THE NETBSD PROJECT

Руководство по NetBSD

Установка, настройка и сопровождение системы

Разработчики NetBSD

Перевод с английского языка

Редакция от 2 января 2026 г.

Об этом издании

Этот документ — перевод на русский язык книги *The NetBSD Guide* (https://www.netbsd.org/docs/guide/en/), входящей в состав документации проекта NetBSD.

Перевод выполнен автоматически с помощью больших языковых моделей и может содержать неточности. Команды, имена файлов, параметры и листинги приведены в исходном виде и не переводились. При расхождениях ориентируйтесь на оригинал.

Оригинал распространяется на условиях лицензии BSD (двухпунктовая лицензия NetBSD). Перевод распространяется на тех же условиях; к переводу применимо уведомление об авторских правах оригинального документа.

Исходный файл: netbsd.html · редакция 2026/01/02

Содержание

Назначение этого руководства 8

Часть I. О NetBSD 9

Глава 1. Что такое NetBSD? 10
1.1. История NetBSD 10
1.2. Возможности NetBSD 10
1.3. Поддерживаемые платформы 11
1.4. Целевые пользователи NetBSD 11
1.5. Приложения для NetBSD 11
1.6. Как получить NetBSD 11

Часть II. Установка системы и связанные вопросы 12

Глава 2. Установка NetBSD: предварительные замечания и подготовка 13
2.1. Предварительные замечания 13
2.2. Подготовка к установке 13
2.3. Контрольный список 16

Глава 3. Пример установки 17
3.1. Введение 17
3.2. Процесс установки 17
3.3. Раскладка клавиатуры 17
3.4. Начало установки 18
3.5. Разделы MBR 22
3.6. Разделы disklabel 25
3.7. Задание имени диска 29
3.8. Последний шанс! 30
3.9. Процесс подготовки диска 31
3.10. Тип установки 32
3.11. Выбор установочного носителя 34
3.12. Распаковка наборов 44
3.13. Настройка системы 45
3.14. Завершение установки 52

Глава 4. Обновление NetBSD 54
4.1. Использование sysinst 54
4.2. Использование sysupgrade 60

Часть III. Конфигурация системы, администрирование и тонкая настройка 62

Глава 5. Первые шаги в NetBSD 63
5.1. Устранение неполадок 63
5.2. Команда man 64
5.3. Редактирование файлов конфигурации 65
5.4. Вход в систему 65
5.5. Смена пароля root 66
5.6. Добавление пользователей 66
5.7. Теневые пароли 67
5.8. Смена раскладки клавиатуры 67
5.9. Системное время 67
5.10. Защищённая оболочка (Secure Shell, ssh(1)) 68
5.11. Базовая настройка в /etc/rc.conf 68
5.12. Основные настройки сети 69
5.13. Монтирование CD-ROM 69
5.14. Монтирование дискеты 70
5.15. Установка дополнительного программного обеспечения 70
5.16. Оповещения о безопасности 71
5.17. Остановка и перезагрузка системы 71

Глава 6. Редактирование 73
6.1. Знакомство с vi 73
6.2. Настройка vi 77
6.3. Использование тегов в vi 78

Глава 7. Система rc.d 79
7.1. Основы 79

7.2. Сценарии rc.d 80
7.3. Роль rcorder и сценариев rc.d 81
7.4. Дополнительная литература 81

Глава 8. Драйверы консоли 82
8.1. wscons 82

Глава 9. Система X Window 89
9.1. Что такое X11 / Xorg? 89
9.2. Настройка 90
9.3. Клавиатура 90
9.4. Монитор 91
9.5. Запуск X 91
9.6. Настройка X 91
9.7. Другие менеджеры окон и рабочие среды 92
9.8. Графический вход с помощью xdm 93
9.9. Использование нескольких серверов X или удалённых серверов X 93
9.10. Дополнительные ресурсы 94

Глава 10. Звук 95
10.1. Настройка устройства звука по умолчанию 95
10.2. Настройка микшера и громкости 95
10.3. Псевдоустройства звука 96
10.4. Команды записи и воспроизведения 97
10.5. Поддержка MIDI 97
10.6. Устройства Intel HD Audio 98

Глава 11. Управление питанием 99
11.1. Основные команды управления питанием 99
11.2. Датчики и мониторинг 101
11.3. Введение в powerd 102

Глава 12. Печать 104
12.1. Включение демона печати 104
12.2. Настройка /etc/printcap 105
12.3. Настройка Ghostscript 106
12.4. Команды управления принтером 108
12.5. Удалённая печать 108

Глава 13. Использование сменных носителей 110
13.1. Инициализация и использование USB-флешек 110
13.2. Инициализация и использование дискет 112
13.3. Как использовать диск ZIP 113
13.4. Чтение компакт-дисков с данными в NetBSD 114
13.5. Чтение компакт-дисков с несколькими сеансами в NetBSD 115
13.6. Предоставление обычным пользователям доступа к компакт-дискам 116
13.7. Монтирование образа ISO 116
13.8. Использование видео-CD в NetBSD 117
13.9. Использование аудио-CD в NetBSD 117
13.10. Создание файла MP3 (MPEG layer 3) с аудио-CD 118
13.11. Использование привода CD-R для компакт-дисков с данными 119
13.12. Использование привода CD-R для создания аудио-CD 120
13.13. Создание аудио-CD из файлов MP3 120
13.14. Копирование аудио-CD 121
13.15. Копирование компакт-диска с данными с помощью двух приводов 121
13.16. Использование перезаписываемых CD-RW 121
13.17. Поддержка DVD 121
13.18. Создание образов ISO с компакт-диска 122
13.19. Получение сведений о томе с компакт-дисков и из образов ISO 122

Глава 14. Драйвер криптографического устройства (CGD) 123
14.1. Обзор 123
14.2. Компоненты системы Crypto-Graphic Disk 124
14.3. Пример: шифрование диска 126
14.4. Пример: зашифрованные компакт-диски и DVD 129
14.5. Пример: зашифрованные устройства iSCSI 132
14.6. Рекомендации и предупреждения 136
14.7. Дополнительная литература 137

Глава 15. Настройка объединённого дискового устройства (CCD) 139
15.1. Установка физических носителей 139
15.2. Настройка поддержки в ядре 140

15.3. Создание disklabel для каждого диска-компонента CCD 140
15.4. Настройка CCD 142
15.5. Инициализация устройства CCD 142
15.6. Создание файловой системы 4.2BSD/UFS на новом устройстве CCD 143
15.7. Монтирование файловой системы 144

Глава 16. NetBSD RAIDframe 145
16.1. Введение в RAIDframe 145
16.2. Настройка поддержки RAIDframe 146
16.3. Пример: корневой диск RAID-1 147

Глава 17. Настройка менеджера логических томов NetBSD (LVM) 169
17.1. Анатомия менеджера логических томов NetBSD 169
17.2. Установка физического носителя 171
17.3. Настройка поддержки в ядре 172
17.4. Создание disklabel для каждого физического тома LVM 172
17.5. Создание физических томов 174
17.6. Создание группы томов 175
17.7. Создание логического тома 175
17.8. Пример: LVM с группами томов, расположенными на raid1 176

Глава 18. Подключаемые модули аутентификации (PAM) 181
18.1. Общие сведения 181
18.2. Введение 181
18.3. Термины и соглашения 181
18.4. Основы PAM 184
18.5. Настройка PAM 187
18.6. Модули PAM 190
18.7. Программирование приложений с PAM 192
18.8. Программирование модулей PAM 192
18.9. Пример приложения с PAM 192
18.10. Пример модуля PAM 195
18.11. Пример функции диалога PAM 197
18.12. Дополнительная литература 198

Глава 19. Настройка производительности NetBSD 200
19.1. Введение 200
19.2. Что учитывать при настройке 201
19.3. Средства визуального мониторинга 203
19.4. Средства мониторинга 206
19.5. Сетевые средства 210
19.6. Учёт 214
19.7. Профилирование ядра 216
19.8. Настройка системы 220
19.9. Настройка ядра 222

Глава 20. Подсистема Veriexec в NetBSD 228
20.1. Как это работает 228
20.2. Файл подписей 228
20.3. Генерация отпечатков 228
20.4. Строгие уровни 230
20.5. Veriexec и многослойные файловые системы 231
20.6. Конфигурация ядра 231

Глава 21. Bluetooth в NetBSD 232
21.1. Введение 232
21.2. Поддерживаемое оборудование 232
21.3. Настройка системы 234
21.4. Устройства человеко-машинного интерфейса 234
21.5. Персональные сети 237
21.6. Последовательные соединения 239
21.7. Звук 240
21.8. Обмен объектами 243
21.9. Устранение неполадок 243

Глава 22. Прочие операции 244
22.1. Установка менеджера загрузки 244
22.2. Удаление disklabel 244
22.3. Динамик 245
22.4. Забыли пароль root? 245
22.5. Файл паролей занят? 246

22.6. Добавление нового жёсткого диска 246
22.7. Как пересоздать устройства в /dev 249

Часть IV. Сеть и связанные вопросы 251

Глава 23. Введение в работу сетей TCP/IP 252
23.1. Аудитория 252
23.2. Поддерживаемые сетевые протоколы 252
23.3. Поддерживаемые среды передачи 252
23.4. Формат адресов TCP/IP 254
23.5. Разбиение на подсети и маршрутизация 256
23.6. Понятия службы имён 258
23.7. IPv6 261

Глава 24. Настройка TCP/IP в NetBSD на практике 267
24.1. Обзор файлов конфигурации сети 267
24.2. Подключение к типичным локальным сетям 268
24.3. Создание небольшой локальной сети вручную 269
24.4. Подключение к домашнему или офисному провайдеру по PPPoE 272
24.5. Настройка интернет-шлюза с помощью NPF 275
24.6. Настройка сетевого моста 276
24.7. Обеспечение правильного порядка инициализации интерфейсов 277
24.8. Некоторые полезные команды 278

Глава 25. Интернет-суперсервер inetd 279
25.1. Обзор 279
25.2. Что такое inetd? 279
25.3. Настройка inetd — /etc/inetd.conf 279
25.4. Службы — /etc/services 281
25.5. Протоколы — /etc/protocols 281
25.6. Удалённый вызов процедур (RPC) — /etc/rpc 281
25.7. Разрешение и запрет хостов — /etc/hosts.{allow,deny} 282
25.8. Добавление службы 282
25.9. Когда использовать inetd, а когда нет 283
25.10. Другие источники 284

Глава 26. Система доменных имён 285
26.1. Основы и понятия DNS 285
26.2. Файлы DNS 290
26.3. Использование DNS 296
26.4. Настройка сервера имён, работающего только как кэш 298

Глава 27. Почта и новости 300
27.1. postfix 302
27.2. fetchmail 304
27.3. Чтение и написание почты в mutt 305
27.4. Стратегия получения почты 306
27.5. Стратегия отправки почты 306
27.6. Дополнительные средства работы с почтой 306
27.7. Новости с tin 308

Глава 28. Введение в протокол резервирования общих адресов (CARP) 310
28.1. Работа CARP 310
28.2. Настройка CARP 311
28.3. Включение поддержки CARP 312
28.4. Пример CARP 312
28.5. Расширенная настройка CARP 313
28.6. Принудительное переключение главного хоста 314

Глава 29. Сетевые службы 315
29.1. Сетевая файловая система (NFS) 315
29.2. Протокол сетевого времени (NTP) 318

Часть V. Виртуализация и эмуляция 320

Глава 30. Использование виртуализации: QEMU и NVMM 321
30.1. Включение монитора виртуальных машин NetBSD 321
30.2. Использование QEMU с NVMM 322
30.3. Настройка мостового соединения на хосте NetBSD 324
30.4. Замечания об использовании NetBSD в качестве гостевой системы 325

Глава 31. Эмуляция Linux 327

31.1. Настройка эмуляции 327
31.2. Структура каталогов 328
31.3. Использование подключаемых модулей Linux для браузеров 329
31.4. Дополнительная литература 330

Часть VI. Сборка системы 331

Глава 32. Получение исходных текстов 332
32.1. Подготовка каталогов 332
32.2. Терминология 332
32.3. Загрузка архивов tar 333
32.4. Получение через CVS 334

Глава 33. Кросс-компиляция NetBSD с помощью build.sh 337
33.1. Сборка инструментария 337
33.2. Настройка ядра вручную 339
33.3. Сборка ядра вручную 340
33.4. Сборка ядра с помощью build.sh 340
33.5. Сборка пользовательского окружения 341
33.6. Сборка системы X Window 341
33.7. Изменение поведения сборки 342

Глава 34. Компиляция ядра 349
34.1. Требования и порядок действий 349
34.2. Установка исходных текстов ядра 349
34.3. Создание файла конфигурации ядра 350
34.4. Сборка ядра вручную 351
34.5. Сборка ядра с помощью build.sh 352
34.6. Установка нового ядра 353
34.7. Если что-то пошло не так 353

Глава 35. Обновление существующей системы из исходных текстов 355
35.1. Порядок ручной сборки и обновления 355
35.2. Использование sysinst 357
35.3. Использование sysbuild и sysupgrade 357
35.4. Подробнее об обновлении файлов конфигурации и загрузки 359

Глава 36. Создание установочных носителей NetBSD 361
36.1. Создание стандартных установочных образов с помощью build.sh 361
36.2. Создание собственных «живых» дисковых образов 361

Приложение A. Сведения 363
A.1. Где взять этот документ 363
A.2. История руководства 363

Приложение B. Вклад в руководство NetBSD 364
B.1. Отправка материалов 364
B.2. Шаблон XML/DocBook 364

Приложение C. Начало работы с XML/DocBook 366
C.1. Что такое XML/DocBook 366
C.2. Установка необходимых инструментов 366
C.3. Работа с инструментами 367
C.4. Ссылки 367

Приложение D. Благодарности 369
D.1. Первоначальные благодарности 369
D.2. Текущие благодарности 369
D.3. Лицензии 370
D.3.1. Первоначальная лицензия Федерико Лупи на это руководство 370
D.3.2. Лицензия Networks Associates Technology на статью о PAM 370
D.3.3. Лицензия Joel Knight на статью о CARP 371

Приложение E. Библиография 373

Назначение этого руководства

В этом руководстве описаны установка и настройка операционной системы NetBSD, а также настройка и администрирование некоторых её подсистем. Оно в первую очередь адресовано людям, пришедшим из других Unix-подобных операционных систем, и призвано служить полезным пособием при множестве мелких проблем, с которыми вы сталкиваетесь, начиная работать с новым инструментом.

Это руководство не является учебником по Unix: предполагается наличие базовых знаний некоторых концепций и инструментов. Например, вы должны знать, что такое файл и каталог, и уметь пользоваться редактором. Существует множество книг, объясняющих базовые концепции Unix и операционных систем, и к одной из них стоит обратиться, если вам нужна дополнительная информация. Лучше выбрать общую книгу и избегать изданий вроде «Изучаем Unix-XYZ, версия 1.2.3.4 за 10 дней», но это вопрос личного вкуса.

Многое ещё предстоит сделать, чтобы завершить это введение в NetBSD: некоторые главы не закончены (а некоторые даже не начаты), и некоторые темы нуждаются в дополнительной проверке. Исправления и дополнения, безусловно, приветствуются.

В настоящее время руководство поддерживается командой www проекта NetBSD (<www@NetBSD.org>). Исправления и предложения следует направлять по этому адресу. См. также Приложение B, *Вклад в руководство NetBSD*.

Часть I. О NetBSD

Глава 1. Что такое NetBSD?

NetBSD — это свободная, быстрая, безопасная и обладающая высокой переносимостью Unix-подобная операционная система с открытым исходным кодом. Она доступна для множества платформ: от 64-битных серверов x86 и настольных ПК до встраиваемых устройств на базе ARM и MIPS. Её чистая архитектура и развитые возможности делают её превосходным выбором как для производственной, так и для исследовательской среды, а поддержка обеспечивается пользователями при полном доступе к исходным текстам. Множество приложений легко доступны через pkgsrc — коллекцию пакетов NetBSD.

1.1. История NetBSD

Первая версия NetBSD (0.8) появилась в 1993 году и берёт начало от операционной системы 4.3BSD Lite — варианта Unix, разработанного в Калифорнийском университете в Беркли (BSD = Berkeley Software Distribution), — а также от системы 386BSD, первого порта BSD для процессора Intel 386. В последующие годы в систему были интегрированы изменения из выпуска 4.4BSD Lite (последнего выпуска группы из Беркли). Ветвь BSD операционной системы Unix сыграла огромную роль и оказала большое влияние на историю Unix-подобных операционных систем, в которые она привнесла множество инструментов, идей и усовершенствований, ставших сегодня стандартными: редактор vi, оболочку C shell, управление заданиями, быструю файловую систему Беркли, надёжные сигналы, поддержку виртуальной памяти и TCP/IP — и это лишь некоторые из них. Эта традиция исследований и разработки сохраняется сегодня в системах BSD и, в частности, в NetBSD.

1.2. Возможности NetBSD

NetBSD работает на огромном множестве аппаратных платформ и обладает высокой переносимостью. Полные исходные тексты ядра и пользовательского окружения NetBSD доступны для всех поддерживаемых платформ; подробности см. на официальном сайте проекта NetBSD.

Основные возможности NetBSD:

- Качество и корректность кода
- Переносимость на широкий спектр оборудования
- Безопасные настройки по умолчанию
- Следование отраслевым стандартам
- Исследования и инновации

Эти характеристики приносят и косвенные преимущества. Например, если вы работаете только на одной платформе, можете решить, что переносимость вас не интересует. Но переносимость связана с качеством кода: без хорошо написанной и хорошо организованной кодовой базы было бы невозможно поддерживать большое число платформ. А качество кода — основа любой хорошей и надёжной программной системы, хотя, как ни странно, мало кто, похоже, это понимает.

Одна из ключевых особенностей NetBSD в том, что её разработчиков не устраивают частичные реализации. У некоторых систем, похоже, есть философия «Если работает — значит, правильно». В этом свете философию NetBSD можно описать как «Работает, только если сделано правильно».

Подумайте, сколько разросшихся программ рушится под тяжестью собственного веса и «возможностей», и вы поймёте, почему NetBSD старается избегать такой ситуации любой ценой.

1.3. Поддерживаемые платформы

NetBSD поддерживает множество платформ, включая популярные i386 и amd64, ARM, SPARC, Alpha, Amiga, Atari, а также машины Apple Macintosh на базе m68k и PowerPC. Технические подробности обо всех них можно найти на сайте NetBSD.

1.4. Целевые пользователи NetBSD

На сайте NetBSD сказано: «Проект NetBSD предоставляет свободно распространяемую и допускающую повторное распространение систему, которую профессионалы, энтузиасты и исследователи могут использовать любым удобным им способом». Это также идеальная система, если вы хотите изучить Unix, главным образом благодаря её следованию стандартам (одна из целей проекта) и потому, что она одинаково хорошо работает как на новейшем оборудовании для ПК, так и на оборудовании, которое многие другие операционные системы считают устаревшим. Чтобы изучать и использовать Unix, не нужно покупать дорогое оборудование; можно воспользоваться старым ПК или Mac с чердака. Важно отметить, что хотя NetBSD работает на старом оборудовании, современное оборудование хорошо поддерживается, и были приняты меры к тому, чтобы поддержка старых машин не снижала производительность на современном оборудовании. Кроме того, если вам нужна Unix-система, которая стабильно работает на самых разных платформах, NetBSD, вероятно, будет лучшим выбором.

1.5. Приложения для NetBSD

Помимо стандартных инструментов Unix для повседневной работы, редакторов, средств форматирования, компиляторов и отладчиков C/C++ и так далее, входящих в базовую систему, существует огромная коллекция пакетов (в настоящее время более 20 000), которые можно установить как двоичные пакеты или собрать из pkgsrc, включая популярное кроссплатформенное программное обеспечение, такое как Firefox, PostgreSQL, Python и Xfce.

1.6. Как получить NetBSD

NetBSD — это операционная система с открытым исходным кодом, и как таковая она свободно доступна для загрузки с cdn.NetBSD.org и других зеркал.

Часть II. Установка системы и связанные вопросы

Глава 2. Установка NetBSD: предварительные замечания и подготовка

2.1. Предварительные замечания

2.1.1. Двойная загрузка

NetBSD можно установить вместе с другими операционными системами на одном жёстком диске.

Если на жёстком диске уже есть операционная система, подумайте, как можно освободить немного места для NetBSD; если NetBSD будет делить диск с другими операционными системами, скорее всего, потребуется создать новый раздел (это вы сделаете с помощью sysinst). Часто это невозможно, если не изменить размер существующего раздела.

К сожалению, изменить размер существующего раздела с помощью sysinst невозможно, но для этого есть некоторые коммерческие продукты (например, Partition Magic) и свободные инструменты (GNU Parted, FIPS, pfdisk).

NetBSD можно также установить на отдельный жёсткий диск.

> **СОВЕТ**
>
> Если вы не чувствуете себя уверенно при создании схемы разделов для двух или более операционных систем и не понимаете риск потери данных в случае ошибки, рекомендуется выделить NetBSD отдельный жёсткий диск. Это устраняет риск повреждения существующей операционной системы.

2.1.2. NetBSD в эмуляции и виртуализации

NetBSD можно установить и запустить поверх других операционных систем, не беспокоясь о разделах. Эмуляторы или среды виртуализации дают быстрый и безопасный способ попробовать NetBSD. Хостовая операционная система остаётся неизменной, а риск повреждения важных данных сводится к минимуму.

Информация о NetBSD в качестве хоста и гостевой системы Xen доступна на веб-странице NetBSD/xen.

Веб-страница NetBSD на эмулируемом оборудовании содержит подробную информацию о различных эмуляторах и поддерживаемых платформах NetBSD. Следует также отметить, что NetBSD работает как гостевая система VMware.

2.2. Подготовка к установке

2.2.1. Документ INSTALL

Первое, что нужно сделать перед установкой NetBSD, — прочитать информацию о выпуске и замечания по установке в одном из файлов `INSTALL`: это официальное описание процедуры установки с информацией, специфичной для платформы, и важными подробностями. Документ доступен в форматах HTML, PostScript, обычного текста и расширенного текстового формата для

использования с more. Эти файлы можно найти в корневом каталоге выпуска NetBSD (на установочном CD или на FTP-сервере). Например, инструкции по установке amd64 доступны по адресу ftp.NetBSD.org/pub/NetBSD/NetBSD-11.0/amd64/INSTALL.html

2.2.2. Разделы

Терминология, которую NetBSD использует для разбиения на разделы, отличается от типичной терминологии DOS/Windows; фактически при работе NetBSD на типичном ПК задействованы две схемы разделов. NetBSD устанавливается в один из четырёх первичных разделов BIOS (разделов, определённых в таблице разделов жёсткого диска).

Внутри раздела BIOS (также называемого *slice*) NetBSD определяет свои BSD-разделы с помощью *disklabel*. Эти разделы видны только NetBSD и обозначаются строчными буквами (начиная с «a»). Например, wd0a обозначает раздел «a» первого диска IDE (wd0), а sd0a — раздел «a» первого диска SCSI. На рисунке 2.1, «Разделы», показаны два первичных раздела BIOS: один используется DOS, другой — NetBSD. NetBSD описывает разметку диска с помощью disklabel.

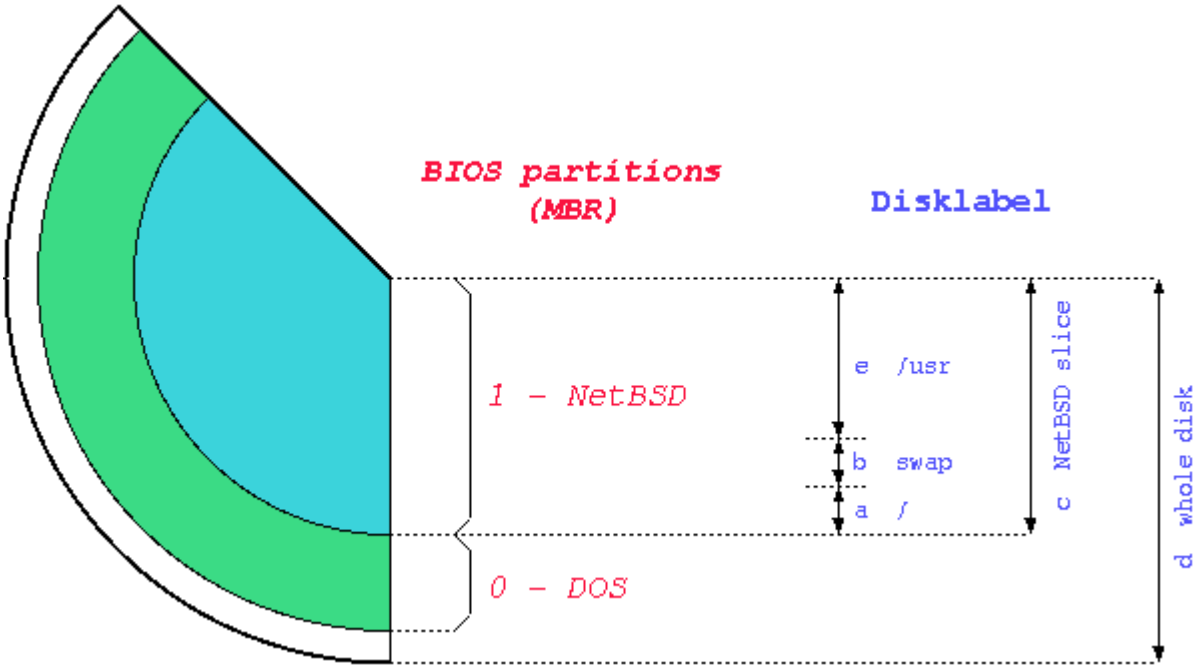


Рисунок 2.1. Разделы

> **ПРИМЕЧАНИЕ**
>
> Значение разделов «c» и «d» типично для порта amd64. В большинстве других портов «c» представляет весь диск.

> **ПРИМЕЧАНИЕ**
>
> Если NetBSD делит жёсткий диск с другой операционной системой (как в предыдущем примере), вы захотите установить *менеджер загрузки*, то есть программу, которая позволяет выбрать, какую ОС запускать при загрузке. sysinst может сделать это за вас и спросит, хотите ли вы установить его. Если у вас нет особых причин этого не делать, позвольте sysinst выполнить этот шаг.

2.2.3. Требования к месту на жёстком диске

Точный объём пространства, необходимый для конкретной установки NetBSD, зависит от используемой платформы и выбранных наборов дистрибутива. Вообще говоря, если на жёстком диске есть несколько ГБ свободного места, этого достаточно для полной установки базовой системы.

2.2.4. Сетевые настройки

Если вы планируете загрузить наборы дистрибутива по сети (это не обязательно, если вы скачали установочный ISO полного размера) и не используете DHCP, запишите свои основные сетевые настройки. Вам потребуются:

- Ваш IP-адрес (пример: 192.168.1.7)
- маска сети (пример: 255.255.255.0)
- IP-адрес вашего шлюза по умолчанию (пример: 192.168.1.1)
- IP-адрес используемого вами DNS-сервера (пример: 145.253.2.75)

2.2.5. Сделайте резервную копию данных и операционных систем!

Прежде чем приступить к установке, убедитесь, что у вас есть надёжная резервная копия всех операционных систем и данных на используемом жёстком диске. Ошибки при разбиении жёсткого диска на разделы могут привести к потере данных. Установленные операционные системы могут перестать загружаться. «Надёжная резервная копия» означает, что процедура резервного копирования и восстановления проверена и работает безупречно!

2.2.6. Подготовка установочных носителей

Система установки NetBSD состоит из двух частей. Первая часть — установочное ядро. Это ядро содержит программу установки NetBSD sysinst и загружается с установочного носителя (например, CD/DVD, USB-накопителя, карты памяти и т. п.). Программа sysinst подготовит диск: разобьёт дисковое пространство на разделы, сделает диск загрузочным и создаст необходимые файловые системы.

Вторая часть системы установки — наборы двоичных файлов дистрибутива: файлы операционной системы NetBSD. Программе установки необходим доступ к наборам дистрибутива. Обычно sysinst берёт эти файлы с установочного носителя, с которого вы загрузились, но она может получать их и по FTP, NFS или из локальной файловой системы.

Проект NetBSD предоставляет полные установочные носители для каждой поддерживаемой аппаратной архитектуры. Обычно это загрузочные образы CD (файлы `.iso`).

2.2.6.1. Загрузка системы установки с USB

Чтобы воспользоваться загрузочным установочным образом USB (на amd64, i386), скачайте файл `img.gz` для вашей аппаратной архитектуры, распакуйте его и скопируйте образ на USB-накопитель. Например, в Unix-подобной системе можно выполнить:

```
# gunzip NetBSD-11.0-amd64-install.img.gz
# dd if=NetBSD-11.0-amd64-install.img of=/dev/your-usb bs=2m
```

Примеры `your-usb`: `/dev/rsd0d` (NetBSD), `/dev/sda` (Linux).

> **ВНИМАНИЕ**
>
> Выбор неверного устройства в **dd** может уничтожить вашу текущую систему. Ещё раз проверьте, что оно не смонтировано и что это именно ваш USB-накопитель. При подключении оно должно появиться в конце вывода **dmesg**, например, если вы видите:
>
> ```
> sd0 at scsibus0 target 0 lun 0: [...], disk removable
> ```
>
> в NetBSD вам нужно будет выбрать `/dev/rsd0d`.

2.2.6.2. Загрузка системы установки с CD

Чтобы воспользоваться загрузочным установочным CD NetBSD, скачайте файл `iso` для вашей аппаратной архитектуры и запишите его на CD или DVD. Этот шаг придётся выполнить самостоятельно, так как программы записи сильно различаются. Убедитесь, что компьютер настроен на загрузку с CD-ROM раньше жёстких дисков, вставьте диск и перезагрузите компьютер.

2.3. Контрольный список

Это контрольный список того, что должно быть уже выяснено и под рукой:

- Доступное дисковое пространство
- Загрузочный носитель с системой установки
- CD/DVD или сервер с наборами дистрибутива
- Сведения о вашей сети (только если вы будете получать наборы дистрибутива по сети и не используете DHCP)
- Проверенная резервная копия
- Копия документа INSTALL

Глава 3. Пример установки

3.1. Введение

Эта глава проведёт вас через процесс установки. Изложенные здесь принципы применимы ко всем способам установки. Разница лишь в том, как программа установки получает наборы дистрибутива. Некоторые детали установки зависят от выпуска NetBSD. Примеры в этой главе созданы с NetBSD 8.0.

> **ПРИМЕЧАНИЕ**
>
> Приведённые ниже экраны установки — всего лишь примеры. Не копируйте их слепо, так как ваше оборудование и параметры конфигурации могут отличаться!

3.2. Процесс установки

Процесс установки логически делится на две части. В первой части вы создаёте раздел для NetBSD и записываете disklabel для этого раздела. Во второй части вы решаете, какие наборы дистрибутива (подмножества операционной системы) хотите установить, а затем распаковываете файлы в только что созданные разделы.

3.3. Раскладка клавиатуры

Программа установки NetBSD sysinst позволяет изменить раскладку клавиатуры во время установки. Если по какой-то причине это у вас не работает, воспользуйтесь схемой из следующей таблицы.

US	IT	DE	FR
-	'	ß	)
/	-	-	!
=	ì	'	-
:	ç	Ö	M
;	ò	ö	m
#	£	§	3
"	°	Ä	%
*	(	(	8
(	)	)	9
)	=	=	0
'	à	ä	ù
`	\	^	@
\	ù	#	`

3.4. Начало установки

Чтобы начать установку NetBSD, вставьте выбранный загрузочный носитель (CD/DVD, USB-накопитель, дискету и т. п.) и перезагрузите компьютер. Ядро с установочного носителя загрузится и начнёт выводить на экран множество сообщений об обнаружении оборудования.

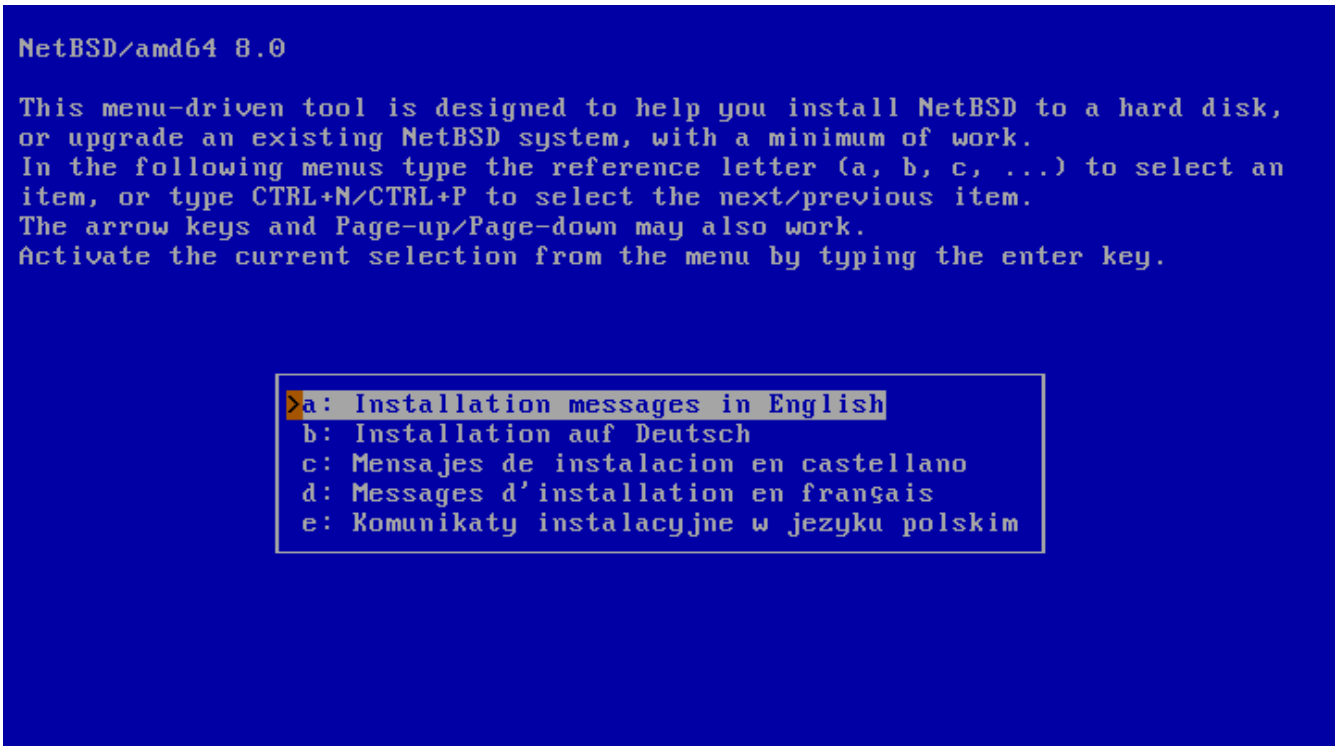


Рисунок 3.1. Выбор языка

Когда ядро загрузится, вы окажетесь в программе установки NetBSD sysinst, как показано на Рисунок 3.1, «Выбор языка». С этого момента следуйте инструкциям, отображаемым на экране, используя документ `INSTALL` в качестве справочника. Документ INSTALL в различных форматах находится в корневом каталоге выпуска NetBSD. Экраны sysinst имеют более или менее одинаковую структуру: в верхней части экрана отображается краткое описание текущей операции или краткое справочное сообщение, а остальную часть занимают интерактивные меню и приглашения. Для выбора используйте клавиши управления курсором, сочетания «Ctrl+N» (следующий) и «Ctrl+P» (предыдущий) или нажмите одну из букв, показанных слева от каждого пункта. Подтвердите выбор нажатием клавиши Return (она же «Enter»).

Начните с выбора языка, который вы хотите использовать в процессе установки.

Следующий экран — Рисунок 3.2, «Выбор типа клавиатуры» — позволит выбрать подходящий тип клавиатуры.

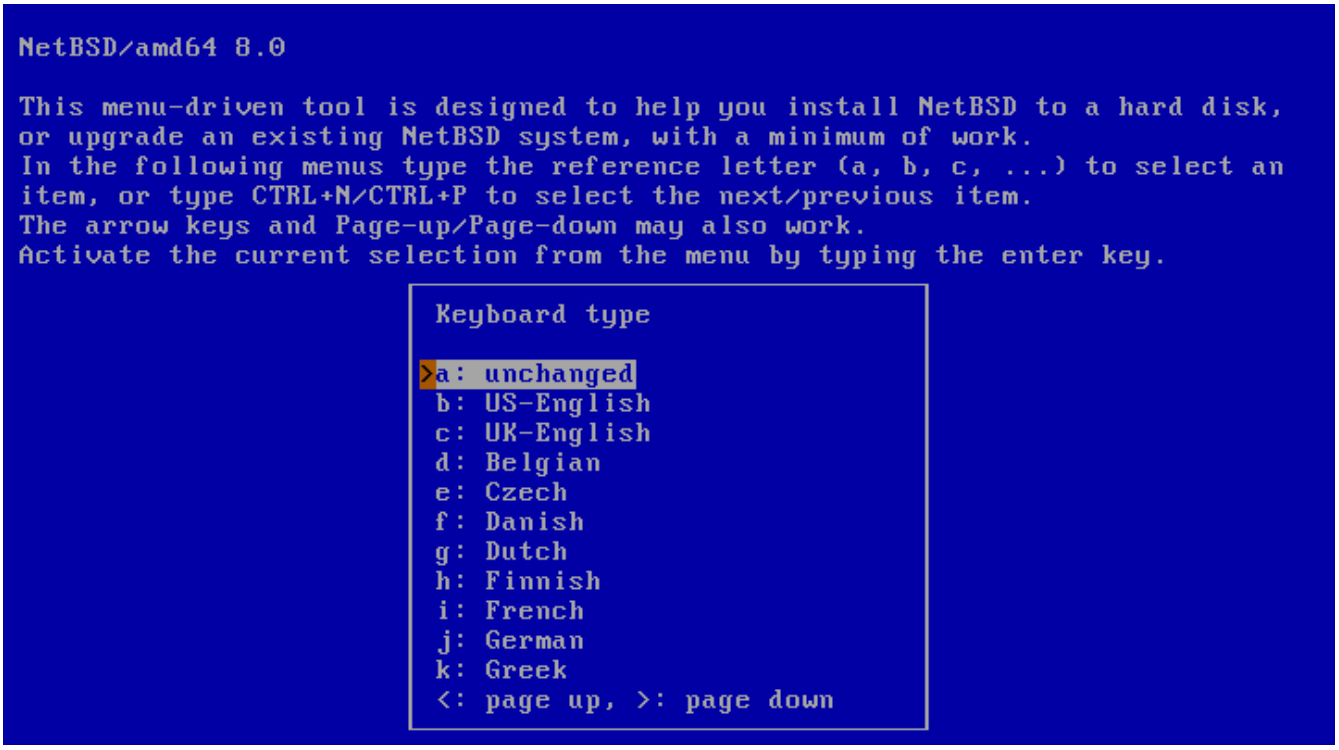


Рисунок 3.2. Выбор типа клавиатуры

После этого вы попадёте в главное меню программы установки (Рисунок 3.3, «Главное меню sysinst»).

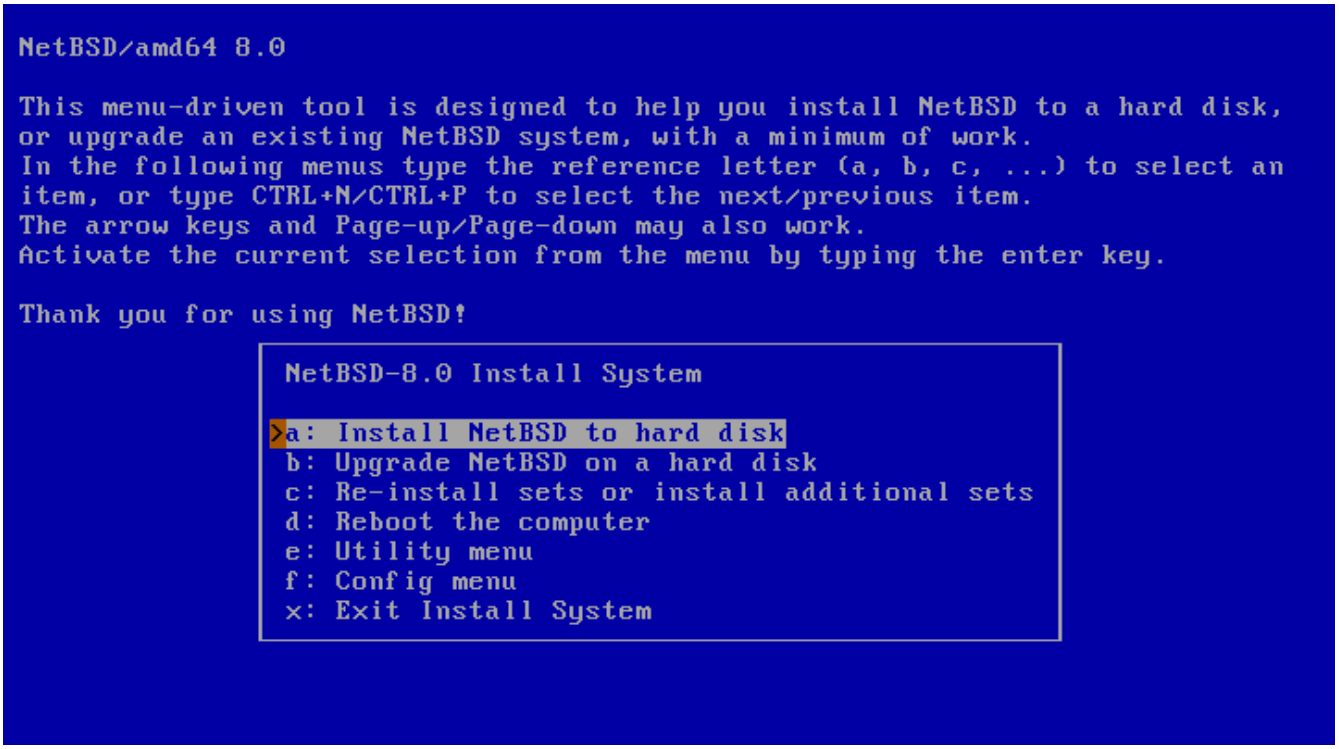


Рисунок 3.3. Главное меню sysinst

Выбор пункта «Install NetBSD to hard disk» переводит вас на следующий экран (Рисунок 3.4, «Подтверждение установки NetBSD»), где необходимо подтвердить, что вы хотите продолжить установку.

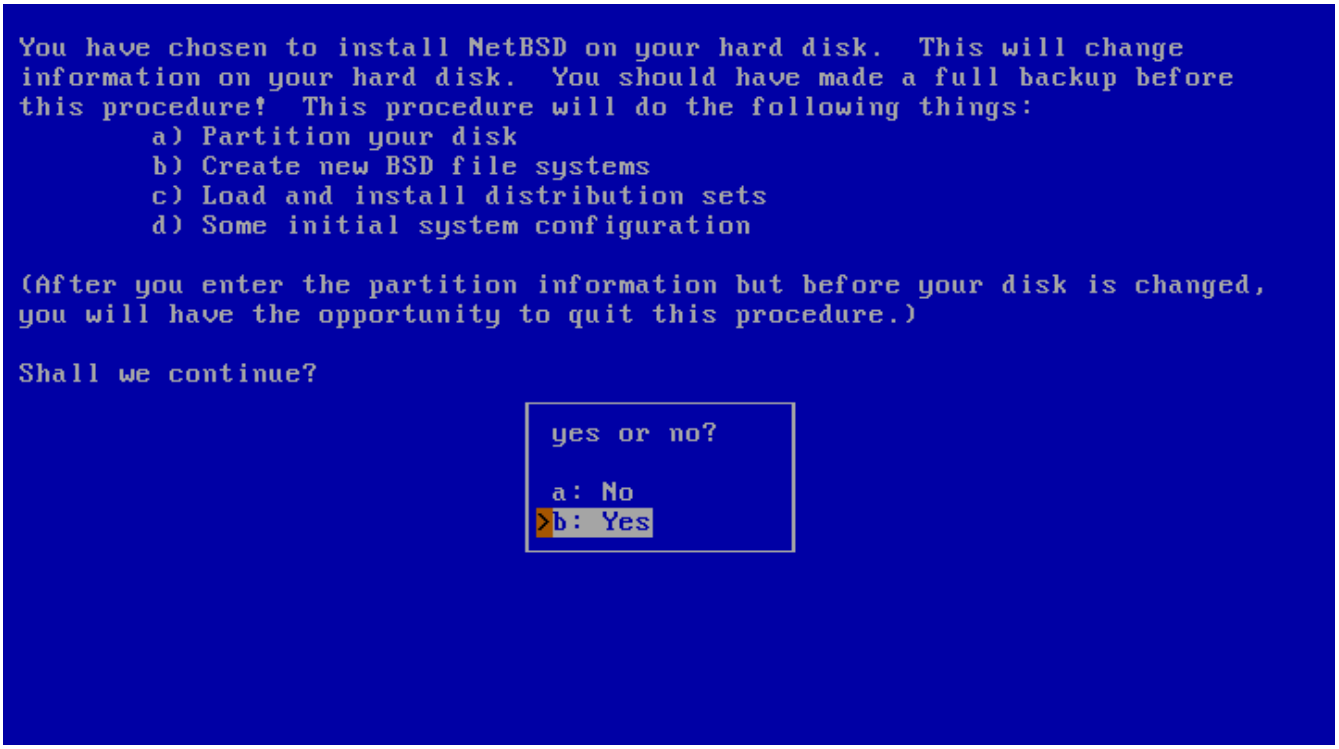


Рисунок 3.4. Подтверждение установки NetBSD

Выбрав «Yes» для продолжения, sysinst отображает список из одного или нескольких дисков и спрашивает, на какой из них вы хотите установить NetBSD. В примере на Рисунок 3.5, «Выбор жёсткого диска» перечислены два диска, и NetBSD будет установлена на «wd0» — первый найденный диск SATA или IDE. Если вы используете диски SCSI или внешние USB-диски, первый из них будет называться «sd0», второй — «sd1» и так далее.

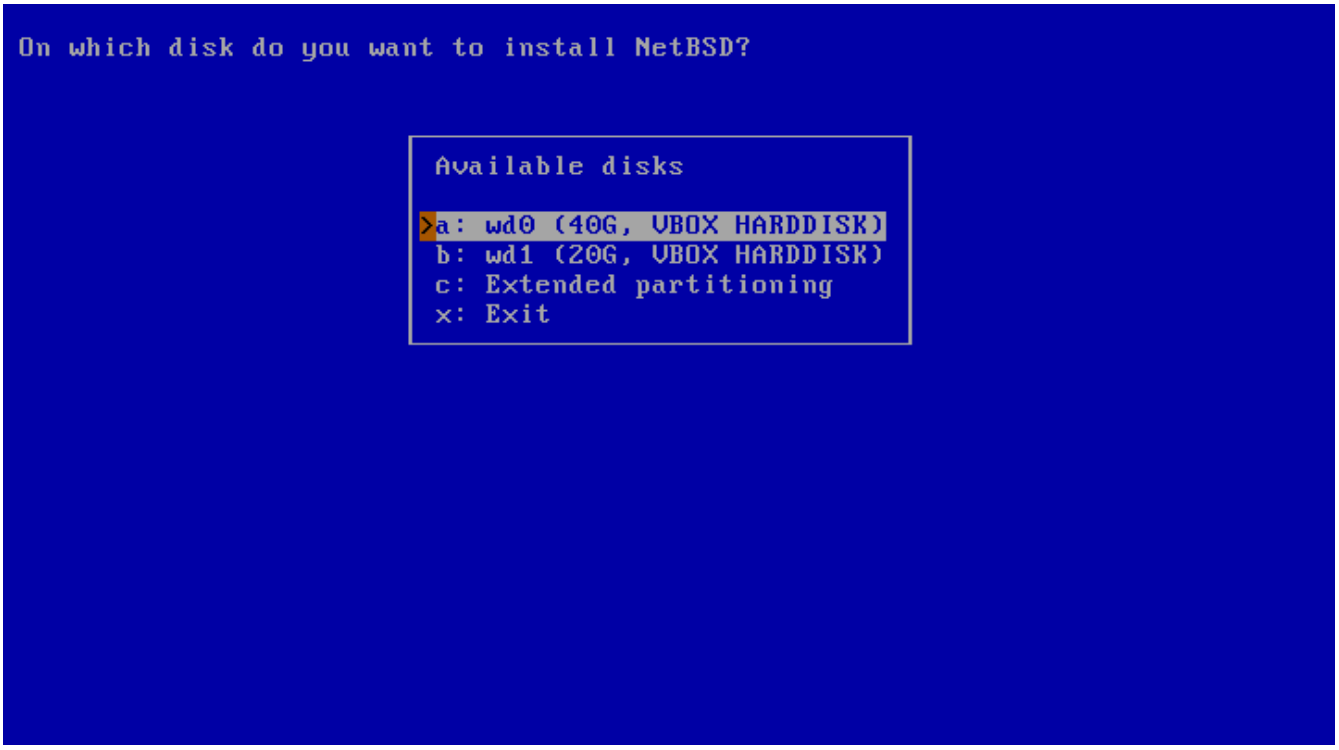


Рисунок 3.5. Выбор жёсткого диска

Затем программа установки попросит вас подтвердить определённую геометрию диска на основе сведений, предоставленных BIOS, как показано на Рисунок 3.6, «Геометрия диска». Почти всегда эти значения верны. Выберите «This is the correct geometry», если только вам не известно, что сведения, предоставляемые вашим BIOS, по имеющимся данным неверны.

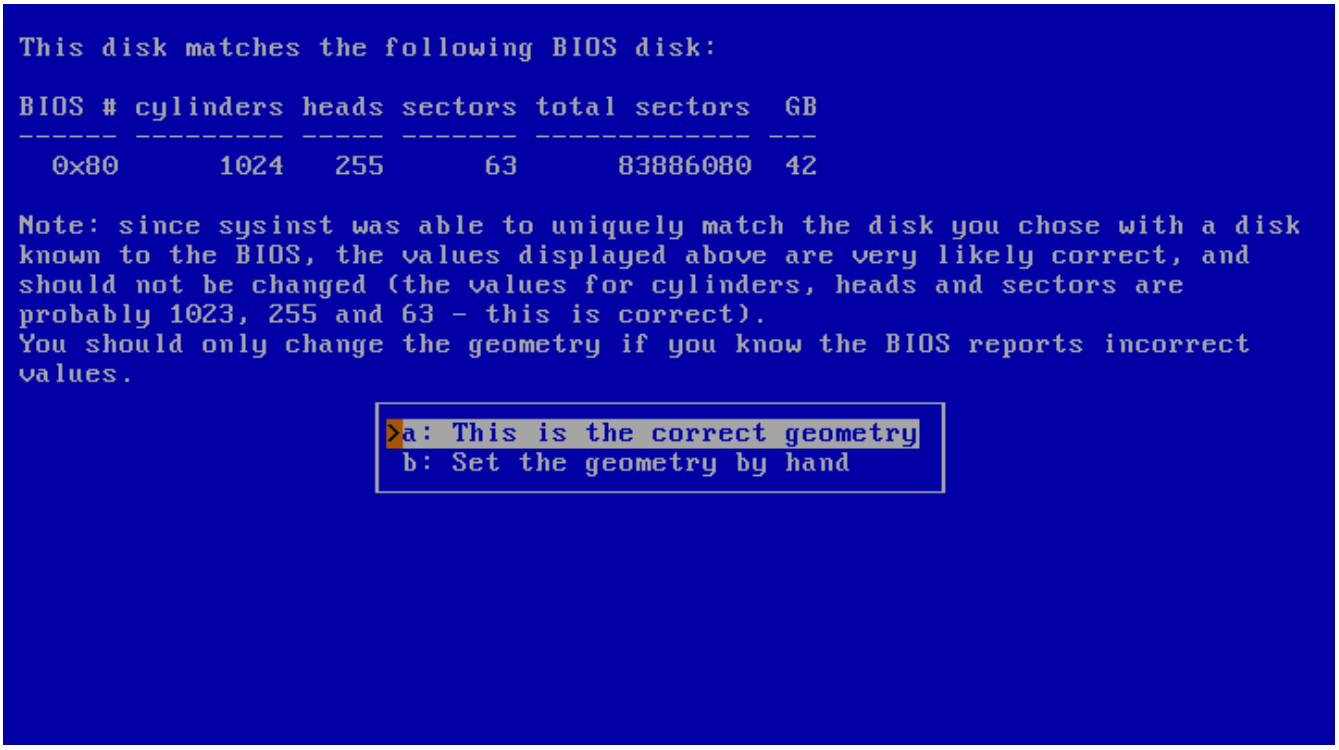


Рисунок 3.6. Геометрия диска

3.5. Разделы MBR

Наступил первый важный этап установки: разбиение жёсткого диска на разделы. Сначала нужно указать, будет ли NetBSD использовать раздел (рекомендуемый вариант) или весь диск. В первом случае по-прежнему можно создать раздел, занимающий весь жёсткий диск (Рисунок 3.7, «Выбор схемы разбиения на разделы»), поэтому мы рекомендуем выбрать этот вариант: он сохраняет таблицу разделов BIOS в формате, совместимом с другими операционными системами.

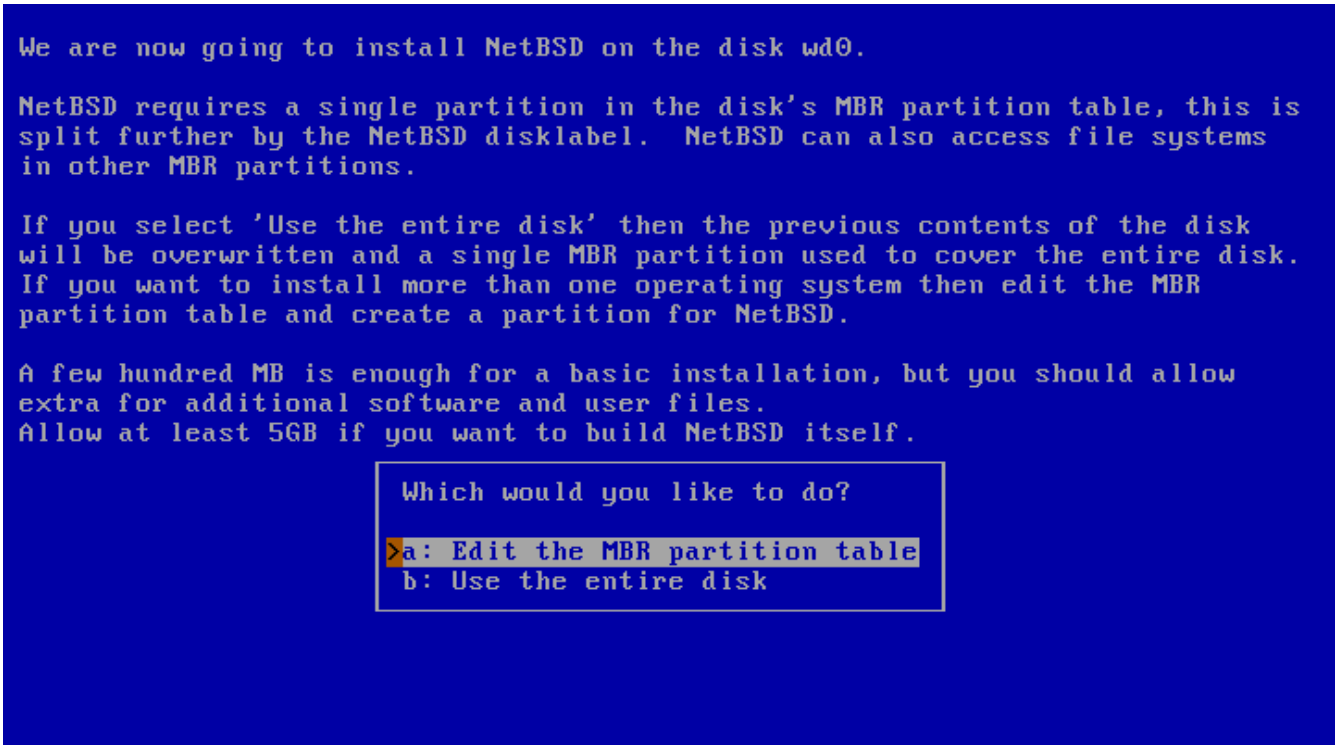


Рисунок 3.7. Выбор схемы разбиения на разделы

Следующий экран показывает текущее состояние таблицы разделов MBR на жёстком диске перед установкой NetBSD. Здесь четыре первичных раздела, и, как видите, в этом примере диск сейчас пуст. Если у вас есть другие разделы, вы можете оставить их и установить NetBSD в раздел, который сейчас не используется, или перезаписать раздел, чтобы использовать его для NetBSD.

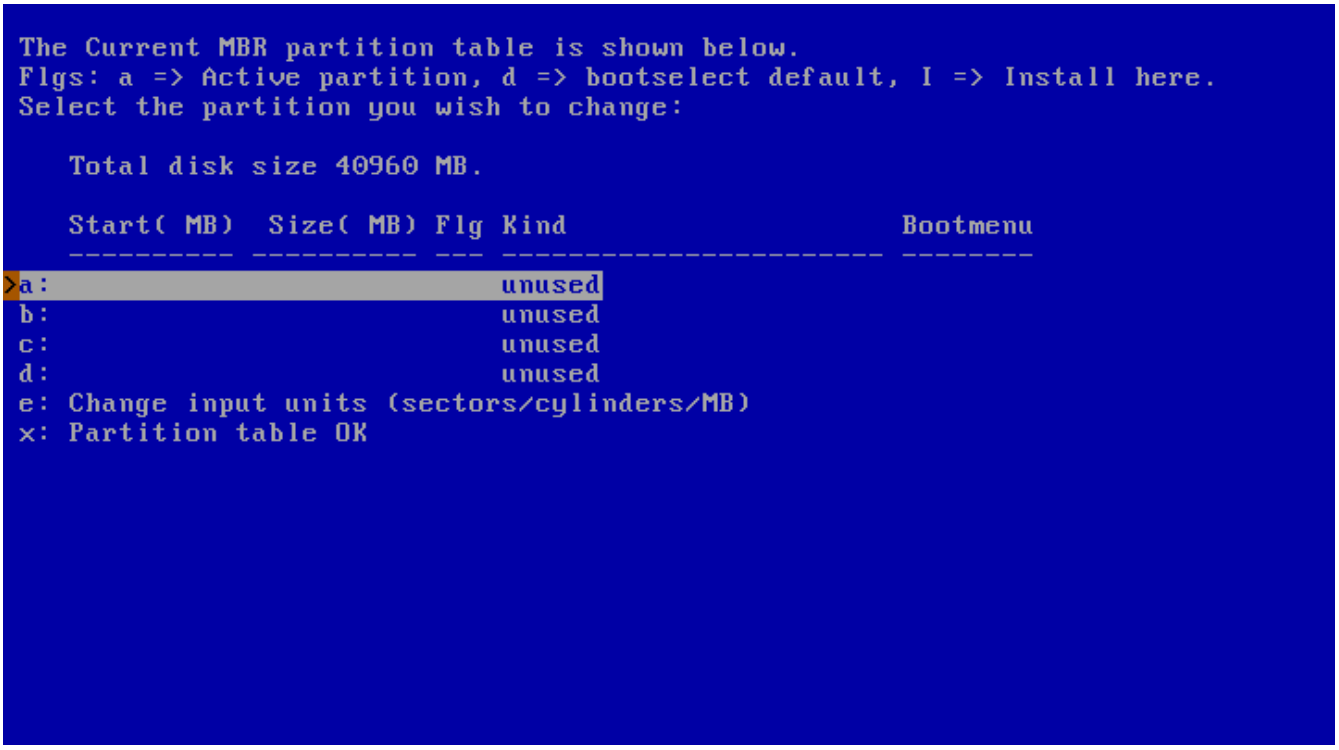


Рисунок 3.8. fdisk

Удалить раздел просто: после выбора раздела появится меню с параметрами для него (Рисунок 3.9, «Параметры раздела»). Измените тип раздела на «Delete partition», чтобы удалить раздел. Разумеется, если вы хотите использовать раздел для NetBSD, можно задать тип раздела «NetBSD».

Создать раздел для NetBSD можно, выбрав раздел, в который вы хотите установить NetBSD. Имена разделов от «a» до «d» соответствуют четырём первичным разделам в других операционных системах. После выбора раздела появится меню с параметрами для него, как показано на Рисунок 3.9, «Параметры раздела».

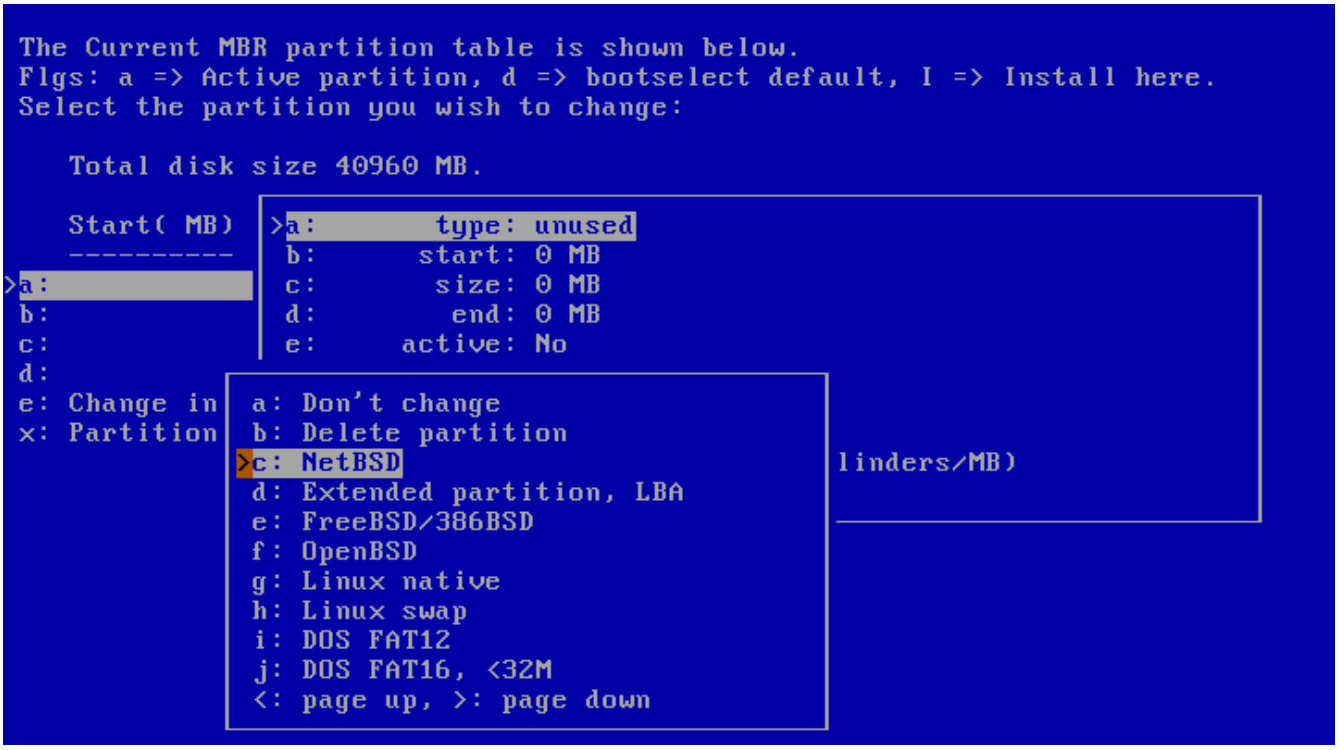


Рисунок 3.9. Параметры раздела

Чтобы создать новый раздел, необходимо указать следующие сведения:

- тип (вид) нового раздела
- первый (начальный) сектор нового раздела
- размер нового раздела

Выберите для нового раздела тип «NetBSD» (с помощью параметра «type»). Программа установки попытается угадать положение «start» по концу предыдущего раздела. При необходимости измените это значение. То же относится к параметру «size»: программа установки попытается заполнить пространство, доступное до следующего раздела или до конца диска (в зависимости от того, что наступит раньше). Вы можете изменить это значение, если оно неверно или если вы не хотите, чтобы NetBSD использовала весь предложенный объём пространства.

После того как вы выбрали тип раздела, начальное положение и размер, стоит задать имя, которое будет использоваться в меню загрузки. Это можно сделать, выбрав пункт «bootmenu» и указав метку, например «NetBSD». Повторите этот шаг для других загрузочных разделов, чтобы с помощью загрузчика NetBSD можно было загружать как NetBSD, так и Windows (или другие операционные системы). Вы также можете выбрать один из помеченных разделов как используемый по умолчанию в меню загрузки. Если вас устраивают параметры разделов, подтвердите выбор, выбрав «Partition OK». Выберите «Partition table OK», чтобы выйти из редактора таблицы разделов MBR.

Если при разбиении на разделы вы допустили ошибку (например, создали перекрывающиеся разделы), sysinst выведет сообщение и предложит вернуться в редактор разделов MBR (но продолжить вы тоже можете). Если данные верны, но раздел NetBSD находится за пределами диапазона секторов, из которых возможна загрузка через BIOS, sysinst предупредит вас и спросит, хотите ли вы всё равно продолжить. Это может привести к проблемам на старых ПК.

> **ПРИМЕЧАНИЕ**
>
> Это не ограничение NetBSD. Некоторые старые версии BIOS не могут загрузить раздел, расположенный за пределами первых 1024 цилиндров. Чтобы полностью разобраться в проблеме, вам следует изучить различные типы BIOS и множество используемых ими схем адресации (*физическая CHS*, *логическая CHS*, *LBA*, ...). В этом руководстве эти вопросы не рассматриваются.
>
> На современных компьютерах (тех, что поддерживают *расширения int13*) можно установить NetBSD в разделы, расположенные за пределами первых 8 ГБ жёсткого диска, при условии, что установлен загрузчик NetBSD.

Далее sysinst предложит установить на жёсткий диск загрузчик. Этот экран показан на рисунке 3.10, «Установка загрузчика».

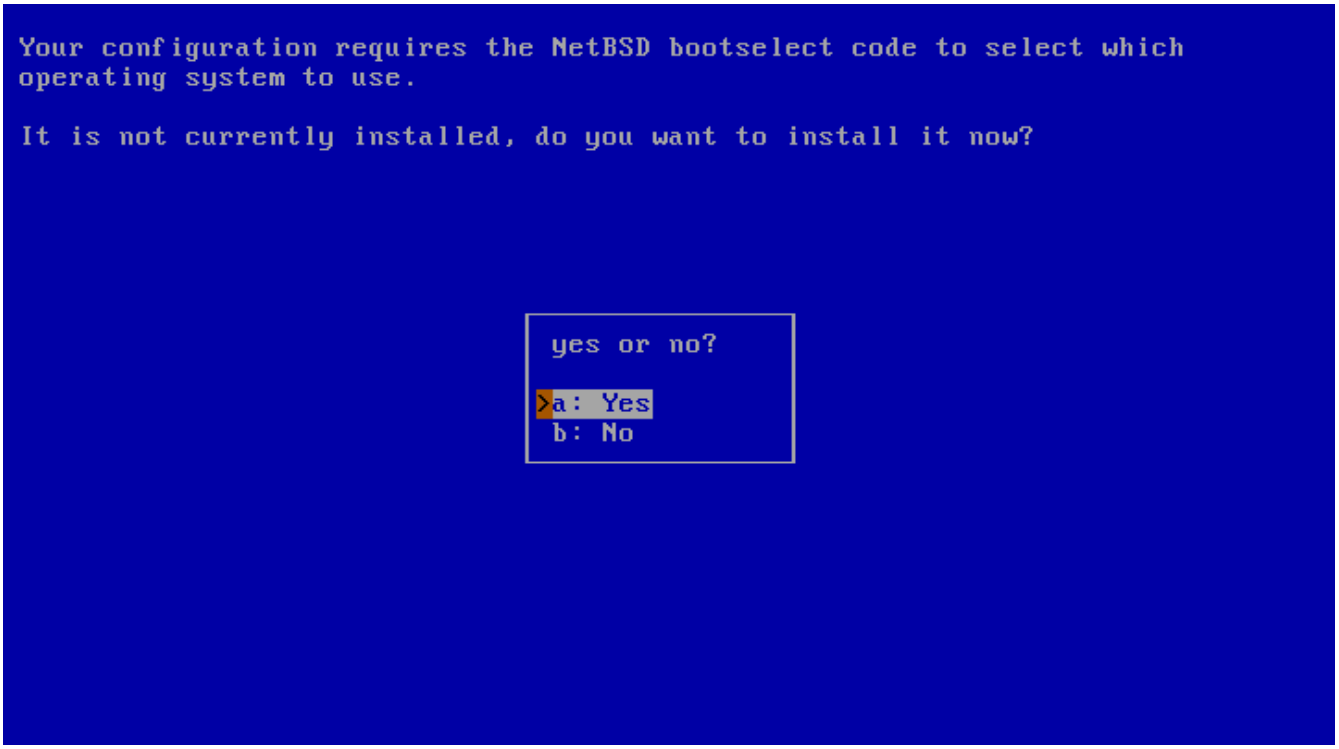


Рисунок 3.10. Установка загрузчика

К этому моменту *разделы BIOS* (в системах BSD они называются *слайсами*) уже созданы. Их также называют *разделами PC BIOS*, *разделами MBR* или *разделами fdisk*.

> **ПРИМЕЧАНИЕ**
>
> Не путайте *слайсы* или *разделы BIOS* с *разделами BSD*: это разные вещи.

3.6. Разделы disklabel

Некоторые платформы, например системы на базе ПК (amd64 и i386), используют разделы MBR в стиле DOS для разделения файловых систем. Раздел MBR, созданный вами ранее в процессе установки, необходим, чтобы другие операционные системы не перезаписали дисковое пространство, выделенное для NetBSD.

NetBSD использует собственную схему разделов, называемую *disklabel*, которая хранится в начале раздела MBR. Дополнительные сведения см. в разделе 2.2.2, «Разделы». В следующих нескольких шагах вы создадите disklabel(5) и зададите размеры разделов NetBSD или воспользуетесь существующими размерами разделов, как показано на рисунке 3.11, «Изменить разделы?».

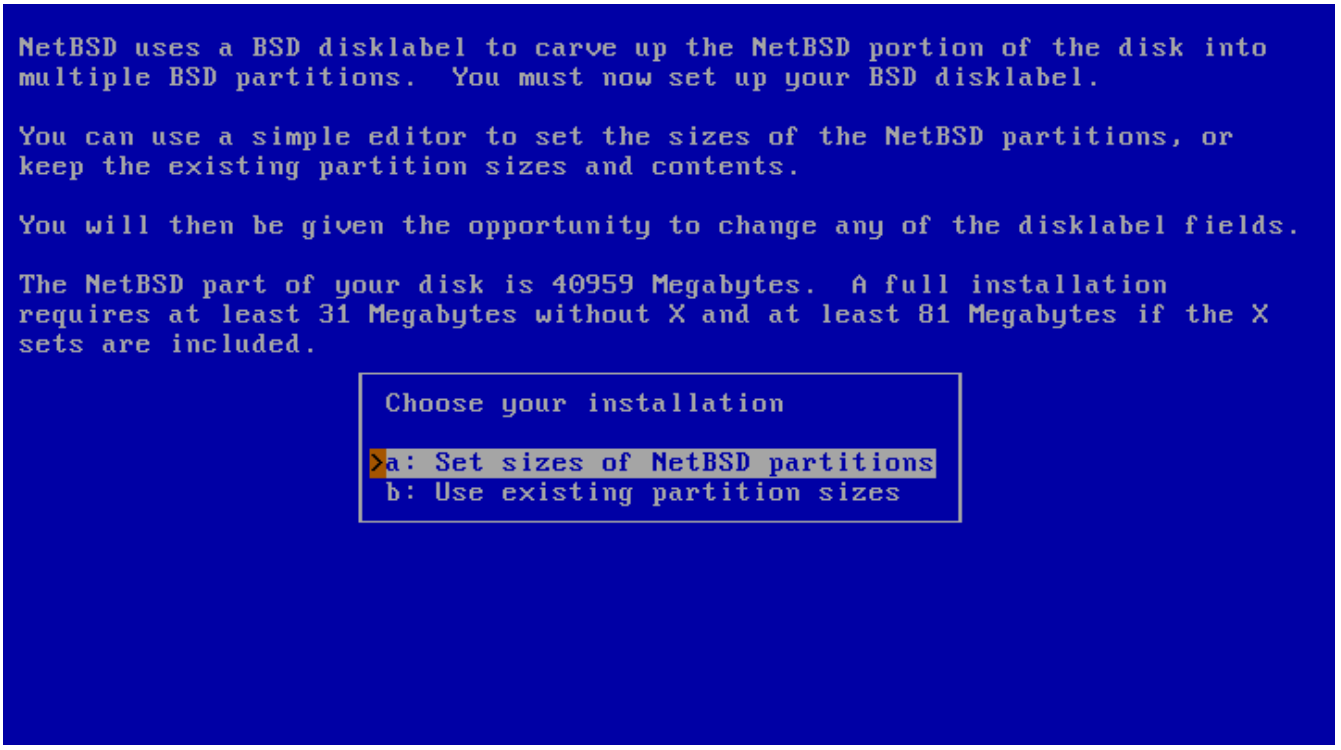


Рисунок 3.11. Изменить разделы?

Если вы решите задать размеры разделов NetBSD, можно определить разделы, которые хотите создать. Программа установки сформирует disklabel на основе этих настроек. Этот экран установки показан на рисунке 3.12, «Задание размеров разделов».

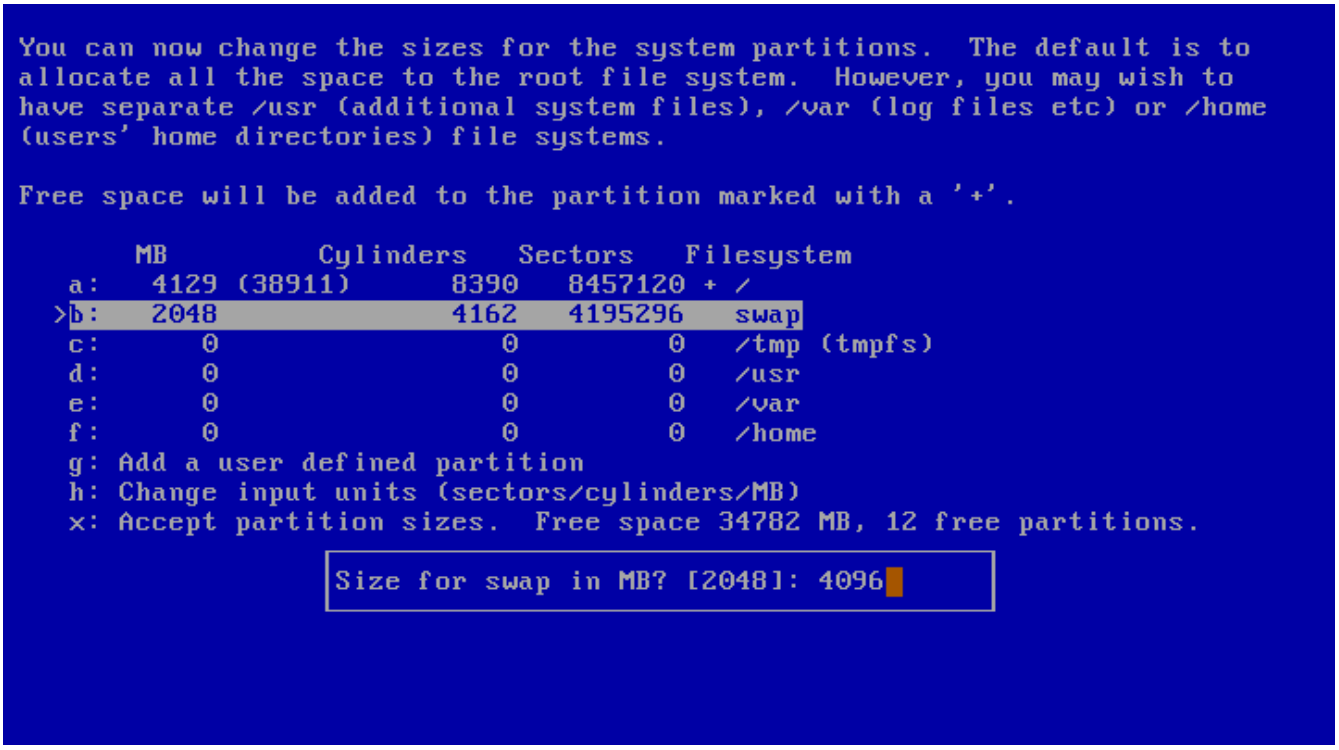


Рисунок 3.12. Задание размеров разделов

Как указано на рисунке 3.3, «Главное меню sysinst», пункты меню установки можно выбирать, нажимая букву, отображаемую слева от них. Учтите, что в этих меню они не всегда соответствуют буквам разделов disklabel BSD. Например, третья строка (буква «c») на рисунке 3.12, «Задание размеров разделов» относится не ко всему разделу NetBSD, а четвёртая строка (буква «d») не соответствует разделу «d» disklabel BSD.

Схема разделов по умолчанию, при которой используется одна большая корневая файловая система `/` (плюс подкачка), хорошо работает с NetBSD, и менять её почти не требуется. На рисунке 3.12, «Задание размеров разделов» показано, как изменить размер раздела подкачки на 4096 МБ. Учтите также, что раздел `/` помечен знаком «+», поэтому он займёт всё оставшееся свободное пространство (не отведённое ни под какой другой раздел). Перевести `/tmp` на *RAM-диск* (mount_tmpfs(8) или mfs(8)) ради дополнительной скорости может быть хорошей идеей. В других схемах разделов могут использоваться отдельные разделы для `/var`, `/usr` и/или `/home`, но решать, нужно ли это вам, следует исходя из собственного опыта. Закончив определять все нужные разделы, выберите «Accept partition sizes».

Следующий шаг — создать disklabel и при необходимости отредактировать его разделы с помощью редактора disklabel (Рисунок 3.13, «Редактор disklabel»). Если вы задали размеры разделов на предыдущем шаге, получившийся disklabel, скорее всего, будет соответствовать вашим пожеланиям. В этом случае можно сразу завершить процесс, выбрав «Partition sizes ok».

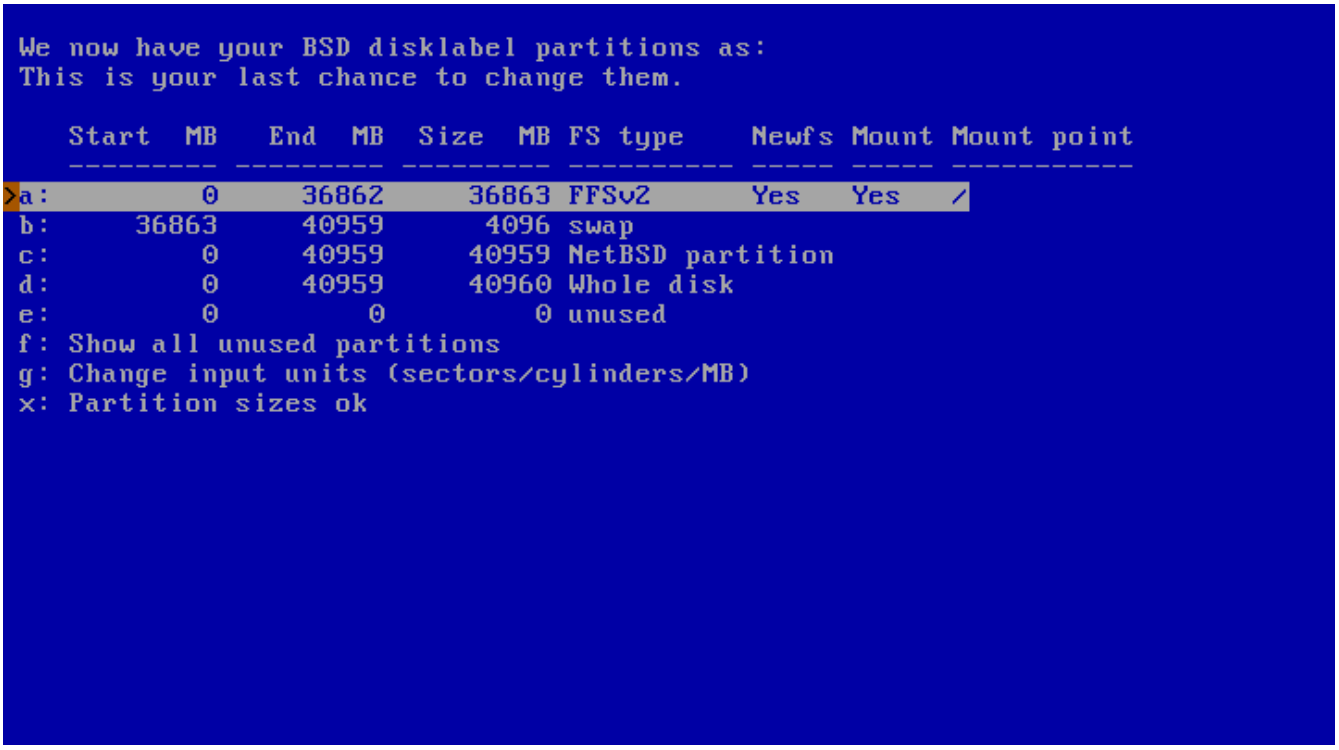


Рисунок 3.13. Редактор disklabel

Буквы на рисунке 3.13, «Редактор disklabel» используются для выбора строки *и* для обозначения соответствующих разделов disklabel BSD в значении, указанном в разделе 2.2.2, «Разделы». В порте amd64 есть два зарезервированных раздела: «c», обозначающий раздел NetBSD, и «d», обозначающий весь диск. Все остальные разделы можно редактировать клавишами управления курсором и клавишей Return либо соответствующими им буквами. Раздел можно добавить, выбрав неиспользуемый слот и задав параметры для этого раздела. Экран редактирования разделов показан на рисунке 3.14, «Редактирование разделов disklabel». Когда все значения вас устроят, выберите «Partition sizes ok».

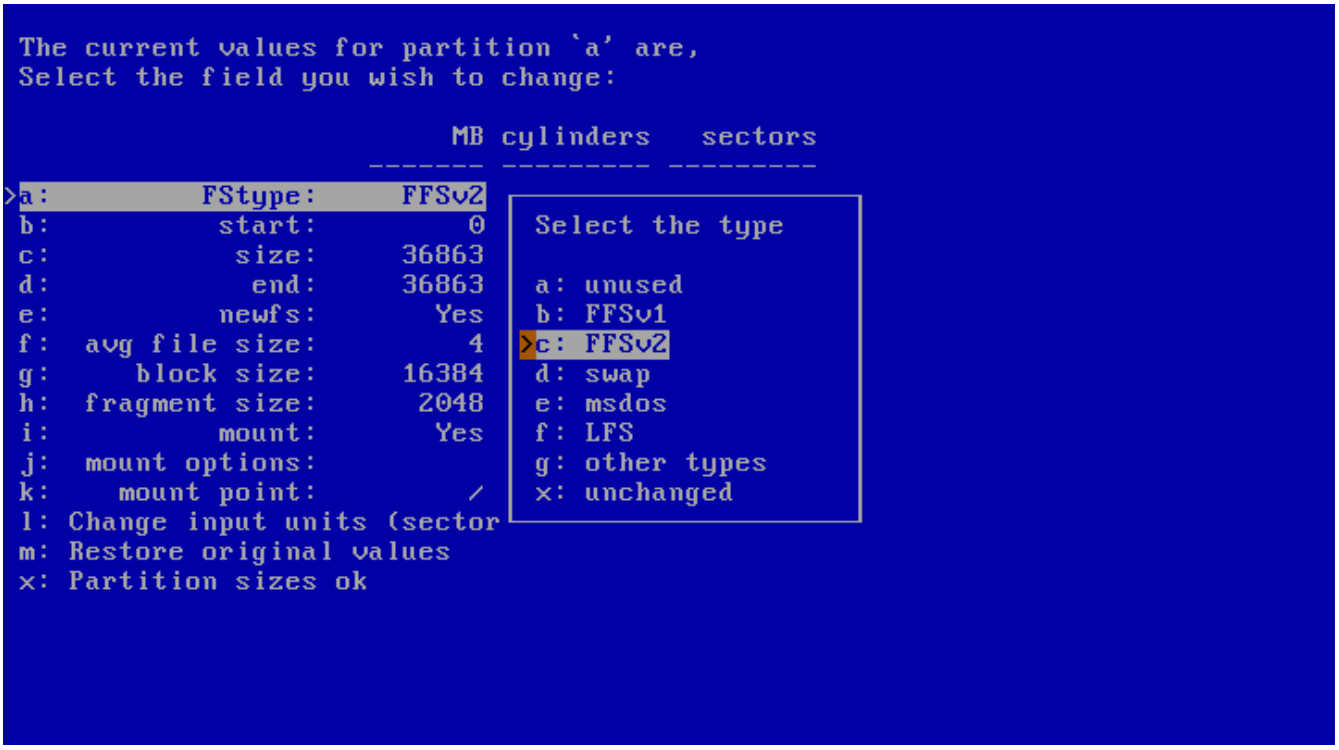


Рисунок 3.14. Редактирование разделов disklabel

3.7. Задание имени диска

После определения разделов в новом disklabel остаётся последний пункт — ввести имя диска NetBSD, как показано на рисунке 3.15, «Именование диска NetBSD». Позже это имя можно использовать, чтобы отличать disklabel внешне одинаковых дисков.

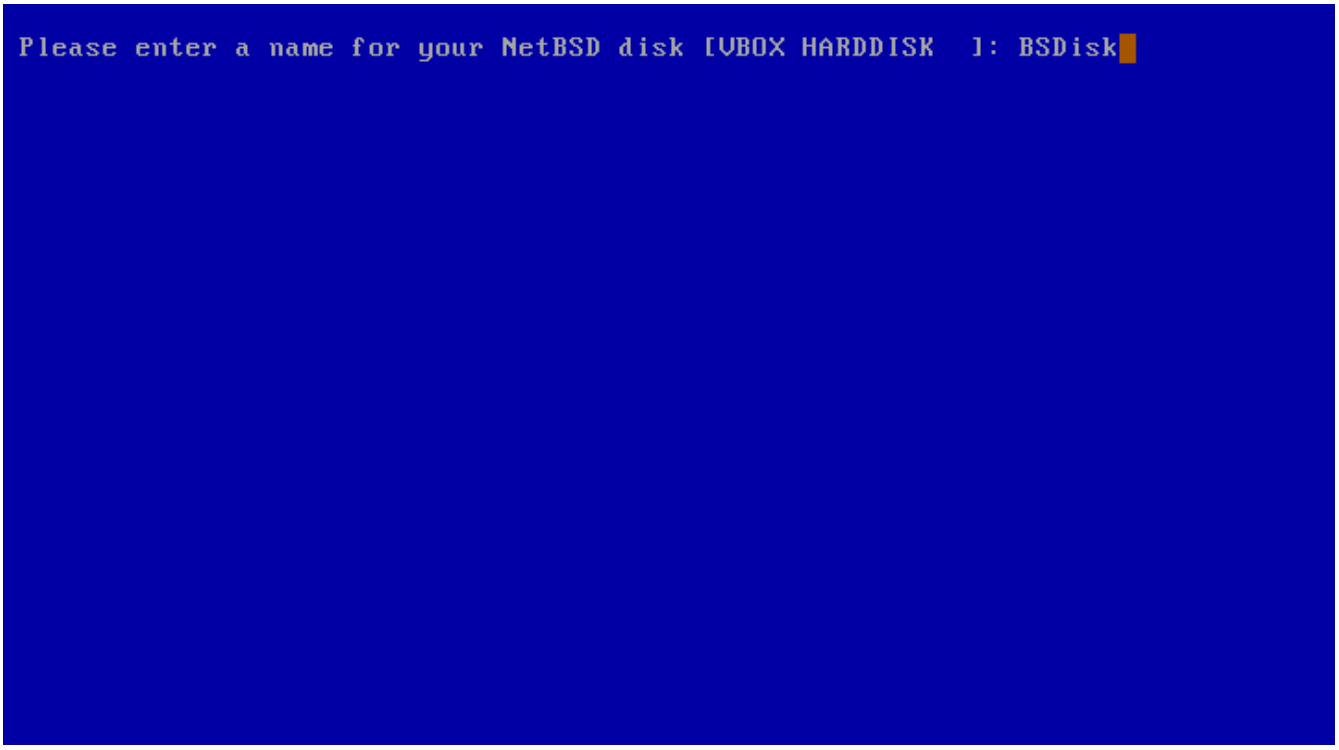


Рисунок 3.15. Именование диска NetBSD

3.8. Последний шанс!

Теперь у программы установки есть все данные, необходимые для подготовки диска. На диск пока ничего не записано, и сейчас у вас последняя возможность прервать процесс установки, прежде чем данные действительно будут записаны. Выберите «no», чтобы прервать процесс установки и вернуться в главное меню, либо продолжите, выбрав «yes».

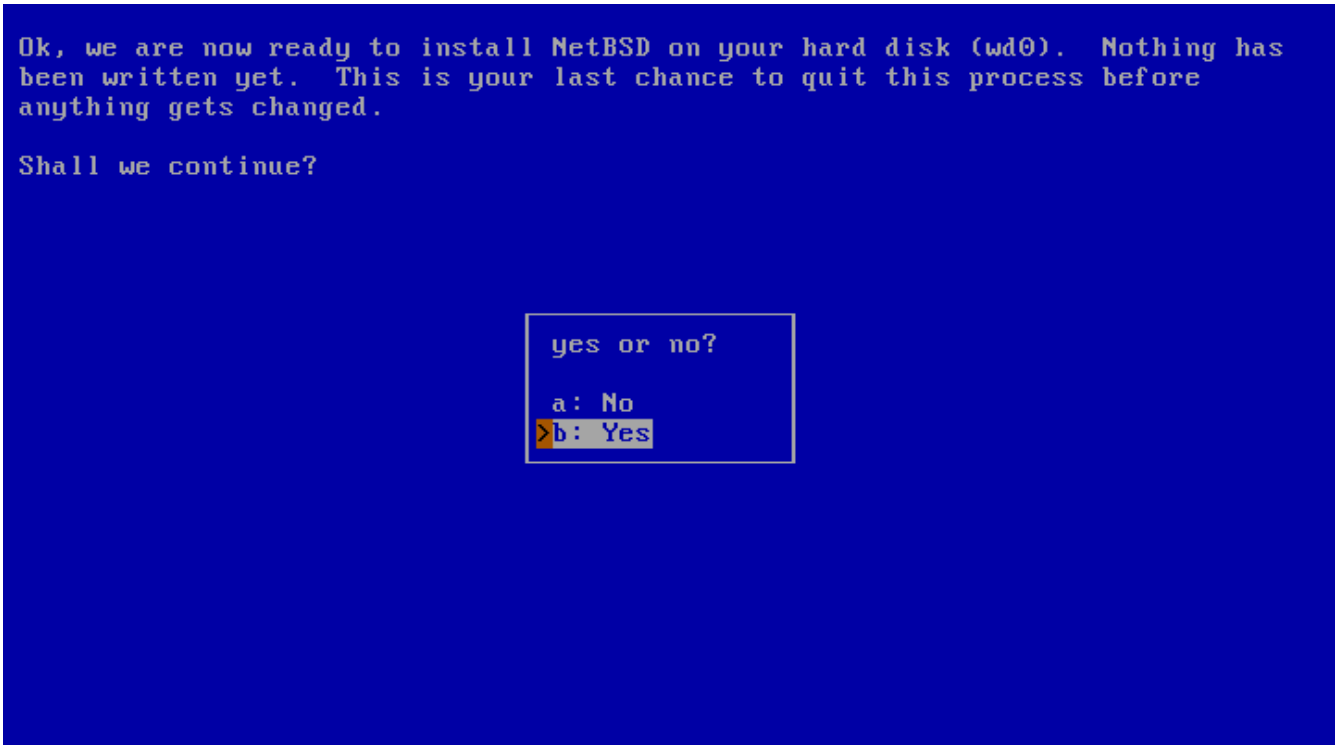


Рисунок 3.16. Последняя возможность прервать установку

3.9. Процесс подготовки диска

После подтверждения того, что sysinst должен подготовить диск, она запустит disklabel(8) для создания разметки разделов NetBSD и newfs(8) для создания файловых систем на диске.

После подготовки разделов NetBSD и их файловых систем возникает следующий вопрос (он показан на рисунке 3.17, «Выбор загрузочных блоков»): какие *загрузочные блоки* устанавливать. Обычно выбирают вариант по умолчанию — *BIOS console*, то есть вывод загрузочных сообщений на дисплей компьютера.

Если у вас парк машин без мониторов, может оказаться удобнее использовать последовательную консоль на одном из последовательных портов. Меню также позволяет изменить скорость последовательного порта, заданную по умолчанию: 9600 бод, 8 бит данных, без контроля чётности и один стоповый бит.

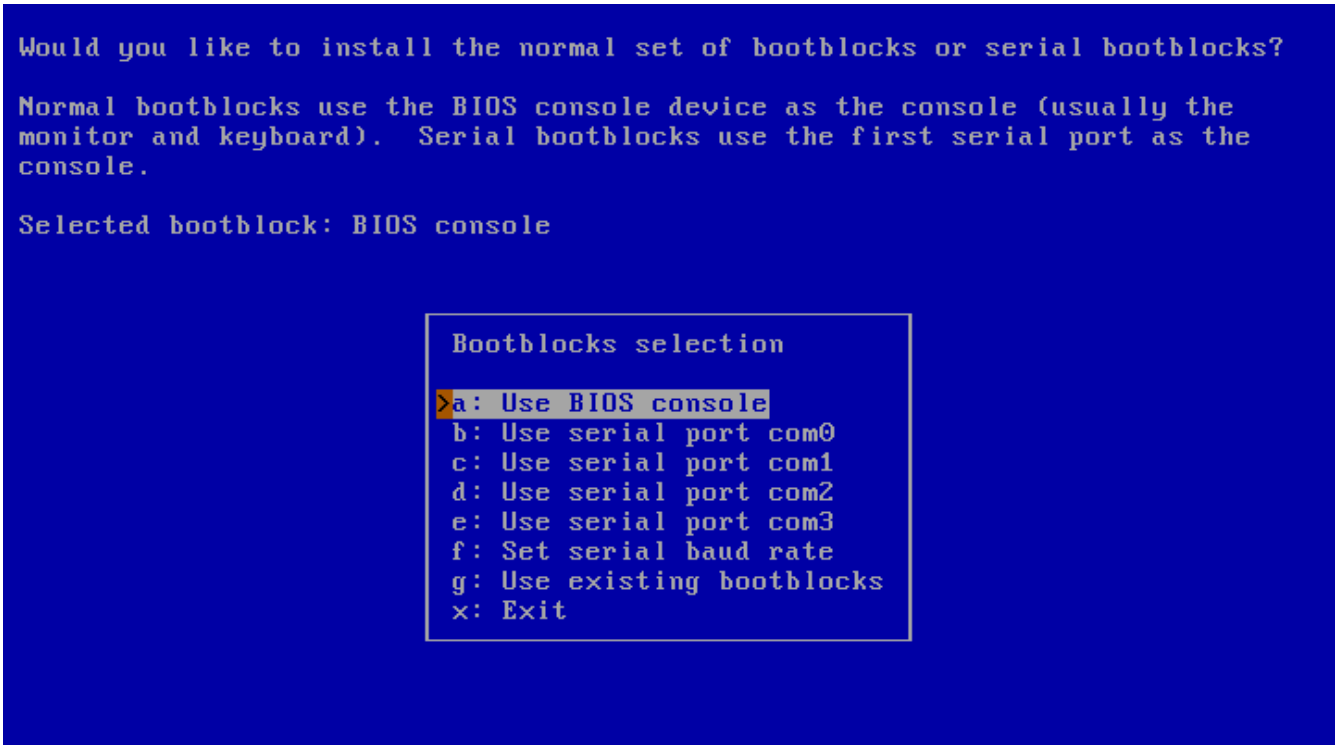


Рисунок 3.17. Выбор загрузочных блоков

3.10. Тип установки

Затем программа установки спросит, хотите ли вы выполнить полную, минимальную или выборочную установку. NetBSD поставляется в виде коллекции наборов дистрибутива. «Full installation» — вариант по умолчанию, он устанавливает все наборы; «Minimal installation» устанавливает только небольшой базовый набор — минимум, необходимый для работоспособной системы. Если выбрать «Custom installation», можно указать, какие наборы вы хотите установить. Этот шаг показан на рисунке 3.18, «Полная или выборочная установка».

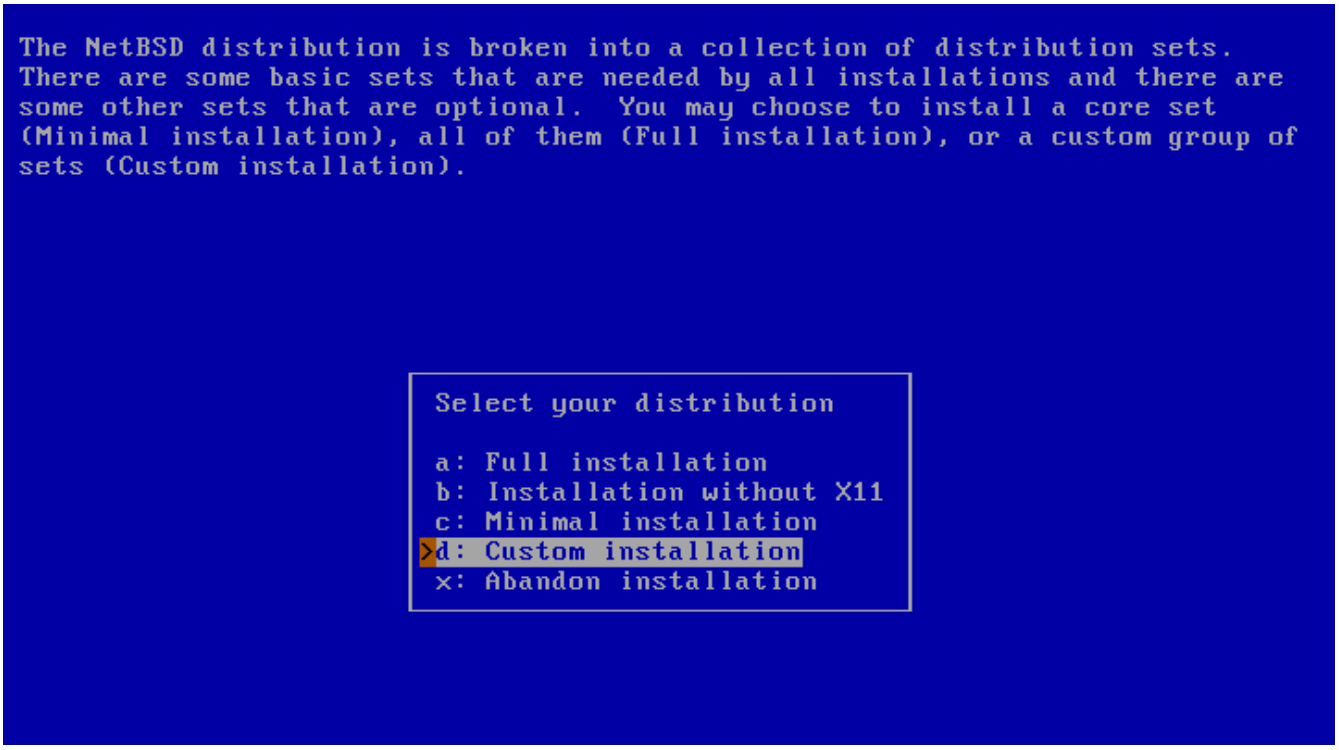


Рисунок 3.18. Полная или выборочная установка

Если вы выберете выборочную установку, sysinst позволит указать, какие наборы дистрибутива устанавливать, как показано на рисунке 3.19, «Выбор наборов дистрибутива». Как минимум необходимо выбрать ядро и наборы «Base» и «Configuration files (/etc)».

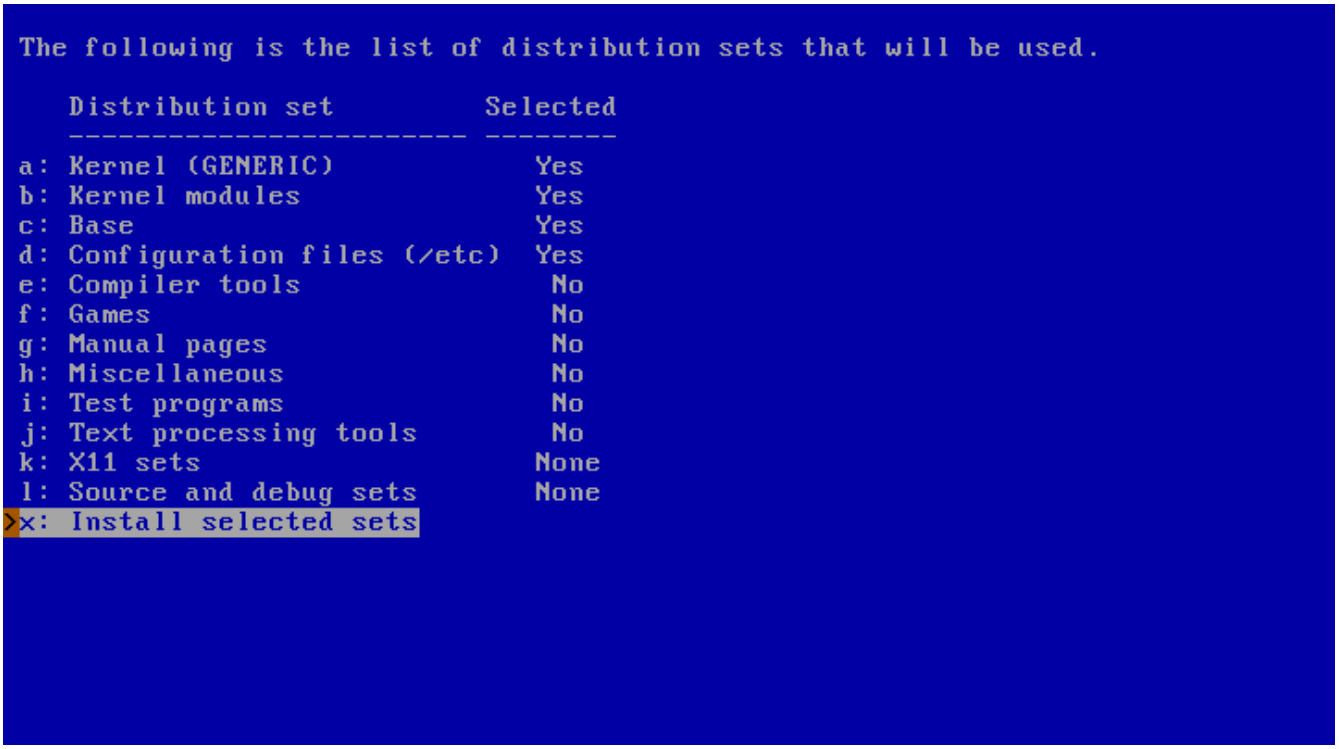


Рисунок 3.19. Выбор наборов дистрибутива

3.11. Выбор установочного носителя

На этом вы завершили первую и самую сложную часть установки!

Вторая половина процесса установки состоит в заполнении файловых систем — распаковке наборов дистрибутива, выбранных вами ранее («Base», «Compiler tools», «Games» и т. д.). Теперь sysinst нужно найти наборы NetBSD, и вы должны указать, где их искать: это может быть тот же носитель, на котором находится sysinst, или другой, по вашему выбору. Меню предлагает несколько вариантов, как показано на рисунке 3.20, «Установочные носители». Эти варианты подробно описаны в документах `INSTALL`.

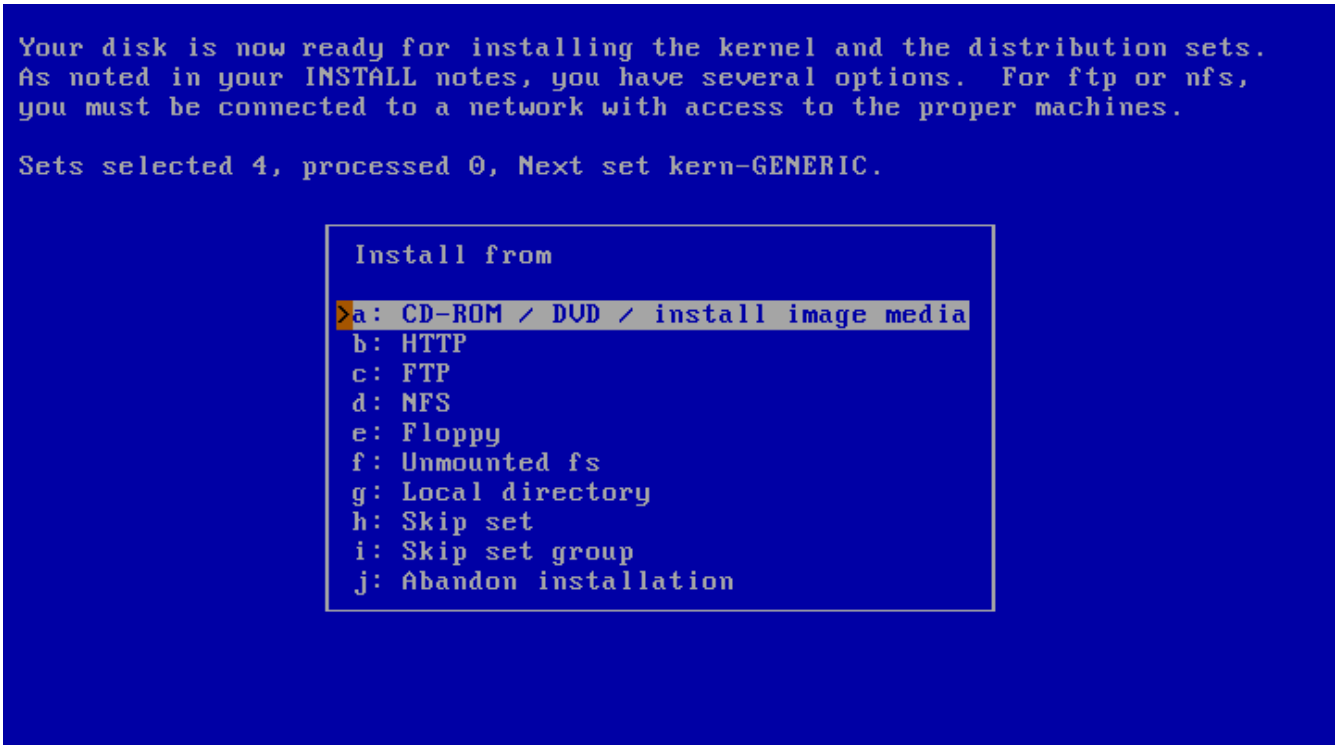


Рисунок 3.20. Установочные носители

3.11.1. Установка с CD-ROM / DVD / установочных образов

Выберите этот вариант, если хотите установить NetBSD с оптического носителя («CD-ROM / DVD») или с другого носителя, например с USB-накопителя. Если сам работающий sysinst был загружен с него, соответствующее устройство будет выбрано автоматически и начнётся извлечение наборов дистрибутива.

ИМЯ УСТРОЙСТВА CD-ROM/DVD ИЛИ ДРУГОГО УСТРОЙСТВА

Если sysinst не может обнаружить CD-ROM/DVD или USB-накопитель, дополнительные сведения о конфигурации оборудования можно получить так:

1. Нажмите «Ctrl+Z», чтобы приостановить sysinst и перейти в приглашение оболочки.
2. Введите команду:

```
# dmesg
```

   Так вы увидите сообщения, которые ядро выводит при запуске, в том числе сведения о необнаруженных или ненастроенных устройствах. Если первый привод CD-ROM или DVD в системе работает правильно, он обычно называется *cd0* — независимо от того, IDE это или SCSI (или даже USB, или FireWire). Первый USB-накопитель при правильной настройке называется *sd0*.
3. Если вывод прокручивается слишком быстро, можно также воспользоваться **more**:

```
# dmesg | more
```

4. Как указано в подсказке, вернуться к установке NetBSD можно, набрав «exit» или «^D» («Ctrl+D»).

3.11.2. Установка из немонтированной файловой системы

Рисунок 3.21, «Монтирование файловой системы» показывает меню установки NetBSD из немонтированной файловой системы. Необходимо указать устройство («Устройство»), тип его файловой системы («Файловая система») и корневой каталог внутри неё («Базовый каталог»). Двоичные установочные наборы и наборы исходных текстов — это файлы `.tgz`. По умолчанию точка монтирования — «mnt» в amd64. Путь формируется следующим образом:

```
/<default mountpoint>/<Base directory>/<Binary set directory> or <Source set
directory>/set.tgz
```

Выберите такое сочетание пунктов «Базовый каталог» и «Каталог двоичных наборов» (или «Каталог наборов исходных текстов»), при котором получается правильный путь в вашей немонтированной файловой системе. Если встречается несколько идущих подряд `/`, фактически учитывается только первый `/`. Указывать «Каталог наборов исходных текстов» нужно только в том случае, если вы ранее выбрали установку исходных текстов. Наборы исходных текстов обычно не входят в установочные образы.

В следующем примере установочные наборы хранятся в файловой системе *MSDOS*, в разделе «e» на устройстве «sd0».

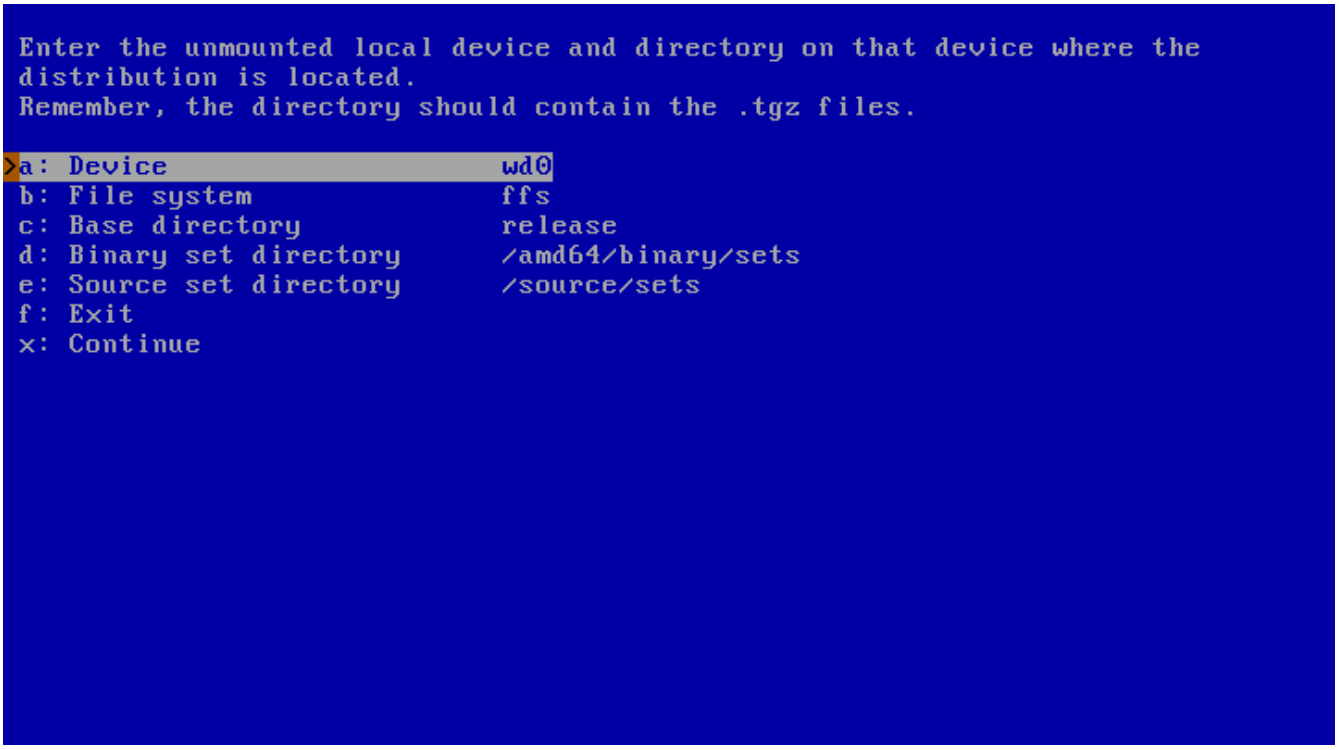


Рисунок 3.21. Монтирование файловой системы

Укажите имя устройства и раздел. Рисунок 3.22, «Монтирование раздела» показывает, как указать устройство «sd0» с разделом «e».

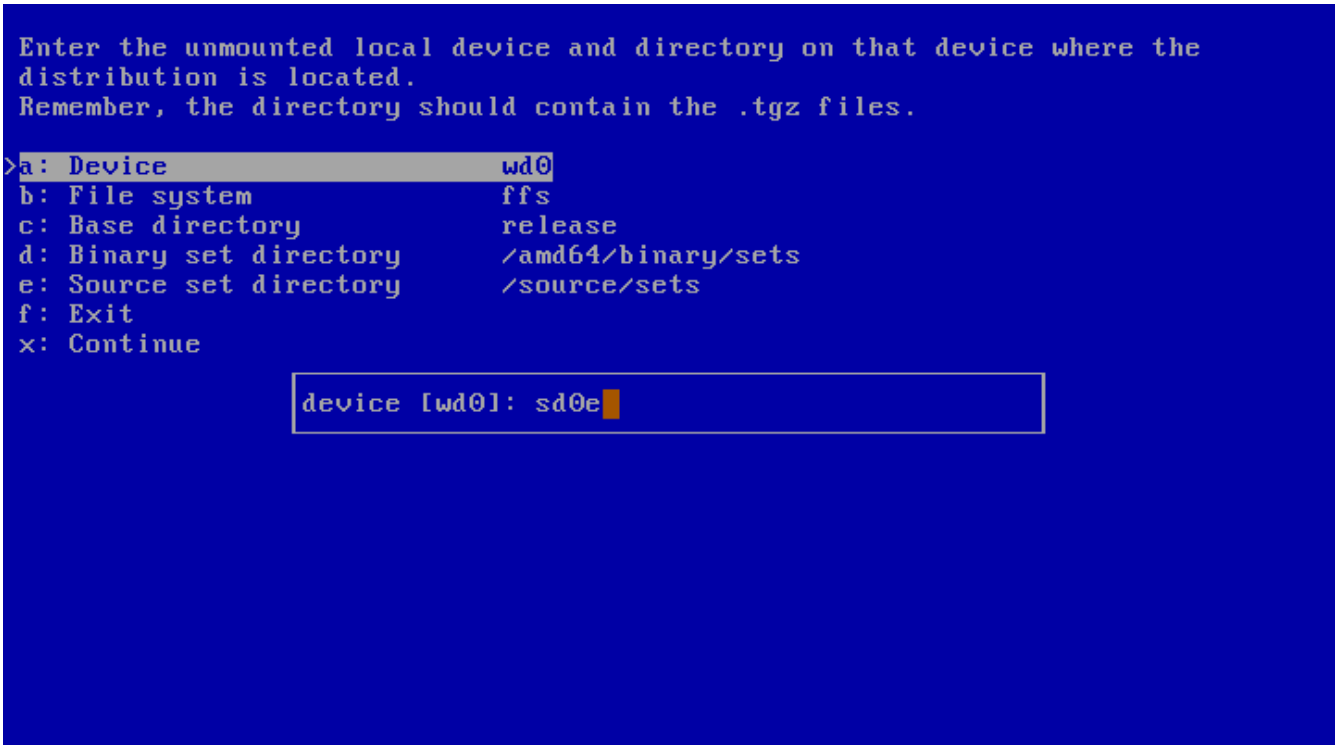


Рисунок 3.22. Монтирование раздела

На рисунке 3.23, «Доступ к файловой системе MSDOS» указан тип файловой системы «msdos». Это значение используется для формирования команды `mount_<File system>` для монтирования тома. Принимается любая строка (обозначающая тип «файловой системы»), из которой получается допустимая команда: например, файловая система NetBSD «ffs» или «ext2fs», файловая система Linux. В этом примере пункт «Базовый каталог» оставлен пустым, а двоичные наборы хранятся в `/sets`, поэтому путь получается таким:

```
/mnt///sets
```

Если не учитывать повторяющиеся `/`, это эквивалентно `/mnt/sets` и является допустимым путём. Выбор «Продолжить» запустит извлечение наборов.

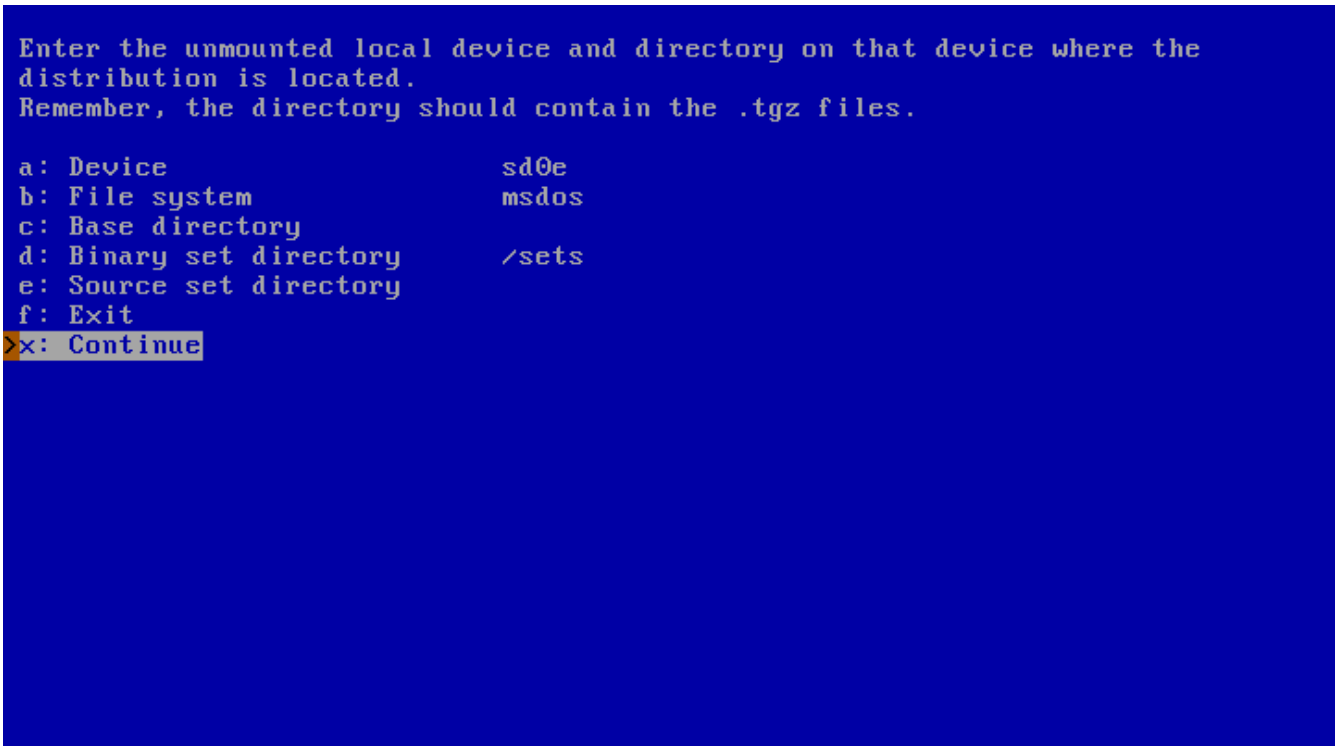


Рисунок 3.23. Доступ к файловой системе MSDOS

3.11.3. Установка по FTP и настройка сети

Если вы решили устанавливать систему из локальной сети или через интернет по FTP, нужно указать sysinst, откуда правильно получить наборы дистрибутива, как показано на рисунке 3.24, «Определение параметров FTP».

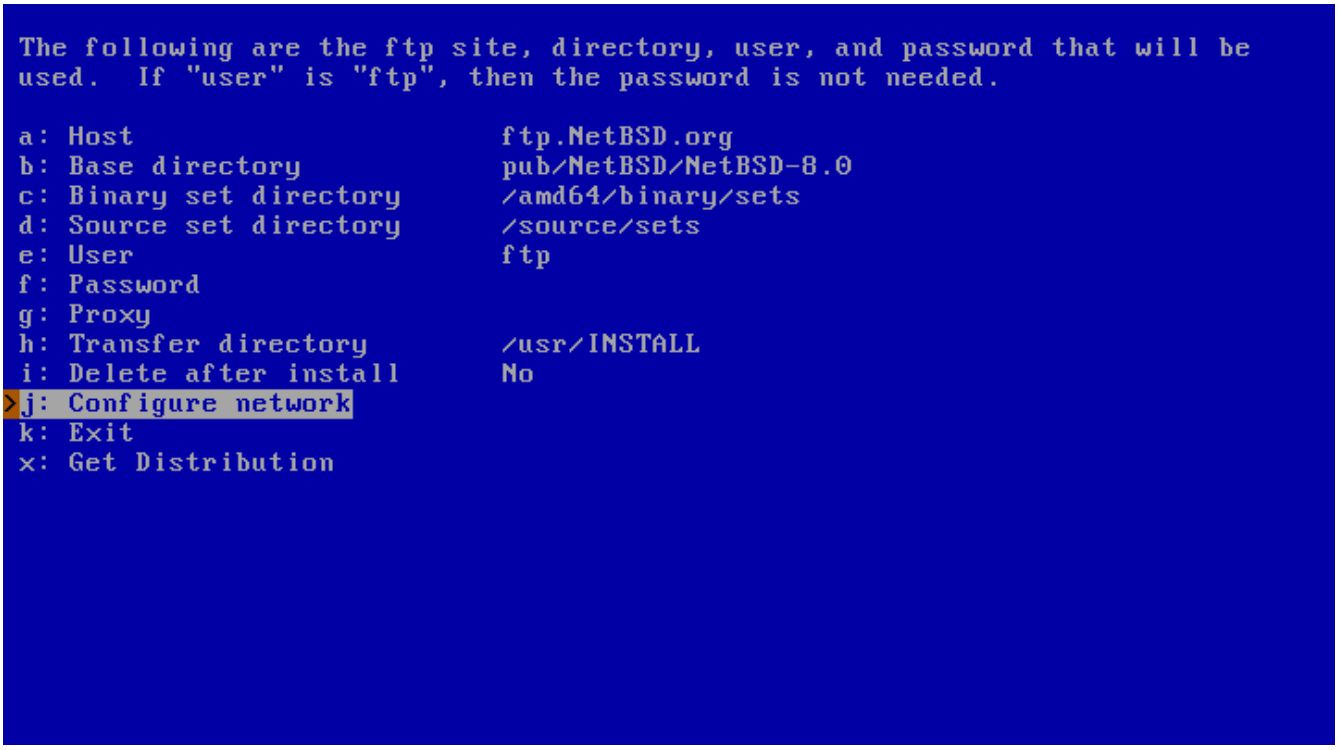


Рисунок 3.24. Определение параметров FTP

Значения по умолчанию в большинстве случаев подходят. Кроме того, прежде чем продолжить, необходимо настроить сетевое подключение: перейдите к соответствующему пункту меню, нажав клавишу «j».

В настоящее время NetBSD поддерживает установку по Ethernet, через USB Ethernet или беспроводное соединение, а также по беспроводной локальной сети. Установка через DSL (PPP over Ethernet) во время установки не поддерживается.

На первом шаге, показанном на рисунке 3.25, «Какой сетевой интерфейс настроить», необходимо выбрать сетевую карту для настройки. sysinst определит список доступных сетевых интерфейсов, покажет их и спросит, какой из них использовать.

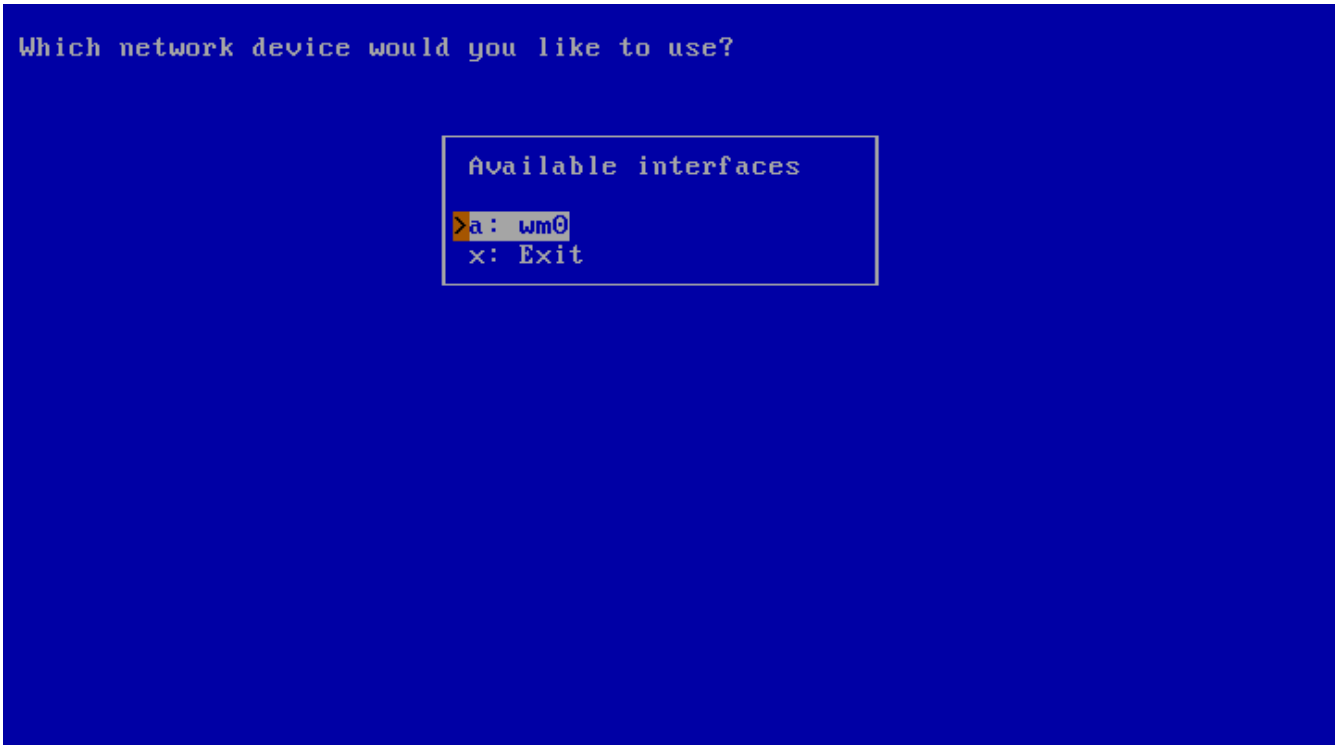


Рисунок 3.25. Какой сетевой интерфейс настроить

ПРИМЕЧАНИЕ

Точные имена сетевых интерфейсов зависят от используемого оборудования. Примеры интерфейсов: «wm» для интерфейсов Intel Gigabit, «ne» для карт NE2000 и совместимых с ними Ethernet-карт, «ath» для беспроводных карт на базе Atheros. Этот список далеко не полон: NetBSD поддерживает гораздо больше сетевых устройств.

Если вашего сетевого устройства нет в списке на рисунке 3.25, «Какой сетевой интерфейс настроить», возможно, оно не было правильно обнаружено. Чтобы получить список сетевых интерфейсов, доступных в системе, прервите процесс установки, нажав «Ctrl+Z», и затем введите

```
# ifconfig -a
wm0: flags=0x8802<BROADCAST,SIMPLEX,MULTICAST> mtu 1500
        capabilities=2bf80<TSO4,IP4CSUM_Rx,IP4CSUM_Tx,TCP4CSUM_Rx>
        capabilities=2bf80<TCP4CSUM_Tx,UDP4CSUM_Rx,UDP4CSUM_Tx,TCP6CSUM_Tx>
        capabilities=2bf80<UDP6CSUM_Tx>
        enabled=0
        ec_capabilities=7<VLAN_MTU,VLAN_HWTAGGING,JUMBO_MTU>
        ec_enabled=0
        address: 08:00:27:7e:85:d7
        media: Ethernet autoselect (1000baseT full-duplex)
        status: active
lo0: flags=0x8048<LOOPBACK,RUNNING,MULTICAST> mtu 33624
```

Если нужный интерфейс не появился, получите дополнительные сведения обо всех устройствах, обнаруженных при загрузке системы. Введите:

```
# dmesg | more
```

Как указано в подсказке, вернуться к установке NetBSD можно, набрав «exit» или «^D» («Ctrl+D»).

Далее у вас есть возможность задать среду передачи данных сети. Нажмите «Enter», чтобы выбрать значение по умолчанию.

ПРИМЕЧАНИЕ

Вряд ли здесь вам понадобится что-либо, кроме значения по умолчанию. Если возникают проблемы, например очень медленная передача или тайм-ауты, можно, скажем, принудительно задать другие параметры дуплекса для Ethernet-карт. Чтобы получить список поддерживаемых сред и параметров среды для конкретного сетевого устройства («wm0», например), выйдите из sysinst, нажав «Ctrl+Z», и введите:

```
# ifconfig -m wm0
wm0: flags=0x8802<BROADCAST,SIMPLEX,MULTICAST> mtu 1500
        capabilities=2bf80<TSO4,IP4CSUM_Rx,IP4CSUM_Tx,TCP4CSUM_Rx>
        capabilities=2bf80<TCP4CSUM_Tx,UDP4CSUM_Rx,UDP4CSUM_Tx,TCP6CSUM_Tx>
        capabilities=2bf80<UDP6CSUM_Tx>
        enabled=0
        ec_capabilities=7<VLAN_MTU,VLAN_HWTAGGING,JUMBO_MTU>
        ec_enabled=0
        address: 08:00:27:7e:85:d7
        media: Ethernet autoselect (1000baseT full-duplex)
        status: active
        supported Ethernet media:
                media none
                media 10baseT
                media 10baseT mediaopt full-duplex
                media 100baseTX
                media 100baseTX mediaopt full-duplex
                media autoselect
```

Здесь могут оказаться полезными несколько значений, выводимых после «media», в том числе ключевые слова вроде «autoselect», а также возможные настройки «mediaopt».

Вернитесь к установке, набрав «exit» или «^D» («Ctrl+D»).

Следующий вопрос, показанный на рисунке 3.26, «Использование автонастройки», — хотите ли вы выполнить автонастройку. Эта процедура использует DHCP (*Dynamic Host Configuration Protocol*). sysinst получит от DHCP ряд значений по умолчанию, которые, скорее всего, окажутся правильными. Эту процедуру рекомендуется выполнить, если только вы не хотите задать статический IP-адрес и/или указать какие-то особые параметры.

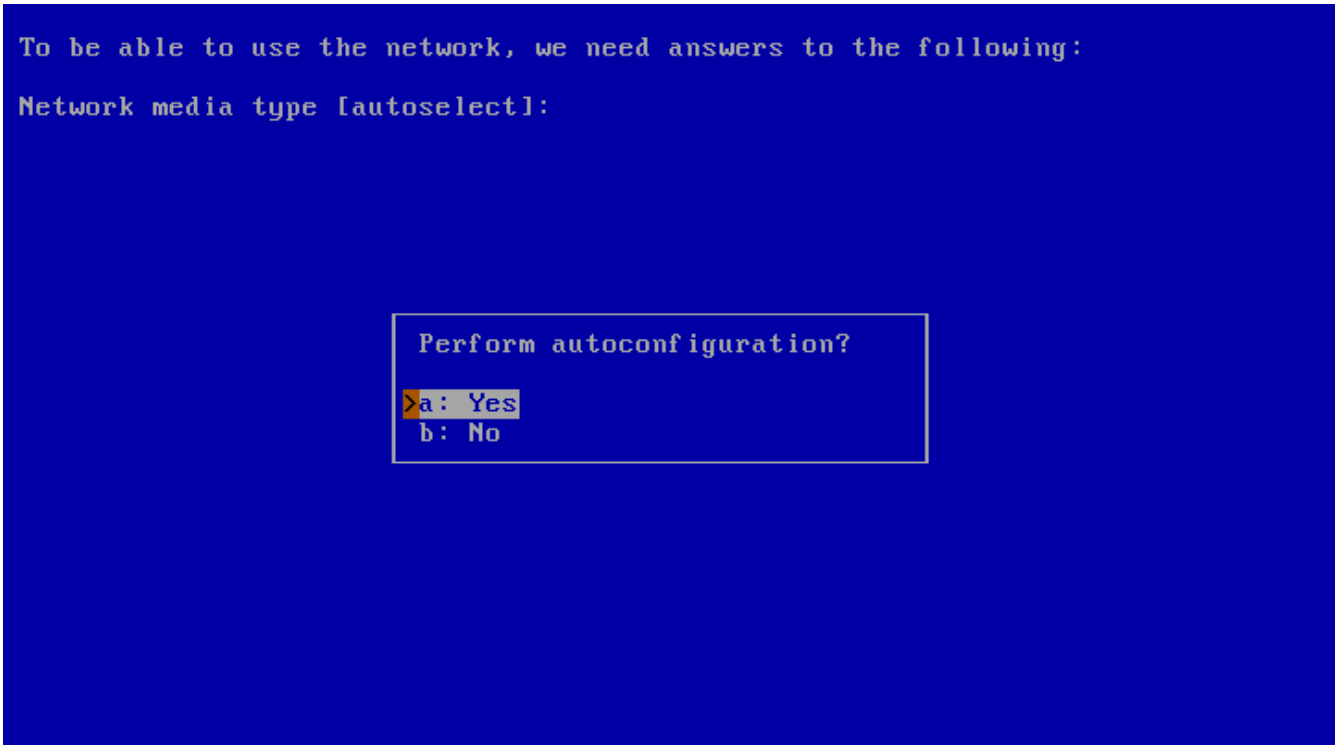


Рисунок 3.26. Использование автонастройки

Затем вас спросят про «Домен DNS»; если машина не входит в зарегистрированный публичный домен, его можно оставить пустым.

В конце этой процедуры показывается список всех настроек, как на рисунке 3.27, «Подтверждение автонастройки». Если они верны, выберите «Да». В противном случае, выбрав «Нет», вы запустите настройку сети заново с самого начала и сможете снова пройти все шаги (а также выполнить настройку вручную).

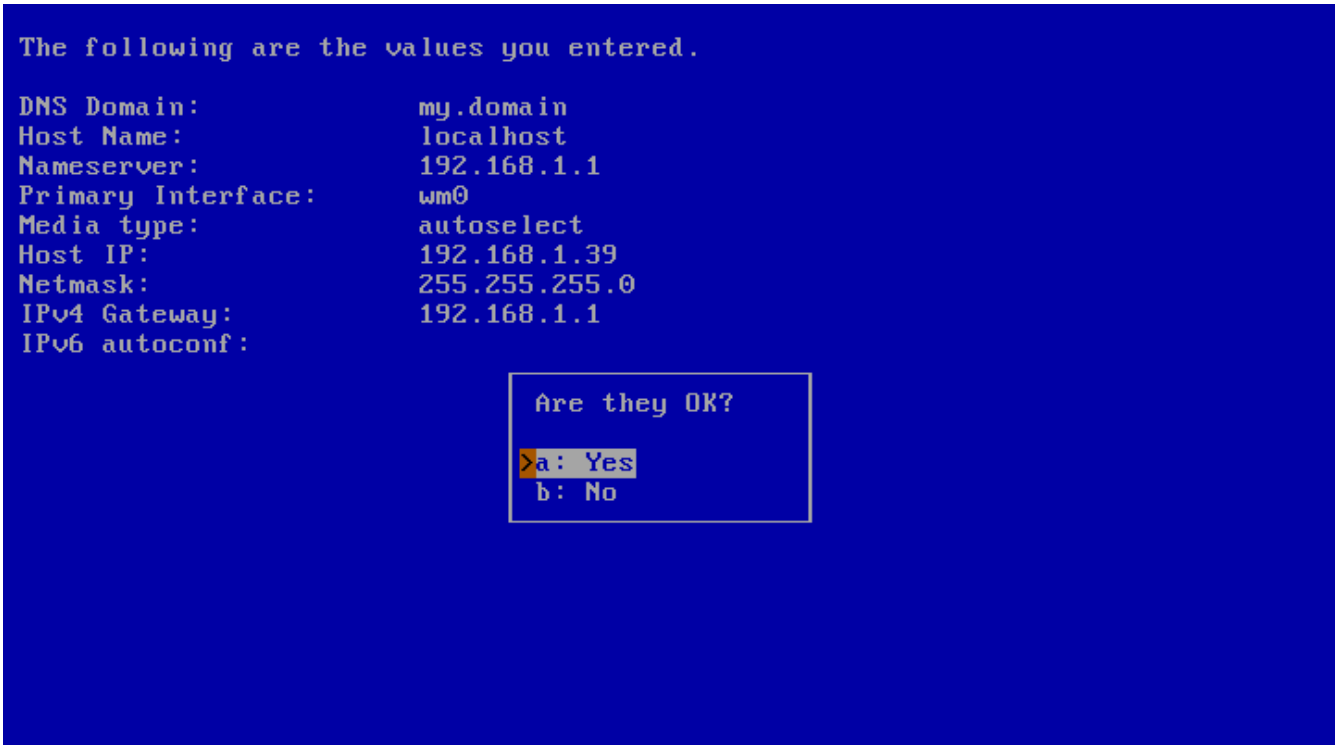


Рисунок 3.27. Подтверждение автонастройки

Если вы выбрали «Нет» в ответ на вопрос с рисунка 3.26, «Использование автонастройки», вам зададут несколько вопросов для настройки сети вручную. Все параметры предлагаются в виде «Имя_параметра [значение_по_умолчанию]:». Нажмите «Enter», чтобы использовать значение по умолчанию. Если значение по умолчанию не задано, параметр останется пустым.

Имя вашего хоста:

Имя, по которому другие машины обычно могут обращаться к вашему компьютеру. При установке не используется.

Домен DNS:

Это имя домена, в котором вы находитесь. Его можно оставить пустым, если вы не в публичном домене.

Ваш IPv4-адрес:

Введите здесь ваш числовой адрес Internet Protocol в записи «точечная четвёрка», например 192.168.1.3. Он будет использоваться как статический IP-адрес вашей сетевой карты.

Маска сети IPv4:

Маска сети для вашей сети, заданная либо в шестнадцатеричном виде («0xffffff00»), либо в виде четырёх чисел через точки («255.255.255.0»).

Шлюз IPv4:

IP-адрес вашего маршрутизатора (или шлюза по умолчанию). Не используйте здесь имя хоста!

Ваш сервер имён:

IP-адрес вашего (первого) DNS-сервера. И снова: не используйте имя хоста.

После того как вы ответите на все вопросы о настройках сети, их список будет показан, как на рисунке 3.27, «Подтверждение автонастройки». У вас будет возможность вернуться и внести

изменения. Если настройки вас устраивают, выберите «Да».

sysinst теперь выполнит несколько команд (подробно здесь не показываемых) для настройки сети: очистку таблицы маршрутизации, установку маршрута по умолчанию и проверку работоспособности сетевого подключения.

Теперь, когда сетевое подключение работает, снова будет показано меню с рисунка 3.24, «Определение параметров FTP». Выберите «Получить дистрибутив», чтобы продолжить: sysinst скачает выбранные файлы наборов во временный каталог, а затем извлечёт их.

3.11.4. Установка по NFS

Если вы хотите установить NetBSD с сервера в локальной сети, альтернативой FTP служит NFS.

> **ПРИМЕЧАНИЕ**
>
> Использование этого способа установки требует умения настроить сервер NFS; эта тема здесь не рассматривается.

Как показано на рисунке 3.28, «Экран установки по NFS», необходимо указать: IP-адрес сервера NFS в поле «„Host“»; каталог, *экспортируемый* сервером NFS, в поле «„Base directory“»; каталог, содержащий установочные наборы, в поле «„Set directory“».

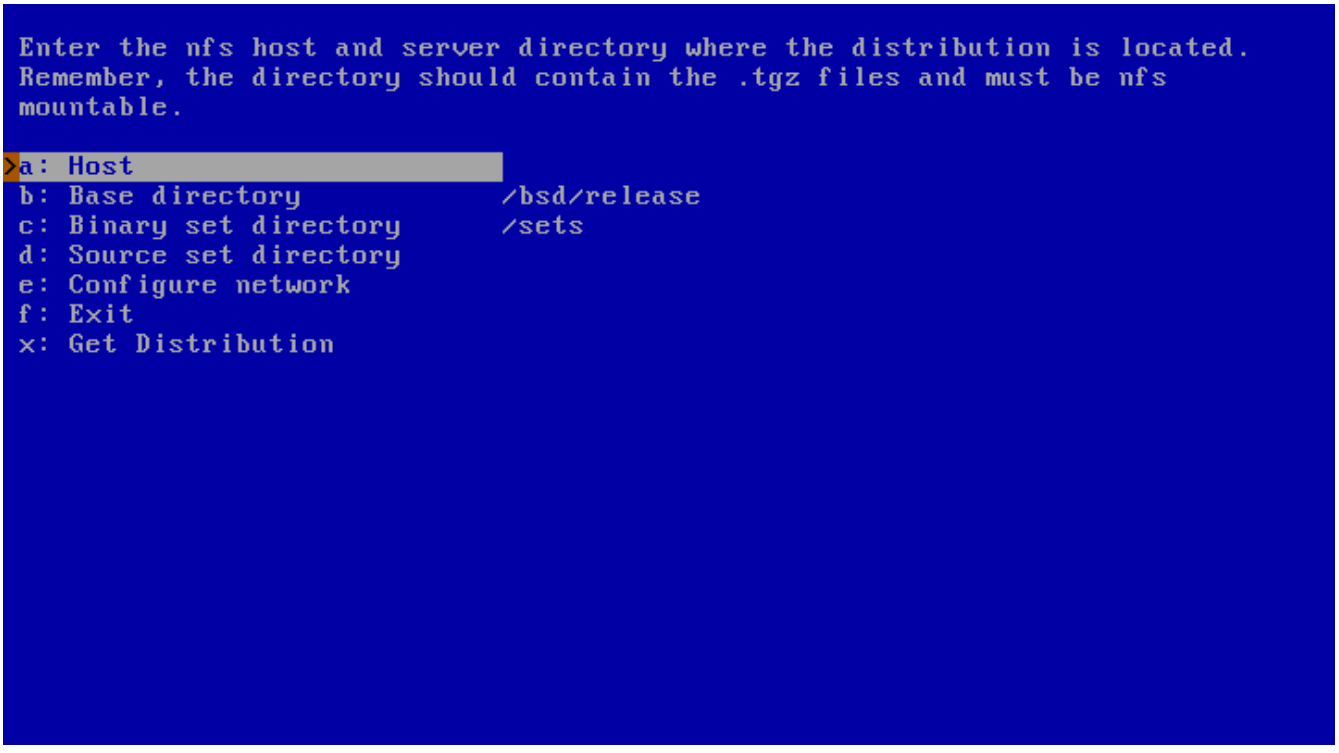


Рисунок 3.28. Экран установки по NFS

Рисунок 3.29, «Пример NFS» показывает пример: хост «„192.168.1.50“» — это сервер NFS, который экспортирует каталог `/home/username/Downloads`. Установочные наборы NetBSD хранятся в каталоге `/home/username/Downloads/sets` на сервере NFS. Выберите «„Continue“», чтобы начать установку наборов дистрибутива.

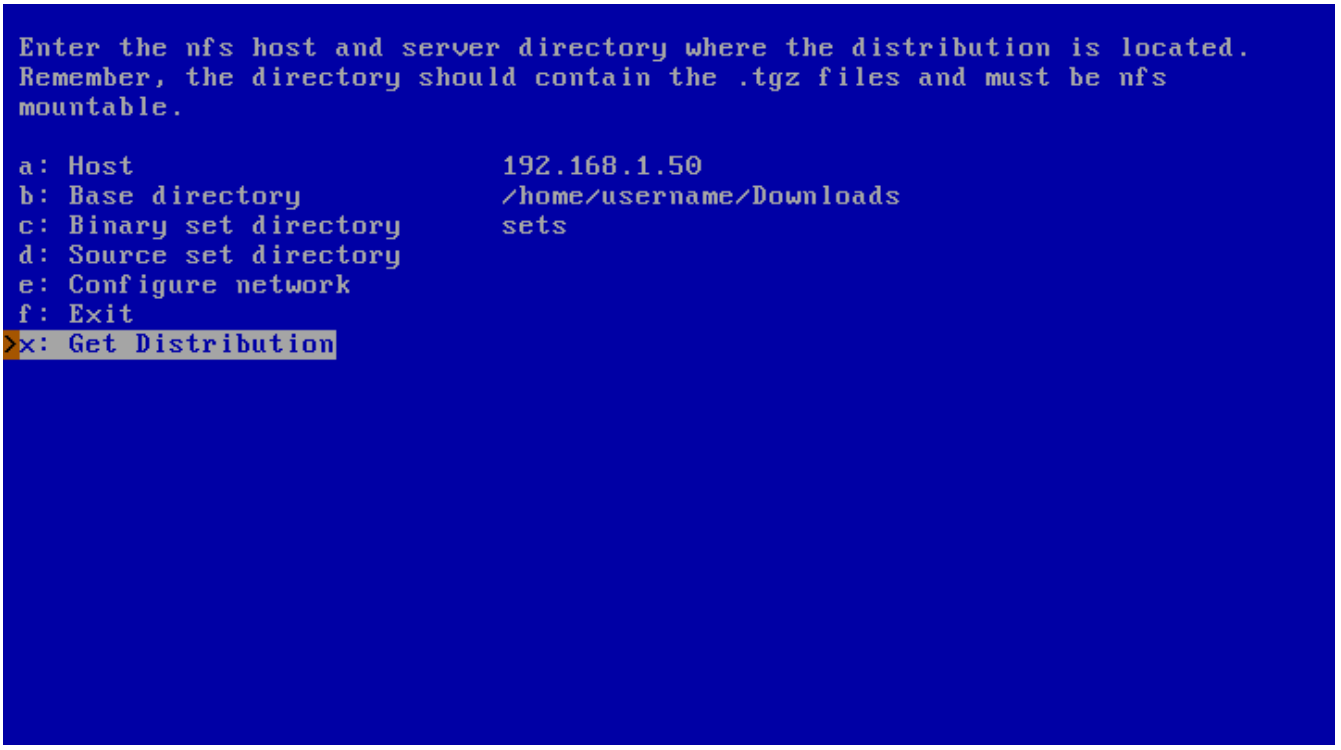


Рисунок 3.29. Пример NFS

3.12. Распаковка наборов

После выбора способа получения наборов дистрибутива и (если это применимо) переноса этих наборов они распаковываются в новую файловую систему NetBSD.

Сообщение (см. рисунок 3.30, «Распаковка наборов завершена») уведомит вас о том, что распаковка наборов завершена и что у вас есть возможность выполнить базовую настройку перед завершением установки NetBSD.



Рисунок 3.30. Распаковка наборов завершена

3.13. Настройка системы

Меню со всеми доступными параметрами настройки выглядит так, как показано на рисунке 3.31, «Меню настройки». После настройки каждого пункта вы возвращаетесь в это меню и можете выбрать другой.

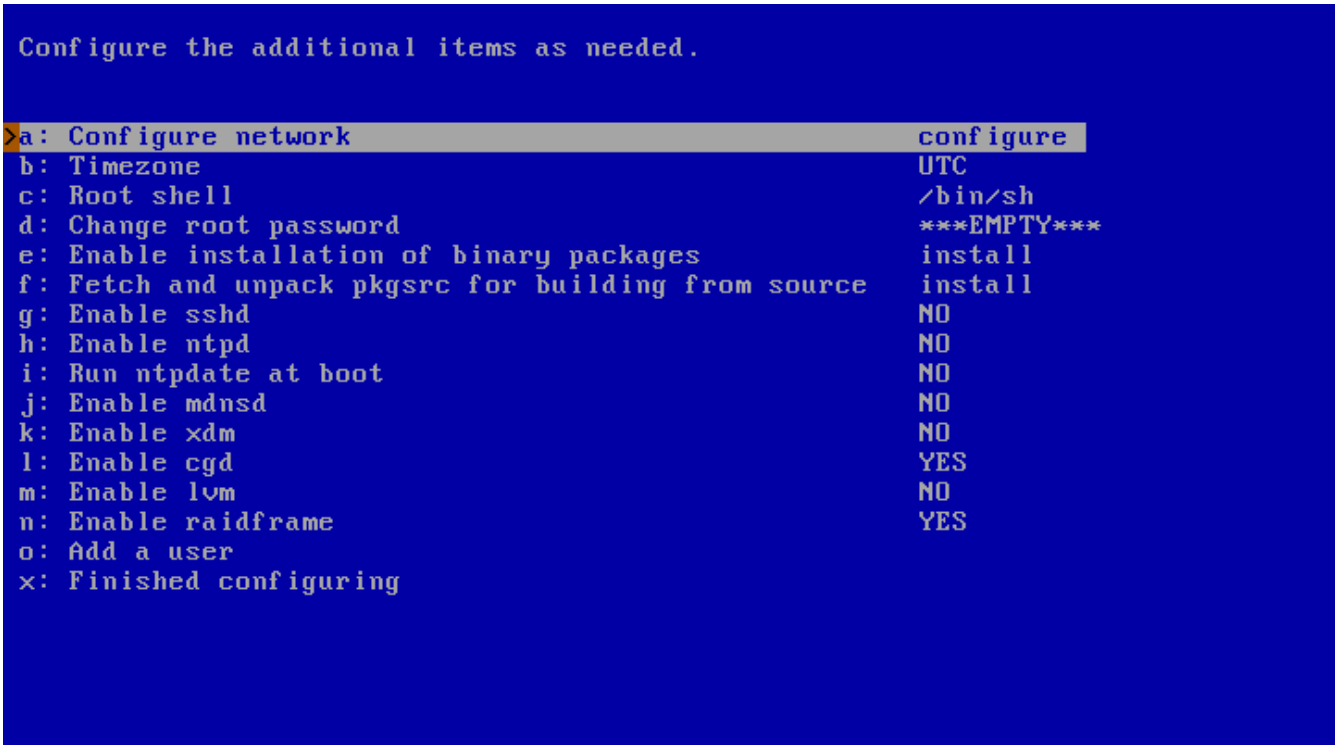


Рисунок 3.31. Меню настройки

Если вы ещё не настроили сеть, это можно сделать сейчас, выполнив ту же процедуру, что описана в разделе 3.11.3, «Установка по FTP и настройка сети».

Часовой пояс тоже можно настроить. По умолчанию используется *всемирное координированное время* (UTC). С помощью двухуровневого меню *Continents/Countries and cities*, показанного на рисунке 3.32, «Выбор часового пояса системы», выберите свой местный часовой пояс клавишей Return. После выбора допустимого значения курсор автоматически переместится на пункт «„Exit“». Затем просто нажмите Return, чтобы выйти из выбора часового пояса.

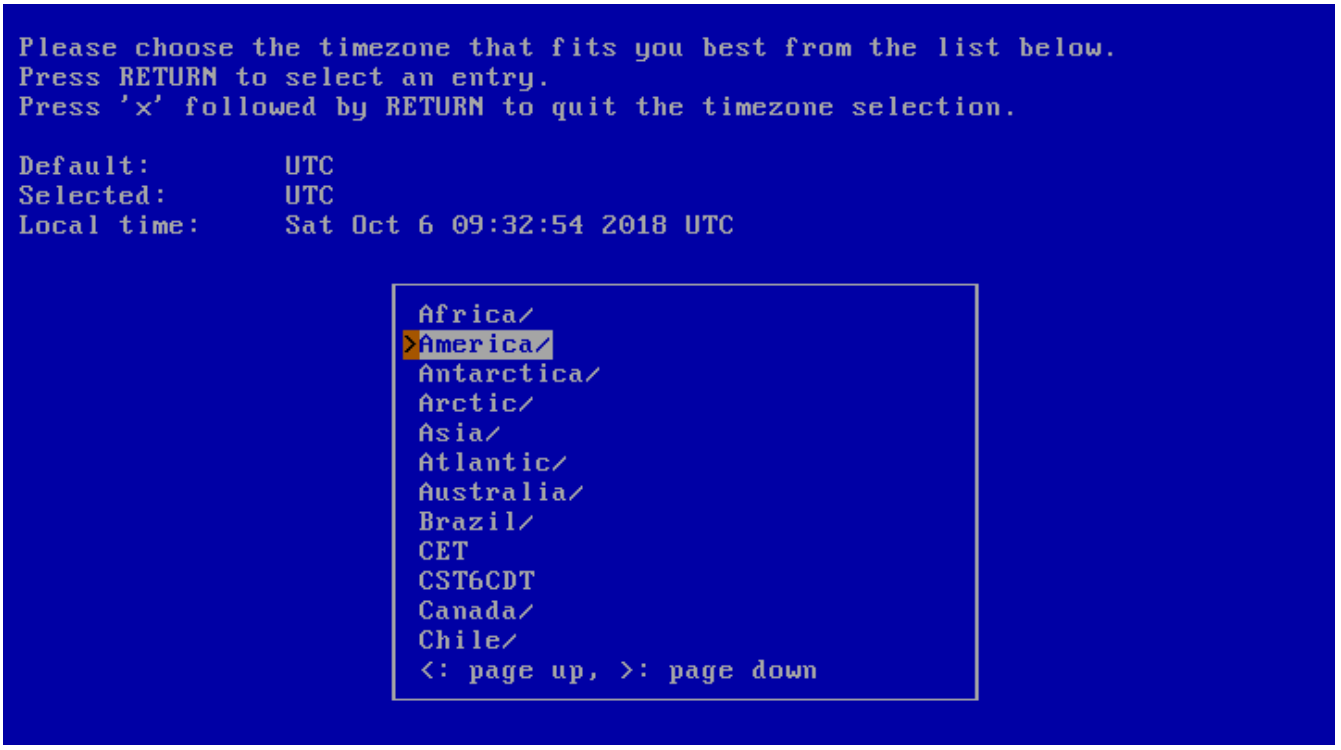


Рисунок 3.32. Выбор часового пояса системы

Следующий пункт рисунка 3.31, «Меню настройки» позволяет выбрать командный интерпретатор — также известный как «„оболочка“» — который будет использоваться для учётной записи root. По умолчанию используется совместимая с Bourne *оболочка Almquist*, sh(1). Другие варианты — *оболочка Korn* (ksh(1)) и *оболочка C* (csh(1)). Если, прочитав это, вы ещё не знаете, какую оболочку предпочитаете, просто используйте вариант по умолчанию — это решение крайне субъективное. Если позже вы передумаете, оболочку root всегда можно изменить.

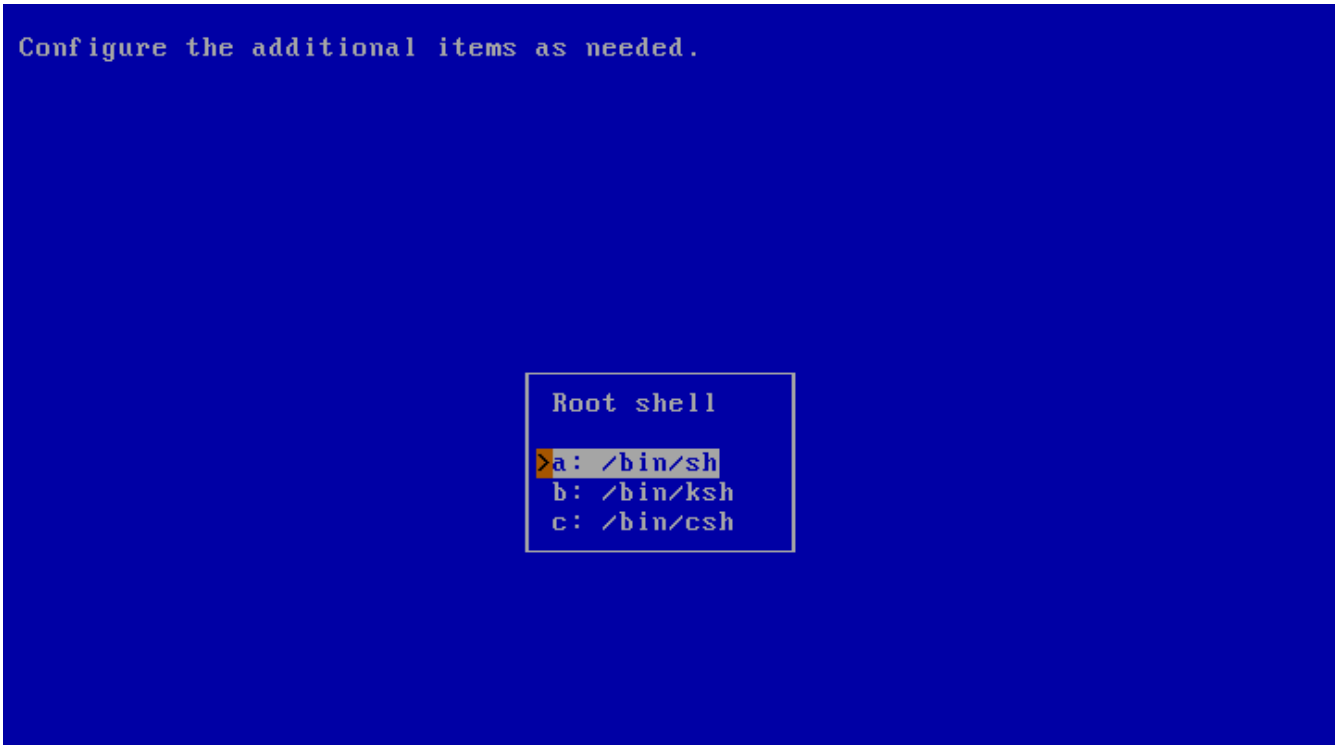


Рисунок 3.33. Выбор оболочки

Учётная запись root по-прежнему не имеет пароля. Из соображений безопасности пароль рекомендуется задать сейчас, выбрав соответствующий пункт на рисунке 3.31, «Меню настройки».



Рисунок 3.34. Задать пароль root?

Когда вы согласитесь задать пароль root, sysinst запустит для вас утилиту passwd(1). Учтите, что вводимый пароль не отображается.

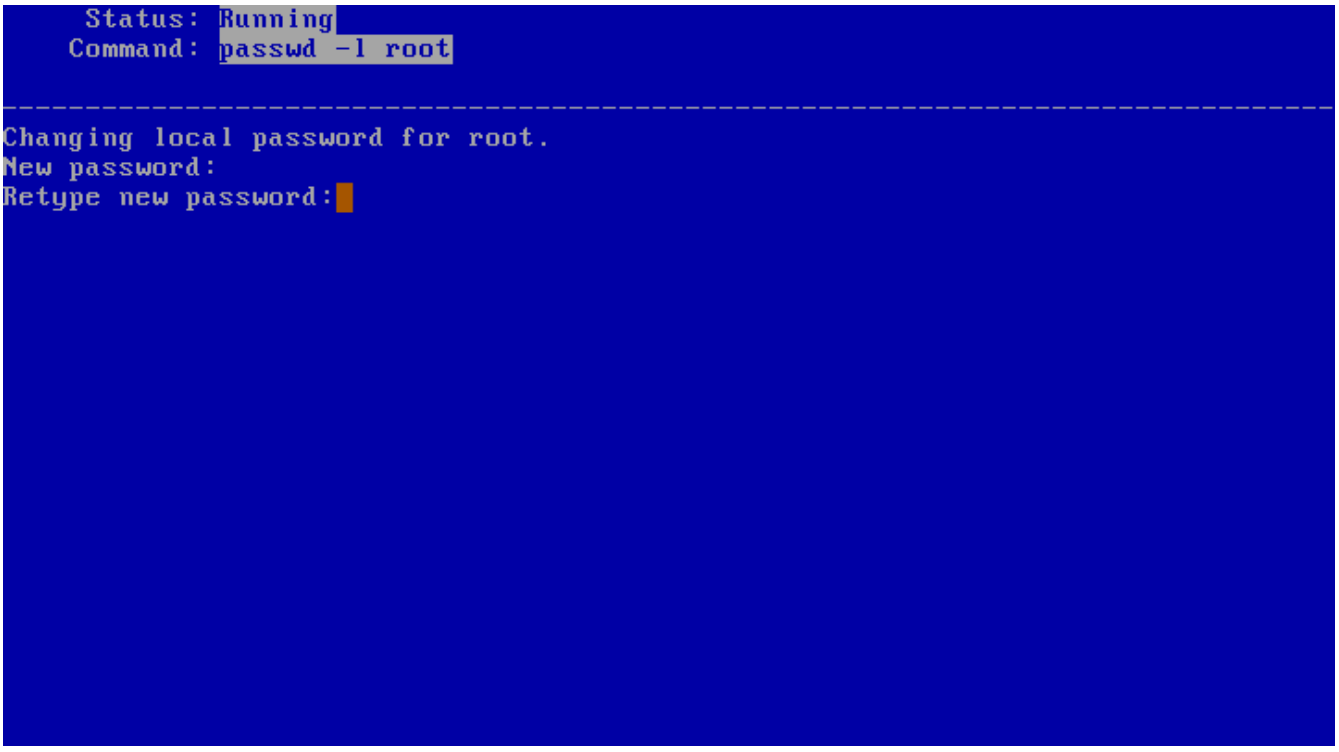


Рисунок 3.35. Задание пароля root

Чтобы упростить последующую установку двоичных пакетов, можно заранее настроить pkgin: выберите «„Enable installation of binary packages“» на рисунке 3.31, «Меню настройки». pkgin будет загружен и установлен с сервера FTP, поэтому убедитесь, что сеть уже настроена. Укажите имя «„Host“», его «„Base directory“» (где хранятся пакеты для всех платформ NetBSD) и «„Package directory“», соответствующий вашей платформе и версии NetBSD. Обычно значения по умолчанию верны.

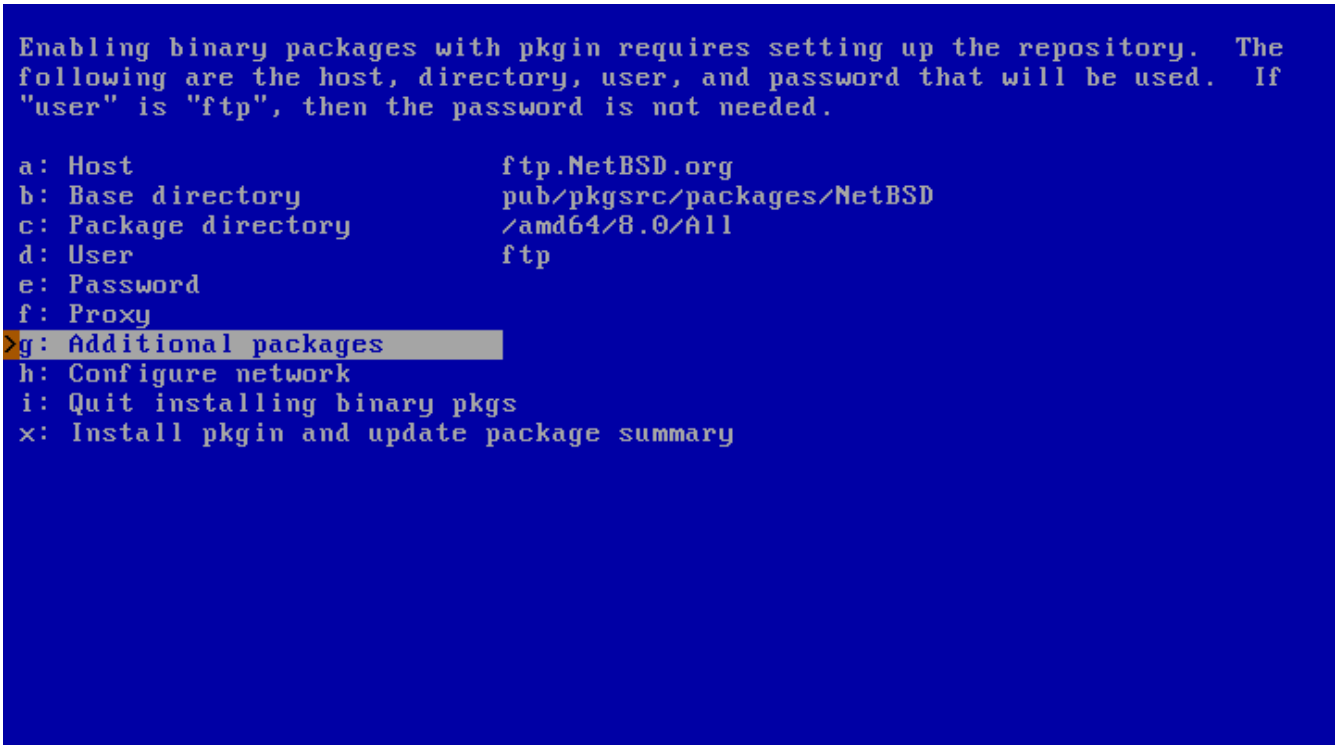


Рисунок 3.36. Включение установки двоичных пакетов

Если выбрать «„ftp“» в поле «„User“», пароль не потребуется. Как показано на рисунке 3.36, «Включение установки двоичных пакетов», можно также установить один или несколько дополнительных пакетов, введя их имена через пробел и нажав в конце «„Enter“». Чтобы перейти к установке, введите «„x“» и нажмите «„Enter“». Команда «„pkgin update“» будет выполнена после установки pkgin, чтобы репозиторий сразу стал актуальным.

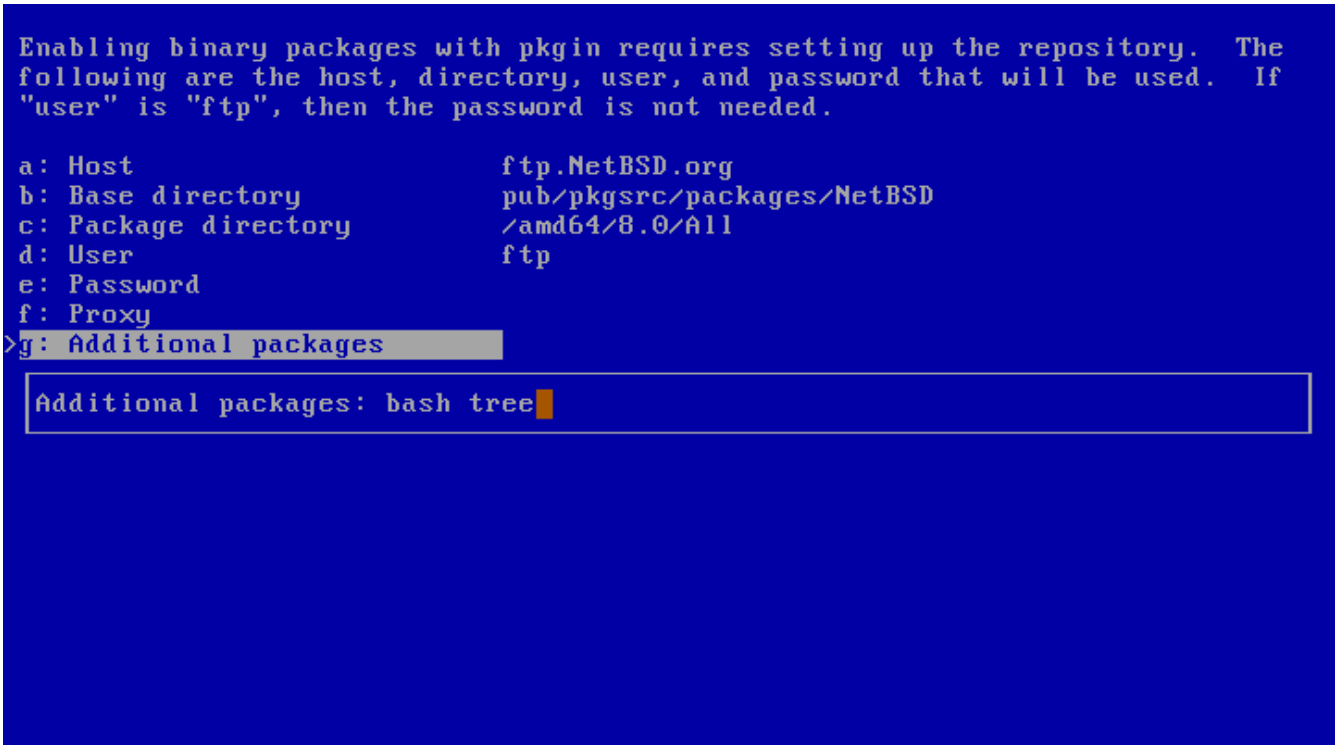


Рисунок 3.37. Дополнительные пакеты

После завершения процедуры sysinst покажет команду для установки дополнительных пакетов. Нажмите «„Enter“», чтобы вернуться в меню настройки.

Если вам нужно или вы хотите собирать пакеты из их исходного кода с помощью pkgsrc, выберите «Загрузить и распаковать pkgsrc для сборки из исходного кода» в меню конфигурации (рисунок 3.31). Как и раньше, укажите имя «Хост»; «Каталог pkgsrc» — это базовый каталог исходных текстов. Значения по умолчанию обычно являются наилучшими. Будет загружен один архивный файл, например `pkgsrc.tgz`: если вы хотите автоматически удалить его после установки pkgsrc, установите курсор на «Удалить после установки» и нажмите «Enter». Чтобы продолжить загрузку, введите «x» и затем нажмите «Enter».

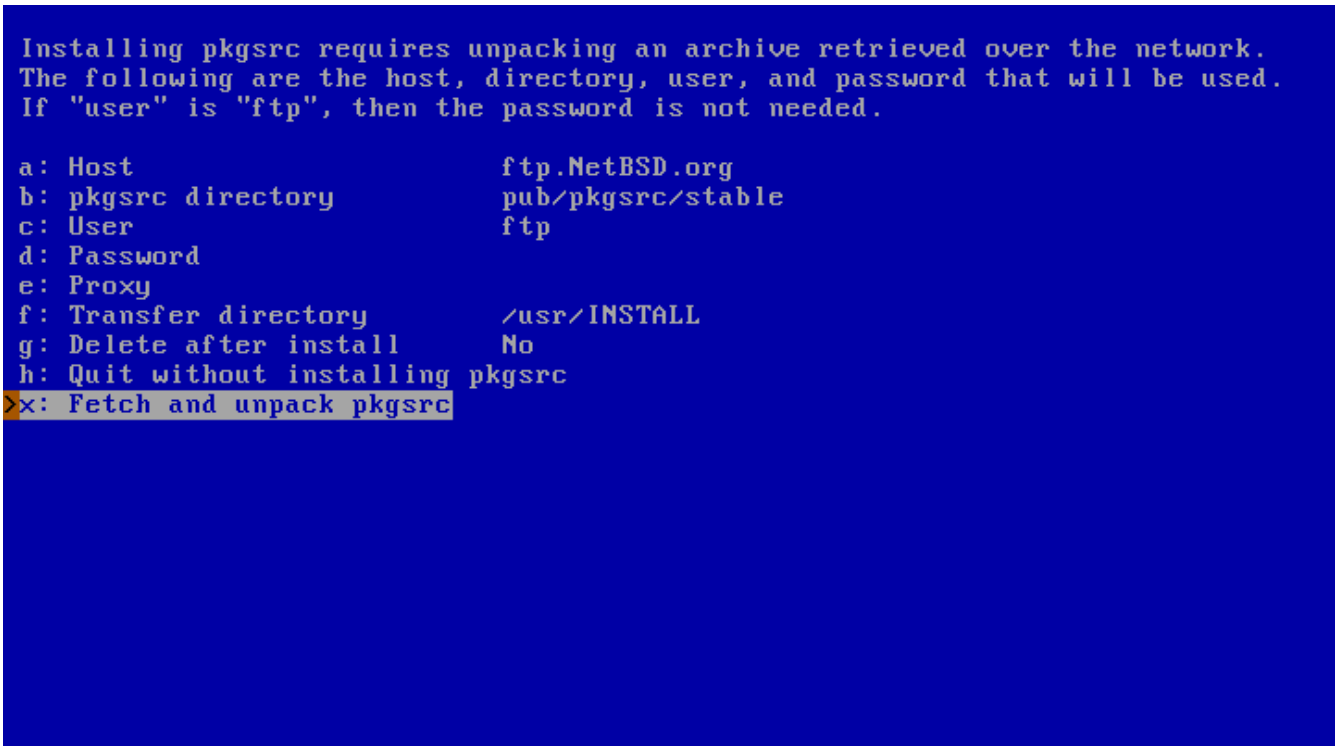


Рисунок 3.38. Загрузка и распаковка pkgsrc

В начальном меню настройки (рисунок 3.31, «Меню настройки») можно также включить некоторые полезные службы, например демон, ожидающий подключений ssh. Сведения о ntpd и ntpdate см. в разделе 29.2, «*Протокол сетевого времени* (NTP)». xdm обеспечивает аутентификацию и управление сеансами пользователей через дисплей X. Использование драйвера криптографических устройств (cgd) показано в главе 14, *Драйвер криптографических устройств (CGD)*. Логический менеджер томов (lvm) описан в главе 17, *Настройка NetBSD Logical Volume Manager (LVM)*, raidframe — в главе 16, *NetBSD RAIDframe*. mdnsd предоставляет службу Multicast DNS, а также DNS Service Discovery в NetBSD: подробности см. в mdnsd(8).

Наконец, меню на рисунке 3.31, «Меню настройки» позволяет добавить в систему обычного пользователя. Основные сведения о пользователях и учётной записи root, а также о группе wheel см. в разделе 5.6, «Добавление пользователей».

Когда вы завершите настройку всех нужных пунктов, выберите «„Finished configuring“» на рисунке 3.31, «Меню настройки».

3.14. Завершение установки

На этом установка завершена.

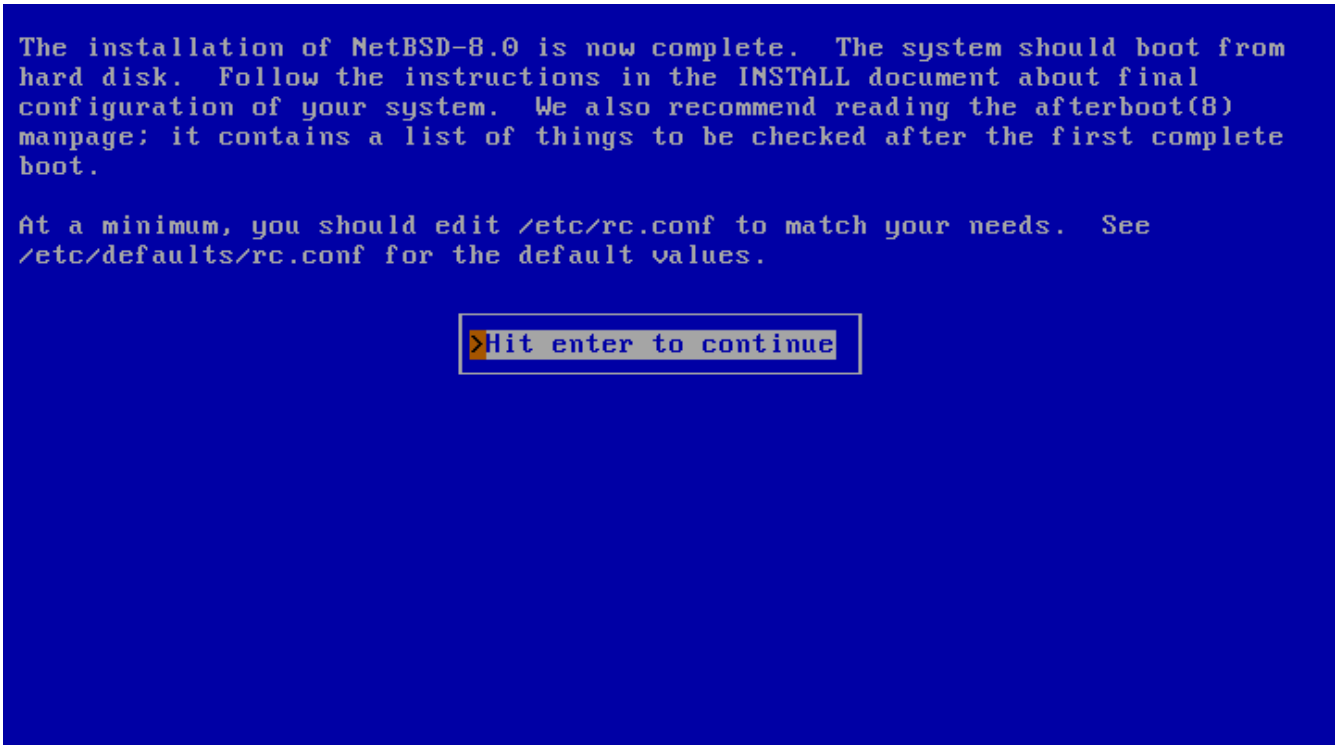


Рисунок 3.39. Установка завершена

После подтверждения установки в появившемся диалоговом окне sysinst вернётся в главное меню. Извлеките установочные носители (CD, дискеты и т. п.) и выберите «„Reboot the computer“», чтобы загрузить только что установленную систему NetBSD.

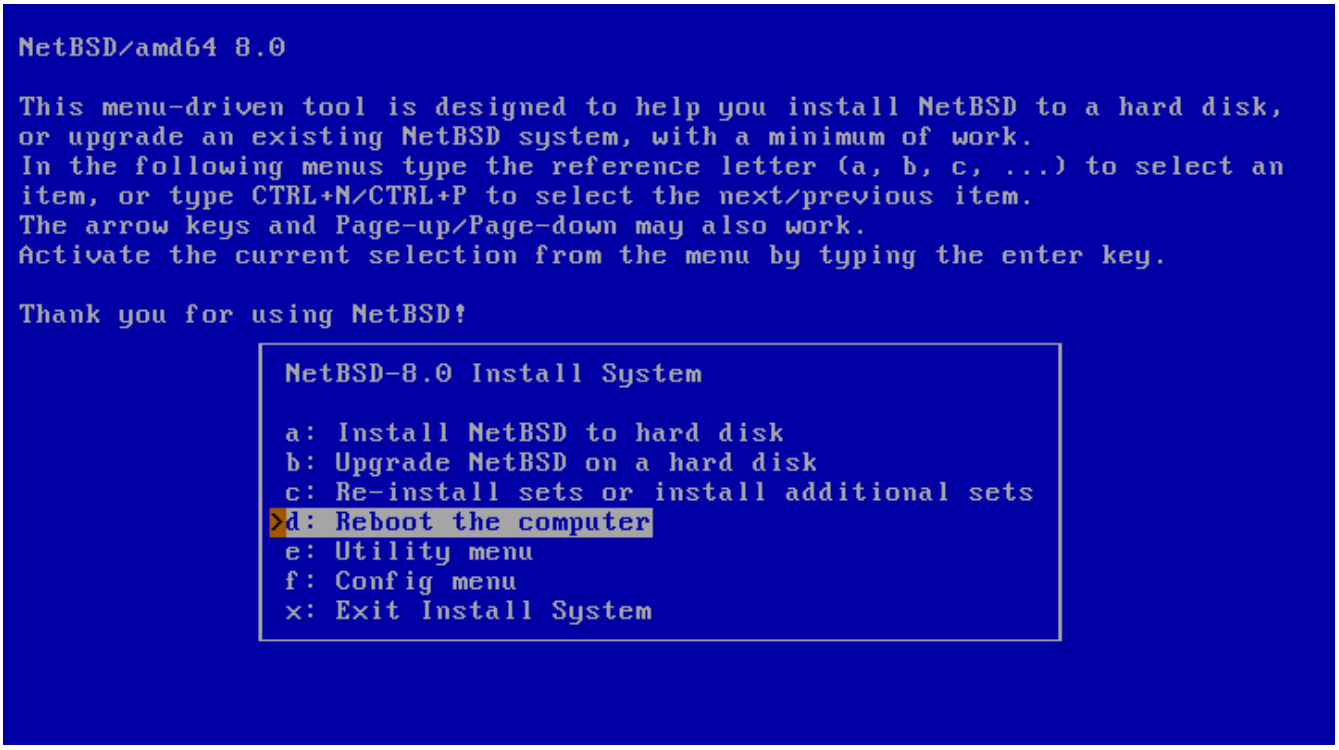


Рисунок 3.40. Перезагрузка для завершения установки

Глава 4. Обновление NetBSD

В этой главе описывается обновление системы NetBSD с использованием двоичных наборов. Существует несколько вариантов выполнения этой процедуры, и следующие разделы проведут вас по каждому из них:

4.1. Использование sysinst

4.1.1. Обзор

Для обновления необходимо иметь загрузочный носитель (CD-ROM, USB-накопитель, дискету и т. п.) и как минимум наборы дистрибутива base и kern. Поскольку уже установленные в системе файлы перезаписываются на своих местах, дополнительное свободное место нужно только для файлов, которые ранее не были установлены, или на случай увеличения размера наборов от выпуска к выпуску. Обычно это не более нескольких мегабайт.

> **ПРИМЕЧАНИЕ**
>
> Поскольку обновление включает замену ядра, загрузочных блоков и большей части системных двоичных файлов, оно может привести к потере данных. Перед началом настоятельно рекомендуется создать резервную копию всех важных данных на разделе NetBSD и на любых других разделах диска.

Процедура обновления похожа на установку, но без разметки жёсткого диска. sysinst попытается объединить настройки, хранящиеся в каталоге `/etc`, с новой версией NetBSD. Кроме того, перед распаковкой наборов проверяются файловые системы. Загрузка двоичных наборов выполняется так же, как при установке.

4.1.2. Документ INSTALL

Перед обновлением необходимо прочитать сведения о выпуске и примечания по обновлению в одном из файлов `INSTALL`: это официальное описание процедуры обновления со сведениями, специфичными для конкретной платформы, и важными подробностями. Его можно найти в корневом каталоге выпуска NetBSD (на установочном CD или на FTP-сервере).

Документ INSTALL рекомендуется распечатать. Он доступен в четырёх форматах: .txt, .ps, .more и .html.

4.1.3. Выполнение обновления

В следующем разделе приведён обзор процесса двоичного обновления. Большинство описанных далее диалоговых окон sysinst аналогичны тем, что появляются при установке. Более подробные описания и пояснения к диалоговым окнам приведены в главе 3, *Пример установки*.

После выбора языка установки и типа клавиатуры появляется главное меню. Выбор пункта «„b: Upgrade NetBSD on a hard disk“» запустит процесс обновления.

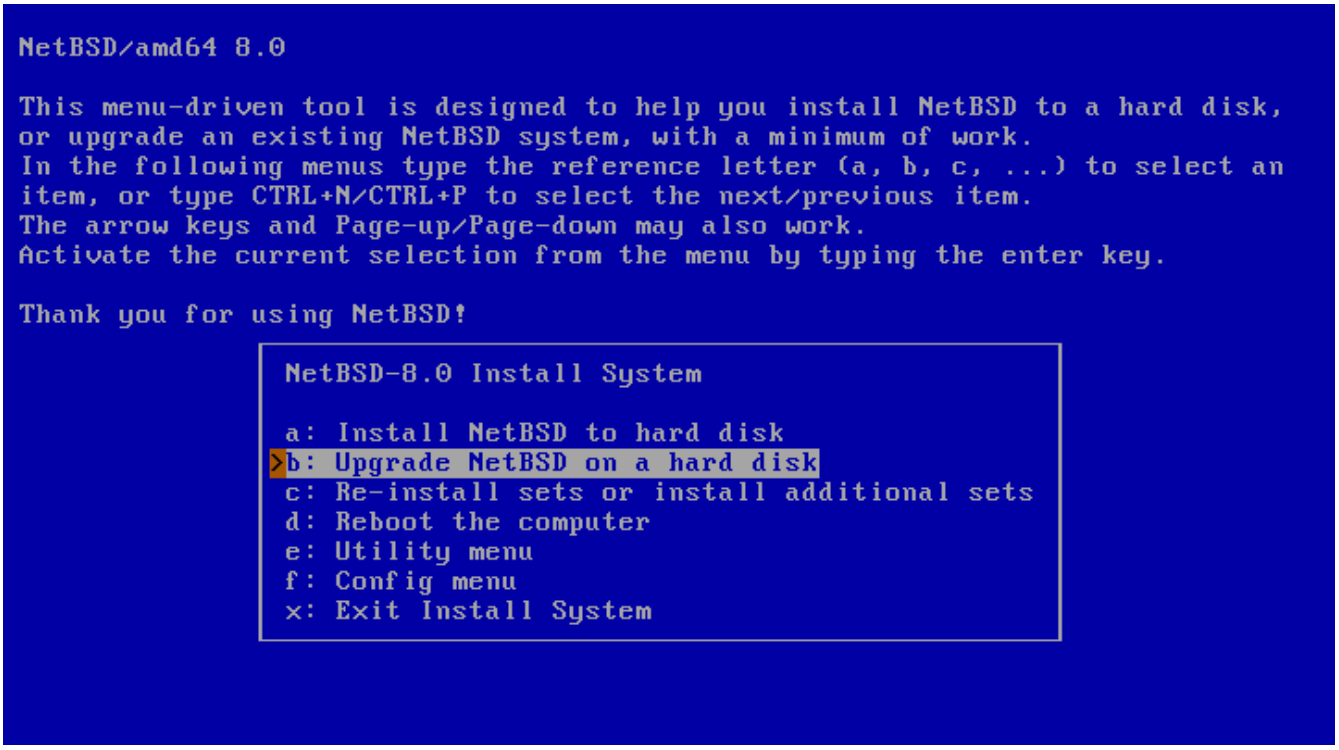


Рисунок 4.1. Начало обновления

Диалоговое окно на рисунке 4.2, «Продолжение обновления» запросит разрешение на продолжение обновления. На этом этапе ещё ничего не изменено, и обновление по-прежнему можно отменить. Это подходящий момент, чтобы спросить себя, сделали ли вы резервную копию и уверены ли вы в том, что сможете из неё восстановиться.



Рисунок 4.2. Продолжение обновления

После выбора «„Yes“» для продолжения следующее диалоговое окно попросит указать жёсткий диск с системой NetBSD, которую нужно обновить.

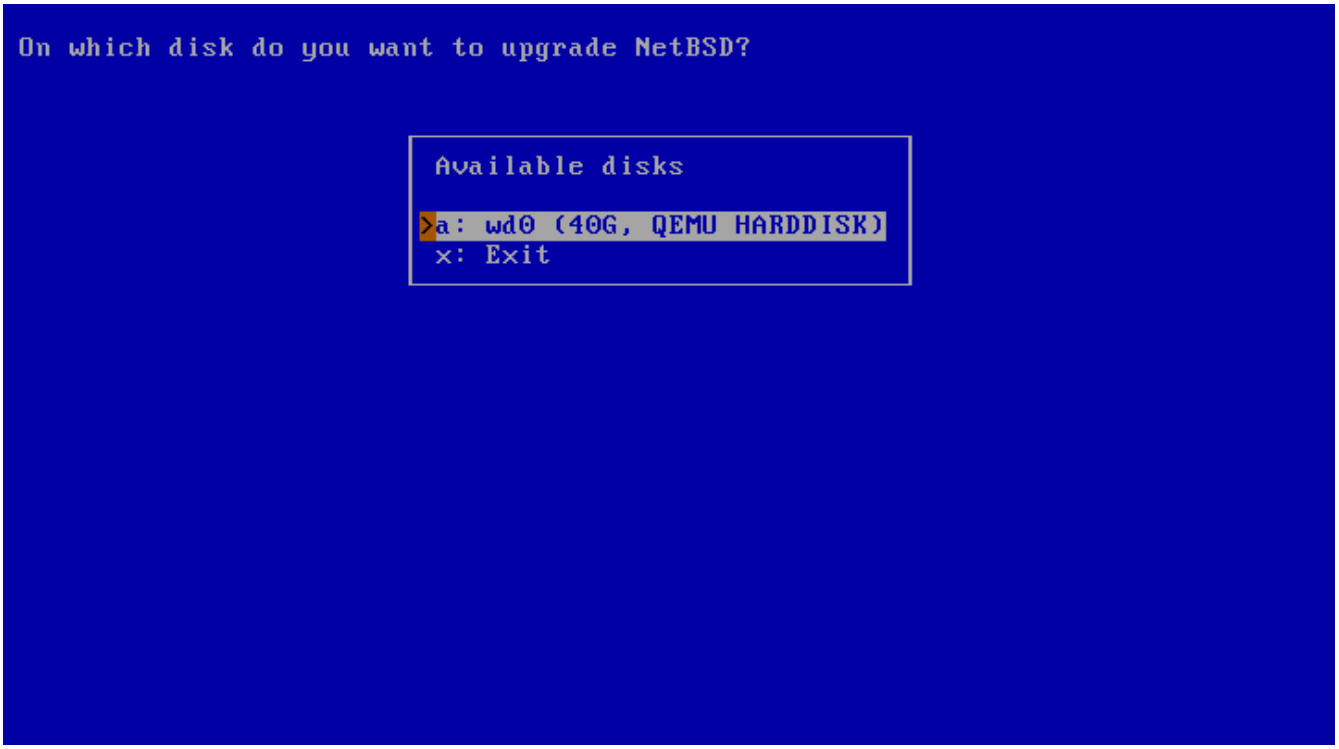


Рисунок 4.3. Выбор жёсткого диска

В системе, использованной в примере, доступен только один жёсткий диск: «„wd0“».
На этом этапе sysinst выполнит проверку файловой системы, чтобы убедиться в её целостности.

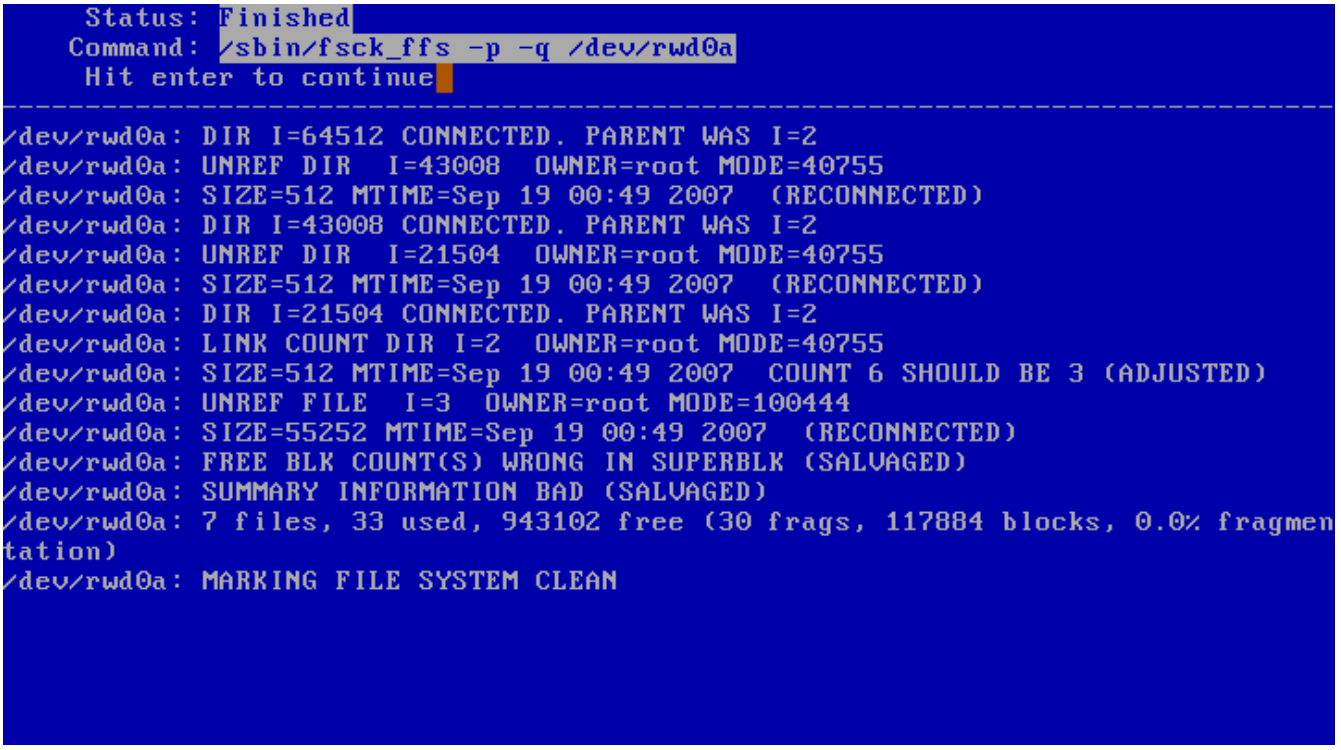


Рисунок 4.4. Проверка файловой системы

Следующий шаг — выбрать тип загрузочных блоков, которые нужно установить.

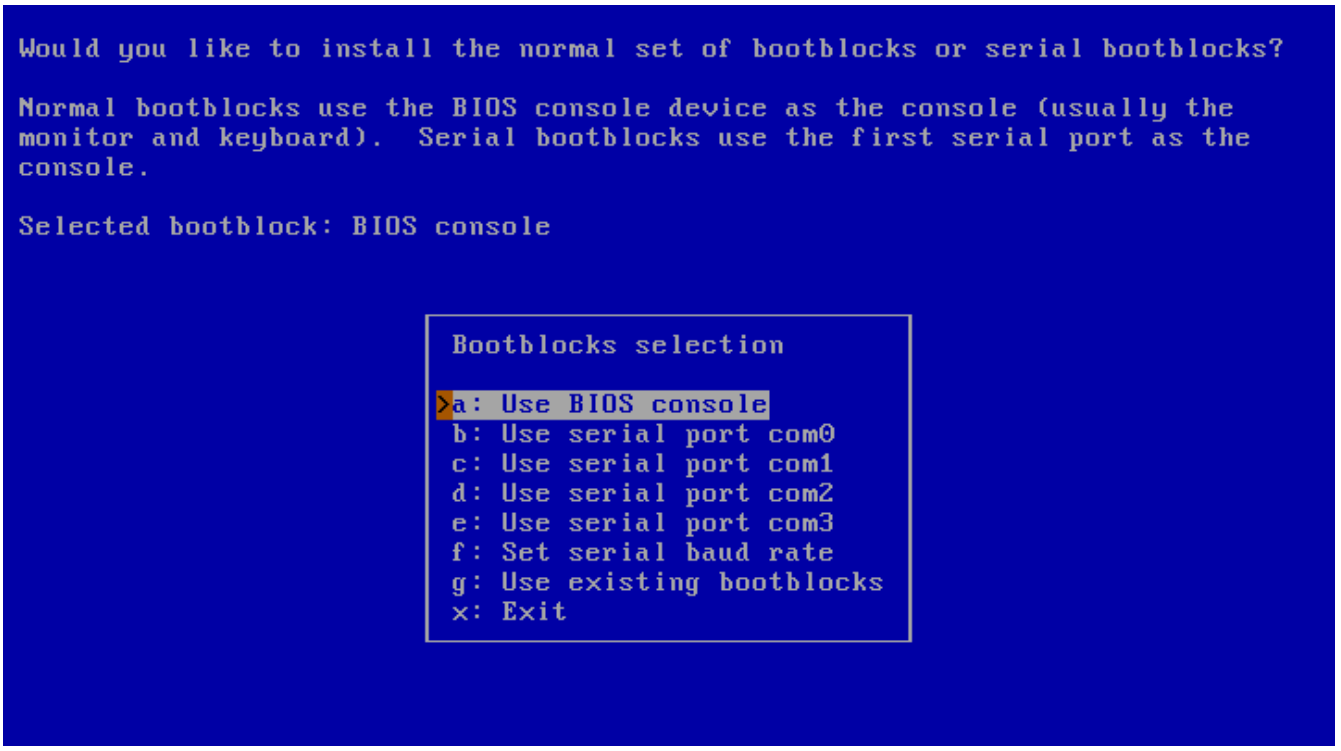


Рисунок 4.5. Выбор загрузочных блоков

В следующем диалоговом окне представлено меню выбора типа установки: «„Full installation“», «„Installation without X11“», «„Minimal installation“» или «„Custom installation“».

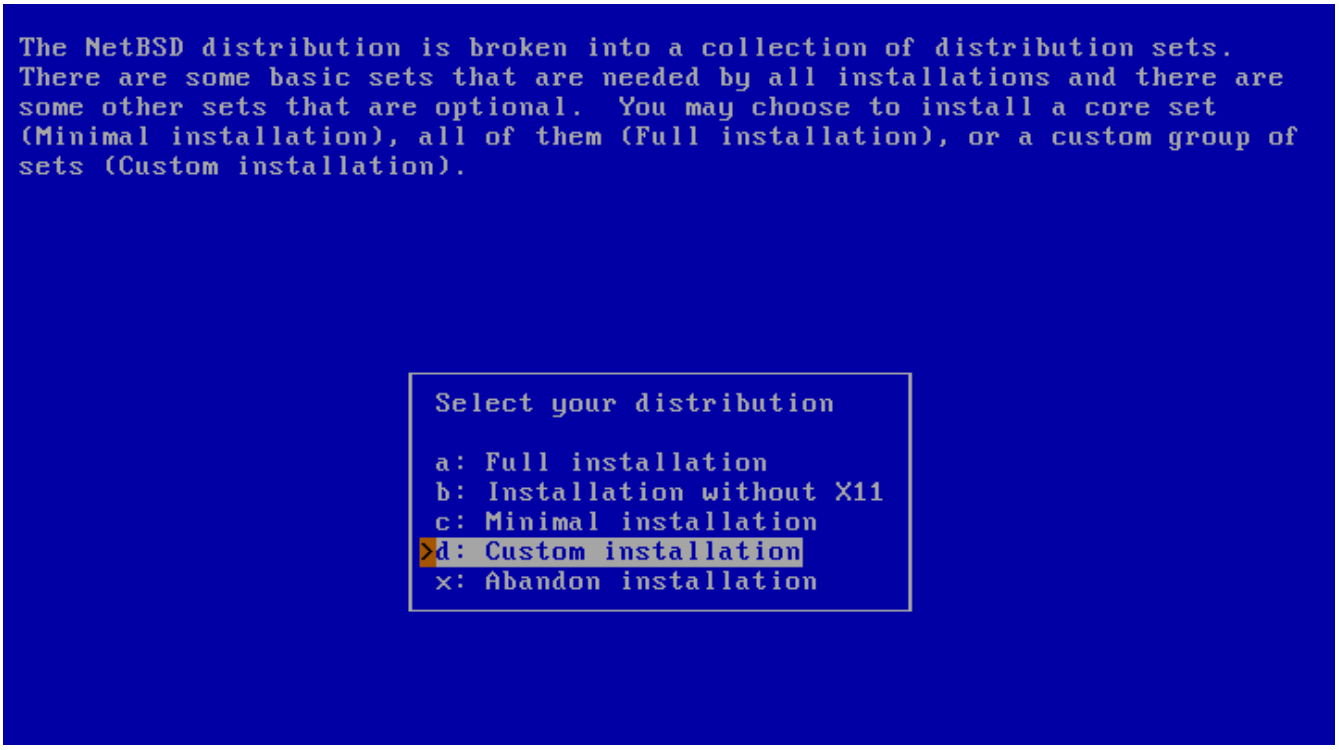


Рисунок 4.6. Выбор наборов дистрибутива

В следующем диалоговом окне нужно выбрать способ установки и приведён список возможных вариантов. Установочный носитель содержит новые наборы дистрибутива NetBSD. В зависимости от выбранного варианта у вас будут запрашивать разные сведения. Например, при установке с CD-ROM или DVD необходимо указать используемое устройство и каталог, в котором находятся наборы, а при установке по FTP — настроить сеть и указать имя хоста FTP-сервера. Подробности см. в разделе 3.11, «Выбор установочного носителя».

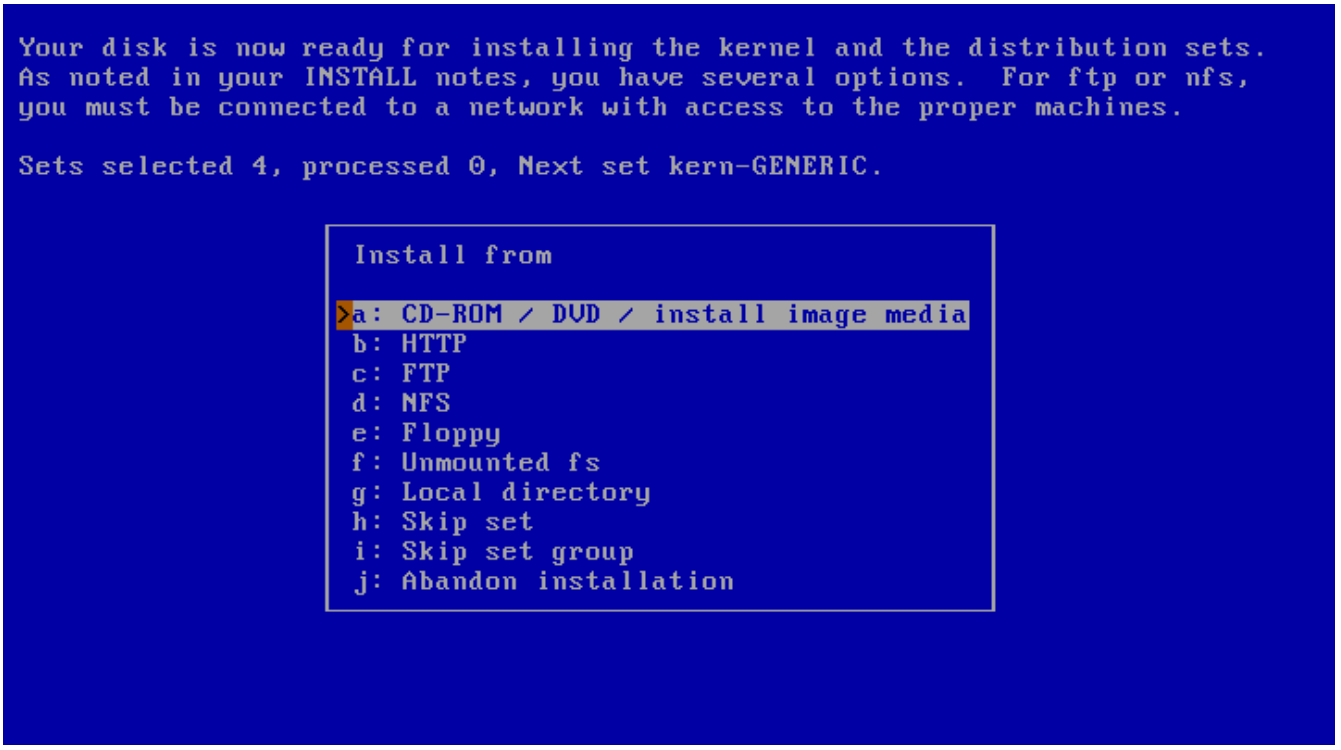


Рисунок 4.7. Установочный носитель

sysinst распакует наборы дистрибутива, заменив ваши старые двоичные файлы. После распаковки этих наборов запускается сценарий postinstall(8), выполняющий различные задачи по очистке системы и обновлению конфигурации. Если postinstall выдаст ошибки, вам придётся самостоятельно решить указанные в них проблемы. Дополнительные сведения см. на странице руководства postinstall. Даже после успешного запуска postinstall целесообразно использовать etcupdate(8), чтобы упростить объединение прочих изменений конфигурации. Следует также прочитать замечания в `INSTALL` об обновлении: там описаны специфические проблемы совместимости.

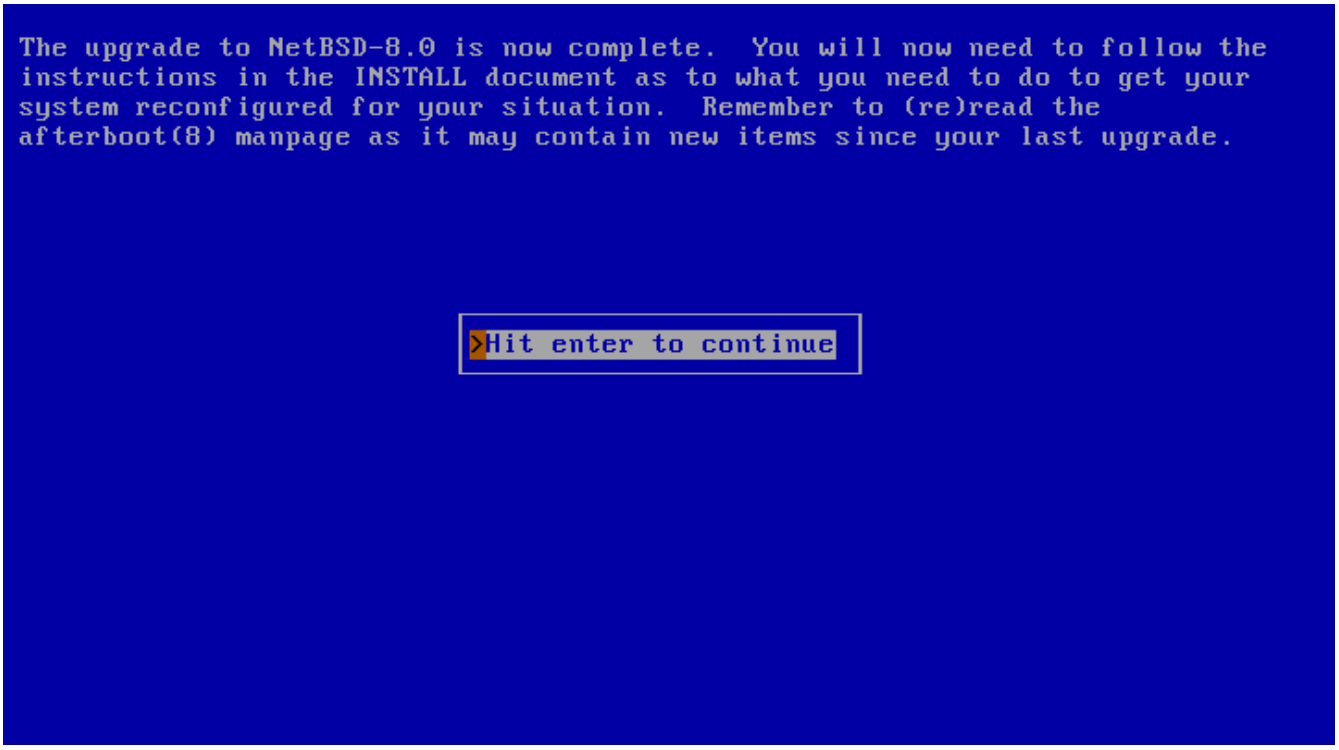


Рисунок 4.8. Обновление завершено

Вернувшись в главное меню, извлеките загрузочный носитель (если это применимо) и перезагрузитесь. Приятной работы с новой версией NetBSD!

4.2. Использование sysupgrade

Утилита sysupgrade (сейчас она находится в `pkgsrc/sysutils/sysupgrade`) позволяет обновить работающую систему до более нового двоичного выпуска.

> **ПРИМЕЧАНИЕ**
>
> Соблюдайте осторожность при обновлении между основными выпусками — следите, чтобы работающее ядро никогда не было старше пользовательского окружения.

Одно из преимуществ sysupgrade — это целостное решение, почти не требующее участия человека: при желании утилита загружает новое ядро и наборы дистрибутива с удалённых сайтов и выполняет обновление без вмешательства пользователя, пока не потребуется объединить новые изменения в файлах конфигурации.

Предположим, вы используете NetBSD/amd64 9.1 и хотите обновиться до NetBSD 9.2. Для этого нужно выполнить следующую команду:

```
# sysupgrade auto https://cdn.NetBSD.org/pub/NetBSD/NetBSD-9.2/amd64
```

Вот и всё, что требуется. Утилита загрузит ядро и наборы, подходящие для вашей машины, распакует их и поможет объединить новые изменения конфигурации. Не забудьте после этого перезагрузиться.

При обновлении между основными выпусками (например, с NetBSD 8.2 на 9.2) сначала обновите ядро и модули:

```
# sysupgrade fetch https://cdn.NetBSD.org/pub/NetBSD/NetBSD-9.2/amd64
# sysupgrade kernel
# sysupgrade modules
# reboot
# sysupgrade sets
# sysupgrade etcupdate
# sysupgrade postinstall
# sysupgrade clean
# reboot
```

Подробности см. на прилагаемой странице руководства sysupgrade(8) и в файле конфигурации `/usr/pkg/etc/sysupgrade.conf`.

Часть III. Конфигурация системы, администрирование и тонкая настройка

Глава 5. Первые шаги в NetBSD

После установки и перезагрузки компьютер загрузится с жёсткого диска. Если всё прошло хорошо, через несколько секунд (или минут — в зависимости от оборудования) вы увидите приглашение входа в систему. Система ещё не настроена полностью, но базовая настройка не сложна. Вы увидите, как быстро настроить некоторые важные вещи, и попутно познакомитесь с основами работы системы.

5.1. Устранение неполадок

5.1.1. Проблемы с загрузкой

Если система не загружается, возможно, загрузчик был установлен неправильно или есть проблема с *MBR* (*главной загрузочной записью*). Загрузите машину с установочного носителя (CD, DVD, дискеты и т. п.), а когда появится меню загрузки, выберите пункт перехода в приглашение загрузчика.

```
type "?" or "help" for help.
> ?
commands are:
boot [xdNx:][filename] [-12acdqsvxz]
     (ex. "hd0a:netbsd.old -s")
ls [path]
dev xd[N[x]]:
consdev {pc|com[0123]|com[0123]kbd|auto}
modules {enabled|disabled}
load {path_to_module}
multiboot [xdNx:][filename] [<args>]
help|?
quit
> boot hd0a:netbsd
```

Теперь система должна загрузиться с жёсткого диска. Если NetBSD загружается с жёсткого диска правильно, то, вероятно, проблема связана с главной загрузочной записью (MBR). Установить загрузчик или изменить его конфигурацию можно командой **fdisk -B**. Подробное описание см. в разделе 22.1, «Установка загрузчика».

5.1.2. Ошибки в конфигурации /etc/rc.conf

Если вы или программа установки совсем не настраивали `/etc/rc.conf` (обычно это делает sysinst), система переведёт вас в *режим однопользовательской работы* и выведет сообщение

```
/etc/rc.conf is not configured. Multiuser boot aborted
```

Когда система предложит выбрать оболочку, просто нажмите RETURN, чтобы попасть в приглашение /bin/sh. Если у вас спросят тип терминала, ответьте `vt220` (или тем, что подходит для вашего терминала) и нажмите RETURN. В зависимости от клавиатуры может потребоваться ввести одну из следующих команд, чтобы клавиша удаления работала правильно:

```
# stty erase '^h'
# stty erase '^?'
```

На этом этапе нужно настроить хотя бы один файл в каталоге `/etc`. Однако корневая файловая система (`/`) смонтирована только для чтения, поэтому сначала нужно сделать её доступной для записи:

```
# /sbin/mount -u -w /
```

Затем взгляните на файл `/etc/rc.conf`. Измените его по своему усмотрению, но обязательно установите «„`rc_configured=YES `“», чтобы больше не оказаться в такой ситуации. Значения по умолчанию для различных программ можно найти в `/etc/defaults/rc.conf`. Более полную документацию можно найти в rc.conf(5).

Закончив, введите в приглашении exit, чтобы выйти из оболочки однопользовательского режима и продолжить многопользовательскую загрузку.

5.2. Команда man

Если вы никогда раньше не работали с Unix-подобной операционной системой, вашим лучшим другом теперь станет команда **man**, которая выводит страницу руководства. Страницы руководства NetBSD — одни из самых лучших и подробных, хотя и очень технические.

Хорошая страница руководства, которую стоит прочитать после первой загрузки новой системы NetBSD, — afterboot(8). Она содержит сведения о различных необходимых и полезных параметрах конфигурации.

man *`name`* выводит страницу руководства для команды «„*`name`*“», а **man -k** *`name`* — список страниц руководства, посвящённых «„*`name`*“» (можно также использовать команду **apropos**).

Чтобы изучить основы работы с командой **man**, введите:

```
# man man
```

Страницы руководства содержат не только сведения о командах, но и описания некоторых возможностей и структур NetBSD. Например, взгляните на страницу руководства hier(7), где подробно описана структура файловой системы, используемая в NetBSD.

```
# man hier
```

Другие подобные страницы — release(7) и pkgsrc(7).

```
# man 8 intro
```

Страницы руководства разделены на несколько разделов в зависимости от того, что они описывают:

1. общие команды (инструменты и утилиты), см. intro(1)
2. системные вызовы и номера ошибок, см. intro(2)
3. библиотеки C, см. intro(3)
4. специальные файлы и поддержка оборудования, см. intro(4)
5. форматы файлов, см. intro(5)

6. игры, см. intro(6)
7. разные информационные страницы, см. intro(7)
8. команды обслуживания и работы системы, см. intro(8)
9. внутреннее устройство ядра, см. intro(9)

Одна и та же тема может встречаться более чем в одном разделе руководства; чтобы просмотреть конкретную страницу, укажите номер раздела в качестве аргумента команды man. Например, *time* встречается в разделе 1 (пользовательская команда time) и в разделе 3 (функция time библиотеки C). Чтобы посмотреть страницу руководства для функции time из библиотеки C, введите:

```
# man 3 time
```

Чтобы просмотреть все доступные страницы:

```
# man -w time
# man -a time
```

5.3. Редактирование файлов конфигурации

Помимо оболочки, текстовый редактор — важнейший инструмент администрирования системы NetBSD.

В базовой системе есть два таких редактора

- ed(1) — построчный текстовый редактор. ed — очень простой текстовый редактор. У него есть командный режим (активный при первом запуске) и режим ввода. Его главное преимущество в том, что он работает даже при неправильно заданном типе терминала. В аварийной ситуации ed стоит знать, но учтите, что vi(1) доступен в `/rescue`, что подводит нас к...
- vi(1) — экранный текстовый редактор. vi — единственный экранный редактор, доступный в базовой установке, и для его работы требуется корректный тип терминала. Дополнительные сведения о редакторе по умолчанию в NetBSD см. в главе 6, *Редактирование*.

> **СОВЕТ**
>
> Прежде чем продолжать, вы должны знать или узнать, как открывать, редактировать и сохранять файлы в vi. Обязательно прочитайте главу 6, *Редактирование*.

5.4. Вход в систему

При первом входе вы используете пользователя `root` — единственную учетную запись, создаваемую по завершении установки. В ответ на приглашение к вводу пароля введите пароль root, заданный при установке. Если вы не задавали пароль, просто нажмите Enter.

```
NetBSD/i386 (Amnesiac) (ttyE0)
login: root
password:
We recommend creating a non-root account and using su(1) for
root access.
#
```

5.5. Смена пароля `root`

Если при установке вы не задали пароль для `root`, сделайте это сейчас с помощью команды **/usr/bin/passwd**.

```
# /usr/bin/passwd
Changing local password for root.
New password:
Retype new password:
```

При вводе пароли не отображаются на экране.

Выбирайте пароль, содержащий цифры и специальные символы (кроме пробела), а также буквы в верхнем и нижнем регистре. Не выбирайте слово из какого-либо языка. Злоумышленники часто используют атаку по словарю.

5.6. Добавление пользователей

Из соображений безопасности при обычной работе и обслуживании системы не следует входить как root. Вместо этого администраторам рекомендуется добавить обычного пользователя, включить его в группу `wheel`, а затем использовать команду su(1), когда требуются права root. NetBSD предлагает утилиту useradd(8) для создания учетных записей пользователей. Например, чтобы создать нового пользователя:

```
# useradd -m joe
```

Значения по умолчанию для команды useradd можно изменить; см. страницу руководства useradd(8).

Учетные записи, которые могут переключаться на root с помощью su, должны входить в группу «wheel». Это можно сделать при создании учетной записи, указав дополнительную группу:

```
# useradd -m -G wheel joe
```

В качестве альтернативы для добавления пользователя в существующую группу можно использовать команду usermod(8):

```
# usermod -G wheel joe
```

Если вы только что создали пользователя, но забыли задать пароль, это можно сделать позже с помощью команды passwd(1).

```
# passwd joe
```

> **ПРИМЕЧАНИЕ**
>
> Вы можете редактировать `/etc/group` напрямую, чтобы добавлять пользователей в группы, но *не* редактируйте `/etc/passwd` напрямую; используйте vipw(8).

5.7. Теневые пароли

Теневые пароли включены по умолчанию. Это означает, что все пароли в `/etc/passwd` заменены на «*»; зашифрованные пароли хранятся в файле `/etc/master.passwd`, который может читать только root. Когда вы запускаете vipw(8) для редактирования файла паролей, программа открывает копию `/etc/master.passwd`; при выходе vipw проверяет копию на корректность, создает новый `/etc/passwd` и устанавливает новый файл `/etc/master.passwd`. Наконец, vipw запускает pwd_mkdb(8), который создает файлы `/etc/pwd.db` и `/etc/spwd.db` — две базы данных, эквивалентные `/etc/passwd` и `/etc/master.passwd`, но обрабатываемые быстрее.

Очень важно *всегда* использовать **vipw** и другие средства администрирования учетных записей (chfn(1), chsh(1), chpass(1), passwd(1)) и *никогда* не изменять напрямую `/etc/master.passwd` или `/etc/passwd`.

5.8. Смена раскладки клавиатуры

Если у вас не US-раскладка клавиатуры, вероятно, вы захотите изменить раскладки. Например, чтобы использовать итальянскую клавиатуру, введите следующую команду:

```
# wsconsctl -k -w encoding=it
encoding -> it
```

Чтобы сохранить раскладку клавиатуры постоянно, добавьте следующую строку в файл `/etc/wscons.conf`:

```
encoding it
```

Список доступных раскладок приведен в разделе 8.1.2.1, «Раскладки клавиатуры».

5.9. Системное время

NetBSD, как и все системы Unix, использует системные часы, отсчитываемые по UTC (всемирное координированное время), и именно на UTC следует устанавливать системные часы. Если вы хотите, чтобы системные часы шли по местному времени (например, потому что у вас система с двойной загрузкой, где установлена Windows), необходимо сообщить об этом NetBSD, добавив `rtclocaltime=YES` в `/etc/rc.conf`:

```
# echo rtclocaltime=YES >> /etc/rc.conf
# service rtclocaltime restart
```

> **ПРИМЕЧАНИЕ**
>
> В качестве альтернативы можно настроить Windows 7 и более поздние версии на использование RTC в UTC. Как упомянуто в этой статье базы знаний Microsoft, для этого нужно добавить ключ реестра типа DWORD с именем RealTimeIsUniversal и значением 1 в HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\TimeZoneInformation.

Число минут к западу от GMT вычисляется автоматически и задается в переменной sysctl `kern.rtc_offset`.

Чтобы отобразить текущее значение переменной `kern.rtc_offset`:

```
# sysctl kern.rtc_offset
kern.rtc_offset = -60
```

Эта автоматическая настройка работает только в том случае, если вы задали правильный часовой пояс с помощью символической ссылки на `/etc/localtime`. Обычно это делается в ходе установки, но если по какой-то причине этого не произошло, можно создать символическую ссылку с файла в каталоге `/usr/share/zoneinfo` на `/etc/localtime`.

В следующем примере часовой пояс задается как Eastern Europe Summer Time:

```
# ln -fs /usr/share/zoneinfo/Europe/Helsinki /etc/localtime
```

5.10. Защищённая оболочка (Secure Shell, ssh(1))

По умолчанию в только что установленной системе NetBSD все службы отключены, и ssh(1) не исключение. Возможно, вы захотите включить его, чтобы иметь возможность входить в систему удаленно. Задайте `sshd=YES` в `/etc/rc.conf` и затем запустите сервер командой

```
# service sshd start
```

При первом запуске сервер сгенерирует новую пару ключей, которая будет сохранена в каталоге `/etc/ssh`.

5.11. Базовая настройка в `/etc/rc.conf`

NetBSD использует `/etc/rc.conf`, чтобы определить, что будет выполняться при загрузке системы. Понимать этот файл важно. Страница руководства rc.conf(5) содержит подробное описание всех доступных параметров.

Файл `/etc/defaults/rc.conf` содержит значения по умолчанию для большинства параметров. Чтобы переопределить значение по умолчанию, новое значение нужно поместить в `/etc/rc.conf`. Определения в нем перекрывают определения в `/etc/defaults/rc.conf` (который следует оставлять без изменений).

```
# man rc.conf
```

Первые изменения:

- Задайте «`rc_configured=YES`» (это изменение уже должно быть выполнено программой установки.)
- Задайте «`dhcpcd=YES`», чтобы настроить сеть вашей системы с помощью DHCP.
- Определите *имя хоста* для своей машины (используйте полное доменное имя, то есть имя, включающее домен). Если у вас автономная машина, можно использовать любое имя (например, `vigor3.your.domain`). Если ваша машина подключена к сети, следует указать правильное имя.
- Если ваш компьютер подключён к локальной сети или к Интернету через маршрутизатор, задайте переменной *defaultroute* IP-адрес маршрутизатора (иногда его называют *шлюзом по умолчанию*). Например, «`defaultroute=192.168.1.1`».

5.12. Основные настройки сети

Чтобы разрешать имена и IP-адреса удалённых хостов, системе нужен доступ к *серверу имён DNS* (удалённому или локальному). Укажите системе, какие серверы имён использовать, добавив IP-адрес одного или нескольких серверов в файл `/etc/resolv.conf`, как в следующем примере:

```
nameserver 145.253.2.75
```

Чтобы задать имена локальных хостов, недоступные через DNS, отредактируйте файл `/etc/hosts`, который имеет следующий вид:

```
IP-address  hostname  host
```

Например:

```
192.168.1.3 vigor3.your.domain vigor3
```

5.13. Монтирование CD-ROM

Новичков часто удивляет, что программа установки прекрасно распознала и смонтировала их CD-ROM, а установленная система, как будто «забыла», как его использовать. В работе с CD-ROM нет никакой особой магии: его можно смонтировать, как любую другую файловую систему. Всё, что нужно знать, — это имя устройства и некоторые параметры команды mount(8). Имя устройства можно узнать с помощью уже упоминавшейся команды dmesg(8). Например, если dmesg выводит:

```
# dmesg | grep ^cd
cd0 at atapibus0 drive 1: <ASUS CD-S400/A, , V2.1H> type 5 cdrom removable
```

имя устройства — `cd0`, и смонтировать CD-ROM можно следующими командами:

```
# mkdir /cdrom
# mount -t cd9660 -o ro /dev/cd0a /cdrom
```

Чтобы упростить задачу, можно добавить строку в файл `/etc/fstab`:

```
/dev/cd0a /cdrom cd9660 ro,noauto 0 0
```

Теперь, не перезагружаясь, можно смонтировать CD-ROM командой:

```
# mount /cdrom
```

Пока CD-ROM смонтирован, извлечь его вручную нельзя; сначала придётся его размонтировать:

```
# umount /cdrom
```

Есть также программная команда, которая размонтирует CD-ROM и извлекает его:

```
# eject /dev/cd0a
```

5.14. Монтирование дискеты

Чтобы смонтировать дискету, нужно знать имя устройства дисковода и тип файловой системы на дискете. Дополнительные сведения об именовании устройств приведены на странице руководства fdc(4): имена различаются в зависимости от точного размера и типа дискеты. Например, для чтения и записи дискеты в формате MS-DOS используется следующая команда:

```
# mount -t msdos /dev/fd0a /mnt
```

Вместо `/mnt` можно использовать любой другой каталог; например, можно создать каталог `/floppy` так же, как вы сделали для CD-ROM. Если вы много работаете с дискетами MS-DOS, установите пакет mtools, который позволяет обращаться к дискете MS-DOS (или разделу жёсткого диска) без необходимости монтировать её. Он очень удобен для быстрого копирования файла на дискету и с неё:

```
# mcopy foo bar a:
# mcopy a:baz.txt baz
# mcopy a:\*.jpg .
```

5.15. Установка дополнительного программного обеспечения

Использование пакетов из pkgsrc

Если вы хотите установить какое-либо программное обеспечение, свободно доступное для UNIX-подобных систем, настоятельно рекомендуем сначала проверить систему пакетов NetBSD — pkgsrc. pkgsrc автоматически вносит все изменения, необходимые для работы программы в NetBSD, включая загрузку и установку других пакетов, от которых она может зависеть.

- См. список доступных пакетов
- Для большинства платформ на FTP-сервере NetBSD доступны готовые двоичные файлы. Чтобы установить их, нужно задать переменную `PKG_PATH` следующим образом (в оболочке sh(1)):

  ```
  # PKG_PATH="https://cdn.NetBSD.org/pub/pkgsrc/packages/NetBSD/$(uname -p)/$(uname -r | cut
  -d_ -f1)/All"
  # export PKG_PATH
  ```

  Теперь суперпользователь `root` может устанавливать приложения командой pkg_add:

  ```
  # pkg_add -v perl
  # pkg_add -v apache
  # pkg_add -v firefox
  ```

  Приведённые выше команды установят язык программирования Perl, веб-сервер Apache и веб-браузер Firefox, а также все пакеты, от которых они зависят.

  Для большинства нетривиальных задач управления двоичными пакетами и обновлениями рекомендуется установить и использовать pkgin. pkgin можно установить из меню настройки после установки в **sysinst** или позже, в работающей системе, с помощью **pkg_add**:

  ```
  # pkg_add -v pkgin
  ```

Он ведёт локальную базу данных пакетов, имеющихся на удалённом сервере; загрузить базу можно командой:

```
# pkgin update
```

Принципы его работы близки к инструментам управления пакетами других операционных систем. Чтобы найти в базе пакетов слово `stat`, выполните

```
# pkgin search WORD
```

Чтобы установить пакет (в данном случае `fscd`), просто введите

```
# pkgin install fluxbox
```

Чтобы обновить установленные пакеты:

```
# pkgin upgrade
```

Прочитайте страницу руководства, чтобы узнать о других действиях, доступных в pkgin.

Все подробности об управлении пакетами можно найти в *руководстве по pkgsrc*

Хранение стороннего программного обеспечения

Во многих UNIX-подобных системах структура каталогов в `/usr/local` зарезервирована для приложений и файлов, не зависящих от системы управления программным обеспечением. Именно из-за этого соглашения большинство разработчиков ожидают, что их программы будут устанавливаться в `/usr/local`. В NetBSD нет каталога `/usr/local`, но при необходимости его можно создать вручную. NetBSD не следит за тем, что установлено в `/usr/local`, поэтому эта задача ложится на вас как на системного администратора.

5.16. Оповещения о безопасности

К тому времени, когда вы установите систему, в выпуске вполне могут быть найдены ошибки. Обо всех существенных и легко исправимых проблемах сообщается на странице //www.NetBSD.org/support/security/. Рекомендуется регулярно её проверять.

5.17. Остановка и перезагрузка системы

Чтобы остановить или перезагрузить систему, используйте одну из двух следующих команд shutdown:

```
# shutdown -h now
# shutdown -r now
```

Две другие команды для тех же задач:

```
# halt
# reboot
```

halt, reboot и shutdown — не синонимы: последняя сложнее. В многопользовательской системе действительно следует использовать shutdown: она позволяет запланировать время выключения и уведомить пользователей, а также корректно останавливает процессы. Дополнительные сведения см. на страницах руководства shutdown(8), halt(8) и reboot(8).

Глава 6. Редактирование

6.1. Знакомство с vi

Нельзя сказать, что редактор vi нуждается в представлении опытным пользователям UNIX. Редактор vi, изначально разработанный Биллом Джоем из Sun Microsystems, — это бесконечно расширяемый, простой в использовании *лёгкий* ASCII-редактор, а также бич всех новичков. В этом разделе мы познакомим с редактором vi новичков и, возможно, подбросим несколько идей и опытным пользователям.

В первой половине раздела рассказывается о редактировании, сохранении, копировании и вставке, а также о навигации по файлу в сеансе vi. Вторая половина — пошаговый пример сеанса vi, который поможет начать работу.

Это вводное руководство по работе с редактором vi, и оно *отнюдь не* является исчерпывающим. Его цель — помочь начинающему пользователю освоить vi настолько, чтобы вносить изменения в файлы и создавать их.

6.1.1. Интерфейс vi

Работа с редактором vi почти ничем не отличается от работы с любым другим терминальным программным обеспечением, за одним исключением: он не использует интерфейс в стиле вкладок (или curses, если угодно), хотя многие версии vi *и применяют* curses, это не даёт такого же внешнего вида и ощущений, как типичный интерфейс на основе curses. Вместо этого он работает в двух режимах — *командном* и *редактирования*. Это может показаться странным, но, если подумать, мало чем отличается от редактирования в оконных редакторах. Возьмём пример: если вы работаете, скажем, в gedit, берёте мышь, выделяете текст, выбираете «вырезать», а затем «вставить», то всё время, пока вы пользуетесь мышью, вы не редактируете (хотя и можете). В vi то же действие выполняется так: удалите всю строку командой **dd** в командном режиме, перейдите к строке, ниже которой нужно вставить текст, и нажмите **p** в командном режиме. Можно даже сказать, что это аналогия «режим мыши против командного режима» (хотя они не совсем идентичны, концептуально идея похожа).

Чтобы начать сеанс vi, достаточно запустить его так же, как любое терминальное приложение:

```
$ vi filename
```

Важно помнить: при редактировании файл загружается в буфер в памяти. Далее в тексте буфер и файл будут упоминаться в соответствующем контексте. Файл *изменяется только* тогда, когда пользователь зафиксирует изменения одной из команд записи.

6.1.2. Переход в режим редактирования

Редактор vi поддерживает ряд параметров, которые можно указать при запуске; пока мы рассмотрим только запуск с настройками по умолчанию. При запуске, как показано выше, редактор по умолчанию начинает работу в командном режиме, поэтому, по сути, вы не можете сразу начать вводить текст в буфер. Чтобы вводить текст, нужно выйти из командного режима. Ниже описаны команды перехода в режим редактирования:

```
a     Append after cursor.

A     Append to end of line.

C     Change the rest of current line.

cw    Change the current word.

i     Insert before cursor.

I     Insert before first non blank line.

o     Open a line below for insert

O     Open a line above for insert.
```

6.1.3. Переключение режимов и сохранение буферов в файлы

Разумеется, знать команды редактирования мало толку, если вы не можете вернуться в командный режим и сохранить файл; чтобы вернуться, просто нажмите клавишу **ESC**. Для ввода некоторых команд нужно использовать двоеточие. Команды записи — как раз одна из таких групп. Для этого просто введите **:**.

Нажатие двоеточия помещает пользователя в приглашение двоеточия (или *командное*, если угодно) в левом нижнем углу экрана. Теперь рассмотрим команды сохранения:

```
:w    Write the buffer to file.

:wq   Write the buffer to file and quit.
```

6.1.4. Копирование и вставка

Что толку в редакторе, если нельзя манипулировать блоками текста? Конечно, vi поддерживает эту возможность, и, как и большинство команд vi, она довольно интуитивна. Чтобы скопировать строку, но *не* удалять её, просто введите **yy** или **Y** в командном режиме — текущая строка будет скопирована в буфер. Чтобы вставить строку в нужное место, перейдите к строке над местом вставки и нажмите клавишу **p** — команду «put». Чтобы переместить строку, просто удалите её целиком командой **dd**, перейдите в нужное место и вставьте.

6.1.4.1. Ой, я не это хотел сделать!

Отмена устроена довольно просто: **u** отменяет последнее действие, а **U** — последнюю удалённую строку или изменения, сделанные в последней строке.

6.1.5. Навигация по буферу

Большинство руководств и учебников по vi начинают с навигации, однако — как и в большинстве редакторов — чтобы перемещаться по файлу, должно быть что-то, по чему можно перемещаться (именно поэтому этот раздел изложен несколько в обратном порядке). В зависимости от разновидности vi (и даже от того, *является* ли она vi, а не, скажем, elvis, nvi или vim) перемещаться можно как в режиме редактирования, так и в командном режиме.

Новичку, на мой взгляд, несколько безопаснее переключаться в командный режим и перемещаться уже в нём, пока не появится достаточная практика. Клавиши навигации для терминалов, которые не распознаются или не поддерживают клавиши со стрелками, приведены ниже:

```
k     Moves the cursor up one line.

j     Moves the cursor down one line.

l     Moves the cursor right one character.

h     Moves the cursor left one character.
```

Если терминал распознаётся и поддерживает их, то в командном режиме для перемещения по буферу можно использовать клавиши со стрелками.

Помимо простой «навигации по одному месту», vi поддерживает переход к строке: достаточно ввести номер строки в приглашении двоеточия. Например, чтобы перейти к строке 223, нажатия клавиш из режима редактирования будут выглядеть так:

```
ESC

:223
```

6.1.6. Поиск по файлу — дополнительное средство навигации

Редактор vi поддерживает поиск с использованием синтаксиса регулярных выражений, однако в командном режиме он вызывается немного иначе. Нужно просто в командном режиме нажать клавишу / и ввести то, что вы ищете; например, предположим, что я ищу выражение *foo*:

```
/foo
```

Вот и всё. Чтобы показать несколько иное выражение, предположим, что я ищу *foo bar*:

```
/foo bar
```

6.1.6.1. Дополнительные команды навигации

Поиск и прокрутка — не единственные способы перемещения по буферу vi. Ниже приведён список кратких команд навигации, доступных в vi:

```
0     Move to beginning of line.

$     Move to end of line.

b     Back up one word.

w     Move forward one word.

G     Move to the bottom of the buffer.

H     Move to the top line on the screen.

L     Move to the last line on the screen.
```

```
M     Move the cursor to the middle of the screen.

N     Scan for next search match but opposite direction.

n     Scan for next search match in the same direction.
```

6.1.7. Пример сеанса работы

Теперь, когда основы рассмотрены, выполним пример сеанса работы, используя несколько описанных выше приёмов. Сначала откроем пустой файл в буфере из командной строки следующим образом:

```
# vi foo.txt
```

Затем переключимся в режим редактирования и введём две строки, разделённые пустой строкой; помните, что буфер пуст, поэтому нажмём клавишу **i**, чтобы вставлять текст перед курсором, и введём некоторый текст:

```
This is some text

there we skipped a line
~
~
~
~
```

Теперь нажмите клавишу **ESC**, чтобы вернуться в командный режим.

Теперь, когда мы в командном режиме, сохраним файл. Сначала нажмите клавишу **:** — курсор должен оказаться в левом нижнем углу сразу после приглашения. В приглашении **:** введите **w** и нажмите клавишу **ENTER** или **RETURN**. Файл только что сохранён. Должно было появиться сообщение об этом; некоторые редакторы vi также сообщают имя, число строк и размер файла.

Пришло время навигации: курсор должен находиться там же, где был при сохранении файла. Попробуйте немного подвигаться с помощью клавиш со стрелками. Если они не работают (или вам просто любопытно), попробуйте клавиши **hjkl** и посмотрите, как они действуют.

Наконец, сделаем ещё две вещи: сначала перейдём к первой строке, а затем к первому символу. Попробуйте другие клавиши навигации командного режима на этой строке — нажмите следующие клавиши несколько раз:

```
$

0

$

0
```

Курсор должен перепрыгнуть в конец строки, затем обратно в начало и снова в конец.

Затем выполните поиск выражения, нажав клавишу /, и введите выражение следующим образом:

```
/we
```

Курсор должен перейти к *первому вхождению we*.

Теперь сохраните файл и выйдите, используя команды write и quit:

```
:wq
```

6.2. Настройка vi

Стандартный редактор, поставляемый с NetBSD, — это, разумеется, vi, самый любимый и самый ненавидимый редактор в мире. Если вы не пользуетесь vi, пропустите этот раздел; в противном случае прочитайте его перед установкой других версий vi. Редактор vi в NetBSD (*nvi*) был написан Китом Бостиком из UCB, чтобы получить свободно распространяемую версию этого редактора, и имеет множество мощных расширений, которые стоит изучить, оставаясь при этом весьма совместимым с исходным vi. Nvi стал стандартной версией vi для BSD.

Среди наиболее интересных расширений:

- Расширенные регулярные выражения (в стиле egrep), включаемые параметром `extended`.
- Стеки тегов.
- Бесконечная отмена (чтобы отменить действие, нажмите **u**; чтобы продолжать отмену, нажмите **.**).
- Инкрементный поиск, включаемый параметром `searchincr`.
- Прокрутка строк влево-вправо, включаемая параметром `leftright`; число прокручиваемых столбцов задаётся параметром `sidescroll`.
- Редактирование истории командной строки, включаемое параметром `cedit`.
- Дополнение имён файлов, включаемое параметром `filec`.
- Фоновые экраны и дисплеи.
- Редактирование с разделённым экраном.

6.2.1. Расширения `.exrc`

В следующем примере показан файл `.exrc` с несколькими включёнными расширенными параметрами.

```
set showmode ruler
set filec=^[
set cedit=^[
```

Первая строка включает отображение позиции курсора (строка и столбец) и текущего режима (Command, Insert, Append) в строке состояния. Вторая строка (где ^[— символ ESC) включает дополнение имён файлов по символу ESC. Третья строка включает редактирование истории командной строки (тоже по символу ESC). Например, ввод «:» и последующее нажатие ESC открывает окно со списком предыдущих команд, которые можно редактировать и выполнять (нажатие Enter на команде выполняет её).

6.2.2. Документация

Набор установки misc (`misc.tgz`) содержит много полезной документации по (n)vi и ex; после установки она доступна в каталоге `/usr/share/doc`. Например:

- Edit: учебное пособие — `/usr/share/doc/usd/edit/edit.{ps.gz,txt}`

- Справочное руководство по Ex — `/usr/share/doc/reference/ref1/ex/reference.{ps.gz,txt}`
- Страница руководства vi — vi(1)
- Введение в редактирование экрана с помощью Vi Уильяма Джоя и Марка Хортона — `/usr/share/doc/usd/vi/vitut.{ps.gz,txt}`
- Справочное руководство по Vi/Ex Кита Бостика — `/usr/share/doc/reference/ref1/vi/vi.{ps.gz,txt}`
- Краткий справочник по Ex/Vi — `/usr/share/doc/usd/vi/summary.{ps.gz,txt}`

Если вы никогда не пользовались vi, *Введение в редактирование экрана с помощью Vi* Уильяма Джоя и Марка Хортона — очень хорошая отправная точка.

Если вы хотите узнать больше о vi и расширениях nvi, прочитайте *Справочное руководство по Vi/Ex* Кита Бостика, где описаны все команды и параметры редактора.

6.3. Использование тегов в vi

Эта тема не имеет прямого отношения к NetBSD, но может быть полезной, например при изучении исходных текстов ядра.

Когда вы изучаете набор исходных текстов в дереве каталогов и подкаталогов, можно упростить работу, используя функцию *тегов* в vi. Метод таков:

1. Перейдите командой **cd** в базовый каталог исходных текстов.

```
$ cd /path
```

2. Введите следующие команды:

```
$ find . -name "*.[ch]" > filelist
$ cat filelist | xargs ctags
```

3. Добавьте следующую строку в `.exrc`

```
set tags=/path/tags
```

(подставьте правильный путь вместо `path`.)

Глава 7. Система rc.d

NetBSD использует отдельные сценарии для управления службами, подобно System V, но без уровней выполнения (runlevels). Эта глава содержит обзор системы rc.d и её настройки.

7.1. Основы

Файлы начальной загрузки системы находятся в каталоге `/etc`. Это:

- `/etc/rc`
- `/etc/rc.conf`
- `/etc/rc.d/*`
- `/etc/rc.local`
- `/etc/rc.shutdown`
- `/etc/rc.subr`
- `/etc/defaults/*`
- `/etc/rc.conf.d/*`

Сначала обзор управляющих и вспомогательных сценариев (они также описаны в rc(8)).

- После того как ядро при загрузке инициализирует все устройства, оно запускает init(8), который, в свою очередь, выполняет `/etc/rc`.
- `/etc/rc` упорядочивает сценарии в `/etc/rc.d` с помощью rcorder(8) и затем выполняет их в этом порядке. Подробности о том, как определяется порядок сценариев rc.d, см. на странице руководства rcorder(8).
- `/etc/rc.subr` содержит общие функции, используемые `/etc/rc` и различными сценариями rc.d.
- При завершении работы системы командой shutdown(8) запускается `/etc/rc.shutdown`, который выполняет сценарии из `/etc/rc.d` в обратном порядке (как определено rcorder(8)). Учтите, что если вы завершаете работу системы командой halt(8), эти сценарии не выполняются.

Дополнительные сценарии вне каталога `rc.d`:

- `/etc/rc.local` — почти последний сценарий, вызываемый при загрузке. Администратор может отредактировать этот сценарий, чтобы запускать локальные демоны, которые не укладываются в модель rc.d.

Сценариями rc.d управляет центральный файл конфигурации `/etc/rc.conf`, который загружает настройки по умолчанию из `/etc/defaults/rc.conf`. Если вы хотите изменить настройку по умолчанию, не редактируйте `/etc/defaults/rc.conf`; вместо этого задайте её в `/etc/rc.conf`. Это переопределит значение по умолчанию.

Полезно прочитать страницу руководства rc.conf(5), чтобы узнать о службах, доступных вам.

В следующем примере показано, как включить демон SSH, который по умолчанию отключён:

```
# cd /etc; grep ssh defaults/rc.conf
sshd=NO                 sshd_flags=""
```

```
# echo "sshd=YES" >> rc.conf
```

Теперь sshd(8) будет запускаться автоматически при старте системы. В следующем разделе описано, как запускать и останавливать службы в любое время.

И последнее: в каталоге `/etc/rc.conf.d/` можно создавать файлы, переопределяющие поведение конкретного сценария rc.d без правки самого сценария.

7.2. Сценарии rc.d

Сами сценарии, управляющие службами, находятся в `/etc/rc.d`. Эти сценарии автоматически запускаются при загрузке, но при необходимости их можно вызвать вручную — через псевдоним service(8) или напрямую. В следующем примере показано, как запустить демон SSH, который мы включили в предыдущем разделе:

```
# service sshd start
Starting sshd.
```

Позже, если вы захотите остановить демон SSH, выполните следующую команду:

```
# service sshd stop
Stopping sshd.
Waiting for PIDS: 123.
```

Сценарии rc.d принимают один из следующих аргументов:

- start
- stop
- restart
- status

Некоторые сценарии могут поддерживать другие аргументы (например, «reload»), но каждый сценарий поддерживает как минимум перечисленные выше команды.

В качестве примера: после добавления новой записи в базу данных named(8) демону можно дать указание перечитать файлы конфигурации следующей командой:

```
# service named reload
Reloading named config files.
```

Учтите, что все рассмотренные выше команды выполняются только в том случае, если соответствующая служба включена в `/etc/rc.conf`. Это требование можно обойти, добавив перед командой «one», как здесь:

```
# service httpd onestart
Starting httpd.
```

Приведённая выше команда позволит запустить службу httpd(8) один раз. Чтобы остановить службу, запущенную таким образом, передайте сценарию «onestop».

7.2.1. Пакеты, устанавливающие сценарии rc.d

Некоторые пакеты устанавливают сценарии rc.d. По умолчанию сценарии rc.d из пакетов находятся в `/usr/pkg/share/examples/rc.d`, и, чтобы их использовать, их нужно вручную скопировать в `/etc/rc.d`. Если перед установкой пакетов задать переменную окружения `PKG_RCD_SCRIPTS=yes`, сценарии rc.d будут автоматически копироваться в `/etc/rc.d`.

7.3. Роль rcorder и сценариев rc.d

Система загрузки любой Unix-системы так или иначе определяет порядок запуска служб. В одних Unix-системах для этого файлы нумеруются и/или раскладываются по отдельным каталогам уровней выполнения. Solaris полагается на то, что шаблоны вроде `/etc/rc[23].d/S*` при раскрытии сортируются в числовом порядке. Другие просто помещают все команды, которые нужно запустить, в один монолитный сценарий (это традиционный метод BSD, и именно так NetBSD работала до появления системы rc.d). В современной NetBSD это делается с помощью сценариев rc.d и их содержимого. Обратите внимание, что NetBSD не использует несколько уровней выполнения.

В начале каждого сценария rc.d есть несколько закомментированных строк, содержащих один из следующих элементов:

- REQUIRE
- PROVIDE
- BEFORE
- KEYWORD

Они описывают зависимости конкретного сценария и позволяют rcorder легко двигаться «вверх» или «вниз» в зависимости от ситуации. В качестве примера приведём информацию о порядке, содержащуюся в `/etc/rc.d/nfsd`:

```
...
 PROVIDE: nfsd
 REQUIRE: rpcbind mountd
...
```

Здесь видно, что этот сценарий предоставляет службу «nfsd» и требует, чтобы сначала были запущены «rpcbind» и «mountd». Утилита rcorder(8) используется при загрузке системы, чтобы просмотреть все сценарии rc.d и определить порядок их запуска.

7.4. Дополнительная литература

Люк Мьюберн (Luke Mewburn), один из основных разработчиков системы rc.d, сделал доклад о ней на USENIX 2001. Доклад доступен в формате PDF.

Глава 8. Драйверы консоли

8.1. wscons

Wscons — это независимый от платформы драйвер консоли рабочей станции в NetBSD. Он обеспечивает полную абстракцию клавиатур и мышей. Это значит, что вы можете подключить несколько клавиатур или мышей, и они будут мультиплексированы в один терминал, а также что он может мультиплексировать несколько виртуальных терминалов в один физический терминал.

Поддержка wscons по умолчанию включена на большинстве архитектур. Включить её вручную можно, добавив `wscons=YES` в ваш `/etc/rc.conf`. Затем настройте нужное число виртуальных консолей, как описано в разделе 8.1.1.1 «Виртуальные консоли», и запустите wscons, введя `service wscons start`, а затем `service ttys restart`. Теперь вы можете переключаться между виртуальными консолями, нажимая Ctrl+Alt+F*n* или аналогичное сочетание в зависимости от платформы.

Wscons состоит из трёх подсистем: wsdisplay, wskbd и wsmouse. Эти подсистемы отвечают за абстракцию всех устройств отображения, клавиатур и мышей соответственно. В следующих разделах рассматривается настройка wscons по подсистемам.

8.1.1. wsdisplay

В этом разделе объясняется, как настроить параметры отображения и экрана.

8.1.1.1. Виртуальные консоли

Число заранее выделяемых виртуальных консолей задаётся следующим параметром конфигурации ядра

```
options     WSDISPLAY_DEFAULTSCREENS=4
```

Другие консоли можно добавить, включив соответствующие строки в файле `/etc/wscons.conf`: из строк, начинающихся с `screen x`, нужно удалить знак комментария (#). В следующем примере к четырём заранее выделенным консолям добавляется пятая:

```
# screens to create
#       idx     screen  emul
#screen 0       -       vt100
screen  1       -       vt100
screen  2       -       vt100
screen  3       -       vt100
screen  4       -       -
#screen 4       80x25bf vt100
#screen 5       80x50   vt100
```

Сценарий `/etc/rc.d/wscons` превращает каждую незакомментированную строку в вызов команды **wsconscfg**: столбцы становятся параметрами вызова. Столбец *idx* становится параметром `index`, столбец *screen* — параметром `-t type` (он определяет тип экрана: число строк и столбцов, количество цветов, …), а столбец *emul* — параметром `-e emul`, который задаёт эмуляцию. Например:

```
screen 3       -       vt100
```

превращается в вызов:

```
wsconscfg -e vt100 3
```

Обратите внимание, что между консолями, заранее выделенными ядром, и консолями, выделяемыми при загрузке через `/etc/wscons.conf`, возможен (безвредный) конфликт. Если при загрузке система пытается выделить уже выделенный экран, выводится следующее сообщение:

```
wsconscfg: WSDISPLAYIO_ADDSCREEN: Device busy
```

Решение — закомментировать соответствующие строки в `/etc/wscons.conf`.

Учтите: хотя можно удалить экран и добавить его с другими настройками, строго говоря, изменить настройки существующего экрана невозможно.

`screen 0` нельзя удалить, если он используется как системная консоль. Это означает, что настройки screen 0 нельзя изменить в работающей системе, если он используется как системная консоль.

Виртуальная консоль также должна быть активна в `/etc/ttys`, чтобы NetBSD запускала программу getty(8) для запроса имени пользователя. Например:

```
console "/usr/libexec/getty Pc"         pc3     off secure
ttyE0   "/usr/libexec/getty Pc"         vt220   on secure
ttyE1   "/usr/libexec/getty Pc"         vt220   on secure
ttyE2   "/usr/libexec/getty Pc"         vt220   on secure
ttyE3   "/usr/libexec/getty Pc"         vt220   off secure
...
```

При запуске X-сервер ищет виртуальную консоль, на которой не запущена программа getty(8), то есть одна консоль должна оставаться в состоянии «off» в `/etc/ttys`. Строка

```
ttyE3   "/usr/libexec/getty Pc"         vt220   off secure
```

из файла `/etc/ttys` используется X-сервером для этой цели. Чтобы использовать экран с номером, отличным от 4, X-серверу нужно передать параметр вида vt*n*, где *n* — номер функциональной клавиши, используемой для активации экрана для X.

Например, включить `screen 7` можно в `/etc/wscons.conf`, а X запустить с параметром `vt8`. Если вы используете xdm, необходимо отредактировать `/etc/X11/xdm/Xservers`. Например:

```
:0 local /usr/X11R7/bin/X +kb dpms -bpp 16 dpms vt8
```

8.1.1.1.1. Как избавиться от сообщения `WSDISPLAYIO_ADDSCREEN: Device busy`

Это сообщение об ошибке обычно появляется, когда wsconscfg пытается добавить экран, который уже существует. Так бывает, если у вас есть строка `screen 0` в файле `/etc/wscons.conf`, поскольку ядро всегда выделяет экран 0 как консольное устройство. В этом случае сообщение об ошибке безвредно, и от него можно избавиться, удалив (или закомментировав) строку `screen 0`.

8.1.1.2. Текстовый режим на 50 строк с wscons

Этот режим включается в `/etc/wscons.conf`. Следующую строку нужно раскомментировать:

```
font ibm  -  8  ibm  /usr/share/pcvt/fonts/vt220l.808
```

Затем нужно изменить следующие строки:

```
#screen 0       80x50   vt100
screen  1       80x50   vt100
screen  2       80x50   vt100
screen  3       80x50   vt100
screen  4       80x50   vt100
screen  5       80x50   vt100
screen  6       80x50   vt100
screen  7       80x50   vt100
```

Эта конфигурация включает восемь экранов, доступ к которым осуществляется сочетанием клавиш Ctrl-Alt-F*n* (где *n* принимает значения от 1 до 8); соответствующие устройства — ttyE0..ttyE7. Чтобы включить их и получить приглашение ко входу, необходимо изменить `/etc/ttys`:

```
ttyE0   "/usr/libexec/getty Pc"         vt220   on secure
ttyE1   "/usr/libexec/getty Pc"         vt220   on secure
ttyE2   "/usr/libexec/getty Pc"         vt220   on secure
ttyE3   "/usr/libexec/getty Pc"         vt220   on secure
ttyE4   "/usr/libexec/getty Pc"         vt220   on secure
ttyE5   "/usr/libexec/getty Pc"         vt220   on secure
ttyE6   "/usr/libexec/getty Pc"         vt220   on secure
ttyE7   "/usr/libexec/getty Pc"         vt220   on secure
```

`screen 0` в качестве системной консоли можно заменить на другой тип экрана во время загрузки на дисплеях VGA. Это параметр конфигурации ядра. Если выбран режим, отличный от 80x25, необходимо убедиться, что в ядро скомпилирован пригодный для использования шрифт — для 80x50 это будет шрифт 8x8.

Со многими видеокартами ATI возникает проблема: они не реализуют стандартную логику переключения шрифтов VGA. Таким картам нужен ещё один параметр ядра, чтобы нестандартный консольный шрифт заработал.

Пример набора параметров конфигурации ядра:

```
options VGA_CONSOLE_SCREENTYPE="\"80x50\""
options VGA_CONSOLE_ATI_BROKEN_FONTSEL
options FONT_VT220L8x8
```

8.1.1.3. Включение консоли с кадровым буфером

Во многих архитектурах существует только один тип режима экрана — графический режим кадрового буфера. На машинах с видеокартами VGA есть второй режим — текстовый. Это оптимизированный режим, специально предназначенный для отображения текста. Поэтому он и является режимом консоли по умолчанию для ядер GENERIC на архитектурах, где видеокарта обычно является картой VGA (i386, amd64).

Однако на машинах с видеокартами VGA, поддерживающими расширение VESA BIOS (VBE), можно включить кадровый буфер.

Режим кадрового буфера VESA настраивается во время работы boot(8) с помощью команды `vesa`.

8.1.1.4. Включение прокрутки назад на консоли

Вы можете включить прокрутку назад на консолях wscons, скомпилировав параметр `WSDISPLAY_SCROLLSUPPORT` в своё ядро. При этом убедитесь, что у вас одновременно не включён параметр `VGA_RASTERCONSOLE`! Инструкции по сборке ядра приведены в главе 34, *Сборка ядра*.

Если у вас работает ядро с параметром `WSDISPLAY_SCROLLSUPPORT`, вы можете прокручивать консоль вверх, нажимая LEFT SHIFT вместе с PAGE UP/DOWN. Обратите внимание, что на системной консоли (ttyE0) это может не работать!

8.1.1.5. Wscons и цвета

8.1.1.5.1. Изменение цвета сообщений ядра

Цвет переднего плана и цвет фона сообщений ядра можно изменить, задав следующие параметры в файлах конфигурации ядра:

```
options WS_KERNEL_FG=WSCOL_xxx
options WS_KERNEL_BG=WSCOL_xxx
```

Цветовые константы `WSCOL_xxx` определены в `src/sys/dev/wscons/wsdisplayvar.h`.

Вы можете легко настроить многие аспекты внешнего вида дисплея: цвета, используемые для вывода обычных сообщений, цвета, используемые для вывода сообщений ядра, и цвет рамки, рисуемой вокруг экрана.

Все эти детали можно изменить либо через параметры ядра, либо с помощью утилиты wsconsctl(8); последний способ может быть предпочтительнее, если вы не хотите компилировать собственное ядро, так как параметры по умолчанию в `GENERIC` подходят для того, чтобы воспользоваться этим советом.

Через wsconsctl(8) можно задать следующие параметры:

- `border`: цвет рамки экрана. Соответствующий параметр ядра — `WSDISPLAY_BORDER_COLOR`.
- `msg.default.attrs`: атрибуты, используемые для вывода обычных сообщений консоли. Соответствующие параметры ядра — `WS_DEFAULT_COLATTR` и `WS_DEFAULT_MONOATTR` (первый используется на цветных дисплеях, второй — на монохромных).
- `msg.default.bg`: цвет фона, используемый для вывода обычных сообщений консоли. Соответствующий параметр ядра — `WS_DEFAULT_BG`.
- `msg.default.fg`: цвет переднего плана, используемый для вывода обычных сообщений консоли. Соответствующий параметр ядра — `WS_DEFAULT_FG`.
- `msg.kernel.attrs`: атрибуты, используемые для вывода сообщений ядра и предупреждений. Соответствующие параметры ядра — `WS_KERNEL_COLATTR` и `WS_KERNEL_MONOATTR` (первый используется на цветных дисплеях, второй — на монохромных).
- `msg.kernel.bg`: цвет фона, используемый для вывода сообщений ядра и предупреждений. Соответствующий параметр ядра — `WS_KERNEL_BG`.
- `msg.kernel.fg`: цвет переднего плана, используемый для вывода сообщений ядра и предупреждений. Соответствующий параметр ядра — `WS_KERNEL_FG`.

В качестве цветов принимаются значения: black, red, green, brown, blue, magenta, cyan и white. Атрибуты представляют собой разделённый запятыми список одного или нескольких флагов, которыми могут быть: reverse, hilit, blink и/или underline.

Например, чтобы воспроизвести вид одной из тех старых машин Amstrad:

```
wsconsctl -d -w border=blue msg.default.bg=blue msg.default.fg=white msg.default.attrs=hilit
```

Или чтобы сообщения ядра выводились красным:

```
wsconsctl -d -w msg.kernel.fg=red
```

Учтите, что в старых версиях NetBSD доступна лишь часть этой функциональности; точнее говоря, изменить цвета ядра можно только путём изменения параметров ядра, как описано выше. Учтите также, что эти возможности поддерживаются не всеми драйверами, поэтому на некоторых архитектурах результат может быть неправильным.

8.1.1.5.2. Настройка использования цветов приложениями на консоли

NetBSD использует базу данных terminfo, чтобы сообщить приложениям о возможностях текущего терминала. Например, некоторые терминалы не поддерживают цвета, некоторые не поддерживают подчёркивание (таковы, например, консольные терминалы VGA на ПК) и т. д. Переменная окружения `TERM` сообщает библиотеке terminfo тип терминала. Затем библиотека обращается за параметрами к своей базе данных.

Значение `TERM` по умолчанию можно посмотреть, набрав `echo $TERM` на интересующем терминале. Обычно это что-то вроде `vt220`. Этот тип терминала не поддерживает цвета. На типичной консоли ПК с 25 строками вместо него можно указать `wsvt25`, чтобы получить цвета. В оболочке C (csh) это делается так:

```
setenv TERM wsvt25
```

В оболочке, совместимой с Bourne (sh, ksh), можно ввести:

```
export TERM=wsvt25
```

Если это не сработает, попробуйте тип терминала `ansi`, который поддерживает цветовые коды ANSI. Однако при этом типе терминала может отсутствовать другая функциональность. Загляните в файл `/usr/share/misc/terminfo` — возможно, вы найдёте подходящий вариант для своего типа консоли.

8.1.1.6. Загрузка альтернативных шрифтов

В `/usr/share/wscons/fonts` есть несколько шрифтов, которые можно загрузить как консольные. Это делается командой wsfontload(8), например: `wsfontload -N ibm -h 8 -e ibm /usr/share/wscons/fonts/vt220l.808`. Эта команда загружает шрифт в кодировке IBM (`-e ibm`) из файла `vt2201.808` высотой восемь пикселей (`-h 8`). Задайте ему имя ibm для дальнейшего использования (`-N ibm`).

Чтобы действительно вывести шрифт на консоль, воспользуйтесь командой `wsconsctl -dw font=ibm`.

Если вы хотите отредактировать шрифт, можно воспользоваться старыми утилитами pcvt из пакета `sysutils/pcvt-utils`.

8.1.2. wskbd

8.1.2.1. Раскладки клавиатуры

Wscons также позволяет задать раскладку, сопоставляющую клавиши различных национальных клавиатур нужным символам. Например, чтобы установить итальянскую раскладку, выполните:

```
# wsconsctl -k -w encoding=it
encoding -> it
```

Эта настройка сохранится до следующей перезагрузки. Чтобы сделать её постоянной, добавьте строку `encoding` в `/etc/wscons.conf`: она будет выполнена автоматически при следующей перезагрузке.

```
# cp /etc/wscons.conf /etc/wscons.conf.orig
# echo encoding it >>/etc/wscons.conf
```

Будьте внимательны и наберите два символа `>`. Если набрать только один `>`, вы перезапишете файл вместо добавления строки. Но именно поэтому мы всегда делаем резервные копии, прежде чем трогать важные файлы!

Полный список раскладок клавиатуры и их вариантов приведён в wskbd(4).

Вкомпилированное в ядро значение по умолчанию можно изменить, добавив `options PCKBD_LAYOUT=KB_`*`encoding`*, где *encoding* — записанный заглавными буквами элемент из приведённого выше списка (например: `PCKBD_LAYOUT=KB_FR`). Варианты можно объединять побитовым ИЛИ (например: `PCKBD_LAYOUT=KB_US|KB_SWAPCTRLCAPS`).

Настройка раскладки клавиатуры в X описана в другом месте.

8.1.2.1.1. Добавление раскладки в wscons

Если ваша любимая раскладка не поддерживается, можно начать разбираться с `src/sys/dev/wscons/wsksymdef.h` и `src/sys/dev/pckbport/wskbdmap_mfii.c`, чтобы сделать собственную. Обязательно отправьте с помощью send-pr запрос на изменение (PR) со своей работой, чтобы другие могли этим воспользоваться!

Проверить раскладку можно с помощью **wsconsctl**, вместо того чтобы вписывать её напрямую в файл раскладки клавиатуры. Например, чтобы задать, что код клавиши 51 без модификаторов должен давать запятую, с shift — вопросительный знак, с alt — точку с запятой, а с alt и shift одновременно — двоеточие, выполните следующую команду:

```
wsconsctl -w "map += keycode 51=comma question semicolon colon"
```

8.1.2.2. Изменение скорости повтора клавиатуры

Скорость повтора клавиатуры можно настроить с помощью утилиты **wsconsctl(8)**. Здесь важны две переменные: `repeat.del1`, задающая задержку перед началом повтора символа, и `repeat.deln`, задающая задержку между повторами символа (после их начала).

Рассмотрим пример: предположим, вы хотите ускорить клавиатуру. Из командной строки можно сделать так:

```
wsconsctl -w repeat.del1=300
wsconsctl -w repeat.deln=40
```

Или, если вы хотите, чтобы это происходило автоматически при каждой загрузке системы, можно добавить следующие строки в `/etc/wscons.conf`:

```
setvar repeat.del1=300
setvar repeat.deln=40
```

8.1.3. wsmouse

8.1.3.1. Поддержка последовательной мыши

Устройство wsmouse (часть wscons) не поддерживает последовательные мыши напрямую. Демон moused(8) предназначен для чтения данных от последовательной мыши, преобразования их в события wsmouse и помещения их в очередь событий wscons, чтобы мышь можно было использовать через уровень абстракции, предоставляемый wsmouse.

Типичный пример использования: `moused -p /dev/tty00`. Эта команда попытается определить тип мыши, подключённой к первому последовательному порту, и начнёт считывать её данные. Страница руководства moused(8) содержит дополнительные примеры.

8.1.3.2. Вырезание и вставка на консоли с помощью wsmoused

На консоли wscons можно помечать (вырезать) текст одной кнопкой мыши, а вставлять его обратно — другой.

Для этого включите «wsmoused» в `/etc/rc.conf` и запустите его:

```
# echo wsmoused=yes >>/etc/rc.conf
# service wsmoused start
```

После этого вы можете помечать текст левой кнопкой мыши и вставлять его правой. Чтобы настроить поведение wsmoused(8), обратитесь к её странице руководства, где также описан формат файла конфигурации wsmoused.conf(5); пример этого файла можно найти в `/usr/share/examples/wsmoused`.

Глава 9. Система X Window

9.1. Что такое X11 / Xorg?

В NetBSD для обеспечения графического интерфейса используется система X Window.

Обратите внимание, что система X Window — довольно минималистичный каркас. Она служит основой для современных рабочих сред, таких как MATE или Xfce, но они не являются частью системы X Window. NetBSD поставляется с системой X Window, но не включает эти рабочие среды: их необходимо устанавливать через pkgsrc.

Начав работать с X, вы обнаружите множество новых терминов, которые поначалу могут показаться запутанными. Основные элементы таковы:

- *Сервер X*, работающий поверх оборудования. Сервер X предоставляет стандартный способ отображения графики (включая шрифты для вывода текста) и получения данных от мыши, клавиатуры и других устройств. На большинстве платформ NetBSD используется дисплейный сервер Xorg(1). Другие серверы X, входящие в NetBSD, включают Xnest(1), который запускает сервер X внутри другого сервера X в виде окна, и Xvfb(1), который запускает сервер X без вывода на физический экран и обычно используется для организации полностью удалённого рабочего стола с помощью `x11/x11vnc`.
- *Клиенты X*. Это программы, с которыми вы непосредственно взаимодействуете. Они работают поверх сервера X. Веб-браузер, например Firefox, — пример клиента X. X обеспечивает прозрачность сети, что означает, что вы можете запускать клиенты X на одной машине, а сервер X (то есть дисплей с видеооборудованием) — на другой. Клиент X выбирает сервер, который будет использовать в качестве дисплея, на основе переменной окружения `DISPLAY`, обычно `:0` для первого сервера и `:1` для второго.
- *Менеджер окон*, работающий поверх сервера X. Менеджер окон — это особый клиент X, которому разрешено управлять размещением окон. Он также может «оформлять» окна стандартными «элементами управления» (обычно они обеспечивают такие действия, как перемещение окон, изменение их размера, свёртывание в значок, закрытие и т. п.). ctwm(1) — менеджер окон по умолчанию в NetBSD.
- *Рабочая среда*, такая как MATE или Xfce. Это комплекты интегрированного программного обеспечения, предназначенные для того, чтобы предоставить вам чётко определённый набор программ и более или менее единый интерфейс для каждой из них. Обычно они включают менеджер окон, файловый менеджер, веб-браузер, почтовый клиент, мультимедийный проигрыватель, текстовый редактор, адресную книгу, программу просмотра справки и т. д. Как вы, вероятно, уже догадались, для использования X рабочая среда не требуется, но многие пользователи захотят её установить.
- *Компоновщик* или *менеджер композиции* работает на сервере X и перенаправляет отрисовку в закадровый буфер, обычно используя для окончательной отрисовки оборудование GPU (графический процессор). Он может обеспечивать дополнительные визуальные эффекты и часто VSync (вертикальную синхронизацию). Некоторые менеджеры окон, обычно входящие в состав больших рабочих сред, включают собственные менеджеры композиции. **xcompmgr** и `x11/picom` — внешние менеджеры композиции.

Система X Window входит в NetBSD в виде отдельных наборов распространения, см. раздел 3.10, „Тип установки“. Её можно добавить в уже установленную систему с помощью sysinst(8).

В NetBSD X11 размещается в иерархии файловой системы `/usr/X11R7`. Поэтому для использования X каталог `/usr/X11R7/bin` должен присутствовать в переменной `PATH` вашей оболочки. См. `~/.profile`.

9.2. Настройка

В большинстве случаев вы сможете начать использовать X вообще без какой-либо настройки, и **startx** будет работать нормально.

Однако в редких случаях настройка сервера X всё же требуется. Этот файл конфигурации находится по адресу `/etc/X11/xorg.conf`. Структура файла конфигурации формально описана в xorg.conf(5).

Чтобы сгенерировать исходный файл конфигурации для вашего сервера X, выполните команду

```
# X -configure
```

Эта команда должна создать файл конфигурации и поместить его в ваш домашний каталог. Чтобы проверить сгенерированный файл конфигурации, выполните, например, такую команду:

```
# X -config ~/xorg.conf.new
```

Если это удалось, вы должны увидеть фон в клетку и курсор в форме буквы X. Подвигайте курсор, чтобы убедиться, что мышь работает. Затем можно переключиться на другой виртуальный терминал (Ctrl-Alt-F#) или войти удалённо и завершить процесс X.

Если приведённая проверка прошла успешно, переместите файл на место как `/etc/X11/xorg.conf`, и всё готово.

9.3. Клавиатура

Даже если вы уже настроили клавиатуру для wscons (см. раздел 8.1, „wscons“), вам нужно настроить её и для X, по крайней мере если вы хотите использовать раскладку, отличную от US.

Простое решение — использовать setxkbmap(1).

Вот пример, показывающий, как использовать ивритскую раскладку клавиатуры, с Ctrl-Alt для переключения раскладок и с дополнительной опцией перестановки позиций клавиш Escape и Caps Lock:

```
setxkbmap -option grp:alt_shift_toggle us,il \
 -option caps:swapescape -option terminate:ctrl_alt_bksp
```

Если вы хотите изменить частоту повтора клавиатуры, её можно задать командой xset(1), которая принимает два аргумента: задержку и частоту соответственно. Следующий пример устанавливает начальную задержку 200 миллисекунд и частоту повтора 30 в секунду:

```
$ xset r 200 30
```

Вы также можете выполнить эту команду в файле `.xinitrc` или `.xsession`. Дополнительную информацию см. ниже (раздел 9.6, „Настройка X“).

9.4. Монитор

Если X работает не в том разрешении, в котором, по вашему мнению, должен, сначала выполните **xrandr** и посмотрите, есть ли нужное разрешение в списке. Если предпочтительное разрешение присутствует в выводе этой команды, вы можете изменить разрешение, например, так:

```
$ xrandr -s 1680x1050
```

xrandr также можно использовать для включения вывода на мониторы, подключённые на ходу. Управлять выходами можно графически с помощью пакета pkgsrc `x11/arandr`.

9.5. Запуск X

Запустить X можно следующей командой:

```
$ startx
```

Если базовая конфигурация сервера X верна, вы окажетесь в среде X с менеджером окон по умолчанию (ctwm). Если вы хотите более развитый менеджер окон или рабочую среду, многие из них доступны в pkgsrc. Сведения о добавлении и смене менеджеров окон см. в разделе 9.7, „Другие менеджеры окон и рабочие среды“.

9.6. Настройка X

Одна из первых вещей, которые вы захотите сделать, — изменить программы, запускаемые при первом старте X. Проще всего скопировать файл `.xinitrc` по умолчанию в свой домашний каталог и изменить его либо создать простой новый с нуля. Например:

```
$ cp /etc/X11/xinit/xinitrc ~/.xinitrc
$ chmod u+w ~/.xinitrc
$ vi ~/.xinitrc
```

Если вы используете xdm(1), вместо `~/.xinitrc.` используется `~/.xsession`.

Следующий пример показывает, как запустить менеджер окон (ctwm) и открыть экземпляр программ xterm и xterm. Цвет фона экрана установлен в «bisque4», который определён в `/usr/X11R7/lib/X11/rgb.txt`.

```
...
# start some programs - a basic clcok
xclock -geometry 50x50-1-1 &
# change the color of the "root window" ("desktop background")
xsetroot -solid bisque4 &
# spawn a terminal
uxterm -geometry 80x34-1+1 -bg OldLace &
exec ctwm -W   # no '&' here
```

При такой схеме, чтобы выйти из X, необходимо выйти из менеджера окон, что обычно делается выбором пункта «exit» в его меню.

Приведённый пример очень прост, но иллюстрирует основы управления клиентами, запускаемыми при старте X. Вы можете выполнить любое количество команд из вашего `.xinitrc`,

включая базовые команды настройки X, такие как **xset b off** для отключения звукового сигнала.

9.7. Другие менеджеры окон и рабочие среды

Если вам не нравится ctwm, который является очень простым менеджером окон, вы можете установить другой менеджер окон или рабочую среду из pkgsrc. В следующем примере используется менеджер окон Openbox, но в `pkgsrc/wm` доступны многие другие.

Openbox можно установить из двоичных пакетов или собрать с помощью pkgsrc. Как всегда, при правильно заданной PKG_PATH, способ с двоичным пакетом таков:

```
# pkgin in openbox
```

Чтобы собрать его с помощью pkgsrc, выполните:

```
# cd /usr/pkgsrc/wm/openbox
# make install
```

Теперь Openbox установлен; чтобы запустить его, нужно изменить файл `.xinitrc`: заменить строку, вызывающую `ctwm`, строкой, вызывающей `openbox`. Например:

```
# start some useful programs
xclock -geometry 50x50-1-1 &
# start window manager:
exec openbox   # no '&' here
```

Команда **startx** запустит сеанс X11 с Openbox. Если файл `.xinitrc` настроен, как в приведённом выше примере, то выбор пункта «Log Out» в меню Openbox завершит сеанс X11.

Установить рабочую среду почти так же просто. Следующий пример показывает, как использовать рабочую среду Xfce.

```
# pkgin in xfce4
```

В зависимости от ваших требований вы можете включить **dbus** как общесистемную службу. Следующий пример показывает, как это сделать. (Если вы не включите dbus как общесистемную службу, **startxfce4** запустит dbus в вашей учётной записи во время инициализации.)

```
# cp /usr/pkg/share/examples/rc.d/dbus /etc/rc.d
# echo dbus=YES >> /etc/rc.conf
# service dbus start
```

Если вы хотите управлять состоянием питания системы из рабочей среды, учётная запись, от имени которой вы собираетесь запускать X, также должна входить в группу «operator» (см. Раздел 5.6, «Добавление пользователей»).

Выполнив приведённые выше команды, отредактируйте свой `.xinitrc`, как описано выше, и замените «openbox» (или «ctwm») на «startxfce4». При следующем запуске **startx** будет запущена рабочая среда Xfce.

9.8. Графический вход с помощью xdm

Если вы постоянно работаете в X и первое, что вы делаете после входа в систему, — это запуск **startx**, вы можете настроить графический вход, который будет делать это автоматически. Это очень просто:

1. Создайте в своём домашнем каталоге файл `.xsession`. Этот файл аналогичен `.xinitrc` и, более того, может быть ссылкой на него.

```
$ ln -s .xinitrc ~/.xsession
```

2. Измените `/etc/rc.conf`, добавив следующую строку:

```
xdm=YES       # x11 display manager
```

3. Запустите xdm(1) (или перезагрузите систему — с этого момента это будет делаться автоматически):

```
# service xdm start
```

Файлы конфигурации xdm находятся в каталоге `/etc/X11/xdm`. Файл `Xservers` задаёт виртуальную консоль, на которой запускается X. По умолчанию используется «vt05» — консоль, на которую можно перейти по «Ctrl+Alt+F5». Если вы хотите использовать другую виртуальную консоль, измените vt05 нужным образом. Чтобы избежать конфликта клавиатуры между getty и xdm, обязательно запускайте xdm на том виртуальном терминале, где getty отключён. Например, если в `Xservers` у вас указано:

```
:0 local /usr/X11R7/bin/X :0 vt04
```

то в `/etc/ttys` у вас должно быть

```
ttyE3   "/usr/libexec/getty Pc"         wsvt25   off secure
```

(Учтите, что vt04 соответствует ttyE3: в `/etc/X11/xdm/Xservers` нумерация начинается с 1, а в `/etc/ttys` — с 0.)

Если вы используете xdm для запуска различных современных рабочих сред, таких как Xfce или MATE, вам потребуется переопределить разрешённые по умолчанию механизмы авторизации, добавив в `/etc/X11/xdm/xdm-config` следующее:

```
DisplayManager*authName:    MIT-MAGIC-COOKIE-1
```

Если вы хотите изменить внешний вид экрана входа xdm, можно изменить файл конфигурации xdm. Например, чтобы изменить цвет фона, добавьте следующую строку в файл `Xsetup_0`:

```
xsetroot -solid SeaGreen
```

9.9. Использование нескольких серверов X или удалённых серверов X

Это простой пример использования нескольких серверов X. Для наглядности мы просто воспользуемся Xnest(1), который создаёт новый сервер X `:1` в виде окна на существующем сервере `:0`:

```
$ Xnest :1 &
```

После этого можно запускать программы на втором сервере или даже другой менеджер окон:

```
$ DISPLAY=:1 uxterm &
$ DISPLAY=:1 ctwm &
```

Используя *перенаправление X11*, программы могут работать на удалённой машине, отображаясь при этом на локальной. Обычно это нужно включить в `/etc/ssh/sshd_config`:

```
X11Forwarding yes
```

Войдите с помощью ssh(1) и запускайте программы X обычным образом:

```
$ ssh -X remote.machine.example.com
$ uxterm &
```

На полностью headless-системе (без монитора) аналогичным образом можно использовать Xvfb(1) (*виртуальный фреймбуфер X*). Полностью виртуальный экран сервера X можно экспортировать по сети с помощью `x11/x11vnc`:

```
$ Xvfb :1 &
$ DISPLAY=:1 ctwm &
$ x11vnc -display :1 -localhost -passwdfile /path/to/password &
```

Обратите внимание, что мы используем ключ `-localhost`. Теоретически он запрещает удалённые подключения, однако на практике мы применяем туннель SSH для перенаправления порта VNC, что добавляет дополнительный уровень безопасности. Чтобы подключиться к машине без монитора:

```
$ ssh -L 5900:hostname:5900 hostname
$ vncviewer localhost &
```

9.10. Дополнительные ресурсы

- *An X Window System Tutorial* — это серия видео, в которой делается попытка объяснить основные понятия системы X Window, включая роль менеджера окон.
- *X Window System User's Guide for X11R3 and R4* (PDF, Web) авторов Valerie Quercia и Tim O'Reilley — классическая книга, в которой описаны некоторые возможности X; теперь её можно бесплатно прочитать в интернете.

Глава 10. Звук

10.1. Настройка устройства звука по умолчанию

С помощью audiocfg(1) можно вывести список аудиоустройств, проверить их и задать устройства звука по умолчанию.

Список всех доступных аудиоустройств выводится командой **audiocfg list**:

```
$ audiocfg list
0: [*] audio0 @ hdafg0: Realtek ALC292
       playback: 2ch, 48000Hz
       record:   2ch, 48000Hz
       (PR) slinear_le 16/16, 2ch, { 32000, 44100, 48000, 88200, 96000, 192000 }
       (PR) slinear_le 20/32, 2ch, { 32000, 44100, 48000, 88200, 96000, 192000 }
       (PR) slinear_le 24/32, 2ch, { 32000, 44100, 48000, 88200, 96000, 192000 }
       (  ) ac3 16/16, 2ch, { 32000, 44100, 48000, 88200, 96000, 192000 }
1: [ ] audio1 @ uaudio0: USB audio
       playback: 2ch, 48000Hz
       record:   1ch, 48000Hz
       (P-) slinear_le 16/16, 2ch, { 48000, 44100 }
       (-R) slinear_le 16/16, 1ch, { 48000, 44100 }
```

Звёздочка рядом с устройством *Realtek ALC292* означает, что в данный момент это устройство по умолчанию, поэтому любое приложение, которое читает из `/dev/audio` или пишет в него, будет воспроизводить или записывать звук через него. Оно также доступно как `/dev/audio0`, а для команд микшера — как `/dev/mixer0`.

Другое устройство, *USB audio*, — дополнительное подключённое устройство. Поскольку оно не является устройством по умолчанию, оно используется только при явном выборе в приложении. Оно доступно как `/dev/audio1`, а для команд микшера — как `/dev/mixer1`.

Строки *playback:* и *record:* показывают выбранный в данный момент аппаратный формат звука. Ниже перечислены другие поддерживаемые форматы. Некоторые устройства задают форматы воспроизведения и записи по отдельности, а другие — оба одновременно. На это указывает *PR*.

audiocfg test ***index*** позволяет проверить воспроизведение: для каждого канала устройства с номером ***index*** проигрывается сигнал длительностью 2 секунды:

```
$ audiocfg test 0
0: [*] audio0 @ hdafg0: Realtek ALC292
  testing channel 0... done
  testing channel 1... done
```

Если доступно больше одного аудиоустройства, с помощью **audiocfg default** ***index*** можно сменить устройство по умолчанию. Этот выбор сохраняется между перезагрузками. Обратите внимание, что в отличие от других команд audiocfg(1) команду **audiocfg default** необходимо запускать от имени root.

10.2. Настройка микшера и громкости

В NetBSD для настройки микширования звука — например, громкости записи и воспроизведения, а также используемых в данный момент источников и приёмников — используется mixerctl(1). Также

доступен интерфейс, который отображает ползунки и управляется стрелками на клавиатуре, — aiomixer(1).

Задайте текущую громкость воспроизведения:

```
$ mixerctl -w outputs.master=50
```

Выведите список доступных регуляторов и настроек:

```
$ mixerctl -av
outputs.master=255,255 volume
inputs.dac=255,255 volume
outputs.auto=255,255 volume delta=13
outputs.headphones=0,0 volume delta=13
outputs.hdmi=255,255 volume delta=13
outputs.select=headphones  [ auto headphones hdmi ]
```

Дополнительные устройства также можно настраивать с помощью mixerctl(1). Например, если вы только что подключили USB-аудиоустройство, оно могло подключиться как `/dev/audio1` и `/dev/mixer1` — это видно с помощью audiocfg(1). Соответственно, настроить его можно командой **mixerctl -d /dev/mixer1**.

10.2.1. Установка настроек микшера по умолчанию при загрузке

Настройки микшера по умолчанию можно применять при загрузке, задав `mixerctl=YES` в `/etc/rc.conf` и указав аргументы в `/etc/mixerctl.conf`. Например, этот `/etc/mixerctl.conf` задаёт громкость воспроизведения и приёмник воспроизведения:

```
outputs.master=120,120
outputs.select=headphones
```

Чтобы настройки устройств микшера автоматически загружались и сохранялись при загрузке и завершении работы, укажите в `/etc/rc.conf` каждое устройство, настройки которого нужно сохранять:

```
mixerctl=YES
mixerctl_mixers="mixer0 mixer1"
```

10.3. Псевдоустройства звука

Устройство pad(4) в NetBSD позволяет получать обратно данные от приложения с помощью виртуального аудиоустройства. Его можно использовать для перенаправления воспроизведения в другое место или для записи воспроизведения приложения.

Устройства `/dev/padN` при открытии на чтение выдают необработанный поток звука в фиксированном формате. В момент открытия они также создают устройство `/dev/audioN`, которое приложение может использовать для вывода. Создание устройства можно наблюдать с помощью dmesg(8).

В следующем примере записывается звук игры `games/jumpnbump` с помощью программы `multimedia/ffmpeg6`, которая одновременно кодирует данные с устройства pad в файл и записывает их обратно на реальное аудиоустройство. Обе программы доступны в коллекции пакетов NetBSD.

```
$ ffmpeg6 -f s16le -ar 44100 -ac 2 -i /dev/pad0 \
    -f wav output.wav -f oss /dev/audio
$ SDL_PATH_DSP=/dev/audio1 jumpnbump
```

10.4. Команды записи и воспроизведения

В состав NetBSD входит ряд команд, позволяющих пользователям воспроизводить и записывать звук из сценариев или из командной строки.

10.4.1. audioplay(1)

С помощью этой команды можно воспроизводить звуковые файлы простых форматов, таких как ULAW и WAVE. Для более сложных задач можно установить одну из многих программ, доступных в системе пакетов, которые позволяют воспроизводить звуковые файлы в разных форматах (например, MP3 и т. п.).

10.4.2. audiorecord(1)

Позволяет записывать звук с микрофона или другого входа в те же простые или необработанные форматы, которые поддерживает audioplay(1).

Следующая команда записывает звук качества CD в файл WAVE с устройства звука по умолчанию. Запись прекратится при завершении процесса:

```
$ audiorecord -d /dev/audio -F wav -e linear -c 2 -P 16 -s 44100 recording.wav
```

Воспроизведите запись (формат определяется по заголовкам WAVE):

```
$ audioplay recording.wav
```

10.4.3. audioctl(1)

audioctl(1) используется для ручной установки некоторых переменных, связанных с вводом-выводом звука, например частот воспроизведения и записи. Это полезно при записи необработанных отсчётов в `/dev/sound` без доступа к полному API audio(4), например из сценария оболочки, но в остальном при обычной работе не используется.

10.5. Поддержка MIDI

NetBSD включает встроенную поддержку MIDI через машинно-независимую систему midi(4). В неё входит поддержка MIDI-устройств USB.

Доступ к MIDI-устройствам поддерживается через прямой доступ к устройствам `/dev/rmidiX` или через устройство-секвенсор `/dev/music`.

Среди цифровых звуковых станций и другого программного обеспечения с поддержкой MIDI в NetBSD из коллекции пакетов есть `audio/lmms` и `audio/fluidsynth`. Некоторые MIDI-программы также входят в состав NetBSD:

10.5.1. midirecord(1)

Программа, позволяющая записывать MIDI-события с устройства в файлы формата Standard MIDI (SMF). Её также можно использовать для проверки устройства и того, что оно работает как ожидается, с помощью параметров **-D** и **-V**.

10.5.2. midiplay(1)

Программа, позволяющая воспроизводить файлы Standard MIDI и RMID на устройстве-секвенсоре MIDI.

10.6. Устройства Intel HD Audio

С 2010-х годов большинство машин x86 имеют аппаратное обеспечение, соответствующее спецификации *Intel HD Audio*. В них используется драйвер hdaudio(4) из NetBSD, и они требуют особого подхода.

10.6.1. Встроенные устройства и разъёмы: DAC и ADC

Для устройств hdaudio(4) текущий выбранный порт воспроизведения (например, встроенный динамик и разъём для наушников на ноутбуке) управляется выбором DAC или ADC. Доступные DAC и ADC можно увидеть в `/var/run/dmesg.boot`:

```
hdafg0 at hdaudio1: Realtek ALC292
hdafg0: DAC00 2ch: Speaker [Built-In], HP Out [Jack]
hdafg0: ADC01 2ch: Mic In [Jack]
hdafg0: ADC02 2ch: Mic In [Built-In]
```

Поэтому, чтобы использовать для записи только встроенный микрофон:

```
$ mixerctl -w record.source=ADC02
```

Использовать все доступные источники:

```
$ mixerctl -w record.source=ADC01,ADC02
```

На некоторых ноутбуках может потребоваться изменить `outputs.dacsel`, чтобы звук воспроизводился только через разъём для наушников; в других есть аппаратное отключение динамиков, и это не нужно.

Глава 11. Управление питанием

В области управления питанием NetBSD поддерживает мониторинг датчиков (включая состояние батареи, температуру процессора и т. д.), регулировку частоты процессора, режим пониженного энергопотребления для устройств, аппаратное выключение, а на некоторых аппаратных средствах — сон (suspend-to-RAM).

Управление питанием в NetBSD в первую очередь реализовано в виде поддержки acpi(4) (Advanced Configuration and Power Interface), хотя датчики поддерживаются и на многих других типах аппаратного обеспечения без ACPI.

11.1. Основные команды управления питанием

11.1.1. Выключение и перезагрузка системы

Систему NetBSD с поддержкой ACPI можно физически выключить, выполнив команды poweroff(8) и reboot(8), однако обычно лучше использовать shutdown(8), чтобы система выключалась с надлежащим предупреждением и успевала корректно остановить работающие приложения и службы.

Немедленно выключить систему:

```
# shutdown -p now
```

Перезагрузить с предупреждением пользователям за 10 минут:

```
# shutdown -r +10
```

11.1.2. Использование спящих состояний ACPI (приостановка и возобновление)

Система с ACPI всегда находится в одном из «спящих состояний»:

S0
: полностью работает

S1
: приостановка с питанием (процессор и жёсткие диски выключены)

S2
: аналогично S3, обычно не реализуется

S3
: приостановка в ОЗУ («сон»: большая часть системы неактивна для экономии энергии, но её можно быстро вернуть в рабочее состояние)

S4
: приостановка на диск («гибернация», в настоящее время в NetBSD не поддерживается)

S5

выключено

Спящее состояние можно изменить с помощью sysctl(8), например на приостановку в ОЗУ:

```
# sysctl -w hw.acpi.sleep.state=3
```

Способ пробуждения системы зависит от аппаратного обеспечения: это может быть нажатие кнопки питания или открытие крышки. Если это поддерживается, система может возобновить работу из приостановленного состояния гораздо быстрее, чем при полной перезагрузке.

Если вы проверили это и убедились, что всё работает как ожидается, возможно, вы захотите запускать это автоматически по событию powerd(8), например при закрытии крышки ноутбука.

11.1.3. Приостановка и возобновление отдельных устройств

Если ваша машина не поддерживает полную приостановку и возобновление ACPI, всё равно может быть возможно приостанавливать и возобновлять отдельные устройства, чтобы экономить энергию, пока они не используются. Это можно сделать с помощью drvctl(8).

Например, в `/var/run/dmesg.boot` сообщается, что в нашем оборудовании есть устройство чтения SD-карт `rtsx0`.

```
rtsx0 at pci1 dev 0 function 0: Realtek Semiconductor RTS5227 PCI-E Card Reader (rev. 0x01)
rtsx0: interrupting at msi4 vec 0
sdmmc0 at rtsx0
```

Мы можем приостановить его:

```
# drvctl -S rtsx0
```

А также можем возобновить его:

```
# drvctl -R rtsx0
```

Если какое-то устройство не приостанавливается или не возобновляется, это также можно использовать для отладки.

11.1.4. Регулировка частоты процессора на ходу

Многие современные машины позволяют динамически регулировать частоту процессора. Более высокая частота обеспечивает лучшую производительность, но увеличивает расход батареи и выделение тепла. В NetBSD частоту процессора можно регулировать на ходу с помощью sysctl(8).

Например, эта машина сейчас работает на частоте 1400 МГц, но также поддерживает режим пониженного энергопотребления 600 МГц:

```
$ sysctl -a | grep freq
machdep.cpu.frequency.target = 1400
machdep.cpu.frequency.current = 1400
machdep.cpu.frequency.min = 600
machdep.cpu.frequency.max = 1400
machdep.cpu.frequency.available = 600 1400
```

Мы можем перейти в режим пониженного энергопотребления, задав целевую частоту:

```
# sysctl -w machdep.cpu.frequency.target=600
```

Многие современные аппаратные средства поддерживают частоту «автоматической регулировки»; обычно это частота, которая заканчивается на 1. В системах без этой возможности из pkgsrc можно установить `sysutils/estd`, чтобы выполнять автоматическую регулировку в зависимости от нагрузки программно, хотя это будет менее эффективно, чем аппаратное масштабирование.

11.1.5. Использование режима энергосбережения IEEE 802.11 (Wi-Fi)

Некоторые сетевые устройства IEEE 802.11 (Wi-Fi) поддерживают режим пониженного энергопотребления, который можно включить с помощью ifconfig(8):

```
# ifconfig iwm0 powersave
```

Вы можете заметить рост сообщаемой задержки от ping(8) и снижение производительности. Однако это может продлить время работы устройства от батареи, поскольку радиомодули в таких устройствах могут потреблять много энергии. Режим можно снова отключить с помощью **ifconfig**:

```
# ifconfig iwm0 -powersave
```

11.2. Датчики и мониторинг

Основная программа командной строки для работы с системой мониторинга датчиков NetBSD — envstat(8). Вот типичный пример вывода **envstat**:

```
$ envstat
                      Current  CritMax  WarnMax  WarnMin  CritMin  Unit
[acpiacad0]
         connected:     FALSE
[acpibat0]
           present:      TRUE
    design voltage:    11.100                                      V
           voltage:    12.270                                      V
        design cap:    23.200                                     Wh
     last full cap:    16.940                                     Wh
            charge:    16.770                     5.000%   1.181%  Wh (99.00%)
       charge rate:       N/A
    discharge rate:       N/A
          charging:     FALSE
      charge state:    NORMAL
[acpitz0]
       temperature:    48.000  128.000
```

`acpiacad0` — это адаптер питания машины. Сейчас он не подключён.

`acpibat0` — это батарея машины, сейчас заряжена на 99%. При 5% на консоль будет выведено предупреждение и в powerd(8) отправлено событие. При 1% система выключится, чтобы предотвратить потерю данных из-за отключения питания.

Также обнаружен датчик температуры процессора `acpitz0`. Он показывает, что текущая температура процессора — 48 градусов Цельсия, а критическая температура — 128 градусов Цельсия. Если критическая температура будет достигнута, система выключится, чтобы предотвратить повреждение оборудования. Об изменениях температуры можно уведомлять powerd(8).

11.3. Введение в powerd

powerd(8) — это демон, который позволяет системе реагировать на события управления питанием, такие как отключение адаптера питания, изменение состояния батареи, закрытие крышки ноутбука или нажатие кнопки «сон».

Как и другие службы, powerd можно включить, отредактировав `/etc/rc.conf`:

```
      powerd=YES
```

И запустить с помощью service(8):

```
# service powerd start
```

powerd работает, выполняя именованный сценарий sh(1) из каталога `/etc/powerd/scripts` при каждом событии питания. В сценариях мы можем использовать в своих интересах команды, которые изучили в предыдущих разделах этой главы.

11.3.1. Пример: использование powerd для приостановки при закрытии крышки

Пример 11.1. /etc/powerd/scripts/lid_switch

```
#!/bin/sh -
#
# Generic script for lid switch events.
#
# Arguments passed by powerd(8):
#
#       device event

case "${2}" in
pressed)
        # Lock the X11 display to prevent tampering
        DISPLAY=:0 /usr/pkg/bin/xlock -mode blank &
        # Wait for 1 second
        sleep 1
        # Suspend
        /sbin/sysctl -w hw.acpi.sleep.state=3
        exit 0
        ;;

released)
        exit 0
        ;;

*)
        logger -p warning "${0}: unsupported event ${2} on device ${1}" >&1
        exit 1
esac
```

11.3.2. Пример: снижение частоты процессора при отключении питания

Пример 11.2. /etc/powerd/scripts/acadapter

```
#!/bin/sh -
#
# Generic script for acadapter events.
#
# Arguments passed by powerd(8):
#
#       device event

case "${2}" in
pressed)
        logger -p info "${0}: Full performance mode" >&1

        # Disable power saving mode on all network interfaces.
        for intf in $(/sbin/ifconfig -l); do
                /sbin/ifconfig $intf -powersave >/dev/null 2>&1
        done

        # Increase CPU frequency
        /sbin/sysctl -w machdep.cpu.frequency.target=2300

        exit 0
        ;;

released)
        logger -p info "${0}: Power saving mode" >&1

        # Enable power saving mode on all network interfaces.
        for intf in $(/sbin/ifconfig -l); do
                /sbin/ifconfig $intf powersave >/dev/null 2>&1
        done

        # Reduce CPU frequency
        /sbin/sysctl -w machdep.cpu.frequency.target=1400

        exit 0
        ;;

*)
        logger -p warning "${0}: unsupported event ${2} on device ${1}" >&1
        exit 1
        ;;
esac
```

Глава 12. Печать

В этой главе описана простая конфигурация для печати с использованием принтера HP Deskjet 690C, подключённого к первому параллельному порту, и системы печати lpd, входящей в состав NetBSD. Сначала система будет настроена на печать текстовых документов, а затем конфигурация будет расширена для печати документов PostScript с помощью программы Ghostscript (`print/ghostscript`). Обратите внимание, что из коллекции пакетов доступны и другие системы печати, например LPRng (`print/LPRng`) и Common Unix Printing System (CUPS) (`print/cups`), которые здесь не рассматриваются.

12.1. Включение демона печати

После установки печать пока невозможна, поскольку демон-диспетчер печати **lpd** не включён. Чтобы включить **lpd**, нужно изменить одну строку в файле `/etc/rc.conf` с:

```
lpd=NO
```

на

```
lpd=YES
```

Изменение вступит в силу при следующей загрузке, но демон можно запустить вручную уже сейчас:

```
# service lpd start
```

Чтобы проверить, активен ли **lpd**, введите следующую команду:

```
# service lpd status
```

Если в выводе предыдущей команды вы не видите записи для lpd, значит демон не активен.

Система lpd настраивается через `/etc/printcap`. Перед настройкой `/etc/printcap` полезно провести проверку принтера, чтобы убедиться, что физическое соединение между компьютером и принтером работает. Проверка отправляет некоторые данные непосредственно на устройство принтера. Если принтер подключён к параллельному порту, это `/dev/lpt0`; если вы используете USB-принтер, попробуйте `/dev/ulpt0`. Дополнительные сведения можно найти на страницах руководства этих устройств (lpt(4), ulpt(4))!

В нашем примере принтер подключён к параллельному порту, поэтому мы запускаем следующее:

```
# lptest 70 5 > /dev/lpt0
```

Чтобы увидеть, как должен выглядеть вывод, выполните ту же команду, не перенаправляя вывод на принтер:

```
# lptest 70 5
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdef
"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefg
#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefgh
$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghi
%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghij
```

Частая проблема — вывод на принтере выровнен по столбцам неправильно и образует «лесенку». Обычно это означает, что принтер настроен начинать новую строку от левого поля после получения и символа (возврат каретки, ASCII 13), и символа (перевод строки, ASCII 10). NetBSD передаёт только символ . Исправить эту проблему можно двумя способами:

- изменив конфигурацию принтера
- использовав простой фильтр принтера (описан ниже)

> **ПРИМЕЧАНИЕ**
>
> В предыдущем примере диспетчер lpd не участвует, поскольку вывод программы отправляется непосредственно на устройство принтера (`/dev/lpt0`) и не ставится в очередь.

12.2. Настройка `/etc/printcap`

В этом разделе объясняется, как настроить принтер из примера для печати текстовых документов.

Принтер должен иметь запись в файле `/etc/printcap`; запись содержит идентификатор принтера (имя принтера) и описание принтера. Идентификатор *lp* используется по умолчанию многими программами. Вот пример записи:

Пример 12.1. /etc/printcap

```
lp|local printer|HP DeskJet 690C:\
        :lp=/dev/lpa0:sd=/var/spool/lpd/lp:lf=/var/log/lpd-errs:\
        :sh:pl#66:pw#80:if=/usr/local/libexec/lpfilter:
```

Формат файла и параметры подробно описаны на странице руководства printcap(5). Обратите внимание, что указан *входной фильтр* (параметр *if*), который устраняет проблему «лесенки»:

```
if=/usr/local/libexec/lpfilter
```

> **ДРАЙВЕР ПРИНТЕРА И ПРИНТЕРЫ HP**
>
> Пример 12.1, «`/etc/printcap`» использует для принтера устройство *lpa0* (драйвер с опросом), а не *lpd0* (драйвер, управляемый прерываниями). При использовании прерываний у некоторых принтеров возникает проблема связи, и HP Deskjet 690C — один из них: печать идёт очень медленно, и на одну страницу PostScript могут уйти часы. Проблема решается использованием драйвера *lpa*. Можно также собрать собственное ядро, в котором lpt работает с опросом.

В записи printcap для принтера также указан каталог-хранилище, который нужно создать; этот каталог будет использоваться демоном lpd для накопления данных, подлежащих печати:

```
# cd /var/spool/lpd
# mkdir lp
# chown daemon:daemon lp
# chmod 770 lp
```

Остаётся только написать входной фильтр `lpfilter`. Единственная задача этого фильтра — настроить принтер на устранение проблемы «лесенки» перед отправкой текста на печать. Принтер, используемый в этом примере, требует следующей строки инициализации: «`<ESC>&k2G`».

Пример 12.2. /usr/local/libexec/lpfilter

```
#!/bin/sh
# Treat LF as CR+LF
printf "\033&k2G" && cat && exit 0
exit 2
```

Сохранив этот сценарий под именем, которое вы указали в `/etc/printcap`, убедитесь, что он исполняемый:

```
# chmod 755 /usr/local/libexec/lpfilter*
```

ПРИМЕЧАНИЕ

Есть ещё один фильтр, который можно использовать:

```
if=/usr/libexec/lpr/lpf:
```

Этот фильтр гораздо сложнее предыдущего. Он написан для обработки вывода **nroff** и поддерживает подчёркивание и надпечатку, раскрывает символы табуляции и преобразует LF в CR + LF. Исходный текст этой программы-фильтра находится в `/usr/src/usr.sbin/lpr/filters/lpf.c`.

Теперь, когда всё на месте, можно снова запустить команду **lptest**, на этот раз через команду **lpr**, которая сначала отправляет данные диспетчеру lpd, затем запускает фильтр и отправляет данные на принтер:

```
# lptest 70 5 | lpr -h
```

Программа **lpr** печатает текст, используя диспетчер для отправки данных на принтер; параметр `-h` отключает печать титульной страницы (на самом деле не обязательную из-за параметра *sh* в `/etc/printcap`). Пользователи, лучше знакомые с системой печати System V, могут также использовать команду lp(1), которая предлагается как альтернатива lpr(1).

12.3. Настройка Ghostscript

Теперь, когда базовая печать работает, можно добавить возможность печати файлов PostScript. Простой принтер, используемый в этом примере, не поддерживает собственную печать файлов PostScript; необходимо использовать программу, способную преобразовать документ PostScript в последовательность команд, понятных принтеру. Для этой цели подойдёт программа Ghostscript,

которую можно найти в коллекции пакетов. В этом разделе объясняется, как настроить lpd на использование Ghostscript для печати файлов PostScript на HP Deskjet 690C.

В `/etc/printcap` будет создан второй идентификатор принтера: этот новый идентификатор будет использовать другой входной фильтр, который вызовет Ghostscript для фактической печати документа PostScript. Таким образом, текстовые документы будут печататься на принтере *lp*, а документы PostScript — на принтере *ps*: обе записи используют один и тот же физический принтер, но разные фильтры печати.

Того же результата можно достичь с помощью других конфигураций. Например, можно было бы использовать единственную запись всего с одним фильтром. Для этого фильтр должен уметь автоматически определять формат печатаемого документа и применять подходящую программу печати. Такой подход проще, но ведёт к более сложному фильтру; если он вам по душе, подумайте об установке программы magicfilter из коллекции пакетов: она делает это и многое другое автоматически.

В нашем случае новый файл `/etc/printcap` выглядит так:

Пример 12.3. /etc/printcap

```
lp|local printer|HP DeskJet 690C:\
        :lp=/dev/lpa0:sd=/var/spool/lpd/lp:lf=/var/log/lpd-errs:\
        :sh:pl#66:pw#80:if=/usr/local/libexec/lpfilter:

ps|Ghostscript driver:\
        :lp=/dev/lpa0:sd=/var/spool/lpd/ps:lf=/var/log/lpd-errs:\
        :mx#0:sh:if=/usr/local/libexec/lpfilter-ps:
```

Параметр `mx#0` очень важен для печати файлов PostScript, так как он снимает ограничения на размер входного файла; документы PostScript обычно бывают очень большими. Параметр `if` указывает на новый фильтр. Появился также новый каталог-хранилище.

Следующие шаги — создание нового каталога-хранилища и программы-фильтра. Для каталога-хранилища процедура та же, что и выше:

```
# cd /var/spool/lpd
# mkdir ps
# chown daemon:daemon ps
# chmod 770 ps
```

Программа-фильтр для вывода PostScript сложнее, чем фильтр для текста: печатаемый файл передаётся интерпретатору, который преобразует его в последовательность команд на языке управления принтера, а затем отправляет результат на принтер. Благодаря операционной системе NetBSD и нескольким мощным свободно распространяемым пакетам нам удалось превратить недорогой цветной принтер в устройство, пригодное для вывода PostScript. Параметры, используемые для настройки Ghostscript, описаны в документации Ghostscript: `cdj550` — это устройство, применяемое для управления принтером HP.

Пример 12.4. /usr/local/libexec/lpfilter-ps

```
#!/bin/sh
# Treat LF as CR+LF
printf "\033&k2G" || exit 2
# Print the postscript file
/usr/pkg/bin/gs -dSAFER -dBATCH -dQUIET -dNOPAUSE -q -sDEVICE=cdj550 \
-sOutputFile=- -sPAPERSIZE=a4 - && exit 0
exit 2
```

Подведём итог: в системе созданы два разных имени принтера, которые указывают на один и тот же физический принтер, но используют разные параметры, разные фильтры и разные каталоги-хранилища. Можно печатать текстовые файлы и файлы PostScript. Для печати файлов PostScript в системе должен быть установлен пакет Ghostscript.

12.4. Команды управления принтером

В этом разделе перечислены некоторые полезные команды BSD для администрирования принтера и заданий печати. Помимо уже упомянутых команд **lpr** и **lpd**, имеются:

lpq
: просмотр очереди заданий принтера.

lprm
: удаление заданий из очереди принтера.

lpc
: проверка системы печати, включение и отключение принтеров и функций принтера.

12.5. Удалённая печать

Систему печати можно настроить так, чтобы печатать на принтере, подключённом к удалённому хосту. Допустим, например, что вы работаете на хосте *wotan* и хотите печатать на принтере, подключённом к хосту *loge*. Файл `/etc/printcap` хоста loge — это файл из примера 12.3, «`/etc/printcap`». С хоста wotan можно будет печатать файлы PostScript с помощью Ghostscript на хосте loge.

Первый шаг — разрешить приём заданий печати, отправляемых с хоста wotan на хост loge. Для этого в файл `/etc/hosts.lpd` на хосте loge нужно добавить строку с именем хоста wotan:

```
# hostname
loge
# cat /etc/hosts.lpd
wotan
```

Формат этого файла очень прост: каждая строка содержит имя хоста, которому разрешено печатать в локальной системе. По умолчанию демон lpd принимает только локальные подключения через доменные сокеты UNIX и не принимает никаких сетевых соединений. Чтобы демон принимал и входящий сетевой трафик, в `/etc/rc.conf` нужно добавить следующее:

```
lpd_flags=""
```

Затем на wotan необходимо настроить файл `/etc/printcap`, чтобы отправлять задания печати на loge. Например:

```
lp|line printer on loge:\
        :lp=:sd=/var/spool/lpd/lp:lf=/var/log/lp-errs:\
        :rm=loge:rp=lp

ps|Ghostscript driver on loge:\
        :lp=:sd=/var/spool/lpd/ps:lf=/var/log/lp-errs:\
        :mx#0:\
        :rm=loge:rp=ps
```

Между этой конфигурацией и конфигурацией из примера 12.3 «`/etc/printcap`» есть четыре основных отличия.

1. Определение «lp» пусто.
2. Параметр «rm» (remote machine) задаёт имя хоста, к которому подключён принтер.
3. Параметр «rp» (remote printer) задаёт имя принтера, подключённого к удалённому хосту.
4. Указывать входные фильтры не нужно, так как будут использоваться определения с хоста loge.
5. Каталоги спулинга всё равно необходимо создать локально на wotan:

```
# cd /var/spool/lpd
# mkdir lp
# chown daemon:daemon lp
# chmod 770 lp
# mkdir ps
# chown daemon:daemon ps
# chmod 770 ps
```

Теперь задания печати для очередей «lp» и «ps» на wotan будут автоматически отправляться на принтер, подключённый к loge.

Глава 13. Использование сменных носителей

13.1. Инициализация и использование USB-флешек

USB-флешки можно применять для обмена данными между машинами. После подключения флешки по выводу dmesg(8) видно, что она распознана как `sd0`:

```
# dmesg
[...]
sd0 at scsibus0 target 0 lun 0: <Kingston, DataTraveler 3.0, > disk removable
sd0: fabricating a geometry
sd0: 14755 MB, 14755 cyl, 64 head, 32 sec, 512 bytes/sect x 30218842 sectors
sd0: fabricating a geometry
[...]
```

> **ПРЕДУПРЕЖДЕНИЕ**
>
> Обратите внимание: приведённые ниже команды уничтожат всё содержимое USB-флешки!

Чтобы инициализировать её, можно записать нули в первые 1 МБ USB-флешки:

```
# dd if=/dev/zero of=/dev/rsd0d bs=1m count=1
1+0 records in
1+0 records out
1048576 bytes transferred in 0.118 secs (8886237 bytes/sec)
```

С помощью fdisk(8) можно создать таблицу разделов. Раздел и файловая система MS-DOS поддерживаются большинством операционных систем и устройств, принимающих USB-диск, поэтому обновим таблицу разделов (`-u`), создав раздел MS-DOS, и сделаем новый раздел активным (`-a`):

```
# fdisk -au sd0
fdisk: primary partition table invalid, no magic in sector 0
fdisk: Cannot determine the number of heads
Disk: /dev/rsd0d
NetBSD disklabel disk geometry:
cylinders: 14755, heads: 64, sectors/track: 32 (2048 sectors/cylinder)
total sectors: 30218842, bytes/sector: 512

BIOS disk geometry:
cylinders: 1023, heads: 255, sectors/track: 63 (16065 sectors/cylinder)
total sectors: 30218842

Partitions aligned to 16065 sector boundaries, offset 63

Do you want to change our idea of what BIOS thinks? [n]

Partition table:
0: <UNUSED>
1: <UNUSED>
2: <UNUSED>
3: <UNUSED>
```

```
Bootselector disabled.
No active partition.
Drive serial number: 0 (0x00000000)
Which partition do you want to change?: [none] 0
The data for partition 0 is:
<UNUSED>
sysid: [0..255 default: 169] 11
start: [0..1881cyl default: 63, 0cyl, 0MB]
size: [0..1881cyl default: 30218779, 1881cyl, 14755MB]
bootmenu: [] (space to clear)

Partition table:
0: Primary DOS with 32 bit FAT (sysid 11)
    start 63, size 30218779 (14755 MB, Cyls 0-1881/9/10)
        PBR is not bootable: All bytes are identical (0x00)
1: <UNUSED>
2: <UNUSED>
3: <UNUSED>
Bootselector disabled.
No active partition.
Drive serial number: 0 (0x00000000)
Which partition do you want to change?: [none]
Do you want to change the active partition? [n] y
Choosing 4 will make no partition active.
active partition: [0..4 default: 4] 0
Are you happy with this choice? [n] y

We haven't written the MBR back to disk yet.  This is your last chance.
Partition table:
0: Primary DOS with 32 bit FAT (sysid 11)
    start 63, size 30218779 (14755 MB, Cyls 0-1881/9/10), Active
        PBR is not bootable: All bytes are identical (0x00)
1: <UNUSED>
2: <UNUSED>
3: <UNUSED>
Bootselector disabled.
First active partition: 0
Drive serial number: 0 (0x00000000)
Should we write new partition table? [n] y
```

Затем с помощью disklabel(8) можно увидеть:

```
# disklabel sd0
# /dev/rsd0d:
type: SCSI
disk: DataTraveler 3.0
label: fictitious
flags: removable
bytes/sector: 512
sectors/track: 32
tracks/cylinder: 64
sectors/cylinder: 2048
cylinders: 14755
total sectors: 30218842
rpm: 3600
interleave: 1
trackskew: 0
cylinderskew: 0
headswitch: 0           # microseconds
track-to-track seek: 0  # microseconds
drivedata: 0

5 partitions:
```

```
#        size    offset     fstype [fsize bsize cpg/sgs]
 d:  30218842         0     unused      0     0        # (Cyl.      0 -  14755*)
 e:  30218779        63      MSDOS                     # (Cyl.      0*-  14755*)
disklabel: boot block size 0
disklabel: super block size 0
```

что присутствует раздел MSDOS `sd0e`.

Наконец, можно создать файловую систему MS-DOS с помощью newfs_msdos(8):

```
# newfs_msdos /dev/rsd0e
/dev/rsd0e: 30189264 sectors in 1886829 FAT32 clusters (8192 bytes/cluster)
MBR type: 11
bps=512 spc=16 res=32 nft=2 mid=0xf0 spt=32 hds=64 hid=0 bsec=30218779 bspf=14741 rdcl=2
infs=1 bkbs=2
```

Теперь её можно использовать и монтировать с помощью mount_msdos(8).

13.2. Инициализация и использование дискет

Дискеты формата PC в основном работают так же, как другие дисковые устройства, например жёсткие диски, только сначала их нужно отформатировать на низком уровне. Чтобы использовать обычную дискету ёмкостью 1440 КБ в первом дисководе, сначала отформатируйте её (от имени root):

```
# fdformat -f /dev/rfd0a
```

Затем создайте на диске единственный раздел с помощью disklabel(8):

```
# disklabel -rw /dev/rfd0a floppy3
```

Создание небольшой файловой системы, оптимизированной по занимаемому месту:

```
# newfs -m 0 -o space -i 16384 -c 80 /dev/rfd0a
```

Теперь дискету можно смонтировать, как любой другой диск. Если же у вас уже есть дискета с файловой системой MS-DOS, к которой нужно просто получить доступ из NetBSD, достаточно выполнить примерно следующее:

```
# mount -t msdos /dev/fd0a /mnt
```

Однако вместо того чтобы обращаться с дискетами как с обычными (более крупными) дисками, часто удобнее вовсе обойтись без файловой системы и просто записать архив файлов напрямую в необработанное устройство. Например:

```
# tar cvfz /dev/rfd0a file1 file2 ...
```

Вариант этого можно проделать и с дискетами MS-DOS, используя пакет `sysutils/mtools`, который удобен тем, что не проходит через буферный кэш ядра и, следовательно, не подвержен опасности извлечения дискеты, пока на ней смонтирована файловая система.

13.3. Как использовать диск ZIP

1. Проверьте, есть ли в системе дисковод ZIP:

```
# dmesg | grep -i zip
sd0 at atapibus0 drive 1: <IOMEGA  ZIP 100       ATAPI, , 14.A> type 0 direct removable
```

Похоже, он есть и распознан как sd0, как и любой SCSI-диск. То, что ZIP здесь ATAPI, значения не имеет — SCSI ZIP тоже отобразится здесь. ZIP помечен как «removable» (сменный), а значит, его можно извлечь командой:

```
# eject sd0
```

2. Вставьте диск ZIP
3. Посмотрите, какие разделы есть на ZIP:

```
# disklabel sd0
# /dev/rsd0d:
type: ATAPI
 ...
8 partitions:
#        size   offset     fstype   [fsize bsize   cpg]
  d:   196608        0     unused        0     0         # (Cyl.    0 - 95)
  h:   196576       32      MSDOS                       # (Cyl.    0*- 95)
disklabel: boot block size 0
disklabel: super block size 0
```

Раздел d

— это весь диск, как обычно на i386.

Раздел h

— это то, что вам нужно, и, как видите, это даже файловая система msdos.

Следовательно, для доступа к разделу zip используйте /dev/sd0h.

4. Смонтируйте его:

```
# mount -t msdos /dev/sd0h /mnt
```

5. Получите доступ к своим файлам:

```
# ls -la /mnt
total 40809
drwxr-xr-x   1 root  wheel     16384 Dec 31  1979 .
drwxr-xr-x  28 root  wheel      1024 Aug  2 22:06 ..
-rwxr-xr-x   1 root  wheel   1474560 Feb 23  1999 boot1.fs
-rwxr-xr-x   1 root  wheel   1474560 Feb 23  1999 boot2.fs
-rwxr-xr-x   1 root  wheel    548864 Feb 23  1999 boot3.fs
-rwxr-xr-x   1 root  wheel  38271173 Feb 23  1999 netbsd19990223.tar.gz
```

6. Размонтируйте ZIP:

```
# umount /mnt
#
```

7. Извлеките ZIP:

```
# eject sd0
#
```

13.4. Чтение компакт-дисков с данными в NetBSD

Компакт-диски с данными могут содержать что угодно — от программ, звуковых файлов (MP3, wav), фильмов (MP3, QuickTime) до исходного кода, текстовых файлов и т. д. Прежде чем обращаться к этим файлам, компакт-диск нужно смонтировать на каталог, во многом так же, как это делается с жёсткими дисками. Как жёсткие диски могут использовать различные файловые системы (ffs, lfs, ext2fs, …), так и у компакт-дисков есть собственная файловая система — «cd9660». Файловая система cd9660 в NetBSD поддерживает файловые системы как без расширений Rockridge и Joliet, так и с ними.

Устройства компакт-дисков называются /dev/cd0a как для SCSI, так и для IDE (ATAPI).

С учётом этого можно начинать:

1. Проверьте, есть ли в системе дисковод компакт-дисков:

```
# dmesg | grep 'cd[0-9]*:'
     cd0 at atapibus0 drive 0: <CD-R/RW RW8040A, , 1.12> type 5 cdrom removable
     cd0: 32-bit data port
     cd0: drive supports PIO mode 4, DMA mode 0
```

   Здесь у нас один дисковод — «cd0». Это дисковод IDE/ATAPI, поскольку он обнаружен на atapibus0. Разумеется, дисковод (точнее, его носитель) является сменным, то есть его можно извлечь. См. ниже.

2. Вставьте компакт-диск

3. Смонтируйте компакт-диск вручную:

```
# mount -t cd9660 /dev/cd0a /mnt
#
```

   Эта команда ничего не должна выводить. Она даёт системе указание смонтировать компакт-диск с /dev/cd0a в /mnt, используя файловую систему «cd9660». Точка монтирования «/mnt» должна быть существующим каталогом.

4. Проверьте содержимое компакт-диска:

```
# ls /mnt
INSTALL.html INSTALL.ps   TRANS.TBL    boot.catalog
INSTALL.more INSTALL.txt  binary       installation
#
```

   Всё выглядит хорошо! Разумеется, это компакт-диск NetBSD. :)

5. Размонтируйте компакт-диск:

```
# umount /mnt
#
```

Если к компакт-диску всё ещё идёт обращение (например, какая-то другая оболочка всё ещё находится в нём после «cd»), это не сработает. При выключении системы компакт-диск будет размонтирован автоматически, так что беспокоиться не о чем.

6. Создание записи в /etc/fstab:

   Если вы не хотите каждый раз набирать полную команду «mount», можно поместить большинство значений в строку файла /etc/fstab:

```
# Device        mountpoint      filesystem  mount options
/dev/cd0a       /cdrom          cd9660      ro,noauto
```

   Убедитесь, что точка монтирования, в нашем примере `/cdrom`, существует:

```
# mkdir /cdrom
#
```

   Теперь можно смонтировать компакт-диск следующей командой:

```
# mount /cdrom
#
```

   Обращайтесь к нему и размонтируйте его, как раньше.

   Компакт-диск не монтируется при загрузке из-за опции монтирования «noauto» — это удобно, поскольку диск в дисководе, скорее всего, будет не всегда. О других полезных опциях см. mount(8) и mount_cd9660(8).

7. Извлеките компакт-диск:

```
# eject cd0
#
```

   Если компакт-диск всё ещё смонтирован, перед извлечением он будет размонтирован, если это возможно.

13.5. Чтение компакт-дисков с несколькими сеансами в NetBSD

Используйте mscdlabel(8), чтобы добавить все сеансы в disklabel компакт-диска, а затем смонтируйте нужный сеанс через соответствующий узел устройства. Возможно, соответствующие узлы устройств в `/dev` придётся создать вручную. Например:

```
# mscdlabel cd1
track (ctl=4) at sector 142312
 adding as 'a'
track (ctl=4) at sector 0
 adding as 'b'
# ls -l /dev/cd1b
ls: /dev/cd1b: No such file or directory
# cd /dev
# ls -l cd1*
brw-r-----  1 root  operator       6,  8 Mar 18 21:55 cd1a
brw-r-----  1 root  operator       6, 11 Mar 18 21:55 cd1d
# mknod cd1b b 6 9
```

чтобы создать `/dev/cd1b`. Обязательно исправьте права доступа для всех новых узлов устройств, которые вы создадите:

```
# ls -l cd1*
brw-r-----  1 root  operator       6,  8 Mar 18 21:55 cd1a
brw-r--r--  1 root  wheel          6,  9 Mar 18 22:23 cd1b
brw-r-----  1 root  operator       6, 11 Mar 18 21:55 cd1d
# chgrp operator cd1b
# chmod 640 cd1b
# ls -l cd1*
brw-r-----  1 root  operator       6,  8 Mar 18 21:55 cd1a
brw-r-----  1 root  operator       6,  9 Mar 18 22:24 cd1b
brw-r-----  1 root  operator       6, 11 Mar 18 21:55 cd1d
```

Теперь вы сможете смонтировать его.

```
# mount /dev/cd1b /mnt
```

13.6. Предоставление обычным пользователям доступа к компакт-дискам

По умолчанию NetBSD разрешает монтировать файловую систему только пользователю «root». Если вы хотите, чтобы это мог делать любой пользователь, выполните следующие действия:

- Предоставьте группе и остальным права доступа к устройству.

  ```
  # chmod go+rw /dev/cd0a
  ```

- Укажите NetBSD, что пользователям разрешено монтировать файловые системы.

  ```
  # sysctl -w vfs.generic.usermount=1
  ```

  Учтите, что это работает для любой файловой системы и любого устройства, а не только для компакт-дисков с файловой системой ISO 9660.

Чтобы после этих команд пользователь мог выполнить монтирование, он должен быть владельцем точки монтирования. Например:

```
$ cd $HOME
$ mkdir cdrom
$ mount -t cd9660 -o nodev,nosuid /dev/cd0a `pwd`/cdrom
```

См. также mount(8), а в качестве альтернативы — *демон автоматического монтирования* amd(8); примеры файлов конфигурации для него можно найти в `/usr/share/examples/amd`.

13.7. Монтирование образа ISO

Иногда бывает полезно смонтировать файл образа ISO9660 ещё до записи компакт-диска: так вы можете просмотреть его содержимое или даже скопировать файлы наружу. Если вы пользователь Linux, то знаете, что для этого используется специальная файловая система *loop*. В NetBSD это делается иначе — с помощью псевдодиска *vnode*.

Покажем, как это сделать, на примере. Предположим, в вашем домашнем каталоге есть образ ISO с именем «mycd.iso»:

1. Сначала настройте новый vnode, «указывающий» на файл ISO:

```
# vnconfig -c vnd0 ~/mycd.iso
```

2. Теперь смонтируйте vnode:

```
# mount -t cd9660 /dev/vnd0a /mnt
```

3. Отлично, содержимое образа появилось в `/mnt`! Перейдите в этот каталог и изучите образ.
4. Когда закончите, размонтируйте образ:

```
# umount /mnt
```

5. И наконец, отмените настройку vnode:

```
# vnconfig -u vnd0
```

Учтите, что эти действия подходят для любого файла, содержащего файловую систему, а не только для образов ISO.

Дополнительные сведения см. на страницах руководства vnd(4) и vnconfig(8).

13.8. Использование видео-CD в NetBSD

Чтобы воспроизводить потоки MPEG Video так, как это умеют многие DVD-проигрыватели в NetBSD, смонтируйте компакт-диск так же, как обычный (с данными) CD (см. Раздел 13.4, «Чтение компакт-дисков с данными в NetBSD»), а затем воспользуйтесь пакетом `multimedia/xine-ui`, `multimedia/mplayer` или `multimedia/gmplayer`, чтобы воспроизвести mpeg-файлы, хранящиеся на диске.

13.9. Использование аудио-CD в NetBSD

Существует два способа работы с аудио-CD:

1. Заставьте привод CD воспроизводить звук на наушники или на звуковую карту, к которой приводы CDROM обычно подключены внутри. Используйте такие программы, как cdplay(1), `audio/xmcd`, «kscd» из пакета `multimedia/kdemultimedia3`, микшерные программы вроде mixerctl(1), `audio/xmix`, `audio/xmmix`, программу `audio/cam` на основе Curses или kmix, входящий в `multimedia/kdemultimedia3`.

   Обычно это хорошо работает как на приводах CDROM с интерфейсами SCSI и IDE (ATAPI), так и на приводах CDRW и DVD.
2. Считывать («извлекать») звуковые дорожки в двоичном виде, не проходя преобразование из цифрового сигнала в аналоговый и обратно. Для этого есть несколько программ:
   - Для большинства приводов ATAPI, SCSI и ряда фирменных приводов CDROM можно использовать пакет `audio/cdparanoia`. С помощью cdparanoia данные можно сохранить в файл или направить в стандартный вывод в формате WAV, AIFF, AIFF-C или raw. В настоящее

время версия cdparanoia для NetBSD требует ключ -g. Гипотетический пример сохранения дорожки 2 в файл WAV выглядит так:

```
$ cdparanoia -g /dev/rcd0d 2 track-02.wav
```

Если вы хотите извлечь все файлы с компакт-диска, пригодится пакетный режим cdparanoia:

```
$ cdparanoia -g /dev/rcd0d -B
```

- Для приводов CD-ROM с интерфейсом ATAPI или SCSI можно использовать пакет `audio/cdd`. Чтобы извлечь дорожку 2 с помощью cdd, введите:

```
# cdd -t 2 `pwd`
```

В текущем каталоге появится файл `track-02.cda`.

- Для приводов CD-ROM с интерфейсом SCSI можно использовать пакет `audio/tosha`. Чтобы извлечь дорожку 2 с помощью tosha, введите:

```
# tosha -d CD-ROM-device -t 2 -o track-02.cda
```

Затем данные можно обработать, например закодировать их в потоки MP3 (см. Раздел 13.10, «Создание файла MP3 (MPEG layer 3) с аудио-CD») или записать их на CD-R (см. Раздел 13.12, «Использование привода CD-R для создания аудио-CD»).

- Чтобы упростить весь процесс — от получения звука до заполнения метаданных дорожки и нормализации звука, — можно воспользоваться пакетом `audio/abcde`.

```
# abcde -d /dev/rcd0d -o mp3 -p -P
```

При этом диск кодируется дорожка за дорожкой, номера дорожек дополняются ведущим 0, а чтение и кодирование выполняются через каналы UNIX без сохранения файлов WAV.

13.10. Создание файла MP3 (MPEG layer 3) с аудио-CD

Основные шаги создания файла MPEG layer 3 (MP3) с аудио-CD (с использованием программ из коллекции пакетов NetBSD) таковы:

1. Извлеките (*rip*) звуковые данные с компакт-диска, как показано в Раздел 13.9, «Использование аудио-CD в NetBSD».
2. Преобразуйте файл в формате аудио-CD в формат WAV. Это нужно делать, только если ваша программа извлечения (например, tosha, cdd) не сделала этого за вас!

   - Используя пакет `audio/sox`, введите:

```
$ sox -s -w -c 2 -r 44100 -t cdr track-02.cda track-02.wav
```

Это преобразует `track-02.cda` в формате raw CD в `track-02.wav` в формате WAV со следующими параметрами: **s**igned (знаковые) 16-битные **w**ords (слова) с 2 **c**hannels (каналами) при **r**ate (частоте дискретизации) 44100 кГц.

3. Закодируйте файл WAV в формат MP3.

   - Используя пакет `audio/bladeenc`, введите:

     ```
     $ bladeenc -128 -QUIT track-02.wav
     ```

     Это закодирует `track-02.wav` в `track-02.mp3` в формате MP3 с битрейтом **128** кбит/с. В документации к bladeenc битрейты описаны подробнее.

   - Используя пакет `audio/lame`, введите:

     ```
     $ lame -p -o -v -V 5 -h track-02.wav track-02.mp3
     ```

     В зависимости от ваших предпочтений и оборудования вы можете выбрать качество пониже.

Полученный файл MP3 можно воспроизвести с помощью любого из пакетов `audio/gqmpeg`, `audio/maplay`, `audio/mpg123` или `audio/splay`.

13.11. Использование привода CD-R для компакт-дисков с данными

Процесс записи компакт-диска состоит из двух шагов: сначала нужно создать «образ» данных, который затем на втором шаге записывается на CD-R.

1. Чтение уже существующего образа ISO

   ```
   # dd if=/dev/rcd0a of=filename.iso bs=2k
   #
   ```

   Либо вы можете создать новый образ ISO самостоятельно:

2. Создание образа ISO

   Поместите все данные, которые хотите записать на компакт-диск, в один каталог. Затем нужно создать дисковый образ ISO ваших данных. Образ хранит данные в том же виде, в каком они позже попадут на компакт-диск, — в формате ISO 9660. Базовый формат ISO9660 понимает только имена файлов 8+3 (максимум восемь букв на имя файла плюс три на расширение). Поскольку это неудобно для имён файлов Unix, чтобы получить более длинные имена, нужно применить так называемое «расширение Rockridge». (В мире Microsoft существует другой набор подобных расширений, позволяющий обойтись с длинными именами файлов; он известен как файловая система Joliet.)

   Образ ISO создаётся командой mkisofs, входящей в пакет `sysutils/cdrtools`.

   Пример: если ваши данные находятся в /usr/tmp/data, создать файл образа ISO /usr/tmp/data.iso можно следующей командой:

   ```
   $ cd /usr/tmp
   $ mkisofs -o data.iso -r data
   Using NETBS000.GZ;1 for  data/binary/kernel/netbsd.INSTALL.gz (netbsd.INSTALL_TINY.gz)
   Using NETBS001.GZ;1 for  data/binary/kernel/netbsd.GENERIC.gz (netbsd.GENERIC_TINY.gz)
     5.92% done, estimate finish Wed Sep 13 21:28:11 2000
    11.83% done, estimate finish Wed Sep 13 21:28:03 2000
    17.74% done, estimate finish Wed Sep 13 21:28:00 2000
    23.64% done, estimate finish Wed Sep 13 21:28:03 2000
    ...
   ```

```
 88.64% done, estimate finish Wed Sep 13 21:27:55 2000
 94.53% done, estimate finish Wed Sep 13 21:27:55 2000
Total translation table size: 0
Total rockridge attributes bytes: 5395
Total directory bytes: 16384
Path table size(bytes): 110
Max brk space used 153c4
84625 extents written (165 Mb)
$
```

Другие параметры, например указание издателя и того, кто подготовил диск, описаны на странице руководства mkisofs(8). В руководстве по созданию загрузочного CD-ROM рассказано, как создать загрузочный компакт-диск.

3. Запись образа ISO на CD-R

   Когда файл образа ISO готов, его нужно просто записать на компакт-диск. Это делается командой «cdrecord» из пакета `sysutils/cdrtools`. Вставьте чистый CD-R — и вперёд:

```
# cdrecord -v dev=/dev/rcd0d data.iso
...
#
```

   После запуска команды «cdrecord» выводит много сведений о вашем приводе, диске и образе, который собирается записать. Затем он отсчитывает 10 секунд — это ваша последняя возможность остановить процесс: нажмите ^C, если хотите прервать запись. Если вы не прервёте её, программа запишет весь образ на компакт-диск и вернёт приглашение оболочки.

   Учтите, что cdrecord(8) работает как с приводами SCSI, так и с IDE (ATAPI).

4. Проверка

   Смонтируйте только что записанный компакт-диск и проверьте его так же, как любой «обычный» CD, см. Раздел 13.4, «Чтение компакт-дисков с данными в NetBSD».

13.12. Использование привода CD-R для создания аудио-CD

Если вы хотите сделать резервную копию одного из своих аудио-CD, извлеките («снимите») звуковые дорожки с диска, а затем запишите их обратно на чистый компакт-диск. Разумеется, это отлично работает и тогда, когда вы извлекаете отдельные дорожки с разных дисков, создавая собственный сборник!

Для этого нужно выполнить следующие шаги:

1. Извлеките («снимите») звуковые дорожки, как описано в Раздел 13.9, «Использование аудио-CD в NetBSD», чтобы получить несколько файлов .wav.
2. Запишите файлы .wav командой cdrecord из пакета `sysutils/cdrtools`:

```
# cdrecord -v dev=/dev/rcd0d -audio -pad *.wav
```

13.13. Создание аудио-CD из файлов MP3

Если вы перевели все свои аудио-CD в MP3 и теперь хотите сделать сборник, например для автомобиля, сначала преобразуйте файлы .mp3 обратно в формат .wav, а затем запишите их как обычный аудио-CD.

Здесь нужно выполнить следующие шаги:

1. Создайте файлы .wav из своих файлов .mp3:

```
$ mpg123 -w foo.wav foo.mp3
```

   Проделайте это для всех файлов MP3, которые хотите включить в свой аудио-CD. Имена файлов .wav значения не имеют.

2. Запишите файлы .wav на компакт-диск, как описано в Раздел 13.12, «Использование привода CD-R для создания аудио-CD».

13.14. Копирование аудио-CD

Чтобы скопировать аудио-CD, не внося пауз, предписанных стандартом CDDA, можно воспользоваться cdrdao:

```
# cdrdao read-cd --device /dev/rcd0d data.toc
# cdrdao write   --device /dev/rcd1d data.toc
```

13.15. Копирование компакт-диска с данными с помощью двух приводов

Если в вашей машине есть и привод CD-R, и привод CD-ROM, скопировать компакт-диск с данными можно следующей командой:

```
# cdrecord dev=/dev/rcd1d /dev/rcd0d
```

Здесь CD-ROM (cd0) содержит копируемый диск, а в CD-R (cd1) вставлен чистый диск. Учтите, что это работает только с компьютерными дисками, содержащими какие-либо данные; с аудио-CD это *не* работает! На практике также стоит добавить что-то вроде «`speed=8`», чтобы немного ускорить процесс.

13.16. Использование перезаписываемых CD-RW

В NetBSD с приводом CD-RW можно обращаться так же, как с приводом CD-R (см. Раздел 13.11, «Использование привода CD-R для компакт-дисков с данными»): создавать образы с помощью mkisofs(8) и записывать их на носитель CD-RW с помощью cdrecord(8).

Если вы хотите стереть CD-RW, это можно сделать с помощью ключа «`blank`» программы cdrecord:

```
# cdrecord dev=/dev/rcd0d blank=fast
```

Есть и другие способы стереть CD-RW; чтобы получить их список, вызовите cdrecord(8) с параметром «`blank=help`». Дополнительные сведения см. на странице руководства cdrecord(8).

13.17. Поддержка DVD

В настоящее время NetBSD поддерживает носители DVD с файловыми системами ISO 9660 и UDF. Сведения о монтировании файловых систем ISO 9660 и UDF можно найти на страницах руководства

mount_cd9660(8) и mount_udf(8) соответственно. DVD, DivX и многие файлы avi можно воспроизводить с помощью `multimedia/ogle` или `multimedia/gmplayer`.

Несколько советов по созданию DVD вы найдёте в сообщениях о growisofs и в этой статье о записи компакт-дисков и DVD в NetBSD.

13.18. Создание образов ISO с компакт-диска

Чтобы создать образ ISO и сохранить контрольную сумму, выполните следующее:

```
# readcd dev=/dev/cd0d f=/tmp/cd.iso
```

Вот альтернативный вариант с использованием dd(1):

```
# dd if=/dev/cd0d of=/tmp/cd.iso bs=2048
```

Если на компакт-диске есть ошибки, остальное можно восстановить так:

```
# dd if=/dev/cd0d of=/tmp/cd.iso bs=2048 conv=noerror
```

Чтобы создать образ ISO с уже смонтированного компакт-диска с данными, сначала смонтируйте диск так:

```
# mount -t cd9660 -r /dev/cd0d /mnt/cdrom
```

Затем получите образ:

```
# mkhybrid -v -l -J -R -o /tmp/my_cd.iso /mnt/cdrom/
```

13.19. Получение сведений о томе с компакт-дисков и из образов ISO

Прочитать данные о томе с несмонтированного компакт-диска можно следующей командой:

```
# file -s /dev/cd0d
```

Прочитать данные о томе из образа ISO можно следующей командой:

```
# isoinfo -d -i /tmp/my_cd.iso
```

Уникальный номер диска с несмонтированного компакт-диска можно получить так:

```
# cd-discid /dev/cd0d
```

Прочитать оглавление несмонтированного компакт-диска можно с помощью этой команды:

```
# cdrecord -v dev=/dev/cd0d -toc
```

Глава 14. Драйвер криптографического устройства (CGD)

Драйвер `cgd` предоставляет функциональность, позволяющую использовать диски или разделы для зашифрованного хранения данных. После ввода соответствующего ключа зашифрованный раздел становится доступен через псевдоустройства `cgd`.

14.1. Обзор

Люди часто хранят конфиденциальную информацию на своих жёстких дисках и беспокоятся о том, что она может попасть в чужие руки. Это особенно актуально для пользователей ноутбуков и других портативных устройств или съёмных носителей, которые могут быть украдены или случайно потеряны.

14.1.1. Зачем нужно шифрование диска?

Инструменты шифрования, ориентированные на отдельные файлы, такие как GnuPG, отлично подходят для шифрования отдельных файлов, которые затем можно передавать по недоверенным сетям и хранить в зашифрованном виде на диске. Но иногда они неудобны, поскольку файл приходится расшифровывать при каждом использовании; это особенно обременительно, когда нужно защитить большое количество файлов. Всякий раз, когда инструмент безопасности неудобен в использовании, есть вероятность, что вы забудете применить его правильно и оставите файлы незащищёнными ради удобства.

Хуже того, на жёстком диске могут по-прежнему оставаться читаемые копии зашифрованного содержимого. Даже если вы перезапишете эти файлы (с помощью **rm -P**) перед тем, как удалить их, ваше прикладное программное обеспечение может создавать временные копии, о которых вы не знаете, или данные могли быть выгружены в область подкачки — и даже ваш жёсткий диск мог незаметно переназначить сбойные секторы, в которых всё ещё остались данные.

Решение простое: никогда не записывать информацию на жёсткий диск в незашифрованном виде. Вместо того чтобы применять ориентированный на файлы подход к шифрованию, рассмотрите ориентированный на блоки подход — виртуальный жёсткий диск, который выглядит как обычный жёсткий диск с обычными файловыми системами, но шифрует и расшифровывает каждый блок по пути к реальному диску и обратно.

14.1.2. Драйверы логических дисков

Устройство `cgd` выглядит и ведёт себя для остальной части операционной системы как любой другой драйвер диска. Вместо прямого управления реальным оборудованием оно реализует логическую функцию, надстроенную над другим блочным устройством. Для него есть специальная программа конфигурации, **cgdconfig**, которая создаёт и настраивает устройство `cgd` и указывает ему на нижележащее дисковое устройство, где будут храниться зашифрованные данные.

NetBSD включает несколько других подобных логических блочных устройств, каждое из которых выполняет какую-то иную функцию там, где `cgd` обеспечивает шифрование. Несколько таких логических блочных устройств можно объединить в стек: `cgd` поверх `vnd` удобно, чтобы создать

зашифрованный том в обычном файле без повторного разбиения диска, а можно создать зашифрованный `raid`, чтобы дополнительно защитить зашифрованные данные от отказа жёсткого диска.

Создав диск `cgd`, вы можете с помощью **disklabel** разбить его на разделы, с помощью **swapctl** включить подкачку на этих разделах или с помощью **newfs** создать файловые системы, а затем с помощью **mount** смонтировать и использовать эти файловые системы, как любой другой новый диск.

14.1.3. Доступность

Драйвер `cgd` написал Roland C. Dowdeswell; он появился в выпуске NetBSD 2.0.

14.2. Компоненты системы Crypto-Graphic Disk

Ряд компонентов и инструментов совместно обеспечивают работоспособность системы `cgd`.

14.2.1. Псевдоустройство драйвера ядра

Чтобы использовать `cgd`, нужно ядро с поддержкой псевдоустройства `cgd`. Убедитесь, что модуль загружен:

```
# modload cgd
```

Если драйвер `cgd` ещё не присутствует или не загружен (в некоторых портах он загружается по умолчанию), добавьте `cgd` в `/etc/modules.conf`.

14.2.2. Шифры

Поддерживаются следующие шифры:

`adiantum` (размер ключа: 256 бит)

Настраиваемый широкоблочный шифр Adiantum. В качестве настройки (tweak) Adiantum для каждого сектора диска используется представление номера сектора диска в порядке little-endian.

Adiantum обеспечивает наилучшую защиту, шифруя целые секторы диска за раз (512 байт), и, как правило, даёт наилучшую производительность на машинах без поддержки аппаратного ускорения AES.

`aes-cbc` (размеры ключа: 128, 192 или 256 бит)

AES в режиме CBC. Вектор инициализации CBC для каждого сектора диска выбирается как результат шифрования в AES представления номера сектора диска в порядке little-endian. Длина ключа по умолчанию — 128 бит.

`aes-xts` (размеры ключа: 256 или 512 бит)

AES в режиме XTS. В качестве настройки XTS для каждого сектора диска используется представление номера сектора диска в порядке little-endian. AES-XTS использует 256-битный или 512-битный ключ, состоящий из пары ключей AES-128 или AES-256. Длина ключа по умолчанию — 256, что означает AES-128.

14.2.3. Устаревшие шифры

Следующие устаревшие шифры поддерживаются для совместимости со старыми дисками.

ВНИМАНИЕ: Эти устаревшие шифры реализованы без защиты от атак по побочным каналам на основе времени, поэтому, например, код JavaScript в веб-браузере, умеющий измерять время дисковых операций, может восстановить секретный ключ. Кроме того, они основаны на 64-битных блочных шифрах и поэтому небезопасны для дисков, размер которых значительно превышает гигабайт. Не используйте их, кроме случаев, когда необходима совместимость со старыми дисками.

`3des-cbc` (размер ключа: 192 бита)

3DES (Triple DES с EDE3) в режиме CBC. Вектор инициализации CBC для каждого сектора диска выбирается как результат шифрования в 3DES представления номера сектора диска в порядке little-endian, что не влияет на безопасность, но снижает производительность.

Примечание: внутри «„биты чётности“» 192-битного ключа игнорируются, поэтому ключевого материала всего 168 бит, а из-за общих атак на 64-битные блочные шифры и атак типа «встреча посередине» на композиции шифров, как в EDE3, стойкость оказывается намного ниже, чем можно ожидать даже для 168-битного ключа.

`blowfish-cbc` (размеры ключа: 40, 48, 56, 64, …, 432, 440 или 448 бит)

Blowfish в режиме CBC. Вектор инициализации CBC для каждого сектора диска выбирается как результат шифрования в Blowfish представления номера сектора диска в порядке little-endian. Настоятельно рекомендуется использовать ключи длиной не менее 128 бит. Применение более коротких ключей не даёт преимуществ в производительности. Длина ключа по умолчанию — 128 бит.

14.2.4. Методы проверки

cgdconfig может проверить диск, чтобы убедиться, что он был расшифрован правильным ключом. Доступны следующие методы проверки:

`none`

Проверка не выполняется. Это опасно, если вы не настраиваете новое устройство `cgd` в первый раз, поскольку ключ вообще не проверяется. Например, ввод неправильной парольной фразы может уничтожить все данные на томе: любые прочитанные данные окажутся мусором, а любые записанные данные превратятся в мусор, если вы когда-нибудь снова откроете том `cgd` с правильной парольной фразой.

`disklabel`

cgdconfig ведёт поиск допустимого disklabel BSD; см. disklabel(5) и disklabel(8).

`mbr`

cgdconfig ведёт поиск допустимой главной загрузочной записи (MBR), традиционно используемой на ПК; см. fdisk(8).

`gpt`

cgdconfig ведёт поиск допустимой таблицы разделов GUID; см. gpt(8).

`ffs`

cgdconfig ведёт поиск допустимой файловой системы FFS — файловой системы, используемой в NetBSD по умолчанию; см. mount_ffs(8).

`re-enter`

Вместо сканирования чего-либо на диске **cgdconfig** вычислит ключ дважды — например, попросив пользователя ввести парольную фразу два раза, — и завершится с ошибкой, если результаты различаются.

14.3. Пример: шифрование диска

В этом разделе пошагово рассматривается пример перевода существующей системы на `cgd` со следующими действиями:

1. Подготовка диска и разделов
2. Затирание всех данных
3. Создание cgd
4. Настройка файлов конфигурации
5. Восстановление резервных копий файлов на зашифрованный диск

14.3.1. Подготовка диска

Сначала решите, какие файловые системы вы хотите перенести на зашифрованное устройство. Как минимум небольшую корневую файловую систему (`/`) придётся оставить незашифрованной, чтобы загрузить ядро и запустить **init**, **cgdconfig** и сценарии **rc.d**, которые настраивают ваш `cgd`. В этом примере мы зашифруем всё, кроме корневой файловой системы (`/`).

Мы собираемся удалить и создать заново разделы и файловые системы, поэтому для восстановления данных потребуется резервная копия. Убедитесь, что у вас есть актуальная надёжная резервная копия на другом диске или машине. Создавайте её в однопользовательском режиме при отмонтированных файловых системах, чтобы получить чистый **dump**. Не забудьте также сохранить резервную копию disklabel жёсткого диска, чтобы у вас была запись о разметке разделов до начала работы.

Когда система работает в однопользовательском режиме, `/` смонтирована для чтения и записи, а всё остальное отмонтировано, удалите с помощью **disklabel** все разделы с данными, которые вы хотите перенести в `cgd`.

Затем создайте один новый раздел во всём только что освобождённом пространстве, скажем, *`wd0e`*. Установите для этого раздела тип `cgd`. Хотя это не имеет особого значения, позже это поможет вам вспомнить, что перед вами не обычная файловая система. По завершении запишите метку диска, чтобы сохранить новую таблицу разделов.

14.3.2. Затирание диска

Мы удалили информацию о таблице разделов, но существующие файловые системы и данные по-прежнему находятся на диске. Даже после того, как мы создадим устройство `cgd`, создадим файловые системы и восстановим данные, некоторые из этих блоков диска могут ещё не быть перезаписаны и по-прежнему содержать наши данные в открытом виде. Это особенно вероятно, если файловые системы в основном пусты. Прежде чем двигаться дальше, мы хотим затереть диск.

Можно было бы с помощью **dd** скопировать `/dev/zero` в новый раздел *`wd0e`*, но тогда диск будет заполнен нулями везде, кроме мест, куда позже будут записаны зашифрованные данные. Возможно, мы не хотим давать злоумышленнику подсказок о том, какие блоки содержат реальные данные, а какие — свободное пространство, поэтому мы хотим записать «шум» во все блоки диска. Для этого мы создадим временный `cgd`, настроенный на случайный неизвестный ключ.

Сначала настроим `cgd` на использование случайного ключа:

```
# cgdconfig -s cgd0 /dev/wd0e aes-xts 256 < /dev/urandom
```

Теперь можно записать нули в «сырой» раздел нашего `cgd` (`/dev/rcgd0d` в NetBSD/i386 и amd64, `/dev/rcgd0c` на большинстве других платформ):

```
# dd if=/dev/zero of=/dev/rcgd0d bs=64k
```

Зашифрованные нули будут выглядеть на диске как случайные данные. Если диск большой, это может занять некоторое время. По завершении удалите конфигурацию `cgd` со случайным ключом:

```
# cgdconfig -u cgd0
```

14.3.3. Создание `cgd`

Программа **cgdconfig**, которая управляет устройствами `cgd`, использует файлы параметров для хранения такой информации, как тип шифрования, длина ключа и случайная соль пароля для каждого `cgd`. Эти файлы очень важны, и их нужно хранить в безопасности — без них вы не сможете расшифровать данные!

Создадим файл параметров и запишем его в место по умолчанию (убедитесь, что каталог `/etc/cgd` существует и имеет права доступа 700):

```
# cgdconfig -g -V disklabel -o /etc/cgd/wd0e aes-cbc 256
```

В результате создаётся файл параметров `/etc/cgd/wd0e`, описывающий `cgd` с методом шифрования `aes-cbc`, методом проверки ключа `disklabel` и длиной ключа `256` бит. Выглядеть он будет примерно так:

```
algorithm aes-cbc;
iv-method encblkno;
keylength 256;
verify_method disklabel;
keygen pkcs5_pbkdf2/sha1 {
        iterations 6275;
        salt AAAAgHTg/jKCd2ZJiOSGrgnadGw=;
};
```

> **ПРИМЕЧАНИЕ**
>
> Считайте этот файл *СВЯЩЕННЫМ, СОЗДАЙТЕ ЕГО РЕЗЕРВНУЮ КОПИЮ* и *СОЗДАЙТЕ РЕЗЕРВНУЮ КОПИЮ ЕЩЁ РАЗ!*

> **СОВЕТ**
>
> При создании файла параметров **cgdconfig** читает из `/dev/random`, чтобы создать соль пароля. Это чтение может заблокироваться, если в пуле случайных чисел накоплено недостаточно энтропии. Это маловероятно, особенно если вы только что перезаписали диск, как на предыдущем шаге, но если такое произошло, нажимайте клавиши на консоли и/или двигайте мышь, пока устройство `rnd` не соберёт достаточно энтропии.

Теперь пора создать наш `cgd`, для чего потребуется парольная фраза. Эту фразу нужно вводить каждый раз при открытии `cgd`, обычно при каждой перезагрузке. Ключ шифрования выводится из этой фразы и соли. Выберите такую фразу, которую вы не забудете и которую не угадают другие.

При первой настройке `cgd` на логическом устройстве нет допустимого disklabel, поэтому механизм проверки, который мы хотим использовать позже, не сработает. На этот раз мы его обходим:

```
# cgdconfig -V re-enter cgd0 /dev/wd0e
```

Парольная фраза будет запрошена дважды для проверки совпадения — на случай опечатки, иначе вы получили бы `cgd`, зашифрованный не той фразой, которую ожидали.

Теперь, когда у нас есть новый `cgd`, нужно разбить его на разделы и создать файловые системы. Создайте заново прежние разделы с теми же размерами и теми же буквенными обозначениями.

> **СОВЕТ**
>
> Не забудьте использовать аргумент **disklabel -I**, поскольку вы создаёте начальную метку для нового диска.

> **ПРИМЕЧАНИЕ**
>
> Хотя размеры новых разделов должны совпадать с размерами старых незашифрованных, смещения будут другими, поскольку они отсчитываются от начала этого виртуального диска.

Затем с помощью **newfs** создайте файловые системы на всех нужных разделах. На этот раз имена разделов будут соответствовать именам диска `cgd`, например:

```
# newfs /dev/rcgd0h
```

14.3.4. Изменение файлов конфигурации

Мы перенесли несколько файловых систем на другой (логический) диск, поэтому нужно соответствующим образом обновить `/etc/fstab`. Каждый раздел сохранит ту же букву (в этом примере), но будет находиться на `cgd0`, а не на `wd0`. Таким образом, записи в `/etc/fstab` будут примерно такими:

```
/dev/wd0a   /     ffs     rw    1 1
/dev/cgd0b  none  swap    sw            0 0
/dev/cgd0b  /tmp  mfs     rw,-s=132m    0 0
/dev/cgd0e  /var  ffs     rw            1 2
/dev/cgd0f  /usr  ffs     rw            1 2
/dev/cgd0h  /home ffs     rw            1 2
```

> **ПРИМЕЧАНИЕ**
>
> `/tmp` должна быть отдельной файловой системой — `mfs` или `ffs` — внутри `cgd`, чтобы ваши временные файлы не хранились в открытом виде в файловой системе `/`.

При каждой перезагрузке ваш `cgd` должен быть настроен заранее, до запуска **fsck** и монтирования файловых систем.

Добавьте следующую строку в `/etc/cgd/cgd.conf`:

```
cgd0    /dev/wd0e
```

При этом файл `/etc/cgd/wd0e` будет использоваться как файл конфигурации для `cgd0`.

Чтобы окончательно включить cgd при каждой загрузке, добавьте следующую строку в `/etc/rc.conf`:

```
cgd=YES
```

Теперь парольная фраза для `/dev/cgd0` должна запрашиваться при каждом запуске `/etc/rc`.

14.3.5. Восстановление данных

Затем смонтируйте новые файловые системы с помощью **mount** и восстановите в них данные с помощью **restore**. Часто полезно сначала правильно смонтировать `/tmp`, так как **restore** может использовать довольно много временного пространства при извлечении большого файла дампа.

Чтобы проверить изменения в конфигурации загрузки, отмонтируйте файловые системы с помощью **umount** и удалите конфигурацию `cgd`, чтобы при выходе из однопользовательской оболочки **rc** запустился как при чистой загрузке, запросил парольную фразу и правильно смонтировал ваши файловые системы. Теперь можно перевести систему в многопользовательский режим и убедиться, что всё работает как раньше.

14.4. Пример: зашифрованные компакт-диски и DVD

14.4.1. Создание зашифрованного компакт-диска или DVD

cgd(4) обеспечивает высоконадёжное шифрование целых разделов или дисков. К сожалению, создание «обычных» компакт-дисков — это не запись disklabel и не запуск newfs по нему. Нельзя также просто вставить CD-R в привод, настроить cgd и рассчитывать, что при синхронизации будут записаны зашифрованные данные. Стандартные компакт-диски содержат как минимум файловую систему ISO-9660, созданную с помощью mkisofs(8) из пакета `sysutils/cdrtools`. Образы ISO *не* могут содержать disklabel или разделы cgd.

Но, конечно, аппаратура чтения и записи CD вовсе не заботится о файловых системах. При желании вы можете записать на CD необработанные данные — или зашифрованную файловую систему FFS, что мы и сделаем здесь. Но предупреждаем: прочитать этот CD не сможет ни одна ОС, кроме NetBSD, — даже другие BSD, из-за отсутствия cgd.

Основные шаги при создании зашифрованного CD:

- Создать (пустой) файл образа
- Зарегистрировать его как виртуальный диск с помощью vnd(4)
- Настроить cgd внутри диска vnd
- Скопировать содержимое в cgd
- Снять всю конфигурацию (сбросить буферы!)
- Записать образ на CD

Первый шаг при создании зашифрованного CD — создать с помощью dd единый файл образа. Образ не может увеличиваться, поэтому сделайте его достаточно большим, чтобы в него поместилось всё содержимое CD. Учтите, что позже весь образ целиком записывается на CD, поэтому для содержимого в 100 МБ всё равно потребуется создать образ размером 700 МБ и выполнить запись на CD объёмом 700 МБ. Несколько сведений о DVD: в маркетинговых материалах DVD имеют объём 4,7 ГБ. 4,7 ГБ = 4,7 x 1024 x 1024 x 1024 = 5046586573 байт. На самом деле DVD вмещает лишь приблизительно 4,7 x 1000 x 1000 x 1000 = 4700000000 байт, то есть около 4482 МБ или примерно 4,37 ГБ. Учитывайте это при создании образов DVD. О CD не беспокойтесь: они вмещают «настоящие» 700 МБ (734003200 байт).

Все следующие команды выполняйте от имени root!

Для CD:

```
# dd if=/dev/zero of=image.img bs=1m count=700
```

или для DVD:

```
# dd if=/dev/zero of=image.img bs=1m count=4482
```

Теперь настройте псевдодиск vnd(4) с этим образом:

```
# vnconfig vnd0 image.img
```

Чтобы использовать cgd, необходимо сгенерировать так называемый файл параметров, описывающий параметры шифрования и содержащий «соль» пароля. Здесь мы назовём его `/etc/cgd/image`. Один файл параметров можно использовать для нескольких зашифрованных разделов (я использую отдельный файл для каждого хоста и общий файл `image` для всех сменных носителей, но решать вам).

В этом примере будет использоваться AES-CBC с длиной ключа 256 бит. Дополнительные сведения и описание других шифров см. в cgd(4) и cgdconfig(8).

Следующая команда создаст файл параметров `/etc/cgd/image`. *НЕ ЗАПУСКАЙТЕ СЛЕДУЮЩУЮ КОМАНДУ ПОВТОРНО* после того, как запишете какой-либо CD: заново созданный файл параметров — это потерянный файл параметров, и вы больше никогда не получите доступ к своему зашифрованному CD (содержащаяся в этом файле «соль» будет отличаться при каждом вызове). Считайте этот файл *СВЯЩЕННЫМ, СДЕЛАЙТЕ ЕГО РЕЗЕРВНУЮ КОПИЮ* и *СДЕЛАЙТЕ РЕЗЕРВНУЮ КОПИЮ ЕЩЁ РАЗ!* Используйте ключ -V, чтобы указать для CD метод проверки «disklabel» (иначе при монтировании cgd не сможет определить, ввели ли вы правильный пароль для CD).

```
# cgdconfig -g -V disklabel aes-cbc 256 > /etc/cgd/image
```

Теперь пора настроить cgd для нашего диска vnd. (На всех платформах, где «c» обозначает весь диск, замените раздел «d» на «c» (там, где «**sysctl kern.rawpartition**» выдаёт «2», а не «3»); если вы работаете на i386 или amd64, «d» вам подходит):

```
# cgdconfig -V re-enter cgd1 /dev/vnd0d /etc/cgd/image
```

Параметр «`-V re-enter`» необходим, пока у cgd ещё нет disklabel, чтобы мы могли получить к нему доступ и настроить его. Этот ключ запрашивает пароль дважды и использует его для шифрования.

Теперь пора создать disklabel внутри cgd. Значения метки по умолчанию подходят, поэтому достаточно вызвать disklabel с

```
# disklabel -e -I cgd1
```

и сразу выйти из vi командой «**:wq**».

Создадим файловую систему на cgd и, наконец, смонтируем её куда-нибудь:

```
# newfs /dev/rcgd1a
# mount /dev/cgd1a /mnt
```

cgd работает! Теперь наполните `/mnt` содержимым. Когда закончите, выполните настройку в обратном порядке. Шаги таковы:

1. Размонтирование cgd1a:

```
# umount /mnt
```

2. Снятие конфигурации cgd:

```
# cgdconfig -u cgd1
```

3. Снятие конфигурации vnd:

```
# vnconfig -u vnd0
```

Следующие команды — примеры записи образов на CD или DVD. Скорректируйте *`dev=`* для cdrecord или `/dev/rcd0d` для growisofs. Обратите внимание, что в NetBSD «*`r`*`cd0d`» *необходим*. Growisofs доступен в пакете `sysutils/dvd+rw-tools`. И ещё раз: используйте «`c`» вместо «`d`», если на вашей платформе это необработанный раздел.

Наконец, запишите файл образа на CD:

```
# cdrecord dev=/dev/rcd0d -v image.img
```

...или на DVD:

```
# growisofs -dvd-compat -Z /dev/rcd0d=image.img
```

Поздравляем! Вы только что создали действительно защищённый CD!

14.4.2. Использование зашифрованного CD/DVD

После создания зашифрованного CD, как описано выше, работа ещё не закончена — а как его снова смонтировать? Можно было бы предположить, что достаточно настроить cgd на `/dev/cd0d`, — но нет, это не так.

NetBSD не может обращаться к файловым системам FFS на носителях, формат которых отличается от 512 байт на сектор. То, что cgd на CD соответствует этому требованию, не помогает, поскольку disklabel CD, в котором находится cgd, имеет размер 2048 байт на сектор.

Но драйвер CD cd(4) достаточно умён, чтобы предоставить доступ «на запись» к (эмулируемому) disklabel на CD. Поэтому перед настройкой cgd давайте посмотрим на disklabel и немного его изменим:

```
# disklabel -e cd0
# /dev/rcd0d:
type: ATAPI
disk: mydisc
label: fictitious
flags: removable
bytes/sector: 2048    # -- Change to 512 (= orig / 4)
sectors/track: 100    # -- Change to 400 (= orig * 4)
tracks/cylinder: 1
sectors/cylinder: 100 # -- Change to 400 (= orig * 4)
cylinders: 164
```

```
total sectors: 16386  # -- Change to value of slice "d" (=65544)
rpm: 300
interleave: 1
trackskew: 0
cylinderskew: 0
headswitch: 0           # microseconds
track-to-track seek: 0  # microseconds
drivedata: 0

4 partitions:
#     size  offset  fstype [fsize bsize cpg/sgs]
 a:   65544   0     4.2BSD  0     0     0  # (Cyl. 0 - 655+)
 d:   65544   0     ISO9660 0     0        # (Cyl. 0 - 655+)
```

Если вы не хотите каждый раз вносить эти изменения вручную, можете воспользоваться инструментом Florian Stoehr **neb-cd512**, который (на момент написания этих строк) находится в pkgsrc-wip и вскоре переедет в `sysutils/neb-cd512`. Исходные тексты neb-cd512 также можно загрузить с http://sourceforge.net/projects/neb-stoehr/ (обязательно используйте neb-cd512, а не neb-wipe!).

Он вызывается от имени root, а в качестве параметра ему передаётся имя диска:

```
# neb-cd512 cd0
```

Теперь, когда disklabel имеет формат 512 б/с, обращаться к CD так же просто:

```
# cgdconfig cgd1 /dev/cd0d /etc/cgd/image
# mount -o ro /dev/cgd1a /mnt
```

Учтите, что cgd *ОБЯЗАТЕЛЬНО* нужно монтировать только для чтения, иначе драйвер cd(4) будет выдавать ошибки недопустимой команды, что в некоторых случаях делает невозможным даже монтирование cgd на CD!

Готово! Наслаждайтесь своим защищённым CD!

```
# ls /mnt
```

Помните, что для извлечения CD необходимо выполнить все шаги в обратном порядке:

```
# umount /mnt
# cgdconfig -u cgd1
# eject cd0
```

14.5. Пример: зашифрованные устройства iSCSI

14.5.1. Создание зашифрованного устройства iSCSI

Чтобы зашифровать устройство iSCSI, мы используем инициатор iSCSI из NetBSD, доступный в NetBSD-6 и новее, и стандартное устройство cgd. В целом настройка зашифрованного устройства таким способом должна занять менее 15 минут, даже если вы не знакомы с iSCSI или cgd.

Подход состоит в том, чтобы разместить vnd поверх файла «хранилища», предоставляемого целью iSCSI. Это в точности то же самое, что и обычно. Поверх этого vnd мы размещаем устройство cgd, которое обеспечивает шифрование всех данных на устройстве iSCSI.

ПРЕДУПРЕЖДЕНИЕ: `cgd` сохраняет в тайне только содержимое тома — он не скрывает шаблоны доступа и не предотвращает и даже не обнаруживает вредоносное вмешательство сети или цели iSCSI в содержимое тома.

14.5.2. Инициализация устройства

Сначала запускается инициатор, указывающий на машину, которая предоставляет хранилище iSCSI (то есть на машину, на которой работает цель iSCSI). В этом примере цель работает на той же машине, что и инициатор (ноутбук, названный в минуту вдохновения inspiron1300). Цель iSCSI объёмом 50 МБ предоставляется как target1.

```
# iscsi-initiator -u agc -h inspiron1300.wherever.co.uk /mnt &
[1] 11196
```

```
# df
Filesystem   1K-blocks       Used      Avail %Cap Mounted on
/dev/dk0      28101396   20862004    5834324  78% /
kernfs               1          1          0 100% /kern
procfs               4          4          0 100% /proc
ptyfs                1          1          0 100% /dev/pts
/dev/puffs           0          0          0 100% /mnt
```

Посмотрев на последнюю строку, можно увидеть, что инициатор работает через устройство puffs. Устройство vnd создаётся поверх хранилища, которое предоставляет цель:

```
# vnconfig vnd0 /mnt/inspiron1300.wherever.co.uk/target1/storage
```

Необходимо добавить disklabel со смещением 63 блока в устройстве iSCSI. Это делается для того, чтобы зашифрованное устройство, которое мы разместим поверх vnd, не конфликтовало с меткой vnd. Тип cgd следует установить в «cgd».

```
# disklabel -e vnd0
# /dev/rvnd0d:
type: vnd
disk: vnd
label: fictitious
flags:
bytes/sector: 512
sectors/track: 32
tracks/cylinder: 64
sectors/cylinder: 2048
cylinders: 50
total sectors: 102400
rpm: 3600
interleave: 1
trackskew: 0
cylinderskew: 0
headswitch: 0           # microseconds
track-to-track seek: 0  # microseconds
drivedata: 0

4 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
 a:    102336        63        cgd   2048 16384 28360  # (Cyl.      0 -     49)
 d:    102400         0     unused      0     0        # (Cyl.      0 -     49)
```

Теперь на устройстве vnd можно создать устройство cgd

```
# cgdconfig -s cgd0 /dev/vnd0a aes-xts 256 < /dev/urandom
```

а хранилище устройства cgd — обнулить

```
# dd if=/dev/zero of=/dev/rcgd0d bs=32k
dd: /dev/rcgd0d: Invalid argument
1601+0 records in
1600+0 records out
52428800 bytes transferred in 16.633 secs (3152095 bytes/sec)
```

Снимите конфигурацию устройства cgd и запишите на cgd disklabel с использованием метода проверки. Примечание: иногда этот процесс завершается не полностью, и его приходится повторять.

```
# cgdconfig -g -V disklabel -o /etc/cgd/vnd0a aes-cbc 256
cgdconfig: could not calibrate pkcs5_pbkdf2
cgdconfig: Failed to generate defaults for keygen
# cgdconfig -g -V disklabel -o /etc/cgd/vnd0a aes-cbc 256
```

Затем устройству cgd можно добавить пароль

```
# cgdconfig -V re-enter cgd0 /dev/vnd0a
/dev/vnd0a's passphrase:
re-enter device's passphrase:
```

Затем создайте disklabel внутри самого cgd

```
# disklabel -I -e cgd0

# /dev/rcgd0d:
type: cgd
disk: cgd
label: fictitious
flags:
bytes/sector: 512
sectors/track: 2048
tracks/cylinder: 1
sectors/cylinder: 2048
cylinders: 49
total sectors: 102336
rpm: 3600
interleave: 1
trackskew: 0
cylinderskew: 0
headswitch: 0           # microseconds
track-to-track seek: 0  # microseconds
drivedata: 0

4 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
 a:    102336         0     4.2BSD   2048 16384 28360  # (Cyl.      0 -     49*)
 d:    102336         0     unused      0     0        # (Cyl.      0 -     49*)
```

Разместив disklabel внутри cgd, теперь можно создать там файловую систему:

```
# newfs /dev/rcgd0a
/dev/rcgd0a: 50.0MB (102336 sectors) block size 8192, fragment size 1024
using 4 cylinder groups of 12.49MB, 1599 blks, 3136 inodes.
super-block backups (for fsck_ffs -b #) at:
32, 25616, 51200, 76784,
```

теперь новую файловую систему в cgd можно смонтировать

```
# df
Filesystem   1K-blocks       Used      Avail %Cap Mounted on
/dev/dk0      28101396   20910216    5786112  78% /
kernfs               1          1          0 100% /kern
procfs               4          4          0 100% /proc
ptyfs                1          1          0 100% /dev/pts
/dev/puffs           0          0          0 100% /mnt
# mount /dev/cgd0a /iscsi
# df
Filesystem   1K-blocks       Used      Avail %Cap Mounted on
/dev/dk0      28101396   20910216    5786112  78% /
kernfs               1          1          0 100% /kern
procfs               4          4          0 100% /proc
ptyfs                1          1          0 100% /dev/pts
/dev/puffs           0          0          0 100% /mnt
/dev/cgd0a       49519          1      47043   0% /iscsi
```

Новую файловую систему, смонтированную в /iscsi, теперь можно использовать как обычно.

14.5.3. Размонтирование зашифрованного устройства

Устройство можно освободить с помощью следующих команд

```
# umount /iscsi
# cgdconfig -u cgd0
# vnconfig -u vnd0
```

14.5.4. Обычное использование

При обычном использовании устройство можно смонтировать. Сначала необходимо настроить инициатор на подключение к устройству:

```
# vnconfig vnd0 /mnt/inspiron1300.wherever.co.uk/target1/storage
# cgdconfig cgd0 /dev/vnd0a
/dev/vnd0a's passphrase:
# mount /dev/cgd0a /iscsi
# ls -al /iscsi
total 3
drwxr-xr-x   2 root  wheel   512 Jan  1  1970 .
drwxr-xr-x  35 root  wheel  1536 Jan  5 08:59 ..
# df
Filesystem   1K-blocks       Used      Avail %Cap Mounted on
/dev/dk0      28101396   20910100    5786228  78% /
kernfs               1          1          0 100% /kern
procfs               4          4          0 100% /proc
ptyfs                1          1          0 100% /dev/pts
/dev/puffs           0          0          0 100% /mnt
/dev/cgd0a       49519          1      47043   0% /iscsi
```

14.6. Рекомендации и предупреждения

Теперь ваши файловые системы зашифрованы внутри `cgd`. Когда машина выключена, данные защищены и не могут быть расшифрованы без парольной фразы. Однако есть ещё несколько опасностей, о которых следует знать, и ещё кое-что, что можно сделать с помощью `cgd`. В этом разделе приведены дополнительные рекомендации и предупреждения, которые помогут эффективно использовать `cgd`.

- Используйте несколько `cgd` для разных видов данных: один смонтирован постоянно, а остальные — только когда это нужно.
- Используйте `cgd`, настроенный поверх `vnd`, созданного из файла на удалённом сетевом файловом сервере (NFS, SMBFS, CODA и т. п.), чтобы безопасно хранить частные данные в общей системе. Это похоже на процедуру использования зашифрованных CD и DVD, описанную в разделе 14.4, «Пример: зашифрованные CD/DVD».

14.6.1. Использование cgd со случайным ключом для подкачки

Следующий раздел в NetBSD 10 будет заменён параметром sysctl «vm.swap_encrypt=1», который обеспечивает лучшую безопасность и более простую настройку.

Возможно, вы захотите использовать для пространства подкачки отдельный `cgd` со случайным ключом, заново генерируя ключ при каждой перезагрузке. Преимущество этого в том, что после перезагрузки машины любое чувствительное содержимое памяти программ, которое могло быть выгружено, становится безвозвратно недоступным, поскольку ключ расшифровки вам никогда не известен.

В примере выше мы создали временный `cgd` со случайным ключом при затирании диска, используя сокращённый вызов **cgdconfig -s**, чтобы избежать создания файла параметров.

Файл параметров **cgdconfig** включает метод генерации ключа «randomkey». Он лучше подходит для «постоянных» конфигураций со случайным ключом и упрощает автоматическую настройку таких томов при загрузке.

Например, если вы хотите преобразовать существующий раздел `/dev/wd0b` в отдельный cgd1 со случайным ключом, используйте следующую команду для создания `/etc/cgd/wd0b`:

```
# cgdconfig -g -o /etc/cgd/wd0b -V none -k randomkey blowfish-cbc
```

При использовании метода генерации ключа randomkey можно применять только метод проверки «none», поскольку содержимое нового `cgd` каждый раз фактически случайно (предыдущие данные расшифровываются случайным ключом). Аналогично новый диск не будет иметь корректной метки или разделов, и **swapctl** будет ругаться на настройку устройств подкачки, не помеченных как таковые в disklabel.

Чтобы автоматизировать процесс разметки диска, подготовьте подходящий disklabel и сохраните его в файл, например `/etc/cgd/wd0b.disklabel`. Обратитесь к disklabel(8) за сведениями о том, как использовать **disklabel** для создания раздела подкачки.

При каждой перезагрузке, чтобы восстановить сохранённую метку на новом `cgd`, создайте файл `/etc/rc.conf.d/cgd` как показано ниже:

```
swap_device="cgd1"
swap_disklabel="/etc/cgd/wd0b.disklabel"
start_postcmd="cgd_swap"

cgd_swap()
{
```

```
        if [ -f $swap_disklabel ]; then
                disklabel -R -r $swap_device $swap_disklabel
        fi
}
```

Ту же методику можно расширить, чтобы с помощью **newfs** заново создавать файловую систему `ffs` для `/tmp`, если вы не хотите использовать `mfs`.

14.6.2. Предупреждения

Избегайте потери данных: убедитесь, что вы всегда можете восстановить свою парольную фразу и файл параметров. Защищайте файл параметров от раскрытия, например храня его на сменных носителях, как описано выше, поскольку содержащаяся в нём соль помогает защититься от словарных атак на парольную фразу.

Хорошо, что данные на диске зашифрованы, но как быть с другими копиями? У вас уже есть как минимум одна такая копия (резервная копия, которую мы использовали при этой настройке), и она не зашифрована. Один из способов решения этой проблемы — направлять вывод **dump** через инструменты шифрования файлов, такие как **gpg**, но убедитесь, что у вас есть все ключи и инструменты, необходимые для расшифровки и **restore** после аварии.

Как и при любом программном шифровании, ключ `cgd` остаётся в памяти ядра, пока устройство настроено, и может быть доступен привилегированным программам и пользователям — например, тем, кто копается в `/dev/kmem`. Настоятельно рекомендуется предпринимать и другие меры безопасности системы, например работать с повышенным securelevel.

Как только тома `cgd` смонтированы как обычные файловые системы, их содержимое доступно, как любой другой файл. Следите за правами доступа к файлам и обеспечьте защиту работающей системы от атак на уровне приложений и сети.

Избегайте использования приостановки и возобновления, особенно на ноутбуках с функцией BIOS «suspend-to-disk». Если злоумышленник сможет возобновить работу вашего ноутбука, пока ключ ещё находится в памяти, или позже прочитать его из образа памяти suspend-to-disk на жёстком диске, весь смысл использования `cgd` теряется.

14.7. Дополнительная литература

Следующие ресурсы содержат дополнительную информацию о CGD и лежащей в его основе криптографии:

Библиография

[nycbug-cgd] *I want my cgd он же: I want an encrypted pseudo-device on my laptop*.

[elric-cgd] Roland Dowdeswell и John Ioannidis. «The CryptoGraphic Disk Driver». *Proceedings of the FREENIX Track: 2003 USENIX Annual Technical Conference*. USENIX Association. 179-186. 9–14 июня 2003 г.

[hubertf-cgd] Feyrer Hubert. *CryptoGraphicFile (CGF), или как хранить конфиденциальные данные на ноутбуке*.

[adiantum] Paul Crowley и Eric Biggers. «Adiantum: шифрование с сохранением длины для процессоров начального уровня». *Transactions on Symmetric Cryptology*. 2018. 4. International Association for Cryptologic Research. 39-61.

[fips46-3] *FIPS PUB 46-3: Data Encryption Standard (DES)*. National Institute of Standards and Technology. United States Department of Commerce. 25 октября 1999 г., отозван 19 мая 2005 г.

[fips197] *FIPS PUB 197: Advanced Encryption Standard (DES)*. National Institute of Standards and Technology. United States Department of Commerce. Ноябрь 2001 г.

[nist-sp800-38a] Morris Dworkin. *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*. NIST Special Publication 800-38A. National Institute of Standards and Technology. United States Department of Commerce. Декабрь 2001 г.

[nist-sp800-38e] Morris Dworkin. *Recommendation for Block Cipher Modes of Operation: the XTS-AES Mode for Confidentiality on Storage Devices*. NIST Special Publication 800-38E. National Institute of Standards and Technology. United States Department of Commerce. Январь 2010 г.

[blowfish] Bruce Schneier. *Алгоритм шифрования Blowfish*. вытеснен алгоритмом Twofish, затем Threefish. 1993 г.

[sweet32] Karthikeyan Bhargavan и Gaétan Leurent. *Sweet32: атаки «дня рождения» на 64-битные блочные шифры в TLS и OpenVPN*.

Глава 15. Настройка объединённого дискового устройства (CCD)

Драйвер CCD позволяет пользователю «объединять» несколько физических дисков в один псевдотом. RAIDframe (см. главу 16, *NetBSD RAIDframe*) тоже позволяет делать это, создавая наборы уровня RAID 0, но он не даёт выполнять распределение полос по дискам с разной геометрией — именно здесь CCD и пригодится. CCD также позволяет задать «чередование» для повышения производительности дисков за счёт потери ёмкости. В этом примере данная возможность не рассматривается.

Для настройки CCD необходимо выполнить следующие шаги:

1. Установите физические носители
2. Настройте поддержку в ядре
3. Создайте disklabel для каждого диска-компонента CCD
4. Настройте файл конфигурации CCD
5. Инициализируйте устройство CCD
6. Создайте файловую систему на новом устройстве CCD
7. Смонтируйте файловую систему CCD

В этом примере рассматривается настройка CCD в NetBSD/sparc 1.5. CCD будет размещён на четырёх дисках SCSI в типовом внешнем корпусе Sun disk pack, подключённом к внешнему 50-контактному порту SCSI.

15.1. Установка физических носителей

Этот шаг выполняется на ваше усмотрение и зависит от платформы и имеющегося оборудования.

Из моего вывода dmesg:

```
Disk #1:
  probe(esp0:0:0): max sync rate 10.00MB/s
  sd0 at scsibus0 target 0 lun 0: <SEAGATE, ST32430N SUN2.1G, 0444> SCSI2 0/direct fixed
  sd0: 2049 MB, 3992 cyl, 9 head, 116 sec, 512 bytes/sect x 4197405 sectors

Disk #2
  probe(esp0:1:0): max sync rate 10.00MB/s
  sd1 at scsibus0 target 1 lun 0: <SEAGATE, ST32430N SUN2.1G, 0444> SCSI2 0/direct fixed
  sd1: 2049 MB, 3992 cyl, 9 head, 116 sec, 512 bytes/sect x 4197405 sectors

Disk #3
  probe(esp0:2:0): max sync rate 10.00MB/s
  sd2 at scsibus0 target 2 lun 0: <SEAGATE, ST11200N SUN1.05, 9500> SCSI2 0/direct fixed
  sd2: 1005 MB, 1872 cyl, 15 head, 73 sec, 512 bytes/sect x 2059140 sectors

Disk #4
  probe(esp0:3:0): max sync rate 10.00MB/s
  sd3 at scsibus0 target 3 lun 0: <SEAGATE, ST11200N SUN1.05, 8808 > SCSI2 0
  sd3: 1005 MB, 1872 cyl, 15 head, 73 sec, 512 bytes/sect x 2059140 sectors
```

15.2. Настройка поддержки в ядре

Для поддержки устройств CCD нужна следующая директива конфигурации ядра. В ядре GENERIC она включена:

```
pseudo-device  ccd  4    # concatenated disk devices
```

В своей конфигурации ядра я также жёстко задаю соответствие идентификаторов SCSI записям устройств в `/dev`, чтобы не случилось неприятностей:

```
sd0     at scsibus0 target 0 lun ?
# SCSI disk drives
sd1     at scsibus0 target 1 lun ?
# SCSI disk drives
sd2     at scsibus0 target 2 lun ?
# SCSI disk drives
sd3     at scsibus0 target 3 lun ?
# SCSI disk drives
sd4     at scsibus0 target 4 lun ?
# SCSI disk drives
sd5     at scsibus0 target 5 lun ?
# SCSI disk drives
sd6     at scsibus0 target 6 lun ?
# SCSI disk drives
```

15.3. Создание disklabel для каждого диска-компонента CCD

Для каждого диска-компонента CCD потребуется создать особую файловую систему. В этом примере disklabel нужно создать для:

```
/dev/rsd0c

/dev/rsd1c

/dev/rsd2c

/dev/rsd3c
```

> **ПРИМЕЧАНИЕ**
>
> Всегда помните: disklabel нужно создавать для символьного устройства, а не для блочного, в `/dev/r{s,w}d*`

> **ПРИМЕЧАНИЕ**
>
> На всех платформах раздел `c` символизирует весь раздел NetBSD и зарезервирован.

Вероятно, вы захотите удалить все существующие disklabel на дисках, входящих в CCD. Это можно сделать одним из двух способов с помощью команды dd(1):

```
# dd if=/dev/zero of=/dev/rsd0c bs=8k count=1
# dd if=/dev/zero of=/dev/rsd1c bs=8k count=1
# dd if=/dev/zero of=/dev/rsd2c bs=8k count=1
# dd if=/dev/zero of=/dev/rsd3c bs=8k count=1
```

Если в вашей платформе для разбиения дисков используется MBR (Master Boot Record), так что разделы NetBSD занимают лишь часть всего диска, а другие операционные системы, например Windows или Linux, используют другие его части, вы можете уничтожить MBR и все разделы на диске с помощью команды:

```
# dd if=/dev/zero of=/dev/rsd0d bs=8k count=1
# dd if=/dev/zero of=/dev/rsd1d bs=8k count=1
# dd if=/dev/zero of=/dev/rsd2d bs=8k count=1
# dd if=/dev/zero of=/dev/rsd3d bs=8k count=1
```

Это сделает все данные на всём диске недоступными. Учтите, что весь диск — это раздел `d` на i386 (и некоторых других платформах), а в остальных случаях (например, на sparc) — `c`. См. параметр sysctl «kern.rawpartition»: «3» означает «d», «2» означает «c».

Disklabel по умолчанию для диска будет выглядеть примерно так:

```
# disklabel -r sd0
[...snip...]
bytes/sector: 512
sectors/track: 116
tracks/cylinder: 9
sectors/cylinder: 1044
cylinders: 3992
total sectors: 4197405
[..snip...]
3 partitions:
#        size   offset    fstype   [fsize bsize   cpg]
  c:  4197405        0     unused     1024  8192        # (Cyl.    0 - 4020*)
```

Вам нужно будет создать на разделе NetBSD диска один «slice» (раздел), охватывающий весь раздел. Раздел должен начинаться не менее чем через один цилиндр от начала диска или раздела, чтобы обеспечить место для специального disklabel CCD. Смещение должно быть равно 1 × sectors/cylinder (см. следующее примечание). Поэтому значение «size» должно быть равно «total sectors» минус 1 × «sectors/cylinder». Отредактируйте свой disklabel соответствующим образом:

```
# disklabel -e sd0
```

> **ПРИМЕЧАНИЕ**
>
> Смещение раздела типа «ccd» должно быть кратно значению «sectors/cylinder».

> **ПРИМЕЧАНИЕ**
>
> Перед редактированием disklabel не забудьте выполнить **export EDITOR=[путь к вашему любимому редактору]**.

> **ПРИМЕЧАНИЕ**
>
> Раздел должен иметь fstype `ccd`.

Поскольку здесь будет всего один раздел, вы можете повторно использовать раздел `c` (обычно зарезервированный для символических целей). Измените disklabel следующим образом:

```
3 partitions:
#        size   offset    fstype   [fsize bsize   cpg]
  c:  4196361     1044       ccd                     # (Cyl. 1 - 4020*)
```

При желании можно настроить и другой раздел вместо `c` — просто соответствующим образом измените приведённый ниже пример:

```
3 partitions:
#        size   offset    fstype   [fsize bsize   cpg]
  a:  4196361     1044       ccd                     # (Cyl. 1 - 4020*)
  c:  4197405        0     unused     1024  8192        # (Cyl. 0 - 4020*)
```

Не забудьте записать метку, когда закончите. Если disklabel не пройдёт свои проверки на разумность, он не примет вашу метку и предложит отредактировать её заново.

15.4. Настройка CCD

Когда все диски размечены должным образом, нужно создать файл конфигурации `/etc/ccd.conf`. По умолчанию этого файла нет, поэтому его придётся создать заново. Формат таков:

```
#ccd    ileave    flags    component    devices
```

> **ПРИМЕЧАНИЕ**
>
> Для параметра «ileave»: если используется значение ноль, диски объединяются, а если значение равно числу «sectors/track», диски чередуются.

Пример для этого случая:

```
# more /etc/ccd.conf
ccd0  0  none /dev/sd0c /dev/sd1c /dev/sd2c /dev/sd3c
```

> **ПРИМЕЧАНИЕ**
>
> Драйвер CCD ожидает, что компонентами будут файлы блочных устройств. Не используйте в конфигурации файлы символьных устройств.

15.5. Инициализация устройства CCD

Убедившись, что конфигурация CCD разумна, вы можете инициализировать устройство командой ccdconfig(8). Настроить:

```
# ccdconfig -C -f /etc/ccd.conf
```

Отменить настройку:

```
# ccdconfig -u -f /etc/ccd.conf
```

Инициализация устройства CCD активирует записи в `/dev`: `/dev/{,r}ccd#`:

```
# ls -la  /dev/{,r}ccd0*
brw-r-----  1 root  operator   9, 0 Apr 28 21:35 /dev/ccd0a
brw-r-----  1 root  operator   9, 1 Apr 28 21:35 /dev/ccd0b
brw-r-----  1 root  operator   9, 2 May 12 00:10 /dev/ccd0c
brw-r-----  1 root  operator   9, 3 Apr 28 21:35 /dev/ccd0d
brw-r-----  1 root  operator   9, 4 Apr 28 21:35 /dev/ccd0e
brw-r-----  1 root  operator   9, 5 Apr 28 21:35 /dev/ccd0f
brw-r-----  1 root  operator   9, 6 Apr 28 21:35 /dev/ccd0g
brw-r-----  1 root  operator   9, 7 Apr 28 21:35 /dev/ccd0h
crw-r-----  1 root  operator  23, 0 Jun 12 20:40 /dev/rccd0a
crw-r-----  1 root  operator  23, 1 Apr 28 21:35 /dev/rccd0b
crw-r-----  1 root  operator  23, 2 Jun 12 20:58 /dev/rccd0c
crw-r-----  1 root  operator  23, 3 Apr 28 21:35 /dev/rccd0d
crw-r-----  1 root  operator  23, 4 Apr 28 21:35 /dev/rccd0e
crw-r-----  1 root  operator  23, 5 Apr 28 21:35 /dev/rccd0f
crw-r-----  1 root  operator  23, 6 Apr 28 21:35 /dev/rccd0g
crw-r-----  1 root  operator  23, 7 Apr 28 21:35 /dev/rccd0h
```

15.6. Создание файловой системы 4.2BSD/UFS на новом устройстве CCD

Теперь вы можете создать disklabel для нового виртуального диска, связанного с вашим CCD:

```
# disklabel -e ccd0
```

Как и раньше, раздел будет только один, поэтому вы можете либо повторно использовать раздел `c`, либо создать отдельный раздел для работы.

```
# disklabel -r ccd0
# /dev/rccd0c:
type: ccd
disk: ccd
label: default label
flags:
bytes/sector: 512
sectors/track: 2048
tracks/cylinder: 1
sectors/cylinder: 2048
cylinders: 6107
total sectors: 12508812
rpm: 3600
interleave: 1
trackskew: 0
cylinderskew: 0
headswitch: 0           # microseconds
track-to-track seek: 0  # microseconds
drivedata: 0
#        size   offset    fstype   [fsize bsize   cpg]
  c: 12508812        0    4.2BSD     1024  8192    16  # (Cyl. 0 - 6107*)
```

Затем файловую систему нужно отформатировать:

```
# newfs /dev/rccd0c
Warning: 372 sector(s) in last cylinder unallocated
/dev/rccd0c:    12508812 sectors in 6108 cylinders of 1 tracks, 2048 sectors
        6107.8MB in 382 cyl groups (16 c/g, 16.00MB/g, 3968 i/g)

super-block backups (for fsck -b #) at:
[...]
```

15.7. Монтирование файловой системы

Создав файловую систему на устройстве CCD, вы можете смонтировать её в точку монтирования в вашей системе. Обязательно монтируйте раздел с меткой типа `ffs` или `4.2BSD`:

```
# mount /dev/ccd0c /mnt
```

Затем:

```
# export BLOCKSIZE=1024; df
Filesystem  1K-blocks     Used   Avail Capacity  Mounted on
/dev/sd6a      376155   320290   37057    89%    /
/dev/ccd0c    6058800        1  5755859     0%    /mnt
```

Поздравляем, теперь у вас работающий CCD. Чтобы устройство CCD настраивалось при загрузке, задайте `ccd=yes` в `/etc/rc.conf`. Чтобы файловая система монтировалась при загрузке, можно поправить `/etc/fstab`:

```
/dev/ccd0c  /home ffs     rw    1 2
```

Глава 16. NetBSD RAIDframe

16.1. Введение в RAIDframe

16.1.1. О RAIDframe

В подсистеме RAID в NetBSD используется программное обеспечение CMU RAIDframe. NetBSD — основная платформа для разработки RAIDframe. В NetBSD есть и другая реализация RAID уровня 0 в ядре — подсистема ccd(4) (см. главу 15, *Настройка объединённого дискового устройства (CCD)*). Прежде чем продолжать, вам следует обладать хотя бы базовыми знаниями о концепциях и терминологии RAID. Также желательно хотя бы в общих чертах знать разные уровни RAID: Adaptec предоставляет отличный справочник, а страница руководства raid(4) тоже содержит краткий обзор.

16.1.2. Предупреждение о целостности данных, резервном копировании и высокой доступности

RAIDframe — это программная реализация RAID, в отличие от аппаратного RAID. Как таковая, она не требует специальных дисковых контроллеров, поддерживаемых NetBSD. Системным администраторам следует серьёзно взвесить, что больше подходит для их «критически важных» приложений — программный или аппаратный RAID. Для некоторых проектов имеет смысл рассмотреть использование многих аппаратных RAID-устройств, поддерживаемых NetBSD. Какой тип RAID использовать, решать вам, но рекомендуется учитывать такие факторы, как управляемость, поддержка со стороны коммерческого поставщика, балансировка нагрузки и отказоустойчивость и т. п.

В зависимости от используемого уровня RAID RAIDframe действительно обеспечивает избыточность при отказе оборудования. Однако он *не* заменяет надёжное резервное копирование! Ошибки в программном обеспечении и ошибки пользователя всё ещё могут привести к потере данных. RAIDframe можно использовать как механизм, облегчающий резервное копирование в системах без резервного оборудования, но это не идеальная конфигурация. Наконец, что касается «высокой доступности», RAID — лишь очень малая составляющая обеспечения доступности данных.

И ещё раз, для надёжности: *делайте резервные копии данных!*

16.1.3. Получение помощи

Если при использовании RAIDframe возникают проблемы, есть несколько вариантов получения помощи.

1. Внимательно прочитайте страницы руководства RAIDframe: raid(4) и raidctl(8).
2. Поищите в архивах списков рассылки. К сожалению, в NetBSD нет списка, посвящённого поддержке RAIDframe. В зависимости от характера проблемы сообщения обычно попадают в разные списки. Как минимум поищите в netbsd-users@NetBSD.org, current-users@NetBSD.org. Также поищите в списке той платформы NetBSD, на которой вы используете RAIDframe: port-`${ARCH}`@NetBSD.org.
3. Поищите в базе данных сообщений о проблемах.
4. Если проблема не исчезла, отправьте сообщение в наиболее подходящий список рассылки (решать вам). Перед отправкой соберите как можно больше подробной информации: приложите

вывод dmesg(8) из `/var/run/dmesg.boot`, конфигурацию вашего ядра config(5), файл `/etc/raid[0-9].conf`, все относящиеся к делу ошибки в `/dev/console`, `/var/log/messages` или в `stdout/stderr` программы raidctl(8). Вывод команды **raidctl -s** (если она доступна) тоже будет полезен. Также укажите подробности предпринятых вами до сих пор шагов по диагностике, точное время начала проблемы и любые замечания о недавних изменениях, которые могли спровоцировать её появление. Помните, что ответа придётся подождать — наберитесь терпения.

16.2. Настройка поддержки RAIDframe

Использование RAID потребует изменений конфигурации программного и аппаратного обеспечения.

16.2.1. Поддержка в ядре

Ядро GENERIC уже содержит поддержку RAIDframe. Если вы собрали собственное ядро для своей среды, в конфигурации ядра должны быть следующие опции:

```
pseudo-device   raid            8       # RAIDframe disk driver
options         RAID_AUTOCONFIG         # auto-configuration of RAID components
```

Поддержку RAID должно обнаружить ядро NetBSD; это можно проверить, посмотрев на вывод команды dmesg(8).

```
# dmesg|grep -i raid
Kernelized RAIDframe activated
```

Исторически ядро также должно было содержать статические сопоставления между адресами шины и узлами устройств в `/dev`. Это обеспечивало согласованность устройств в наборах RAID в случае отказа устройства после перезагрузки. Однако начиная с NetBSD 1.6 вместо статического сопоставления устройств рекомендуется использовать средства автоматической конфигурации RAIDframe. Средства автоматической конфигурации позволяют дискам перемещаться по системе, и RAIDframe автоматически определит, какие компоненты принадлежат каким наборам RAID.

16.2.2. Резервирование питания и кеширование дисков

Если в вашей системе есть источник бесперебойного питания (UPS) и/или резервированные блоки питания, стоит подумать о включении кешей чтения и записи на дисках. В системах с резервированным питанием это повысит производительность дисков. В системах без резервирования питания кеш записи может поставить под угрозу целостность данных RAID при потере питания.

Утилиту dkctl(8) можно использовать для этого на всех видах дисков, поддерживающих эту операцию (SCSI, EIDE, SATA, …):

```
# dkctl wd0 getcache
/dev/rwd0d: read cache enabled
/dev/rwd0d: read cache enable is not changeable
/dev/rwd0d: write cache enable is changeable
/dev/rwd0d: cache parameters are not savable
# dkctl wd0 setcache rw
# dkctl wd0 getcache
/dev/rwd0d: read cache enabled
/dev/rwd0d: write-back cache enabled
/dev/rwd0d: read cache enable is not changeable
```

```
/dev/rwd0d: write cache enable is changeable
/dev/rwd0d: cache parameters are not savable
```

16.3. Пример: корневой диск RAID-1

В этом примере объясняется, как настроить корневой диск RAID-1. В RAID-1 компоненты зеркалируются, поэтому при отказе одного компонента сервер может полностью сохранять работоспособность. Цель — обеспечить такой уровень избыточности, при котором система при отказе любого из дисков-компонентов RAID сможет:

- продолжать нормальную работу до тех пор, пока не удастся запланировать окно обслуживания.
- Или, в маловероятном случае, когда отказ компонента приводит к перезагрузке системы, быстро перенастроить систему на загрузку с оставшегося компонента (зависит от платформы).

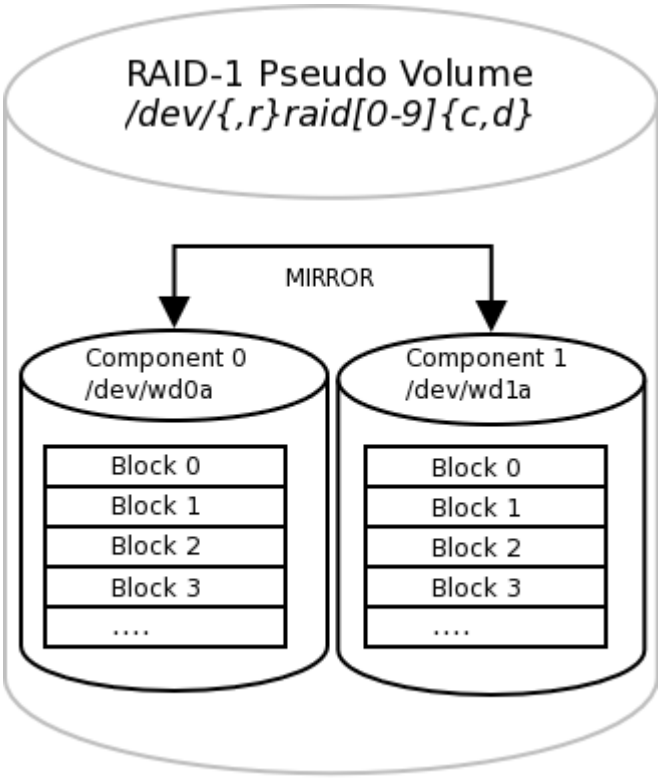


Рисунок 16.1. Логическая схема диска RAID-1

Поскольку RAID-1 обеспечивает и избыточность, и рост производительности, практичнее всего применять его для важных «системных» разделов, таких как `/`, `/usr`, `/var`, `swap` и т. д., где операции чтения происходят чаще операций записи. Для других файловых систем, например `/home` или `/var/{application}`, можно рассмотреть другие уровни RAID (см. ссылки выше). Если бы речь шла просто о создании обычного тома RAID-1 для некорневой файловой системы, можно было бы воспользоваться шаблонными примерами из страницы руководства, но поскольку корневой том должен быть загрузочным, при начальной настройке необходимо выполнить некоторые особые шаги.

> **ПРИМЕЧАНИЕ**
>
> В этом примере описан процесс, который лишь незначительно отличается для платформ x86 и sparc64. Для того чтобы избежать чрезмерного дублирования материала, различия, которые всё же существуют и носят косметический характер, будут отмечены в разделах, подобных этому. Если процесс существенно отличается, он разделяется на отдельные шаги, зависящие от платформы.

16.3.1. Схема псевдопроцесса

Хотя можно было бы разработать гораздо более изящный процесс, используя собственную копию NetBSD, установленную на специально изготовленный сменный носитель, в настоящее время установочный носитель NetBSD не содержит инструментов и поддержки RAIDframe, поэтому описанный ниже псевдопроцесс стал де-факто стандартом настройки корневого раздела RAID-1.

1. Установите стандартную NetBSD на Disk0 вашей системы.

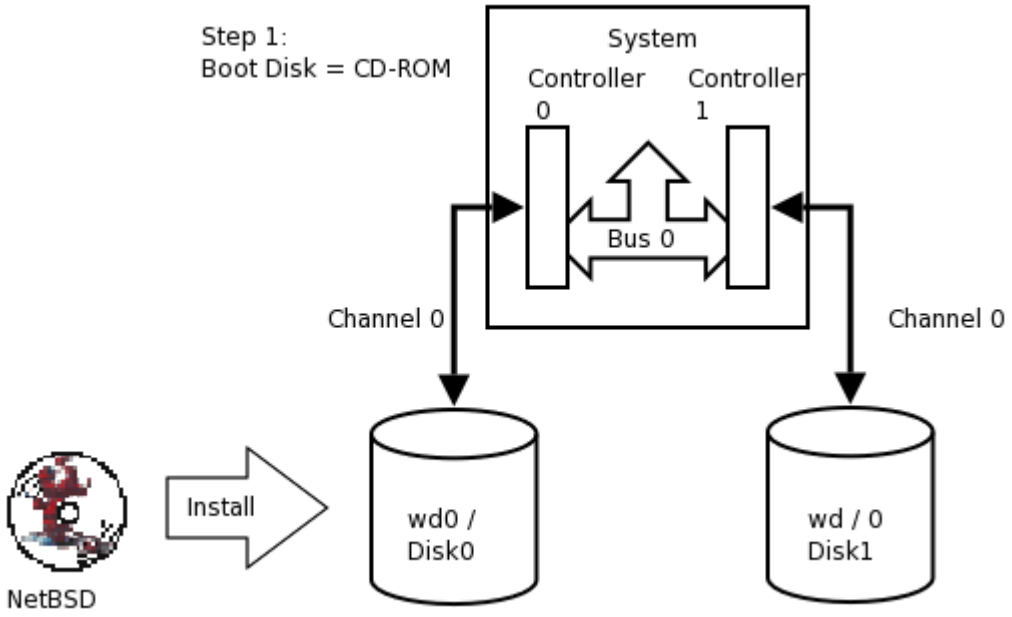


Рисунок 16.2. Обычная установка на Disk0/wd0

2. Используя установленную систему на Disk0/wd0, создайте набор RAID, состоящий только из Disk1/wd1.

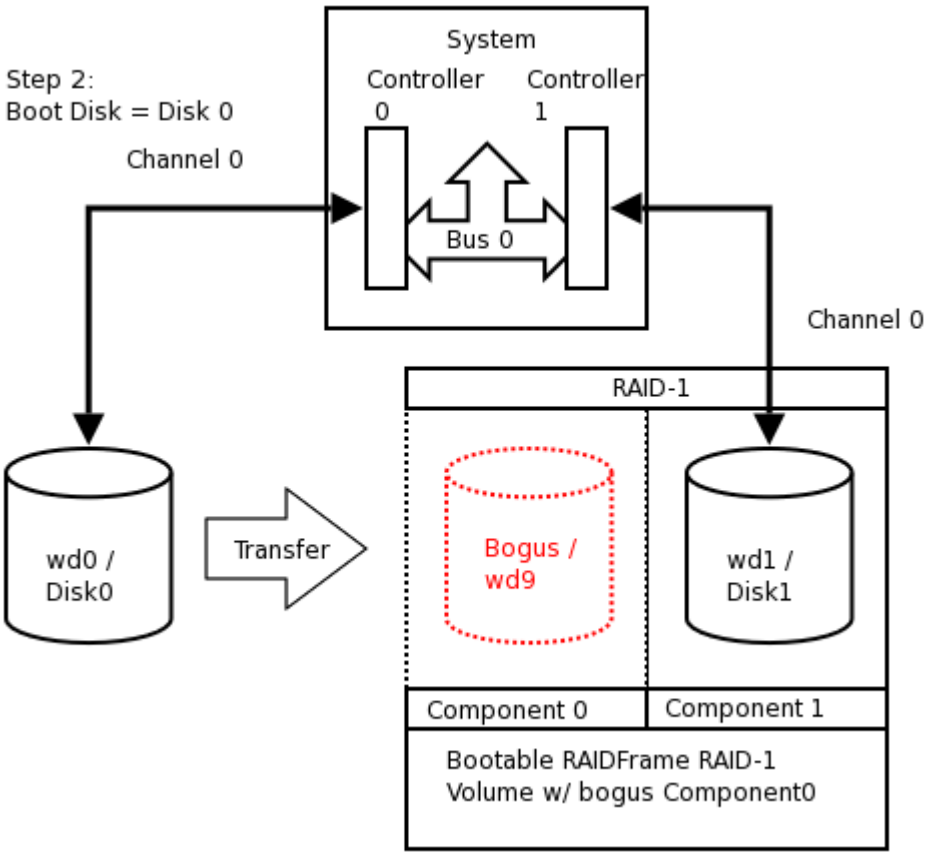


Рисунок 16.3. Создание набора RAID

3. Перезагрузите систему с Disk1/wd1, на котором находится только что созданный том RAID.

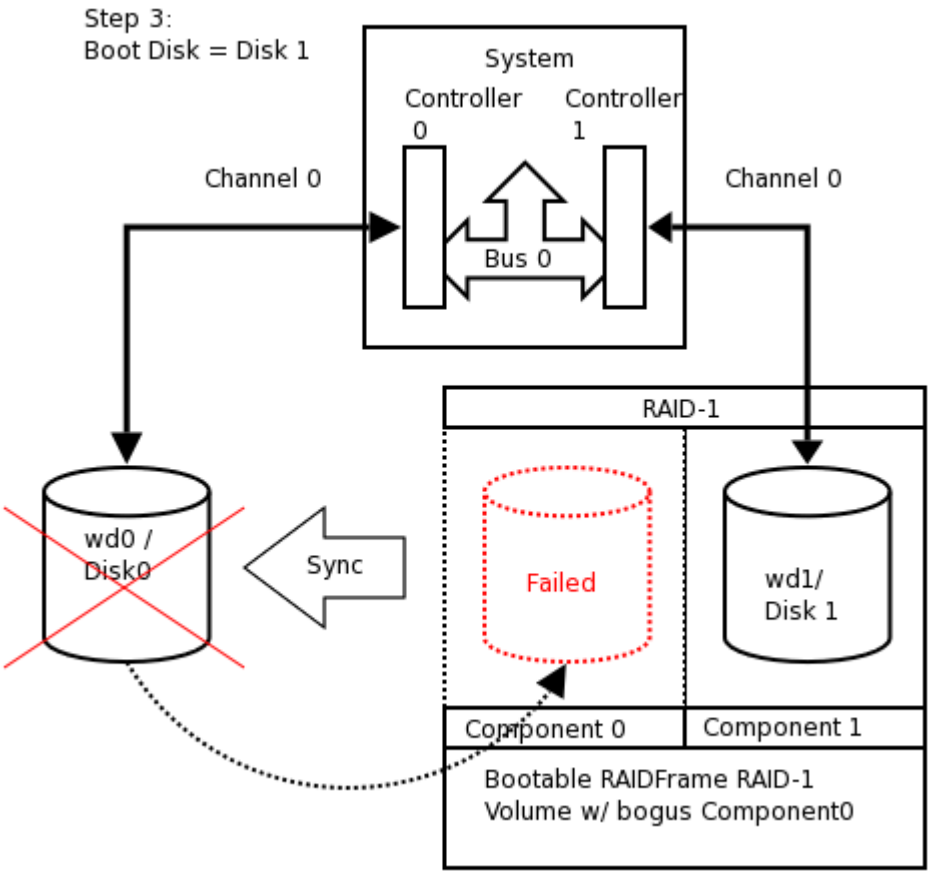


Рисунок 16.4. Перезагрузка с использованием Disk1/wd1 набора RAID

4. Добавьте Disk0/wd0 обратно в набор RAID и повторно синхронизируйте его.

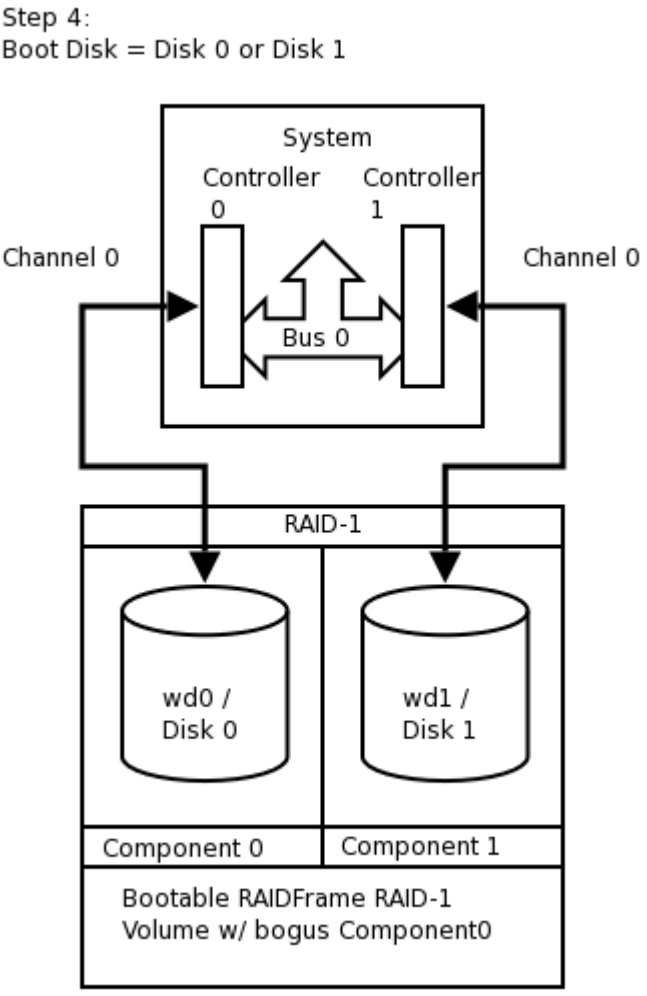


Рисунок 16.5. Зеркалирование Disk1/wd1 обратно на Disk0/wd0

16.3.2. Обзор оборудования

В настоящее время загрузка с RAID-1 поддерживается платформами NetBSD alpha, amd64, i386, pmax, sparc, sparc64 и vax. Загрузка с любого другого уровня RAID не поддерживается. Загрузка с набора RAID реализуется за счёт того, что загрузчик первого этапа обучается понимать разделы 4.2BSD/FFS и RAID. Коду первого загрузочного блока достаточно знать о разделах диска и файловых системах ровно столько, чтобы прочитать загрузочные блоки второго этапа. Поэтому в любой момент BIOS (прошивка) системы должен уметь читать диск с установленными загрузочными блоками первого этапа. На платформе x86 настройка этого целиком зависит от производителя карты контроллера (host bus adapter), к которой подключены ваши диски. На sparc64 этим управляет прошивка IEEE 1275 Sun OpenBoot.

В этой статье предполагаются два одинаковых диска IDE (`/dev/wd{0,1}`), которые мы собираемся зеркалировать (RAID-1). Эти диски обозначаются так:

```
# grep ^wd /var/run/dmesg.boot
wd0 at atabus0 drive 0: <WDC WD100BB-75CLB0>
wd0: drive supports 16-sector PIO transfers, LBA addressing
wd0: 9541 MB, 19386 cyl, 16 head, 63 sec, 512 bytes/sect x 19541088 sectors
wd0: drive supports PIO mode 4, DMA mode 2, Ultra-DMA mode 5 (Ultra/100)
wd0(piixide0:0:0): using PIO mode 4, Ultra-DMA mode 2 (Ultra/33) (using DMA data transfers)
```

```
wd1 at atabus1 drive 0: <WDC WD100BB-75CLB0>
wd1: drive supports 16-sector PIO transfers, LBA addressing
wd1: 9541 MB, 19386 cyl, 16 head, 63 sec, 512 bytes/sect x 19541088 sectors
wd1: drive supports PIO mode 4, DMA mode 2, Ultra-DMA mode 5 (Ultra/100)
wd1(piixide0:1:0): using PIO mode 4, Ultra-DMA mode 2 (Ultra/33) (using DMA data transfers)
```

> **ПРИМЕЧАНИЕ**
>
> Если вы используете SCSI, замените `/dev/{,r}wd{0,1}` на `/dev/{,r}sd{0,1}`

В этом примере оба диска перемычками переведены в режим Master и подключены к разным каналам одного контроллера. Никогда не стоит размещать оба диска на одной шине одного контроллера: так создаётся единая точка отказа. В идеале диски должны находиться на разных каналах разных контроллеров. У некоторых SCSI-контроллеров несколько каналов на одном контроллере, однако сброс SCSI-шины на одном канале может неблагоприятно повлиять на другой канал, если микросхема ASIC/IC окажется перегруженной. Компромисс при использовании двух контроллеров — двойной расход пропускной способности системной шины. Для простоты в этом примере показаны два диска на разных каналах одного контроллера.

> **ПРИМЕЧАНИЕ**
>
> RAIDframe требует, чтобы все компоненты были одного размера. На самом деле при разных размерах компонентов он использует наименьший общий знаменатель. Для иллюстрации в примере используются два диска с одинаковой геометрией. Кроме того, учитывайте доступность сменных дисков на случай, если компонент выйдет из строя из-за критического отказа оборудования.

> **СОВЕТ**
>
> Два диска с одинаковыми номерами моделей производителя могут иметь разную геометрию, если на накопителе есть «grown defects» (дефекты, возникшие в процессе эксплуатации). С помощью низкоуровневой программы изучите таблицу grown defects диска. Такие диски, очевидно, плохо подходят для RAID, и их следует избегать.

16.3.3. Первоначальная установка на Disk0/wd0

Выполните самую обычную установку на Disk0/wd0. Следуйте инструкциям файла INSTALL для вашей платформы. Установите все наборы, но не утруждайте себя настройкой чего-либо, кроме ядра, так как остальное будет перезаписано. См. также главу 2, *Установка NetBSD: предварительные замечания и подготовка*.

> **СОВЕТ**
>
> На x86 во время установки с помощью sysinst на вопрос, хотите ли вы «использовать весь диск для NetBSD», ответьте «да».

После завершения установки вам следует изучить вывод disklabel(8) и fdisk(8) / sunlabel(8) в вашей системе:

```
# df
Filesystem   1K-blocks        Used       Avail %Cap Mounted on
```

```
/dev/wd0a       9487886      502132     8511360   5% /
```

На x86:

```
# disklabel -r wd0
type: unknown
disk: Disk00
label:
flags:
bytes/sector: 512
sectors/track: 63
tracks/cylinder: 16
sectors/cylinder: 1008
cylinders: 19386
total sectors: 19541088
rpm: 3600
interleave: 1
trackskew: 0
cylinderskew: 0
headswitch: 0           # microseconds
track-to-track seek: 0  # microseconds
drivedata: 0

16 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
 a:  19276992        63     4.2BSD   1024  8192 46568  # (Cyl.      0* - 19124*)
 b:    264033  19277055       swap                     # (Cyl.  19124* - 19385)
 c:  19541025        63     unused      0     0        # (Cyl.      0* - 19385)
 d:  19541088         0     unused      0     0        # (Cyl.      0 - 19385)

# fdisk /dev/rwd0d
Disk: /dev/rwd0d
NetBSD disklabel disk geometry:
cylinders: 19386, heads: 16, sectors/track: 63 (1008 sectors/cylinder)
total sectors: 19541088

BIOS disk geometry:
cylinders: 1023, heads: 255, sectors/track: 63 (16065 sectors/cylinder)
total sectors: 19541088

Partition table:
0: NetBSD (sysid 169)
    start 63, size 19541025 (9542 MB, Cyls 0-1216/96/1), Active
1: <UNUSED>
2: <UNUSED>
3: <UNUSED>
Bootselector disabled.
First active partition: 0
```

На Sparc64 команда и вывод немного отличаются:

```
# disklabel -r wd0
type: unknown
disk: Disk0
[...snip...]
8 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
 a:  19278000         0     4.2BSD   1024  8192 46568  # (Cyl.      0 -  19124)
 b:    263088  19278000       swap                     # (Cyl.  19125 -  19385)
 c:  19541088         0     unused      0     0        # (Cyl.      0 -  19385)

# sunlabel /dev/rwd0c
```

```
sunlabel> P
a: start cyl =      0, size = 19278000 (19125/0/0 - 9413.09Mb)
b: start cyl =  19125, size =   263088 (261/0/0 - 128.461Mb)
c: start cyl =      0, size = 19541088 (19386/0/0 - 9541.55Mb)
```

16.3.4. Подготовка Disk1/wd1

Когда на Disk0/wd0 установлена стандартная NetBSD, можно приступать. Disk1/wd1 будет виден системе и не будет ею использоваться. Чтобы настроить Disk1/wd1, с помощью disklabel(8) выделите весь второй диск под набор RAID-1.

> **СОВЕТ**
>
> Лучший способ убедиться, что Disk1/wd1 полностью пуст, — обнулить первые несколько секторов диска с помощью dd(1). Это сотрёт MBR (x86) или метку диска Sun (sparc64), а также метку диска NetBSD. Если вы ошибётесь на любом этапе настройки RAID, вы всегда сможете повторить эту процедуру, чтобы вернуть диск в пустое состояние.

> **ПРИМЕЧАНИЕ**
>
> На sparc64 используйте `/dev/rwd1c` вместо `/dev/rwd1d`!

```
# dd if=/dev/zero of=/dev/rwd1d bs=8k count=1
1+0 records in
1+0 records out
8192 bytes transferred in 0.003 secs (2730666 bytes/sec)
```

После этого на x86 проверьте, что исчезли и метка MBR, и метка диска NetBSD. На sparc64 проверьте, что метка диска Sun также исчезла.

На x86:

```
# fdisk /dev/rwd1d

fdisk: primary partition table invalid, no magic in sector 0
Disk: /dev/rwd1d
NetBSD disklabel disk geometry:
cylinders: 19386, heads: 16, sectors/track: 63 (1008 sectors/cylinder)
total sectors: 19541088

BIOS disk geometry:
cylinders: 1023, heads: 255, sectors/track: 63 (16065 sectors/cylinder)
total sectors: 19541088

Partition table:
0: <UNUSED>
1: <UNUSED>
2: <UNUSED>
3: <UNUSED>
Bootselector disabled.

# disklabel -r wd1

[...snip...]
16 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
```

```
 c:  19541025        63     unused      0     0        # (Cyl.      0* - 19385)
 d:  19541088         0     unused      0     0        # (Cyl.      0 - 19385)
```

На sparc64:

```
# sunlabel /dev/rwd1c

sunlabel: bogus label on `/dev/wd1c' (bad magic number)

# disklabel -r wd1

[...snip...]
3 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
 c:  19541088         0     unused      0     0        # (Cyl.      0 -  19385)
disklabel: boot block size 0
disklabel: super block size 0
```

Теперь, когда вы уверены, что второй диск пуст, на x86 необходимо создать MBR на втором диске, используя значения, полученные выше для Disk0/wd0. Не забудьте пометить раздел NetBSD активным, иначе система не загрузится. Кроме того, необходимо создать метку NetBSD (disklabel) на Disk1/wd1, которая позволит разместить на нём том RAID. На sparc64 достаточно просто выполнить disklabel(8) для второго диска — будет записана правильная метка диска Sun.

> **СОВЕТ**
>
> disklabel(8) использует для редактирования метки переменную окружения оболочки `$EDITOR`. По умолчанию используется vi(1)

На x86:

```
# fdisk -0ua /dev/rwd1d
fdisk: primary partition table invalid, no magic in sector 0
Disk: /dev/rwd1d
NetBSD disklabel disk geometry:
cylinders: 19386, heads: 16, sectors/track: 63 (1008 sectors/cylinder)
total sectors: 19541088

BIOS disk geometry:
cylinders: 1023, heads: 255, sectors/track: 63 (16065 sectors/cylinder)
total sectors: 19541088

Do you want to change our idea of what BIOS thinks? [n]

Partition 0:
<UNUSED>
The data for partition 0 is:
<UNUSED>
sysid: [0..255 default: 169]
start: [0..1216cyl default: 63, 0cyl, 0MB]
size: [0..1216cyl default: 19541025, 1216cyl, 9542MB]
bootmenu: []
Do you want to change the active partition? [n] y
Choosing 4 will make no partition active.
active partition: [0..4 default: 0] 0
Are you happy with this choice? [n] y

We haven't written the MBR back to disk yet.  This is your last chance.
Partition table:
```

```
0: NetBSD (sysid 169)
    start 63, size 19541025 (9542 MB, Cyls 0-1216/96/1), Active
1: <UNUSED>
2: <UNUSED>
3: <UNUSED>
Bootselector disabled.
Should we write new partition table? [n] y

# disklabel -r -e -I wd1
type: unknown
disk: Disk1
label:
flags:
bytes/sector: 512
sectors/track: 63
tracks/cylinder: 16
sectors/cylinder: 1008
cylinders: 19386
total sectors: 19541088
[...snip...]
16 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
 a:  19541025        63       RAID                     # (Cyl.      0*-19385)
 c:  19541025        63     unused      0     0        # (Cyl.      0*-19385)
 d:  19541088         0     unused      0     0        # (Cyl.      0 -19385)
```

На sparc64:

```
# disklabel -r -e -I wd1
type: unknown
disk: Disk1
label:
flags:
bytes/sector: 512
sectors/track: 63
tracks/cylinder: 16
sectors/cylinder: 1008
cylinders: 19386
total sectors: 19541088
[...snip...]
3 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
 a:  19541088         0       RAID                     # (Cyl.      0 -  19385)
 c:  19541088         0     unused      0     0        # (Cyl.      0 -  19385)

# sunlabel /dev/rwd1c
sunlabel> P
a: start cyl =      0, size = 19541088 (19386/0/0 - 9541.55Mb)
c: start cyl =      0, size = 19541088 (19386/0/0 - 9541.55Mb)
```

> **ПРИМЕЧАНИЕ**
>
> На x86 разделы **c:** и **d:** зарезервированы. **c:** представляет часть диска, отведённую под NetBSD. **d:** представляет весь диск. Поскольку мы хотим отдать весь раздел MBR NetBSD под RAID, а **a:** находится в границах **c:**, разделы **a:** и **c:** имеют одинаковые значения размера и смещения. Смещение должно начинаться на границе дорожки (величина, кратная числу секторов на дорожку в метке диска). На sparc64, однако, **c:** представляет весь раздел NetBSD в метке диска Sun, а **d:** не зарезервирован. Учтите также, что на sparc64 разделы **c:** и **a:** не требуют смещения от начала диска, но если оно всё же понадобится, оно должно начинаться на границе цилиндра (величина, кратная числу секторов на цилиндр).

16.3.5. Инициализация устройства RAID

Далее создадим файл конфигурации для набора RAID / тома. Традиционно файлы конфигурации RAIDframe размещаются в `/etc` , читаются и инициализируются при загрузке, однако, поскольку мы создаём загрузочный том RAID, данные конфигурации фактически будут записаны в том RAID с помощью функции «auto-configure». Поэтому файлы нужны только при первоначальной настройке, и они не должны находиться в `/etc`.

```
# vi /var/tmp/raid0.conf
START array
1 2 0

START disks
absent
/dev/wd1a

START layout
128 1 1 1

START queue
fifo 100
```

Обратите внимание, что `absent` означает несуществующий диск. Это позволит создать том RAID с фиктивным компонентом, который позже будет заменён на Disk0/wd0.

Затем настроим устройство RAID и зададим ему уникальный серийный номер. В этом примере используется схема «YYYYMMDD*Revision*». Формат выбираете вы сами, однако выбранная схема должна гарантировать, что никакие два набора RAID не используют один и тот же серийный номер одновременно.

После этого инициализируем набор RAID в первый раз, спокойно игнорируя ошибки, связанные с фиктивным компонентом.

```
# raidctl -v -C /var/tmp/raid0.conf raid0
Ignoring missing component at column 0
raid0: Component absent being configured at col: 0
         Column: 0 Num Columns: 0
         Version: 0 Serial Number: 0 Mod Counter: 0
         Clean: No Status: 0
Number of columns do not match for: absent
absent is not clean!
raid0: Component /dev/wd1a being configured at col: 1
         Column: 0 Num Columns: 0
         Version: 0 Serial Number: 0 Mod Counter: 0
         Clean: No Status: 0
Column out of alignment for: /dev/wd1a
```

```
Number of columns do not match for: /dev/wd1a
/dev/wd1a is not clean!
raid0: There were fatal errors
raid0: Fatal errors being ignored.
raid0: RAID Level 1
raid0: Components: component0[**FAILED**] /dev/wd1a
raid0: Total Sectors: 19540864 (9541 MB)
# raidctl -v -I 2009122601 raid0
# raidctl -v -i raid0
Initiating re-write of parity
raid0: Error re-writing parity!
Parity Re-write status:

# tail -1 /var/log/messages
Dec 26 00:00:30  /netbsd: raid0: Error re-writing parity!
# raidctl -v -s raid0
Components:
          component0: failed
           /dev/wd1a: optimal
No spares.
component0 status is: failed.  Skipping label.
Component label for /dev/wd1a:
   Row: 0, Column: 1, Num Rows: 1, Num Columns: 2
   Version: 2, Serial Number: 2009122601, Mod Counter: 7
   Clean: No, Status: 0
   sectPerSU: 128, SUsPerPU: 1, SUsPerRU: 1
   Queue size: 100, blocksize: 512, numBlocks: 19540864
   RAID Level: 1
   Autoconfig: No
   Root partition: No
   Last configured as: raid0
Parity status: DIRTY
Reconstruction is 100% complete.
Parity Re-write is 100% complete.
Copyback is 100% complete.
```

16.3.6. Настройка файловых систем

> **ВНИМАНИЕ**
>
> Корневая файловая система должна начинаться с сектора 0 устройства RAID. Иначе первичный загрузчик не сможет найти вторичный загрузчик.

Устройство RAID теперь настроено и доступно. Устройство RAID — это псевдоустройство диска. Оно создаётся с меткой диска по умолчанию. Теперь необходимо определить правильные размеры разделов disklabel для вашей рабочей среды. Для упрощения в этом примере 8,5 гигабайта отводится под `/` как **/dev/raid0a**, а остальное — под `swap` как **/dev/raid0b**.

> **ВНИМАНИЕ**
>
> Это нереалистичная схема диска для рабочего сервера; в руководстве NetBSD правильная методика разбиения диска описана подробнее. См. главу 2, *Установка NetBSD: предварительные замечания и подготовка*

ПРИМЕЧАНИЕ

Учтите, что 1 ГБ — это 2*1024*1024=2097152 блока (1 блок равен 512 байтам, или 0,5 килобайта). Независимо от того, из какого оборудования состоит набор RAID, псевдодиск RAID всегда имеет 512 байт на сектор.

ПРИМЕЧАНИЕ

В нашем примере пространство, выделенное базовому разделу `a:`, из которого состоит набор RAID, отличалось для x86 и sparc64, поэтому общее число секторов томов RAID различается:

На x86:

```
 # disklabel -r -e -I raid0
type: RAID
disk: raid
label: fictitious
flags:
bytes/sector: 512
sectors/track: 128
tracks/cylinder: 8
sectors/cylinder: 1024
cylinders: 19082
total sectors: 19540864
rpm: 3600
interleave: 1
trackskew: 0
cylinderskew: 0
headswitch: 0 # microseconds
track-to-track seek: 0 # microseconds
drivedata: 0

#        size    offset     fstype [fsize bsize cpg/sgs]
 a:  19015680         0     4.2BSD      0     0     0  # (Cyl.      0 - 18569)
 b:    525184  19015680       swap                     # (Cyl.  18570 - 19082*)
 d:  19540864         0     unused      0     0        # (Cyl.      0 - 19082*)
```

На sparc64:

```
# disklabel -r -e -I raid0
[...snip...]
total sectors: 19539968
[...snip...]
3 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
 a:  19251200         0     4.2BSD      0     0     0  # (Cyl.      0 -  18799)
 b:    288768  19251200       swap                     # (Cyl.  18800 -  19081)
 c:  19539968         0     unused      0     0        # (Cyl.      0 -  19081)
```

Затем отформатируйте только что созданный раздел `/` как файловую систему 4.2BSD FFSv1:

```
# newfs -O 1 /dev/rraid0a
/dev/rraid0a: 9285.0MB (19015680 sectors) block size 16384, fragment size 2048
        using 51 cylinder groups of 182.06MB, 11652 blks, 23040 inodes.
super-block backups (for fsck -b #) at:
32, 372896, 745760, 1118624, 1491488, 1864352, 2237216, 2610080, 2982944,
.....................................................................
```

```
# fsck -fy /dev/rraid0a
** /dev/rraid0a
** File system is already clean
** Last Mounted on
** Phase 1 - Check Blocks and Sizes
** Phase 2 - Check Pathnames
** Phase 3 - Check Connectivity
** Phase 4 - Check Reference Counts
** Phase 5 - Check Cyl groups
1 files, 1 used, 4679654 free (14 frags, 584955 blocks, 0.0% fragmentation)
```

16.3.7. Перенос системы на RAID

Новые файловые системы RAID теперь готовы к использованию. Мы монтируем их в `/mnt` и копируем все файлы со старой системы. Это можно сделать с помощью dump(8) или pax(1).

```
# mount /dev/raid0a /mnt
# df -h /mnt
Filesystem        Size       Used      Avail %Cap Mounted on
/dev/raid0a       8.9G       2.0K       8.5G   0% /mnt
# cd /; pax -v -X -rw -pe . /mnt
[...snip...]
```

Теперь установленная NetBSD находится на файловой системе RAID. Нужно исправить точки монтирования в новой копии `/etc/fstab`, иначе система не запустится правильно. Замените все вхождения `wd0` на `raid0`.

Подкачку следует отключать при выключении системы, чтобы избежать ошибок чётности на устройстве RAID. Это делается одной простой строкой в `/etc/rc.conf`.

```
# vi /mnt/etc/rc.conf
swapoff=YES
```

Далее необходимо установить загрузчик на Disk1/wd1. Если не установить загрузчик на Disk1/wd1, то при отказе Disk0/wd0 система станет незагружаемой, и RAID-1 потеряет смысл.

> **СОВЕТ**
>
> Поскольку меню BIOS/CMOS во многих системах на базе x86 вводят в заблуждение относительно порядка загрузки устройств, я настоятельно рекомендую использовать опцию «-o timeout=X», поддерживаемую загрузчиком первого этапа x86. Задайте уникальные значения для каждого диска в качестве ориентира, чтобы легко определять, с какого диска загружается система.

> **ВНИМАНИЕ**
>
> Хотя может показаться логичным установить загрузочный блок первого этапа в `/dev/rwd1{c,d}` с помощью installboot(8), начиная с NetBSD 1.6.x это уже не так. Если вы допустите эту ошибку, загрузочный сектор будет непоправимо повреждён, и процесс придётся начать заново.

На x86 установите загрузчик в `/dev/rwd1a` :

```
# /usr/sbin/installboot -o timeout=30 -v /dev/rwd1a /usr/mdec/bootxx_ffsv2
File system:         /dev/rwd1a
Primary bootstrap:   /usr/mdec/bootxx_ffsv2
```

```
Ignoring PBR with invalid magic in sector 0 of `/dev/rwd1a'
Boot options:        timeout 30, flags 0, speed 9600, ioaddr 0, console pc
```

ПРИМЕЧАНИЕ

Начиная с NetBSD 6.x типом файловой системы по умолчанию на платформах x86 является FFSv2, а не FFSv1. Убедитесь, что используете правильный файл загрузочного блока первого этапа `/usr/mdec/bootxx_ffsv{1,2}` при запуске команды installboot(8).

Чтобы узнать, какой тип файловой системы используется в данный момент, можно воспользоваться командой file(1) или dumpfs(8):

```
# /usr/bin/file -s /dev/rwd1a
/usr/bin/file -s /dev/rwd1a
/dev/rwd1a: Unix Fast File system [v2] (little-endian), last mounted on ...
```

или

```
# /usr/sbin/dumpfs -s /dev/rwd1a
file system: /dev/rwd1a
format  FFSv2
endian  little-endian
...
```

На sparc64 также установите загрузчик в `/dev/rwd1a` , однако ключ «-o» не поддерживается (и не нужен благодаря OpenBoot):

```
# /usr/sbin/installboot -v /dev/rwd1a /usr/mdec/bootblk
File system:         /dev/rwd1a
Primary bootstrap:   /usr/mdec/bootblk
Bootstrap start sector: 1
Bootstrap byte count:   5140
Writing bootstrap
```

Наконец, набор RAID нужно сделать автоматически настраиваемым, а систему — перезагрузить. После перезагрузки всё монтируется с устройств RAID.

```
# raidctl -v -A root raid0
raid0: Autoconfigure: Yes
raid0: Root: Yes
# tail -2 /var/log/messages
raid0: New autoconfig value is: 1
raid0: New rootpartition value is: 1
# raidctl -v -s raid0
[...snip...]
   Autoconfig: Yes
   Root partition: Yes
   Last configured as: raid0
[...snip...]
# shutdown -r now
```

> **ПРЕДУПРЕЖДЕНИЕ**
>
> Всегда используйте shutdown(8) при выключении. Никогда не используйте просто reboot(8). reboot(8) не выполнит должным образом сценарии RC при завершении работы и не отключит подкачку безопасно. Это приведёт к появлению «грязной» чётности при каждой перезагрузке.

16.3.8. Первая загрузка с RAID

На этом этапе временно настройте систему на загрузку с Disk1/wd1. Подробности этого процесса описаны в примечаниях к разделу 16.3.10, «Проверка загрузочных блоков». Система теперь должна загрузиться, а все файловые системы должны находиться на устройствах RAID. RAID будет работоспособен с одним компонентом, однако набор работает не полностью, поскольку фиктивный диск (wd9) отказал.

```
# egrep -i "raid|root" /var/run/dmesg.boot
raid0: RAID Level 1
raid0: Components: component0[**FAILED**] /dev/wd1a
raid0: Total Sectors: 19540864 (9541 MB)
boot device: raid0
root on raid0a dumps on raid0b
root file system type: ffs

# df -h
Filesystem    Size     Used     Avail Capacity  Mounted on
/dev/raid0a   8.9G     196M      8.3G     2%    /
kernfs        1.0K     1.0K        0B   100%    /kern

# swapctl -l
Device      1K-blocks     Used    Avail Capacity  Priority
/dev/raid0b    262592        0   262592     0%    0
# raidctl -s raid0
Components:
          component0: failed
           /dev/wd1a: optimal
No spares.
component0 status is: failed.  Skipping label.
Component label for /dev/wd1a:
   Row: 0, Column: 1, Num Rows: 1, Num Columns: 2
   Version: 2, Serial Number: 2009122601, Mod Counter: 65
   Clean: No, Status: 0
   sectPerSU: 128, SUsPerPU: 1, SUsPerRU: 1
   Queue size: 100, blocksize: 512, numBlocks: 19540864
   RAID Level: 1
   Autoconfig: Yes
   Root partition: Yes
   Last configured as: raid0
Parity status: DIRTY
Reconstruction is 100% complete.
Parity Re-write is 100% complete.
Copyback is 100% complete.
```

16.3.9. Добавление Disk0/wd0 в RAID

Теперь добавим Disk0/wd0 в качестве компонента RAID. Это уничтожит исходную структуру файловой системы. На x86 метка MBR останется нетронутой (вспомните, что метка wd0 всё равно была скопирована на wd1), поэтому обнулять Disk0/wd0 не нужно. Однако нужно заново разметить Disk0/wd0 так, чтобы раскладка метки NetBSD совпадала с Disk1/wd1. Затем мы добавим Disk0/wd0 в

набор RAID как «hot-spare» и запустим восстановление чётности для всех устройств RAID, фактически включив Disk0/wd0 в набор RAID-1 и «синхронизировав» оба диска.

```
# disklabel -r wd1 > /tmp/disklabel.wd1
# disklabel -R -r wd0 /tmp/disklabel.wd1
```

В качестве последней проверки перед переходом к следующему шагу можно воспользоваться diff(1), чтобы убедиться, что метки дисков Disk0/wd0 и Disk1/wd1 совпадают. Эти файлы также стоит сохранить в качестве резервной копии на случай аварии.

```
# disklabel -r wd0 > /tmp/disklabel.wd0
# disklabel -r wd1 > /tmp/disklabel.wd1
# diff /tmp/disklabel.wd0 /tmp/disklabel.wd1
# fdisk /dev/rwd0 > /tmp/fdisk.wd0
# fdisk /dev/rwd1 > /tmp/fdisk.wd1
# diff /tmp/fdisk.wd0 /tmp/fdisk.wd1
# mkdir /root/RFbackup
# cp -p /tmp/{disklabel,fdisk}* /root/RFbackup
```

Когда вы убедитесь в этом, добавьте Disk0/wd0 как запасной компонент и запустите восстановление:

```
# raidctl -v -a /dev/wd0a raid0
/netbsd: Warning: truncating spare disk /dev/wd0a to 241254528 blocks
# raidctl -v -s raid0
Components:
          component0: failed
           /dev/wd1a: optimal
Spares:
           /dev/wd0a: spare
[...snip...]
# raidctl -F component0 raid0
RECON: initiating reconstruction on col 0 -> spare at col 2
 11% |****                                   | ETA:    04:26 \
```

Время восстановления зависит от скорости вашего оборудования. За ходом процесса можно наблюдать в другом терминале:

```
# raidctl -S raid0
Reconstruction is 0% complete.
Parity Re-write is 100% complete.
Copyback is 100% complete.
Reconstruction status:
  17% |******                                 | ETA: 03:08 -
```

После восстановления оба диска должны быть «оптимальными».

```
# tail -f /var/log/messages
raid0: Reconstruction of disk at col 0 completed
raid0: Recon time was 1290.625033 seconds, accumulated XOR time was 0 us (0.000000)
raid0:  (start time 1093407069 sec 145393 usec, end time 1093408359 sec 770426 usec)
raid0: Total head-sep stall count was 0
raid0: 305318 recon event waits, 1 recon delays
raid0: 1093407069060000 max exec ticks

# raidctl -v -s raid0
Components:
```

```
           component0: spared
           /dev/wd1a: optimal
Spares:
     /dev/wd0a: used_spare
     [...snip...]
```

Когда восстановление завершится, нужно установить загрузчик на Disk0/wd0. На x86 установите загрузчик в `/dev/rwd0a`:

```
# /usr/sbin/installboot -o timeout=15 -v /dev/rwd0a /usr/mdec/bootxx_ffsv2
File system:         /dev/rwd0a
Primary bootstrap:   /usr/mdec/bootxx_ffsv2
Boot options:        timeout 15, flags 0, speed 9600, ioaddr 0, console pc
```

На sparc64:

```
# /usr/sbin/installboot -v /dev/rwd0a /usr/mdec/bootblk
File system:         /dev/rwd0a
Primary bootstrap:   /usr/mdec/bootblk
Bootstrap start sector: 1
Bootstrap byte count:   5140
Writing bootstrap
```

И наконец, перед продолжением перезагрузите машину в последний раз. Это необходимо, чтобы перевести Disk0/wd0 из состояния «used_spare» в качестве «Component0» в состояние «optimal». О проверке чистой чётности после каждой перезагрузки см. примечания в следующем разделе.

```
# shutdown -r now
```

16.3.10. Проверка загрузочных блоков

На этом этапе нужно убедиться, что оборудование вашей системы способно правильно загружаться с использованием загрузочных блоков на любом из дисков. На x86 это зависит от оборудования: процедуру можно выполнить через меню CMOS/BIOS материнской платы или меню настройки карты контроллера.

На x86 с помощью меню вашей машины задайте порядок и приоритет загрузочных устройств так, чтобы Disk1/wd1 шёл перед Disk0/wd0. В примерах ниже показан типичный Award BIOS.

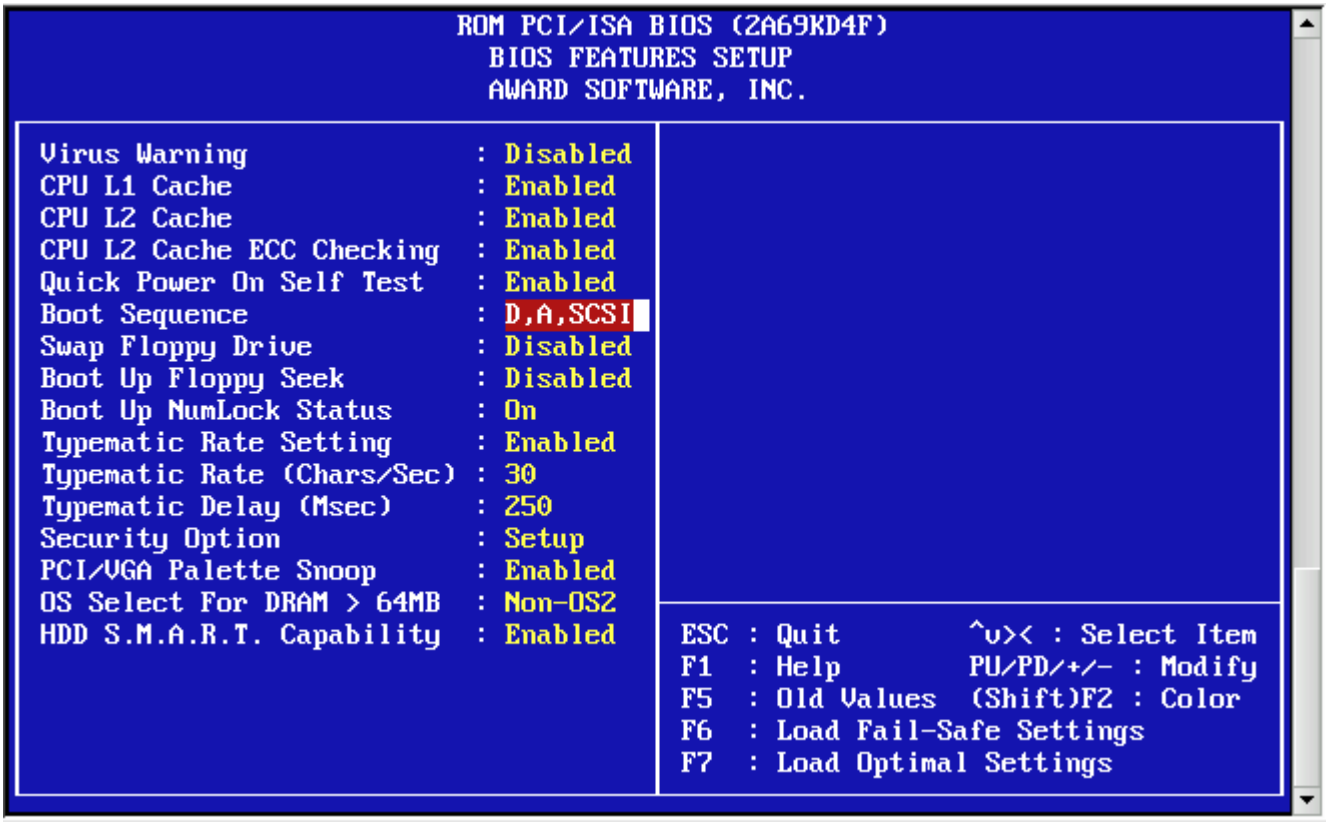


Рисунок 16.6. Award BIOS i386: загрузка с Disk1/wd1

Сохраните изменения и выйдите.

```
>> NetBSD/i386 BIOS Boot, Revision 5.2 (from NetBSD 5.0.2)
>> (builds@b7, Sun Feb 7 00:30:50 UTC 2010)
>> Memory: 639/130048 k
Press return to boot now, any other key for boot menu
booting hd0a:netbsd - starting in 30
```

Понять, что BIOS читает Disk1/wd1, можно по тому, что тайм-аут загрузчика равен 30 секундам вместо 15. После перезагрузки снова войдите в BIOS и верните порядок загрузки дисков к значению по умолчанию:

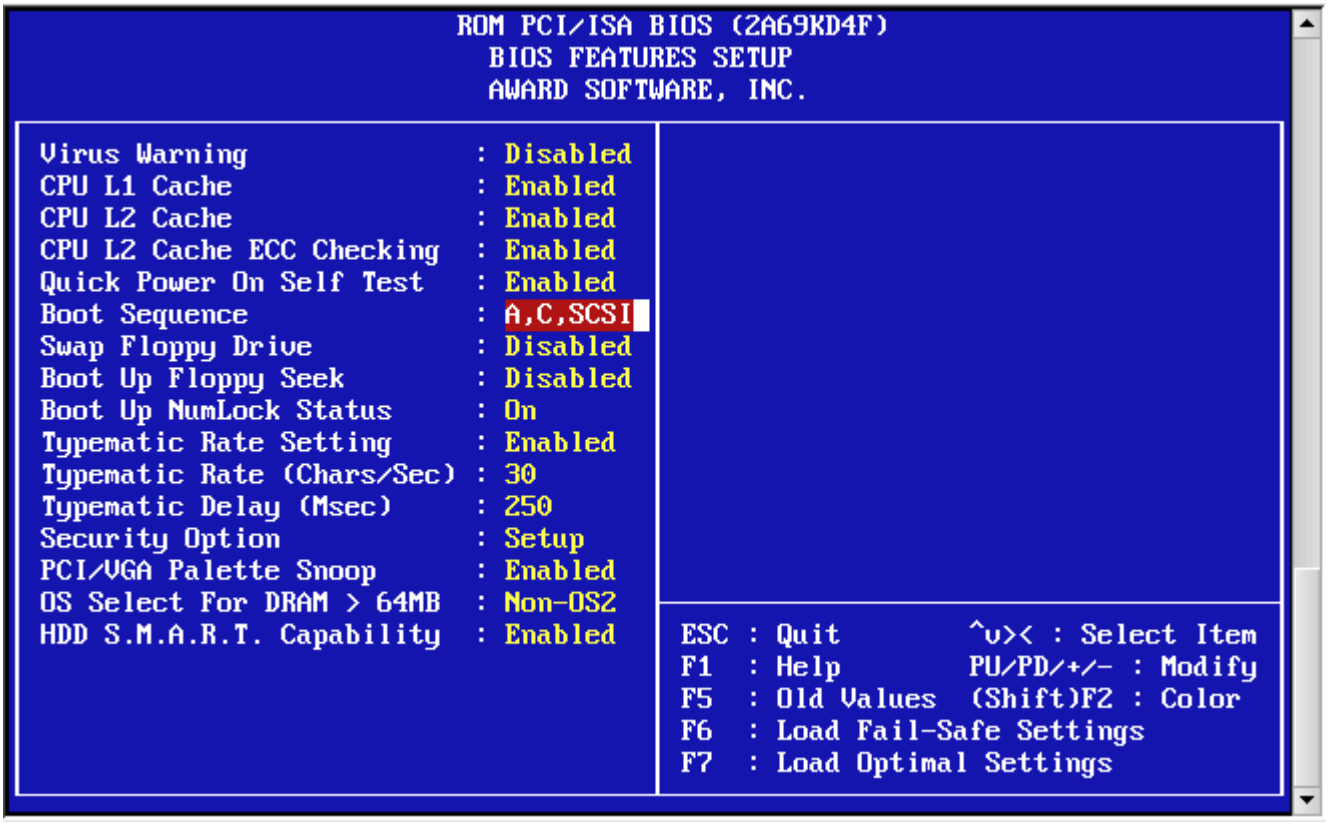


Рисунок 16.7. Award BIOS i386: загрузка с Disk0/wd0

Сохраните изменения и выйдите.

```
>> NetBSD/x86 BIOS Boot, Revision 5.9 (from NetBSD 6.0)
>> Memory: 640/261120 k

     1. Boot normally
     2. Boot single use
     3. Disable ACPI
     4. Disable ACPI and SMP
     5. Drop to boot prompt

Choose an option; RETURN for default; SPACE to stop countdown.Option 1 will be chosen in 0
seconds.
```

Обратите внимание, как ваше настроенное ядро определяет назначения контроллеров, шин и дисков независимо от того, какой диск BIOS назначает загрузочным. Это ожидаемое поведение.

На sparc64 воспользуйтесь командой Sun OpenBoot **devalias**, чтобы убедиться, что загрузка возможна с обоих дисков:

```
Sun Ultra 5/10 UPA/PCI (UltraSPARC-IIi 400MHz), No Keyboard
OpenBoot 3.15, 128 MB memory installed, Serial #nnnnnnnn.
Ethernet address 8:0:20:a5:d1:3b, Host ID: nnnnnnnn.

ok devalias
[...snip...]
cdrom /pci@1f,0/pci@1,1/ide@3/cdrom@2,0:f
disk /pci@1f,0/pci@1,1/ide@3/disk@0,0
disk3 /pci@1f,0/pci@1,1/ide@3/disk@3,0
disk2 /pci@1f,0/pci@1,1/ide@3/disk@2,0
```

```
disk1 /pci@1f,0/pci@1,1/ide@3/disk@1,0
disk0 /pci@1f,0/pci@1,1/ide@3/disk@0,0
[...snip...]

ok boot disk0 netbsd
Initializing Memory [...]
Boot device /pci/pci/ide@3/disk@0,0 File and args: netbsd
NetBSD IEEE 1275 Bootblock
>> NetBSD/sparc64 OpenFirmware Boot, Revision 1.13
>> (builds@b7.netbsd.org, Wed Jul 29 23:43:42 UTC 2009)
loadfile: reading header
elf64_exec: Booting [...]
symbols @ [....]
 Copyright (c) 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005,
     2006, 2007, 2008, 2009
     The NetBSD Foundation, Inc.  All rights reserved.
 Copyright (c) 1982, 1986, 1989, 1991, 1993
     The Regents of the University of California.  All rights reserved.
[...snip...]
```

А для второго диска:

```
ok boot disk2 netbsd
Initializing Memory [...]
Boot device /pci/pci/ide@3/disk@2,0: File and args:netbsd
NetBSD IEEE 1275 Bootblock
>> NetBSD/sparc64 OpenFirmware Boot, Revision 1.13
>> (builds@b7.netbsd.org, Wed Jul 29 23:43:42 UTC 2009)
loadfile: reading header
elf64_exec: Booting [...]
symbols @ [....]
 Copyright (c) 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005,
     2006, 2007, 2008, 2009
     The NetBSD Foundation, Inc.  All rights reserved.
 Copyright (c) 1982, 1986, 1989, 1991, 1993
     The Regents of the University of California.  All rights reserved.
[...snip...]
```

При каждой загрузке в выводе dmesg(8) ядра NetBSD должно появляться следующее:

```
Kernelized RAIDframe activated
raid0: RAID Level 1
raid0: Components: /dev/wd0a /dev/wd1a
raid0: Total Sectors: 19540864 (9541 MB)
boot device: raid0
root on raid0a dumps on raid0b
root file system type: ffs
```

Убедившись, что загрузка возможна с обоих дисков, проверяйте, что чётность RAID остаётся чистой после каждой перезагрузки:

```
# raidctl -v -s raid0
Components:
           /dev/wd0a: optimal
           /dev/wd1a: optimal
No spares.
[...snip...]
Component label for /dev/wd0a:
   Row: 0, Column: 0, Num Rows: 1, Num Columns: 2
   Version: 2, Serial Number: 2009122601, Mod Counter: 67
```

```
   Clean: No, Status: 0
   sectPerSU: 128, SUsPerPU: 1, SUsPerRU: 1
   Queue size: 100, blocksize: 512, numBlocks: 19540864
   RAID Level: 1
   Autoconfig: Yes
   Root partition: Yes
   Last configured as: raid0
Component label for /dev/wd1a:
   Row: 0, Column: 1, Num Rows: 1, Num Columns: 2
   Version: 2, Serial Number: 2009122601, Mod Counter: 67
   Clean: No, Status: 0
   sectPerSU: 128, SUsPerPU: 1, SUsPerRU: 1
   Queue size: 100, blocksize: 512, numBlocks: 19540864
   RAID Level: 1
   Autoconfig: Yes
   Root partition: Yes
   Last configured as: raid0
Parity status: clean
Reconstruction is 100% complete.
Parity Re-write is 100% complete.
Copyback is 100% complete.
```

Глава 17. Настройка менеджера логических томов NetBSD (LVM)

NetBSD LVM обеспечивает управление логическими томами в системах NetBSD с хорошо известным интерфейсом, таким же, как у инструментов Linux LVM2.

LVM в NetBSD построена на Linux lvm2tools и libdevmapper, а также на драйвере device-mapper для ядра, специально написанном для NetBSD и распространяемом по лицензии BSD.

Драйвер LVM позволяет пользователю эффективно и рационально управлять доступным дисковым пространством. Дисковое пространство нескольких дисков и разделов, называемое «физическими томами», можно добавлять в «группы томов» — пул доступного дискового пространства для «логических разделов», иначе называемых логическими томами.

Логические тома можно увеличивать и уменьшать по желанию с помощью утилит LVM.

Основной строительный блок — физический том. Это диск или часть диска, используемая для хранения данных.

Физические тома объединяются в группы томов, или VG. Обычно группы томов служат для объединения хранилища в пределах одной функциональной единицы. Поэтому типичные группы томов можно было бы назвать «Audio», «Multimedia» или «Documents». Разделяя требования к хранилищу по функциональному признаку, вы распространяете одинаковую отказоустойчивость и избыточность на всю функциональную единицу.

Для настройки LVM нужно выполнить следующие шаги:

1. Установите физические носители
2. Настройте поддержку в ядре
3. Настройте систему, установите инструменты
4. *Необязательный шаг*

   Создайте disklabel для каждого тома — участника LVM
5. Инициализируйте дисковые устройства LVM
6. Создайте группу томов из инициализированных дисков
7. Создайте логический том из созданной группы томов
8. Создайте файловую систему на новом устройстве LV
9. Смонтируйте файловую систему LV

В этом примере рассматривается настройка LVM в NetBSD/i386.

17.1. Анатомия менеджера логических томов NetBSD

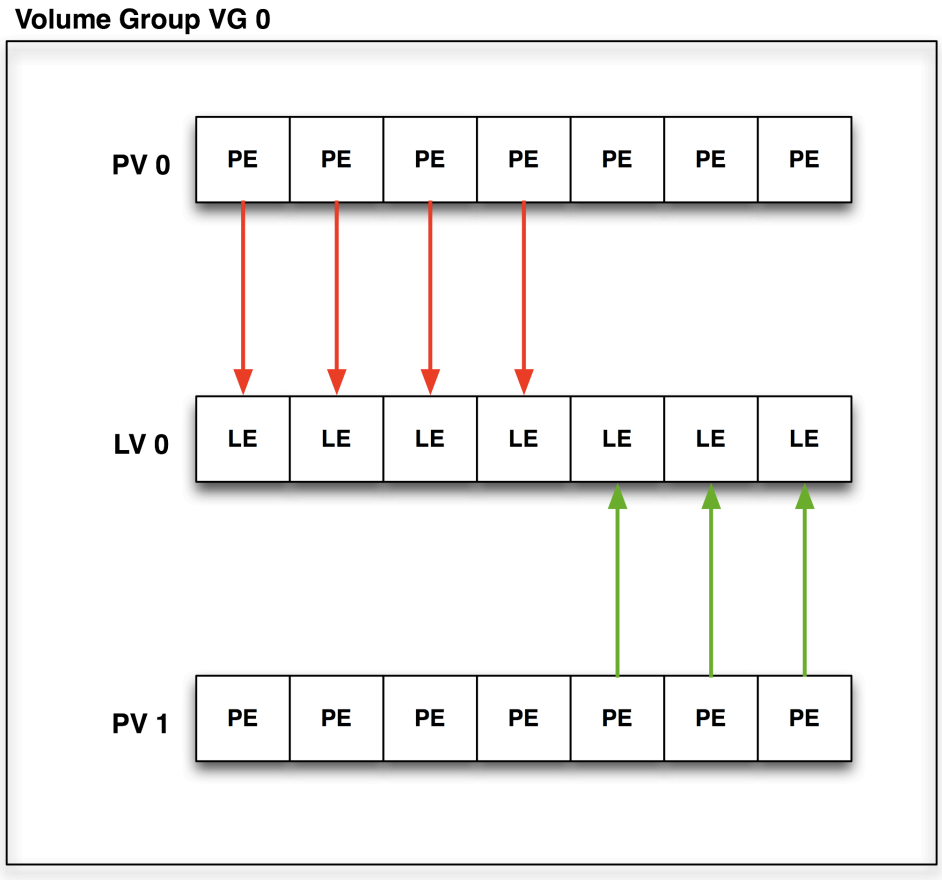


Рисунок 17.1. Анатомия управления логическими томами

1. **Группа томов**

   Группа томов — это пул дискового пространства, из которого пользователь создаёт логические тома и в который можно добавлять физические тома. Это основная единица администрирования в реализации LVM для NetBSD.

2. **Физический том**

   Физический том — основная единица структуры LVM. Каждый PV состоит из небольших участков дискового пространства, называемых физическими экстентами (Physical Extents). В каждой группе томов должен быть хотя бы один PV. PV можно создать на жёстких дисках или на устройствах, подобных жёсткому диску, — например raid, ccd или cgd.

3. **Логический том**

   Логический том — это логический раздел, созданный из дискового пространства, выделенного группе томов. LV можно отформатировать с помощью newfs и смонтировать, как любое другое псевдодисковое устройство. Инструменты LVM используют функциональность, предоставляемую драйвером device-mapper в ядре, для создания LV.

4. **Физические экстенты**

   Каждый физический том разбит на участки дискового пространства. Размер по умолчанию — 4 МБ. Размер каждого LV округляется до размера PE. LV создаётся отображением логических экстентов (Logical Extents) в LV на физические экстенты (Physical Extents) в группе томов.

5. **Логические экстенты**

Каждый логический том разбит на участки дискового пространства, называемые логическими экстентами. Размер экстента одинаков для всех логических томов в группе томов.

6. **Отображение физических экстентов**

   Каждый LV состоит из «LE», отображённых на «PE» посредством целевого отображения. В настоящее время определены следующие отображения.

   - **Линейное отображение**

     линейно назначит диапазон PE логическим экстентам.

     ```
     For example it can map 100 PEs from PV 1 to LV 1 and
                         another 100 PEs from PV 0.
     ```

   - **Полосное отображение (stripe)**

     будет чередовать участки логических экстентов между несколькими физическими томами.

7. **Снимки**

   Одна из возможностей LVM — «снимки» (snapshots). Хотя в стандартной NetBSD драйвер «fss» позволяет создавать снимки файловой системы на уровне файловой системы, механизм снимков в LVM позволяет администратору создать логическое блочное устройство, которое представляет точную копию логического тома, зафиксированную на некоторый момент времени. Этот механизм требует, чтобы снимок был сделан в момент, когда данные на логическом томе находятся в согласованном состоянии.

> **ПРЕДУПРЕЖДЕНИЕ**
>
> Функция снимков в LVM в NetBSD реализована не полностью, и её не следует использовать в рабочей эксплуатации.

17.2. Установка физического носителя

Этот шаг выполняется на ваше усмотрение и зависит от вашей платформы и имеющегося в вашем распоряжении оборудования. LVM можно использовать с разделами disklabel или даже со стандартными разделами, созданными с помощью fdisk.

Из моего вывода «dmesg»:

```
Disk #1:
            probe(esp0:0:0): max sync rate 10.00MB/s
            sd0 at scsibus0 target 0 lun 0: <SEAGATE, ST32430N SUN2.1G, 0444> SCSI2 0/direct
fixed
            sd0: 2049 MB, 3992 cyl, 9 head, 116 sec, 512 bytes/sect x 4197405 sectors

            Disk #2
            probe(esp0:1:0): max sync rate 10.00MB/s
            sd1 at scsibus0 target 1 lun 0: <SEAGATE, ST32430N SUN2.1G, 0444> SCSI2 0/direct
fixed
            sd1: 2049 MB, 3992 cyl, 9 head, 116 sec, 512 bytes/sect x 4197405 sectors

            Disk #3
            probe(esp0:2:0): max sync rate 10.00MB/s
            sd2 at scsibus0 target 2 lun 0: <SEAGATE, ST11200N SUN1.05, 9500> SCSI2 0/direct
fixed
            sd2: 1005 MB, 1872 cyl, 15 head, 73 sec, 512 bytes/sect x 2059140 sectors
```

```
            Disk #4
            probe(esp0:3:0): max sync rate 10.00MB/s
            sd3 at scsibus0 target 3 lun 0: <SEAGATE, ST11200N SUN1.05, 8808 > SCSI2 0
            sd3: 1005 MB, 1872 cyl, 15 head, 73 sec, 512 bytes/sect x 2059140 sectors
```

17.3. Настройка поддержки в ядре

Для поддержки устройств LVM нужна следующая директива конфигурации ядра. Она предоставляется в виде модуля ядра, поэтому в стандартное ядро NetBSD не требуется вносить дополнительные изменения.

```
      pseudo-device dm
```

Если вы не хотите пересобирать ядро только ради поддержки LVM, можно использовать модуль ядра dm. Модуль ядра devmapper можно загрузить в систему. Чтобы узнать текущее состояние модулей в ядре, используется **modstat**:

```
vm1# modstat
        NAME            CLASS   SOURCE  REFS    SIZE    REQUIRES
        cd9660          vfs     filesys 0       21442   -
        coredump        misc    filesys 1       2814    -
        exec_elf32      misc    filesys 0       6713    coredump
        exec_script     misc    filesys 0       1091    -
        ffs             vfs     boot    0       163040  -
        kernfs          vfs     filesys 0       10201   -
        ptyfs           vfs     filesys 0       7852    -
```

При выполнении **modload dm** будет загружен модуль ядра dm:

```
vm1# modstat
          NAME            CLASS   SOURCE  REFS    SIZE    REQUIRES
          cd9660          vfs     filesys 0       21442   -
          coredump        misc    filesys 1       2814    -
          dm              misc    filesys 0       14448   -
          exec_elf32      misc    filesys 0       6713    coredump
          exec_script     misc    filesys 0       1091    -
          ffs             vfs     boot    0       163040  -
          kernfs          vfs     filesys 0       10201   -
          ptyfs           vfs     filesys 0       7852    -
```

17.4. Создание disklabel для каждого физического тома LVM

Для каждого диска, служащего физическим томом в LVM, потребуется создать специальную файловую систему. В этом примере мне нужно будет создать disklabel для:

```
            /dev/rsd0d

            /dev/rsd1d

            /dev/rsd2d

            /dev/rsd3d
```

Следует учитывать, что можно использовать драйвер vnd в NetBSD, чтобы обычное пространство файловой системы выглядело в системе как дисковое устройство.

> **ПРИМЕЧАНИЕ**
>
> Всегда помните: disklabel нужно создавать для символьного устройства, а не для блочного, в `/dev/r{s,w}d*`

> **ПРИМЕЧАНИЕ**
>
> На всех платформах, кроме amd64 и i386, где для этого используется раздел `d`, срез `c` символизирует весь раздел NetBSD и зарезервирован.

Вероятно, вы захотите удалить все существующие disklabel на дисках — физических томах в LVM. Это можно сделать одним из двух способов с помощью команды dd(1):

```
# dd if=/dev/zero of=/dev/rsd0d bs=8k count=1
            # dd if=/dev/zero of=/dev/rsd1d bs=8k count=1
            # dd if=/dev/zero of=/dev/rsd2d bs=8k count=1
            # dd if=/dev/zero of=/dev/rsd3d bs=8k count=1
```

Если в вашей платформе для разбиения дисков используется MBR (Master Boot Record), так что разделы NetBSD занимают лишь часть всего диска, а другие операционные системы, например Windows или Linux, используют другие его части, вы можете уничтожить MBR и все разделы на диске с помощью команды:

```
# dd if=/dev/zero of=/dev/rsd0d bs=8k count=1
            # dd if=/dev/zero of=/dev/rsd1d bs=8k count=1
            # dd if=/dev/zero of=/dev/rsd2d bs=8k count=1
            # dd if=/dev/zero of=/dev/rsd3d bs=8k count=1
```

Это сделает все данные на всём диске недоступными. Учтите, что весь диск — это раздел `d` на i386 (и некоторых других платформах), а в остальных случаях (например, на sparc) — `c`. См. параметр sysctl «kern.rawpartition»: «3» означает «d», «2» означает «c».

Disklabel по умолчанию для диска будет выглядеть примерно так:

```
# disklabel -r sd0
            [...snip...]
            bytes/sector: 512
            sectors/track: 63
            tracks/cylinder: 16
            sectors/cylinder: 1008
            cylinders: 207
            total sectors: 208896
            rpm: 3600
            interleave: 1
            trackskew: 0
            cylinderskew: 0
            headswitch: 0           # microseconds
            track-to-track seek: 0  # microseconds
            drivedata: 0
```

```
4 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
a:    208896         0     4.2BSD      0     0     0  # (Cyl.      0 -    207*)
d:    208896         0     unused      0     0        # (Cyl.      0 -    207*)
```

Вам потребуется создать на разделе NetBSD один «срез», который занимает весь раздел. Срез должен начинаться как минимум через два сектора после конца части диска, занятой disklabel. На i386 это сектор «63». Поэтому значение «size» должно быть равно «total sectors» минус 2× «sectors». Отредактируйте disklabel соответствующим образом:

```
# disklabel -e sd0
```

> **ПРИМЕЧАНИЕ**
>
> Смещение среза типа «4.2BSD» должно быть кратно значению «sectors».

> **ПРИМЕЧАНИЕ**
>
> Перед редактированием disklabel не забудьте выполнить **export EDITOR=[путь к вашему любимому редактору]**.

> **ПРИМЕЧАНИЕ**
>
> Срез должен иметь тип файловой системы `4.2BSD`.

Поскольку на этом разделе будет только один срез, можно задействовать срез `d` (обычно зарезервированный для символических целей). Измените disklabel следующим образом:

```
3 partitions:
#        size   offset    fstype   [fsize bsize   cpg]
d:  4197403       65      4.2BSD                     # (Cyl. 1 - 4020*)
```

При желании можно настроить для использования срез, отличный от `d`, — просто соответствующим образом измените приведённое ниже:

```
3 partitions:
#        size   offset    fstype   [fsize bsize   cpg]
a:  4197403       65      4.2BSD                     # (Cyl. 1 - 4020*)
c:  4197405       0       unused     1024  8192        # (Cyl. 0 - 4020*)
```

Не забудьте записать метку, когда закончите. Если disklabel не пройдёт свои проверки на разумность, он не примет вашу метку и предложит отредактировать её заново.

17.5. Создание физических томов

После того как все диски правильно размечены, вам нужно создать на них физические тома. Каждый раздел или диск, добавляемый в LVM, должен иметь в начале заголовок физического тома. В этом заголовке хранится вся информация, например группа томов, к которой принадлежит физический том.

```
# lvm pvcreate /dev/rwd1[ad]
```

Состояние физического тома можно посмотреть с помощью команды pvdisplay.

```
# lvm pvdisplay
```

17.6. Создание группы томов

После того как все диски правильно размечены с заголовком физического тома, из них нужно создать группу томов. Группа томов — это пул PE, из которого администратор может создавать логические тома «разделы».

```
# lvm vgcreate vg0 /dev/rwd1[ad]
```

- vg0 — имя группы томов
- /dev/rwd1[ad] — физический том

Впоследствии группу томов можно расширить или уменьшить с помощью команд vgextend и vgreduce. Эти команды добавляют физические тома в VG.

```
# lvm vgextend vg0 /dev/rwd1[ad]

# lvm vgreduce vg0 /dev/rwd1[ad]
```

Состояние группы томов можно посмотреть с помощью команды vgdisplay.

```
# lvm vgdisplay vg0
```

17.7. Создание логического тома

После создания группы томов администратор может создавать тома «логических разделов».

```
# lvm lvcreate  -L 20M -n lv1 vg0
```

- vg0 — имя группы томов
- -L 20M — размер логического тома
- -n lv1 — имя логического тома

Логический том можно впоследствии расширить или уменьшить с помощью команд lvextend и lvreduce.

```
# lvm lvextend -L+20M /dev/vg0/lv1
```

```
# lvm lvreduce -L-20M /dev/vg0/lv1
```

> **ПРИМЕЧАНИЕ**
>
> Чтобы уменьшить раздел lv, сначала нужно уменьшить файловую систему с помощью resize_ffs(8) (которая по состоянию на NetBSD 9.0 пока не поддерживает уменьшение FFSv2).

Состояние логического тома можно посмотреть с помощью команды lvdisplay.

```
# lvm lvdisplay lv0/lv1
```

После перезагрузки все работающие LV в определённой группе томов можно активировать командой

```
# lvm vgchange -a y
```

17.8. Пример: LVM с группами томов, расположенными на raid1

Мотивация использования диска raid 1 в качестве диска физического тома для группы томов — надёжность диска. С PV на диске raid 1 логические тома можно использовать даже после отказа диска.

17.8.1. Загрузка драйвера Device-Mapper

Прежде чем начать работу с инструментами LVM, нужно убедиться, что драйвер dm в NetBSD был правильно скомпилирован в ядро или загружен как модуль. Проще всего узнать, доступен ли драйвер dm, запустив modstat. Дополнительные сведения см. в разделе Настройка поддержки в ядре.

17.8.2. Подготовка установки raid1

Следуя примеру конфигурации raid, описанному в разделе Конфигурация Raid 1, пользователь настроит чистый дисковый массив raid1 из двух дисков в режиме зеркала.

```
# vi /var/tmp/raid0.conf
START array
1 2 0

START disks
/dev/wd2a
/dev/wd1a

START layout
128 1 1 1

START queue
fifo 100
```

```
# raidctl -v -C /var/tmp/raid0.conf raid0
raid0: Component /dev/wd1a being configured at col: 0
Column: 0 Num Columns: 0
Version: 0 Serial Number: 0 Mod Counter: 0
Clean: No Status: 0
Column out of alignment for: /dev/wd2a
Number of columns do not match for: /dev/wd2a
/dev/wd2a is not clean!
raid0: Component /dev/wd1a being configured at col: 1
Column: 0 Num Columns: 0
Version: 0 Serial Number: 0 Mod Counter: 0
Clean: No Status: 0
Column out of alignment for: /dev/wd1a
Number of columns do not match for: /dev/wd1a
/dev/wd1a is not clean!
raid0: There were fatal errors
raid0: Fatal errors being ignored.
raid0: RAID Level 1
raid0: Components: /dev/wd2a /dev/wd1a
raid0: Total Sectors: 19540864 (9541 MB)
# raidctl -v -I 2004082401 raid0
# raidctl -v -i raid0
Initiating re-write of parity
# tail -1 /var/log/messages
raid0: Error re-writing parity!
# raidctl -v -s raid0
Components:
/dev/wd2a: optimal
/dev/wd1a: optimal
No spares.
Component label for /dev/wd1a:
Row: 0, Column: 1, Num Rows: 1, Num Columns: 2
Version: 2, Serial Number: 2004082401, Mod Counter: 7
Clean: No, Status: 0
sectPerSU: 128, SUsPerPU: 1, SUsPerRU: 1
Queue size: 100, blocksize: 512, numBlocks: 19540864
RAID Level: 1
Autoconfig: No
Root partition: No
Last configured as: raid0
Parity status: DIRTY
Reconstruction is 100% complete.
Parity Re-write is 100% complete.
Copyback is 100% complete.
Component label for /dev/wd2a:
Row: 0, Column: 1, Num Rows: 1, Num Columns: 2
Version: 2, Serial Number: 2004082401, Mod Counter: 7
```

```
Clean: No, Status: 0
sectPerSU: 128, SUsPerPU: 1, SUsPerRU: 1
Queue size: 100, blocksize: 512, numBlocks: 19540864
RAID Level: 1
Autoconfig: No
Root partition: No
Last configured as: raid0
Parity status: DIRTY
Reconstruction is 100% complete.
Parity Re-write is 100% complete.
Copyback is 100% complete.
```

Рисунок 17.2. Пример конфигурации raid 1

После настройки raid нужно создать disklabel на raid-диске.
В i386:

```
 # disklabel -r -e -I raid0
type: RAID
disk: raid
label: fictitious
flags:
bytes/sector: 512
sectors/track: 128
tracks/cylinder: 8
sectors/cylinder: 1024
cylinders: 19082
total sectors: 19540864
rpm: 3600
interleave: 1
trackskew: 0
cylinderskew: 0
headswitch: 0 # microseconds
track-to-track seek: 0 # microseconds
drivedata: 0

#        size    offset     fstype [fsize bsize cpg/sgs]
a:  19540789        65     4.2BSD      0     0     0  # (Cyl.      0 - 18569)
d:  19540864         0     unused      0     0        # (Cyl.      0 - 19082*)
```

На sparc64:

```
# disklabel -r -e -I raid0
[...snip...]
total sectors: 19539968
[...snip...]
2 partitions:
#        size    offset     fstype [fsize bsize cpg/sgs]
a:  19540793        65     4.2BSD      0     0     0  # (Cyl.      0 -  18799)
c:  19539968         0     unused      0     0        # (Cyl.      0 -  19081)
```

Разделы следует создавать со смещением 65, так как секторы с номерами меньше 65 помечены как доступные только для чтения и поэтому не могут быть перезаписаны.

17.8.3. Создание PV и VG на raid-диске

Физические тома можно создавать на любом устройстве, подобном диску, и на любом его разделе: так, на sparc64 можно использовать разделы a или d, но не раздел c. PV помечает выбранный раздел как используемый LVM и добавляет в него необходимую информацию.

PV создаётся на символьном устройстве диска, как и любая другая дисковая операция в NetBSD.

```
# lvm pvcreate /dev/rraid0a
```

В нашем примере я создам группу томов vg00. Первый параметр команды vgcreate — имя группы томов, второй — PV, созданный на raid. Если позже вы обнаружите, что размера VG недостаточно и требуется больше места, его можно будет добавить с помощью команды **vgextend**.

```
# lvm vgcreate vg00 /dev/rraid0a

# lvm vgextend vg00 /dev/rraid1a
```

> **ПРЕДУПРЕЖДЕНИЕ**
>
> Если вы добавите в группу томов PV, не основанный на raid, ваши данные больше не будут в безопасности. Поэтому, если вы хотите сохранить данные в безопасности, добавляйте в VG PV на основе raid.

17.8.4. Создание LV из VG, расположенной на raid-диске

В нашем примере мы создадим логический том с именем lv0. Если позже вы обнаружите, что размера LV вам недостаточно, его можно увеличить с помощью команды **lvresize**.

> **ПРИМЕЧАНИЕ**
>
> При изменении размера LV необходимо также изменить размер файловой системы, иначе при монтировании LV вы не увидите никаких изменений в файловой системе.

> **ПРЕДУПРЕЖДЕНИЕ**
>
> Учтите, что для уменьшения LV сначала нужно уменьшить файловую систему (а уменьшение файловых систем FFSv2 по состоянию на NetBSD 9.0 пока не поддерживается).
>
> Это означает, что для файловых систем FFSv2 параметр -L-* в NetBSD недоступен.

```
# lvm lvcreate -n lv0 -L 2G vg00

# lvm lvresize -L+2G vg00/lv0
```

Все узлы устройств lv создаются в каталоге `/dev/vg00/`. Файловую систему на LV можно создать этой командой. После создания файловой системы LV можно смонтировать в систему.

```
# newfs -O2 /dev/vg00/rlv0
```

```
# mount /dev/vg00/lv0 /mnt/
```

17.8.5. Интеграция LV в систему

Для правильной интеграции LVM необходимо включить rc.d-скрипт lvm, который обнаруживает LV при загрузке и активирует их. Кроме того, нужно добавить запись для логического тома в файл `/etc/fstab`.

```
# cat /etc/rc.conf
[snip]
lvm=yes
```

```
# cat /etc/fstab
/dev/wd0a               /       ffs     rw              1 1
/dev/vg00/lv0           /lv0/   ffs     rw              1 1
[snip]
```

Глава 18. Подключаемые модули аутентификации (PAM)

18.1. Общие сведения

В этой статье описаны основные принципы и механизмы библиотеки Pluggable Authentication Modules (PAM), а также объясняется, как настроить PAM, как интегрировать PAM в приложения и как писать модули PAM.

О лицензии этой главы см. в разделе D.3.2, «Лицензия Networks Associates Technology на статью о PAM».

18.2. Введение

Библиотека Pluggable Authentication Modules (PAM) — это обобщённый API для служб, связанных с аутентификацией, который позволяет системному администратору добавлять новые методы аутентификации, просто устанавливая новые модули PAM, и изменять политики аутентификации, редактируя файлы конфигурации.

PAM была определена и разработана в 1995 году Vipin Samar и Charlie Lai из Sun Microsystems и с тех пор мало изменилась. В 1997 году консорциум Open Group опубликовал предварительную спецификацию X/Open Single Sign-on (XSSO), которая стандартизировала API PAM и добавила расширения для единого (точнее, интегрированного) входа в систему. На момент написания этой статьи спецификация ещё не была принята в качестве стандарта.

Хотя эта статья посвящена в первую очередь FreeBSD 5.x и NetBSD 3.x, которые обе используют OpenPAM, она в равной степени применима к FreeBSD 4.x, использующей Linux-PAM, и к другим операционным системам, таким как Linux и Solaris™.

18.3. Термины и соглашения

18.3.1. Определения

Терминология, связанная с PAM, довольно запутана. Ни оригинальная статья Samar и Lai, ни спецификация XSSO не содержали попытки формально определить термины для различных действующих лиц и сущностей, участвующих в PAM, а те термины, которые в них всё же используются (но не определяются), иногда вводят в заблуждение и неоднозначны. Первой попыткой установить последовательную и непротиворечивую терминологию стала аналитическая записка, написанная Andrew G. Morgan (автором Linux-PAM) в 1999 году. Хотя предложенная Morgan терминология стала огромным шагом вперёд, по мнению автора этой статьи, она отнюдь не идеальна. Далее следует попытка, во многом навеянная Morgan, определить точные и однозначные термины для всех действующих лиц и сущностей, участвующих в PAM.

учётная запись

Набор учётных данных, которые заявитель запрашивает у арбитра.

заявитель

Пользователь или сущность, запрашивающая аутентификацию.

арбитр

Пользователь или сущность, обладающая привилегиями, необходимыми для проверки учётных данных заявителя, и полномочиями удовлетворить или отклонить запрос.

цепочка

Последовательность модулей, которые вызываются в ответ на запрос PAM. Цепочка содержит сведения о порядке вызова модулей, о том, какие аргументы им передавать и как интерпретировать результаты.

клиент

Приложение, которое инициирует запрос аутентификации от имени заявителя и получает от него необходимые данные аутентификации.

функциональная группа

Одна из четырёх основных групп функций, предоставляемых PAM: аутентификация, управление учётными записями, управление сеансами и обновление маркера аутентификации.

модуль

Набор одной или нескольких связанных функций, реализующих определённую функциональную группу аутентификации, собранных в один (обычно динамически загружаемый) двоичный файл и идентифицируемых одним именем.

политика

Полный набор операторов конфигурации, описывающих, как обрабатывать запросы PAM для конкретной службы. Политика обычно состоит из четырёх цепочек — по одной на каждую функциональную группу, хотя некоторые службы используют не все четыре функциональные группы.

сервер

Приложение, действующее от имени арбитра: взаимодействует с клиентом, получает данные аутентификации, проверяет учётные данные заявителя и удовлетворяет или отклоняет запросы.

служба

Класс серверов, предоставляющих сходные или связанные функции и требующих сходной аутентификации. Политики PAM определяются для каждой службы отдельно, поэтому все серверы, указывающие одно и то же имя службы, подчиняются одной и той же политике.

сеанс

Контекст, в рамках которого сервер оказывает услуги заявителю. Одна из четырёх функциональных групп PAM — управление сеансами — занимается исключительно созданием и разрушением этого контекста.

маркер

Порция информации, связанная с учётной записью, например пароль или парольная фраза, которую заявитель должен предъявить, чтобы подтвердить свою личность.

транзакция

Последовательность запросов от одного и того же заявителя к одному и тому же экземпляру одного и того же сервера, начинающаяся с аутентификации и установки сеанса и заканчивающаяся завершением сеанса.

18.3.2. Примеры использования

В этом разделе на нескольких простых примерах поясняются значения некоторых из определённых выше терминов.

18.3.2.1. Клиент и сервер совпадают

В этом простом примере `alice` с помощью su(1) переключается на `root`.

```
$ whoami
alice
$ ls -l `which su`
-r-sr-xr-x  1 root  wheel  10744 Dec  6 19:06 /usr/bin/su
$ su -
Password: xi3kiune
# whoami
root
```

- Заявитель — `alice`.
- Учётная запись — `root`.
- Процесс su(1) одновременно является и клиентом, и сервером.
- Маркер аутентификации — `xi3kiune`.
- Арбитр — `root`; именно поэтому su(1) является setuid-программой `root`.

18.3.2.2. Клиент и сервер разделены

В примере ниже `eve` пытается установить соединение ssh(1) с `login.example.com`, просит войти в систему как `bob` — и ей это удаётся. Bob мог бы выбрать пароль получше!

```
$ whoami
eve
$ ssh bob@login.example.com
bob@login.example.com's password: god
Last login: Thu Oct 11 09:52:57 2001 from 192.168.0.1
NetBSD 3.0 (LOGIN) #1: Thu Mar 10 18:22:36 WET 2005

Welcome to NetBSD!
$
```

- Заявитель — `eve`.
- Клиент — процесс ssh(1), запущенный у Eve.
- Сервер — процесс sshd(8) на `login.example.com`
- Учётная запись — `bob`.
- Маркер аутентификации — `god`.
- Хотя в этом примере это не показано, арбитром является `root`.

18.3.2.3. Пример политики

Ниже приведена политика по умолчанию в FreeBSD для `sshd`:

```
sshd    auth            required        pam_nologin.so  no_warn
sshd    auth            required        pam_unix.so     no_warn try_first_pass
sshd    account         required        pam_login_access.so
sshd    account         required        pam_unix.so
```

```
sshd    session         required        pam_lastlog.so  no_fail
sshd    password        required        pam_permit.so
```

- Эта политика применяется к службе `sshd` (которая не обязательно ограничивается сервером sshd(8)).
- `auth`, `account`, `session` и `password` — это функциональные группы.
- `pam_nologin.so`, `pam_unix.so`, `pam_login_access.so`, `pam_lastlog.so` и `pam_permit.so` — это модули. Из этого примера ясно, что `pam_unix.so` реализует как минимум две функциональные группы (аутентификацию и управление учётными записями).

Политики PAM в FreeBSD и NetBSD имеют некоторые различия:

- По умолчанию вся конфигурация находится в каталоге `/etc/pam.d`.
- Если конфигурация отсутствует, вы не получите доступа к системе — в отличие от FreeBSD, где по умолчанию действует политика, разрешающая аутентификацию.
- Для аутентификации NetBSD требует, чтобы присутствовал хотя бы один модуль `required`, `requisite` или `binding`.

18.4. Основы PAM

18.4.1. Функциональные группы и примитивы

API PAM предоставляет шесть различных примитивов аутентификации, сгруппированных в четыре функциональные группы; они описаны ниже.

`auth`

Аутентификация. Эта функциональная группа занимается аутентификацией заявителя и установлением учётных данных учётной записи. Она предоставляет два примитива:

- pam_authenticate(3) аутентифицирует заявителя — обычно запрашивая маркер аутентификации и сравнивая его со значением, хранящимся в базе данных или полученным от сервера аутентификации.
- pam_setcred(3) устанавливает учётные данные записи, такие как идентификатор пользователя, членство в группах и ограничения ресурсов.

`account`

Управление учётными записями. Эта функциональная группа решает не связанные с аутентификацией вопросы доступности учётной записи, например ограничения доступа по времени суток или по нагрузке на сервер. Она предоставляет один примитив:

- pam_acct_mgmt(3) проверяет, доступна ли запрошенная учётная запись.

`session`

Управление сеансами. Эта функциональная группа выполняет задачи, связанные с установкой и завершением сеанса, например учёт входов в систему. Она предоставляет два примитива:

- pam_open_session(3) выполняет задачи, связанные с установкой сеанса: добавляет запись в базы данных `utmp` и `wtmp`, запускает агент SSH и т. п.
- pam_close_session(3) выполняет задачи, связанные с завершением сеанса: добавляет запись в базы данных `utmp` и `wtmp`, останавливает агент SSH и т. п.

`password`

Управление паролями. Эта функциональная группа служит для смены маркера аутентификации, связанного с учётной записью, — потому что срок его действия истёк или потому что пользователь хочет его изменить. Она предоставляет один примитив:

- pam_chauthtok(3) изменяет маркер аутентификации, при необходимости проверяя, что его достаточно трудно угадать, что он не использовался ранее, и т. п.

18.4.2. Модули

Модули — одно из центральных понятий в PAM; в конце концов, именно они образуют букву «M» в слове «PAM». Модуль PAM — это самодостаточный фрагмент программного кода, реализующий примитивы одной или нескольких функциональных групп для одного конкретного механизма; так, для функциональной группы аутентификации возможными механизмами являются, например, база данных паролей UNIX®, NIS, LDAP и Radius.

18.4.2.1. Именование модулей

FreeBSD и NetBSD реализуют каждый механизм в одном модуле с именем `pam_`*`mechanism`*`.so` (например, `pam_unix.so` для механизма UNIX®). В других реализациях иногда имеются отдельные модули для отдельных функциональных групп, и в имя модуля включается как имя механизма, так и имя функциональной группы. Приведём один пример: в Solaris™ есть модуль `pam_dial_auth.so.1`, который обычно используется для аутентификации пользователей дозвона. Кроме того, почти каждый модуль имеет страницу руководства с тем же именем, то есть pam_unix(8) объясняет, как работает модуль `pam_unix.so`.

18.4.2.2. Версии модулей

Первоначальная реализация PAM в FreeBSD, основанная на Linux-PAM, не использовала номера версий для модулей PAM. Это часто создавало проблемы со старыми приложениями, которые могли быть скомпонованы со старыми версиями системных библиотек, поскольку не было способа загрузить подходящую версию требуемых модулей.

OpenPAM, напротив, ищет модули с тем же номером версии, что и библиотека PAM (в настоящее время 2 в FreeBSD и 0 в NetBSD), и прибегает к модулю без номера версии только в том случае, если не удалось загрузить модуль с версией. Благодаря этому можно поставлять устаревшие модули для устаревших приложений и в то же время позволять новым (или только что собранным) приложениям использовать самые новые модули.

Хотя модули PAM в Solaris™ обычно имеют номер версии, настоящего версионирования нет, поскольку номер является частью имени модуля и должен указываться в конфигурации.

18.4.2.3. Путь к модулям

Единого каталога для хранения модулей PAM нет. В FreeBSD они находятся в `/usr/lib`, а в NetBSD их можно найти в `/usr/lib/security`.

18.4.3. Цепочки и политики

Когда сервер инициирует транзакцию PAM, библиотека PAM пытается загрузить политику для службы, указанной в вызове pam_start(3). Политика определяет, как следует обрабатывать запросы аутентификации, и задаётся в файле конфигурации. В этом состоит второе центральное понятие PAM: возможность для администратора настраивать политику безопасности системы (в широком смысле слова), просто редактируя текстовый файл.

Политика состоит из четырёх цепочек — по одной для каждой из четырёх функциональных групп PAM. Каждая цепочка представляет собой последовательность операторов конфигурации; каждый из них задаёт модуль, который нужно вызвать, некоторые (необязательные) параметры, передаваемые модулю, и управляющий флаг, описывающий, как интерпретировать код возврата модуля.

Чтобы разобраться в файлах конфигурации PAM, необходимо понимать управляющие флаги. Существует несколько различных управляющих флагов:

`binding`
: Если модуль завершается успешно и ни один из предыдущих модулей в цепочке не завершился с ошибкой, цепочка немедленно прекращается и запрос удовлетворяется. Если модуль завершается с ошибкой, остаток цепочки выполняется, но в итоге запрос отклоняется.

  Этот управляющий флаг был введён компанией Sun в Solaris™ 9 (SunOS™ 5.9) и поддерживается также в OpenPAM.

`required`
: Если модуль завершается успешно, выполняется остаток цепочки и запрос удовлетворяется, если только не завершится с ошибкой какой-либо другой модуль. Если модуль завершается с ошибкой, остаток цепочки также выполняется, но в итоге запрос отклоняется.

`requisite`
: Если модуль завершается успешно, выполняется остаток цепочки и запрос удовлетворяется, если только не завершится с ошибкой какой-либо другой модуль. Если модуль завершается с ошибкой, цепочка немедленно прекращается и запрос отклоняется.

`sufficient`
: Если модуль завершается успешно и ни один из предыдущих модулей в цепочке не завершился с ошибкой, цепочка немедленно прекращается и запрос удовлетворяется. Если модуль завершается с ошибкой, его результат игнорируется и выполняется остаток цепочки.

  Поскольку семантика этого флага может быть несколько запутанной, особенно когда он применяется к последнему модулю в цепочке, рекомендуется вместо него использовать управляющий флаг `binding`, если реализация его поддерживает.

`optional`
: Модуль выполняется, но его результат игнорируется. Если все модули в цепочке помечены как `optional`, все запросы всегда будут удовлетворяться.

Когда сервер вызывает один из шести примитивов PAM, PAM извлекает цепочку для функциональной группы, к которой относится примитив, и вызывает каждый модуль из цепочки в порядке их перечисления, пока не дойдёт до конца или не определит, что дальнейшая обработка не нужна (либо потому, что модуль `binding` или `sufficient` завершился успешно, либо потому, что модуль `requisite` завершился с ошибкой). Запрос удовлетворяется тогда и только тогда, когда был вызван хотя бы один модуль и все модули, кроме необязательных, завершились успешно.

Обратите внимание, что один и тот же модуль может — хотя это бывает не очень часто — несколько раз присутствовать в одной цепочке. Например, модуль, который ищет имена пользователей и пароли на сервере каталогов, можно вызвать несколько раз с разными параметрами, задающими разные серверы каталогов, к которым нужно обращаться. PAM рассматривает разные вхождения одного и того же модуля в одной цепочке как разные, не связанные друг с другом модули.

18.4.4. Транзакции

Ниже описан жизненный цикл типичной транзакции PAM. Учтите, что при сбое любого из этих шагов сервер должен сообщить клиенту подходящее сообщение об ошибке и прервать транзакцию.

1. При необходимости сервер получает учётные данные арбитра посредством механизма, не зависящего от PAM, — чаще всего благодаря тому, что он запущен от имени `root` или является setuid-программой `root`.
2. Сервер вызывает pam_start(3), чтобы инициализировать библиотеку PAM, указать имя своей службы и целевую учётную запись, а также зарегистрировать подходящую функцию диалога.
3. Сервер получает различные сведения о транзакции (например, имя пользователя-заявителя и имя хоста, на котором работает клиент) и передаёт их в PAM с помощью pam_set_item(3).
4. Сервер вызывает pam_authenticate(3) для аутентификации заявителя.
5. Сервер вызывает pam_acct_mgmt(3), чтобы проверить, что запрошенная учётная запись доступна и действительна. Если пароль верен, но срок его действия истёк, pam_acct_mgmt(3) вернёт `PAM_NEW_AUTHTOK_REQD` вместо `PAM_SUCCESS`.
6. Если предыдущий шаг вернул `PAM_NEW_AUTHTOK_REQD`, сервер вызывает pam_chauthtok(3), чтобы заставить клиента сменить маркер аутентификации для запрошенной учётной записи.
7. Теперь, когда заявитель должным образом аутентифицирован, сервер вызывает pam_setcred(3), чтобы установить учётные данные запрошенной учётной записи. Он может это сделать, поскольку действует от имени арбитра и обладает учётными данными арбитра.
8. После установления правильных учётных данных сервер вызывает pam_open_session(3), чтобы открыть сеанс.
9. Затем сервер выполняет ту службу, которую запросил клиент, — например, предоставляет заявителю оболочку.
10. Завершив обслуживание клиента, сервер вызывает pam_close_session(3), чтобы закрыть сеанс.
11. Наконец, сервер вызывает pam_end(3), чтобы уведомить библиотеку PAM о завершении работы и о том, что она может освободить все ресурсы, выделенные в ходе транзакции.

18.5. Настройка PAM

18.5.1. Файлы политики PAM

18.5.1.1. Файл `/etc/pam.conf`

Традиционный файл политики PAM — `/etc/pam.conf`. Этот файл содержит все политики PAM вашей системы. Каждая строка файла описывает один шаг цепочки, как показано ниже:

```
login   auth    required        pam_nologin.so  no_warn
```

Поля следуют в таком порядке: имя службы, имя функциональной группы, управляющий флаг, имя модуля и аргументы модуля. Любые дополнительные поля интерпретируются как дополнительные аргументы модуля.

Для каждой пары «служба / функциональная группа» строится отдельная цепочка, поэтому порядок, в котором идут строки для одной и той же службы и функциональной группы, важен, а порядок, в котором перечислены отдельные службы и функциональные группы, — нет. В примерах из оригинальной статьи о PAM строки конфигурации были сгруппированы по функциональным

группам, и поставляемый с Solaris™ файл `pam.conf` до сих пор устроен так же, но в конфигурации, поставляемой с FreeBSD, строки сгруппированы по службам. Годится и тот, и другой способ; и тот, и другой одинаково разумен.

18.5.1.2. Каталог `/etc/pam.d`

OpenPAM и Linux-PAM поддерживают альтернативный механизм конфигурации, который является предпочтительным в FreeBSD и NetBSD. При такой схеме каждая политика хранится в отдельном файле, имя которого совпадает с именем службы, к которой она применяется. Эти файлы находятся в `/etc/pam.d/`.

В этих файлах политик для отдельных служб только четыре поля вместо пяти в `pam.conf`: поле имени службы опускается. Поэтому вместо строки из примера `pam.conf` из предыдущего раздела в файле `/etc/pam.d/login` будет такая строка:

```
auth    required        pam_nologin.so  no_warn
```

Благодаря такому упрощённому синтаксису можно использовать одну и ту же политику для нескольких служб, связав каждое имя службы с одним и тем же файлом политики. Например, чтобы использовать одну политику для служб `su` и `sudo`, можно поступить так:

```
# cd /etc/pam.d
# ln -s su sudo
```

Это работает потому, что имя службы определяется по имени файла, а не задаётся в файле политики, поэтому один и тот же файл можно использовать для нескольких служб с разными именами.

Поскольку политика каждой службы хранится в отдельном файле, механизм `pam.d` также значительно упрощает установку дополнительных политик для сторонних пакетов программ.

18.5.1.3. Порядок поиска политики

Как показано выше, политики PAM могут находиться в нескольких местах. Если для конкретной службы файл конфигурации не найден, вместо него используется `/etc/pam.d/other`. Если и этого файла нет, в `/etc/pam.conf` выполняется поиск записей, соответствующих указанной службе, а при их отсутствии — записей службы «other».

Важно понимать, что система конфигурации PAM строится вокруг цепочек.

18.5.2. Разбор строки конфигурации

Как объяснено в разделе Файлы политики PAM, каждая строка в `/etc/pam.conf` состоит из четырёх или более полей: имя службы, имя функциональной группы, управляющий флаг, имя модуля и ноль или более аргументов модуля.

Имя службы, как правило (хотя и не всегда), совпадает с именем приложения, к которому относится оператор. Если вы не уверены, обратитесь к документации соответствующего приложения, чтобы выяснить, какое имя службы оно использует.

Учтите, что при использовании `/etc/pam.d/` вместо `/etc/pam.conf` имя службы задаётся именем файла политики и опускается в самих строках конфигурации, которые тогда начинаются с имени функциональной группы.

Функциональная группа — одно из четырёх ключевых слов, описанных в разделе Функциональные группы и примитивы.

Точно так же управляющий флаг — одно из четырёх ключевых слов, описанных в разделе Цепочки и политики, которое указывает, как интерпретировать код возврата модуля. Linux-PAM поддерживает альтернативный синтаксис, позволяющий задать действие, сопоставляемое каждому возможному коду возврата, но его следует избегать, поскольку он нестандартен и тесно связан со способом, которым Linux-PAM диспетчеризует вызовы служб (а он сильно отличается от способа, используемого Solaris™ и OpenPAM). Неудивительно, что OpenPAM этот синтаксис не поддерживает.

18.5.3. Политики

Чтобы правильно настроить PAM, необходимо понимать, как интерпретируются политики.

Когда приложение вызывает pam_start(3), библиотека PAM загружает политику для указанной службы и строит четыре цепочки модулей (по одной на каждую функциональную группу). Если одна или несколько из этих цепочек пусты, вместо них подставляются соответствующие цепочки из политики службы `other`.

Когда приложение позже вызывает один из шести примитивов PAM, библиотека PAM извлекает цепочку для соответствующей функциональной группы и вызывает нужную функцию службы в каждом модуле из цепочки в том порядке, в котором они перечислены в конфигурации. После каждого вызова функции службы тип модуля и код ошибки, возвращённый функцией службы, используются для определения дальнейших действий. За несколькими исключениями, о которых говорится ниже, действует следующая таблица:

Таблица 18.1. Сводка по выполнению цепочки PAM

	`PAM_SUCCESS`	`PAM_IGNORE`	`other`
binding	if (!fail) break;	-	fail = true;
required	-	-	fail = true;
requisite	-	-	fail = true; break;
sufficient	if (!fail) break;	-	-
optional	-	-	-

Если значение `fail` истинно в конце цепочки или при достижении «break», диспетчер возвращает код ошибки, возвращённый первым модулем, который завершился с ошибкой. В противном случае он возвращает `PAM_SUCCESS`.

Первое примечательное исключение состоит в том, что код ошибки `PAM_NEW_AUTHTOK_REQD` трактуется как успех, за тем исключением, что если ни один модуль не завершился с ошибкой и хотя бы один модуль вернул `PAM_NEW_AUTHTOK_REQD`, диспетчер вернёт `PAM_NEW_AUTHTOK_REQD`.

Второе исключение состоит в том, что pam_setcred(3) обрабатывает модули `binding` и `sufficient` так, как если бы они были `required`.

Третье и последнее исключение состоит в том, что pam_chauthtok(3) проходит всю цепочку дважды (один раз для предварительных проверок и один раз для фактической установки пароля), причём на предварительном этапе он обрабатывает модули `binding` и `sufficient` так, как если бы они были `required`.

18.6. Модули PAM

18.6.1. Общие модули

18.6.1.1. pam_deny(8)

Модуль pam_deny(8) — один из самых простых доступных модулей; на любой запрос он отвечает `PAM_AUTH_ERR`. Он полезен для быстрого отключения службы (добавьте его в начало каждой цепочки) или для завершения цепочек из модулей `sufficient`.

18.6.1.2. pam_echo(8)

Модуль pam_echo(8) просто передаёт свои аргументы функции диалога как сообщение `PAM_TEXT_INFO`. В основном он полезен для отладки, но может также служить для вывода сообщений вроде «Несанкционированный доступ будет преследоваться по закону» перед началом процедуры аутентификации.

18.6.1.3. pam_exec(8)

Модуль pam_exec(8) считает свой первый аргумент именем программы, которую нужно выполнить, а остальные аргументы передаёт этой программе как аргументы командной строки. Одно из возможных применений — запуск при входе в систему программы, монтирующей домашний каталог пользователя.

18.6.1.4. pam_ftpusers(8)

Модуль pam_ftpusers(8) завершается успешно тогда и только тогда, когда пользователь указан в `/etc/ftpusers`. В настоящее время в NetBSD этот модуль не понимает расширенный синтаксис ftpd(8), но в будущем это будет исправлено.

18.6.1.5. pam_group(8)

Модуль pam_group(8) принимает или отклоняет заявителей в зависимости от их членства в определённой группе (обычно `wheel` для su(1)). Он предназначен в первую очередь для сохранения традиционного поведения BSD-утилиты su(1), но имеет и много других применений, например исключение определённых групп пользователей из конкретной службы.

В NetBSD есть аргумент `authenticate`, при котором пользователя просят пройти аутентификацию с помощью его собственного пароля.

18.6.1.6. pam_guest(8)

Модуль pam_guest(8) разрешает вход гостем с использованием фиксированных имён входа. К паролю можно предъявить различные требования, но по умолчанию допускается любой пароль, если имя входа соответствует гостевой учётной записи. Модуль pam_guest(8) легко использовать для реализации анонимного входа по FTP.

18.6.1.7. pam_krb5(8)

Модуль pam_krb5(8) предоставляет функции для проверки подлинности пользователя и установки специфичных для пользователя учётных данных с помощью Kerberos 5. Он запрашивает у пользователя пароль и получает новый Kerberos TGT для принципала. TGT проверяется путём получения служебного билета для локального хоста. Вновь полученные учётные данные сохраняются в кэше учётных данных, а переменная среды KRB5CCNAME устанавливается соответствующим образом. Кэш учётных данных следует уничтожать при выходе из системы с помощью kdestroy(1).

18.6.1.8. pam_ksu(8)

Модуль pam_ksu(8) предоставляет только службы аутентификации для Kerberos 5, чтобы определить, имеет ли заявитель право получить привилегии целевой учётной записи.

18.6.1.9. pam_lastlog(8)

Модуль pam_lastlog(8) предоставляет только службы управления сеансами. Он записывает сведения о сеансе в базы данных utmp(5), utmpx(5), wtmp(5), wtmpx(5), lastlog(5) и lastlogx(5).

18.6.1.10. pam_login_access(8)

Модуль pam_login_access(8) реализует примитив управления учётными записями и обеспечивает соблюдение ограничений на вход, заданных в таблице login.access(5).

18.6.1.11. pam_nologin(8)

Модуль pam_nologin(8) отказывает во входе всем, кроме root, если существует файл `/var/run/nologin`. Обычно этот файл создаёт shutdown(8), когда до запланированного выключения остаётся менее пяти минут.

18.6.1.12. pam_permit(8)

Модуль pam_permit(8) — один из самых простых доступных модулей; на любой запрос он отвечает `PAM_SUCCESS`. Он полезен как заполнитель для служб, в которых одна или несколько цепочек иначе оказались бы пустыми.

18.6.1.13. pam_radius(8)

Модуль pam_radius(8) предоставляет службы аутентификации на основе протокола RADIUS (Remote Authentication Dial In User Service).

18.6.1.14. pam_rhosts(8)

Модуль pam_rhosts(8) предоставляет только службы аутентификации. Он сообщает об успехе тогда и только тогда, когда идентификатор целевого пользователя не равен 0 и удалённый хост и пользователь указаны в `/etc/hosts.equiv` или в файле `~/.rhosts` целевого пользователя.

18.6.1.15. pam_rootok(8)

Модуль pam_rootok(8) сообщает об успехе тогда и только тогда, когда реальный идентификатор пользователя процесса, который его вызывает (предполагается, что этот процесс запущен заявителем), равен 0. Это полезно для служб, не связанных с сетью, таких как su(1) или passwd(1), к которым `root` должен иметь автоматический доступ.

18.6.1.16. pam_securetty(8)

Модуль pam_securetty(8) предоставляет только службы учётных записей. Он используется, когда заявитель пытается пройти аутентификацию как суперпользователь, а процесс привязан к незащищённому терминалу TTY.

18.6.1.17. pam_self(8)

Модуль pam_self(8) сообщает об успехе тогда и только тогда, когда имя заявителя совпадает с именем целевой учётной записи. Он наиболее полезен для служб, не связанных с сетью, таких как su(1), где личность заявителя можно легко проверить.

18.6.1.18. pam_ssh(8)

Модуль pam_ssh(8) предоставляет и службы аутентификации, и службы сеансов. Служба аутентификации позволяет пользователям, у которых в каталоге `~/.ssh` есть защищённые парольной фразой секретные ключи SSH, аутентифицироваться, вводя свою парольную фразу. Служба сеансов запускает ssh-agent(1) и заранее загружает в него ключи, расшифрованные на этапе аутентификации. Эта возможность особенно полезна при локальном входе — как в X (с помощью xdm(1) или другого менеджера входа в X, поддерживающего PAM), так и на консоли.

По сути этот модуль реализует схему аутентификации по паролю. Следует позаботиться о том, чтобы использовать этот модуль только в защищённом сеансе (защищённый TTY, шифрованный сеанс и т. п.), иначе парольная фраза SSH пользователя может быть скомпрометирована.

Следует также дополнительно обдумать использование pam_ssh(8). Пользователи часто полагают, что прав доступа к файлам достаточно для защиты их ключей SSH, и поэтому используют слабые парольные фразы или не используют их вовсе. Поскольку у системного администратора нет действенных средств контроля качества парольных фраз SSH, это потенциально подвергает систему рискам безопасности.

18.6.1.19. pam_unix(8)

Модуль pam_unix(8) реализует традиционную аутентификацию по паролю UNIX®, используя getpwnam(3) в FreeBSD или getpwnam_r(3) в NetBSD для получения пароля целевой учётной записи и сравнения его с паролем, указанным заявителем. Он также предоставляет службы управления учётными записями (соблюдение сроков действия учётных записей и паролей) и службы смены пароля. Пожалуй, это самый полезный модуль, поскольку подавляющему большинству администраторов нужно сохранить привычное поведение хотя бы для некоторых служб.

18.6.2. Модули PAM, специфичные для NetBSD

18.6.2.1. pam_skey(8)

Модуль pam_skey(8) реализует методы аутентификации по одноразовым паролям S/Key (OTP), используя базу данных `/etc/skeykeys`.

18.7. Программирование приложений с PAM

Этот раздел ещё не написан.

18.8. Программирование модулей PAM

Этот раздел ещё не написан.

18.9. Пример приложения с PAM

Ниже приведена минимальная реализация su(1) с использованием PAM. Обратите внимание, что в ней используется специфичная для OpenPAM функция диалога openpam_ttyconv(3), прототип которой объявлен в `security/openpam.h`. Если вы хотите собрать это приложение в системе с другой библиотекой PAM, вам придётся предоставить собственную функцию диалога. Надёжную функцию диалога на удивление трудно реализовать; функция, приведённая в подразделе Пример функции диалога PAM, — хорошая отправная точка, но её не следует использовать в реальных приложениях.

```
#include <sys/param.h>
#include <sys/wait.h>

#include <err.h>
#include <pwd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <syslog.h>
#include <unistd.h>

#include <security/pam_appl.h>
#include <security/openpam.h>   /* for openpam_ttyconv() */

extern char **environ;

static pam_handle_t *pamh;
static struct pam_conv pamc;

static void
usage(void)
{

        fprintf(stderr, "Usage: su [login [args]]\n");
        exit(1);
}

int
main(int argc, char *argv[])
{
        char hostname[MAXHOSTNAMELEN];
        const char *user, *tty;
        char **args, **pam_envlist, **pam_env;
        struct passwd *pwd;
        int o, pam_err, status;
        pid_t pid;

        while ((o = getopt(argc, argv, "h")) != -1)
                switch (o) {
                case 'h':
                default:
                        usage();
                }

        argc -= optind;
        argv += optind;

        if (argc > 0) {
                user = *argv;
                --argc;
                ++argv;
        } else {
                user = "root";
        }

        /* initialize PAM */
        pamc.conv = &openpam_ttyconv;
        pam_start("su", user, &pamc, &pamh);

        /* set some items */
        gethostname(hostname, sizeof(hostname));
        if ((pam_err = pam_set_item(pamh, PAM_RHOST, hostname)) != PAM_SUCCESS)
                goto pamerr;
        user = getlogin();
```

```
        if ((pam_err = pam_set_item(pamh, PAM_RUSER, user)) != PAM_SUCCESS)
                goto pamerr;
        tty = ttyname(STDERR_FILENO);
        if ((pam_err = pam_set_item(pamh, PAM_TTY, tty)) != PAM_SUCCESS)
                goto pamerr;

        /* authenticate the applicant */
        if ((pam_err = pam_authenticate(pamh, 0)) != PAM_SUCCESS)
                goto pamerr;
        if ((pam_err = pam_acct_mgmt(pamh, 0)) == PAM_NEW_AUTHTOK_REQD)
                pam_err = pam_chauthtok(pamh, PAM_CHANGE_EXPIRED_AUTHTOK);
        if (pam_err != PAM_SUCCESS)
                goto pamerr;

        /* establish the requested credentials */
        if ((pam_err = pam_setcred(pamh, PAM_ESTABLISH_CRED)) != PAM_SUCCESS)
                goto pamerr;

        /* authentication succeeded; open a session */
        if ((pam_err = pam_open_session(pamh, 0)) != PAM_SUCCESS)
                goto pamerr;

        /* get mapped user name; PAM may have changed it */
        pam_err = pam_get_item(pamh, PAM_USER, (const void **)&user);
        if (pam_err != PAM_SUCCESS || (pwd = getpwnam(user)) == NULL)
                goto pamerr;

        /* export PAM environment */
        if ((pam_envlist = pam_getenvlist(pamh)) != NULL) {
                for (pam_env = pam_envlist; *pam_env != NULL; ++pam_env) {
                        putenv(*pam_env);
                        free(*pam_env);
                }
                free(pam_envlist);
        }

        /* build argument list */
        if ((args = calloc(argc + 2, sizeof *args)) == NULL) {
                warn("calloc()");
                goto err;
        }
        *args = pwd->pw_shell;
        memcpy(args + 1, argv, argc * sizeof *args);

        /* fork and exec */
        switch ((pid = fork())) {
        case -1:
                warn("fork()");
                goto err;
        case 0:
                /* child: give up privs and start a shell */

                /* set uid and groups */
                if (initgroups(pwd->pw_name, pwd->pw_gid) == -1) {
                        warn("initgroups()");
                        _exit(1);
                }
                if (setgid(pwd->pw_gid) == -1) {
                        warn("setgid()");
                        _exit(1);
                }
                if (setuid(pwd->pw_uid) == -1) {
                        warn("setuid()");
                        _exit(1);
                }
```

```
                execve(*args, args, environ);
                warn("execve()");
                _exit(1);
        default:
                /* parent: wait for child to exit */
                waitpid(pid, &status, 0);

                /* close the session and release PAM resources */
                pam_err = pam_close_session(pamh, 0);
                pam_end(pamh, pam_err);

                exit(WEXITSTATUS(status));
        }

pamerr:
        fprintf(stderr, "Sorry\n");
err:
        pam_end(pamh, pam_err);
        exit(1);
}
```

18.10. Пример модуля PAM

Ниже приведена минимальная реализация pam_unix(8), предоставляющая только службы аутентификации. Она должна собираться и работать с большинством реализаций PAM, но использует расширения OpenPAM, если они доступны: обратите внимание на использование pam_get_authtok(3), что чрезвычайно упрощает запрос пароля у пользователя.

```
#include <sys/param.h>

#include <pwd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#include <security/pam_modules.h>
#include <security/pam_appl.h>

#ifndef _OPENPAM
static char password_prompt[] = "Password:";
#endif

#ifndef PAM_EXTERN
#define PAM_EXTERN
#endif

PAM_EXTERN int
pam_sm_authenticate(pam_handle_t *pamh, int flags,
        int argc, const char *argv[])
{
#ifndef _OPENPAM
        const void *ptr;
        const struct pam_conv *conv;
        struct pam_message msg;
        const struct pam_message *msgp;
        struct pam_response *resp;
#endif
        struct passwd *pwd;
        const char *user;
        char *crypt_password, *password;
```

```
        int pam_err, retry;

        /* identify user */
        if ((pam_err = pam_get_user(pamh, &user, NULL)) != PAM_SUCCESS)
                return (pam_err);
        if ((pwd = getpwnam(user)) == NULL)
                return (PAM_USER_UNKNOWN);

        /* get password */
#ifndef _OPENPAM
        pam_err = pam_get_item(pamh, PAM_CONV, &ptr);
        if (pam_err != PAM_SUCCESS)
                return (PAM_SYSTEM_ERR);
        conv = ptr;
        msg.msg_style = PAM_PROMPT_ECHO_OFF;
        msg.msg = password_prompt;
        msgp = &msg;
#endif
        password = NULL;
        for (retry = 0; retry < 3; ++retry) {
#ifdef _OPENPAM
                pam_err = pam_get_authtok(pamh, PAM_AUTHTOK,
                    (const char **)&password, NULL);
#else
                resp = NULL;
                pam_err = (*conv->conv)(1, &msgp, &resp, conv->appdata_ptr);
                if (resp != NULL) {
                        if (pam_err == PAM_SUCCESS)
                                password = resp->resp;
                        else
                                free(resp->resp);
                        free(resp);
                }
#endif
                if (pam_err == PAM_SUCCESS)
                        break;
        }
        if (pam_err == PAM_CONV_ERR)
                return (pam_err);
        if (pam_err != PAM_SUCCESS)
                return (PAM_AUTH_ERR);

        /* compare passwords */
        if ((!pwd->pw_passwd[0] && (flags & PAM_DISALLOW_NULL_AUTHTOK)) ||
            (crypt_password = crypt(password, pwd->pw_passwd)) == NULL ||
            strcmp(crypt_password, pwd->pw_passwd) != 0)
                pam_err = PAM_AUTH_ERR;
        else
                pam_err = PAM_SUCCESS;
#ifndef _OPENPAM
        free(password);
#endif
        return (pam_err);
}

PAM_EXTERN int
pam_sm_setcred(pam_handle_t *pamh, int flags,
        int argc, const char *argv[])
{

        return (PAM_SUCCESS);
}

PAM_EXTERN int
pam_sm_acct_mgmt(pam_handle_t *pamh, int flags,
```

```
        int argc, const char *argv[])
{

        return (PAM_SUCCESS);
}

PAM_EXTERN int
pam_sm_open_session(pam_handle_t *pamh, int flags,
        int argc, const char *argv[])
{

        return (PAM_SUCCESS);
}

PAM_EXTERN int
pam_sm_close_session(pam_handle_t *pamh, int flags,
        int argc, const char *argv[])
{

        return (PAM_SUCCESS);
}

PAM_EXTERN int
pam_sm_chauthtok(pam_handle_t *pamh, int flags,
        int argc, const char *argv[])
{

        return (PAM_SERVICE_ERR);
}

#ifdef PAM_MODULE_ENTRY
PAM_MODULE_ENTRY("pam_unix");
#endif
```

18.11. Пример функции диалога PAM

Приведённая ниже функция диалога — сильно упрощённая версия openpam_ttyconv(3) из OpenPAM. Она полностью работоспособна и должна дать читателю хорошее представление о том, как должна вести себя функция диалога, но она слишком проста для реального использования. Даже если вы не используете OpenPAM, можете свободно загрузить исходный код и адаптировать openpam_ttyconv(3) к своим задачам; мы считаем, что она настолько надёжна, насколько это разумно возможно для функции диалога, ориентированной на tty.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include <security/pam_appl.h>

int
converse(int n, const struct pam_message **msg,
        struct pam_response **resp, void *data)
{
        struct pam_response *aresp;
        char buf[PAM_MAX_RESP_SIZE];
        int i;

        data = data;
        if (n <= 0 || n > PAM_MAX_NUM_MSG)
```

```
                return (PAM_CONV_ERR);
        if ((aresp = calloc(n, sizeof *aresp)) == NULL)
                return (PAM_BUF_ERR);
        for (i = 0; i < n; ++i) {
                aresp[i].resp_retcode = 0;
                aresp[i].resp = NULL;
                switch (msg[i]->msg_style) {
                case PAM_PROMPT_ECHO_OFF:
                        aresp[i].resp = strdup(getpass(msg[i]->msg));
                        if (aresp[i].resp == NULL)
                                goto fail;
                        break;
                case PAM_PROMPT_ECHO_ON:
                        fputs(msg[i]->msg, stderr);
                        if (fgets(buf, sizeof buf, stdin) == NULL)
                                goto fail;
                        aresp[i].resp = strdup(buf);
                        if (aresp[i].resp == NULL)
                                goto fail;
                        break;
                case PAM_ERROR_MSG:
                        fputs(msg[i]->msg, stderr);
                        if (strlen(msg[i]->msg) > 0 &&
                            msg[i]->msg[strlen(msg[i]->msg) - 1] != '\n')
                                fputc('\n', stderr);
                        break;
                case PAM_TEXT_INFO:
                        fputs(msg[i]->msg, stdout);
                        if (strlen(msg[i]->msg) > 0 &&
                            msg[i]->msg[strlen(msg[i]->msg) - 1] != '\n')
                                fputc('\n', stdout);
                        break;
                default:
                        goto fail;
                }
        }
        *resp = aresp;
        return (PAM_SUCCESS);
 fail:
        for (i = 0; i < n; ++i) {
                if (aresp[i].resp != NULL) {
                        memset(aresp[i].resp, 0, strlen(aresp[i].resp));
                        free(aresp[i].resp);
                }
        }
        memset(aresp, 0, n * sizeof *aresp);
        *resp = NULL;
        return (PAM_CONV_ERR);
}
```

18.12. Дополнительная литература

Библиография

Статьи

[sun-pam] *Службы входа, независимые от технологий аутентификации*. Vipin Samar и Charlie Lai. Sun Microsystems.

[opengroup-singlesignon] *Предварительная спецификация X/Open Single Sign-on*. The Open Group. 1-85912-144-6. Июнь 1997 г.

[kernelorg-pamdraft] *Pluggable Authentication Modules*. Andrew G. Morgan. 6 октября 1999 г.

Руководства пользователя

[sun-pamadmin] *Администрирование PAM*. Sun Microsystems.

Связанные веб-страницы

[openpam-website] *Домашняя страница OpenPAM*. Dag-Erling Smørgrav. ThinkSec AS.

[linuxpam-website] *Домашняя страница Linux-PAM*. Andrew G. Morgan.

[solarispam-website] *Домашняя страница PAM для Solaris*. Sun Microsystems.

Глава 19. Настройка производительности NetBSD

19.1. Введение

19.1.1. Обзор

В этом разделе рассматриваются разнообразные вопросы настройки производительности. Он охватывает эту тему от точки зрения системного администратора до точки зрения системного программиста. Само искусство настройки производительности очень старо. Настроить что-либо означает заставить это работать более эффективно: идёт ли речь о техническом сервере на базе NetBSD или о пылесосе, цель — улучшить что-то, будь то способ выполнения чего-либо, принцип работы или устройство.

19.1.1.1. Что такое настройка производительности?

Взгляд с высоты птичьего полёта подсказывает, что всё, что мы делаем, ориентировано на задачи; это относится и к системе NetBSD. При загрузке система автоматически начинает выполнять множество задач. Пользователь, входя в систему, обычно имеет целый набор задач, которые нужно выполнить. Однако в рамках этих документов настройка производительности означает строго одно: повышение эффективности работы системы NetBSD.

Когда человек думает о «настройке», ему чаще всего приходит в голову какое-нибудь повышение скорости или уменьшение размера ядра; хотя это и способы повышения производительности, они не единственные цели, которых администратор может добиваться ради повышения эффективности. Для наших целей настройка производительности означает следующее: *обеспечить работу системы NetBSD в оптимальном состоянии.*

Это может означать самые разные вещи, не обязательно ускорение. Хороший пример — параметры форматирования файловой системы: на системе с большим количеством мелких файлов (скажем, с репозиторием исходных текстов) администратору может понадобиться увеличить число inode, уменьшив их размер (например, до 1024k), и тем самым увеличить их количество. В этом случае число inode возросло, однако это избавляет администратора от неприятных сообщений об исчерпании inode, что в конечном счёте делает систему эффективнее.

Настройка обычно сводится к поиску и устранению узких мест. В большинстве случаев такие узкие места оказываются ложными: например, выпуск Mozilla, который не слишком хорошо справляется с апплетами Java, может заставить Mozilla нагружать процессор, особенно если апплеты сделаны некачественно. Случаи, когда процессы, кажется, уходят в никуда и съедают процессор, почти всегда решаются командой kill. Однако бывает, что на устранение узких мест требуется гораздо больше времени: скажем, сервер, синхронизируемый через rsync, просто становится всё больше и больше. Постепенно производительность начинает падать, и администратору, возможно, придётся что-то предпринять для ускорения, однако ситуация несопоставима, скажем, с аварией вроде мгновенного скачка загрузки процессора.

19.1.1.2. Когда нужно настраивать?

Многим пользователям NetBSD редко приходится настраивать систему. Ядро GENERIC может работать вполне нормально, а компоновка и конфигурация системы могут вполне справляться со своей задачей. В то же время как правило полезно знать, как настраивать систему. Чаще всего

настройка требуется из-за внезапно возникшей проблемы с узким местом (которая может возникнуть случайно) или постепенной потери производительности. В каком-то смысле это случается со всеми когда-нибудь: один процесс, съедающий процессор, — такое же узкое место, как и постепенный рост объёма подкачки. Поэтому вопрос должен стоять не столько о том, когда настраивать, сколько о том, когда учиться настройке.

Ещё один повод заняться настройкой: если вы можете настроить систему профилактически (и думаете, что это может понадобиться), сделайте это. Пример — система, которой нужно было быстро перезагружаться. Вместо того чтобы ждать, я сделал всё возможное, чтобы сократить ядро и убедиться, что не работает абсолютно ничего лишнего; я даже убрал драйверы, у которых были устройства, но которые никогда не использовались (lp). В результате время перезагрузки сократилось почти на две трети. В конечном счёте настроить систему до того, как это стало проблемой, было разумным шагом.

19.1.1.3. Что эти документы не охватывают

Прежде чем закончить введение, считаю важным отметить, чего эти документы не будут охватывать. Это руководство касается только основной системы NetBSD. Иными словами, в нём не рассматривается настройка конфигурации веб-сервера для повышения его производительности; однако может быть упомянуто, как настроить NetBSD, чтобы она лучше работала в качестве веб-сервера. Логика здесь простая: веб-серверы, программное обеспечение баз данных и т. п. — сторонние и почти неограниченны по числу. Я мог бы легко увязнуть в деталях, которые не относятся к системе NetBSD. К тому же почти всё стороннее программное обеспечение имеет собственную документацию по настройке.

19.1.1.4. Как построены примеры

Поскольку документации в виде страниц руководства предостаточно, обсуждаются только используемые параметры и аргументы, причём с примерами. В некоторых случаях материал сокращён для краткости и не рассматривается подробно, потому что его попросту слишком много. Например, не будет обсуждаться каждая запись драйвера устройства в ядре, но будет приведён пример того, как определить, нужен ли конкретной системе тот или иной драйвер. Ничто в этом руководстве не является незыблемым: настройка и производительность очень субъективны; скорее, это руководство, из которого читатель узнает, что умеют некоторые из доступных ему инструментов.

19.2. Что учитывать при настройке

Настраивать систему не так уж сложно, если подходить к этому проактивно. В этом документе настройка рассматривается с позиции «до того, как это возникло». Хотя настройка в свободное время заметно проще, чем, скажем, на сервере, который почти полностью загружен и имеет 0,1 % простоя, есть всё же несколько вещей, которые стоит обдумать перед тем, как приступать к настройке, — желательно ещё до установки системы.

19.2.1. Общая конфигурация системы

Разумеется, то, как настроена система, имеет огромное значение. Иногда можно упустить мелкие детали, которые на самом деле способны вызвать долговременные проблемы с производительностью.

19.2.1.1. Файловые системы и диски

То, как файловая система распределена по дискам, очень важно. На аппаратных RAID-системах это не так важно, но многие пользователи NetBSD используют NetBSD именно на старом оборудовании, где аппаратный RAID попросту недоступен. Мысль о том, чтобы `/` находился близко к первому диску, хороша, но, например, если есть несколько дисков, любой из которых может стать первым, будет ли самый производительный из них тем, на котором окажется `/`? В связи с этим: разумно ли выделять `/usr`? Будет ли система интенсивно использовать, скажем, `/usr/pkgsrc`? Возможно, имеет смысл поставить быстрый диск и смонтировать его в `/usr/pkgsrc`, а возможно, и нет. Как и всё в настройке производительности, это субъективно.

19.2.1.2. Настройка подкачки

Что касается размера подкачки, существуют три школы мысли, а насчёт использования нескольких файлов подкачки с приоритетами и того, как это делать, — около пятидесяти. В вопросах размера подкачки у производителей (по крайней мере у большинства коммерческих) обычно есть собственные формулы для каждой ОС. Например, для определённой версии HP-UX с определённой версией Oracle формула была такой:

```
2.5 GB * Number_of_processor
```

Ну, всё это на самом деле зависит от того, как используется база данных и насколько она велика: например, если она настолько большая, что её приходится распределять, эта формула плохо подходит.

Следующая школа мысли относительно размера подкачки несколько странная, но в ней есть смысл: она говорит, что, если возможно, нужно получить эталонный объём памяти, используемой системой. Выглядит это примерно так:

1. Запустите машину и оцените общую потребность в памяти, запустив одновременно всё, что когда-либо может понадобиться. Базы данных, веб-серверы… что угодно. Суммируйте объём.
2. Добавьте несколько МБ запаса.
3. Вычтите из этого итога объём физической оперативной памяти.

Если оставшийся объём в три раза превышает размер физической памяти, подумайте о том, чтобы добавить памяти. Проблема, конечно, в том, чтобы понять, что нужно и сколько места это займёт. Есть у этого метода и ещё один недостаток: некоторые программы ведут себя неважно. Вопиющий пример плохо работающего программного обеспечения — веб-браузеры. В некоторых версиях Netscape при возникновении неполадки браузер имел обыкновение уходить в разгон и съедать место подкачки. Так что чем больше свободного места, тем больше времени, чтобы его убить.

Наконец, последний, но не менее важный — проверенный временем метод PHYSICAL_RAM * 2. На современных машинах и даже на более старых (разумеется, при ограниченном круге задач) он, похоже, работает лучше всего.

В целом трудно сказать, когда начнётся подкачка. Даже на небольших машинах с 16 МБ памяти (и меньше) NetBSD всегда хорошо работала у большинства людей, пока не запущено плохо ведущее себя программное обеспечение.

19.2.2. Системные службы

На серверах системные службы оказывают большое влияние. Чтобы они работали наилучшим образом, почти всегда требуется какое-то изменение на сетевом уровне или принципиальное повышение скорости базовой системы (что, собственно, и составляет предмет этого раздела). Бывают

случаи, когда службы можно улучшить простыми решениями. Пример: ftp-сервер начинает работать медленнее, а тут выходит новый выпуск ftp-сервера, поставляемого с системой, который, как оказалось, работает быстрее. Обновив программное обеспечение ftp, мы получаем прирост производительности.

Ещё один хороший пример, касающийся служб, — извечный вопрос: «Использовать inetd или не использовать inetd?» Отличный пример службы — pop3. Соединения pop3 вполне могут забить inetd. При этом сама служба pop3 начинает постепенно деградировать, а другие службы, мультиплексируемые через inetd, тоже деградируют (в некоторых случаях сильнее, чем pop3). Может помочь настройка pop3 так, чтобы она работала вне inetd и самостоятельно.

19.2.3. Ядро NetBSD

Ядро NetBSD, очевидно, играет ключевую роль в том, насколько хорошо работает система; пересборка и настройка ядра рассматриваются далее в тексте, но здесь стоит обсудить это на высоком уровне.

Настройка ядра NetBSD действительно затрагивает три основные области:

1. удаление ненужных драйверов
2. настройка параметров
3. системные настройки

19.2.3.1. Удаление ненужных драйверов

Удаление из ядра ненужных драйверов даёт несколько результатов: во-первых, система загружается быстрее, поскольку ядро меньше; во-вторых, опять же из-за меньшего размера ядра больше памяти остаётся пользователям и процессам; в-третьих, ядро, как правило, реагирует быстрее.

19.2.3.2. Настройка параметров

Настройка таких параметров, как включение и отключение отдельных подсистем, конкретного оборудования и файловых систем, также может повысить производительность примерно так же, как удаление ненужных драйверов. Очень простой пример — FTP-сервер, который отдаёт только ftp-файлы, и ничего больше. На таком сервере достаточно поддержки собственной файловой системы и, возможно, нескольких параметров, помогающих ускорить работу. Зачем ему, например, поддержка NTFS? К тому же при необходимости поддержку NTFS можно добавить позже. В противоположном случае рабочей станции может потребоваться поддержка множества различных типов файловых систем для совместного использования и доступа к файлам.

19.2.3.3. Системные настройки

Общесистемные настройки контролируются ядром; примеры — параметры файловых систем, сетевые параметры и основные параметры ядра, такие как максимальное число процессов. Почти все системные настройки можно хотя бы просмотреть или изменить с помощью механизма sysctl. Примеры использования sysctl приведены далее.

19.3. Средства визуального мониторинга

NetBSD поставляется вместе с системой с набором средств мониторинга производительности. Большинство этих средств распространены во всех системах UNIX. В этом разделе приведены примеры использования некоторых средств с расшифровкой вывода.

19.3.1. Монитор процессов top

Монитор top делает именно то, что заявлено: он показывает процессы, нагружающие процессор. Чтобы запустить монитор, просто введите top в приглашении. Без аргументов вывод должен выглядеть так:

```
load averages:  0.09,  0.12,  0.08                                      20:23:41
21 processes:  20 sleeping, 1 on processor
CPU states:  0.0% user,  0.0% nice,  0.0% system,  0.0% interrupt,  100% idle
Memory: 15M Act, 1104K Inact, 208K Wired, 22M Free, 129M Swap free

  PID USERNAME PRI NICE   SIZE   RES STATE     TIME   WCPU    CPU COMMAND
13663 root       2    0  1552K 1836K sleep     0:08  0.00%  0.00% httpd
  127 root      10    0   129M 4464K sleep     0:01  0.00%  0.00% mount_mfs
22591 root       2    0   388K 1156K sleep     0:01  0.00%  0.00% sshd
  108 root       2    0   132K  472K sleep     0:01  0.00%  0.00% syslogd
22597 jrf       28    0   156K  616K onproc    0:00  0.00%  0.00% top
22592 jrf       18    0   828K 1128K sleep     0:00  0.00%  0.00% tcsh
  203 root      10    0   220K  424K sleep     0:00  0.00%  0.00% cron
    1 root      10    0   312K  192K sleep     0:00  0.00%  0.00% init
  205 root       3    0    48K  432K sleep     0:00  0.00%  0.00% getty
  206 root       3    0    48K  424K sleep     0:00  0.00%  0.00% getty
  208 root       3    0    48K  424K sleep     0:00  0.00%  0.00% getty
  207 root       3    0    48K  424K sleep     0:00  0.00%  0.00% getty
13667 nobody     2    0  1660K 1508K sleep     0:00  0.00%  0.00% httpd
 9926 root       2    0   336K  588K sleep     0:00  0.00%  0.00% sshd
  200 root       2    0    76K  456K sleep     0:00  0.00%  0.00% inetd
  182 root       2    0    92K  436K sleep     0:00  0.00%  0.00% portsentry
  180 root       2    0    92K  436K sleep     0:00  0.00%  0.00% portsentry
13666 nobody    -4    0  1600K 1260K sleep     0:00  0.00%  0.00% httpd
```

Утилита top хорошо подходит для поиска процессов, нагружающих процессор, «убежавших» процессов или групп процессов, которые могут создавать проблемы. Показанный выше вывод говорит о том, что эта система в хорошем состоянии. Теперь следующий вывод покажет совсем иные результаты:

```
load averages:  0.34,  0.16,  0.13                                      21:13:47
25 processes:  24 sleeping, 1 on processor
CPU states:  0.5% user,  0.0% nice,  9.0% system,  1.0% interrupt, 89.6% idle
Memory: 20M Act, 1712K Inact, 240K Wired, 30M Free, 129M Swap free

  PID USERNAME PRI NICE   SIZE   RES STATE     TIME   WCPU    CPU COMMAND
 5304 jrf       -5    0    56K  336K sleep     0:04 66.07% 19.53% bonnie
 5294 root       2    0   412K 1176K sleep     0:02  1.01%  0.93% sshd
  108 root       2    0   132K  472K sleep     1:23  0.00%  0.00% syslogd
  187 root       2    0  1552K 1824K sleep     0:07  0.00%  0.00% httpd
 5288 root       2    0   412K 1176K sleep     0:02  0.00%  0.00% sshd
 5302 jrf       28    0   160K  620K onproc    0:00  0.00%  0.00% top
 5295 jrf       18    0   828K 1116K sleep     0:00  0.00%  0.00% tcsh
 5289 jrf       18    0   828K 1112K sleep     0:00  0.00%  0.00% tcsh
  127 root      10    0   129M 8388K sleep     0:00  0.00%  0.00% mount_mfs
  204 root      10    0   220K  424K sleep     0:00  0.00%  0.00% cron
    1 root      10    0   312K  192K sleep     0:00  0.00%  0.00% init
  208 root       3    0    48K  432K sleep     0:00  0.00%  0.00% getty
  210 root       3    0    48K  424K sleep     0:00  0.00%  0.00% getty
  209 root       3    0    48K  424K sleep     0:00  0.00%  0.00% getty
  211 root       3    0    48K  424K sleep     0:00  0.00%  0.00% getty
  217 nobody     2    0  1616K 1272K sleep     0:00  0.00%  0.00% httpd
  184 root       2    0   336K  580K sleep     0:00  0.00%  0.00% sshd
  201 root       2    0    76K  456K sleep     0:00  0.00%  0.00% inetd
```

На первый взгляд кажется очевидным, какой процесс нагружает систему, однако в данном случае интересно, почему это происходит. Программа bonnie — это средство тестирования производительности диска, способное записывать большие файлы разных размеров и разными способами. Предыдущий вывод показывает лишь то, что программа bonnie нагружает процессор, но не отвечает на вопрос почему.

19.3.1.1. Другие полезные возможности top

Внимательное изучение страницы руководства top(1) показывает, что с помощью top можно делать гораздо больше, например менять приоритет процессов и завершать их. Кроме того, можно задавать фильтры для просмотра процессов.

19.3.2. Утилита sysstat

Как указано на странице руководства sysstat(1), утилита sysstat показывает различные системные статистики с использованием библиотеки curses. Во время её работы экран разделён на две части: в верхнем окне отображается текущая средняя нагрузка, а содержимое нижней части зависит от команд пользователя. Исключение из режима с разделением экрана — включённое отображение vmstat, которое занимает весь экран. Ниже показано, как выглядит sysstat на довольно незагруженной системе, когда она была запущена без аргументов:

```
                /0   /1   /2   /3   /4   /5   /6   /7   /8   /9   /10
Load Average    |

                      /0   /10  /20  /30  /40  /50  /60  /70  /80  /90  /100
               <idle> XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

В основном сплошной простой, поэтому теперь посмотрим на вывод с аргументами — в данном случае **sysstat inet.tcp**, который выглядит так:

```
                /0   /1   /2   /3   /4   /5   /6   /7   /8   /9   /10
Load Average    |

   0 connections initiated          19 total TCP packets sent
   0 connections accepted           11   data
   0 connections established         0   data (retransmit)
                                     8   ack-only
   0 connections dropped             0   window probes
   0   in embryonic state            0   window updates
   0   on retransmit timeout         0   urgent data only
   0   by keepalive                  0   control
   0   by persist
                                    29 total TCP packets received
  11 potential rtt updates          17   in sequence
  11 successful rtt updates          0   completely duplicate
   9 delayed acks sent               0   with some duplicate data
   0 retransmit timeouts             4   out of order
   0 persist timeouts                0   duplicate acks
   0 keepalive probes               11   acks
   0 keepalive timeouts              0   window probes
                                     0   window updates
```

Вот это уже информативно. Первый опрос накопительный, поэтому при запуске sysstat в выводе можно увидеть довольно много информации. Это интересно, а теперь посмотрим на кэш буферов с помощью **sysstat bufcache**:

```
                    /0   /1   /2   /3   /4   /5   /6   /7   /8   /9   /10
     Load Average

There are 1642 buffers using 6568 kBytes of memory.

File System          Bufs used   %   kB in use   %  Bufsize kB   %  Util %
/                          877  53        6171  93        6516  99      94
/var/tmp                     5   0          17   0          28   0      60

Total:                     882  53        6188  94        6544  99
```

Опять довольно скучная система, но эти сведения полезно иметь под рукой. Хотя на всё это приятно смотреть, пора создать на системе искусственную нагрузку, чтобы увидеть, как sysstat можно использовать в качестве средства мониторинга производительности. Как и в случае с top, для высокой нагрузки на подсистемы ввода-вывода и небольшой нагрузки на процессор будет использоваться bonnie++. Затем снова посмотрим на bufcache, чтобы увидеть, есть ли заметные отличия:

```
                    /0   /1   /2   /3   /4   /5   /6   /7   /8   /9   /10
     Load Average   |||

There are 1642 buffers using 6568 kBytes of memory.

File System          Bufs used   %   kB in use   %  Bufsize kB   %  Util %
/                          811  49        6422  97        6444  98      99

Total:                     811  49        6422  97        6444  98
```

Сначала обратите внимание, что средняя нагрузка резко выросла — это, конечно, ожидаемо; затем, хотя большинство чисел близки, обратите внимание, что загрузка составляет 99%. Всё время, пока работала bonnie++, процент загрузки оставался на уровне 99 — это, разумеется, логично, однако в реальной ситуации поиска неисправностей это может указывать на процесс, выполняющий интенсивный ввод-вывод с одним конкретным файлом или файловой системой.

19.4. Средства мониторинга

Помимо экранных мониторов и средств, система NetBSD поставляется с набором средств, ориентированных на командную строку. Многие из средств, входящих в состав системы NetBSD, можно встретить и в других системах UNIX и UNIX-подобных системах.

19.4.1. fstat

Утилита fstat(1) сообщает состояние открытых файлов в системе. Хотя многие администраторы не считают её средством мониторинга производительности, она помогает выяснить, не использует ли конкретный пользователь или процесс чрезмерное количество файлов, не создаёт ли большие файлы, и получить подобные сведения.

Ниже приведён пример вывода fstat:

```
USER     CMD          PID   FD MOUNT      INUM MODE         SZ|DV R/W
jrf      tcsh       21607   wd /         29772 drwxr-xr-x     512 r
jrf      tcsh       21607    3* unix stream c057acc0<-> c0553280
jrf      tcsh       21607    4* unix stream c0553280 <-> c057acc0
root     sshd       21597   wd /             2 drwxr-xr-x     512 r
root     sshd       21597    0 /         11921 crw-rw-rw-    null rw
```

```
nobody   httpd       5032   wd /            2 drwxr-xr-x     512 r
nobody   httpd       5032    0 /        11921 crw-rw-rw-    null r
nobody   httpd       5032    1 /        11921 crw-rw-rw-    null w
nobody   httpd       5032    2 /        15890 -rw-r--r--  353533 rw
...
```

Назначение полей довольно очевидно; и снова: это средство, хотя и не столь ориентировано на производительность, как другие, может пригодиться при выяснении сведений об использовании файлов.

19.4.2. iostat

Команда iostat(8) делает именно то, что следует из её названия: она сообщает состояние подсистем ввода-вывода в системе. Обычно iostat запускают с заданным числом опросов и интервалом между ними, например так:

```
$ iostat -c 5 -w 5
      tty             wd0             cd0             fd0             md0             cpu
 tin tout  KB/t t/s MB/s   KB/t t/s MB/s   KB/t t/s MB/s   KB/t t/s MB/s  us ni sy in id
   0    1  5.13   1 0.00   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0  0  0  0 100
   0   54  0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0  0  0  0 100
   0   18  0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0  0  0  0 100
   0   18  8.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0  0  0  0 100
   0   28  0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0  0  0  0 100
```

Приведённый выше вывод получен на очень спокойном ftp-сервере. Поля соответствуют различным устройствам ввода-вывода: tty (который, как ни странно, самый активный, потому что работает сам iostat), wd0 — основной диск IDE, cd0 — привод CD-ROM, fd0 — дисковод гибких дисков, а также файловая система в памяти.

А теперь посмотрим, удастся ли нагрузить систему по полной. Сначала — большая ftp-передача: tar-архив исходных текстов netbsd-current и одновременно запущенная программа тестирования диска bonnie++.

```
$ iostat -c 5 -w 5
      tty             wd0             cd0             fd0             md0             cpu
 tin tout  KB/t t/s MB/s   KB/t t/s MB/s   KB/t t/s MB/s   KB/t t/s MB/s  us ni sy in id
   0    1  5.68   1 0.00   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0  0  0  0 100
   0   54 61.03 150 8.92   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   1  0 18  4 78
   0   26 63.14 157 9.71   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   1  0 20  4 75
   0   20 43.58  26 1.12   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   0  0  9  2 88
   0   28 19.49  82 1.55   0.00   0 0.00   0.00   0 0.00   0.00   0 0.00   1  0  7  3 89
```

Как и следовало ожидать, обратите внимание, что wd0 очень активен; интересно в этом выводе то, что ввод-вывод процессора, по-видимому, растёт пропорционально wd0. Это вполне логично, однако стоит отметить, что наблюдать такое можно только потому, что этот ftp-сервер почти не используется. Если бы, например, подсистема ввода-вывода процессора уже была под умеренной нагрузкой, а дисковая подсистема — под такой же, как сейчас, могло бы показаться, что узким местом является процессор, тогда как на самом деле им был бы диск. В таком случае видно, что одного средства редко достаточно для полного анализа проблемы. Беглый взгляд на процессы, вероятно, подскажет нам (после наблюдения за iostat), какие процессы создают проблемы.

19.4.3. ps

С помощью команды ps(1), то есть состояния процессов, можно узнать много информации о системе. Чаще всего команду ps используют, чтобы выделить конкретный процесс по имени, группе, владельцу и т. п. Запущенная без параметров и аргументов, ps просто выводит сведения о пользователе, который её выполняет.

```
$ ps
  PID TT STAT    TIME COMMAND
21560 p0 Is   0:00.04 -tcsh
21564 p0 I+   0:00.37 ssh jrf.odpn.net
21598 p1 Ss   0:00.12 -tcsh
21673 p1 R+   0:00.00 ps
21638 p2 Is+  0:00.06 -tcsh
```

Не слишком захватывающе. Назначение полей очевидно, за исключением STAT — это фактически состояние, в котором находится процесс. Все флаги описаны на странице руководства; в приведённом примере I означает простой, S — сон, R — готовность к выполнению, + означает, что процесс находится на переднем плане, а s — что процесс является лидером сеанса. Всё это вполне логично при взгляде на флаги: например, PID 21560 — это оболочка, она простаивает и (как и следовало ожидать) является лидером сеанса.

В большинстве случаев в списке процессов ищут что-то вполне конкретное. Например, все процессы показывает параметр -a; чтобы увидеть все процессы, включая процессы без управляющего терминала, используется -ax; а для гораздо более подробного списка (по сути, всё плюс сведения о влиянии процессов) — aux:

```
# ps aux
USER     PID %CPU %MEM    VSZ  RSS TT STAT STARTED    TIME COMMAND
root       0  0.0  9.6      0 6260 ?? DLs  16Jul02 0:01.00 (swapper)
root   23362  0.0  0.8    144  488 ?? S    12:38PM 0:00.01 ftpd -l
root   23328  0.0  0.4    428  280 p1 S    12:34PM 0:00.04 -csh
jrf    23312  0.0  1.8    828 1132 p1 Is   12:32PM 0:00.06 -tcsh
root   23311  0.0  1.8    388 1156 ?? S    12:32PM 0:01.60 sshd: jrf@ttyp1
jrf    21951  0.0  1.7    244 1124 p0 S+    4:22PM 0:02.90 ssh jrf.odpn.net
jrf    21947  0.0  1.7    828 1128 p0 Is    4:21PM 0:00.04 -tcsh
root   21946  0.0  1.8    388 1156 ?? S     4:21PM 0:04.94 sshd: jrf@ttyp0
nobody  5032  0.0  2.0   1616 1300 ?? I    19Jul02 0:00.02 /usr/pkg/sbin/httpd
...
```

Опять же, назначение большинства полей очевидно, за исключением VSZ и RSS, которые могут немного запутать. RSS — реальный размер процесса в единицах по 1024 байта, а VSZ — виртуальный размер. Всё это прекрасно, но опять же: чем может помочь ps? Хотя бы тем — взгляните на эту изменённую версию того же вывода:

```
# ps aux
USER     PID %CPU %MEM    VSZ  RSS TT STAT STARTED    TIME COMMAND
root       0  0.0  9.6      0 6260 ?? DLs  16Jul02 0:01.00 (swapper)
root   23362  0.0  0.8    144  488 ?? S    12:38PM 0:00.01 ftpd -l
root   23328  0.0  0.4    428  280 p1 S    12:34PM 0:00.04 -csh
jrf    23312  0.0  1.8    828 1132 p1 Is   12:32PM 0:00.06 -tcsh
root   23311  0.0  1.8    388 1156 ?? S    12:32PM 0:01.60 sshd: jrf@ttyp1
jrf    21951  0.0  1.7    244 1124 p0 S+    4:22PM 0:02.90 ssh jrf.odpn.net
jrf    21947  0.0  1.7    828 1128 p0 Is    4:21PM 0:00.04 -tcsh
root   21946  0.0  1.8    388 1156 ?? S     4:21PM 0:04.94 sshd: jrf@ttyp0
```

```
nobody  5032  9.0  2.0   1616 1300 ?? I    19Jul02 0:00.02 /usr/pkg/sbin/httpd
...
```

Учитывая, что на этом сервере, судя по нашей базовой линии, система относительно спокойна, PID 5032 имеет необычно большую долю %CPU. Иногда из-за этого растут и значения TIME. Вывод ps можно отфильтровать с помощью grep по PID, имени пользователя и имени процесса и тем самым отследить процессы, у которых могут быть проблемы.

19.4.4. vmstat

С помощью vmstat(1) можно отслеживать и измерять сведения, относящиеся к виртуальной памяти. Как и iostat, vmstat можно запускать с числом опросов и интервалом. Ниже приведён пример вывода с параметрами -c 5 -w 5, как в примере с iostat:

```
# vmstat -c 5 -w 5
 procs   memory     page                       disks         faults      cpu
 r b w   avm   fre  flt  re  pi   po   fr   sr w0 c0 f0 m0   in   sy  cs us sy id
 0 7 0 17716 33160    2   0   0    0    0    0  1  0  0  0  105   15   4  0  0 100
 0 7 0 17724 33156    2   0   0    0    0    0  1  0  0  0  109    6   3  0  0 100
 0 7 0 17724 33156    1   0   0    0    0    0  1  0  0  0  105    6   3  0  0 100
 0 7 0 17724 33156    1   0   0    0    0    0  0  0  0  0  107    6   3  0  0 100
 0 7 0 17724 33156    1   0   0    0    0    0  0  0  0  0  105    6   3  0  0 100
```

Снова относительно спокойно; ради последовательности будет использована точно такая же нагрузка, как в примере с iostat. Нагрузка — большая передача файлов и программа тестирования bonnie.

```
# vmstat -c 5 -w 5
 procs   memory     page                       disks         faults      cpu
 r b w   avm   fre  flt  re  pi   po   fr   sr w0 c0 f0 m0   in   sy  cs us sy id
 1 8 0 18880 31968    2   0   0    0    0    0  1  0  0  0  105   15   4  0  0 100
 0 8 0 18888 31964    2   0   0    0    0    0 130  0  0  0 1804 5539 1094 31 22 47
 1 7 0 18888 31964    1   0   0    0    0    0 130  0  0  0 1802 5500 1060 36 16 49
 1 8 0 18888 31964    1   0   0    0    0    0 160  0  0  0 1849 5905 1107 21 22 57
 1 7 0 18888 31964    1   0   0    0    0    0 175  0  0  0 1893 6167 1082  1 25 75
```

Результат немного отличается. Обратите внимание: поскольку большая часть работы была связана с вводом-выводом, фактически используемой памяти было немного. Но так как в этой системе для `/tmp` используется mfs, она вполне может пострадать. Взгляните:

```
# vmstat -c 5 -w 5
 procs   memory     page                       disks         faults      cpu
 r b w   avm   fre  flt  re  pi   po   fr   sr w0 c0 f0 m0   in   sy  cs us sy id
 0 2 0 99188   500    2   0   0    0    0    0  1  0  0  0  105   16   4  0  0 100
 0 2 0111596   436  592   0 587  624  586 1210 624  0  0  0  741  883 1088  0 11 89
 0 3 0123976   784  666   0 662  643  683 1326 702  0  0  0  828  993 1237  0 12 88
 0 2 0134692  1236  581   0 571  563  595 1158 599  0  0  0  722  863 1066  0  9 90
 2 0 0142860   912  433   0 406  403  405  808 429  0  0  0  552  602 768  0  7 93
```

Довольно пугающая картина. Она получена запуском bonnie в `/tmp` на файловой системе в памяти. Если бы это продолжалось слишком долго, возможно, система начала бы работать в режиме непрерывной подкачки (thrashing). Заметьте: хотя подсистема виртуальной памяти подвергалась серьёзной нагрузке, процессоры всё же не были слишком перегружены.

19.5. Сетевые средства

Иногда проблема производительности связана не с конкретной машиной, а с сетью или каким-либо устройством в сети — другим хостом, маршрутизатором и т. п. То, что делают другие машины, предоставляющие службы или какое-либо подключение к конкретной системе NetBSD, и как они себя ведут, может очень сильно влиять на производительность самой системы NetBSD или на то, как её воспринимают пользователи. Очень показательный пример — внезапное исчезновение DNS-сервера, которым пользуется машина NetBSD. Запросы выполняются долго и в конце концов завершаются неудачей. Неопытный человек, вошедший в систему NetBSD, несомненно (если у него нет других данных) обвинит в этом систему NetBSD. Одна из моих любимых фраз — «интернет сломался» — обычно означает, что либо служба DNS, либо маршрутизатор или шлюз вышел из сети. Как бы то ни было, система NetBSD достаточно вооружена, чтобы выяснить, какие сетевые проблемы возникают — будь то неисправность локальной системы или что-то иное.

19.5.1. ping

Классическая утилита ping(8) может сказать, есть ли обычная связность, а также работает ли разрешение имён хостов (в зависимости от того, что предписывает `nsswitch.conf`). Ниже приведён типичный вывод ping в локальной сети с указанным числом 3:

```
# ping -c 3 marie
PING marie (172.16.14.12): 56 data bytes
64 bytes from 172.16.14.12: icmp_seq=0 ttl=255 time=0.571 ms
64 bytes from 172.16.14.12: icmp_seq=1 ttl=255 time=0.361 ms
64 bytes from 172.16.14.12: icmp_seq=2 ttl=255 time=0.371 ms

----marie PING Statistics----
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.361/0.434/0.571/0.118 ms
```

ping не только сообщает, жив ли хост, но и показывает, сколько времени это заняло, и в самом конце выдаёт полезные подробности. Если имя хоста не разрешается, можно указать просто IP-адрес:

```
# ping -c 1 172.16.20.5
PING ash (172.16.20.5): 56 data bytes
64 bytes from 172.16.20.5: icmp_seq=0 ttl=64 time=0.452 ms

----ash PING Statistics----
1 packets transmitted, 1 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.452/0.452/0.452/0.000 ms
```

Как и у любого другого средства, значения времени здесь весьма субъективны, особенно когда речь идёт о сети. Например, хотя времена в примерах хорошие, взгляните на ping localhost:

```
# ping -c 4 localhost
PING localhost (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=255 time=0.091 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=255 time=0.129 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=255 time=0.120 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=255 time=0.122 ms

----localhost PING Statistics----
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.091/0.115/0.129/0.017 ms
```

Гораздо меньше, потому что запрос не покидал машину. Ping можно использовать для сбора сведений о том, насколько хорошо работает сеть. Он также хорош для локализации проблем: например, если в сети есть три примерно одинаковые по размеру системы NetBSD и у одной из них просто ужасные времена ping, скорее всего, что-то не так именно с этой машиной.

19.5.2. traceroute

Команда traceroute(8) отлично подходит для проверки доступности маршрута и обнаружения проблем на конкретном маршруте. Например, вот трассировка между ftp-сервером из примера и ftp.NetBSD.org:

```
# traceroute ftp.NetBSD.org
traceroute to ftp.NetBSD.org (204.152.184.75), 30 hops max, 40 byte packets
 1  208.44.95.1 (208.44.95.1)  1.646 ms  1.492 ms  1.456 ms
 2  63.144.65.170 (63.144.65.170)  7.318 ms  3.249 ms  3.854 ms
 3  chcg01-edge18.il.inet.qwest.net (65.113.85.229)  35.982 ms  28.667 ms  21.971 ms
 4  chcg01-core01.il.inet.qwest.net (205.171.20.1)  22.607 ms  26.242 ms  19.631 ms
 5  snva01-core01.ca.inet.qwest.net (205.171.8.50)  78.586 ms  70.585 ms  84.779 ms
 6  snva01-core03.ca.inet.qwest.net (205.171.14.122)  69.222 ms  85.739 ms  75.979 ms
 7  paix01-brdr02.ca.inet.qwest.net (205.171.205.30)  83.882 ms  67.739 ms  69.937 ms
 8  198.32.175.3 (198.32.175.3)  72.782 ms  67.687 ms  73.320 ms
 9  so-1-0-0.orpa8.pf.isc.org (192.5.4.231)  78.007 ms  81.860 ms  77.069 ms
10  tun0.orrc5.pf.isc.org (192.5.4.165)  70.808 ms  75.151 ms  81.485 ms
11  ftp.NetBSD.org (204.152.184.75)  69.700 ms  69.528 ms  77.788 ms
```

В целом неплохо. Трассировка прошла от хоста до локального маршрутизатора, затем вышла в сеть провайдера и, наконец, в интернет на поиск конечного пункта назначения. То, как толковать результаты traceroute, опять же субъективно, но аномально высокие времена на отдельных участках маршрута могут указывать на узкое место в сетевом оборудовании. Как и в случае с ping, если подозрение вызывает сам хост, запустите traceroute с другого хоста до того же пункта назначения. А теперь худший случай:

```
# traceroute www.microsoft.com
traceroute: Warning: www.microsoft.com has multiple addresses; using 207.46.230.220
traceroute to www.microsoft.akadns.net (207.46.230.220), 30 hops max, 40 byte packets
 1  208.44.95.1 (208.44.95.1)  2.517 ms  4.922 ms  5.987 ms
 2  63.144.65.170 (63.144.65.170)  10.981 ms  3.374 ms  3.249 ms
 3  chcg01-edge18.il.inet.qwest.net (65.113.85.229)  37.810 ms  37.505 ms  20.795 ms
 4  chcg01-core03.il.inet.qwest.net (205.171.20.21)  36.987 ms  32.320 ms  22.430 ms
 5  chcg01-brdr03.il.inet.qwest.net (205.171.20.142)  33.155 ms  32.859 ms  33.462 ms
 6  205.171.1.162 (205.171.1.162)  39.265 ms  20.482 ms  26.084 ms
 7  sl-bb24-chi-13-0.sprintlink.net (144.232.26.85)  26.681 ms  24.000 ms  28.975 ms
 8  sl-bb21-sea-10-0.sprintlink.net (144.232.20.30)  65.329 ms  69.694 ms  76.704 ms
 9  sl-bb21-tac-9-1.sprintlink.net (144.232.9.221)  65.659 ms  66.797 ms  74.408 ms
10  144.232.187.194 (144.232.187.194)  104.657 ms  89.958 ms  91.754 ms
11  207.46.154.1 (207.46.154.1)  89.197 ms  84.527 ms  81.629 ms
12  207.46.155.10 (207.46.155.10)  78.090 ms  91.550 ms  89.480 ms
13  * * *
.......
```

В этом случае сервер Microsoft не удаётся найти либо из-за нескольких адресов, либо потому, что где-то по пути система или сервер не может ответить на информационный запрос. В такой момент можно попробовать ping; в случае с Microsoft ping не отвечает — скорее всего, где-то в их сети отключён ICMP.

19.5.3. netstat

Ещё одна проблема, которая может возникнуть в системе NetBSD, — неполадки с таблицей маршрутизации. Они не всегда являются виной системы. Команды route(8) и netstat(1) показывают сведения о маршрутах и сетевых соединениях (соответственно).

Команду route можно использовать для просмотра и изменения таблиц маршрутизации, а netstat — для отображения сведений о сетевых соединениях и маршрутах. Сначала приведём вывод route show:

```
# route show
Routing tables

Internet:
Destination      Gateway            Flags
default          208.44.95.1        UG
loopback         127.0.0.1          UG
localhost        127.0.0.1          UH
172.15.13.0      172.16.14.37       UG
172.16.0.0       link#2             U
172.16.14.8      0:80:d3:cc:2c:0    UH
172.16.14.10     link#2             UH
marie            0:10:83:f9:6f:2c   UH
172.16.14.37     0:5:32:8f:d2:35    UH
172.16.16.15     link#2             UH
loghost          8:0:20:a7:f0:75    UH
artemus          8:0:20:a8:d:7e     UH
ash              0:b0:d0:de:49:df   UH
208.44.95.0      link#1             U
208.44.95.1      0:4:27:3:94:20     UH
208.44.95.2      0:5:32:8f:d2:34    UH
208.44.95.25     0:c0:4f:10:79:92   UH

Internet6:
Destination      Gateway            Flags
default          localhost          UG
default          localhost          UG
localhost        localhost          UH
::127.0.0.0      localhost          UG
::224.0.0.0      localhost          UG
::255.0.0.0      localhost          UG
::ffff:0.0.0.0   localhost          UG
2002::           localhost          UG
2002:7f00::      localhost          UG
2002:e000::      localhost          UG
2002:ff00::      localhost          UG
fe80::           localhost          UG
fe80::%ex0       link#1             U
fe80::%ex1       link#2             U
fe80::%lo0       fe80::1%lo0        U
fec0::           localhost          UG
ff01::           localhost          U
ff02::%ex0       link#1             U
ff02::%ex1       link#2             U
ff02::%lo0       fe80::1%lo0        U
```

Раздел флагов показывает состояние и признак шлюза. В данном случае мы видим U, H и G (U — up, H — host, G — gateway; остальные флаги описаны на странице руководства).

Теперь небольшой вывод netstat с параметрами -r (маршрутизация) и -n (показывать сетевые номера):

```
Routing tables

Internet:
Destination        Gateway            Flags     Refs     Use    Mtu  Interface
default            208.44.95.1        UGS          0   330309   1500  ex0
127                127.0.0.1          UGRS         0        0  33228  lo0
127.0.0.1          127.0.0.1          UH           1     1624  33228  lo0
172.15.13/24       172.16.14.37       UGS          0        0   1500  ex1
172.16             link#2             UC          13        0   1500  ex1
...
Internet6:
Destination                   Gateway                   Flags     Refs     Use
  Mtu  Interface
::/104                        ::1                       UGRS        0        0
33228  lo0 =>
::/96                         ::1                       UGRS        0        0
```

Приведённый выше вывод немного подробнее. Итак, чем это может помочь? Хороший пример — когда маршруты между сетями меняются, пока пользователи подключены. Я видел это несколько раз, когда кто-то целыми днями перезагружал маршрутизаторы после каждого изменения. Несколько пользователей звонили и говорили, что их выбрасывает из системы и что войти обратно очень долго не получается. Как выяснилось, клиенты, подключавшиеся к системе, перенаправлялись на другой маршрутизатор (с очень длинным маршрутом) для повторного подключения. У их соединений я наблюдал флаг M, то есть Modified dynamically (by redirect). Я удалил маршруты, попросил их подключиться заново и сразу же связался с провинившимся техником.

19.5.4. tcpdump

Последнее и, безусловно, не по важности — tcpdump(8), сетевой анализатор, способный извлечь массу информации. Здесь будет приведён пример вывода и объяснение некоторых наиболее полезных параметров tcpdump.

Ниже — небольшой фрагмент работы tcpdump сразу после запуска:

```
# tcpdump
tcpdump: listening on ex0
14:07:29.920651 mail.ssh > 208.44.95.231.3551: P 2951836801:2951836845(44) ack 2
476972923 win 17520 <nop,nop,timestamp 1219259 128519450> [tos 0x10]
14:07:29.950594 12.125.61.34 >  208.44.95.16: ESP(spi=2548773187,seq=0x3e8c) (DF)
14:07:29.983117 smtp.somecorp.com.smtp > 208.44.95.30.42828: . ack 420285166 win
16500 (DF)
14:07:29.984406 208.44.95.30.42828 > smtp.somecorp.com.smtp: . 1:1376(1375) ack 0
 win 7431 (DF)
...
```

Учитывая, что этот сервер — почтовый, показанное вполне логично; однако утилита весьма многословна, поэтому я предпочитаю сначала запускать tcpdump без параметров и записывать текстовый вывод в файл, чтобы разбирать его позже, вот так:

```
# tcpdump > tcpdump.out
tcpdump: listening on ex0
```

Так что же именно мы ищем в этой мешанине? Короче говоря, всё, что кажется неуместным: например, неправильные длины пакетов (когда их много) проявятся как improper lens или malformed packets (по сути, мусор). Если же мы ищем что-то конкретное, tcpdump может помочь — в зависимости от проблемы.

19.5.4.1. Конкретное использование tcpdump

Это лишь примеры нескольких вещей, которые можно делать с помощью tcpdump.

Поиск дублирующихся IP-адресов:

```
tcpdump -e host ip-address
```

Например:

```
tcpdump -e host 192.168.0.2
```

Проблемы маршрутизации:

```
tcpdump icmp
```

Сторонних средств существует множество, однако NetBSD поставляется с хорошим набором инструментов для поиска проблем производительности на сетевом уровне.

19.6. Учёт

NetBSD можно настроить так, чтобы каждый выполненный в системе процесс записывался в специальный файл учёта. В этом разделе рассматриваются основы общесистемного учёта.

19.6.1. Включение учёта

Проще всего включить общесистемный учёт, задав `accounting=YES` в `/etc/rc.conf` и запустив соответствующую службу:

```
# service accounting start
```

Можно также включить его вручную, запустив accton(8) от имени root:

```
# touch /var/account/acct
# accton /var/account/acct
```

`/var/account/acct` — файл учёта по умолчанию. Выбор именно этого пути делает использование других команд учёта, таких как lastcomm(1) и sa(8), немного удобнее. Однако можно указать любой файл.

Чтобы остановить учёт, можно отключить переменную `accounting` в `/etc/rc.conf` (и/или остановить службу `accounting`) либо (что неочевидно) запустить **accton** без аргументов.

19.6.2. Чтение данных учёта

Для чтения данных учёта можно использовать два средства:

- lastcomm(1)
- sa(8)

19.6.2.1. lastcomm

Команда lastcomm(1) показывает последние выполненные команды по порядку — все. Однако можно выбрать и по пользователю. Вот пример вывода:

```
$ lastcomm jrf
last       -       jrf      ttyp3      0.00 secs Tue Sep  3 14:39 (0:00:00.02)
man        -       jrf      ttyp3      0.00 secs Tue Sep  3 14:38 (0:01:49.03)
sh         -       jrf      ttyp3      0.00 secs Tue Sep  3 14:38 (0:01:49.03)
less       -       jrf      ttyp3      0.00 secs Tue Sep  3 14:38 (0:01:49.03)
lastcomm   -       jrf      ttyp3      0.02 secs Tue Sep  3 14:38 (0:00:00.02)
stty       -       jrf      ttyp3      0.00 secs Tue Sep  3 14:38 (0:00:00.02)
tset       -       jrf      ttyp3      0.00 secs Tue Sep  3 14:38 (0:00:01.05)
hostname   -       jrf      ttyp3      0.00 secs Tue Sep  3 14:38 (0:00:00.02)
ls         -       jrf      ttyp0      0.00 secs Tue Sep  3 14:36 (0:00:00.00)
...
```

Довольно удобно. **lastcomm** берёт сведения из расположенного по умолчанию файла `/var/account/acct`, однако с помощью параметра -f можно указать другой файл.

Как нетрудно понять, вывод **lastcomm** может оказаться тяжеловатым на больших многопользовательских системах. Здесь-то и вступает в игру команда **sa**.

19.6.2.2. sa

Команду sa(8) можно использовать для работы с данными учёта. Её также можно использовать в интерактивном режиме для создания отчётов. Ниже приведён вывод sa по умолчанию:

```
$ sa
      77       18.62re        0.02cp        8avio        0k
       3        4.27re        0.01cp       45avio        0k   ispell
       2        0.68re        0.00cp       33avio        0k   mutt
       2        1.09re        0.00cp       23avio        0k   vi
      10        0.61re        0.00cp        7avio        0k   ***other
       2        0.01re        0.00cp       29avio        0k   exim
       4        0.00re        0.00cp        8avio        0k   lastcomm
       2        0.00re        0.00cp        3avio        0k   atrun
       3        0.03re        0.00cp        1avio        0k   cron*
       5        0.02re        0.00cp       10avio        0k   exim*
      10        3.98re        0.00cp        2avio        0k   less
      11        0.00re        0.00cp        0avio        0k   ls
       9        3.95re        0.00cp       12avio        0k   man
       2        0.00re        0.00cp        4avio        0k   sa
      12        3.97re        0.00cp        1avio        0k   sh
...
```

Слева направо поля означают: общее число вызовов, реальное время в минутах, суммарное пользовательское и системное время в минутах, среднее число операций ввода-вывода за одно выполнение, средний объём занятой памяти (core) в пересчёте на процессорное время и имя команды.

sa также позволяет создавать сводные файлы или отчёты с помощью некоторых параметров. Например, вот вывод с сортировкой по среднему использованию памяти в пересчёте на процессорное время:

```
$ sa -k
      86       30.81re        0.02cp        8avio        0k
      10        0.61re        0.00cp        7avio        0k   ***other
       2        0.00re        0.00cp        3avio        0k   atrun
       3        0.03re        0.00cp        1avio        0k   cron*
```

```
      2        0.01re        0.00cp       29avio        0k   exim
      5        0.02re        0.00cp       10avio        0k   exim*
      3        4.27re        0.01cp       45avio        0k   ispell
      4        0.00re        0.00cp        8avio        0k   lastcomm
     12        8.04re        0.00cp        2avio        0k   less
     13        0.00re        0.00cp        0avio        0k   ls
     11        8.01re        0.00cp       12avio        0k   man
      2        0.68re        0.00cp       33avio        0k   mutt
      3        0.00re        0.00cp        4avio        0k   sa
     14        8.03re        0.00cp        1avio        0k   sh
      2        1.09re        0.00cp       23avio        0k   vi
```

На больших системах команда **sa** очень полезна.

19.6.3. Как применять учёт на практике

Отчёты учёта помогают предсказывать тенденции. Например, на системе, где всё чаще используются **cc** и **make**, может оказаться, что через несколько месяцев потребуются некоторые изменения, чтобы система продолжала работать на оптимальном уровне. Другой хороший пример — нагрузка на веб-сервер. Если она начинает постепенно расти, возможно, придётся что-то предпринять, прежде чем это станет проблемой. К счастью, средства учёта NetBSD позволяют достаточно обоснованно предсказать такие действия и спланировать их заранее.

19.7. Профилирование ядра

Профилирование ядра обычно применяют, когда нужно сравнить изменения в новом ядре с предыдущим или найти какую-либо низкоуровневую проблему производительности. Независимо друг от друга записываются два набора данных о поведении профилируемого кода: частота вызовов функций и время, проведённое в каждой функции.

19.7.1. Начало работы

Сначала загляните в раздел 19.9, «Настройка ядра» и в главу 34, *Сборка ядра*. Единственное отличие в процедуре подготовки ядра с включённым профилированием — при запуске config добавьте параметр -p. Область сборки — `../compile/<KERNEL_NAME>.PROF`; например, для ядра GENERIC это будет `../compile/GENERIC.PROF`.

Ниже — краткая сводка о том, как собрать ядро с включённым профилированием на платформе amd64; предполагается, что нужные исходные тексты находятся в `/usr/src` и используется конфигурация GENERIC, — конечно, так бывает не всегда:

1. `cd /usr/src/sys/arch/amd64/conf`
2. `config -p GENERIC`
3. `cd ../compile/GENERIC.PROF`
4. `make depend && make`
5. `cp /netbsd /netbsd.old`
6. `cp netbsd /`
7. `reboot`

Когда новое ядро установлено и система перезагружена, пора включить мониторинг и начать смотреть результаты.

19.7.1.1. Использование kgmon

Чтобы запустить kgmon:

```
$ kgmon -b
kgmon: kernel profiling is running.
```

Затем направьте данные в файл `gmon.out`:

```
$ kgmon -p
```

Теперь пора сделать вывод удобочитаемым:

```
$ gprof /netbsd > gprof.out
```

Поскольку gmon ищет `gmon.out`, он должен найти его в текущем рабочем каталоге.

Запустив только kgmon, вы можете не получить нужных сведений; однако если вы сравниваете различия между двумя разными ядрами, следует использовать заведомо исправную базовую линию. Учтите, что обычно полезно нагрузить ту подсистему, о которой идёт речь, — как при снятии базовых показателей, так и с новым (или другим) ядром.

19.7.2. Расшифровка вывода kgmon

Теперь, когда kgmon умеет запускаться, собирать и разбирать информацию, пора действительно посмотреть на эти данные. В данном случае ядро GENERIC работало с включённым профилированием около часа, при этом выполнялись только системные процессы и не было никакой мешающей нагрузки; в разделе о внесении неисправности пример будет достаточно показательным, чтобы проблему можно было легко обнаружить даже при минимальной нагрузке.

19.7.2.1. Плоский профиль

Плоский профиль — это список функций, число их вызовов и затраченное время (в секундах). Ниже приведён пример вывода, полученного на спокойной системе:

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ns/call  ns/call  name
 99.77    163.87   163.87                             idle
  0.03    163.92     0.05      219 228310.50 228354.34  _wdc_ata_bio_start
  0.02    163.96     0.04      219 182648.40 391184.96  wdc_ata_bio_intr
  0.01    163.98     0.02     3412  5861.66  6463.02  pmap_enter
  0.01    164.00     0.02      548 36496.35 36496.35  pmap_zero_page
  0.01    164.02     0.02                             Xspllower
  0.01    164.03     0.01   481968    20.75    20.75  gettick
  0.01    164.04     0.01     6695  1493.65  1493.65  VOP_LOCK
  0.01    164.05     0.01     3251  3075.98 21013.45  syscall_plain
...
```

Как и ожидалось, наибольший процент оказался у idle, однако кое-что всё же происходило: например, чуть ниже находится функция *vn_lock*:

```
...
  0.00    164.14     0.00     6711     0.00     0.00  VOP_UNLOCK
```

```
  0.00    164.14     0.00     6677     0.00  1493.65  vn_lock
  0.00    164.14     0.00     6441     0.00     0.00  genfs_unlock
```

Это ожидаемо, поскольку блокировки всё равно выполняются.

19.7.2.2. Профиль графа вызовов

Граф вызовов — это расширенная версия плоского профиля, показывающая последующие вызовы из перечисленных функций. Сначала приведём пример вывода:

```
                     Call graph (explanation follows)


granularity: each sample hit covers 4 byte(s) for 0.01% of 164.14 seconds

index % time    self  children    called     name
                                                 <spontaneous>
[1]     99.8  163.87    0.00                 idle [1]
-----------------------------------------------
                                                 <spontaneous>
[2]      0.1    0.01    0.08                 syscall1 [2]
                0.01    0.06    3251/3251        syscall_plain [7]
                0.00    0.01     414/1660        trap [9]
-----------------------------------------------
                0.00    0.09     219/219         Xintr14 [6]
[3]      0.1    0.00    0.09     219         pciide_compat_intr [3]
                0.00    0.09     219/219         wdcintr [5]
-----------------------------------------------
...
```

Тут можно немного запутаться. Номер индекса соответствует числу в конце строки, например,

```
...
                0.00    0.01      85/85          dofilewrite [68]
[72]     0.0    0.00    0.01      85          soo_write [72]
                0.00    0.01      85/89          sosend [71]
...
```

Здесь мы видим, что первой была вызвана dofilewrite; теперь можно посмотреть на номер индекса для 64 и увидеть, что там происходило:

```
...
                0.00    0.01     101/103         ffs_full_fsync <cycle 6> [58]
[64]     0.0    0.00    0.01     103         bawrite [64]
                0.00    0.01     103/105         VOP_BWRITE [60]
...
```

И так далее — таким образом можно построить «визуальную трассировку».

В конце графа вызовов, сразу после раздела terms, находится индекс по именам функций, который также помогает сопоставлять индексы.

19.7.3. Применение на практике

В этом примере я изменил участок ядра, который, как я знаю, создаст проблему, бьющую в глаза.

Вот верхняя часть плоского профиля после примерно часа работы системы при минимальном вмешательстве пользователей:

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  us/call  us/call  name
 93.97    139.13   139.13                             idle
  5.87    147.82     8.69       23 377826.09 377842.52  check_exec
  0.01    147.84     0.02      243    82.30    82.30  pmap_copy_page
  0.01    147.86     0.02      131   152.67   152.67  _wdc_ata_bio_start
  0.01    147.88     0.02      131   152.67   271.85  wdc_ata_bio_intr
  0.01    147.89     0.01     4428     2.26     2.66  uvn_findpage
  0.01    147.90     0.01     4145     2.41     2.41  uvm_pageactivate
  0.01    147.91     0.01     2473     4.04  3532.40  syscall_plain
  0.01    147.92     0.01     1717     5.82     5.82  i486_copyout
  0.01    147.93     0.01     1430     6.99    56.52  uvm_fault
  0.01    147.94     0.01     1309     7.64     7.64  pool_get
  0.01    147.95     0.01      673    14.86    38.43  genfs_getpages
  0.01    147.96     0.01      498    20.08    20.08  pmap_zero_page
  0.01    147.97     0.01      219    45.66    46.28  uvm_unmap_remove
  0.01    147.98     0.01      111    90.09    90.09  selscan
...
```

Как видно, разница в производительности большая. Сразу заметно, что время простоя стало заметно меньше. Главное отличие в том, что одна конкретная функция имеет большое время по всем показателям при очень малом числе вызовов. Эта функция — *check_exec*. Поначалу это не показалось бы странным, если бы выполнялось много команд, но по сравнению с плоским профилем первого измерения пропорции выглядят неправильно:

```
...
  0.00    164.14     0.00       37     0.00 62747.49  check_exec
...
```

В первом измерении вызов выполняется 37 раз и показывает лучшую производительность. Очевидно, что-то не так в самой этой функции или вокруг неё. Чтобы исключить другие функции, может помочь взгляд на граф вызовов; вот первое вхождение *check_exec*

```
...
-----------------------------------------------
                0.00    8.69      23/23          syscall_plain [3]
[4]      5.9    0.00    8.69      23         sys_execve [4]
                8.69    0.00      23/23          check_exec [5]
                0.00    0.00      20/20          elf32_copyargs [67]
...
```

Обратите внимание, что время 8.69, по-видимому, влияет на две предыдущие функции. Возможно, что-то не так с ними, однако следующее вхождение *check_exec*, похоже, доказывает обратное:

```
...
-----------------------------------------------
                8.69    0.00      23/23          sys_execve [4]
[5]      5.9    8.69    0.00      23         check_exec [5]
...
```

Теперь видно, что проблема, скорее всего, кроется в *check_exec*. Конечно, проблемы не всегда настолько просты, но вот незамысловатый код, который был вставлен сразу после *check_exec* (функция находится в `sys/kern/kern_exec.c`):

```
...
        /* A Cheap fault insertion */
        for (x = 0; x < 100000000; x++) {
                y = x;
        }
..
```

Не бог весть что, но достаточно, чтобы профилирование показало большое изменение.

19.7.4. Итоги

Профилирование ядра может многое прояснить и даёт гораздо более точный метод поиска проблем производительности, которые трудно найти обычными средствами; к тому же это не так сложно, как думает большинство: если вы умеете собирать ядро, вы сможете заставить профилирование работать.

19.8. Настройка системы

Теперь, когда средства мониторинга и анализа рассмотрены, пора заняться собственно методами. В этом разделе рассматриваются средства и методы, влияющие на производительность системы без повторной сборки ядра; в следующем разделе рассматривается настройка ядра путём его пересборки.

19.8.1. Использование sysctl

Утилиту sysctl можно использовать для просмотра, а в некоторых случаях и изменения системных параметров. Параметров, которые можно просмотреть и изменить, так много, что все их здесь не показать; однако для первого примера приведём простое использование sysctl для просмотра системной переменной окружения PATH:

```
$ sysctl user.cs_path
user.cs_path =
/usr/bin:/bin:/usr/sbin:/sbin:/usr/pkg/bin:/usr/pkg/sbin:/usr/local/bin:/usr/local/sbin
```

Довольно просто. Теперь кое-что, действительно связанное с производительностью. Допустим, в системе с множеством пользователей возникают проблемы с открытием файлов; возможно, проблема решится, если проверить и, возможно, увеличить параметр kern.maxfiles. Но сначала посмотрим:

```
$ sysctl kern.maxfiles
kern.maxfiles = 1772
```

А чтобы изменить его, нужно от имени root указать параметр -w:

```
# sysctl -w kern.maxfiles=1972
kern.maxfiles: 1772 -> 1972
```

Учтите: после перезагрузки системы вернётся старое значение. Есть два способа это исправить: во-первых, изменить этот параметр в ядре и пересобрать его; во-вторых (и проще) добавить эту строку в `/etc/sysctl.conf`:

```
kern.maxfiles=1972
```

19.8.2. tmpfs и mfs

Реализации «ramdisk» в NetBSD кэшируют все данные в оперативной памяти, а если она заполнена, в качестве резервного хранилища используется пространство подкачки. В NetBSD есть две реализации: традиционная файловая система BSD на основе памяти «mfs» и более современная «tmpfs». Первая может только увеличиваться в размере, а вторая также может уменьшаться, если место больше не нужно.

На больших многопользовательских системах решить, когда использовать файловую систему в памяти, а когда нет, бывает непросто. Однако в некоторых случаях это вполне оправданно: например, на машине разработки, которой в каждый момент пользуется только один разработчик, хорошим местом может стать каталог obj или некоторые временные каталоги для сборок. В таком случае это оправданно на машинах с достаточным объёмом оперативной памяти. С другой стороны, если в системе всего 16 МБ памяти и `/var/tmp` использует mfs, могут возникнуть серьёзные проблемы с приложениями.

В ядре GENERIC по умолчанию включены и tmpfs, и mfs. Чтобы использовать их для конкретного каталога, сначала определите, где находится пространство подкачки, которое вы хотите задействовать; в данном примере беглый взгляд в `/etc/fstab` показывает, что раздел подкачки — `/dev/wd0b`:

```
mail% cat /etc/fstab
/dev/wd0a / ffs rw 1 1
/dev/wd0b none swap sw 0 0
/kern /kern kernfs rw
```

Эта система — почтовый сервер, поэтому я хочу использовать tmpfs только для `/tmp`; кроме того, в этой системе я связал `/tmp` с `/var/tmp`, чтобы сэкономить место (они на одном диске). Всё, что нужно, — добавить следующую запись:

```
/dev/wd0b /var/tmp tmpfs rw 0 0
```

Если вы хотите использовать «mfs» вместо «tmpfs», укажите в указанном выше месте именно её.

А теперь предупреждение: убедитесь, что упомянутые каталоги пусты и ничто их не использует, когда вы монтируете файловую систему в памяти! После изменения `/etc/fstab` можно выполнить **mount -a** или перезагрузить систему.

19.8.3. Журналирование

Журналирование — это механизм, который сначала помещает записываемые данные в так называемый «журнал», а на втором шаге данные из журнала записываются на диск. В случае краха системы данные, которые не были записаны на диск, но находятся в журнале, можно воспроизвести, и диск таким образом придёт в согласованное состояние. Главный эффект этого в том, что после аварийной перезагрузки не требуется проверка файловой системы (fsck).

Журналирование можно включить, добавив «log» в параметры файловой системы в `/etc/fstab`. Вот пример, включающий журналирование для корневой файловой системы (`/`), а также для `/var` и `/usr`:

```
/dev/wd0a /    ffs rw,log 1 1
/dev/wd0e /var ffs rw,log 1 2
/dev/wd0g /usr ffs rw,log 1 2
```

19.8.4. LFS

LFS, файловая система с журнальной структурой, записывает данные на диск иногда слишком агрессивно, что приводит к заторам. Чтобы ограничить скорость записи, можно использовать следующие параметры sysctl:

```
vfs.sync.delay
vfs.sync.filedelay
vfs.sync.dirdelay
vfs.sync.metadelay
vfs.lfs.flushindir
vfs.lfs.clean_vnhead
vfs.lfs.dostats
vfs.lfs.pagetrip
vfs.lfs.stats.segsused
vfs.lfs.stats.psegwrites
vfs.lfs.stats.psyncwrites
vfs.lfs.stats.pcleanwrites
vfs.lfs.stats.blocktot
vfs.lfs.stats.cleanblocks
vfs.lfs.stats.ncheckpoints
vfs.lfs.stats.nwrites
vfs.lfs.stats.nsync_writes
vfs.lfs.stats.wait_exceeded
vfs.lfs.stats.write_exceeded
vfs.lfs.stats.flush_invoked
vfs.lfs.stats.vflush_invoked
vfs.lfs.stats.clean_inlocked
vfs.lfs.stats.clean_vnlocked
vfs.lfs.stats.segs_reclaimed
vfs.lfs.ignore_lazy_sync
```

Помимо настройки этих параметров, может быть полезно отключить кэширование с обратной записью на устройствах wd(4). Подробности см. на странице руководства dkctl(8).

Дополнительные сведения можно найти в архивах списков рассылки NetBSD. См. это и это письмо.

19.9. Настройка ядра

Хотя многие системные параметры можно изменить с помощью sysctl, многого также можно добиться за счёт более совершенного системного программного обеспечения, правильной компоновки системы и управления службами (например, переноса их в inetd и обратно). Однако настройка ядра даёт более высокую производительность, даже если выигрыш кажется незначительным.

19.9.1. Подготовка к пересборке ядра

Сначала получите исходные тексты ядра для нужного выпуска, как описано в главе 32, *Получение исходных текстов*; для получения дополнительных сведений о сборке ядра рекомендуется прочитать главу 34, *Сборка ядра*. Учтите, что этот документ можно использовать и для настройки -current, однако сначала следует прочитать документацию Tracking -current, поскольку многое изложенное там повторяется здесь.

19.9.2. Настройка ядра

Настройка ядра в NetBSD может показаться устрашающей — из-за множества зависимостей между строками самого файла конфигурации. Однако у этого метода есть преимущество: чтобы настроить новое ядро, по сути нужен только текстовый редактор и вывод dmesg. Файл конфигурации ядра находится в `src/sys/arch/ARCH/conf`, где ARCH — ваша архитектура (например, на SPARC он будет в `src/sys/arch/sparc/conf`).

Найдя файл конфигурации ядра, скопируйте его и удалите (закомментируйте) все ненужные записи. Здесь вам и пригодится dmesg(8). Чистый вывод dmesg(8) покажет все устройства, обнаруженные ядром при загрузке. По выводу dmesg(8) можно определить, какие параметры устройств действительно нужны.

19.9.2.1. Примеры элементов конфигурации

В этом примере ядро ftp-сервера перенастраивается так, чтобы работать с минимальным набором драйверов, параметров и всего остального, что может ускорить его работу (опять же, не обязательно меньшим по размеру, хотя так и будет). Первым делом стоит взглянуть на некоторые основные элементы конфигурации. Итак, в `/usr/src/sys/arch/amd64/conf` файл GENERIC копируется в FTP, после чего файл FTP редактируется.

В начале файла есть группа параметров, начинающихся с maxusers; их мы не трогаем, хотя на больших многопользовательских системах, возможно, стоит немного увеличить это значение. Далее — поддержка процессоров; судя по выводу dmesg, видно следующее:

```
cpu0: Intel Pentium II/Celeron (Deschutes) (686-class), 400.93 MHz
```

Это означает, что нужно использовать только параметр I686_CPU. В следующем разделе все параметры оставлены без изменений, кроме PIC_DELAY, который рекомендуется, если машина не старая. В данном случае он включён, поскольку 686 — «относительно новый».

От последнего раздела и вплоть до параметров совместимости на этой конкретной системе действительно не нужно было ничего менять. Однако в разделе совместимости есть несколько параметров, которые включать не нужно; опять же потому, что эта машина строго является FTP-сервером, все параметры совместимости были отключены.

Следующий раздел — файловые системы; для этого сервера, опять же, включёнными нужно оставить очень немногие. Оставлены были следующие:

```
# File systems
file-system     FFS             # UFS
file-system     LFS             # log-structured file system
file-system     MFS             # memory file system
file-system     CD9660          # ISO 9660 + Rock Ridge file system
file-system     FDESC           # /dev/fd
file-system     KERNFS          # /kern
file-system     NULLFS          # loopback file system
file-system     PROCFS          # /proc
file-system     UMAPFS          # NULLFS + uid and gid remapping
...
options         SOFTDEP         # FFS soft updates support.
...
```

Далее идёт раздел сетевых параметров. Оставлены были только следующие:

```
options         INET            # IP + ICMP + TCP + UDP
options         INET6           # IPV6
```

```
options         IPFILTER_LOG    # ipmon(8) log support
```

IPFILTER_LOG — полезный параметр, поскольку на сервере будет работать ipf.

Следующий раздел — подробные сообщения различных подсистем; поскольку эта машина уже работает и серьёзных проблем не было, все они закомментированы.

19.9.2.2. Некоторые драйверы

Настраиваемых элементов в файле конфигурации относительно немного, и с ними легко разобраться, а вот драйверы устройств — совсем другое дело. В следующих примерах рассматриваются два драйвера и сокращаются соответствующие им «области» в файле. Сначала небольшой пример: в dmesg привод CD-ROM выглядит следующими строками:

```
...
cd0 at atapibus0 drive 0: <CD-540E, , 1.0A> type 5 cdrom removable
cd0: 32-bit data port
cd0: drive supports PIO mode 4, DMA mode 2, Ultra-DMA mode 2
pciide0: secondary channel interrupting at irq 15
cd0(pciide0:1:0): using PIO mode 4, Ultra-DMA mode 2 (using DMA data transfer
...
```

Теперь пора найти этот фрагмент в файле конфигурации. Обратите внимание, что привод «cd» находится на atapibus и требует поддержки pciide. В данном случае интерес представляет раздел конфигурации ядра «IDE and related devices». Здесь стоит отметить, что рядом с разделом IDE находятся также разделы ISA, PCMCIA и т. д.; на этой машине в выводе dmesg(8) устройств PCMCIA нет, поэтому логично, что все упоминания PCMCIA можно удалить. Но сначала — привод «cd».

В начале раздела IDE находится следующее:

```
...
wd*     at atabus? drive ? flags 0x0000
...
atapibus* at atapi?
...
```

Что ж, совершенно очевидно, что эти строки нужно оставить. Далее идёт вот что:

```
...
# ATAPI devices
# flags have the same meaning as for IDE drives.
cd*     at atapibus? drive ? flags 0x0000       # ATAPI CD-ROM drives
sd*     at atapibus? drive ? flags 0x0000       # ATAPI disk drives
st*     at atapibus? drive ? flags 0x0000       # ATAPI tape drives
uk*     at atapibus? drive ? flags 0x0000       # ATAPI unknown
...
```

Единственным типом устройств, который был в выводе dmesg(8), был cd; остальное можно закомментировать.

Следующий пример немного сложнее — сетевые интерфейсы. На этой машине их два:

```
...
ex0 at pci0 dev 17 function 0: 3Com 3c905B-TX 10/100 Ethernet (rev. 0x64)
ex0: interrupting at irq 10
ex0: MAC address 00:50:04:83:ff:b7
UI 0x001018 model 0x0012 rev 0 at ex0 phy 24 not configured
ex1 at pci0 dev 19 function 0: 3Com 3c905B-TX 10/100 Ethernet (rev. 0x30)
```

```
ex1: interrupting at irq 11
ex1: MAC address 00:50:da:63:91:2e
exphy0 at ex1 phy 24: 3Com internal media interface
exphy0: 10baseT, 10baseT-FDX, 100baseTX, 100baseTX-FDX, auto
...
```

На первый взгляд может показаться, что устройств на самом деле три, однако более внимательный взгляд на эту строку:

```
exphy0 at ex1 phy 24: 3Com internal media interface
```

показывает, что физических карт только две; как и в случае с CD-ROM, достаточно просто удалить имена, которых нет в dmesg. В начале раздела сетевых интерфейсов находится:

```
...
# Network Interfaces

# PCI network interfaces
an*     at pci? dev ? function ?        # Aironet PC4500/PC4800 (802.11)
bge*    at pci? dev ? function ?        # Broadcom 570x gigabit Ethernet
en*     at pci? dev ? function ?        # ENI/Adaptec ATM
ep*     at pci? dev ? function ?        # 3Com 3c59x
epic*   at pci? dev ? function ?        # SMC EPIC/100 Ethernet
esh*    at pci? dev ? function ?        # Essential HIPPI card
ex*     at pci? dev ? function ?        # 3Com 90x[BC]
...
```

Здесь есть устройство ex. Значит, всё остальное в разделе PCI можно удалить. Кроме того, каждую строку вплоть до этой:

```
exphy*  at mii? phy ?                   # 3Com internal PHYs
```

можно закомментировать, как и всё оставшееся.

19.9.2.3. Несколько проходов

Когда я настраиваю ядро, мне нравится делать это удалённо в сеансе X Window: в одном окне вывод dmesg, в другом — файл конфигурации. Иногда требуется несколько проходов, чтобы собрать сильно урезанное ядро, поскольку легко случайно удалить зависимости.

19.9.3. Сборка нового ядра

Теперь пора собрать ядро и установить его. В каталоге conf на ftp-сервере следующая команда подготавливает сборку:

```
$ config FTP
```

По завершении появится сообщение с напоминанием выполнить make depend; далее:

```
$ cd ../compile/FTP
$ make depend && make
```

Когда всё готово, я делаю резервную копию старого ядра и ставлю на его место новое:

```
# cp /netbsd /netbsd.orig
# cp netbsd /
```

Теперь перезагрузитесь. Если ядро не загружается, остановите процесс загрузки по приглашению и введите `boot netbsd.orig`, чтобы загрузиться с предыдущего ядра.

19.9.4. Уменьшение размера ядра NetBSD

При сборке ядра для встраиваемых систем часто требуется изменить двоичный файл ядра, чтобы уменьшить занимаемое место или объём памяти.

19.9.4.1. Удаление разделов ELF и отладочной информации

Мы уже знаем, как убрать из ядра поддержку ненужных драйверов и параметров, экономя таким образом память и место, но можно сэкономить ещё несколько килобайт, удалив отладочные символы и два раздела ELF, если они не нужны: `.comment` и `.ident`. Они служат для хранения строк RCS, которые можно просмотреть с помощью ident(1), и строки версии gcc(1). В следующих примерах предполагается, что `TOOLDIR` находится в `/usr/src/tooldir.NetBSD-2.0-i386`, а целевая архитектура — `i386`.

```
$ /usr/src/tooldir.NetBSD-2.0-i386/bin/i386--netbsdelf-objdump -h /netbsd

/netbsd:     file format elf32-i386

Sections:
Idx Name          Size      VMA       LMA       File off  Algn
  0 .text         0057a374  c0100000  c0100000  00001000  2**4
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .rodata       00131433  c067a380  c067a380  0057b380  2**5
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .rodata.str1.1 00035ea0  c07ab7b3  c07ab7b3  006ac7b3  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .rodata.str1.32 00059d13  c07e1660  c07e1660  006e2660  2**5
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 link_set_malloc_types 00000198  c083b374  c083b374  0073c374  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  5 link_set_domains 00000024  c083b50c  c083b50c  0073c50c  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 link_set_pools 00000158  c083b530  c083b530  0073c530  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 link_set_sysctl_funcs 000000f0  c083b688  c083b688  0073c688  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  8 link_set_vfsops 00000044  c083b778  c083b778  0073c778  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  9 link_set_dkwedge_methods 00000004  c083b7bc  c083b7bc  0073c7bc  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 10 link_set_bufq_strats 0000000c  c083b7c0  c083b7c0  0073c7c0  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 11 link_set_evcnts 00000030  c083b7cc  c083b7cc  0073c7cc  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 12 .data         00048ae4  c083c800  c083c800  0073c800  2**5
                  CONTENTS, ALLOC, LOAD, DATA
 13 .bss          00058974  c0885300  c0885300  00785300  2**5
                  ALLOC
 14 .comment      0000cda0  00000000  00000000  00785300  2**0
                  CONTENTS, READONLY
 15 .ident        000119e4  00000000  00000000  007920a0  2**0
                  CONTENTS, READONLY
```

В третьем столбце виден размер разделов в шестнадцатеричном виде. Сложив размеры `.comment` и `.ident`, мы узнаем, сколько сэкономим, удалив их: около 120 КБ (= 52640 + 72164 = 0xcda0 + 0x119e4). Чтобы удалить эти разделы и отладочные символы, если они есть, воспользуемся strip(1):

```
# cp /netbsd /netbsd.orig
# /usr/src/tooldir.NetBSD-2.0-i386/bin/i386--netbsdelf-strip -S -R .ident -R .comment /netbsd
# ls -l /netbsd /netbsd.orig
-rwxr-xr-x  1 root  wheel  8590668 Apr 30 15:56 netbsd
-rwxr-xr-x  1 root  wheel  8757547 Apr 30 15:56 netbsd.orig
```

Поскольку мы удалили и отладочные символы, общий объём сэкономленного дискового пространства составляет около 160 КБ.

19.9.4.2. Сжатие ядра

На некоторых архитектурах загрузчик может загружать сжатое ядро. Этот метод позволяет сэкономить несколько мегабайт дискового пространства, но загрузчику потребуется больше времени на загрузку ядра.

```
# cp /netbsd /netbsd.plain
# gzip -9 /netbsd
```

Чтобы увидеть, сколько места мы сэкономили:

```
$ ls -l /netbsd.plain /netbsd.gz
-rwxr-xr-x  1 root  wheel  8757547 Apr 29 18:05 /netbsd.plain
-rwxr-xr-x  1 root  wheel  3987769 Apr 29 18:05 /netbsd.gz
```

Учтите, что можно использовать только сжатие gzip с помощью gzip(1); bzip2 не поддерживается загрузчиками NetBSD!

Глава 20. Подсистема Veriexec в NetBSD

Veriexec — подсистема проверки целостности файлов в NetBSD. Она реализована в ядре и потому может обеспечить некоторую защиту даже в случае компрометации root.

20.1. Как это работает

Veriexec работает, загружая в ядро файл спецификации, который также называют **файлом подписей**. Этот файл содержит сведения о файлах, которые Veriexec должен контролировать, их цифровой отпечаток (вместе с алгоритмом хеширования, использованным для его получения), а также различные флаги, которые будут рассмотрены позже.

В настоящее время Veriexec поддерживает следующие алгоритмы хеширования: **MD5**, **SHA1**, **SHA256**, **SHA384**, **SHA512** и **RMD160**.

20.2. Файл подписей

Запись в файле подписей Veriexec выглядит так:

```
        /path/to/file algorithm fingerprint flags
```

Первый элемент — путь — всегда должен быть абсолютным. Алгоритм — один из перечисленных выше, а fingerprint — отпечаток в формате ASCII.

20.3. Генерация отпечатков

Отпечатки ASCII для каждого алгоритма можно создать с помощью следующих средств:

Таблица 20.1. Средства создания отпечатков для Veriexec

Алгоритм	***Средство***
MD5	`/usr/bin/cksum -a md5`
SHA1	`/usr/bin/cksum -a sha1`
SHA256	`/usr/bin/cksum -a sha256`
SHA384	`/usr/bin/cksum -a sha384`
SHA512	`/usr/bin/cksum -a sha512`
RMD160	`/usr/bin/cksum -a rmd160`

Например, чтобы создать отпечаток MD5 для `/bin/ls`:

```
        % cksum -a md5 < /bin/ls
        a8b525da46e758778564308ed9b1e493
```

А чтобы создать отпечаток SHA512 для `/bin/ps`:

```
        % cksum -a sha512 < /bin/ps

381d4ad64fd47800897446a2026eca42151e03adeae158db5a34d12c529559113d928a9fef9a7c4615d257688d1da4
645db004081030d7f080bb7198067eb890
```

С каждой записью может быть связано ноль или более флагов. В настоящее время эти флаги указывают, как следует обращаться к файлу, описываемому записью. Учтите, что этот тип доступа проверяется только на строгом уровне 2 (режим IPS) и выше.

Можно использовать следующие типы доступа: «DIRECT», «INDIRECT» и «FILE».

- **DIRECT** означает, что файл выполняется напрямую, а не вызывается как интерпретатор некоторого сценария и не открывается редактором. Обычно большинство используемых вами программ доступны именно в этом режиме:

  ```
          % ls /tmp
          % cp ~/foo /tmp/bar
          % rm ~/foo
  ```

- **INDIRECT** означает, что файл выполняется косвенно и вызывается для интерпретации сценария. Обычно так бывает, когда первой строкой сценария является конструкция #!. Например, если первая строка вашего сценария выглядит так:

  ```
          #!/bin/sh
  ```

  и вы запускаете его так:

  ```
          % ./script.sh
  ```

  то `/bin/sh` будет выполнен косвенно — он будет вызван для интерпретации сценария.

- Записи **FILE** относятся ко всему, что не является (или не должно быть) исполняемым файлом. Сюда входят разделяемые библиотеки, файлы конфигурации и т. п.

  Несколько примеров записей файла подписей Veriexec:

  ```
          /bin/ls         MD5 dc2e14dc84bdefff4bf9777958c1b20b DIRECT
          /usr/bin/perl   MD5 914aa8aa47ebd79ccd7909a09ed61f81 INDIRECT
          /etc/pf.conf    MD5 950e1dd6fcb3f27df1bf6accf7029f7d FILE
  ```

Veriexec позволяет указать в одной записи несколько способов доступа к файлу. Например, хотя `/usr/bin/perl` в основном используется как интерпретатор, может потребоваться и возможность запускать его напрямую:

```
        /usr/bin/perl MD5 914aa8aa47ebd79ccd7909a09ed61f81 DIRECT, INDIRECT
```

Сценариям оболочки, использующим конструкцию #!, чтобы быть «исполняемыми», также требуются два типа доступа: нужен «DIRECT», чтобы их можно было выполнять, и нужен «FILE», чтобы ядро могло передать их содержимое указанному в них интерпретатору:

```
        /usr/src/build.sh MD5 e80dbb4c047ecc1d84053174c1e9264a DIRECT, FILE
```

Чтобы упростить создание файлов подписей и сделать сами файлы подписей более читаемыми, Veriexec позволяет использовать следующие псевдонимы:

Таблица 20.2. Псевдонимы типов доступа Veriexec

Псевдоним	***Развёртывается в***
PROGRAM	*DIRECT*
INTERPRETER	*INDIRECT*
SCRIPT	*DIRECT, FILE*
LIBRARY	*FILE*

Создав файл подписей, сохраните его как `/etc/signatures` и включите Veriexec в `rc.conf`:

```
        veriexec=YES
```

20.4. Строгие уровни

Поскольку разные люди могут использовать Veriexec для разных целей, мы также определяем четыре строгих уровня от 0 до 3, называемых режимами «learning», «IDS», «IPS» и «lockdown».

На **строгом уровне 0**, в режиме обучения, Veriexec действует пассивно и лишь предупреждает об аномалиях. В сочетании с уровнем подробности 1 работа системы в этом режиме помогает точно настроить файл подписей. Это также единственный строгий уровень, на котором можно загружать в ядро новые записи.

Строгий уровень 1, или режим IDS, запрещает доступ к файлам с несовпадающим отпечатком. Этот режим подходит в основном тем, кто просто хочет не допустить доступ к файлам, которые могли быть злонамеренно изменены злоумышленником.

Строгий уровень 2, режим IPS, делает шаг к защите целостности контролируемых файлов. Помимо запрета доступа к файлам с несовпадающим отпечатком, он также запрещает запись, не позволяет удалять контролируемые файлы и навязывает способ доступа к ним (как указано в файле подписей).

Режим lockdown (**строгий уровень 3**) можно применять в особо ответственных ситуациях, например на специально собранных машинах узкого назначения, или как последнюю линию обороны, когда злоумышленник скомпрометировал систему и нужно не дать удалить следы, чтобы можно было провести анализ после инцидента. Он запрещает создание новых файлов и доступ к файлам, не контролируемым Veriexec.

Рекомендуется сначала запустить Veriexec на строгом уровне 0 с уровнем подробности 1, чтобы точно настроить файл подписей и убедиться, что нужные приложения работают правильно, и только затем повышать строгий уровень (и понижать уровень подробности). С помощью `/etc/sysctl.conf` можно автоматически повышать строгий уровень до нужного после перезагрузки:

```
        kern.veriexec.strict=1
```

20.5. Veriexec и многослойные файловые системы

Veriexec можно использовать на файловых системах NFS на стороне клиента и на многослойных файловых системах, таких как union. Достаточно указать файлы, находящиеся на этих файловых системах, в файле `/etc/signatures` и смонтировать эти файловые системы до загрузки отпечатков.

Если вы собираетесь использовать многослойные файловые системы, необходимо включить отпечатки защищаемых файлов на каждом слое. Иначе кто-то сможет перезаписать файл, защищённый Veriexec, через другой слой стека многослойной файловой системы. В последующих версиях NetBSD это ограничение может быть устранено.

Рекомендуется: если вы не собираетесь использовать ни многослойные файловые системы, ни NFS, эти возможности следует отключить в конфигурации ядра. Если многослойные файловые системы нужны, необходимо выполнить указания из предыдущего абзаца и убедиться, что защищаемые файлы имеют отпечатки на всех слоях. Кроме того, после выполнения всех монтирований следует повысить securelevel до 2:

```
        kern.securelevel=2
```

чтобы не допустить монтирования новых слоёв, которые могли бы подорвать защиту Veriexec.

20.6. Конфигурация ядра

Чтобы использовать Veriexec, помимо создания файла подписей нужно включить (раскомментировать) его в файле конфигурации ядра (например, `/usr/src/sys/arch/i386/conf/GENERIC`):

```
        pseudo-device veriexec
```

Затем нужно включить алгоритмы хеширования, которые вы хотите поддерживать:

```
        options VERIFIED_EXEC_FP_MD5
        options VERIFIED_EXEC_FP_SHA1
        options VERIFIED_EXEC_FP_RMD160
        options VERIFIED_EXEC_FP_SHA512
        options VERIFIED_EXEC_FP_SHA384
        options VERIFIED_EXEC_FP_SHA256
```

В зависимости от версии операционной системы и платформы они могут быть уже включены. После этого пересоберите и переустановите ядро; дальнейшие указания см. в главе 34, *Сборка ядра*.

Если устройства Veriexec `/dev/veriexec` нет, его можно создать вручную, выполнив следующую команду:

```
# cd /dev
# sh MAKEDEV veriexec
```

Глава 21. Bluetooth в NetBSD

21.1. Введение

Bluetooth — цифровой радиопротокол, используемый для связи на коротких расстояниях с малой мощностью. NetBSD включает поддержку стека протоколов Bluetooth и некоторую интеграцию профилей служб в инфраструктуру устройств NetBSD.

Нижние уровни стека протоколов Bluetooth, относящиеся к собственно радиосвязи между устройствами, реализованы внутри контроллера Bluetooth, который взаимодействует с хост-компьютером по протоколу «интерфейс контроллера хоста» (HCI), доступному через сокетный интерфейс необработанных пакетов BTPROTO_HCI.

Большинство протоколов и служб Bluetooth работают поверх «протокола управления и адаптации канального уровня» (L2CAP), доступного через сокетный интерфейс BTPROTO_L2CAP. Он обеспечивает последовательные пакетные соединения с удалёнными устройствами, до 64 тысяч каналов. При открытии канала L2CAP требуемый протокол или служба определяются по значению «мультиплексор протоколов и служб» (PSM).

Обнаружение служб в среде Bluetooth обеспечивается библиотечными функциями sdp(3) и демоном sdpd(8), который ведёт базу данных локально зарегистрированных служб и предоставляет эти сведения удалённым устройствам, выполняющим запросы. Средство sdpquery(1) можно использовать для запросов к локальной и к удалённым базам данных служб.

Безопасность соединений Bluetooth можно включить с помощью параметров шифрования и аутентификации команды btconfig(8), которые применяются ко всем соединениям базового диапазона, устанавливаемым контроллером, либо, при необходимости, включить шифрование и аутентификацию для отдельных соединений RFCOMM и L2CAP. При запросе аутентификации каждая сторона вводит PIN (обычно его вводит оператор; у некоторых устройств с ограниченным вводом PIN фиксирован). Контроллер использует этот PIN для генерации ключа соединения (Link Key) и сообщает его хосту, у которого он может быть запрошен для аутентификации последующих соединений. В NetBSD демон bthcid(8) отвечает за хранение ключей соединения и ответы на запросы ключей соединения, а также предоставляет интерфейс, позволяющий непривилегированным пользователям задавать PIN с помощью клиента PIN, например btpin(1).

21.2. Поддерживаемое оборудование

Поскольку контроллеры Bluetooth обычно общаются с хостом по стандартному протоколу HCI, описанному в документации «Bluetooth 2.0 Core», стек Bluetooth в NetBSD совместим с большинством контроллеров и требует только драйвер интерфейса:

- bcsp(4) предоставляет дисциплину линии tty(4) для передачи и приёма пакетов BlueCore Serial Protocol по последовательной линии, как описано в спецификации «BlueCore Serial Protocol (BCSP)».
- bt3c(4) предоставляет интерфейс к плате 3Com Bluetooth PC Card модели 3CRWB6096-A.
- btbc(4) обеспечивает поддержку устройств PCMCIA AnyCom BlueCard (LSE041, LSE039, LSE139).
- btuart(4) предоставляет дисциплину линии tty(4) для передачи и приёма пакетов Bluetooth по последовательной линии, как описано в спецификации «Bluetooth Host Controller Interface

[Transport Layer] specification, Vol 4 part A».

- sbt(4) обеспечивает поддержку адаптеров Bluetooth Secure Digital IO.
- ubt(4) обеспечивает работу со всеми USB-контроллерами Bluetooth, соответствующими спецификации «HCI USB Transport Layer».

Если оборудование поддерживается стеком Bluetooth в NetBSD, в выводе **dmesg** появятся сообщения автонастройки, например:

```
bt3c0 at pcmcia0 function 0: <3COM, 3CRWB60-A, Bluetooth PC Card>

ubt0 at uhub1 port 4 configuration 1 interface 0
ubt0: Cambridge Silicon Radio Bluetooth USB Adapter, rev 2.00/19.58, addr 4

ubt1 at uhub1 port 2 configuration 1 interface 0
ubt1: Broadcom Belkin Bluetooth Device, rev 1.10/0.01, addr 5
```

Если поддержка не вкомпилирована, её можно добавить в файл конфигурации ядра для любой платформы, поддерживающей USB и/или PCMCIA (см. раздел 19.9, «Настройка ядра»), с помощью следующих объявлений по мере необходимости:

```
# Bluetooth Controller and Device support

pseudo-device   bcsp                    # BlueCore Serial Protocol
pseudo-device   btuart                  # Bluetooth HCI UART

# Bluetooth PCMCIA Controllers
bt3c* at pcmcia? function ?              # 3Com 3CRWB6096-A
btbc* at pcmcia? function ?              # AnyCom BlueCard LSE041/039/139

# Bluetooth SDIO Controllers
sbt* at sdmmc?

# Bluetooth USB Controllers
ubt* at uhub? port ?

# Bluetooth Device Hub
bthub* at bcsp?
bthub* at bt3c?
bthub* at btbc?
bthub* at btuart?
bthub* at sbt?
bthub* at ubt?

# Bluetooth HID support
bthidev* at bthub?

# Bluetooth Mouse
btms* at bthidev? reportid ?
wsmouse* at btms? mux 0

# Bluetooth Keyboard
btkbd* at bthidev? reportid ?
wskbd* at btkbd? console ? mux 1

# Bluetooth Audio support
btsco* at bthub?
```

Некоторым старым USB-адаптерам Bluetooth на микросхеме Broadcom BCM2033 требуется загрузка встроенного ПО, прежде чем они смогут работать; такие устройства подключаются как ugen(4). Используйте пакет «sysutils/bcmfw» из коллекции пакетов NetBSD, чтобы загрузить встроенное ПО и включить их.

21.3. Настройка системы

Чтобы полностью включить службы Bluetooth в NetBSD, в файл `/etc/rc.conf` следует добавить следующую строку.

```
bluetooth=YES
```

а затем либо перезагрузиться, либо выполнить следующую команду:

```
# service bluetooth start
```

Настройка контроллеров Bluetooth выполняется программой btconfig(8); указанный выше аргумент включает только базовые возможности, другие полезные параметры см. на странице руководства. Дополнительные параметры btconfig для конкретного устройства, скажем utb0, можно задать, добавив для него строку в файл `/etc/rc.conf`.

```
btconfig_ubt0="name MyComputerName"
```

> **ВАЖНО**
> bthcid(8) *должен* быть запущен, чтобы устанавливать аутентифицированные соединения с удалёнными устройствами; аутентификацию может запросить любое из устройств.

21.4. Устройства человеко-машинного интерфейса

Поддержка «устройств человеко-машинного интерфейса» (HID), которые работают по протоколу USB HID через пару каналов L2CAP, обеспечивается драйвером bthidev(4). В настоящее время поддерживаются клавиатуры и мыши, которые подключаются к wscons(4) как обычно.

21.4.1. Мыши

Мыши Bluetooth можно подключить к системе с помощью драйвера btms(4), используя btdevctl(8).

Сначала нужно определить BDADDR устройства. Возможно, он напечатан на коробке, но проще всего перевести устройство в режим обнаружимости и выполнить поиск устройств с помощью соответствующего контроллера:

```
% btconfig ubt0 inquiry
Device Discovery on ubt0 .... 1 response
  1: bdaddr 00:14:51:c1:b9:2d (unknown)
   : name "Mighty Mouse"
   : class: [0x002580] Peripheral Mouse <Limited Discoverable>
   : page scan rep mode 0x01
   : page scan period mode 0x02
```

```
    : page scan mode 0x00
    : clock offset 6944
```

Для удобства можно добавить адрес в файл `/etc/bluetooth/hosts`, чтобы обращаться к мыши по псевдониму:

```
# echo "00:14:51:c1:b9:2d mouse" >>/etc/bluetooth/hosts
```

Теперь можно опросить мышь; скорее всего, она запросит аутентификацию, прежде чем принимать подключения. Фиксированный PIN должен быть указан в документации, хотя часто используется «0000». Сначала задайте PIN с помощью программы btpin(1):

```
% btpin -d ubt0 -a mouse -p 0000
# btdevctl -d ubt0 -a mouse -s HID
local bdaddr: 00:08:1b:8d:ba:6d
remote bdaddr: 00:14:51:c1:b9:2d
link mode: auth
device type: bthidev
control psm: 0x0011
interrupt psm: 0x0013
Collection page=Generic_Desktop usage=Mouse
  Input id=2 size=1 count=1 page=Button usage=Button_1 Variable, logical range 0..1
  Input id=2 size=1 count=1 page=Button usage=Button_2 Variable, logical range 0..1
  Input id=2 size=1 count=1 page=Button usage=Button_3 Variable, logical range 0..1
  Input id=2 size=1 count=1 page=Button usage=Button_4 Variable, logical range 0..1
  Input id=2 size=4 count=1 page=0x0000 usage=0x0000 Const Variable, logical range 0..1
Collection page=Generic_Desktop usage=Pointer
  Input id=2 size=8 count=1 page=Generic_Desktop usage=X Variable Relative, logical range
-127..127
  Input id=2 size=8 count=1 page=Generic_Desktop usage=Y Variable Relative, logical range
-127..127
  Input id=2 size=8 count=1 page=Consumer usage=AC_Pan Variable Relative, logical range
-127..127
  Input id=2 size=8 count=1 page=Generic_Desktop usage=Wheel Variable Relative, logical range
-127..127
End collection
  Input id=2 size=8 count=1 page=0x00ff usage=0x00c0 Variable, logical range -127..127
Feature id=71 size=8 count=1 page=0x0006 usage=0x0020 Variable NoPref Volatile, logical range
0..100
End collection
```

Это означает, что мышь ответила на SDP-запрос и показаны возможности устройства. Учтите, что для Bluetooth-мышей аутентификация включена по умолчанию. Теперь можно подключить устройство к системе:

```
# btdevctl -d ubt0 -a mouse -s HID -A
```

при этом на системной консоли должны появиться сообщения:

```
bthidev0 at bthub0 remote-bdaddr 00:14:51:c1:b9:2d link-mode auth
btms0 at bthidev1 reportid 2: 4 buttons, W and Z dirs.
wsmouse1 at btms0 mux 0
bthidev1: reportid 71 not configured
```

```
bthidev1: connected
```

и мышь должна заработать.

Возможности устройства кэшируются btdevctl(8), и чтобы повторно подключать мышь при запуске системы, добавьте запись в `/etc/bluetooth/btdevctl.conf`. Драйвер bthidev(4) попытается подключиться один раз, но мыши обычно спят, и их, возможно, придётся «разбудить» — в этом случае они сами инициируют подключение к компьютеру-хосту.

21.4.2. Клавиатуры

Bluetooth-клавиатуры можно подключить к системе с помощью драйвера btkbd(4), используя btdevctl(8).

Сначала нужно определить BDADDR устройства. Возможно, он напечатан на коробке, но проще всего перевести устройство в режим обнаружимости и выполнить поиск устройств с помощью соответствующего контроллера:

```
% btconfig ubt0 inquiry
Device Discovery on ubt0 .... 1 response
  1: bdaddr 00:0a:95:45:a4:a0 (unknown)
   : name "Apple Wireless Keyboard"
   : class: [0x002540] Peripheral Keyboard <Limited Discoverable>
   : page scan rep mode 0x01
   : page scan period mode 0x00
   : page scan mode 0x00
   : clock offset 18604
```

Для удобства можно добавить адрес в файл `/etc/bluetooth/hosts`, чтобы обращаться к клавиатуре по псевдониму:

```
# echo "00:0a:95:45:a4:a0 keyboard" >>/etc/bluetooth/hosts
```

Теперь можно опросить клавиатуру; скорее всего, она запросит аутентификацию, прежде чем принимать подключения. PIN нужно будет ввести на клавиатуре, а сгенерировать случайный PIN можно с помощью программы btpin(1).

```
% btpin -d ubt0 -a keyboard -r -l 8
PIN: 18799632
# btdevctl -d ubt0 -a keyboard -s HID

    < ENTER PIN ON BLUETOOTH KEYBOARD NOW >

local bdaddr: 00:08:1b:8d:ba:6d
remote bdaddr: 00:0a:95:45:a4:a0
link mode: encrypt
device type: bthidev
control psm: 0x0011
interrupt psm: 0x0013
Collection page=Generic_Desktop usage=Keyboard
  Input id=1 size=1 count=1 page=Keyboard usage=Keyboard_LeftControl Variable, logical range
0..1
  Input id=1 size=1 count=1 page=Keyboard usage=Keyboard_LeftShift Variable, logical range
0..1
  Input id=1 size=1 count=1 page=Keyboard usage=Keyboard_LeftAlt Variable, logical range 0..1
```

```
  Input id=1 size=1 count=1 page=Keyboard usage=Keyboard_Left_GUI Variable, logical range 0..1
  Input id=1 size=1 count=1 page=Keyboard usage=Keyboard_RightControl Variable, logical range
0..1
  Input id=1 size=1 count=1 page=Keyboard usage=Keyboard_RightShift Variable, logical range
0..1
  Input id=1 size=1 count=1 page=Keyboard usage=Keyboard_RightAlt Variable, logical range 0..1
  Input id=1 size=1 count=1 page=Keyboard usage=Keyboard_Right_GUI Variable, logical range
0..1
  Input id=1 size=8 count=1 page=0x0000 usage=0x0000 Const, logical range 0..1
 Output id=1 size=1 count=1 page=LEDs usage=Num_Lock Variable, logical range 0..1
 Output id=1 size=1 count=1 page=LEDs usage=Caps_Lock Variable, logical range 0..1
 Output id=1 size=1 count=1 page=LEDs usage=Scroll_Lock Variable, logical range 0..1
 Output id=1 size=1 count=1 page=LEDs usage=Compose Variable, logical range 0..1
 Output id=1 size=1 count=1 page=LEDs usage=Kana Variable, logical range 0..1
 Output id=1 size=3 count=1 page=0x0000 usage=0x0000 Const, logical range 0..1
  Input id=1 size=8 count=6 page=Keyboard usage=No_Event, logical range 0..255
  Input id=1 size=1 count=1 page=Consumer usage=Eject Variable Relative, logical range 0..1
  Input id=1 size=1 count=1 page=Consumer usage=Mute Variable Relative, logical range 0..1
  Input id=1 size=1 count=1 page=Consumer usage=Volume_Up Variable, logical range 0..1
  Input id=1 size=1 count=1 page=Consumer usage=Volume_Down Variable, logical range 0..1
  Input id=1 size=1 count=4 page=0x0000 usage=0x0000 Const, logical range 0..1
End collection
```

Это означает, что клавиатура ответила на SDP-запрос и показаны возможности устройства. Учтите, что шифрование включено по умолчанию, поскольку поддержка зашифрованных соединений обязательна для Bluetooth-клавиатур. Теперь можно подключить устройство к системе:

```
# btdevctl -d ubt0 -a keyboard -s HID -A
```

при этом на системной консоли должны появиться сообщения:

```
bthidev1 at bthub0 remote-bdaddr 00:0a:95:45:a4:a0 link-mode encrypt
btkbd0 at bthidev0 reportid 1
wskbd1 at btkbd0 mux 1
wskbd1: connecting to wsdisplay0
bthidev1: connected
```

и клавиатура должна заработать.

Возможности устройства кэшируются btdevctl(8), и чтобы повторно подключать клавиатуру при запуске системы, добавьте запись в `/etc/bluetooth/btdevctl.conf`. Драйвер bthidev(4) попытается подключиться один раз при подключении, но если клавиатуры в этот момент нет, нажатие клавиши может её разбудить, и она инициирует подключение к последнему спаренному хосту.

21.5. Персональные сети

Службы персональных сетей по Bluetooth предоставляет демон btpand(8), который может выполнять все роли профиля PAN и подключает удалённые устройства к системе через виртуальный Ethernet-интерфейс tap(4).

21.5.1. Пользователь персональной сети

Роль «пользователь персональной сети» (PAN User) — это клиент, который обращается к сетевым службам на другом устройстве. Например, чтобы подключиться к Интернету через смартфон с

профилем NAP, убедитесь, что телефон обнаружим, затем:

```
% btconfig ubt0 inquiry
Device Discovery from device: ubt0 .... 1 response
  1: bdaddr 00:17:83:30:bd:5e (unknown)
   : name "HTC Touch"
   : class: [0x5a020c] Smart Phone <Networking> <Capturing> <Object Transfer>
        <Telephony>
   : page scan rep mode 0x01
   : clock offset 9769
   : rssi -42

# echo "00:17:83:30:bd:5e phone" >>/etc/bluetooth/hosts
```

Вы увидите, что у телефона должен быть установлен флаг в поле Class of Device. Проверка службы NAP:

```
% sdpquery -a phone search NAP
ServiceRecordHandle: 0x00010000
ServiceClassIDList:
    Network Access Point
ProtocolDescriptorList:
    L2CAP (PSM 0x000f)
    BNEP (v1.0; IPv4, ARP, IPv6)
LanguageBaseAttributeIDList:
    en.UTF-8 base 0x0100
BluetoothProfileDescriptorList:
    Network Access Point, v1.0
ServiceName: "Network Access Point"
ServiceDescription: "Bluetooth NAP Service"
SecurityDescription: None
NetAccessType: 100Mb Ethernet
MaxNetAccessRate: 100000
```

показывает, что служба NAP доступна и предоставляет протоколы IPv4, ARP и IPv6.

Скорее всего, телефон запросит аутентификацию, прежде чем разрешит подключения к службе NAP, поэтому перед первым подключением может потребоваться ввести PIN, который можно сгенерировать случайно. Затем запустите btpand(8):

```
% btpin -d ubt0 -a phone -r -l 6
PIN: 862048
# btpand -d ubt0 -a phone -s NAP

    < ENTER PIN ON PHONE NOW >

Searching for NAP service at 00:17:83:30:bd:5e
Found PSM 15 for service NAP
Opening connection to service 0x1116 at 00:17:83:30:bd:5e
Using interface tap0 with addr 00:10:60:e1:50:3d
```

Наконец, нужно настроить интерфейс tap(4), но на телефоне должен работать DHCP-сервер, так что dhcpcd(8) сделает это за вас.

```
# dhcpcd tap0
```

Теперь можно работать во Всемирной паутине, но следите за расходом трафика, если у вас нет безлимитного тарифа.

21.6. Последовательные соединения

Последовательные соединения по Bluetooth обеспечивает протокол RFCOMM, который предоставляет до 30 каналов, мультиплексируемых в одном канале L2CAP. Доступ к этому потоковому протоколу можно получить через интерфейс сокетов BTPROTO_RFCOMM или с помощью программы rfcomm_sppd(1).

Например, можно установить последовательное соединение со службой «Dial Up Networking» (DUN) мобильного телефона, чтобы подключаться к Интернету по PPP. Сначала нужно определить BDADDR телефона и для удобства добавить его в `/etc/bluetooth/hosts`. Переведите телефон в режим обнаружимости и выполните поиск с помощью соответствующего контроллера:

```
% btconfig ubt0 inquiry
Device Discovery from device: ubt0 ..... 1 response
  1: bdaddr 00:16:bc:00:e8:48 (unknown)
   : name "Nokia 6103"
   : class: [0x520204] Cellular Phone <Networking> <Object Transfer> <Telephony>
   : page scan rep mode 0x01
   : page scan period mode 0x02
   : page scan mode 0x00
   : clock offset 30269

# echo "00:16:bc:00:e8:48 phone" >>/etc/bluetooth/hosts
```

Теперь можно опросить телефон и убедиться, что он поддерживает профиль DUN:

```
% sdpquery -d ubt0 -a phone search DUN
ServiceRecordHandle: 0x00010003
ServiceClassIDList:
    Dialup Networking
    Generic Networking
ProtocolDescriptorList:
    L2CAP
    RFCOMM (channel 1)
BrowseGroupList:
    Public Browse Root
LanguageBaseAttributeIDList:
    en.UTF-8 base 0x0100
BluetoothProfileDescriptorList:
    Dialup Networking, v1.0
ServiceName: "Dial-up networking"
```

Скорее всего, телефон запросит аутентификацию, прежде чем разрешит подключения к службе DUN, поэтому перед первым подключением может потребоваться ввести PIN, который можно сгенерировать случайно. Чтобы проверить, что соединение работает нормально, можно запустить **rfcomm_sppd** в режиме stdio; нажмите `^C`, чтобы отключиться и вернуться в оболочку. Например:

```
% btpin -d ubt0 -a phone -r -l 6
PIN: 904046
% rfcomm_sppd -d ubt0 -a phone -s DUN

    < ENTER PIN ON PHONE NOW >
```

```
rfcomm_sppd[24635]: Starting on stdio...
at
OK
ati
Nokia

OK
ati3
Nokia 6103

OK
at&v
ACTIVE PROFILE:
E1 Q0 V1 X5 &C1 &D2 &S0 &Y0
+CMEE=0 +CSTA=129 +CBST=0,0,1 +CRLP=61,61,48,6 +CR=0 +CRC=0 +CLIP=0,2
+CLIR=0,2 +CSNS=0 +CVHU=1 +DS=0,0,2048,32 +DR=0 +ILRR=0
+CHSN=0,0,0,0 +CHSR=0 +CPBS="SM"
S00:000 S01:000 S02:043 S03:013 S04:010 S05:008 S07:060 S08:002
S10:100 S12:050 S25:000

OK
^C
rfcomm_sppd[24635]: Completed on stdio
```

Чтобы pppd(8) автоматически подключался к службе DUN вашего телефона при установлении исходящих соединений, добавьте следующую строку в файл `/etc/ppp/options` вместо обычного объявления tty:

```
pty "rfcomm_sppd -d ubt0 -a phone -s DUN -m encrypt"
```

21.7. Звук

Изохронные звуковые соединения (SCO) можно создавать на радиоканале базовой полосы либо через интерфейс сокетов BTPROTO_SCO, либо с помощью драйвера звукового устройства btsco(4). Хотя спецификация допускает до трёх таких соединений между устройствами, текущий стек Bluetooth способен сколько-нибудь сносно обслуживать только одно из них.

> **ВАЖНО**
>
> При использовании звука SCO с USB-контроллерами Bluetooth потребуется включить изохронные данные и рассчитать MTU, который будет использовать устройство; см. ubt(4) и btconfig(8).

> **ПРИМЕЧАНИЕ**
>
> Звук SCO не работает должным образом с драйвером bt3c(4); для наилучшего результата используйте USB-контроллер.

21.7.1. Гарнитуры со звуком SCO

Звуковые соединения с Bluetooth-гарнитурами возможны с помощью драйвера звука btsco(4) и программы bthset(1). Сначала нужно определить BDADDR гарнитуры и, вероятно, для удобства

создать псевдоним в файле `/etc/bluetooth/hosts`. Переведите гарнитуру в режим обнаружимости и выполните поиск с помощью соответствующего контроллера:

```
% btconfig ubt0 inquiry
Device Discovery from device: ubt0 ..... 1 response
 1: bdaddr 00:07:a4:23:10:83 (unknown)
  : name "JABRA 250"
  : class: [0x200404] Wearable Headset <Audio>
  : page scan rep mode 0x01
  : page scan period mode 0x00
  : page scan mode 0x00
  : clock offset 147

# echo "00:07:a4:23:10:83 headset" >>/etc/bluetooth/hosts
```

При первом подключении потребуется выполнить сопряжение с гарнитурой; фиксированный PIN должен быть указан в руководстве (часто используется «0000»). btdevctl(8) опросит устройство и подключит драйвер звука btsco(4).

```
% btpin -d ubt0 -a headset -p 0000
# btdevctl -d ubt0 -a headset -s HSET -A
local bdaddr: 00:08:1b:8d:ba:6d
remote bdaddr: 00:07:a4:23:10:83
link mode: none
device type: btsco
mode: connect
channel: 1
```

при этом на системной консоли должны появиться сообщения:

```
btsco0 at bthub0 remote-bdaddr 00:07:a4:23:10:83 channel 1
audio1 at btsco0: full duplex
```

Чтобы использовать звуковое устройство, нужно открыть управляющее соединение с помощью bthset(1), которое передаёт информацию о громкости микшеру.

```
% bthset -m /dev/mixer1 -v
Headset Info:
        mixer: /dev/mixer1
        laddr: 00:08:1b:8d:ba:6d
        raddr: 00:07:a4:23:10:83
        channel: 1
        vgs.dev: 0, vgm.dev: 1
```

и теперь вы сможете передавать отсчёты с частотой 8 кГц в `/dev/audio1` и из него с помощью любой программы, поддерживающей звук, например audioplay(1) или audiorecord(1). Изменение значений микшера должно работать при воспроизведении, хотя при открытии соединения гарнитура может сбрасывать громкость к последним известным настройкам.

```
% audiorecord -d /dev/audio1 voice.au

        < TALK NONSENSE NOW >
```

```
^C
% audioplay -d /dev/audio voice.au

        < THATS REALLY WHAT YOU SOUND LIKE >

% audioplay -d /dev/audio1 voice.au

        < IN THE HEADSET >
```

Возможности устройства кэшируются btdevctl(8); чтобы драйвер btsco(4) подключался снова при запуске системы, добавьте запись в `/etc/bluetooth/btdevctl.conf`.

21.7.2. Режим Hands-free со звуком SCO

Звуковые соединения с мобильными телефонами Bluetooth по профилю Handsfree возможны с помощью программы «comms/bthfp» из коллекции пакетов NetBSD.

Сначала нужно определить BDADDR телефона и, вероятно, для удобства создать псевдоним в файле `/etc/bluetooth/hosts`. Переведите телефон в режим обнаружимости и выполните поиск с помощью соответствующего контроллера:

```
% btconfig ubt0 inquiry
Device Discovery from device: ubt0 ..... 1 response
  1: bdaddr 00:16:bc:00:e8:48 (unknown)
   : name "Nokia 6103"
   : class: [0x520204] Cellular Phone <Networking;gt; <Object Transfer;gt; <Telephony;gt;
   : page scan rep mode 0x01
   : page scan period mode 0x02
   : page scan mode 0x00
   : clock offset 10131

# echo "00:16:bc:00:e8:48 phone" >>/etc/bluetooth/hosts
```

Теперь можно опросить телефон и убедиться, что он поддерживает профиль Handsfree:

```
% sdpquery -d ubt0 -a phone search HF
ServiceRecordHandle: 0x00010006
ServiceClassIDList:
    Handsfree Audio Gateway
    Generic Audio
ProtocolDescriptorList:
    L2CAP
    RFCOMM (channel 13)
BrowseGroupList:
    Public Browse Root
LanguageBaseAttributeIDList:
    en.UTF-8 base 0x0100
BluetoothProfileDescriptorList:
    Handsfree, v1.5
ServiceName: "Voice Gateway"
Network: Ability to reject a call
SupportedFeatures:
    3 Way Calling
    Echo Cancellation/Noise Reduction
    Voice Recognition
    In-band Ring Tone
```

и вы сможете использовать программу bthfp для доступа к профилю Handsfree. При первом подключении может потребоваться PIN для сопряжения с телефоном, который можно сгенерировать случайно с помощью btpin(1):

```
% btpin -d ubt0 -a phone -r -l 6
PIN: 349163
% bthfp -d ubt0 -a phone -v

      < ENTER PIN ON PHONE NOW >
Handsfree channel: 13
Press ? for commands
Connecting.. ok
< AT+BRSF=20
> +BRSF: 47
Features: [0x002f] <3 way calling> <EC/NR> <Voice Recognition> <In-band ringtone> <reject
ability>
> OK
< AT+CIND=?
> +CIND: ("call",(0,1)),("service",(0,1)),("call_setup",(0-3)),("callsetup",(0-3))
> OK
< AT+CIND?
> +CIND: 0,1,0,0
> OK
< AT+CMER=3,0,0,1
> OK
< AT+CLIP=1
> OK
Service Level established
```

Когда телефон зазвонит, просто нажмите `a`, чтобы ответить, и звук должен пойти через устройство `/dev/audio`. Учтите, что для разговора с собеседником потребуется подключённый микрофон.

21.8. Обмен объектами

В настоящее время в NetBSD нет собственных средств OBEX; см. пакеты «comms/obexapp» или «comms/obexftp» из коллекции пакетов NetBSD.

21.9. Устранение неполадок

Когда кажется, что ничего не происходит, может оказаться полезной программа hcidump из пакета «sysutils/netbt-hcidump» коллекции пакетов NetBSD. Она умеет выводить дамп пакетов, которые входят в контроллеры Bluetooth в NetBSD и выходят из них, что очень помогает выявить проблему.

Глава 22. Прочие операции

В этой главе собраны различные темы в произвольном порядке

22.1. Установка менеджера загрузки

Программа установки NetBSD Sysinst обычно устанавливает менеджер загрузки NetBSD на жёсткий диск. При необходимости менеджер загрузки можно также установить или перенастроить позже с помощью команды **fdisk**. Например:

```
# fdisk -B wd0
```

Если NetBSD не загружается с жёсткого диска, можно загрузиться с установочной дискеты и запустить ядро с жёсткого диска. Вставьте установочный диск и в приглашении загрузчика введите следующую команду:

```
> boot wd0a:netbsd
```

Эта команда загружает ядро с жёсткого диска (используйте правильное устройство, например sd0a для диска SCSI).

> **ПРИМЕЧАНИЕ**
>
> Иногда **fdisk -B** не даёт ожидаемого результата (по крайней мере, так было у меня), вероятно, при установке или удалении других операционных систем. В этом случае попробуйте выполнить **fdisk -i**, а затем снова запустите **fdisk** из NetBSD.

22.2. Удаление disklabel

Хотя эту операцию не нужно выполнять часто, полезно знать, как её делать при необходимости. Убедитесь, что вы точно понимаете, что делаете, прежде чем выполнять такие операции. Например:

```
# dd if=/dev/zero of=/dev/rwd0c bs=8k count=1
```

Предыдущая команда удаляет disklabel (но не таблицу разделов MBR). Чтобы полностью очистить диск, нужно использовать всё устройство `rwd0d`. Например:

```
# dd if=/dev/zero of=/dev/rwd0d bs=8k
```

Приведённые выше команды работают как ожидается только на платформах i386 и amd64 NetBSD. На других платформах имя всего устройства оканчивается на c, а не на d (например, `rwd0c`).

22.3. Динамик

Чтобы вывести звук через динамик (например, в конце длинного сценария), можно использовать в конфигурации ядра драйвер *spkr*, который отображается на `/dev/speaker`. Например:

```
echo 'BPBPBPBPBP' > /dev/speaker
```

> **ПРИМЕЧАНИЕ**
>
> Устройство *spkr* не включено в универсальном ядре GENERIC; требуется настроенное ядро.

22.4. Забыли пароль root?

Если вы забыли пароль root, это ещё не всё потеряно: систему можно восстановить, выполнив следующие шаги — загрузиться в однопользовательском режиме, смонтировать / и изменить пароль root. Подробнее:

1. Загрузитесь в однопользовательском режиме: когда появится приглашение загрузчика и начнётся пятисекундный отсчёт, введите следующую команду:

   ```
   > boot -s
   ```

2. В следующем приглашении

   ```
   Enter pathname of shell or RETURN for sh:
   ```

   нажмите Enter.

3. Введите следующие команды:

   ```
   # fsck -y /
   # mount -u /
   # fsck -y /usr
   # mount /usr
   ```

4. Измените пароль root:

   ```
   # passwd root
   Changing local password for root.
   New password: (not echoed)
   Retype new password: (not echoed)
   #
   ```

5. Выйдите из оболочки, чтобы перейти в многопользовательский режим.

   ```
   # exit
   ```

Если вы получили ошибку «Password file is busy», см. раздел ниже.

22.5. Файл паролей занят?

Если при попытке изменить пароль вы получаете загадочное сообщение «Password file is busy», вероятно, файл `/etc/ptmp` не был удалён из системы. Этот файл — временная копия файла `/etc/master.passwd`; проверьте, что вы не теряете важную информацию, и удалите его:

```
# rm /etc/ptmp
```

> **ПРИМЕЧАНИЕ**
>
> Если файл `/etc/ptmp` существует, вы также можете получить предупреждение при запуске системы. Например:
>
> ```
> root: password file may be incorrect - /etc/ptmp exists
> ```

22.6. Добавление нового жёсткого диска

В этом разделе описано, как добавить новый жёсткий диск в уже работающую систему NetBSD.

22.6.1. Новый диск SATA и GPT

В следующем примере добавляется новый жёсткий диск SATA. Можно использовать и диски других типов, например NVMe/M.2, заменив `wd#` на `ld#`.

Когда диск SATA физически подключён к порту SATA, пора перезагрузить компьютер и проверить, что устройство правильно обнаружено, с помощью команды **dmesg**. Это пример сообщения ядра для диска Seagate Barracuda:

```
wd1 at atabus0 drive 1: <ST380013AS>
wd1: drive supports 16-sector PIO transfers, LBA48 addressing
wd1: 76293 MB, 155010 cyl, 16 head, 63 sec, 512 bytes/sect x 156250080 sectors
wd1: 32-bit data port
wd1: drive supports PIO mode 4, DMA mode 2, Ultra-DMA mode 6 (Ultra/133)
```

Теперь можно создать разделы с помощью команды **gpt**. Сначала проверьте текущее состояние диска:

```
# gpt show wd1
start         size  index  contents
         0  156250080
```

В этом примере жёсткий диск пуст — на нём нет ни таблицы разделов, ни дисковой метки. Чтобы создать, например, системный раздел EFI, а за ним раздел NetBSD, выполните команды:

```
# gpt create wd1
# gpt add -t efi -l EFI -s 1m wd1
Partition 1 added, use:
  dkctl wd1 addwedge <wedgename> xxx yyyyyyy <type>
to create a wedge for it
# gpt add -t ffs -l netbsd-data wd1
Partition 2 added, use:
  dkctl wd1 addwedge <wedgename> xxx yyyyyyy <type>
```

```
to create a wedge for it
```

Для простоты дайте NetBSD указание автоматически пересоздать wedge диска для только что определённых разделов. В зависимости от используемой конфигурации ядра это может не потребоваться:

```
# dkctl wd1 makewedges
```

Предположим, что получились wedge диска `dk2` и `dk3`. Имена узлов устройств можно увидеть с помощью **dkctl wd1 listwedges**. Большинство команд также принимают `NAME=xxx`, где `xxx` — метка раздела GPT.

Последняя операция — создание файловых систем для только что определённых разделов:

```
# newfs_msdos -F 32 -L EFISYS dk2

# newfs -O2ea dk3
```

Теперь диск готов к использованию, и разделы можно смонтировать. Например:

```
# mount /dev/dk3 /mnt

# mount /dev/dk2 /boot/efi
```

22.6.2. Новый диск SCSI и disklabel

В следующем примере добавляется новый контроллер SCSI и новый жёсткий диск, подключённый к нему. Если добавлять новый контроллер не нужно, пропустите соответствующую часть и переходите к настройке жёсткого диска. Установка жёсткого диска IDE/SATA с использованием disklabel выполняется так же; отличается только имя устройства (`wd#` вместо `sd#`).

Как всегда, прежде чем покупать новое оборудование, ознакомьтесь со списком совместимого оборудования NetBSD и убедитесь, что новое устройство поддерживается системой.

Когда контроллер SCSI физически установлен в систему и новый жёсткий диск подключён, пора перезагрузить компьютер и проверить, что устройство правильно обнаружено, с помощью команды **dmesg**. Вот пример вывода для контроллера NCR-875:

```
ncr0 at pci0 dev 15 function 0: ncr 53c875 fast20 wide scsi
ncr0: interrupting at irq 10
ncr0: minsync=12, maxsync=137, maxoffs=16, 128 dwords burst, large dma fifo
ncr0: single-ended, open drain IRQ driver, using on-chip SRAM
ncr0: restart (scsi reset).
scsibus0 at ncr0: 16 targets, 8 luns per target
sd0(ncr0:2:0): 20.0 MB/s (50 ns, offset 15)
sd0: 2063MB, 8188 cyl, 3 head, 172 sec, 512 bytes/sect x 4226725 sectors
```

Если устройство не появляется в выводе, проверьте, поддерживается ли оно используемым ядром; при необходимости соберите настроенное ядро (см. Глава 34, *Сборка ядра*).

Теперь можно создать разделы с помощью команды **fdisk**. Сначала проверьте текущее состояние диска:

```
# fdisk sd0
NetBSD disklabel disk geometry:
cylinders: 8188 heads: 3 sectors/track: 172 (516 sectors/cylinder)

BIOS disk geometry:
cylinders: 524 heads: 128 sectors/track: 63 (8064 sectors/cylinder)

Partition table:
0: sysid 6 (Primary 'big' DOS, 16-bit FAT (> 32MB))
    start 63, size 4225473 (2063 MB), flag 0x0
        beg: cylinder    0, head   1, sector  1
        end: cylinder  523, head 127, sector 63
1: <UNUSED>
2: <UNUSED>
3: <UNUSED>
```

В этом примере жёсткий диск уже содержит раздел DOS, который будет удалён и заменён собственным разделом NetBSD. Команда **fdisk -u sd0** позволяет изменять разделы в интерактивном режиме. Изменённые данные будут записаны на диск только перед выходом, и fdisk запросит подтверждение перед записью, так что вы можете работать без спешки.

Геометрия диска

Геометрия диска, которую показывает fdisk, может показаться запутанной. dmesg сообщает 4226725 секторов при C/H/S 8188/3/172, но 8188*3*172 даёт 4225008, а не 4226725. Дело в том, что у большинства современных дисков нет фиксированной геометрии, и число секторов на дорожку меняется в зависимости от цилиндра: единственный интересный параметр — число секторов. Диск сообщает значения C/H/S, но это фиктивная геометрия: значение 172 получено делением общего числа секторов (4226725) на 8188, а затем на 3.

Ещё больше путаницы вносит BIOS, использующий ещё одну «фиктивную» геометрию (C/H/S 524/128/63), которая даёт в сумме 4225536 — значение, более близкое к реальному, чем 425008. Для разбиения диска мы будем использовать геометрию BIOS, чтобы сохранить совместимость с другими операционными системами, хотя при этом потеряем часть секторов (4226725 - 4225536 = 1189 секторов = 594 КБ).

Чтобы создать разделы BIOS, нужно использовать команду **fdisk -u**; результат будет следующим:

```
Partition table:
0: sysid 169 (NetBSD)
    start 63, size 4225473 (2063 MB), flag 0x0
        beg: cylinder    0, head   1, sector  1
        end: cylinder  523, head 127, sector 63
1: <UNUSED>
2: <UNUSED>
3: <UNUSED>
```

Теперь пора создать disklabel для раздела NetBSD. Правильные шаги для этого таковы:

```
# disklabel sd0 > tempfile
# vi tempfile
# disklabel -R -r sd0 tempfile
```

Если вы попытаетесь создать disklabel сразу с помощью

```
# disklabel -e sd0
```

вы получите следующее сообщение

```
disklabel: ioctl DIOCWDINFO: No disk label on disk;
use "disklabel -I" to install initial label
```

потому что disklabel на диске ещё не существует.

Теперь создадим несколько разделов disklabel, отредактировав файл `tempfile`, как уже объяснялось. Результат:

```
#      size   offset   fstype [fsize bsize   cpg]
a:  2048004       63   4.2BSD   1024  8192    16 # (Cyl.  0*- 3969*)
c:  4226662       63   unused      0     0       # (Cyl.  0*- 8191*)
d:  4226725        0   unused      0     0       # (Cyl.  0 - 8191*)
e:  2178658  2048067   4.2BSD   1024  8192    16 # (Cyl.  3969*- 8191*)
```

> **ПРИМЕЧАНИЕ**
>
> После создания disklabel его можно оптимизировать, изучив вывод команды **newfs -N /dev/rsd0a**, которая предупреждает о наличии нераспределённых секторов в конце раздела disklabel. Значения, сообщаемые newfs, можно использовать для корректировки размеров разделов в итеративном процессе.

Последняя операция — создание файловых систем для только что определённых разделов (*a* и *e*).

```
# newfs /dev/rsd0a
# newfs /dev/rsd0e
```

Теперь диск готов к использованию, и оба раздела можно смонтировать. Например:

```
# mount /dev/sd0a /mnt
```

Если это удалось, можно добавить запись о разделе в `/etc/fstab`.

22.7. Как пересоздать устройства в /dev

Сначала перейдите в однопользовательский режим, оставив разделы смонтированными «rw» (для чтения и записи). Это можно сделать, просто набрав **shutdown now**, пока вы находитесь в многопользовательском режиме, либо перезагрузиться с ключом `-s` и сделать `/` и `/dev` доступными для записи, выполнив.

```
# mount -u /
# mount -u /dev
```

Затем:

```
# mkdir /newdev
# cd /newdev
# cp /dev/MAKEDEV* .
# sh ./MAKEDEV all
# cd /
# mv dev olddev
# mv newdev dev
# rm -r olddev
```

Или, если вы загрузили все исходные тексты в `/usr/src`:

```
# mkdir /newdev
# cd /newdev
# cp /usr/src/etc/MAKEDEV.local .
# ( cd /usr/src/etc ; make MAKEDEV )
# cp /usr/src/etc/obj*/MAKEDEV .
# sh ./MAKEDEV all
# cd /
# mv dev olddev; mv newdev dev
# rm -r olddev
```

Определить $arch можно с помощью

```
# uname -m
```

или

```
# sysctl hw.machine_arch
```

Часть IV. Сеть и связанные вопросы

Глава 23. Введение в работу сетей TCP/IP

23.1. Аудитория

В этом разделе объясняются различные аспекты работы сетей. Он призван помочь людям, мало знакомым с сетями, сделать первые шаги. Раздел разделён на три большие части. Мы начнём с общего обзора того, как работают сети, и познакомим с основными понятиями. Затем во второй части подробно рассмотрим настройку сетей различных типов, а третья часть раздела о сетях охватывает ряд «продвинутых» тем, выходящих за рамки базовых операций, представленных в первых двух разделах.

Предполагается, что читатель знаком с базовыми задачами администрирования системы: как стать root, редактировать файлы, изменять права доступа, останавливать процессы и т. п. Дополнительные сведения по этой теме см. в других главах этого руководства по NetBSD и, например, в [AeleenFrisch]. Кроме того, вы должны уметь обращаться с утилитами, которые мы здесь настраиваем, то есть знать, как пользоваться telnet, FTP, … Я не буду объяснять базовые возможности этих утилит — обращайтесь к соответствующим страницам руководства, перечисленным источникам или, конечно, к другим частям этого документа.

Это введение в работу сетей TCP/IP написано с намерением дать начинающим базовые знания. Если вы действительно хотите понять, в чём суть дела, прочитайте [CraigHunt]. Эта книга не ограничивается основами, а идёт дальше и подробно объясняет все концепции, службы и способы их настройки. Она великолепна, я её обожаю! :-)

23.2. Поддерживаемые сетевые протоколы

NetBSD поддерживает несколько наборов протоколов, большинство из которых унаследованы от предшественника NetBSD — 4.4BSD — и впоследствии расширены и улучшены. Первый и самый важный сегодня — Transmission Control Protocol/Internet Protocol (TCP/IP) организации DARPA. Другие наборы протоколов, доступные в NetBSD, включают Stream Control Transmission Protocol (SCTP) и набор протоколов AppleTalk от Apple. Они используются лишь в некоторых специальных приложениях.

Сегодня TCP/IP — самый распространённый из перечисленных выше протоколов. Он реализован почти на любом оборудовании и в любой операционной системе, а также является наиболее используемым протоколом в неоднородных средах. Поэтому если вы просто хотите подключить свой компьютер с NetBSD к другой машине дома или интегрировать его в сеть компании или университета, TCP/IP — правильный выбор.

23.3. Поддерживаемые среды передачи

Стек протоколов TCP/IP ведёт себя одинаково независимо от используемой среды передачи, и NetBSD поддерживает широкий их спектр, в том числе Ethernet (10/100 Мбит/с, 1/10/40/100 Гбит/с), USB, последовательную линию и FireWire (IEEE 1394).

23.3.1. Ethernet

Ethernet — среда, обычно используемая для построения локальных сетей (LAN) из соединённых машин в пределах ограниченной территории, например офиса, компании или университетского городка. Ethernet основан на шинной структуре, к которой может подключаться множество машин, а обмен всегда происходит между двумя узлами одновременно. Когда два или более узла хотят передавать одновременно, оба возобновляют передачу через некоторый тайм-аут. Техническое название этого — CSMA/CD (Carrier Sense w/ Multiple Access and Collision Detection).

Изначально оборудование Ethernet состояло из толстого (жёлтого) кабеля, к которому машины подключались специальными разъёмами, прокалывающими внешнюю оболочку кабеля. Его преемник назывался 10base5 и использовал разъёмы типа BNC, врезавшиеся в специальные T-разъёмы, и терминаторы на обоих концах шины. Сегодня Ethernet в основном используется с витыми парами, которые применяются в системе сжатой шины, реализованной в коммутаторах или концентраторах. Именно витые пары дали название этому типу среды — 10baseT для сетей 10 Мбит/с и 100baseT для сетей 100 Мбит/с. В коммутируемых средах существует также различие в том, может ли обмен между узлом и коммутатором происходить в полудуплексном или полнодуплексном режиме.

23.3.2. IEEE 802.11 (Wi-Fi)

IEEE 802.11 (обычно называемый Wi-Fi) — основное средство подключения мобильных устройств к локальной сети.

IEEE 802.11 в основном работает в двух радиодиапазонах: 2,4 ГГц (режимы b, g и n) и 5 ГГц (режимы a и ac). Диапазон 2,4 ГГц обычно более загружен, но меньше теряет в производительности из-за стен и других препятствий.

Основной протокол защиты соединений Wi-Fi — WPA (Wi-Fi Protected Access). В типовой конфигурации он шифрует соединение между точкой доступа и её клиентами с помощью пароля.

23.3.3. Последовательная линия

Недостаток последовательного соединения в том, что оно медленнее других способов. NetBSD может использовать не более 115200 бит/с, что намного медленнее, например, минимальных 10 Мбит/с у Ethernet.

Существуют два возможных протокола для подключения хоста под управлением NetBSD к другому хосту по последовательной линии (возможно, через телефонную линию):

- IP по последовательной линии (SLIP)
- Протокол «точка — точка» (PPP)

Выбор зависит от того, используете ли вы соединение по коммутируемой линии через модем или статическое соединение (нуль-модем или выделенная линия). Если вы дозваниваетесь для IP-соединения, разумно использовать PPP, так как он даёт возможность автоматического согласования IP-адресов и маршрутов, что вручную бывает довольно мучительно. Если вы хотите подключиться к напрямую соединённой машине, используйте SLIP: он поддерживается практически любой операционной системой и проще настраивается с фиксированными адресами и маршрутами.

Настройка PPP при прямом соединении несколько сложнее, так как легко превысить время ожидания начального рукопожатия; с SLIP такого начального рукопожатия нет: вы запускаете одну сторону, и когда у другой стороны появляется первый пакет, она отправляет его по линии.

[RFC1331] и [RFC1332] описывают PPP и TCP/IP поверх PPP. SLIP определён в [RFC1055].

23.4. Формат адресов TCP/IP

В текущих реализациях TCP/IP использует 4-байтовые (32-битные) адреса (IPv4), называемые также IP-номерами (номерами Internet Protocol), для адресации хостов.

TCP/IP позволяет любым двум машинам общаться напрямую. Для этого все хосты в данной сети должны иметь уникальный IP-адрес. Чтобы это обеспечить, IP-адреса распределяет одна центральная организация — InterNIC. Она выдаёт определённые диапазоны адресов (сетевые адреса) непосредственно сайтам, желающим участвовать в интернете, или интернет-провайдерам, которые передают адреса своим клиентам.

Если ваш университет или компания подключены к Интернету, у них есть (как минимум) один такой сетевой адрес для собственного использования, обычно назначенный не напрямую InterNIC, а через поставщика услуг Интернета (ISP).

Если вы хотите просто запустить свою частную сеть дома, см. ниже, как «построить» собственные IP-адреса. Однако если вы хотите подключить свою машину к (настоящему :-) Интернету, вам следует получить IP-адреса у своего сетевого администратора или провайдера.

IP-адреса обычно записывают в нотации «точечная четвёрка» — четыре байта записываются в десятичном виде (старший байт первым) через точки. Например, 132.199.15.99 был бы допустимым адресом. Другой способ записи IP-адресов — одно 32-битное шестнадцатеричное слово, например 0x84c70f63. Это не так удобно, как точечная четвёрка, но иногда тоже весьма полезно. (См. ниже!)

Получить сеть означает не что иное, как установить некоторые из упомянутых 32 бит адреса в определённые значения. Эти биты, используемые для идентификации сети, называются сетевыми битами. Остальные биты можно использовать для адресации хостов в этой сети, поэтому они называются битами хоста. Рисунок 23.1, «Адреса IPv4 делятся на более значимые биты сети и менее значимые биты хоста» иллюстрирует это разделение.

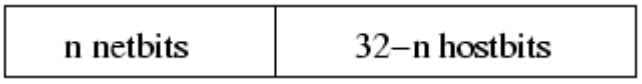


Рисунок 23.1. Адреса IPv4 делятся на более значимые биты сети и менее значимые биты хоста

В приведённом выше примере сетевой адрес — 132.199.0.0 (в сетевых адресах биты хоста устанавливаются в 0), а адрес хоста в этой сети — 15.99.

Откуда известно, что адрес хоста занимает 16 бит? Это назначает провайдер, у которого вы получаете свои сетевые адреса. В используемой сегодня бесклассовой междоменной маршрутизации (CIDR) поля хоста обычно занимают от 2 до 16 бит, а число сетевых бит записывается после сетевого адреса через «/», например 132.199.0.0/16 означает, что в данной сети 16 сетевых бит. Когда говорят о «размере» сети, обычно называют только «/16», «/24» и т. д.

До появления CIDR существовало четыре класса сетей. Каждый из них начинается с определённого битового шаблона, его идентифицирующего. Вот эти четыре класса:

- Класс A начинается с «0» в качестве старшего бита. Следующие семь бит адреса класса A идентифицируют сеть, а остальные 24 бита можно использовать для адресации хостов. Таким образом, в одной сети класса A может быть 2^{24} хостов. Маловероятно, что вы (или ваш университет, компания или кто угодно ещё) получите целый адрес класса A.

  Запись CIDR для сети класса A с её восемью сетевыми битами — «/8».

- Класс B начинается с «10» в качестве старших бит. Следующие 14 бит используются для адреса сети, а остальные 16 бит позволяют адресовать более 65000 хостов. Адреса класса B сегодня

выдаются очень редко; раньше они были обычны для компаний и университетов, пока адресное пространство IPv4 не стало дефицитным.

Запись CIDR для сети класса B с её 16 сетевыми битами — «/16».

Возвращаясь к нашему примеру выше, видно, что 132.199.15.99 (или 0x84c70f63, что здесь уместнее!) находится в сети класса B, так как 0x84… = **10**00… (по основанию 2).

Следовательно, адрес 132.199.15.99 можно разделить на сетевой адрес 132.199.0.0 и адрес хоста 15.99.

- Класс C определяется тем, что старшие биты (MSB) равны «110», что допускает только 256 (фактически только 254, см. ниже) хостов в каждой из 2^{21} возможных сетей класса C. Адреса класса C обычно встречаются у (небольших) компаний.

  Запись CIDR для сети класса C с её 24 сетевыми битами — «/24».
- Есть и другие адреса, начинающиеся с «111». Они используются для специальных целей (например, для адресов multicast) и здесь не представляют интереса.

Обратите внимание, что биты, используемые для идентификации класса сети, входят в сетевой адрес.

Для отделения адресов хостов от сетевых адресов удобно использовать «маску сети». В этой маске все сетевые биты равны «1», а биты хоста — «0». Таким образом, при логическом И IP-адреса и маски сети остаётся сетевой адрес.

Продолжая наш пример: 255.255.0.0 — возможная маска сети для 132.199.15.99. При применении этой маски остаётся сетевой адрес 132.199.0.0.

Для адресов в нотации CIDR указанное число сетевых бит говорит также о том, сколько старших бит адреса нужно установить в «1», чтобы получить маску сети для соответствующей сети. При классовой адресации каждый класс сети имеет фиксированную маску сети по умолчанию:

- Класс A (/8): маска сети по умолчанию: 255.0.0.0, первый байт адреса: 1-127
- Класс B (/16): маска сети по умолчанию: 255.255.0.0, первый байт адреса: 128-191
- Класс C (/24): маска сети по умолчанию: 255.255.255.0, первый байт адреса: 192-223

Ещё стоит упомянуть «широковещательный адрес». При отправке на этот адрес сообщение получат *все* хосты соответствующей сети. Широковещательный адрес характеризуется тем, что все биты хоста в нём установлены в «1».

Если снова взять 132.199.15.99 с маской сети 255.255.0.0, широковещательный адрес получится 132.199.255.255.

Теперь вы спросите: а что, если я хочу, чтобы адрес хоста состоял из всех бит «0» или «1»? Что ж, это не сработает, так как сетевой и широковещательный адреса должны существовать! Из-за этого сеть класса B (/16) может содержать не более 2^{16}-2 хостов, а сеть класса C (/24) — не более 2^{8}-2 = 254 хостов.

Помимо всех этих категорий адресов существует специальный IP-адрес 127.0.0.1, который всегда указывает на «локальный» хост, то есть, обращаясь к 127.0.0.1, вы обращаетесь к самому себе, не запуская никакой сетевой активности. Иногда это полезно, чтобы использовать службы, установленные на собственной машине, или поэкспериментировать, если у вас нет других хостов для вашей сети.

Соберём вместе то, что мы представили в этом разделе:

IP-адрес

32-битный адрес, содержащий сетевые биты и биты хоста.

Сетевой адрес

IP-адрес, в котором все биты хоста установлены в «0».

Маска сети

32-битная маска с «1» для сетевых бит и «0» для бит хоста.

Широковещательный адрес

IP-адрес, в котором все биты хоста установлены в «1».

Адрес localhost

IP-адрес локального хоста всегда 127.0.0.1.

23.5. Разбиение на подсети и маршрутизация

После стольких разговоров о масках сети, сетевых адресах, адресах хостов и прочих адресах я должен признать, что это не вся правда.

Представьте ситуацию в вашем университете, у которого обычно есть адрес класса B (/16), позволяющий иметь в этой сети до 2^{16} ~= 65534 хостов. Может быть, было бы неплохо иметь все эти хосты в одной сети, но это просто невозможно из-за ограничений обычно используемых сегодня сред передачи.

Например, при использовании тонкого Ethernet максимальная длина кабеля — 185 метров. Даже с повторителями между сегментами, которые восстанавливают сигнал, этого недостаточно, чтобы охватить все места, где находятся машины. Кроме того, на одном проводе Ethernet может быть не более 1024 хостов, и при приближении к этому пределу вы заметно потеряете в производительности.

И что, теперь всё пропало? У вас есть адрес, допускающий более 60000 хостов, но вы привязаны к среде, которая допускает куда меньше?

Ну, конечно, нет! :-)

Идея в том, чтобы разделить «большую» сеть класса B на несколько меньших сетей, обычно называемых подсетями. В таких подсетях разрешается иметь, скажем, 254 хоста (то есть вы делите одну большую сеть класса B на несколько сетей класса C!).

Для этого вы настраиваете маску сети так, чтобы в ней было больше сетевых бит и меньше бит хоста. Обычно это делается по границе байта, но к этому вас никто не принуждает. Так что обычно ваша маска сети будет не 255.255.0.0, как предполагается для сети класса B, а 255.255.255.0.

В нотации CIDR вы теперь пишете «/24» вместо «/16», показывая, что для идентификации сети и подсети используется 24 бита адреса вместо 16, которые использовались раньше.

Это даёт вам один дополнительный сетевой байт для назначения каждой (физической!) сети. Все 254 хоста этой подсети теперь могут напрямую общаться друг с другом, а вы можете построить 256 таких сетей класса C. Этого должно хватить для ваших нужд.

Чтобы объяснить это лучше, продолжим наш пример. Пусть у нашего хоста 132.199.15.99 (далее я буду называть его `dusk`; о назначении имён хостов мы поговорим позже) маска сети 255.255.255.0, то есть он находится в подсети 132.199.15.0/24. Введём ещё несколько хостов, чтобы было с чем поэкспериментировать, см. Рисунок 23.2, «Наша демонстрационная сеть».

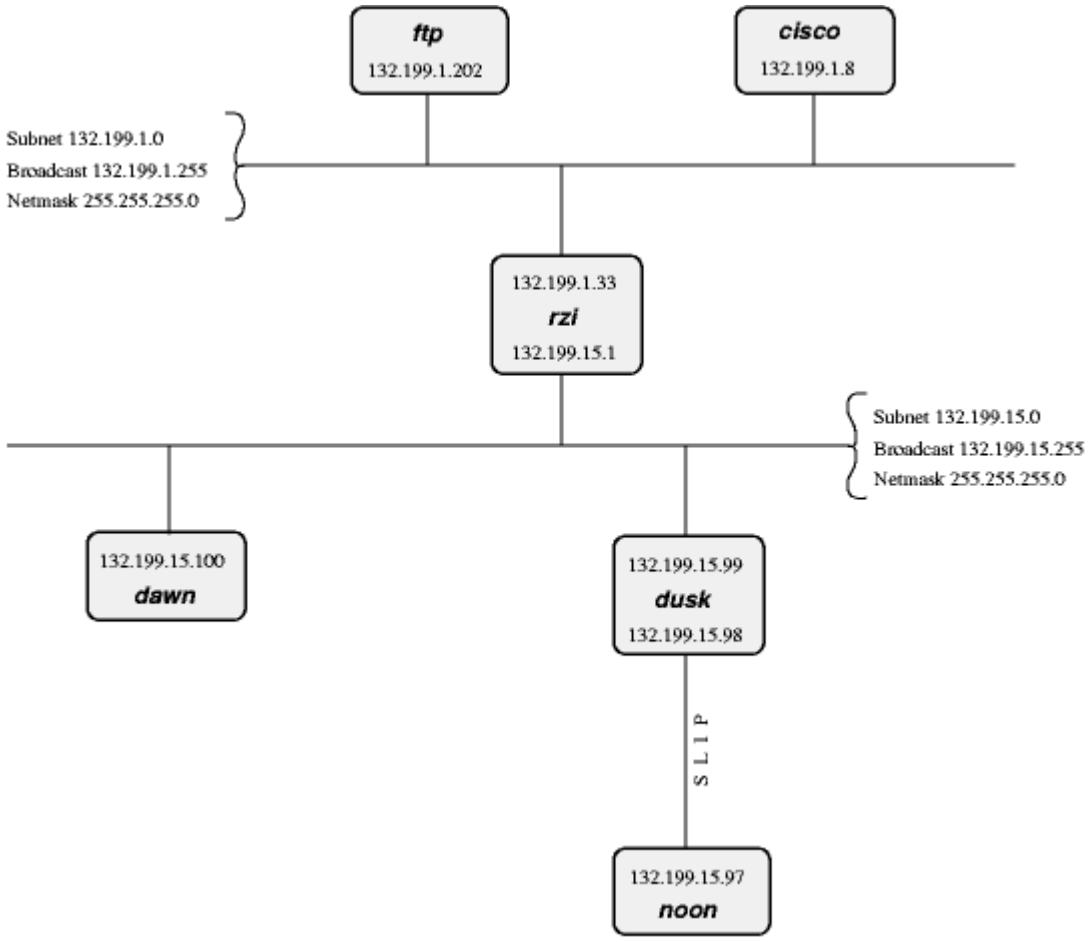


Рисунок 23.2. Наша демонстрационная сеть

В приведённой выше сети `dusk` может напрямую общаться с `dawn`, так как оба находятся в одной подсети. (К подсети 132.199.15.0/24 подключены и другие хосты, но сейчас они для нас не важны)

Но что, если `dusk` хочет связаться с хостом в другой подсети?

Тогда трафик пойдёт через один или несколько шлюзов (маршрутизаторов), которые подключены к двум подсетям. Поэтому у маршрутизатора всегда есть два разных адреса — по одному для каждой подсети, в которой он находится. Функционально маршрутизатор прозрачен, то есть обращаться к нему, чтобы достичь хостов на «другой» стороне, не нужно. Вместо этого вы адресуете пакеты этому хосту напрямую, и они будут правильно до него маршрутизированы.

Пример. Допустим, `dusk` хочет получить файлы с локального FTP-сервера. Поскольку `dusk` не может напрямую достичь `ftp` (он находится в другой подсети), все его пакеты будут переданы его «маршрутизатору по умолчанию» `rzi` (132.199.15.1), который знает, куда их пересылать.

`Dusk` знает адрес своего маршрутизатора по умолчанию в своей сети (`rzi`, 132.199.15.1) и будет пересылать ему любые пакеты, не относящиеся к той же подсети, то есть все IP-пакеты, в которых третий байт адреса не равен 15.

Затем (маршрутизатор по умолчанию) передаёт пакеты соответствующему хосту, так как он также находится в сети FTP-сервера.

В этом примере *все* пакеты пересылаются в сеть 132.199.1.0/24 просто потому, что это магистраль сети — самая важная её часть, по которой проходит весь трафик между несколькими подсетями. Почти все сети, кроме 132.199.15.0/24, подключены к магистрали аналогичным образом.

Но что, если бы мы подключили к 132.199.15.0/24 другую подсеть вместо 132.199.1.0/24? Возможно, как показано на Рисунок 23.3, «Подключение одной подсети к другой».

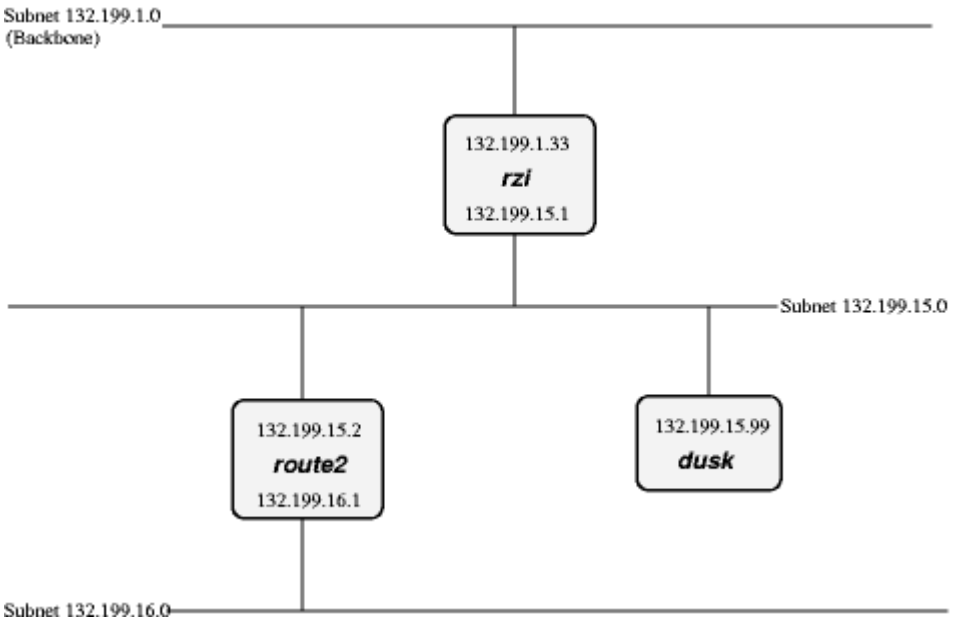


Рисунок 23.3. Подключение одной подсети к другой

Если теперь мы хотим достичь хоста, находящегося в подсети 132.199.16.0/24, из `dusk`, маршрутизировать пакеты через `rzi` не получится — их придётся отправлять напрямую на `route2` (132.199.15.2). `Dusk` должен знать, что такие пакеты нужно пересылать на `route2`, а все остальные — на `rzi`.

Настраивая `dusk`, вы указываете ему пересылать все пакеты для подсети 132.199.16.0/24 на `route2`, а все остальные — на `rzi`. Вместо того чтобы задавать маршрут по умолчанию как 132.199.1.0/24, 132.199.2.0/24 и т. д., можно использовать 0.0.0.0.

Возвращаясь к Рисунок 23.2, «Наша демонстрационная сеть», видим похожую проблему, когда `dawn` хочет отправить данные `noon`, который подключён к `dusk` через последовательную линию. Если смотреть на IP-адреса, `noon` кажется подключённым к сети 132.199.15.0, но на самом деле это не так. Вместо этого `dusk` используется как шлюз, и `dawn` должен отправлять свои пакеты `dusk`, который затем перешлёт их `noon`. Способ, которым `dusk` заставляют принимать пакеты, предназначенные не ему, а другому хосту (`noon`), называется «proxy ARP».

То же самое происходит, когда хосты из других подсетей хотят отправить данные `noon`. Им приходится отправлять пакеты `dusk` (возможно, через `rzi`),

23.6. Понятия службы имён

В предыдущих разделах, говоря о хостах, мы обращались к ним по их IP-адресам. Это было необходимо, чтобы ввести различные виды адресов. Когда речь идёт о хостах вообще, удобнее давать им «имена», как мы делали, говоря о маршрутизации.

Большинству приложений безразлично, что вы им передаёте — IP-адрес или имя хоста. Однако внутри они используют IP-адреса, и существует несколько способов сопоставить имена хостов IP-адресам, каждый со своим способом настройки. В этом разделе мы познакомим с идеей каждого метода, а в следующей главе поговорим о настройке.

Сопоставление имён хостов (и доменных имён) с IP-адресами выполняет программное обеспечение, называемое «распознавателем» (resolver). Это не отдельная служба, а библиотечные процедуры, компонуемые с каждым приложением, использующим сетевые вызовы. Затем

распознаватель пытается преобразовать (отсюда и название;-) заданные вами имена хостов в IP-адреса. Подробности о распознавателе см. в [RFC1034] и [RFC1035].

Имена хостов обычно имеют длину до 256 символов и содержат буквы, цифры и дефисы («-»); регистр не учитывается.

Как и в случае сетей и подсетей, хосты можно (и желательно) группировать в домены и поддомены. Получая сетевой адрес, вы обычно получаете от провайдера и доменное имя. Как и с подсетями, вводить поддомены или нет — решаете вы. В отличие от IP-адресов, (под)домены напрямую не связаны с (под)сетями; например, один домен может содержать хосты из нескольких подсетей.

Рисунок 23.2, «Наша демонстрационная сеть» показывает это: обе подсети 132.199.1.0/24 и 132.199.15.0/24 (а также другие) входят в поддомен «`rz.uni-regensburg.de`». Домен, который Регенсбургский университет получил от своего IP-провайдера, — «`uni-regensburg.de`» («`.de`» — это Deutschland, Германия), а поддомен «`rz`» — это Rechenzentrum, вычислительный центр.

Имена хостов, поддомены и домены разделяются точками («.»). Можно также использовать несколько уровней поддоменов, хотя это не очень распространено. Пример — `fox_in.socs.uts.edu.au`.

Имя хоста, включающее (под)домен, называется также полностью определённым доменным именем (FQDN). Например, IP-адрес 132.199.15.99 принадлежит хосту с FQDN `dusk.rz.uni-regensburg.de`.

Выше я говорил, что IP-адрес 127.0.0.1 всегда принадлежит локальному хосту независимо от того, каков «настоящий» IP-адрес этого хоста. Поэтому 127.0.0.1 всегда сопоставляется с именем «`localhost`».

Существуют три разных способа преобразования имён хостов в IP-адреса: `/etc/hosts`, Domain Name Service (DNS) и Network Information Service (NIS).

23.6.1. `/etc/hosts`

Первый и самый простой способ преобразования имён хостов в IP-адреса — таблица, указывающая, какой IP-адрес принадлежит какому имени (именам). Эта таблица хранится в файле `/etc/hosts` и имеет следующий формат:

```
IP-address        hostname [nickname [...]]
```

Строки, начинающиеся с символа решётки («`#`»), считаются комментариями. Остальные строки содержат один IP-адрес и соответствующее имя (имена) хостов.

Одно имя хоста не может принадлежать нескольким IP-адресам, даже если, говоря о маршрутизации, я создал у вас такое впечатление. Например, у `rzi` на самом деле два разных имени для каждого из двух адресов: `rzi` и `rzia` (но не спрашивайте меня, какое имя какому адресу соответствует!).

Давать хосту несколько псевдонимов удобно, если вы хотите, чтобы ваш любимый хост, предоставляющий какую-то службу, назывался этим именем, как обычно делается с FTP-серверами. Первое (самое левое) имя обычно является настоящим (каноническим) именем хоста.

Помимо псевдонимов удобно задавать полное имя хоста (включая домен) как его каноническое имя, а в качестве псевдонима использовать только имя хоста (без домена).

Важно: Запись, сопоставляющая localhost с 127.0.0.1, *обязательно* должна быть в `/etc/hosts`!

23.6.2. Служба доменных имён (DNS)

`/etc/hosts` имеет принципиальный недостаток, особенно в больших сетях: когда добавляется хост или меняется его адрес, приходится изменять все файлы `/etc/hosts` на всех машинах! Это не только отнимает время, но и весьма вероятно приведёт к ошибкам и несоответствиям, а значит, к проблемам.

Другой подход — хранить для сети только одну таблицу (базу данных) имён хостов и заставлять всех клиентов обращаться к этому «серверу имён». Обновления будут вноситься только на сервере имён.

В этом и состоит основная идея Domain Name Service (DNS).

Обычно для каждого домена существует один сервер имён (отсюда DNS), и каждый хост (клиент) в этом домене знает, в каком домене он находится и к какому серверу имён обращаться по своему домену.

Когда DNS получает запрос о хосте, которого нет в его домене, он переправляет запрос в DNS, который либо является DNS нужного домена, либо знает, какой DNS спросить об этом домене. Если DNS, которому переправлен запрос, не знает, как его обработать, он переправляет запрос дальше в DNS на уровень выше. Это не бесконечно: существуют несколько «корневых» серверов, которые знают обо всех доменах.

Подробности о DNS см. в Глава 26, *Система доменных имён*.

23.6.3. Сетевая информационная служба (NIS/YP)

Yellow Pages (YP) была придумана в Sun Microsystems. Название было изменено на Network Information Service (NIS), так как YP уже была торговой маркой British Telecom. Так что, когда я говорю о NIS, вы знаете, что имеется в виду.;-)

В системе Unix довольно много файлов конфигурации, и часто хочется поддерживать только один комплект этих файлов для нескольких хостов. Такие хосты объединяются в NIS-домен (который *ничего* общего не имеет с доменами, построенными с помощью DNS!) и обычно входят в один кластер рабочих станций.

Примеры файлов конфигурации, общих для этих хостов, — `/etc/passwd`, `/etc/group` и, наконец, `/etc/hosts`.

Так что вы можете «злоупотребить» NIS, чтобы получить единое преобразование имён в адреса на всех хостах одного (NIS-)домена.

Есть только один недостаток, который мешает использовать NIS для такого преобразования на практике: в отличие от DNS, NIS не даёт способа разрешить имена хостов, отсутствующие в таблице хостов. Нет никакого «хоста на уровень выше», к которому NIS-сервер мог бы обратиться, поэтому преобразование завершится неудачей! NIS+ от Sun принимает меры против этой проблемы, но поскольку NIS+ доступна только в системах Solaris, нам сейчас от этого мало пользы.

Не поймите меня неправильно: NIS — хорошая вещь для управления, например, сведениями о пользователях (`/etc/passwd`, …) в кластерах рабочих станций, просто она не слишком полезна для разрешения имён хостов.

23.6.4. Прочее

Описанные выше методы разрешения имён — это то, что обычно используется сегодня для преобразования имён хостов в IP-адреса, но они не единственные. В принципе подошёл бы любой механизм базы данных, но ни один из них в NetBSD не реализован. Кратко рассмотрим, с чем можно столкнуться.

Поскольку в NIS данные не имеют иерархии, NIS+ призвана помочь в этой области. Таблицы можно организовать так, что если сервер домена не может ответить на запрос, найдётся другой домен «выше», который сможет это сделать. Например, можно сделать домен, перечисляющий всех хостов (пользователей, группы, …), действующих во всей компании, домен, определяющий то же для каждого подразделения, и т. д. Сегодня NIS+ используется мало: даже Sun вернулась к тому, чтобы поставлять по умолчанию NIS.

В прошлом веке стандарт X.500 был разработан, чтобы вмещать как простые базы данных вроде `/etc/hosts`, так и сложные иерархические системы, которые можно найти, например, в современном DNS. X.500 не имел особого успеха, главным образом потому, что пытался делать слишком много сразу. Сегодня доступна урезанная версия — Lightweight Directory Access Protocol (LDAP), которая в последние годы становится популярной для управления данными о пользователях, а также о хостах и другом в организациях малого и среднего размера.

23.7. IPv6

23.7.1. Чем хорош IPv6?

Когда людям говорят о переходе с IPv4 на IPv6, обычно слышишь вопрос «зачем?». На самом деле есть несколько веских причин перейти на новую версию:

- Большее адресное пространство
- Поддержка мобильных устройств
- Встроенная безопасность

23.7.1.1. Большее адресное пространство

Большее адресное пространство, которое предлагает IPv6, — самое очевидное её улучшение по сравнению с IPv4. Если сегодняшняя архитектура интернета основана на 32-битных адресах, то в новой версии для адресации доступно 128 бит. Благодаря увеличенному адресному пространству больше не нужно прибегать к обходным решениям вроде NAT. Это даёт полную, неограниченную IP-связность для современных мобильных телефонов и устройств IoT.

23.7.1.2. Мобильность

Говоря о мобильных устройствах и IP, стоит отметить ещё один важный момент: для поддержки мобильности нужен особый протокол, и его реализация — так называемый «Mobile IP» — входит в требования к любому стеку IPv6. Таким образом, если у вас работает IPv6, вы получаете поддержку роуминга между разными сетями, причём все обновляются, когда вы покидаете одну сеть и входите в другую. Поддержка роуминга возможна и в IPv4, но чтобы всё заработало, приходится преодолевать немало препятствий. С IPv6 этого не требуется, так как поддержка мобильности была одним из требований к проекту IPv6. Дополнительные сведения о проблемах, которые нужно решать с Mobile IP в IPv4, см. в [RFC3024].

23.7.1.3. Безопасность

Помимо поддержки мобильности, ещё одним требованием к преемнику нынешней версии Internet Protocol была безопасность. В результате стеки протоколов IPv6 обязаны включать IPsec. IPsec позволяет аутентифицировать, шифровать и сжимать любой IP-трафик. В отличие от протоколов прикладного уровня вроде SSL или SSH, он может обрабатывать весь IP-трафик между двумя узлами, не требуя изменений в приложениях. Преимущество в том, что все приложения на машине получают выгоду от шифрования и аутентификации, а политики можно задавать в расчёте на хост (или даже на

сеть), а не на приложение или службу. Введение в IPsec с путеводителем по документации можно найти в [RFC2411], основной протокол описан в [RFC2401].

23.7.2. Изменения по сравнению с IPv4

После краткого обзора всех важных возможностей IPv6 мы перейдём здесь к основам IPv6 подробнее. Предполагается, что вы немного понимаете, как работает IPv4, а изменения в IPv6 будут выделены. Начнём с адресов IPv6 и того, как они устроены, затем рассмотрим различные типы адресов, что стало с широковещательной передачей, а после обсуждения уровня IP перейдём к изменениям в разрешении имён и к тому, что нового в DNS для IPv6.

23.7.2.1. Адресация

Адрес IPv4 — это 32-битное значение, обычно записываемое в представлении «точечная четвёрка», где каждая «четвёрка» обозначает значение байта от 0 до 255, например:

```
127.0.0.1
```

Теоретически это позволяет подключить к интернету 2^{32}, или около 4 миллиардов, хостов. Из-за группировки не все адреса доступны сегодня.

В IPv6 используются 128 бит, что даёт 2^{128} теоретически адресуемых хостов. Это позволяет адресовать поистине огромное число машин и наверняка покрывает все сегодняшние потребности, а также все те новомодные КПК и сотовые телефоны с IP-телефонией в ближайшем будущем — без малейшего напряжения. Записывая адреса IPv6, их обычно делят на группы по 16 бит, записываемые четырьмя шестнадцатеричными цифрами, а группы разделяют двоеточиями. Пример:

```
fe80::2a0:d2ff:fea5:e9f5
```

Здесь показана особая вещь: несколько подряд идущих нулей можно один раз в адресе IPv6 сократить двойным «::». Так, приведённый выше адрес эквивалентен fe80:0:00:000:2a0:d2ff:fea5:e9f5 — ведущие нули внутри групп можно опускать, а использовать «::» в адресе IPv6 можно только один раз.

Чтобы адресами было удобнее управлять, их делят на две части: биты, идентифицирующие сеть, в которой находится машина, и биты, идентифицирующие машину в (под)сети. Эти биты называют сетевыми битами (netbits) и битами хоста (hostbits), и как в IPv4, так и в IPv6 сетевые биты — это «левые», старшие биты IP-адреса, а биты хоста — «правые», младшие биты, как показано на рисунке 23.4, «IPv6-адреса также делятся на более старшие биты сети и более младшие биты хоста».

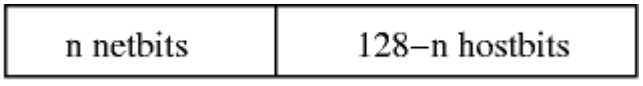


Рисунок 23.4. IPv6-адреса также делятся на более старшие биты сети и более младшие биты хоста

В IPv4 граница проводится с помощью маски сети, которой можно замаскировать все биты сети и хоста. Типичные примеры — 255.255.0.0, где 16 бит используются для адресации сети и 16 бит — для машины, или 255.255.255.0, где ещё 8 бит позволяют адресовать 256 подсетей, например в сети класса B.

Когда адресация перешла от классовой к маршрутизации CIDR, границы между битами сети и хоста перестали совпадать с 8-битными границами, и в результате маски сети стали выглядеть

некрасиво и не особенно удобно. Взамен для заданного адреса используется число битов сети, обозначающее границу, например

```
10.0.0.0/24
```

эквивалентно маске сети 255.255.255.0 (24 единичных бита). Та же схема используется в IPv6:

```
2001:638:a01:2::/64
```

означает, что в используемом здесь адресе первые (самые левые) 64 бита служат адресом сети, а последние (самые правые) 64 бита идентифицируют машину в сети. Биты сети обычно называют «префиксом» (сети), а «prefixlen» здесь был бы равен 64 битам.

Распространённые схемы адресации в IPv4 — это (старые) сети класса B и класса C. В сети класса C (/24) провайдер выделяет вам 24 бита, а 8 бит остаются в вашем распоряжении. Если вы захотите добавить к этому разбиение на подсети, у вас получатся «неравномерные» маски сети, с которыми довольно неудобно работать. Проще в таких случаях сети класса B (/16): провайдер выделяет только 16 бит, а остальные можно разбивать на подсети, то есть делить самые правые биты на две части — одну для адресации подсети на месте, другую для адресации хостов в этой подсети. Обычно это делается по границам байтов (8 бит). Использование маски сети 255.255.255.0 (или префикса /24) позволяет гибко управлять даже более крупными сетями. Разумеется, есть верхний предел — 254 машины на подсеть и 256 подсетей.

При 128 битах, доступных для адресации в IPv6, обычно используется та же схема, только поля шире. Провайдеры обычно выделяют сети /48, что оставляет 16 бит для разбиения на подсети и 64 бита хоста.

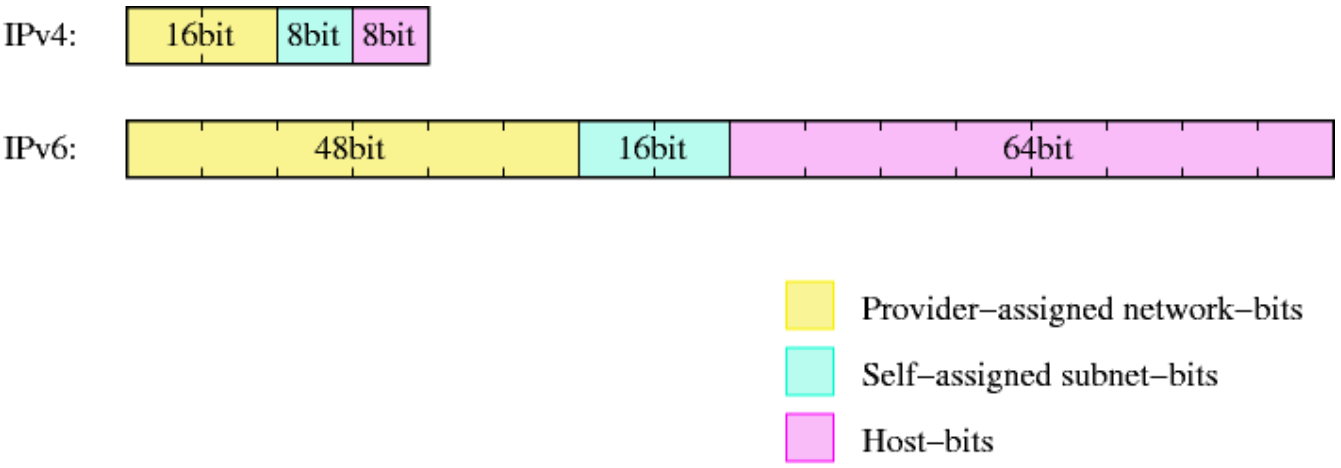


Рисунок 23.5. IPv6-адреса имеют структуру, похожую на адреса класса B

Пространства для сетей и подсетей здесь вполне достаточно, а вот использование 64 бит для адресации хостов кажется расточительством. Вряд ли вы захотите разместить несколько миллиардов хостов в одной подсети, так в чём же смысл?

Идея идентификаторов хоста фиксированной ширины 64 бита состоит в том, что они не назначаются вручную, как это обычно делается для IPv4 сегодня. Вместо этого адреса хостов IPv6 рекомендуется (но не обязательно!) строить из так называемых адресов EUI64. Адреса EUI64 — как следует из названия — имеют ширину 64 бита и получаются из MAC-адресов соответствующего сетевого интерфейса. Например, для ethernet 6-байтовый (48-битный) MAC-адрес обычно

дополняется в середине шестнадцатеричными битами «fffe», а один бит устанавливается, чтобы отметить адрес как уникальный (что верно для Ethernet), например MAC-адрес

```
01:23:45:67:89:ab
```

даёт адрес EUI64

```
03:23:45:ff:fe:67:89:ab
```

что, в свою очередь, даёт такие биты хоста для адреса IPv6:

```
::0323:45ff:fe67:89ab
```

Эти биты хоста теперь можно использовать для автоматического назначения хостам адресов IPv6, что поддерживает автоматическую настройку хостов IPv6: для получения полного адреса IPv6 нужны только первые (сетевые/подсетевые) биты, а IPv6 также предлагает решение для их автоматического назначения.

В сети машин, работающих по IP, обычно есть один маршрутизатор, который служит шлюзом во внешние сети. В мире IPv6 этот маршрутизатор отправляет информацию «объявления маршрутизатора», которую клиенты должны либо принимать во время работы, либо запрашивать при запуске системы. Информация объявлений маршрутизатора включает данные об адресе маршрутизатора и о том, какой префикс адресов он маршрутизирует. С этой информацией и сгенерированным хостом адресом EUI64 хост IPv6 может вычислить свой IP-адрес, и ручное назначение адресов не требуется. Разумеется, маршрутизаторам по-прежнему нужна некоторая настройка.

Создаваемая ими информация объявлений маршрутизатора входит в протокол обнаружения соседей (Neighbor Discovery Protocol, NDP, см. [RFC2461]), который является преемником протокола ARP из IPv4. В отличие от ARP, NDP не только ищет IPv6-адреса по MAC-адресам (часть neighbor solicitation/advertisement), но и выполняет аналогичную службу для маршрутизаторов и обслуживаемых ими префиксов, что используется для автоматической настройки хостов IPv6, как описано в предыдущем абзаце.

23.7.2.2. Множественные адреса

В IPv4 у хоста обычно один IP-адрес на сетевой интерфейс или даже на машину, если IP-стек это поддерживает. Лишь очень редкие приложения, вроде веб-серверов, приводят к тому, что у машины оказывается больше одного IP-адреса. В IPv6 всё иначе. На каждом интерфейсе есть не только глобально уникальный IP-адрес, но и ещё два адреса, представляющих интерес: адрес канала (link local) и адрес сайта (site local). Адрес link local имеет префикс fe80::/64, а биты хоста строятся из адреса EUI64 интерфейса. Адрес link local используется только для связи с хостами и маршрутизаторами в той же сети; из других подсетей эти адреса не видны и недоступны. При желании можно выбрать либо глобальные адреса (выделяемые провайдером), либо адреса site local. Адресам site local назначается сетевой адрес fec0::/10, и подсети и хосты можно адресовать так же, как в сетях, выделенных провайдером. Разница только в том, что эти адреса не будут видны внешним машинам, поскольку те находятся в другой сети, а их адреса «site local» — в другой физической сети (если они вообще назначены). Как и в случае сети 10/8 в IPv4, адреса site local можно использовать, но не обязательно. В IPv6 чаще всего хостам назначают link-local и глобальный IP-адрес. Адреса site local сегодня довольно редки и не заменяют глобально уникальные адреса, если требуется глобальная связность.

23.7.2.3. Многоадресная рассылка

В мире IP есть три способа обратиться к хосту: одноадресная передача (unicast), широковещательная (broadcast) и многоадресная (multicast). Самый распространённый — обратиться к хосту напрямую, используя его unicast-адрес. В IPv4 unicast-адрес — это «обычный» IP-адрес, назначенный одному хосту, со всеми заданными битами адреса. Широковещательный адрес, используемый для обращения ко всем хостам одной IP-подсети, имеет биты сети, равные адресу сети, и все биты хоста, равные «1» (что легко сделать с помощью маски сети и нескольких битовых операций). Многоадресные адреса служат, чтобы охватить ряд хостов в одной группе multicast, которыми могут быть машины по всему интернету. Чтобы участвовать, машины должны явно вступать в группы multicast, а для многоадресных адресов используются специальные IPv4-адреса, выделенные из подсети 224/8. В IPv4 multicast применяется не очень широко, и его используют лишь немногие приложения, вроде утилит аудио- и видеовещания MBone.

В IPv6 unicast-адреса используются так же, как и в IPv4, — ничего удивительного: все биты сети и хоста назначаются, чтобы идентифицировать целевую сеть и машину. Широковещательная передача в IPv6 в том виде, в каком она была в IPv4, больше недоступна — здесь в игру вступает multicast. Адреса в сети ff::/8 зарезервированы для многоадресных приложений, и есть два специальных многоадресных адреса, которые заменяют широковещательные адреса из IPv4. Один — многоадресный адрес «все маршрутизаторы», другой — для «все хосты». Эти адреса относятся к конкретной подсети, то есть маршрутизатор, подключённый к двум разным подсетям, может адресовать все хосты/маршрутизаторы в любой из подсетей, к которым он подключён. Адреса таковы:

- ff0*X*::1 для всех хостов и
- ff0*X*::2 для всех маршрутизаторов,

где «*X*» — идентификатор области (scope ID) канала, определяющий сеть. Обычно он начинается с «1» для области «node local», «2» — для первого канала и т. д. Учтите, что два сетевых интерфейса вполне могут быть подключены к одному каналу, что даёт двойную пропускную способность:

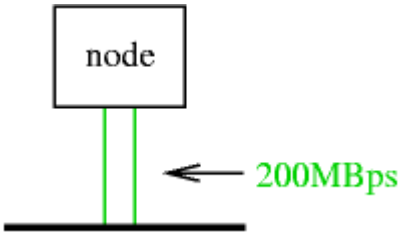


Рисунок 23.6. Несколько интерфейсов, подключённых к одному каналу, дают только один идентификатор области для канала

Одно из применений многоадресной рассылки «все хосты» — в коде neighbor solicitation протокола NDP: любая машина, желающая связаться с другой машиной, отправляет запрос группе «все хосты», и предполагается, что нужная машина ответит.

23.7.2.4. Разрешение имён в IPv6

После долгого разговора об адресации в IPv6 те, кто ещё читает, надеются, что есть подходящий способ скрыть все эти длинные и уродливые IPv6-адреса за красивыми именами хостов, как это делается в IPv4. И конечно, он есть.

Разрешение имён хостов в IP-адреса в IPv4 обычно выполняется одним из трёх способов: с помощью простой таблицы в `/etc/hosts`, с помощью Network Information Service (NIS, ранее YP) или через Domain Name System (DNS).

На момент написания NIS/NIS+ поверх IPv6 имеется только в Solaris 8 — как для содержимого баз данных, так и для транспорта, с использованием расширения RPC.

Простая таблица соответствия вида адрес<->имя, как в файле `/etc/hosts`, поддерживается во всех стеках IPv6. В реализации KAME, используемой в NetBSD, файл `/etc/hosts` содержит как IPv6-адреса, так и IPv4-адреса. Простой пример — запись «localhost» в установке NetBSD по умолчанию:

```
127.0.0.1               localhost
::1                     localhost
```

В DNS принципиально новых понятий нет. Разрешение имён для IPv6 выполняется с помощью записей AAAA, которые — как видно из названия — указывают на объект в четыре раза большего размера, чем запись A. Запись AAAA, как и A, содержит имя хоста в левой части, а в правой — IPv6-адрес, например

```
noon            IN      AAAA    3ffe:400:430:2:240:95ff:fe40:4385
```

Для обратного разрешения IPv4 использует зону in-addr.arpa и записывает под ней байты (в десятичном виде) в обратном порядке, то есть более старшие байты находятся правее. В IPv6 всё похоже, только вместо десятичных чисел используются шестнадцатеричные цифры, представляющие 4 бита, а ресурсные записи находятся в другом домене — ip6.int.

Итак, чтобы обеспечить обратное разрешение для указанного выше хоста, добавьте в свой `/etc/named.conf` примерно следующее:

```
zone "0.3.4.0.0.0.4.0.e.f.f.3.IP6.INT" {
      type master;
      file "db.reverse";
};
```

а в файл зоны db.reverse поместите (помимо обычных записей, таких как SOA и NS):

```
5.8.3.4.0.4.e.f.f.f.5.9.0.4.2.0.2.0.0.0 IN   PTR   noon.ipv6.example.com.
```

Здесь адрес записан в обратном порядке, по одной шестнадцатеричной цифре, начиная с младшей (самой правой), причём цифры разделены точками, как обычно в файлах зон.

При настройке DNS для IPv6 нужно учитывать версию используемого программного обеспечения DNS. BIND 8.x понимает записи AAAA, но не обеспечивает разрешение имён через IPv6. Для этого нужен BIND 9.x. Кроме того, BIND 9.x поддерживает ряд ресурсных записей, которые сейчас обсуждаются, но ещё официально не введены. Самая заметная из них — запись A6, упрощающая смену провайдера и префикса.

Подводя итог: в этом разделе речь шла о технических различиях между IPv4 и IPv6 в части адресации и разрешения имён. Некоторые детали, например параметры заголовка IP, QoS и потоки, намеренно опущены, чтобы не усложнять документ сверх необходимости.

Глава 24. Настройка TCP/IP в NetBSD на практике

24.1. Обзор файлов конфигурации сети

Ниже приведён список файлов, используемых для настройки сети. Их применение — некоторые из них уже встречались в первых главах — описывается в следующих разделах.

`/etc/hosts`
: Файл локальной базы данных хостов. Каждая строка содержит сведения об известном хосте: интернет-адрес, имя хоста и псевдонимы. Небольшие сети можно настроить, используя только файл hosts, без *сервера имён*. См. hosts(5)

`/etc/resolv.conf`
: Этот файл определяет, как должны работать процедуры, обеспечивающие доступ к интернет-системе доменных имён. Обычно он содержит адреса DNS-серверов. См. resolv.conf(5)

`/etc/sysctl.conf`
: Этот файл используется для настройки параметров ядра, например включения пересылки пакетов на шлюзе. См. sysctl.conf(5).

`/etc/ifconfig.xxx`
: Этот файл используется для автоматической настройки сетевых интерфейсов при загрузке, см. ifconfig.if(5)

`/etc/npf.conf`
: Содержит конфигурацию межсетевого экрана для NetBSD Packet Filter, см. npf.conf(5) и `/usr/share/examples/npf`.

`/etc/dhcpcd.conf`
: Содержит конфигурацию клиента DHCP. DHCP используется для автоматического получения IPv4-адресов по Ethernet, но dhcpcd(8) также применяется для делегирования префикса IPv6 и DHCPv6. См. dhcpcd.conf(5) и `/usr/share/examples/dhcpcd`.

`/etc/dhcpd.conf`
: Содержит конфигурацию сервера DHCP. DHCP используется для автоматического назначения IPv4-адресов клиентам. См. dhcpd.conf(5) и `/usr/share/examples/dhcpd`.

`/etc/mygate`
: Содержит IP-адрес шлюза IPv4. Используется для настройки маршрута по умолчанию, когда DHCP не применяется. Можно также задать `defaultroute=""` в `/etc/rc.conf`.

`/etc/mygate6`
: Содержит IP-адрес шлюза IPv6. Используется для настройки маршрута по умолчанию, когда автонастройка не применяется. Можно также задать `defaultroute6=""` в `/etc/rc.conf`.

`/etc/nsswitch.conf`
: Файл конфигурации подсистемы переключения служб имён. Он управляет тем, как процесс ищет в различных базах данных сведения о хостах, пользователях, группах и т. п. В частности, этот файл задаёт порядок поиска по базам данных. Например, строка:

```
hosts:    files dns mdnsd
```

указывает, что база данных hosts берётся из двух источников — *files* (локальный файл `/etc/hosts`) и *DNS* (интернет-система доменных имён), — и что локальные файлы просматриваются раньше DNS.

Обычно изменять этот файл не требуется, если только не нужно включить Multicast DNS. См. nsswitch.conf(5).

`/etc/hostapd.conf`

Используется для настройки беспроводной точки доступа IEEE 802.11 (Wi-Fi). См. hostapd.conf(5) и `/usr/share/examples/hostapd`.

`/etc/wpa_supplicant.conf`

Используется для настройки клиента IEEE 802.11 (Wi-Fi). См. wpa_supplicant.conf(5) и `/usr/share/examples/wpa_supplicant`.

24.2. Подключение к типичным локальным сетям

В локальных сетях с централизованным управлением можно рассчитывать на доступ в интернет через какой-либо маршрутизатор, на наличие DNS-сервера и, что важнее всего, DHCP-сервера, который по запросу выдаёт клиентам IP-адреса. Чтобы клиент NetBSD заработал в такой среде, обычно достаточно задать

```
dhcpcd=YES
```

в `/etc/rc.conf`, и IP-адрес будет задан автоматически, файл `/etc/resolv.conf` будет создан, а маршрутизация настроена на маршрутизатор по умолчанию.

24.2.1. Подключение через IEEE 802.11 (Wi-Fi)

WPA Supplicant позволяет подключаться к сетям Wi-Fi по паролю, а также предоставляет единообразный интерфейс для подключения к точкам доступа.

Помимо работающего dhcpcd(8), в системе, которая подключается к точке доступа по Wi-Fi, как правило, должен быть включён wpa_supplicant(8):

```
ifconfig_iwm0="up"
dhcpcd=YES
wpa_supplicant=YES
```

Во время работы:

```
# ifconfig iwm0 up
# service dhcpcd start
# service wpa_supplicant start
```

В данном случае наш интерфейс Wi-Fi называется `iwm0`. Он переводится в состояние `up` с помощью ifconfig(8). Список обнаруженных интерфейсов Wi-Fi можно получить с помощью wlanctl(8):

```
# wlanctl -a
```

```
iwm0: mac 10:02:xx:xx:xx:xx bss 00:00:00:00:00:00
```

Следующий файл `/etc/wpa_supplicant.conf` настраивает NetBSD на автоматическое подключение к двум точкам доступа. Можно добавить и больше.

Пример 24.1. /etc/wpa_supplicant.conf

```
# Allow wpa_cli(8) to configure wpa_supplicant
ctrl_interface=/var/run/wpa_supplicant
ctrl_interface_group=wheel
update_config=1

# Automatically connect to the unprotected network "metalab".
network={
        ssid="metalab"
        key_mgmt=NONE
        priority=100
}

# Automatically connect to the protected network "discord" using the password "XXX".
network={
        ssid="discord"
        psk="XXX"
}
```

Добавив точки доступа, перезагрузите конфигурацию wpa_supplicant(8):

```
# service wpa_supplicant reload
# service dhcpcd restart
```

Когда WPA supplicant запущен, для поиска сетей можно использовать wpa_cli(8):

```
# wpa_cli scan
# wpa_cli scan_results
Selected interface 'iwm0'
16:01:33.578: bssid / frequency / signal level / flags / ssid
xx:xx:xx:xx:xx:xx       5180    91      [WPA2-PSK-CCMP][ESS]    FRITZ!Box 1000 EZ
```

24.3. Создание небольшой локальной сети вручную

В этом разделе описано, как *вручную* настроить локальную сеть, чтобы объяснить основы работы сетевого стека. Обычно эта настройка выполняется автоматически с помощью dhcpcd(8), см. раздел 24.2, «Подключение к типичным локальным сетям»

Сначала нужно установить сетевые карты и подключить их к коммутатору или напрямую друг к другу.

Затем проверьте, распознаёт ли ядро сетевые карты, изучив вывод команды **dmesg**. В следующем примере ядро правильно распознало клон NE2000:

```
...
ne0 at isa0 port 0x280-0x29f irq 9
ne0: NE2000 Ethernet
```

```
ne0: Ethernet address 00:c2:dd:c1:d1:21
...
```

Следующая команда показывает текущую конфигурацию сетевой карты:

```
# ifconfig ne0
ne0: flags=8822<BROADCAST,NOTRAILERS,SIMPLEX,MULTICAST> mtu 1500
address: 00:50:ba:aa:a7:7f
media: Ethernet autoselect (10baseT)
inet6 fe80::250:baff:feaa:a77f%ne0 prefixlen 64 scopeid 0x1
```

Настроить сетевую карту программно очень просто. Карте назначается IP-адрес «192.168.1.1».

```
# ifconfig ne0 inet 192.168.1.1 netmask 0xffffff00
```

Учтите, что сети 10.0.0.0/8 и 192.168.0.0/16 зарезервированы для частных сетей — именно такую сеть мы здесь и настраиваем.

Если повторить предыдущую команду, результат будет другим:

```
# ifconfig ne0
ne0: flags=8863<UP,BROADCAST,NOTRAILERS,RUNNING,SIMPLEX,MULTICAST> mtu 1500
address: 00:50:ba:aa:a7:7f
media: Ethernet autoselect (10baseT)
inet 192.168.1.1 netmask 0xffffff00 broadcast 192.168.1.255
inet6 fe80::250:baff:feaa:a77f%ne0 prefixlen 64 scopeid 0x1
```

Вывод **ifconfig** изменился: теперь печатается IP-адрес и появились два новых флага — «UP» и «RUNNING». Если интерфейс не «UP», система не будет использовать его для отправки пакетов.

Хосту назначен IP-адрес 192.168.1.1, который принадлежит набору адресов, зарезервированных для внутренних сетей, недоступных из интернета. Настройка завершена, и теперь её нужно проверить; если в сети есть другой активный хост, можно попробовать *ping*. Например, если 192.168.1.2 — адрес активного хоста:

```
# ping 192.168.1.2
PING ape (192.168.1.2): 56 data bytes
64 bytes from 192.168.1.2: icmp_seq=0 ttl=255 time=1.286 ms
64 bytes from 192.168.1.2: icmp_seq=1 ttl=255 time=0.649 ms
64 bytes from 192.168.1.2: icmp_seq=2 ttl=255 time=0.681 ms
64 bytes from 192.168.1.2: icmp_seq=3 ttl=255 time=0.656 ms
^C
----ape PING Statistics----
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.649/0.818/1.286/0.312 ms
```

При текущей настройке после следующей загрузки конфигурацию сетевой карты придётся повторить. Чтобы не повторять настройку карты при каждой загрузке, добавьте в `/etc/rc.conf` следующие строки:

```
auto_ifconfig=yes
ifconfig_ne0="inet 192.168.1.1 netmask 0xffffff00"
```

В этом примере переменная `ifconfig_ne0` задана потому, что ядро распознало сетевую карту как *ne0*; если вы используете другой адаптер, подставьте его имя вместо ne0.

При следующей загрузке сетевая карта будет настроена автоматически.

Если у вас есть маршрутизатор, подключённый к интернету, его можно использовать как маршрутизатор по умолчанию: он будет обрабатывать все ваши пакеты. Для этого задайте параметру `defaultroute` в `/etc/rc.conf` IP-адрес маршрутизатора:

```
defaultroute=192.168.0.254
```

Обязательно указывайте IP-адрес маршрутизатора по умолчанию, а не его имя, если ваш DNS-сервер находится за этим маршрутизатором. В таком случае до DNS-сервера нельзя было бы добраться, чтобы разрешить имя маршрутизатора по умолчанию, и наоборот, — получилась бы проблема курицы и яйца.

Чтобы обращаться к хостам локальной сети, и при условии, что хостов действительно очень мало, добавьте в `/etc/hosts` адреса всех хостов, принадлежащих внутренней сети. Например:

Пример 24.2. /etc/hosts

```
#
# Host Database
# This file should contain the addresses and aliases
# for local hosts that share this file.
# It is used only for "ifconfig" and other operations
# before the nameserver is started.
#
#
127.0.0.1             localhost
::1                   localhost
#
# RFC 1918 specifies that these networks are "internal".
# 10.0.0.0    10.255.255.255
# 172.16.0.0  172.31.255.255
# 192.168.0.0 192.168.255.255

192.168.1.1   ape.insetti.net ape
192.168.1.2   vespa.insetti.net vespa
192.168.1.0   insetti.net
```

Чтобы настроить машину как клиент DNS, отредактируйте `/etc/resolv.conf` и укажите адрес DNS-сервера, а также необязательное доменное имя, которое будет добавляться к хостам без домена для получения полного доменного имени при разрешении. Если IP-адрес вашего DNS-сервера — 192.168.1.2 и он настроен обслуживать «home.net», поместите в `/etc/resolv.conf` следующее:

```
# /etc/resolv.conf
domain home.net
nameserver 192.168.1.2
```

Итак, для настройки сети нужно сделать следующее: установить сетевые адаптеры и физически их подключить. Затем настроить их (командой **ifconfig**) и, наконец, изменить файл `/etc/rc.conf`, чтобы настроить интерфейс и, возможно, маршрутизатор по умолчанию, а также скорректировать `/etc/resolv.conf` и `/etc/nsswitch.conf`, если предполагается использовать DNS. Такого способа управления сетью достаточно для небольших сетей без изощрённых потребностей.

24.4. Подключение к домашнему или офисному провайдеру по PPPoE

Многие домашние и офисные провайдеры используют PPP (point to point protocol), чтобы предоставить клиентам доступ в интернет. NetBSD включает функциональность pppoe(4) (PPP over Ethernet), с помощью которой можно подключаться к модему, общающемуся с провайдером, как правило, не по Ethernet, что позволяет использовать NetBSD в качестве шлюза в небольших домашних и офисных сетях.

Начнём с настройки ядра для работы с PPPoE — увеличим размер очереди tty. Этот параметр можно сделать постоянным, отредактировав `/etc/sysctl.conf`:

```
# sysctl -w kern.tty.qsize=32768
```

Теперь создайте интерфейс с помощью ifconfig(8):

```
# ifconfig pppoe0 create
# ifconfig inet 0.0.0.0 0.0.0.1 down
# ifconfig re0 up
# pppoectl -e re0 pppoe0
```

Мы используем интерфейс `re0` нашего компьютера для подключения к DSL-модему.

Затем настройте PPPoE-соединение, указав имя пользователя и пароль, выданные провайдером:

```
# pppoectl pppoe0 myauthproto=pap 'myauthname=XXX' 'myauthsecret=YYY' hisauthproto=none
```

Теперь можно проверить первое соединение. Поскольку что-то может пойти не так, пока ограничим число попыток:

```
# pppoectl pppoe0 max-auth-failure=1
# ifconfig pppoe0 up
# pppoectl -d pppoe0
pppoe0: state = session
        Session ID: 0x254f
        PADI retries: 0
        PADR retries: 0
```

Этот пример вывода показывает работающую настройку. Сеанс PPPoE установлен и всё ещё используется (state = session). Теперь можно проверить согласование IP в PPP:

```
# ifconfig pppoe0
pppoe0: flags=8851<UP,POINTOPOINT,RUNNING,SIMPLEX,MULTICAST> mtu 1492
        inet 117.80.111.85 -> 118.5.113.169 netmask 0xff000000
```

Сделать настройку постоянной можно, создав `/etc/ifconfig.pppoe0`:

Пример 24.3. /etc/ifconfig.pppoe0

```
create
# Mark the physical interface used by this PPPoE interface up
! /sbin/ifconfig re0 up
# Let $int use re0 as its Ethernet interface
! /sbin/pppoectl -e re0 $int
# Configure authentication
! /sbin/pppoectl $int myauthproto=pap 'myauthname=XXX' 'myauthsecret=YYY' hisauthproto=none
# Configure the PPPoE interface itself. These addresses are magic
# meaning we don't care about either address and let the remote
# ppp choose them.
0.0.0.0 0.0.0.1 up
```

Чтобы автоматически получать route(8) во внешний мир, мы используем ifwatchd(8). Создайте следующие сценарии:

Пример 24.4. /etc/ppp/ip-up

```
#!/bin/sh
/sbin/route add default $5
```

Пример 24.5. /etc/ppp/ip-down

```
#!/bin/sh
/sbin/route delete default $5
```

И сделайте их исполняемыми для root:

```
# chmod +x /etc/ppp/ip-up /etc/ppp/ip-down
```

Теперь отредактируйте `/etc/rc.conf`, чтобы включить ifwatchd:

```
ifwatchd=YES
ifwatchd_flags="-u /etc/ppp/ip-up -d /etc/ppp/ip-down pppoe0"
```

И запустите службу:

```
# service ifwatchd start
```

24.4.1. Настройка VLAN

Типичное PPPoE-соединение требует задать на внешнем интерфейсе идентификатор VLAN. В NetBSD это делается созданием интерфейса vlan(4):

```
# ifconfig vlan0 create
# ifconfig vlan0 vlan 6 vlanif pppoe0
```

Пример 24.6. /etc/ifconfig.vlan0

```
create
vlan 6 vlanif pppoe0
```

Чтобы `vlan0` создавался в нужный момент, см. раздел 24.7, «Обеспечение правильного порядка инициализации интерфейсов».

24.4.2. Настройка ограничения MSS

Некоторые системы за неправильно настроенными межсетевыми экранами пытаются использовать Path-MTU-Discovery, тогда как их межсетевой экран блокирует все сообщения ICMP. Это недопустимая, но нередкая конфигурация. Как правило, удалённые серверы с такой конфигурацией вам не подконтрольны, но подключаться к ним всё равно может понадобиться — например, для интернет-банкинга.

Без особых мер такие системы не смогут отправлять крупные порции данных системе, подключённой через PPPoE. Но есть обходной путь: сделать вид, что большие пакеты не поддерживаются, отправив маленький параметр MSS (maximum segment size) при начальном рукопожатии TCP.

Для соединений, исходящих от вашей системы с PPPoE, это делается установкой переменной sysctl(8) `net.inet.tcp.mss_ifmtu` в 1, то есть добавлением в `/etc/sysctl.conf` строки:

```
# Obey interface MTUs when calculating MSS
net.inet.tcp.mss_ifmtu=1
```

Для соединений, исходящих от систем за вашим PPPoE-маршрутизатором, нужно настроить ограничение MSS в межсетевом экране, как в этом примере `/etc/npf.conf`:

```
procedure "norm4" {
        normalize: "random-id", "max-mss" 1440
}

procedure "norm6" {
        normalize: "random-id", "max-mss" 1420
}

group "external" on "pppoe0" {
        pass stateful out final family inet4 all apply "norm4"
        pass stateful out final family inet6 all apply "norm6"
}
```

Дополнительные сведения о настройке NPF см. в разделе 24.5, «Настройка интернет-шлюза с помощью NPF»

24.4.3. Получение адресов IPv6 с помощью делегирования префикса

Чтобы получить адрес IPv6, ядро NetBSD нужно настроить на приём объявлений маршрутизатора IPv6 с помощью sysctl(8):

```
# sysctl -w net.inet6.ip6.accept_rtadv=1
```

Этот параметр можно сделать постоянным, отредактировав `/etc/sysctl.conf`.

Многие провайдеры реализуют IPv6 поверх PPP посредством делегирования префикса. Делегирование префикса можно настроить с помощью dhcpcd(8), например в таком `/etc/dhcpcd.conf`:

Пример 24.7. /etc/dhcpcd.conf

```
duid
ipv6only
require dhcp_server_identifier
option interface_mtu
noipv6rs
slaac private
interface pppoe0
  option rapid_commit
  ipv6rs
  iaid 1
  ia_na 1
  ia_pd 2/::/64 re1/1
```

С этой конфигурацией запуска rtadvd(8) на интерфейсе `re1` должно быть достаточно, чтобы назначить клиентам адреса IPv6.

Если IPv6 всё ещё не работает, попробуйте убедиться, что `ipv6-icmpand` и `dhcpv6` могут проходить через ваш межсетевой экран.

24.5. Настройка интернет-шлюза с помощью NPF

npf(7) (NetBSD Packet Filter) — межсетевой экран NetBSD. Его можно использовать для защиты локальной сети от угроз внешнего интернета, а также для преобразования сетевых адресов (NAT), чтобы IPv4-пакеты попадали на нужный компьютер.

Некоторые примеры использования NPF находятся в подкаталоге `/usr/share/examples/npf`. Загляните в файл `soho_gw-npf.conf` — там приведён пример конфигурации для небольшого домашнего или офисного шлюза.

Чтобы использовать NetBSD в качестве шлюза, необходимо включить параметры sysctl(8) для пересылки пакетов. Их можно добавить в `/etc/sysctl.conf`.

```
# sysctl -w net.inet.ip.forwarding=1
net.inet.ip.forwarding = 1
# sysctl -w net.inet6.ip6.forwarding=1
net.inet6.ip6.forwarding = 1
```

И включите NPF в `/etc/rc.conf`:

```
npf=YES
```

Следующая конфигурация выполняет обычное NAT, используя `re0` как внешнюю сеть, а `re1` как внутренний сетевой интерфейс. Если у вас несколько внутренних сетевых интерфейсов, возможно, их стоит объединить мостом. См. раздел 24.6, «Настройка сетевого моста»

Пример 24.8. /etc/npf.conf

```
$ext_if = "re0"
$int_if = "re1"
$ext_addrs = { ifaddrs($ext_if) }
$localnet = { 192.168.0.0/24 }

# Allow pings.
alg "icmp"

# Perform IPv4 NAT.
map inet4($ext_if) dynamic $localnet -> inet4($ext_if)

group "external" on $ext_if {
        # Allow all outbound traffic
        pass stateful out all
        # Block all incoming traffic
        block in all
}

group "internal" on $int_if {
        # We trust the internal network.
        pass in final all
        pass out final all
}

group default {
        pass final on lo0 all
        block all
}
```

Обычно стоит настроить dhcpd(8), чтобы клиентам автоматически назначались IP-адреса из правильного диапазона:

Пример 24.9. /etc/dhcpd.conf

```
subnet 192.168.0.0 netmask 255.255.255.0 {
        option routers 192.168.0.1;
        option domain-name-servers 9.9.9.9;
        option subnet-mask 192.168.0.0;
        range 192.168.0.100 192.168.0.254;
        default-lease-time 604800; # default lease 7 days
}
```

24.6. Настройка сетевого моста

Мост позволяет объединить разные физические сети в одну логическую, то есть соединить их на уровне 2 модели ISO-OSI, а не на уровне 3, как это сделал бы маршрутизатор. Он позволяет адресовать несколько сетевых интерфейсов как один. Драйвер «bridge» в NetBSD обеспечивает функциональность моста в системах NetBSD.

24.6.1. Пример моста

В этом примере две физические сети объединяются в одну логическую сеть 192.168.1.0 с помощью моста NetBSD. Машина NetBSD, которая будет работать как мост, имеет два интерфейса, ne0 и ne1, каждый из которых подключён к одной физической сети.

Когда система готова, можно создать мост — это делается командой brconfig(8). Сначала нужно создать интерфейс моста. Следующая команда ifconfig(8) создаст интерфейс `bridge0`:

```
$ ifconfig bridge0 create
```

Убедитесь, что к этому моменту интерфейсы ne0 и ne1 подняты. Следующий шаг — добавить интерфейсы ne0 и ne1 в мост.

```
$ brconfig bridge0 add ne0 add ne1 up
```

Эту конфигурацию можно задать автоматически, создав файл `/etc/ifconfig.interface`, в данном случае `/etc/ifconfig.bridge0`, со следующим содержимым:

```
create
!brconfig $int add ne0 add ne1 up
```

> **ПРИМЕЧАНИЕ**
>
> В NetBSD 10.0 и более поздних версиях в качестве конечной точки моста потребуется использовать *vether* вместо *tap*. В предыдущих выпусках *vether* недоступен.

После настройки моста его конфигурацию можно вывести командой **brconfig -a**. Помните, что если вы хотите назначить машине-мосту IP-адрес, его можно присвоить только одному из интерфейсов, входящих в мост. Виртуальный интерфейс tap(4) также можно создать и настроить как конечную точку моста, например в `/etc/ifconfig.tap0`:

Пример 24.10. /etc/ifconfig.tap0

```
create
inet 192.168.0.1 netmask 255.255.255.0
up
!ifconfig bridge0 create
!brconfig bridge0 add $int add ne0 add ne1 up
```

24.7. Обеспечение правильного порядка инициализации интерфейсов

В предыдущем примере (раздел 24.6, «Настройка сетевого моста») мы создали bridge(4) и tap(4), работа которых зависит от других сетевых интерфейсов. Это может стать проблемой, если rc(8) инициализирует их раньше интерфейсов, от которых они зависят. К счастью, в `/etc/rc.conf` можно задать нужный порядок инициализации:

```
auto_ifconfig=NO
net_interfaces="ne0 ne1 pppoe0 bridge0 tap0"
```

С этими строками первым будет прочитан `/etc/ifconfig.ne0`, а последним — `/etc/ifconfig.tap0`. То же самое относится к случаю, когда строки вида `ifconfig_ne0="up"` используются в `/etc/rc.conf` вместо отдельных файлов конфигурации.

24.8. Некоторые полезные команды

Для диагностики проблем могут быть полезны следующие команды:

ifconfig(8)

Показывает конфигурацию сетевых интерфейсов и позволяет её изменять.

ping(8)

Попытка достучаться до хоста и измерить задержку.

netstat(1)

Показывает активные соединения.

npfctl(8)

npfctl show показывает текущую конфигурацию межсетевого экрана, а **npfctl validate filename** позволяет проверить правильность конфигурации перед её загрузкой.

route(8)

route show показывает таблицы маршрутизации, другие команды позволяют ими управлять.

traceroute(8)

Показывает маршрут, который проходят пакеты до места назначения.

sysstat(1)

sysstat ifstat можно использовать для наблюдения за сетевыми интерфейсами.

tcpdump(8)

Можно использовать для наблюдения за трафиком TCP/IP.

Глава 25. Интернет-суперсервер inetd

«Интернет-суперсервер», или inetd(8), есть во всех Unix-подобных системах и предоставляет многие из доступных базовых сетевых служб. В этой главе описана связь между этим демоном и несколькими файлами конфигурации в каталоге `/etc/`.

25.1. Обзор

В этом документе мы рассмотрим простое определение inetd(8), то, как работают несколько файлов, связанных с inetd(8) (заметьте, что эти файлы не связаны с другим программным обеспечением), как добавить службу в inetd(8), а также некоторые соображения о том, когда стоит использовать inetd(8) для конкретной службы, а когда службе лучше работать вне inetd(8).

25.2. Что такое inetd?

В традиционных сценариях Unix один серверный (демон) процесс ожидает подключений на определённом порту и обрабатывает входящие запросы. Но если машина предоставляет много служб, потребовалось бы много процессов-демонов, которые большей частью простаивают, но всё равно расходуют ресурсы, например память. Интернет-суперсервер inetd — это подход к решению данной проблемы. Он прослушивает ряд портов и, получив запрос, определяет, какую программу запустить для его обработки, и запускает экземпляр этой программы.

Ниже приведена очень простая схема, иллюстрирующая inetd(8):

```
 pop3  ------  |
               |
 ftpd -------  | INETD | ---- Internet / DMZ / Switch / Whatever . . .
               |
cvsupserver -  |
```

На схеме выше видна общая идея. Процесс inetd(8) получает запрос и запускает подходящий серверный процесс. inetd(8) выполняет программное мультиплексирование. Здесь важно отметить один момент, касающийся безопасности: во многих других UNIX-подобных системах в качестве средства повышения безопасности inetd(8) используется пакет tcpwrappers. В NetBSD функциональность tcpwrapper встроена в inetd(8) с помощью libwrap.

25.3. Настройка inetd — `/etc/inetd.conf`

Работа inetd(8) управляется его собственным файлом конфигурации, который, как ни удивительно, называется `/etc/inetd.conf`, см. inetd.conf(5). Файл `inetd.conf` в основном включает службы, которые системный администратор хотел бы мультиплексировать через inetd(8), и сопоставляет их программам, указывая, какую программу запускать для входящих запросов на каком порту.

inetd.conf(5) — это ASCII-файл, содержащий по одной службе в строке и несколько полей в каждой строке. Основная структура полей такова:

```
service-name socket-type protocol wait/nowait user:group server-program arguments
```

имя службы:

Имя службы указывает порт, который должен прослушивать inetd(8). Это либо десятичное число, либо имя, совпадающее с именем службы, заданным в `/etc/services`.

тип сокета:

Тип сокета связи. Разные типы: «stream» — для потока TCP, «dgram» — для службы UDP, «raw» — для сырого сокета, «rdm» — для надёжно доставляемых сообщений, «seqpacket» — для сокета упорядоченных пакетов. Наиболее распространены типы «stream» и «dgram».

протокол

Используемый протокол — в основном «tcp», «tcp6», «udp» и «udp6» для служб, ориентированных на поток, через Transmission Control Protocol, или служб, ориентированных на датаграммы, через User Datagram Protocol. Стоит отметить, что «tcp» и «udp» означают использование протокола по умолчанию (сейчас это IPv4), «tcp4» явно означает связь только по IPv4, а «tcp6» и «udp6» — только по IPv6. Кроме того, протоколы на основе удалённого вызова процедур (RPC) можно задать как «rpc/tcp» или «rpc/udp».

wait/nowait

Это поле сообщает inetd(8), должен ли он ждать завершения работы программы сервера или сразу продолжать обрабатывать новые подключения. Многие подключения к серверным процессам требуют ответов после завершения передачи данных, тогда как другие типы могут непрерывно передавать данные по одному соединению; последний случай — «nowait», первый — «wait». В большинстве случаев это поле соответствует типу сокета: например, потоковое соединение (чаще всего) имеет в этом поле значение «nowait».

user[:group]

Это поле задаёт имя пользователя и, необязательно, имя группы, от имени которых работает серверный процесс, запускаемый inetd(8).

программа сервера

Это поле — полный путь к программе, которая запускается.

аргументы программы

Это поле содержит вектор аргументов argv[] запускаемой программы, включая имя программы и дополнительные аргументы, которые системному администратору может понадобиться указать для запускаемой программы сервера.

Всё это немало для усвоения, и с некоторыми полями системный администратор может делать и другие вещи. Вот пример строки из файла `inetd.conf`:

```
ftp       stream  tcp    nowait  root   /usr/libexec/ftpd    ftpd -ll
```

Слева направо: имя службы — «ftp», тип сокета — «stream», протокол — «tcp», inetd(8) не будет ждать завершения серверного процесса («nowait»), процесс работает от имени пользователя «root», путь — `/usr/libexec/ftpd`, а имя программы и аргументы — «ftpd -ll». Обратите внимание, что в последнем поле имя программы отличается от имени службы.

25.4. Службы — `/etc/services`

Следующий файл — база данных имён служб, находящаяся в `/etc/services`. Этот файл в основном содержит сведения, сопоставляющие имя службы номеру порта. Формат файла `/etc/services` таков:

```
service-name port-number/protocol-name [aliases]
```

«service-name» — имя службы, «port-number» — номер порта, назначенный службе, «protocol-name» — «tcp» или «udp», а если нужны псевдонимы для порта, их можно добавить как «aliases», разделив пробельными символами. Комментарии можно добавлять после знака решётки (#).

Рассмотрим в качестве примера записи «ssh»:

```
ssh             22/tcp          # Secure Shell
ssh             22/udp
```

Как видим, слева направо: имя службы — «ssh», номер порта — «22», протоколы — «tcp» и «udp». Обратите внимание, что для каждого протокола, который может использовать служба, есть отдельная запись (даже на одном и том же порту).

25.5. Протоколы — `/etc/protocols`

Ещё один файл, который читает inetd(8), — `/etc/protocols`. В нём содержатся сведения о протоколах интернета DARPA. Формат базы данных имён протоколов таков:

```
protocol-name number [aliases]
```

где «protocol-name» описывает полезную нагрузку IP-пакета, например «tcp» или «udp». «number» — официальный номер протокола, назначенный IANA, после которого можно добавить необязательные псевдонимы.

Рассмотрим в качестве примера седьмую запись в базе `/etc/protocols`:

```
tcp     6       TCP             # transmission control protocol
```

Слева направо: имя протокола — «tcp», номер — «6», а единственный указанный псевдоним — «TCP», относящийся к Transmission Control Protocol, как следует из комментария в этой строке.

25.6. Удалённый вызов процедур (RPC) — `/etc/rpc`

База данных номеров RPC-программ, используемая службами с типом протокола «rpc» в inetd.conf(5), хранится в `/etc/rpc` и содержит сопоставление имён номерам RPC-программ. Формат файла таков:

```
server-name program-number aliases
```

Например, вот запись nfs:

```
nfs             100003  nfsprog
```

25.7. Разрешение и запрет хостов — `/etc/hosts.{allow,deny}`

Как упоминалось выше, inetd(8) в NetBSD содержит пакет tcpwrapper, встроенный через библиотеку libwrap. Благодаря этому inetd(8) может разрешать или запрещать доступ к каждой службе более тонко, чем просто разрешить службу всем или вообще её не включать. Управление доступом задаётся в файлах `/etc/hosts.allow` и `/etc/hosts.deny`, см. страницу руководства hosts_access(5).

Каждый из этих двух файлов содержит несколько строк, описывающих ограничения доступа для определённого сервера. Доступ разрешён, если разрешение указано в `/etc/hosts.allow`. Если служба указана не в `/etc/hosts.allow`, а в `/etc/hosts.deny`, доступ запрещается. Если служба не указана ни в одном файле, доступ разрешён, что соответствует стандартному поведению inetd(8).

Каждая строка в `/etc/hosts.allow` и `/etc/hosts.deny` содержит службу либо по имени (как оно задано в поле для argv[0] в `/etc/inetd.conf`, например «ftpd» вместо «ftp»), либо специальную службу «ALL», которая, очевидно, относится ко всем службам. За именем службы через двоеточие следует ряд ограничений доступа, которыми могут быть имена хостов, домены, отдельные IP-адреса, целые IP-подсети или иные ограничения; все подробности см. в hosts_access(5).

Пример конфигурации, которая в основном открыта, но запрещает доступ к службам определённому хосту и всем машинам из определённого домена, выглядел бы так:

```
# /etc/hostname.deny:
ALL: some.host.name, .some.domain
```

Другой пример — почти полностью закрытая конфигурация, запрещающая доступ всем, кроме очень немногих машин, потребует записей как в `/etc/hosts.allow`, так и в `/etc/hosts.deny`. Запись для `/etc/hosts.deny` была бы такой:

```
# /etc/hosts.deny
ALL: ALL
```

Запись, разрешающая доступ нескольким хостам, помещается в `/etc/hosts.allow`:

```
# /etc/hosts.allow
ALL: friend.host.domain otherfriend.otherhost.otherdomain
```

25.8. Добавление службы

Системному администратору нередко требуется добавить в свою систему службу, которой ещё нет в inetd(8), или он может захотеть перевести службу под управление inetd, потому что нагрузка у неё небольшая. Обычно это довольно просто, поэтому в качестве примера рассмотрим добавление версии POP3 в систему NetBSD.

В данном случае мы получили и установили пакет «cucipop», который находится в `pkgsrc/mail/cucipop`. Этот сервер довольно прост в использовании, единственные странности — другие пути. Поскольку это POP3, мы знаем, что это потоковое соединение со значением «nowait». Запуск от имени «root» подойдёт, отличается только расположение программы и имя самой программы.

Итак, первая половина новой записи в `/etc/inetd.conf` выглядит так:

```
pop3   stream  tcp     nowait  root
```

После установки pkgsrc поместил cucipop в `/usr/pkg/sbin/cucipop`. Таким образом, следующее поле выглядит так:

```
pop3   stream  tcp     nowait  root /usr/pkg/sbin/cucipop
```

Наконец, мы хотим использовать формат почтовых ящиков Berkeley, поэтому наша серверная программа должна вызываться с параметром `-Y`. В итоге вся запись выглядит так:

```
pop3   stream  tcp     nowait  root /usr/pkg/sbin/cucipop cucipop -Y
```

Мы добавили службу с именем «pop3» в `/etc/inetd.conf`. Следующее, что нужно проверить, — умеет ли система сопоставить имя службы номеру порта в `/etc/services`:

```
# grep ^pop3 /etc/services
pop3            110/tcp         # POP version 3
pop3            110/udp
pop3s           995/tcp                 # pop3 protocol over TLS/SSL (was spop3)
pop3s           995/udp                 # pop3 protocol over TLS/SSL (was spop3)
```

Здесь нас интересуют записи «pop3», то есть они уже содержатся в файле `/etc/services`, поставляемом с NetBSD.

Теперь, чтобы inetd(8) начал использовать новую запись, просто перезапустите его с помощью сценария rc:

```
# service inetd restart
```

Всё готово. В большинстве случаев у используемого вами программного обеспечения есть документация, где указана нужная запись; если её нет, иногда помогает поискать что-то похожее на серверную программу, которую вы добавляете. Классический пример — сервер MUD со встроенным telnet. Можно почти без изменений взять запись telnet и поменять нужные части.

25.9. Когда использовать inetd, а когда нет

Решение о том, добавлять службу в inetd(8) или выводить её оттуда, обычно основано на нагрузке на сервер. Например, в большинстве систем демону telnet не требуется столько новых подключений, сколько, скажем, почтовому серверу. Чаще всего администратору приходится на ощупь определять, стоит ли переносить службу.

Хороший пример, который я видел, — почтовые службы, такие как smtp и pop. Я настроил почтовый сервер, где pop3 работал через inetd(8), а exim работал самостоятельно; я ошибочно предположил, что всё будет работать нормально, поскольку пользователей мало — я сам и одна диагностическая учётная запись. Сервер также был настроен как резервный MX и релей на случай отказа другого, сильно нагруженного. Когда я провёл несколько тестов, то обнаружил огромную задержку при удалённых подключениях по pop. Причина была в моём постоянном заборе почты и в том, что диагностический пользователь непрерывно отправлял диагностику туда-обратно. В итоге мне пришлось вывести службу pop3 из inetd(8).

Причина переноса службы на самом деле довольно интересна. Когда какая-то служба начинает интенсивно использоваться, она, конечно, создаёт нагрузку на систему. Если служба работает внутри мета-демона inetd(8), последствия высокой нагрузки на неё могут навредить и другим службам, использующим inetd(8). Если мультиплексор получает слишком много запросов к одной конкретной

службе, это начнёт сказываться на производительности других служб, использующих inetd(8). В такой ситуации решение — вывести проблемную службу из-под управления inetd(8), чтобы улучшилось время отклика и самой службы, и inetd(8).

25.10. Другие источники

Ниже приведены дополнительные материалы и сведения по темам, затронутым в этом документе.

Страницы руководства NetBSD:

- inetd(8)
- protocols(5)
- rpc(5)
- services(5)
- hosts_access(5)

Прочие ссылки:

- IANA: номера протоколов и службы назначения
- RFC1700: назначенные номера

Глава 26. Система доменных имён

Использование системы доменных имён обсуждалось в предыдущих главах — без подробностей о настройке сервера, предоставляющего эту службу. В этой главе описана настройка простого небольшого домена с одним сервером имён системы доменных имён (DNS) в системе NetBSD. Она включает краткое объяснение и обзор DNS; дополнительную информацию можно получить в каталоге ресурсов DNS (DNSRD) по адресу http://www.dns.net/dnsrd/.

26.1. Основы и понятия DNS

DNS — широко используемая *служба имён* в интернете и других сетях TCP/IP. Сетевые протоколы, форматы данных и файлов, а также другие аспекты DNS являются стандартами интернета, определены в ряде документов RFC и описаны в ряде других справочных и учебных работ. DNS имеет распределённую архитектуру «клиент — сервер». Существуют эталонные реализации сервера и клиента, но они не являются частью стандарта. Для многих платформ доступен ряд дополнительных реализаций.

26.1.1. Службы имён

Службы имён служат для сопоставления текстовых имён и тех или иных конфигурационных данных. *Сервер имён* хранит это соответствие, а клиенты запрашивают у сервера имён *разрешение* имени в связанные с ним данные.

Читателю следует хорошо понимать основные принципы сопоставления имён хостов IP-адресам и спецификации классов IP-адресов, см. раздел 23.6, «Понятия службы имён».

В случае DNS конфигурационные данные, привязанные к имени, имеют форму стандартных *ресурсных записей* (RR). Эти текстовые имена подчиняются определённым структурным соглашениям.

26.1.2. Пространство имён DNS

DNS представляет собой иерархическое пространство имён, во многом похожее на файловую систему UNIX, изображаемое в виде перевёрнутого дерева с *корнем* наверху.

```
TOP-LEVEL                                .org

                                           |

MID-LEVEL                             .diverge.org

                      ________________________|_________________________

                     |                        |                         |

BOTTOM-LEVEL strider.diverge.org   samwise.diverge.org   wormtongue.diverge.org
```

При желании систему можно разделить логически ещё дальше, в разных точках. В примере выше показаны три узла в домене diverge.org, но мы могли бы разделить diverge.org и на поддомены, например «strider.net1.diverge.org», «samwise.net2.diverge.org» и «wormtongue.net2.diverge.org»; в этом случае 2 узла находятся в «net2.diverge.org», а один — в «net1.diverge.org».

Существуют каталоги имён, некоторые из которых могут быть подкаталогами других имён. Такие каталоги иногда называют *зонами*. Предусмотрены символические ссылки, перенаправляющие запросы информации об одном имени к записям, привязанным к другому имени. Каждое имя, распознаваемое DNS, называется *доменным именем*, независимо от того, представляет ли оно информацию о конкретном хосте, каталог подчинённых доменных имён (или и то и другое, или что-то ещё).

Однако, в отличие от большинства схем имён файловых систем, доменные имена записываются так, что самое внутреннее имя находится слева, а домены более высокого уровня — последовательно справа, вплоть до корневого каталога, если это необходимо. Разделителем при записи доменных имён служит точка — «.».

Как и путевые имена файловых систем, доменные имена можно записывать абсолютным или относительным образом, хотя в деталях есть некоторые различия. Например, нет никакой возможности косвенно сослаться на родительский домен, как это делается с помощью каталога UNIX `..`. Многие (но не все) резолверы предлагают механизм пути поиска, благодаря которому частично заданные имена можно разрешать относительно дополнительно перечисленных поддоменов, помимо собственного домена клиента. Имена, полностью заданные вплоть до корня, называются *полными доменными именами* (Fully Qualified Domain Names) или *FQDN*. Определяющая особенность FQDN — она записывается с завершающей точкой. То же имя без завершающей точки может рассматриваться как относительное к некоторому другому поддомену. Такое редко происходит без злого умысла, но отчасти из-за этой возможности в ряде ситуаций в качестве параметров конфигурации требуются именно FQDN.

В Интернете существуют установившиеся соглашения об именах первых нескольких уровней дерева — на этом уровне иерархия достигает отдельной организации. Эта организация отвечает за установление и поддержание соглашений ниже по дереву, в пределах своего домена.

26.1.3. Ресурсные записи

Ресурсные записи домена хранятся в стандартизованном формате в текстовом файле ASCII, который часто называют *файлом зоны*. Обычно используются перечисленные ниже ресурсные записи (некоторые другие определены, но применяются редко или больше не применяются). В ряде случаев с одним именем может быть связано несколько типов RR и даже несколько записей одного типа.

Распространённые ресурсные записи DNS

A: адрес

Эта запись содержит числовой IP-адрес, связанный с именем.

CNAME: каноническое имя

Эта запись содержит каноническое имя (FQDN, для которого имеется связанная запись A) имени хоста, к которому привязана данная запись. Этот тип записи служит для создания псевдонимов имён: он даёт ссылку на другое имя, с которым связаны соответствующие RR. Если с именем связана запись CNAME, то это псевдоним, и никакие другие RR не могут быть привязаны к тому же имени.

Такие записи часто используются для указания на хосты, предоставляющие определённую службу, например сервер FTP или HTTP. Если службу нужно перенести на другой хост, можно изменить псевдоним, и то же имя будет указывать на новый хост.

PTR: указатель

Эта запись содержит текстовое имя. Такие записи привязаны к именам, построенным особым образом из числовых IP-адресов, и служат для обратного отображения IP-адреса в текстовое имя. Подробнее это описано в разделе 26.1.8, «Обратное разрешение».

NS: сервер имён

Этот тип записи используется для *делегирования* поддерева пространства доменных имён другому серверу имён. Запись содержит FQDN сервера имён DNS, располагающего сведениями о поддомене, и привязана к имени этого поддомена. Так устанавливается иерархическая структура DNS. Делегирование подробнее описано в разделе 26.1.4, «Делегирование».

MX: обмен почтой

Эта запись содержит FQDN хоста, который будет принимать электронную почту по SMTP для указанного домена, вместе со значением приоритета, используемым для выбора MX-хоста при пересылке почты. Она служит для указания других серверов, готовых принимать и накапливать почту для домена, если основной MX какое-то время недоступен. Она также применяется для направления почты на центральный сервер, если это нужно, а не на каждую отдельную рабочую станцию.

HINFO: сведения о хосте

Содержит две строки, предназначенные для описания аппаратного обеспечения хоста и платформы операционной системы. Для некоторых систем определены стандартные строки, но их применение не является обязательным. Некоторые узлы из соображений безопасности не раскрывают эту информацию.

TXT: текст

Текстовое поле произвольной формы, иногда используемое как поле комментария, а иногда дополняемое дополнительным смыслом, который определяется локальными соглашениями.

SOA: начало полномочий

Эта запись обязательна для каждого файла зоны. В ней перечислены основной сервер имён и адрес электронной почты лица, ответственного за домен, а также значения по умолчанию для ряда полей, связанных с поддержанием согласованности на нескольких серверах и кэшированием результатов DNS-запросов.

26.1.4. Делегирование

С помощью записей NS можно делегировать полномочия на части пространства имён DNS ниже определённой точки дерева, а ещё меньшие части ниже этой точки — делегировать дальше. Именно здесь становится важным различие между доменом и зоной. Любое имя в DNS называется доменом, и этот термин относится к этому имени и ко всем подчинённым именам ниже него в дереве. Границы зоны уже, и они определяются делегированиями. Зона начинается с делегирования (или с корня) и охватывает все имена домена ниже этой точки, за исключением имён ниже любого последующего делегирования.

Это различие важно для реализации: зона — единая административная сущность (с единственной записью SOA), и все данные зоны указываются в одном файле, который называется *файлом зоны*. При желании файл зоны может содержать более одного разделённого точками уровня дерева пространства имён — для этого в имена в этом файле включаются точки. Чтобы упростить администрирование и предотвратить чрезмерно большие файлы зон, сервер DNS вполне законно может делегировать полномочия самому себе, разделив домен на несколько зон, хранящихся на одном и том же сервере.

26.1.5. Делегирование нескольким серверам

Для избыточности обычно (и часто по административным требованиям) бывает так, что сведения о зоне предоставляет не один сервер имён. Также обычно, что по крайней мере один из этих серверов

расположен на некотором расстоянии (с точки зрения сетевой топологии) от остальных, чтобы знания об этой зоне не становились недоступны при нарушении связности. Каждый сервер имён указывается в записи NS, привязанной к имени зоны и хранящейся в родительской зоне на сервере, отвечающем за родительский домен. Так те, кто ищет в иерархии имён сверху вниз, могут обратиться к любому из серверов, чтобы продолжить сужение поиска. Это иногда называют *обходом дерева*.

В Интернете есть ряд серверов имён, которые называются *корневыми серверами имён*. Эти серверы предоставляют сведения о самых верхних уровнях дерева пространства доменных имён. Они особенны тем, что их адреса должны быть заранее настроены в серверах имён как отправная точка для поиска других серверов. Изолированным сетям, не имеющим доступа к этим серверам, возможно, потребуется собственный корневой сервер имён.

26.1.6. Вторичные серверы, кэширование и запись SOA

Чтобы поддерживать согласованность между этими серверами, один из них обычно настраивается как *основной*, и все административные изменения вносятся именно на нём. Остальные серверы настраиваются как *вторичные* и передают содержимое зоны с основного. Такая модель работы не является обязательной, и если того требуют внешние обстоятельства, вместо неё можно использовать несколько основных серверов, но тогда согласованность придётся поддерживать другими средствами. Серверы DNS, хранящие ресурсные записи для зоны, будь то основные или вторичные серверы, называются *авторитетными* для этой зоны. Один сервер DNS может быть авторитетным для нескольких зон.

Получая ответы на запросы, серверы имён могут *кэшировать* результаты. Это заметно благоприятно сказывается на скорости запросов, нагрузке от запросов на серверы верхнего уровня и использовании сети. Это также одна из главных причин объёма памяти, занимаемого процессом сервера имён.

Есть ряд параметров, важных для поддержания согласованности между вторичными серверами и кэшами. Значения этих параметров для конкретной зоны хранятся в записи SOA. Вот эти поля:

Поля записи SOA

Serial

Порядковый номер файла зоны. Его следует увеличивать всякий раз, когда данные домена изменяются. Когда вторичный сервер хочет проверить, актуальны ли его данные, он проверяет порядковый номер в записи SOA основного сервера.

Refresh (обновление)

Время в секундах, определяющее, как часто вторичный сервер должен проверять порядковый номер на основном сервере и начинать новую передачу, если у основного сервера данные новее.

Retry (повтор)

Если вторичному серверу не удалось подключиться к основному по истечении времени обновления (например, хост выключен), это значение в секундах задаёт, как часто следует повторять подключение.

Expire (истечение)

Если повторные попытки не достигли основного сервера за указанное число секунд, вторичный сервер уничтожает свои копии файлов данных зоны и перестаёт отвечать на запросы об этом домене. Это не позволяет очень старым и потенциально неточным данным оставаться в обращении.

TTL

Это поле задаёт время в секундах, в течение которого ресурсные записи этой зоны должны оставаться действительными в кэшах других серверов имён. Если данные изменчивы, это значение должно быть небольшим. TTL — распространённая аббревиатура от «Time To Live» (время жизни).

26.1.7. Разрешение имён

Клиенты DNS настраиваются с адресами серверов DNS. Обычно это серверы, авторитетные для домена, членом которого является клиент. Все запросы на разрешение имён начинаются с запроса к одному из этих локальных серверов. Запросы DNS бывают двух видов:

- *Рекурсивный* запрос просит сервер имён разрешить имя полностью и вернуть результат. Если запрос нельзя удовлетворить напрямую, сервер имён ищет в своей конфигурации и кэшах сервер выше в дереве доменов, у которого может быть больше сведений. В худшем случае это будет список заранее настроенных серверов для корневого домена. Эти адреса возвращаются в ответе, который называется *ссылкой* (referral). После этого локальный сервер имён должен отправить свой запрос одному из этих серверов.
- Обычно это *итеративный* запрос, который просит второй сервер имён либо дать авторитетный ответ, либо сообщить адреса серверов имён (записи NS), перечисленных в его таблицах или кэшах как авторитетные для соответствующей зоны. Затем локальный сервер имён выполняет итеративные запросы, спускаясь по дереву вниз, пока не будет найден авторитетный ответ (положительный или отрицательный) и возвращён клиенту.

В некоторых конфигурациях — например, когда межсетевой экран препятствует прямым IP-соединениям между клиентами DNS и внешними серверами имён, или когда узел подключён к остальному миру по медленному каналу, — сервер имён можно настроить со сведениями о *пересыльщике* (forwarder). Это внешний сервер имён, к которому локальный сервер имён должен обращаться как клиент, прося внешний сервер выполнить полный рекурсивный поиск имени и вернуть результат в одном запросе (который затем можно кэшировать), а не отвечать ссылками.

26.1.8. Обратное разрешение

DNS обеспечивает разрешение от текстового имени к ресурсной записи, например к записи A с IP-адресом. Он не даёт способа, кроме полного перебора, сопоставить в обратном направлении; нет механизма, позволяющего узнать, какое имя привязано к конкретной RR.

Для многих типов RR это не имеет реальных последствий, однако часто полезно определить по имени хост, которому принадлежит конкретный IP-адрес. Вместо того чтобы усложнять проектирование и реализацию механизма базы данных DNS, добавляя функции сопоставления в обоих направлениях, DNS использует существующие механизмы и создаёт специальное пространство имён, заполненное записями PTR, для разрешения IP-адреса в имя. Такое разрешение часто называют *обратным разрешением*, хотя этот термин и неточен по своей сути.

Это достигается следующим образом:

- Резервируется обычное доменное имя, назначение которого — отображение IP-адресов. Используется имя «in-addr.arpa.», отражающее исторические истоки Интернета в программе финансирования Агентства перспективных исследовательских проектов Министерства обороны США.
- Затем этот домен делится и делегируется в соответствии со структурой IP-адресов. IP-адреса часто записывают в *десятичной точечной записи*, где каждый октет четырёхоктетного адреса записывается в десятичном виде и отделяется точками. Диапазоны IP-адресов обычно

делегируются так, что чем меньше делегируемая часть, тем больше совпадающих крайних левых частей адреса. Поэтому, чтобы делегирование домена обратного просмотра было удобным, при работе с иерархическим пространством имён DNS порядок переворачивается: домены более высокого уровня располагаются в имени справа.

- Каждый байт IP-адреса записывается как ASCII-представление числа в десятичном виде; октеты следуют в обратном порядке, разделяются точками и дополняются доменным именем in-addr.arpa. Например, чтобы определить имя хоста сетевого устройства с IP-адресом 11.22.33.44, этот алгоритм даст строку «44.33.22.11.in-addr.arpa.», которая является допустимым структурированным доменным именем. Затем на сервер имён отправляется обычный запрос службы имён с запросом записи PTR, привязанной к сформированному имени.
- Запись PTR, если она будет найдена, содержит FQDN хоста.

Одно из следствий этого — возможны несовпадения. Разрешение имени в запись A, а затем разрешение имени, построенного из адреса в этой записи A, в запись PTR может дать запись PTR, не содержащую исходного имени. В DNS нет ограничения, требующего, чтобы «обратное» отображение совпадало с «прямым». В некоторых обстоятельствах это полезное свойство, особенно когда требуется, чтобы с несколькими именами были связаны записи A, содержащие один и тот же IP-адрес.

Хотя в DNS такого ограничения нет, некоторые серверные программы и сетевые библиотеки отвергают подключения от хостов, не прошедших следующую проверку:

- в информации о состоянии, прилагаемой к входящему подключению, есть IP-адрес источника запроса;
- выполняется поиск PTR, чтобы получить FQDN хоста, устанавливающего подключение;
- затем для возвращённого имени выполняется поиск A, и подключение отвергается, если IP-адрес источника не указан среди возвращённых записей A.

Это делается как мера безопасности, помогающая выявлять и предотвращать выдачу злонамеренных узлов за другие узлы путём настройки собственных записей PTR, которые возвращают имена хостов, принадлежащих другой организации.

26.2. Файлы DNS

Теперь рассмотрим, как на самом деле настроить небольшую сеть с поддержкой DNS. Мы продолжим использовать примеры из главы 24, *Настройка TCP/IP в NetBSD на практике*, то есть предполагаем, что:

- наша сеть IP работает правильно;
- у нас правильно работает IPNAT;
- сейчас все хосты используют DNS провайдера;

Нашим сервером имён будет хост «strider», на котором также работает IPNAT, а два наших клиента используют «strider» как шлюз. Не так уж важно, какой интерфейс установлен на «strider»; для примера скажем, что это коммутируемое соединение 56k.

Итак, прежде чем идти дальше, посмотрим на файл `/etc/hosts` на «strider» — до того, как мы внесли изменения для использования DNS.

Пример 26.1. Файл /etc/hosts на strider

```
127.0.0.1       localhost
192.168.1.1     strider
192.168.1.2     samwise sam
192.168.1.3     wormtongue worm
```

Это не слишком большая сеть, но стоит отметить, что те же правила применимы и к более крупным сетям, если говорить в контексте этого раздела.

Другое допущение, которое мы хотим сделать: настраиваемый домен — `diverge.org`, и этот домен известен только в нашей внутренней сети, а не во всём мире. Помимо статического IP-адреса, потребовалась бы надлежащая регистрация IP-адреса сервера имён как основного. Это в основном административные вопросы, которые здесь не рассматриваются.

Операционная система NetBSD предоставляет набор файлов конфигурации для настройки DNS. Помимо файла `/etc/named.conf` по умолчанию, в каталоге `/etc/namedb` хранятся следующие файлы:

- `localhost`
- `127`
- `loopback.v6`
- `root.cache`

В этом разделе вы увидите изменённые версии этих файлов; настоятельно рекомендую сделать резервные копии исходных файлов для справки.

> **ПРИМЕЧАНИЕ**
>
> Примеры в этой главе относятся к BIND основной версии 8, однако следует отметить, что формат базы данных имён и других файлов конфигурации почти на 100% совместим между версиями. Единственное отличие, которое я заметил, — информация «$TTL» не требовалась.

26.2.1. `/etc/named.conf`

Первый файл, который мы рассмотрим, — `/etc/named.conf`. Это файл конфигурации для bind (отсюда и запоминающееся имя). Настроить такую систему, как наша, относительно просто. Сначала вот как выглядит мой файл:

```
options {
        directory "/etc/namedb";
        allow-transfer { 192.168.1.0/24; };
        allow-query { 192.168.1.0/24; };
        listen-on port 53 { 192.168.1.1; };
};

zone "localhost" {
   type master;
   notify no;
   file "localhost";
};

zone "127.IN-ADDR.ARPA" {
   type master;
   notify no;
   file "127";
};
```

```
zone "0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.ip6.int" {
   type master;
   file "loopback.v6";
};

zone "diverge.org" {
   type master;
   notify no;
   file "diverge.org";
};

zone "1.168.192.in-addr.arpa" {
   type master;
   notify no;
   file "1.168.192";
};

zone "." in {
   type hint;
   file "root.cache";
};
```

Учтите, что в моём `named.conf` раздел корня («.») идёт последним: в Интернете есть другой домен с именем diverge.org (он, кстати, принадлежит мне), поэтому я хочу, чтобы резолвер обращался к Интернету в последнюю очередь. В большинстве систем это обычно не так.

Здесь очень важно помнить ещё одно: если у вас внутренняя конфигурация, то есть нет действующего подключения к Интернету и/или нет необходимости обращаться к корневым серверам, закомментируйте зону корня («.»). Иначе могут возникнуть проблемы с поиском, если какой-то клиент решит обратиться к домену в Интернете, который наш сервер не смог бы разрешить самостоятельно.

Выглядит довольно запутанно, но при более внимательном рассмотрении оказывается, что многие строки в каждом разделе несколько избыточны. Поэтому объяснять их придётся всего несколько раз.

Пройдёмся по разделам `named.conf`:

26.2.1.1. options

В этом разделе определяются некоторые глобальные параметры; самый заметный из них — расположение таблиц DNS: в этой конкретной системе они помещаются в `/etc/namedb`, как указано в параметре «directory».

Далее идут остальные параметры:

allow-transfer

Этот параметр перечисляет удалённые серверы DNS, работающие как вторичные, которым разрешена передача зон, то есть разрешено считывать все данные DNS сразу. По соображениям конфиденциальности это следует ограничить только вторичными серверами DNS.

allow-query

Этот параметр определяет, из какой сети хосты вообще могут опрашивать этот сервер имён. Ограничение запросов только локальной сетью (192.168.1.0/24) не даёт запросам приходить на внешний интерфейс сервера DNS и предотвращает возможные проблемы с конфиденциальностью.

listen-on port

Этот параметр определяет порт и связанные с ним IP-адреса, на которых этот сервер будет запускать named(8). Опять же, «внешний» интерфейс здесь не прослушивается, чтобы запросы не

поступали «снаружи».

Остальная часть файла `named.conf` состоит из «зон». Зона — это область, к которой можно привязать элементы для разрешения: например, у домена могут быть привязаны имена хостов, разрешаемые в IP-адреса, а у обратной зоны — IP-адреса, которые разрешаются обратно в имена хостов. С каждой зоной связан файл, а в нём — таблица для разрешения именно этой зоны. Как легко заметить, их формат в `named.conf` поразительно похож, поэтому я выделю только одну из их записей:

26.2.1.2. зона «diverge.org»

type

Тип зоны обычно «master» во всех случаях, кроме корневой зоны «.» и зон, для которых предоставляется вторичная (резервная) служба; в последнем случае тип, очевидно, «secondary».

notify

Хотите ли вы отправлять уведомления вторичным серверам при изменении зоны? В этой конфигурации, очевидно, нет, поэтому здесь задано «no».

file

Этот параметр задаёт имя файла в нашем каталоге `/etc/namedb`, где находятся записи, относящиеся к этой зоне. Для зоны «diverge.org» используется файл `/etc/namedb/diverge.org`.

26.2.2. `/etc/namedb/localhost`

По большей части файлы зон выглядят довольно похоже, однако у каждого есть свои особенности. Вот как выглядит файл `localhost`:

Пример 26.2. localhost

```
 1|$TTL    3600
 2|@              IN SOA  strider.diverge.org. root.diverge.org. (
 3|                        1       ; Serial
 4|                        8H      ; Refresh
 5|                        2H      ; Retry
 6|                        1W      ; Expire
 7|                        1D)     ; Minimum TTL
 8|               IN NS   localhost.
 9|localhost.     IN      A       127.0.0.1
10|               IN      AAAA    ::1
```

Разберём по строкам:

Строка 1:

Это время жизни для результатов поиска, определяющее, как долго другие серверы DNS будут кэшировать это значение, прежде чем отбросить его. Обычно это значение одинаково во всех файлах.

Строка 2:

Эта строка обычно одинакова во всех файлах зон, кроме `root.cache`. Она определяет так называемый заголовок «Start Of Authority» (SOA), содержащий основные сведения о зоне. Особый интерес в этой строке представляют «strider.diverge.org.» и «root.diverge.org.» (обратите внимание на завершающие точки!). Очевидно, одно из них — имя этого сервера, а другое — контакт для

этого сервера DNS; в большинстве случаев root выглядит несколько неопределённо, поэтому в контактных данных предпочтительно использовать обычную учётную запись электронной почты, заменив «@» на «.» (например, у меня было бы «jrf.diverge.org.»).

Строка 3:

Эта строка — порядковый номер, идентифицирующий «версию» набора данных зоны (файла). Порядковый номер следует увеличивать при каждом изменении файла; обычный формат — либо начать со значения «1» и увеличивать его при каждом изменении, либо использовать значение «YYYYMMDDNN», кодирующее в порядковом номере год (YYYY), месяц (MM), день (DD) и изменение в течение дня (NN).

Строка 4:

Это частота обновления сервера; в этом файле она задана как один раз в 8 часов.

Строка 5:

Частота повторных попыток.

Строка 6:

Истечение срока поиска.

Строка 7:

Минимальное время жизни.

Строка 8:

Это строка сервера имён, в которой ресурсная запись «NS» показывает, что «localhost» — единственный сервер DNS, выдающий данные для этой зоны (это «@», означающий имя зоны, использованное в файле `named.conf`, то есть «diverge.org»), — ну, «localhost».

Строка 9:

Это запись localhost, которая с помощью ресурсной записи «A» указывает, что имя «localhost» должно разрешаться в IP-адрес 127.0.0.1 для запросов IPv4 (которые специально запрашивают запись «A»).

Строка 10:

Эта строка — запись IPv6, которая возвращает ::1, когда кто-то запрашивает IPv6-адрес (специально запрашивая запись AAAA) для «localhost.».

26.2.3. `/etc/namedb/zone.127.0.0`

Это файл (или зона) обратного просмотра для разрешения особого IP-адреса 127.0.0.1 обратно в «localhost»:

```
1| $TTL    3600
2| @               IN SOA  strider.diverge.org. root.diverge.org. (
3|                         1       ; Serial
4|                         8H      ; Refresh
5|                         2H      ; Retry
6|                         1W      ; Expire
7|                         1D)     ; Minimum TTL
8|                  IN NS   localhost.
9| 1.0.0            IN PTR  localhost.
```

В этом файле все строки совпадают с файлом зоны localhost, за исключением строки 9 — это запись обратного просмотра (PTR). Используемая здесь зона снова «@», которая получила значение,

заданное в `named.conf`, то есть «127.in-addr.arpa». Это особый «домен», применяемый для обратного просмотра IP-адресов в имена хостов. Чтобы это работало, четыре байта IPv4-адреса записываются в обратном порядке и к ним добавляется домен «in-addr.arpa»: так, для разрешения IP-адреса «127.0.0.1» запрашивается запись PTR для «1.0.0.127.in-addr.arpa», что и определено в этой строке.

26.2.4. `/etc/namedb/diverge.org`

Этот файл зоны заполнен записями для всех наших хостов. Вот как он выглядит:

```
 1| $TTL    3600
 2| @              IN SOA  strider.diverge.org. root.diverge.org. (
 3|                         1       ; serial
 4|                         8H      ; refresh
 5|                         2H      ; retry
 6|                         1W      ; expire
 7|                         1D )    ; minimum seconds
 8|                  IN NS   strider.diverge.org.
 9|                  IN MX   10 strider.diverge.org.    ; primary mail server
10|                  IN MX   20 samwise.diverge.org.    ; secondary mail server
11| strider          IN A     192.168.1.1
12| samwise          IN A     192.168.1.2
13| www              IN CNAME samwise.diverge.org.
14| worm             IN A     192.168.1.3
```

Здесь много нового, поэтому рассмотрим только те строки, которые новые:

Строка 9

Эта строка показывает наш почтовый обменник (MX), в данном случае это «strider». Число перед «strider.diverge.org.» — номер приоритета: чем меньше число, тем выше приоритет. Наша конфигурация устроена так: если «strider» не сможет обработать почту, это сделает «samwise».

Строка 11

CNAME означает каноническое имя, то есть псевдоним существующего имени хоста, у которого должна быть запись A. Итак, мы задали псевдонимы для следующего:

```
www.diverge.org to samwise.diverge.org
```

Остальные записи — просто отображения IP-адреса в полное имя (записи A).

26.2.5. `/etc/namedb/1.168.192`

Этот файл зоны — обратный файл для всех записей хостов: он отображает IP-номера, которые мы используем в частной сети, обратно в имена хостов. Формат похож на вариант «localhost», с очевидным исключением: адреса другие из-за другой зоны, заданной в файле `named.conf`, а именно «0.168.192.in-addr.arpa»:

```
 1|$TTL    3600
 2|@              IN SOA  strider.diverge.org. root.diverge.org. (
 3|                       1       ; serial
 4|                       8H      ; refresh
 5|                       2H      ; retry
 6|                       1W      ; expire
 7|                       1D )    ; minimum seconds
 8|               IN NS   strider.diverge.org.
 9|1              IN PTR  strider.diverge.org.
```

```
10|2             IN PTR  samwise.diverge.org.
11|3             IN PTR  worm.diverge.org.
```

26.2.6. `/etc/namedb/root.cache`

Этот файл содержит список корневых серверов имён, к которым ваш сервер обращается, когда получает запросы за пределами своего домена, на которые не может ответить сам. Вот первые несколько строк файла корневой зоны:

```
;
;       This file holds the information on root name servers needed to
;       initialize cache of Internet domain name servers
;       (e.g. reference this file in the "cache  .  <file>"
;       configuration file of BIND domain name servers).
;
;       This file is made available by InterNIC
;       under anonymous FTP as
;           file                /domain/db.cache
;           on server           FTP.INTERNIC.NET
;       -OR-                    RS.INTERNIC.NET
;
;       last update:    Jan 29, 2004
;       related version of root zone:   2004012900
;
;
; formerly NS.INTERNIC.NET
;
.                        3600000  IN  NS    A.ROOT-SERVERS.NET.
A.ROOT-SERVERS.NET.      3600000      A     198.41.0.4
;
; formerly NS1.ISI.EDU
;
.                        3600000      NS    B.ROOT-SERVERS.NET.
B.ROOT-SERVERS.NET.      3600000      A     192.228.79.201
;
; formerly C.PSI.NET
;
.                        3600000      NS    C.ROOT-SERVERS.NET.
C.ROOT-SERVERS.NET.      3600000      A     192.33.4.12
;
...
```

Этот файл можно получить у ISC по адресу http://www.isc.org/, и обычно он входит в дистрибутив BIND. Файл `root.cache` включён в набор «etc» операционной системы NetBSD.

В этом разделе описаны важнейшие файлы и параметры сервера DNS. Дополнительные сведения см. в документации BIND в `/usr/src/dist/bind/doc/bog` и в named.conf(5).

26.3. Использование DNS

В этом разделе мы рассмотрим, как запустить DNS и настроить «strider», чтобы он использовал собственные службы DNS.

Настроить автоматический запуск named совсем просто. В `/etc/rc.conf` достаточно задать `named=yes`. Дополнительные параметры можно указать в `named_flags`; например, я предпочитаю использовать `-g nogroup -u nobody`, чтобы процесс «named» работал не от root. Возможно, вы также захотите задать `named_chrootdir=/var/chroot/named`, чтобы **bind** работал в chroot. При первом запуске это перенесёт файлы конфигурации и базу данных кэша в каталог chroot. (Примечание: если

вы так сделаете, добавляйте к путям **bind** в оставшейся части этого документа префикс `/var/chroot/named`.)

Помимо возможности запускать «named» при загрузке, им можно управлять командой **ndc**. Вкратце команда **ndc** может остановить, запустить или перезапустить процесс сервера named. Она умеет и многое другое. Перед использованием её нужно настроить для связи с процессом «named»; подробности о настройке каналов связи между «ndc» и процессом «named» см. на страницах руководства rndc(8) и named.conf(5).

Далее мы хотим, чтобы «strider» при поиске обращался к самому себе. Здесь два простых шага; сначала определим порядок разрешения. В такой небольшой сети, вероятно, у каждого хоста есть копия таблицы hosts, поэтому можно сначала использовать `/etc/hosts`, а затем DNS. Однако в более крупных сетях гораздо проще использовать DNS. В любом случае файл, в котором определяется порядок служб имён, используемых для разрешения, — `/etc/nsswitch.conf`. Вот часть типичного `nsswitch.conf`:

```
. . .
group_compat:   nis
hosts:          files dns
netgroup:       files [notfound=return] nis
. . .
```

Нас интересует строка «hosts». «files» означает, что система сначала использует файл `/etc/hosts` для преобразования IP в имя, а если не найдёт запись, обратится к DNS.

Следующий файл — `/etc/resolv.conf`, который служит для настройки поиска DNS («разрешения») на стороне клиента. Формат довольно понятен сам по себе, но мы всё равно его разберём:

```
domain diverge.org
search diverge.org
nameserver 192.168.1.1
```

Вкратце этот файл сообщает резолверу, что эта машина принадлежит домену «diverge.org»: это значит, что к запросам, содержащим только имя хоста без «.», добавляется этот домен, чтобы получить FQDN. Если такой поиск не удался, далее перебираются домены из строки «search». И наконец, строка «nameserver» задаёт IP-адреса одного или нескольких серверов DNS, которые следует использовать для разрешения запросов DNS.

Чтобы проверить наш сервер имён, можно использовать несколько команд, например:

```
# host sam
sam.diverge.org has address 192.168.1.2
```

Как видите, домен здесь был добавлен автоматически — использовалось значение из `/etc/resolv.conf`. Вот ещё один пример — вывод команды **host www.yahoo.com**:

```
$ host www.yahoo.com
www.yahoo.com is an alias for www.yahoo.akadns.net.
www.yahoo.akadns.net has address 68.142.226.38
www.yahoo.akadns.net has address 68.142.226.39
www.yahoo.akadns.net has address 68.142.226.46
www.yahoo.akadns.net has address 68.142.226.50
www.yahoo.akadns.net has address 68.142.226.51
www.yahoo.akadns.net has address 68.142.226.54
www.yahoo.akadns.net has address 68.142.226.55
www.yahoo.akadns.net has address 68.142.226.32
```

Другие команды для отладки DNS, помимо host(1), — это nslookup(8) и dig(1). Учтите, что ping(8) *не* подходит для отладки DNS, так как для поиска имени он будет использовать то, что настроено в `/etc/nsswitch.conf`.

На этом сервер настроен правильно. Настройка клиентских хостов проще: достаточно задать в `/etc/nsswitch.conf` и `/etc/resolv.conf` те же значения, что и на сервере.

26.4. Настройка сервера имён, работающего только как кэш

Сервер имён, работающий только как кэш, не имеет локальных зон: все получаемые запросы передаются корневым серверам, а ответы накапливаются в локальном кэше. В следующий раз ответ будет получен быстрее, потому что данные уже есть в кэше сервера. Поскольку сервер этого типа не обслуживает локальные зоны, для разрешения имён локальных хостов по-прежнему придётся использовать уже знакомый файл `/etc/hosts`.

Поскольку NetBSD предоставляет значения по умолчанию для всех файлов, нужных серверу, работающему только как кэш, его достаточно включить и запустить — и он сразу готов к работе! Чтобы включить named, добавьте `named=yes` в `/etc/rc.conf` и укажите системе использовать его, добавив следующую строку в файл `/etc/resolv.conf`:

```
# cat /etc/resolv.conf
nameserver 127.0.0.1
```

Теперь можно запустить named:

```
# service named restart
```

26.4.1. Проверка сервера

Теперь, когда сервер работает, можно проверить его с помощью программы nslookup(8):

```
$ nslookup
Default server: localhost
Address: 127.0.0.1

>
```

Попробуем разрешить имя хоста, например «www.NetBSD.org»:

```
> www.NetBSD.org
Server:  localhost
Address:  127.0.0.1

Name:    www.NetBSD.org
Address:  204.152.190.12
```

Если повторить запрос во второй раз, результат будет немного другим:

```
> www.NetBSD.org
Server:  localhost
Address:  127.0.0.1

Non-authoritative answer:
```

```
Name:    www.NetBSD.org
Address:  204.152.190.12
```

Как вы, вероятно, заметили, адрес тот же, но появилось сообщение «Non-authoritative answer». Это сообщение означает, что ответ пришёл не от авторитетного сервера домена NetBSD.org, а из кэша нашего собственного сервера.

Результаты этой первой проверки подтверждают, что сервер работает правильно.

Можно также попробовать команды host(1) и dig(1), которые дают следующий результат.

```
$ host www.NetBSD.org
www.NetBSD.org has address 204.152.190.12
$
$ dig www.NetBSD.org

; <<>> DiG 8.3 <<>> www.NetBSD.org
;; res options: init recurs defnam dnsrch
;; got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19409
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 5, ADDITIONAL: 0
;; QUERY SECTION:
;;      www.NetBSD.org, type = A, class = IN

;; ANSWER SECTION:
www.NetBSD.org.         23h32m54s IN A  204.152.190.12

;; AUTHORITY SECTION:
NetBSD.org.             23h32m54s IN NS  uucp-gw-1.pa.dec.com.
NetBSD.org.             23h32m54s IN NS  uucp-gw-2.pa.dec.com.
NetBSD.org.             23h32m54s IN NS  ns.NetBSD.org.
NetBSD.org.             23h32m54s IN NS  adns1.berkeley.edu.
NetBSD.org.             23h32m54s IN NS  adns2.berkeley.edu.

;; Total query time: 14 msec
;; FROM: miyu to SERVER: 127.0.0.1
;; WHEN: Thu Nov 25 22:59:36 2004
;; MSG SIZE  sent: 32  rcvd: 175
```

Как видите, dig(1) выдаёт довольно много данных, и ожидаемый ответ можно найти в «ANSWER SECTION». Остальные данные могут оказаться полезны при отладке проблем с DNS.

Глава 27. Почта и новости

В этой главе объясняется, как настроить NetBSD для работы с почтой и новостями. Описывается только простая, но очень распространённая конфигурация: настройка хоста, подключённого к Интернету через модем и провайдера. Эту главу можно считать продолжением главы 24, *Настройка TCP/IP в NetBSD на практике* при аналогичной конфигурации сети. Даже такая «простая» настройка оказывается трудной, если вы не знаете, с чего начать, или читали только вводную либо техническую документацию. Общее описание настройки почты и новостей выходит за рамки этого руководства; прочитайте хорошую книгу по администрированию Unix (некоторые очень хорошие книги перечислены на сайте NetBSD).

В этой главе также кратко описана настройка (но не использование) двух популярных приложений: mutt для почты и tin для новостей. Использование не описывается, потому что эти программы просты в работе и хорошо документированы. Очевидно, что mutt и tin — не обязательный выбор: существует множество других подобных приложений, но, на мой взгляд, они хорошая отправная точка, поскольку широко используются, просты, хорошо работают и не занимают слишком много места на диске и памяти. Обе программы работают в консольном режиме; если вы предпочитаете графические приложения, для X также есть большой выбор.

Коротко говоря, для описанной в этой главе настройки требуются программы:

- postfix
- fetchmail
- mutt
- tin

С базовой системой устанавливается только postfix; остальные программы можно установить из коллекции пакетов NetBSD — pkgsrc.

> **ПРИМЕЧАНИЕ**
>
> Поскольку sendmail широко распространён и несколько программ, таких как fetchmail, рассчитаны на работу с ним, postfix включает обёртку командной строки, которая принимает синтаксис командной строки sendmail, но работает с postfix. Дополнительные сведения см. в sendmail(1).

Прежде чем продолжить, помните, что ни одна из программ, представленных в этой главе, не обязательна: есть другие приложения, выполняющие похожие задачи, и многие пользователи предпочитают их. Читая списки рассылки, вы встретите разные мнения. Можно использовать и другие схемы отправки и получения почты: описанная здесь — лишь отправная точка; разобравшись, как она работает, вы, вероятно, захотите изменить её под свои нужды или выбрать совсем другой способ.

Прямо противоположный пример — использование такого приложения, как Mozilla: оно делает всё и избавляет вас от необходимости настраивать множество компонентов. С Mozilla можно просматривать Интернет, отправлять и получать почту и читать новости. К тому же настройка очень проста. Однако за это приходится платить: Mozilla — «закрытая» программа, которая не будет легко взаимодействовать с другими стандартными утилитами Unix.

Другая возможность — использовать emacs для чтения почты и новостей. Emacs не нуждается в представлении пользователям Unix, но на случай, если вы не знаете: это расширяемый редактор (хотя называть emacs редактором несколько уничижительно), который превращается в полноценную рабочую среду и позволяет читать почту и новости, а также выполнять множество операций. Для многих людей emacs — единственная нужная им среда, и они используют её для всей своей работы. Настройка emacs для почты и новостей описана в руководстве по emacs.

В остальной части главы речь пойдёт о хосте, подключённом к Интернету по PPP-соединению через последовательный модем к провайдеру.

- имя локального хоста — «ape», а внутренняя сеть — «insetti.net», то есть FQDN (Fully Qualified Domain Name) — «ape.insetti.net».
- имя пользователя для входа на ape — «carlo».
- провайдер называется BigNet.
- почтовый сервер провайдера — «mail.bignet.it».
- сервер новостей провайдера — «news.bignet.it».
- учётная запись пользователя («carlo») у провайдера — «alan» с паролем «pZY9o».

Сначала немного основной терминологии:

MUA (агент пользователя почты)

программа для чтения и написания почты. Например: mutt, elm и pine, а также простая программа mail, устанавливаемая с базовой системой.

MTA (агент передачи почты)

программа, передающая почту между двумя хостами, а также локально (на одном хосте). MTA определяет путь, по которому почта будет следовать к месту назначения. В других системах BSD (и не только) стандартный MTA — sendmail; другие примеры — qmail, exim и Microsoft Exchange.

MDA (агент доставки почты)

программа, обычно используемая MTA, которая доставляет почту; например, она физически помещает сообщения в почтовый ящик получателя. Так, postfix использует один или несколько MDA для доставки почты, а procmail — ещё один известный MDA.

рисунок 27.1, «Структура почтовой системы» изображает почтовую систему, которую мы хотим настроить. Между локальной сетью (или единственным хостом) и провайдером есть модемное PPP-соединение. «Пузыри» с толстой рамкой (postfix, fetchmail, mutt) — это программы, запускаемые пользователем вручную; остальные пузыри — программы, запускаемые автоматически. Числа в кружках обозначают логические шаги почтового цикла:

1. На шаге (1) почта загружается с сервера POP провайдера с помощью fetchmail, который передаёт сообщения обёртке sendmail из postfix, чтобы та поместила сообщения в почтовый ящик пользователя.
2. На шаге (2) пользователь запускает mutt (или другой MUA), чтобы читать почту, отвечать и писать новые сообщения.
3. На шаге (3) пользователь «отправляет» почту из mutt. Сообщения накапливаются в области spool.
4. На шаге (4) пользователь вызывает обёртку sendmail из postfix, чтобы передать сообщения на сервер SMTP провайдера, который доставит их к конечному адресату (возможно, через другие почтовые серверы). Сервер SMTP провайдера действует как *ретранслятор* нашей почты.

Соединение с провайдером должно быть установлено только на шагах (1) и (4); на остальных шагах оно не нужно.

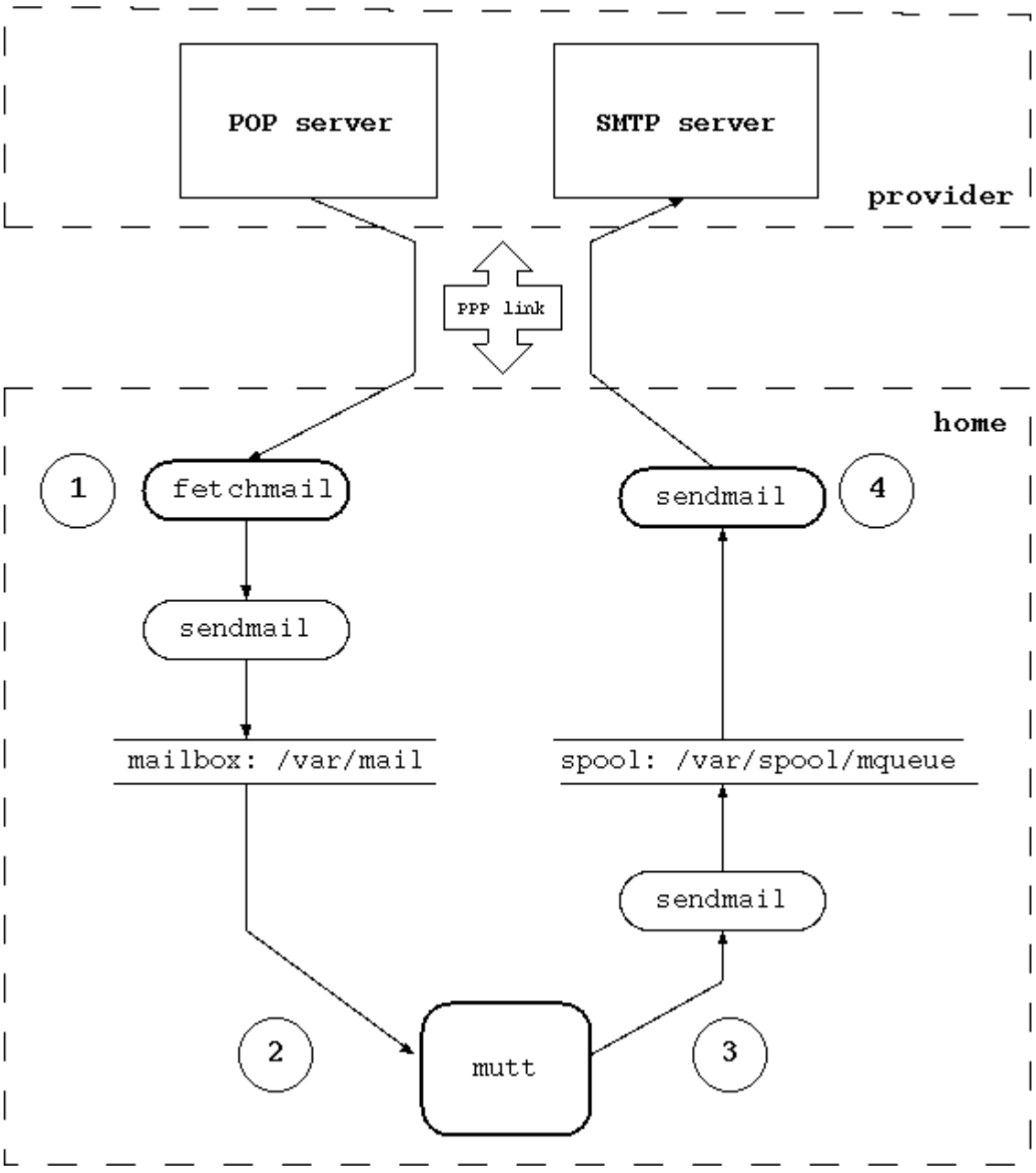


Рисунок 27.1. Структура почтовой системы

27.1. postfix

Когда MTA должен доставить локальное сообщение, оно доставляется напрямую. Если сообщение предназначено другому домену, MTA должен выяснить адрес почтового сервера для этого домена. Postfix использует службу DNS (описанную в главе 26, *Система доменных имён*), чтобы найти почтовый обменник, обслуживающий почту для данного домена, и затем доставляет сообщение на этот почтовый сервер.

Postfix управляется набором файлов конфигурации и баз данных, важнейшие из которых — `/etc/postfix/main.cf` и `/etc/postfix/master.cf`.

Первая задача, которую нужно решить, — локальная сеть, с которой мы имеем дело, является внутренней, то есть недоступна напрямую из Интернета. Это значит, что используемые внутри имена ничего не значат в Интернете; короче говоря, «ape.insetti.net» недоступен для внешнего хоста: никто не сможет ответить на письмо, отправленное с таким обратным адресом (многие почтовые системы даже отвергнут сообщение в порядке борьбы со спамом, так как оно пришло с неизвестного хоста). Настоящий адрес, видимый всем, назначается провайдером, и поэтому необходимо преобразовать локальный адрес «carlo@ape.insetti.net» в реальный адрес «alan@bignet.it». Postfix, если он правильно настроен, сделает это при передаче сообщений.

Вероятно, вы также захотите настроить postfix так, чтобы отправлять письма на почтовый сервер провайдера, используя его как *ретранслятор*. В конфигурации, описанной в этой главе, postfix не обращается напрямую к почтовому серверу получателя (как описано выше), а ретранслирует всю свою почту на почтовый сервер провайдера.

> **ПРИМЕЧАНИЕ**
>
> Почтовый сервер провайдера действует как *ретранслятор*: он доставляет почту, не предназначенную для его собственного домена, на другой почтовый сервер. Он выступает посредником между двумя серверами.

Поскольку соединение с провайдером активно не всегда, запускать postfix как демон в `/etc/rc.conf` не обязательно: его можно отключить строкой «`postfix=NO`». В результате postfix придётся запускать вручную, когда нужно передать почту провайдеру. Локальная почта доставляется правильно, даже если postfix не работает как демон.

Начнём настройку postfix.

27.1.1. Настройка обобщённого отображения

Этот тип конфигурации использует новый файл `/etc/postfix/generic`, содержащий отображение имён хостов, которое postfix применяет для перезаписи внутренних имён хостов.

Итак, первый шаг — написать файл отображения:

```
carlo@ape.insetti.net                   alan@bignet.it
root@ape.insetti.net                    alan@bignet.it
news@ape.insetti.net                    alan@bignet.it
```

Эти записи отображают почту, отправленную пользователями, указанными слева, в глобально действительные адреса электронной почты, указанные справа, так что почта выглядит отправленной именно с этого адреса.

Для эффективности `generic` нужно преобразовать в двоичный файл следующей командой:

```
# postmap /etc/postfix/generic
```

Теперь создадим файл конфигурации-прототип, из которого мы получим файл конфигурации postfix.

```
# vi /etc/postfix/main.cf
```

Для простоты покажем только те переменные, которые нужно изменить:

```
relayhost = mail.bignet.it
smtp_generic_maps = hash:/etc/postfix/generic
```

Эта конфигурация указывает postfix перезаписывать адреса вида «ape.insetti.net», используя реальные имена из файла `/etc/postfix/generic`. Она также указывает, что почту следует отправлять на сервер «mail.bignet.it». Значение параметров подробно описано в postconf(5).

Последний шаг — перезагрузить конфигурацию. Это легко сделать так:

```
# service postfix reload
postfix/postfix-script: refreshing the Postfix mail system
```

Теперь всё готово к отправке почты.

27.1.2. Проверка конфигурации

Наконец postfix настроен и готов к работе, но перед отправкой настоящей почты лучше выполнить несколько простых проверок. Сначала попробуем отправить локальное письмо следующей командой (обёртка sendmail из postfix):

```
$ sendmail carlo
Subject: test

Hello world
.
```

Точно следуйте приведённому примеру: оставьте пустую строку после Subject: и завершите сообщение строкой, содержащей только одну точку. Теперь вы сможете прочитать сообщение в своём почтовом клиенте и убедиться, что поле From: правильно перезаписано.

```
From: alan@bignet.it
```

27.1.3. Использование альтернативного MTA

```
$ ls -l /usr/sbin/sendmail
lrwxr-xr-x  1 root  wheel  21 Nov  1 01:14 /usr/sbin/sendmail@ -> /usr/sbin/mailwrapper
```

Назначение mailwrapper — позволить использовать альтернативный MTA вместо postfix (например, sendmail). Если вы планируете использовать другой почтовый агент, советую прочитать страницы руководства mailwrapper(8) и mailer.conf(5), которые написаны очень понятно.

27.2. fetchmail

Если кто-то отправляет мне почту, она принимается и хранится провайдером и не передаётся автоматически на локальные хосты; поэтому её нужно загрузить. Fetchmail — очень популярная программа, которая загружает почту с удалённого почтового сервера (используя, например, протокол Post Office Protocol, POP) и передаёт её локальной системе для доставки (обычно через обёртку sendmail из postfix). Она мощная, но простая в использовании и настройке: после установки нужно создать файл `~/.fetchmailrc`, и программа готова к работе (файл `~/.fetchmailrc` содержит пароль, поэтому требуются соответствующие права доступа к нему).

Вот пример `.fetchmailrc`:

```
poll mail.bignet.it
protocol POP3
username alan there with password pZY9o is carlo here
flush
mda "/usr/sbin/sendmail -oem %T"
```

Последняя строка («mda ...») используется только в том случае, если postfix не работает в системе как демон. Учтите, что указанный в этом файле почтовый сервер POP (mail.bignet.it) служит только для получения почты и не обязательно совпадает с почтовым ретранслятором, который postfix использует для отправки почты.

После настройки файла `.fetchmailrc` для загрузки и доставки почты в локальную систему можно использовать следующую команду:

```
$ fetchmail
```

Теперь сообщения можно прочитать в mutt.

27.3. Чтение и написание почты в mutt

Mutt — одна из самых популярных почтовых программ: она «лёгкая», проста в использовании и имеет множество возможностей. Страница руководства mutt очень скупа; настоящая документация находится в `/usr/pkg/share/doc/mutt/`, в частности `manual.txt`.

Конфигурация mutt задаётся файлом `~/.muttrc`. Проще всего создать его, скопировав пример файла muttrc из mutt (обычно `/usr/pkg/share/examples/mutt/sample.muttrc`) в домашний каталог и изменив его. Следующий пример показывает, как добиться некоторых результатов:

- Сохранять копию отправленной почты.
- Определить каталог и два файла для входящей и исходящей почты, сохраняемой mutt (в этом примере каталог — `~/Mail`, а файлы — `incoming` и `outgoing`).
- Определить несколько цветов.
- Определить псевдоним.

```
set copy=yes
set edit_headers
set folder="~/Mail"
unset force_name
set mbox="~/Mail/incoming"
set record="~/Mail/outgoing"
unset save_name

bind pager <up> previous-page
bind pager <down> next-page

color normal white black
color hdrdefault blue black
color indicator white blue
color markers red black
color quoted cyan black
color status white blue
color error red white
color underline yellow black
```

```
mono quoted standout
mono hdrdefault underline
mono indicator underline
mono status bold

alias pippo Pippo Verdi <pippo.verdi@pluto.net>
```

Чтобы запустить mutt:

```
$ mutt
```

Учтите, что mutt поддерживает цвет, но это зависит от настроек терминала. В X можно использовать «xterm-color», например:

```
$ env TERM=xterm-color mutt
```

27.4. Стратегия получения почты

В этом разделе описан простой способ получения и чтения почты. Соединение с провайдером устанавливается только на время, необходимое для загрузки сообщений; затем почта читается в автономном режиме.

1. Установите соединение с провайдером.
2. Запустите **fetchmail**.
3. Отключите соединение.
4. Прочитайте почту в mutt.

27.5. Стратегия отправки почты

Когда письмо написано и «отправлено» в mutt, сообщения нужно передать провайдеру с помощью postfix. Почта отправляется из mutt командой **y**, но это на самом деле её не отправляет: сообщения ставятся в очередь в области spool; если postfix не работает как демон, его нужно запустить вручную, иначе сообщения останутся на жёстком диске. Необходимые шаги:

1. Напишите письмо в mutt, отправьте его и выйдите из mutt. Проверить, есть ли сообщения в очереди почты postfix и какие именно, можно программой mailq(1).
2. Установите соединение с провайдером.
3. Если ваш провайдер требует «SMTP-after-POP», то есть сначала хочет убедиться, кто вы, прежде чем разрешить отправку почты (и избежать спама), сначала нужно снова запустить **fetchmail**.
4. Введите команду **/usr/sbin/postfix flush**, чтобы передать сообщения из очереди провайдеру.
5. Отключите соединение, когда очередь опустеет.

27.6. Дополнительные средства работы с почтой

Когда вы только начинаете пользоваться почтой, у вас, вероятно, нет очень сложных требований, и уже описанной стандартной конфигурации хватит для всех нужд. Но у многих пользователей число ежедневных сообщений со временем растёт, и становится необходимой более рациональная организация хранения почты — например, разделение почты на разные почтовые ящики по темам.

Если, скажем, вы подписаны на список рассылки, вы, скорее всего, будете получать много сообщений каждый день и захотите хранить их отдельно от остальной почты. Вскоре вы обнаружите, что тратите слишком много времени каждый день, повторяя одни и те же ручные операции по упорядочиванию почтовых ящиков.

Зачем повторять одни и те же операции вручную, если можно поручить их программе? Существует множество инструментов, которые можно добавить в свою почтовую систему, чтобы повысить её гибкость и автоматически обрабатывать сообщения. Среди самых известных и используемых:

- procmail — развитый агент доставки почты и универсальный почтовый фильтр для локальной почты, который автоматически обрабатывает входящую почту по заданным пользователем наборам правил. Он легко интегрируется с sendmail/postfix.
- spamassassin или spamprobe — для борьбы со спамом.
- metamail — средство для обработки вложений.
- formail — форматировщик почты.

В оставшейся части раздела будет приведён пример конфигурации procmail для очень распространённого случая: автоматическая доставка в заданный пользователем почтовый ящик всех сообщений, приходящих из списка рассылки. Конфигурация postfix будет изменена так, чтобы напрямую вызывать procmail (procmail станет *локальным почтовым агентом*, используемым sendmail), и будет создан собственный файл конфигурации procmail.

Сначала нужно установить procmail с помощью системы пакетов (`mail/procmail`) или **pkg_add**.

Затем нужно изменить конфигурацию postfix, чтобы использовать procmail как локальный почтовый агент:

```
mailbox_command = /usr/pkg/bin/procmail
```

Эта строка задаёт путь к программе procmail (узнать, куда установлен procmail, можно командой **which procmail**).

Последний шаг — создание файла конфигурации procmail, содержащего рецепты доставки почты.

Допустим, например, что вы подписались на список рассылки о розах, адрес которого — «roses@flowers.org», и что каждое сообщение из этого списка содержит в заголовке следующую строку:

```
Delivered-To: roses@flowers.org
```

Предполагая, что вы хотите автоматически раскладывать все письма, приходящие через этот список, в локальную папку почты «roses_list», файл конфигурации procmail (`.procmailrc`) выглядит так:

```
PATH=/bin:/usr/bin:/usr/pkg/bin
MAILDIR=$HOME/Mail
LOGFILE=$MAILDIR/from

:0
* ^Delivered-To: roses@flowers.org
roses_list
```

Предыдущий файл содержит только одно правило, начинающееся со строки, содержащей «:0». Следующая строка определяет все сообщения, содержащие строку «Delivered-To: roses@flowers.org», а последняя строка говорит, что выбранные сообщения должны попасть в почтовый ящик `roses_list`

(который вам следует создать в $MAILDIR). Остальные сообщения будут доставлены в почтовый ящик по умолчанию. Учтите, что $MAILDIR — тот же каталог, который вы настроили в mutt:

```
set folder="~/Mail"
```

Разумеется, список рассылки — лишь пример; procmail — очень универсальное средство, позволяющее фильтровать почту по множеству критериев. Как обычно, за подробностями обращайтесь к страницам руководства: procmail(1), procmailrc(5) и procmailex(5) (последняя содержит много примеров файлов конфигурации).

27.7. Новости с tin

Слово *новости* (news) обозначает набор сообщений, публикуемых в группах новостей USENET — службе, доступной в Интернете. Каждая группа новостей содержит статьи на определённую тему. Читать группу новостей — не то же самое, что читать список рассылки: подписавшись на список рассылки, вы получаете статьи по почте и читаете их обычной почтовой программой вроде mutt, которую используете и для ответов. Новости же читаются напрямую с сервера новостей специальной программой — *программой чтения новостей*, например tin. С помощью tin можно подписаться на интересующие группы и следить за *потоками* (threads). Поток — это последовательность статей, которые все происходят от одной статьи, которую можно назвать «исходной». Короче говоря, сообщение отправляется в группу, кто-то отвечает, другие отвечают тем, кто ответил первым, и так далее, образуя древовидную структуру сообщений и ответов: без программы чтения новостей понять правильную последовательность сообщений невозможно.

После установки tin (как обычно, из коллекции пакетов) остаётся только вписать имя сервера NNTP в файл `/usr/pkg/etc/nntp/server`, который, возможно, придётся сначала создать. Например:

```
news.bignet.it
```

После этого программу можно запустить командой **tin**. На экране появится примерно следующее:

```
$ tin
Connecting to news.bignet.it...
news.bignet.it InterNetNews NNRP server INN 1.7.2 08-Dec-1997 ready (posting ok).
Reading groups from active file...
Checking for new groups...
Reading attributes file...
Reading newsgroups file...
Creating newsrc file...
Autosubscribing groups...
Reading newsrc file...
```

Наберитесь терпения при первом подключении: tin загружает огромный список групп новостей, на которые можно подписаться, и это занимает несколько минут. Когда загрузка закончится, откроется главный экран программы; обычно группы не отображаются; чтобы увидеть список групп, нажмите **y**. Чтобы подписаться на группу, перейдите на её имя и нажмите **y**.

После того как вы подписались на несколько групп новостей, tin можно запускать быстрее командой **tin -Q**. Поиск новых групп отключён (`-q`), ищутся только активные группы (`-n`), а описания групп новостей не загружаются (`-d`): при этом в tin нельзя будет использовать команду **y** (yank). Когда tin запущен с этим параметром, он не может определить, модерируется группа новостей или нет.

Учтите, что если вы подключаетесь из внутренней сети (как в нашем примере), то при отправке («публикации») сообщения адрес в его начале будет неверным (потому что это внутренний адрес). Чтобы решить эту проблему, используйте параметр «mail_address» в файле конфигурации tin (`~/.tin/tinrc`) или задайте переменную окружения REPLYTO.

Глава 28. Введение в протокол резервирования общих адресов (CARP)

О лицензии этой главы см. в разделе D.3.3, «Лицензия Joel Knight на статью о CARP».

CARP — это Common Address Redundancy Protocol (протокол резервирования общих адресов). Его основное назначение — позволить нескольким хостам в одном сегменте сети совместно использовать IP-адрес. CARP — безопасная и свободная альтернатива Virtual Router Redundancy Protocol и Hot Standby Router Protocol.

CARP работает, позволяя группе хостов в одном сегменте сети совместно использовать IP-адрес. Эта группа хостов называется «группой резервирования». Группе резервирования назначается IP-адрес, общий для всех её членов. Внутри группы один хост назначается основным (master), а остальные — резервными (backup). Основной хост — тот, который в текущий момент «владеет» общим IP-адресом; он отвечает на любой трафик и запросы ARP, направленные к этому адресу. Каждый хост может одновременно входить более чем в одну группу резервирования.

Одно из распространённых применений CARP — создание группы резервирующих межсетевых экранов. Виртуальный IP-адрес, назначенный группе резервирования, настраивается на клиентских машинах как шлюз по умолчанию. Если основной межсетевой экран выйдет из строя или будет выведен из работы, IP-адрес перейдёт к одному из резервных межсетевых экранов, и обслуживание продолжится без перебоев.

Хотя высокорезервированное и отказоустойчивое оборудование уменьшает потребность в CARP, оно её не устраняет. Никакая аппаратная отказоустойчивость не поможет, если кто-то выдернет шнур питания или если ваш системный администратор введёт reboot не в том окне. CARP также позволяет сделать цикл «патч — перезагрузка» незаметным для пользователей и упрощает проверку обновления программного или аппаратного обеспечения: если обновление не сработает, вы можете вернуться к резервной системе, пока не устраните проблему.

Однако есть ситуации, в которых CARP не поможет. Архитектура CARP требует, чтобы члены группы находились в одной физической подсети и имели статический IP-адрес, хотя с появлением директивы carpdev необходимость в IP-адресах на физических интерфейсах отпала. Точно так же службы, требующие постоянного соединения с сервером (например, SSH или IRC), не будут прозрачно перенесены на другую систему — хотя в этом случае CARP помогает свести простой к минимуму. Сам по себе CARP не синхронизирует данные между приложениями: например, он не будет вручную дублировать данные между машинами с помощью rsync или другого подходящего для вашего приложения средства.

CARP поддерживает как IPv4, так и IPv6.

28.1. Работа CARP

Главный хост группы регулярно отправляет в локальную сеть объявления, чтобы резервные хосты знали, что он ещё работает. Если резервные хосты в течение заданного периода времени не получают объявления от главного, одна из них берёт на себя обязанности главного (тот резервный хост, у которого настроены наименьшие значения advbase и advskew). На одном сегменте сети может существовать несколько групп CARP. Объявления CARP содержат идентификатор виртуального хоста

(Virtual Host ID), который позволяет членам группы определить, к какой группе резервирования относится объявление.

Чтобы злоумышленник в сегменте сети не мог подделать объявления CARP, для каждой группы можно задать пароль. После этого каждый пакет CARP, отправляемый в группу, защищается с помощью SHA1 HMAC.

28.2. Настройка CARP

Каждая группа резервирования представлена виртуальным сетевым интерфейсом carp(4). Соответственно, CARP настраивается с помощью ifconfig(8). Доступны следующие параметры:

`carpN`
: Имя виртуального интерфейса carp(4), где N — целое число, обозначающее номер интерфейса (например, carp0).

`vhid`
: Идентификатор виртуального хоста (Virtual Host ID). Это уникальное число, используемое для того, чтобы другие узлы сети могли определить группу резервирования. Допустимые значения — от 1 до 255. Это позволяет нескольким группам резервирования существовать в одной сети.

`password`
: Пароль аутентификации, используемый при обмене данными с другими хостами с поддержкой CARP в этой группе резервирования. Он должен быть одинаковым на всех членах группы резервирования.

`carpdev`
: Этот необязательный параметр задаёт физический сетевой интерфейс, принадлежащий данной группе резервирования. По умолчанию CARP пытается определить, какой интерфейс использовать, отыскивая физический интерфейс в той же подсети, что и комбинация ipaddress и mask, заданная для интерфейса carp(4).

`advbase`
: Этот необязательный параметр задаёт, как часто (в секундах) объявлять о том, что мы являемся членом группы резервирования. Значение по умолчанию — 1 секунда. Допустимые значения — от 1 до 255.

`advskew`
: Этот необязательный параметр задаёт величину смещения advbase при отправке объявлений CARP. Манипулируя advbase, можно выбирать главный хост CARP. Чем больше число, тем менее предпочтителен хост при выборе главного. Значение по умолчанию — 0. Допустимые значения — от 1 до 254.

`state`
: Переводит интерфейс carp(4) в определённое состояние. Допустимые состояния: init, backup и master

`ipaddress`
: Это общий IP-адрес, назначенный группе резервирования. Этот адрес не обязан находиться в той же подсети, что и IP-адрес на физическом интерфейсе (если он есть). Однако он должен быть одинаковым на всех хостах группы.

`mask`

Маска подсети общего IP-адреса.

Другими аспектами поведения CARP можно управлять с помощью sysctl(8)

`net.inet.carp.allow`

Принимать ли входящие пакеты CARP. По умолчанию — 1 (да).

`net.inet.carp.preempt`

Разрешает хостам в группе резервирования с лучшими значениями advbase и advskew вытеснять главный хост. Кроме того, этот параметр включает переключение всех интерфейсов при выходе из строя одного из них. Если один физический интерфейс с поддержкой CARP выходит из строя, CARP изменит advskew на 240 на всех остальных интерфейсах с поддержкой CARP, по сути выполнив переключение на резерв. По умолчанию этот параметр равен 0 (отключено).

`net.inet.carp.log`

Записывать в журнал некорректные пакеты CARP. По умолчанию — 0 (отключено).

`net.inet.carp.arpbalance`

Балансировать трафик между несколькими хостами группы резервирования. По умолчанию — 0 (отключено). Дополнительные сведения см. в carp(4).

28.3. Включение поддержки CARP

Поддержка CARP по умолчанию не включена.

Чтобы использовать carp(4), нужно ядро с поддержкой псевдоустройства `carp`. Убедитесь, что в файле конфигурации ядра есть следующая строка:

```
pseudo-device   carp            # CARP
```

После настройки псевдоустройства `carp` в конфигурации ядра необходимо пересобрать ядро и перезагрузиться, чтобы включить поддержку carp(4).

28.4. Пример CARP

Пример конфигурации CARP:

```
# sysctl -w net.inet.carp.allow=1
# ifconfig carp0 create
# ifconfig carp0 vhid 1 pass lanpasswd \
     carpdev em0 advskew 100 10.0.0.1 255.255.255.0
```

При этом настраивается следующее:

- Разрешает приём пакетов CARP (это настройка по умолчанию)
- Создаёт интерфейс carp(4).
- Настраивает carp0 для виртуального хоста №1, включает пароль (lanpasswd), назначает em0 интерфейсом, принадлежащим группе, и делает этот хост резервным благодаря значению advskew, равному 100 (при условии, конечно, что на главном хосте advskew меньше 100). Общий IP-адрес, назначенный этой группе, — 10.0.0.1/255.255.255.0.

Выполнение ifconfig для carp0 показывает состояние интерфейса:

```
# ifconfig carp0
carp0: flags=8802<UP,BROADCAST,SIMPLEX,MULTICAST> mtu 1500
     carp: BACKUP carpdev em0 vhid 1 advbase 1 advskew 100
     inet 10.0.0.1 netmask 0xffffff00 broadcast 10.0.0.255
```

28.5. Расширенная настройка CARP

В следующем примере создаётся кластер из двух высокодоступных резервируемых межсетевых экранов. Приведённая ниже схема показывает, чего мы хотим добиться:

```
         +----| WAN/Internet |----+
         |                        |
      em1|                        |em1
      +-----+                  +-----+
      | fw1 |                  | fw2 |
      +-----+                  +-----+
      em0|                        |em0
         |                        |
      ---+-------Shared LAN-------+---
```

Оба межсетевых экрана подключены к локальной сети через em0, а к сети WAN/Интернет — через em1. IP-адреса приведены ниже:

- Межсетевой экран 1 (fw1), em0: 172.16.0.1
- Межсетевой экран 1 (fw1), em1: 192.0.2.1
- Межсетевой экран 2 (fw2), em0: 172.16.0.2
- Межсетевой экран 2 (fw2), em1: 192.0.2.2

IP-адреса, которые должны быть общими для групп резервирования:

- Общий IP-адрес WAN/Интернет: 192.0.2.100
- Общий IP-адрес LAN: 172.16.0.100

Согласно принятой сетевой политике предпочтительным главным хостом будет межсетевой экран 1 (fw1).

Следующая конфигурация предназначена для межсетевого экрана 1 (fw1):

```
#Enable preemption and group interface failover
# sysctl -w net.inet.carp.preempt=1
```

```
#Configure CARP on the LAN side
# ifconfig carp0 create
# ifconfig carp0 vhid 1 pass lanpasswd carpdev em0 \
     172.16.0.100 255.255.255.0
```

```
#Configure CARP on the WAN side
# ifconfig carp1 create
# ifconfig carp1 vhid 2 pass wanpasswd carpdev em1 \
```

```
     192.0.2.100 255.255.255.0
```

Как отмечалось ранее, согласно нашей политике предпочтительным главным хостом должен быть межсетевой экран 1. При настройке межсетевого экрана 2 мы задаём для `advskew` большее значение, так как он менее предпочтителен в роли главного.

Следующая конфигурация предназначена для межсетевого экрана 2 (fw2):

```
#Enable preemption and group interface failover
# sysctl -w net.inet.carp.preempt=1
```

```
#Configure CARP on the LAN side
# ifconfig carp0 create
# ifconfig carp0 vhid 1 pass lanpasswd carpdev em0 \
     advskew 128 172.16.0.100 255.255.255.0
```

```
#Configure CARP on the WAN side
# ifconfig carp1 create
# ifconfig carp1 vhid 2 pass wanpasswd carpdev em1 \
     advskew 128 192.0.2.100 255.255.255.0
```

28.6. Принудительное переключение главного хоста

Иногда возникает необходимость намеренно переключить или понизить в статусе главный узел. Примеры: остановка главного узла для обслуживания или при поиске неисправности. Цель в данном случае — корректно переключить трафик на один из резервных хостов, чтобы пользователи не заметили последствий.

Чтобы выполнить переключение, отключите интерфейс carp(4) на главном узле. Из-за этого главный узел начнёт объявлять себя с «бесконечными» advbase и advskew. Резервные хосты увидят это и немедленно возьмут на себя роль главного.

```
# ifconfig carp0 down
```

Глава 29. Сетевые службы

29.1. *Сетевая файловая система* (NFS)

Теперь, когда сеть работает, можно совместно использовать файлы и каталоги по сети с помощью сетевой файловой системы (NFS). С точки зрения совместного использования файлов компьютер, предоставляющий доступ к своим файлам и каталогам, называется *сервером*, а компьютер, использующий эти файлы и каталоги, — *клиентом*. Компьютер может быть одновременно и клиентом, и сервером.

Предупреждение: в NFS нет аутентификации, а управление доступом осуществляется только по сетевому адресу. NFS следует развёртывать только в ограниченных или защищённых межсетевым экраном сетях (например, с npf(7); см. Раздел 24.5, «Настройка интернет-шлюза с помощью NPF»). Развёртывать её в открытом интернете небезопасно.

- На сервере необходимо включить демоны rpcbind(8), mountd(8), lockd(8) и statd(8), а также параметр `nfs_server` (который включает nfsd(8)) в файле `/etc/rc.conf`:

```
rpcbind=yes
mountd=yes
nfs_server=yes
lockd=yes
statd=yes
```

- На клиенте необходимо включить демоны rpcbind(8), lockd(8) и statd(8), а также параметр `nfs_client` в файле `/etc/rc.conf`:

```
rpcbind=yes
nfs_client=yes
lockd=yes
statd=yes
```

- Сервер должен перечислить экспортируемые файловые системы в файле `/etc/exports` (см. exports(5)), а затем выполнить команду **service mountd reload**.

  Предупреждение: экспортировать можно только файловые системы *целиком*, а не их поддеревья. Вы можете экспортировать подкаталоги файловой системы как возможные точки монтирования для клиентов, но это откроет клиентам *всё* содержимое файловой системы, в которой они находятся. Если вы хотите экспортировать одно поддерево и запретить доступ к другим поддеревьям, экспортируемое поддерево должно находиться в собственной файловой системе.

Клиентский хост может получить доступ к удалённой файловой системе через NFS, если:

- Серверный хост экспортирует клиенту точку монтирования в файловой системе. Список точек монтирования, экспортируемых сервером NFS, можно посмотреть командой **showmount -e** (см. showmount(8)). Например:

```
# showmount -e 192.168.1.2
Exports list on 192.168.1.2:
```

```
/home                           host1 host2 host3
```

- Клиентский хост монтирует удалённый каталог с помощью fstab(5) или командой **mount 192.168.1.2:/home /home** (см. mount_nfs(8)).

29.1.1. Пример настройки NFS

Описываемый сценарий таков: пять клиентских машин (cli1, ..., cli5) совместно используют некоторые каталоги на сервере (buzz.toys.org). Часть каталогов, экспортируемых сервером, закреплена за конкретным клиентом, остальные каталоги общие для всех клиентских машин. Все клиенты загружаются с сервера и должны смонтировать эти каталоги.

С сервера экспортируются следующие каталоги:

`/export/cli`*X*`/root`
: пять корневых каталогов для пяти клиентских машин. У каждого клиента свой корневой каталог, где *X* = 1, 2, 3, 4 или 5.

`/export/cli`*X*`/swap`
: Пять каталогов подкачки для пяти машин.

`/export/common/usr`
: Каталог `/usr`; общий для всех клиентских хостов.

`/usr/src`
: Общий каталог `/usr/src` для всех клиентских машин.

На сервере существуют следующие файловые системы:

```
/dev/ra0a on /
/dev/ra0f on /usr
/dev/ra1a on /usr/src
/dev/ra2a on /export
```

Каждому клиенту нужны следующие файловые системы:

```
buzz:/export/cliX/root   on /
buzz:/export/common/usr  on /usr
buzz:/usr/src            on /usr/src
```

Конфигурация сервера такова:

```
# /etc/exports
/usr/src  -network 192.168.1.0 -mask 255.255.255.0
/export   -alldirs -maproot=root -network 192.168.1.0 -mask 255.255.255.0
```

Учтите, что все экспортируемые каталоги находятся в файловых системах, отличных от `/` и `/usr`, чтобы клиенты не могли перезаписать важные файлы, такие как `/bin/sh` и `/usr/bin/less`, или прочитать секретные файлы сервера, такие как `/root/.ssh/id_ed25519` и `/etc/krb5.keytab`.

На клиентских машинах файл `/etc/fstab` содержит:

```
buzz:/export/cliX/root  /        nfs rw
buzz:/export/common/usr /usr     nfs ro,nodev,nosuid
buzz:/usr/src           /usr/src nfs rw,nodev,nosuid
```

Каждая клиентская машина подставляет свой номер вместо символа «*X*» в первой строке предыдущего примера.

29.1.2. Настройка автоматического монтирования NFS для `/net` с помощью amd(8)

29.1.2.1. Введение

Проблема с монтированием NFS (и не только) заключается в том, что обычно для этого требуются права root, а это может быть довольно неудобно для пользователей. С помощью amd(8) можно настроить определённый каталог (обычно `/net`), в котором любой пользователь может выполнить любое NFS-монтирование, если файловая система, к которой требуется доступ, действительно экспортируется сервером NFS.

Чтобы проверить, экспортирует ли сервер файловую систему и какие именно, используйте команду **showmount** с ключом `-e` (export):

```
$ showmount -e wuarchive.wustl.edu
Exports list on wuarchive.wustl.edu:
/export/home                       onc.wustl.edu
/export/local                      onc.wustl.edu
/export/adm/log                    onc.wustl.edu
/usr                               onc.wustl.edu
/                                  onc.wustl.edu
/archive                           Everyone
```

Если затем вы хотите смонтировать каталог, чтобы получить доступ ко всему, что в нём находится (например, `/archive/systems/unix/NetBSD`), просто перейдите в этот каталог:

```
$ cd /net/wuarchive.wustl.edu/archive/systems/unix/NetBSD
```

Файловая система будет смонтирована (командой **amd**), и вы сможете обращаться к любым файлам так, как если бы каталог был смонтирован суперпользователем вашей системы.

29.1.2.2. Практическая настройка

Настроить такой каталог `/net` можно, выполнив следующие шаги (включая базовую настройку **amd**):

1. в файле `/etc/rc.conf` задайте следующую переменную:

   ```
   amd=yes
   ```

2. **mkdir /amd**

3. **mkdir /net**

4. Взяв за основу `/usr/share/examples/amd/amd.conf`, поместите следующее в `/etc/amd.conf`:

   ```
   [ /net ]
   map_name =              /etc/amd/net
   map_type =              file
   ```

5. Взяв за образец `/usr/share/examples/amd/net`, поместите следующее в `/etc/amd/net`:

   ```
   /defaults       type:=host;rhost:=${key};fs:=${autodir}/${rhost}/root
   *             host==${key};type:=link;fs:=/                          \
   ```

```
                host!=${key};opts:=ro,soft,intr,nodev,nosuid,noconn
```

6. Перезагрузите систему или запустите (перезапустите) **amd** вручную:

```
# service amd restart
```

29.2. *Протокол сетевого времени* (NTP)

Довольно часто системные часы идут неверно, порой на несколько минут: по какой-то странной причине компьютерные часы не отличаются высокой точностью. Проблема усугубляется, если вы администрируете множество сетевых хостов: синхронизация часов легко превращается в кошмар. Для решения этой проблемы на помощь приходит протокол NTP (версия 3): с его помощью можно синхронизировать часы сети рабочих станций, используя один или несколько серверов NTP.

Благодаря протоколу NTP можно скорректировать часы одной рабочей станции, а также синхронизировать целую сеть. Протокол NTP довольно сложен: он определяет иерархическую структуру серверов «главный — подчинённый», разделённую на страты. Вершину иерархии занимают серверы страта 1, подключённые к внешним часам (например, к приёмнику сигналов точного времени), что обеспечивает высокую точность. Ниже серверы страта 2 синхронизируют свои часы с серверами страта 1 и так далее. Точность снижается по мере продвижения к нижним уровням. Такая иерархическая структура предотвращает перегрузку, которая возникла бы, если бы все хосты обращались к одним и тем же (немногим) серверам страта 1. Если, например, вы хотите синхронизировать сеть, не подключайте все хосты к одному и тому же публичному серверу страта 1. Вместо этого создайте локальный сервер, который подключается к главному серверу, а остальные хосты синхронизируют свои часы с локальным сервером.

К счастью, для использования средств NTP не нужно разбираться в деталях протокола и его реализации (если вам интересно, обратитесь к RFC 1305) — достаточно знать, как настроить и запустить несколько программ. Базовая система NetBSD уже содержит необходимые средства для работы с этим протоколом (и другими протоколами точного времени, как мы увидим), основанные на реализации xntp. В этом разделе описан простой способ всегда иметь правильное системное время.

Сначала необходимо найти адреса публичных серверов NTP, которые будут использоваться в качестве эталона; подробный список можно найти на http://support.ntp.org/bin/view/Servers/WebHome. Например, для Италии можно использовать три сервера stratum 1: tempo.cstv.to.cnr.it, ntp1.inrim.it и ntp2.inrim.it.

Затем, чтобы скорректировать системные часы, выполните от имени root следующую команду:

```
# ntpdate -b ntp1.inrim.it ntp2.inrim.it
```

(замените имена серверов из примера на те, которые вы действительно используете. Ключ `-b` указывает **ntpdate** устанавливать системное время с помощью системного вызова settimeofday, а не плавно подстраивать его с помощью adjtime (как по умолчанию). Этот ключ рекомендуется, когда разница между местным и правильным временем может быть значительной.

Как вы убедились, пользоваться ntpdate несложно. Следующий шаг — запускать его автоматически, чтобы системное время всегда было правильным. Если у вас есть постоянное подключение к интернету, программу можно запускать при загрузке с помощью следующей строки в `/etc/rc.conf`:

```
ntpdate=YES      ntpdate_hosts="ntp1.inrim.it"
```

Имя используемого сервера NTP задаётся в переменной `ntpdate_hosts`; если оставить это поле пустым, загрузочный сценарий попытается извлечь имя из файла `/etc/ntp.conf`.

Если у вас нет постоянного подключения к интернету (например, соединение через модем по коммутируемой линии через провайдера), запускать ntpdate можно из сценария `ip-up`, как описано в Глава 24, *Настройка TCP/IP в NetBSD на практике*. В этом случае добавьте в сценарий `ip-up` следующую строку:

```
/usr/sbin/ntpdate -s -b ntp1.inrim.it
```

(путь обязателен, иначе сценарий, вероятно, не найдёт исполняемый файл). Ключ `-s` перенаправляет вывод журналирования со стандартного вывода (как по умолчанию) в системный механизм syslog(3), поэтому сообщения от ntpdate обычно попадают в `/var/log/messages`.

Помимо ntpdate есть и другие полезные команды NTP. Кроме того, можно превратить один из локальных хостов в сервер NTP для остальных хостов сети. Локальный сервер будет синхронизировать свои часы с публичным сервером. Для такой конфигурации необходимо использовать демон **ntpd** и создать файл конфигурации `/etc/ntp.conf`. Например:

```
server ntp1.inrim.it
    server ntp2.inrim.it
```

ntpd также можно запускать из `rc.conf`, используя соответствующий параметр:

```
ntpd=YES
```

Если вы хотите синхронизировать сеть, NTP — не единственный вариант: можно использовать также демон timed или команду rdate(8). Демон timed был разработан для 4.3BSD.

Демон timed также использует иерархию «главный — подчинённый»: запущенный на хосте, он запрашивает сетевое время у главного и соответствующим образом корректирует локальные часы. Можно использовать смешанную структуру, задействуя и timed, и ntpd. Один из локальных хостов получает правильное время от публичного сервера NTP и становится главным для timed для остальных хостов сети, которые становятся его клиентами и синхронизируют свои часы с помощью timed. Это означает, что на локальном сервере должны работать и NTP, и timed; нужно следить, чтобы они не мешали друг другу (timed должен запускаться с ключом `-F hostname`, чтобы не пытаться корректировать локальные часы).

Наконец, с помощью rdate(8) можно однократно синхронизироваться с заданным хостом, во многом подобно ntpdate(8). На этом хосте должна быть включена служба «time» (порт 37) в файле `/etc/inetd.conf`.

Часть V. Виртуализация и эмуляция

Глава 30. Использование виртуализации: QEMU и NVMM

Виртуальная машина — это виртуальный компьютер («гость»), работающий внутри другого компьютера («хост»). Виртуальные машины полезны для тестирования, запуска разных операционных систем, изоляции частей системы и многого другого.

nvmm(4) (NetBSD Virtual Machine Monitor) — собственный гипервизор NetBSD. Обычно он используется как «ускоритель» для программного обеспечения виртуальных машин QEMU. Он ускоряет работу виртуальных машин на вашем хосте NetBSD за счёт использования расширений виртуализации процессора. В настоящее время требуется процессор с поддержкой AMD SVM или Intel VMX, но в будущем могут быть добавлены другие бэкенды для иных архитектур. QEMU можно использовать и без ускорителя, но производительность при этом значительно ниже.

Другие гипервизоры, поддерживаемые NetBSD, включают Intel HAXM (также используется с QEMU) и Xen, архитектура которого существенно отличается.

При работе с современными операционными системами в качестве гостевых систем обычно следует использовать паравиртуализированный ввод-вывод вместо эмуляции реальных аппаратных устройств средствами QEMU. В NetBSD это поддерживается драйверами virtio(4).

30.1. Включение монитора виртуальных машин NetBSD

> **ПРИМЕЧАНИЕ**
>
> На многих компьютерах (особенно ноутбуках) аппаратные средства виртуализации по умолчанию отключены. Возможно, необходимые функции придётся включить в прошивке при загрузке.
>
> Перед загрузкой модуля NVMM убедитесь, что модули в `/stand` соответствуют используемой версии ядра NetBSD и актуальны.

Монитор виртуальных машин NetBSD по умолчанию не активен. Его необходимо активировать, загрузив модуль `nvmm` с помощью modload(8):

```
# modload nvmm
```

Проверьте, что NVMM загружен, с помощью modstat(8):

```
# modstat | grep nvmm
nvmm                    misc     filesys  -        0       - -
```

Модуль можно загружать автоматически при старте системы, добавив следующую строку в `/etc/modules.conf`:

```
nvmm
```

Загрузка NVMM при старте системы также позволит ей работать со значением secmodel_securelevel(9), равным 1, что запрещает загрузку модулей после старта. Однако, поскольку

NVMM блокирует такие операции, как приостановка, возможно, вы захотите выгрузить его:

```
# modunload nvmm
```

По умолчанию устройство `/dev/nvmm` принадлежит пользователю `root` и группе `nvmm`. Скорее всего, вы захотите запускать виртуальные машины от имени пользователя, не являющегося root, например, по соображениям безопасности.

Можно добавить пользователей в группу `nvmm`, чтобы разрешить им запускать виртуальные машины. Например, с помощью useradd(8).

Либо можно назначить владельцем устройства `/dev/nvmm` пользователя, которому разрешено запускать виртуальные машины:

```
# chown nia /dev/nvmm
```

Текущее состояние NVMM можно посмотреть с помощью nvmmctl(8):

```
$ nvmmctl identify
nvmm: Kernel API version 2
nvmm: State size 1008
nvmm: Max machines 128
nvmm: Max VCPUs per machine 256
nvmm: Max RAM per machine 128G
nvmm: Arch Mach conf 0
nvmm: Arch VCPU conf 0x3<CPUID,TPR>
nvmm: Guest FPU states 0x3<x87,SSE>
```

30.2. Использование QEMU с NVMM

QEMU — это эмулятор процессора и виртуальная машина, которая может использовать NVMM в качестве ускорителя. По умолчанию он не входит в состав NetBSD. Однако он доступен в pkgsrc как `emulators/qemu` и может быть установлен с помощью **pkgin**:

```
# pkgin install qemu
```

30.2.1. Запуск QEMU с ускорением

Эта команда запускает виртуальную машину в окне X11 с ускорением NVMM, тем же типом процессора, что и у хост-машины, двумя ядрами процессора и одним гигабайтом памяти:

```
$ qemu-system-x86_64 -accel nvmm \
        -cpu max -smp cpus=2 -m 1G \
        -display sdl,gl=on \
        -cdrom NetBSD-9.1-amd64.iso
```

Без ускорения гостевая система будет работать намного медленнее, так как каждую инструкцию процессора придётся эмулировать.

Кроме того, работающую виртуальную машину можно увидеть с помощью nvmmctl(8):

```
$ nvmmctl list
Machine ID VCPUs RAM  Owner PID Creation Time
```

```
---------- ----- ---- --------- ------------------------
0          2     147M 10982     Sat May  8 10:09:59 2021
```

30.2.2. Создание виртуального диска

Как правило, вы захотите создать на хосте виртуальный диск для размещения вашей виртуальной машины. Мы создадим образ `qcow2`, поскольку он обеспечивает лучшую производительность и более универсален, чем образ в формате raw:

```
$ qemu-img create -f qcow2 netbsd.qcow2 16G
```

Лучшую производительность обеспечивает блочное устройство VirtIO. Чтобы использовать его, добавьте следующие аргументы в `qemu-system-x86_64`:

```
-drive file=netbsd.qcow2,if=none,id=hd0 \
-device virtio-blk-pci,drive=hd0
```

В более старых операционных системах может не быть драйверов VirtIO — в этом случае можно использовать обычный эмулируемый диск:

```
-hda netbsd.qcow2
```

30.2.3. Добавление энтропии в гостевую систему

Операционным системам нужен хороший источник случайных чисел для безопасности системы, криптографии и т. п. В виртуальной машине его в идеале предоставляет хост-машина, имеющая более широкий доступ к нижележащему оборудованию. Вы можете легко подключить устройство-генератор случайных чисел VirtIO, добавив в QEMU следующие аргументы:

```
-object rng-random,filename=/dev/urandom,id=viornd0 \
-device virtio-rng-pci,rng=viornd0
```

Это не требует дополнительной настройки на хост-машине.

Энтропия обычно необходима для защищённой связи. Дополнительные сведения об энтропии см. в entropy(7).

30.2.4. Использование сети

Простейший способ организовать работу в сети с QEMU — так называемая «пользовательская сеть» (user networking). Это означает, что низкоуровневые операции с сокетами (raw socket), такие как ping(8), работать не будут, а обычные протоколы TCP/IP (HTTP/FTP и т. п.) будут. Другой способ — мостовое соединение, см. Раздел 30.3, «Настройка мостового соединения на хосте NetBSD».

Наиболее производительный тип устройства — `virtio-net-pci`:

```
-netdev user,id=vioif0 -device virtio-net-pci,netdev=vioif0
```

Для работы со старыми гостевыми операционными системами, не поддерживающими VirtIO, хорошим выбором будет Intel Gigabit Ethernet:

```
-netdev user,id=wm0 -device e1000,netdev=wm0
```

Или карта AMD PCnet — для очень старых гостевых операционных систем:

```
-netdev user,id=pcn0 -device pcnet,netdev=pcn0
```

30.2.5. Использование звука

На хосте NetBSD для включения звука можно использовать следующие аргументы QEMU:

```
-audiodev oss,id=oss,out.dev=/dev/audio,in.dev=/dev/audio \
-device ac97,audiodev=oss
```

`ac97` — классический стандартизованный звуковой драйвер для систем x86.

Возможно, вы захотите изменить используемое устройство `/dev/audioX`, см. Глава 10, *Звук*.

Возможно, для плавного воспроизведения потребуется дополнительная настройка, см. Раздел 30.4.3, «Плавное воспроизведение звука и задержка в виртуальных машинах».

30.2.6. Использование графики (или её отсутствия)

Эти аргументы создадут окно X11 с включённым OpenGL (для плавного масштабирования при изменении размера окна), используя совместимое с VMware устройство VGA и USB-мышь:

```
-display sdl,gl=on -vga vmware \
-usb -device usb-mouse,bus=usb-bus.0
```

В состав X11 для NetBSD входит видеодрайвер VMware, поэтому дисплей будет настроен автоматически при запуске startx(1), а его параметры можно скорректировать с помощью xrandr(1).

Дисплей VNC позволит удалённо подключаться клиентом VNC, например `net/tigervnc`, что удобно при запуске QEMU с параметром `--daemonize` на сервере:

```
-display vnc=unix:/home/nia/.qemu-myvm-vnc -vga vmware
-usb -device usb-mouse,bus=usb-bus.0
```

Более простой вариант — дисплей `curses`, предпочтительный для систем, которым достаточно текстового вывода в терминале:

```
-display curses
```

Дополнительные сведения о настройке X11 см. в Глава 9, *Система X Window*.

Дополнительные сведения о безопасной настройке VNC см. в документации QEMU по VNC.

30.3. Настройка мостового соединения на хосте NetBSD

Хотя пользовательская сеть QEMU проста в использовании и не требует прав root, она, как правило, медленнее мостового соединения через устройство tap(4) и не позволяет использовать в гостевой системе диагностические средства, такие как ping(8).

Чтобы настроить мостовое соединение на хосте NetBSD, сначала запомните основной сетевой интерфейс хост-машины. Найдите интерфейс, которому назначен адрес и у которого есть маршрут во внешний мир, с помощью ifconfig(8).

В этом примере основной интерфейс хост-машины — `wm0`. Все эти команды выполняются на хост-машине.

Создайте виртуальный интерфейс tap(4):

```
# ifconfig tap0 create
# ifconfig tap0 descr "NetBSD VM" up
```

Создайте bridge(4), соединяющий реальный интерфейс с виртуальным:

```
# ifconfig bridge0 create
# ifconfig bridge0 descr "LAN VM bridge" up
# brconfig bridge0 add tap0 add wm0
```

Настройте NetBSD на выполнение всех этих действий при загрузке, отредактировав `/etc/ifconfig.tap0`:

```
create
descr "NetBSD VM" up
! ifconfig bridge0 create
! ifconfig bridge0 descr "LAN VM bridge" up
! brconfig bridge0 add tap0 add wm0
```

Теперь можно передать QEMU аргументы для работы с мостовым соединением:

```
-netdev tap,id=tap0,ifname=tap0,script=no -device virtio-net-pci,netdev=tap0
```

Дополнительные сведения о настройке сети в NetBSD см. в Глава 24, *Настройка TCP/IP в NetBSD на практике*.

30.4. Замечания об использовании NetBSD в качестве гостевой системы

30.4.1. Некорректное завершение работы виртуальной машины, восстановление данных и fsck

ИЗБЕГАЙТЕ НЕКОРРЕКТНОГО ЗАВЕРШЕНИЯ РАБОТЫ! Имеется в виду нажатие Ctrl+C или завершение процесса виртуальной машины. В QEMU диски редко синхронизируются, и потеря данных почти неизбежна.

Возможно, вы захотите добавить параметры монтирования `log,noatime` в `/etc/fstab` рядом с `rw`, чтобы ускорить fsck(8). Можно также включить параметр `sync`, но это существенно снизит производительность.

Всегда корректно завершайте работу NetBSD командой shutdown(8) и делайте резервные копии.

30.4.2. Виртуальные машины NetBSD без IPv6

Иногда сетевые средства QEMU настраивают недопустимый маршрут IPv6 в конфигурациях только с IPv4, из-за чего программы, например средства работы с пакетами NetBSD, предпочитают IPv6 и подолгу ждут истечения тайм-аута, прежде чем завершить операцию.

Обойдите это, отредактировав `/etc/rc.conf` так, чтобы предпочитались адреса IPv4:

```
ip6addrctl=YES
ip6addrctl_policy="ipv4_prefer"
```

30.4.3. Плавное воспроизведение звука и задержка в виртуальных машинах

Как правило, виртуальные машины не могут обеспечить такое же плавное воспроизведение с низкой задержкой, как реальное оборудование. Для плавного воспроизведения может потребоваться увеличить задержку звука NetBSD внутри виртуальной машины:

```
$ sysctl -w hw.audio0.blk_ms=100
```

Чтобы задавать это автоматически при загрузке, добавьте соответствующую строку в `/etc/sysctl.conf`.

Проверить вывод звука можно в виртуальной машине. Убедитесь, что audiocfg(1) воспроизводит непрерывный сигнал для каждого канала:

```
$ audiocfg test 0
```

30.4.4. Изменение разрешения консоли в виртуальной машине x86

> **ПРИМЕЧАНИЕ**
>
> На реальном оборудовании, где разрешение экрана уже правильно задано ядром, это отключит графическое ускорение.

Если вы хотите увеличить размер консоли x86, введите следующее в приглашении загрузчика NetBSD:

```
> vesa 1024x768x32
```

Эту настройку можно сделать постоянной в `/boot.cfg`.

Глава 31. Эмуляция Linux

Версии NetBSD для amd64, i386, alpha, mac68k, macppc и многих других платформ могут выполнять большое количество нативных программ Linux с помощью слоя эмуляции Linux. Обычно, думая об эмуляции, вы представляете себе что-то медленное и неэффективное, поскольку эмуляция часто должна воспроизводить в программном виде инструкции оборудования и даже целые архитектуры (как правило, старых машин). В случае эмуляции Linux всё радикально иначе: это лишь тонкий программный слой, в основном для системных вызовов, которые и без того очень похожи в обеих системах. Сам код приложения выполняется на полной скорости вашего процессора, поэтому производительность при эмуляции Linux не снижается, а ощущения точно такие же, как от нативных приложений NetBSD.

31.1. Настройка эмуляции

Установка эмуляции Linux описана в странице руководства compat_linux(8); при использовании системы пакетов требуется всего два шага.

31.1.1. Настройка ядра

Если вы используете ядро GENERIC, ничего делать не нужно: совместимость с Linux уже включена.

Если вы используете ядро, собранное с изменениями, проверьте, что включены следующие параметры:

```
option COMPAT_LINUX
option EXEC_ELF32
```

или следующие параметры, если вы собираетесь использовать 64-битные двоичные файлы ELF:

```
option COMPAT_LINUX
option EXEC_ELF64
```

после сборки ядра с указанными параметрами можно приступать к установке необходимого программного обеспечения.

31.1.2. Установка библиотек Linux

Обычно приложения компонуются с разделяемыми библиотеками, а приложениям Linux нужны разделяемые библиотеки Linux. Разделяемые библиотеки можно взять из любого дистрибутива Linux, если он не слишком старый, но рекомендуемый способ — установить предоставляемые библиотеки из openSUSE с помощью системы пакетов.

Все двоичные файлы Linux находятся исключительно в отдельных корневых каталогах внутри `/emul/linux` и `/emul/linux32`. При поиске разделяемых объектов, необходимых программам Linux, ядро всегда сначала просматривает эти пути.

В pkgsrc предоставляется ряд полезных разделяемых объектов Linux для запуска как 64-, так и 32-битных приложений. Абсолютный минимум, необходимый для запуска динамически

скомпонованных приложений Linux, обеспечивают пакеты `suse131_base` и `suse131_32_base` (или, при использовании двоичных пакетов, `suse_base-13` и `suse32_base-13`). Многие другие библиотеки также поставляются отдельными пакетами.

Некоторые пакеты в pkgsrc поставляются как двоичные файлы Linux и при установке также устанавливают все необходимые зависимости SUSE. Однако это встречается нечасто. Один из таких пакетов — `adoptopenjdk11-bin`.

Узнать, какие библиотеки требуются программе Linux, можно с помощью readelf(1):

```
$ readelf -d ./runner
Dynamic section at offset 0x3a2e94 contains 40 entries:
  Tag        Type           Name/Value
 0x00000001 (NEEDED)    Shared library: [libstdc++.so.6]
 0x00000001 (NEEDED)    Shared library: [libz.so.1]
 0x00000001 (NEEDED)    Shared library: [libXxf86vm.so.1]
 0x00000001 (NEEDED)    Shared library: [libGL.so.1]
 0x00000001 (NEEDED)    Shared library: [libopenal.so.1]
 0x00000001 (NEEDED)    Shared library: [libm.so.6]
 0x00000001 (NEEDED)    Shared library: [librt.so.1]
 0x00000001 (NEEDED)    Shared library: [libpthread.so.0]
 0x00000001 (NEEDED)    Shared library: [libdl.so.2]
 0x00000001 (NEEDED)    Shared library: [libcrypto.so.1.0.0]
 0x00000001 (NEEDED)    Shared library: [libXext.so.6]
 0x00000001 (NEEDED)    Shared library: [libX11.so.6]
 0x00000001 (NEEDED)    Shared library: [libXrandr.so.2]
 0x00000001 (NEEDED)    Shared library: [libGLU.so.1]
 0x00000001 (NEEDED)    Shared library: [libssl.so.1.0.0]
 0x00000001 (NEEDED)    Shared library: [libgcc_s.so.1]
 0x00000001 (NEEDED)    Shared library: [libc.so.6]
```

Например, приложению, которому требуются `libcrypto.so.1.0.0`, `libXext.so.6` и `libGL.so.1`, понадобятся `openssl`, `x11` и `glx` в дополнение к пакету `base` SUSE.

31.1.3. Запуск программ Linux

После установки нужных библиотек для запуска программы Linux не требуется никаких особых действий — просто введите команду (если вы получили её не через pkgsrc, укажите полный путь в файловой системе). Ядро определит, что это исполняемый файл Linux, и запустит его в соответствующем режиме трансляции.

31.2. Структура каталогов

Если посмотреть на результат установки библиотек и программ Linux, можно обнаружить, что `/emul/linux` — это символическая ссылка на `/usr/pkg/emul/linux`, где созданы следующие каталоги:

```
bin/

dev/

etc/

lib/

lib64/

proc/
```

```
sbin/

usr/

var/
```

> **ПРИМЕЧАНИЕ**
>
> Всегда обращайтесь к `/emul/linux`, а не к `/usr/pkg/emul/linux`. Последний — деталь реализации, которая может измениться в будущем.

Сколько места требуется для программного обеспечения эмуляции Linux? В одной системе у нас получилась следующая цифра:

```
# cd /usr/pkg/emul
# du -k /emul/linux/
...
399658  /emul/linux/
```

31.3. Использование подключаемых модулей Linux для браузеров

Подключаемые модули Linux для браузеров на основе Mozilla можно использовать в обычных сборках Firefox для NetBSD с помощью nspluginwrapper — обёртки, которая обеспечивает взаимодействие между обычным браузером и внешним подключаемым модулем. В настоящее время nspluginwrapper надёжно работает только с браузерами на основе Mozilla, скомпонованными с GTK2+ (GTK1+ не поддерживается). nspluginwrapper можно установить через pkgsrc:

```
# cd /usr/pkgsrc/www/nspluginwrapper
# make install
```

Затем подключаемые модули можно установить в два этапа: сначала модуль нужно установить в системе (например, через pkgsrc). После этого пользователи, желающие использовать этот модуль, должны зарегистрировать его с помощью **nspluginwrapper**.

В этом коротком примере мы рассмотрим установку подключаемого модуля Macromedia Flash. Первый этап можно выполнить, установив модуль Flash через pkgsrc:

```
# cd /usr/pkgsrc/multimedia/ns-flash
# make install
```

После этого непривилегированный пользователь может зарегистрировать модуль Flash:

```
$ nspluginwrapper -i /usr/pkg/lib/netscape/plugins/libflashplayer.so
```

После этого модуль должен быть зарегистрирован правильно. Проверить это можно с помощью ключа `-l` программы **nspluginwrapper** (**nspluginwrapper -l**). Если модуль присутствует в списке, перезапустите Firefox и проверьте, что модуль установлен, введя *about:plugins* в адресной строке.

31.4. Дополнительная литература

Для более глубокого понимания эмуляции Linux (и не только) могут быть полезны следующие статьи:

Библиография

[chap-linux-further-implementing-linux-emul-on-netbsd] *Реализация эмуляции Linux в NetBSD* . Peter Seebach. Май 2004.

Часть VI. Сборка системы

Глава 32. Получение исходных текстов

Чтобы читать исходные тексты NetBSD с локального диска или собирать систему, необходимо загрузить исходные тексты NetBSD. В этой главе описано несколько способов получения исходных текстов NetBSD, хотя предпочтительный метод — загрузить архивы tar и затем обновляться через cvs(1).

32.1. Подготовка каталогов

Традиционно исходные тексты ядра и пользовательского окружения NetBSD размещаются в `/usr/src`. По умолчанию этого каталога в установленной системе NetBSD нет, поэтому сначала его нужно создать. Поскольку он находится в системном каталоге, для его создания потребуются права root, а также нужно обеспечить, чтобы ваша обычная учётная запись могла в него записывать. Для примера предполагается, что имя входа непривилегированного пользователя — *`carlo`*. Замените его на действительное имя входа в вашей системе:

```
$ su
Password: ********
# mkdir /usr/src
# chown <carlo> /usr/src
```

Если вам нужны исходные тексты системы X Window, следует также подготовить `/usr/xsrc`:

```
# mkdir /usr/xsrc
# chown <carlo> /usr/xsrc
```

> **ПРИМЕЧАНИЕ**
>
> Учтите, что для последующих шагов права root не требуются и не рекомендуются, поэтому этот подготовительный шаг следует выполнить первым. Все операции CVS можно (и нужно) выполнять как обычный пользователь, поэтому откажитесь от прав root:
>
> ```
> # exit
> $
> ```

32.2. Терминология

Прежде чем приступать к получению или загрузке необходимых файлов, возможно, вы захотите узнать, что означают «Официальные выпуски», «Ветви сопровождения» и другие связанные термины. Эта информация доступна в разделе Глоссарий и схемы выпусков NetBSD.

32.3. Загрузка архивов tar

Иногда быстрее начать с загрузки архива tar с исходными текстами. Архивы tar (см. tar(1)) для ряда выпусков и ветвей можно загрузить с ftp.NetBSD.org (или любого другого зеркала). Эти архивы содержат каталоги `CVS`, поэтому вы можете продолжать обновлять дерево исходных текстов с помощью cvs(1), как описано в разделе о CVS.

Учтите, что архивы tar с исходными текстами для стабильных ветвей обновляются только раз в три дня.

32.3.1. Загрузка исходных текстов выпуска NetBSD

Исходные файлы выпуска не изменяются после его создания.

Архивы tar с исходными текстами конкретного выпуска доступны по пути `/pub/NetBSD/NetBSD-<RELEASE-NUMBER>/source/sets/` на ftp.NetBSD.org (или зеркале), где *`<RELEASE-NUMBER>`* — нужный вам выпуск (например, 10.1).

Чтобы получить исходные тексты выпуска NetBSD с помощью архивов tar, просто выполните:

```
$ ftp -i ftp://ftp.NetBSD.org/pub/NetBSD/NetBSD-10.1/source/sets/
Trying 2001:470:a085:999::21:21 ...
Connected to ftp.NetBSD.org.
220 ftp.NetBSD.org FTP server (NetBSD-ftpd 20180428) ready.
331 Guest login ok, type your name as password.
[...]
250 CWD command successful.
ftp> mget *.tgz
local: gnusrc.tgz remote: gnusrc.tgz
229 Entering Extended Passive Mode (|||61968|)
150 Opening BINARY mode data connection for 'gnusrc.tgz' (152604808 bytes).
[...]
ftp> quit
221-
221 Thank you for using the FTP service on ftp.NetBSD.org.
```

Теперь у вас должно быть 5 файлов:

```
$ ls *.tgz
gnusrc.tgz      sharesrc.tgz    src.tgz         syssrc.tgz      xsrc.tgz
```

Теперь необходимо распаковать их все:

```
$ for file in *.tgz
> do
> tar -xzf $file -C /
> done
```

32.3.2. Загрузка исходных текстов стабильной ветви NetBSD

```
$ ftp -i ftp://ftp.NetBSD.org/pub/NetBSD/NetBSD-release-10/tar_files/src/
Trying 2001:470:a085:999::21:21 ...
Connected to ftp.NetBSD.org.
220 ftp.NetBSD.org FTP server (NetBSD-ftpd 20180428) ready.
331 Guest login ok, type your name as password.
[...]
250 CWD command successful.
ftp> mget *.tar.gz
```

```
local: bin.tar.gz remote: bin.tar.gz
229 Entering Extended Passive Mode (|||56011|)
150 Opening BINARY mode data connection for 'bin.tar.gz' (996075 bytes).
[...]
ftp> quit
221-
    Data traffic for this session was 429024351 bytes in 25 files.
    Total traffic for this session was 429035139 bytes in 26 transfers.
221 Thank you for using the FTP service on ftp.NetBSD.org.
```

Теперь у вас должно быть 25 файлов:

```
$ ls *.tar.gz
bin.tar.gz          doc.tar.gz          libexec.tar.gz      tools.tar.gz
common.tar.gz       etc.tar.gz          regress.tar.gz      top-level.tar.gz
compat.tar.gz       external.tar.gz     rescue.tar.gz       usr.bin.tar.gz
config.tar.gz       extsrc.tar.gz       sbin.tar.gz         usr.sbin.tar.gz
crypto.tar.gz       games.tar.gz        share.tar.gz
dist.tar.gz         include.tar.gz      sys.tar.gz
distrib.tar.gz      lib.tar.gz          tests.tar.gz
```

Теперь необходимо распаковать их все:

```
$ for file in *.tar.gz
> do
> tar -xzf $file -C /usr
> done
```

32.3.3. Загрузка исходных текстов ветви разработки NetBSD-current

Чтобы загрузить архивы tar NetBSD-current, расположенные по пути `/pub/NetBSD/NetBSD-current/tar_files/src`, выполните те же шаги, что и в Раздел 32.3.2, «Загрузка исходных текстов стабильной ветви NetBSD», но с использованием этого пути.

Возможно, вам также понадобятся исходные тексты системы X Window, доступные по пути: `/pub/NetBSD/NetBSD-current/tar_files/xsrc`.

32.4. Получение через CVS

CVS (Concurrent Versions System) можно использовать для получения дерева исходных текстов NetBSD или для его поддержания в актуальном состоянии с учётом изменений, вносимых в исходные тексты NetBSD. Через cvs(1) доступны два основных модуля исходных текстов: «src» и «xsrc».

Список поддерживаемых в настоящее время ветвей доступен в `src/doc/BRANCHES` (см. пункт «Status» в разделе «Release branches»).

> **ОСТОРОЖНО!**
>
> Будьте внимательны при выборе правильной и нужной метки ветви, чтобы случайно не *понизить* версию дерева исходных текстов.

Прежде чем выполнить начальную (полную) выгрузку исходных текстов NetBSD через *анонимный CVS*, необходимо задать переменную окружения `CVSROOT`, которая указывает cvs(1), откуда загружать файлы:

```
$ export CVSROOT="anoncvs@anoncvs.NetBSD.org:/cvsroot"
```

Убедитесь, что переменная окружения `CVS_RSH` имеет значение «ssh».

```
$ export CVS_RSH="ssh"
```

В приведённых ниже примерах используется ключ `-P`, который указывает CVS удалять пустые каталоги.

32.4.1. Получение выпуска NetBSD

Исходные файлы выпуска не изменяются после его создания.

Чтобы получить исходные тексты NetBSD (ядра и пользовательского окружения) для конкретного выпуска, выполните следующую команду, задав `CVSROOT`, как показано выше:

```
$ cd /usr
$ cvs checkout -r <TAG> -P src
```

где *<TAG>* — метка выпуска, которую нужно выгрузить, например «netbsd-10-1-RELEASE». Если вам нужен более поздний уровень патчей, используйте, например, «netbsd-10-1-RELEASE».

Например, чтобы получить исходные тексты NetBSD 10.1, используйте метку «netbsd-10-1-RELEASE»:

```
$ cvs checkout -r netbsd-10-1-RELEASE -P src
```

Чтобы получить исходные тексты системы X Window, достаточно «выгрузить» модуль «xsrc». Например:

```
$ cvs checkout -r netbsd-10-1-RELEASE -P xsrc
```

32.4.2. Получение стабильной ветви NetBSD

Стабильные ветви NetBSD — одна из разновидностей «Ветвей сопровождения». Обратитесь к Раздел 32.2, «Терминология».

Если вы хотите следить за стабильной ветвью, просто передайте имя ветви ключу cvs(1) `-r`.

Например, если вам нужна самая свежая версия «netbsd-9», достаточно использовать эту метку:

```
$ cd /usr
$ cvs checkout -r netbsd-9 -P src
```

А для модуля «xsrc»:

```
$ cvs checkout -r netbsd-9 -P xsrc
```

Если вы выгрузили исходные тексты стабильной ветви в `/usr/src` и хотите обновить их, чтобы получить последние исправления безопасности и ошибок, выполните:

```
$ cd /usr/src
$ cvs update -dP
```

То же самое относится к модулю «xsrc», но в этом случае сначала нужно сменить рабочий каталог на `/usr/xsrc`.

32.4.3. Получение ветви разработки NetBSD-current

Чтобы получить исходные тексты NetBSD-current, просто опустите «`-r` *<BRANCH>*» и замените его на «`-A`»:

```
$ cd /usr
$ cvs checkout -A -P src
```

Модуль «xsrc» получается так же:

```
$ cd /usr
$ cvs checkout -A -P xsrc
```

Чтобы обновить дерево исходных текстов NetBSD-current, добавьте флаг `-A`:

```
$ cd /usr/src
$ cvs update -A -dP
```

То же самое относится к модулю «xsrc», но в этом случае сначала нужно сменить рабочий каталог на `/usr/xsrc`.

32.4.4. Сохранение некоторых параметров cvs(1)

Если вы раз за разом вводите для CVS одни и те же параметры, возможно, стоит сделать их значениями по умолчанию, добавив в `.cvsrc` в вашем домашнем каталоге. В следующем примере **cvs update** будет добавлять в дерево недостающие новые каталоги, а также удалять каталоги, ставшие пустыми. Там же показано, как заставить **cvs rdiff** и **cvs diff** использовать формат unified программы diff(1). Последняя строка в примере делает работу CVS немного менее многословной.

Пример 32.1. .cvsrc

```
update  -dP
rdiff   -u
diff    -u
cvs     -q
```

Глава 33. Кросс-компиляция NetBSD с помощью `build.sh`

В NetBSD используется инфраструктура `build.sh`, позволяющая собрать как ядро операционной системы, так и всё пользовательское окружение — для той же платформы, что и хост-машина, либо для другой платформы с помощью кросс-компиляции. `build.sh` создаёт все средства, необходимые для сборки NetBSD для заданной платформы, и предоставляет их готовыми к использованию в работе над проектом.

В этой главе мы покажем, как с помощью `build.sh` сначала создать инструментарий, включающий компилятор, ассемблер, компоновщик и т. д., а затем использовать его для кросс-компиляции ядра и пользовательского окружения NetBSD для AArch64 на хост-машине с AMD64. Хотя обычная сборка ядра рассматривается в Глава 34, *Компиляция ядра*, для сборки NetBSD целиком обычно предпочитают сценарий `build.sh`.

Перед началом мы разместили исходные тексты в подкаталоге домашнего каталога нашего пользователя: `/home/nia/cvs/src`. Хотя `/usr/src` — традиционное место для исходных текстов NetBSD, как описано в Глава 32, *Получение исходных текстов*, мы хотим собирать операционную систему как непривилегированный (не root) пользователь.

Более подробное описание инфраструктуры `build.sh` можно найти в статье Luke Mewburn и Matthew Green и их презентации с конференции BSDCon 2003, а также в файле `/usr/src/BUILDING`.

33.1. Сборка инструментария

Первый шаг кросс-разработки — получить все необходимые средства. В терминологии NetBSD это называется «инструментарием» (toolchain), и он включает совместимую с BSD программу make(1), компиляторы C/C++, компоновщик, ассемблер, config(8), а также довольно много средств, которые требуются только при кросс-компиляции полного выпуска NetBSD, что здесь не рассматривается.

Команда создания инструментария довольно проста и использует сценарий `src/build.sh` из NetBSD. Учтите, что все приведённые здесь команды можно выполнять как обычный (не root) пользователь:

```
$ cd /home/nia/cvs/src
$ ./build.sh -U -O ~/obj -j2 -m evbarm -a aarch64 tools
```

- Ключ «-U» указывает, что сборка выполняется непривилегированным пользователем, как правило, когда исходные тексты расположены не в `/usr/src`
- Ключ «-O» задаёт каталог для скомпилированных объектных файлов.
- Ключ «-j2» задаёт число параллельно выполняемых сборок. Возможно, его стоит выбрать равным числу ядер процессора на вашей хост-системе.
- Ключи «-m evbarm -a aarch64» указывают, что сборка выполняется для типа машины *evbarm* и типа процессора *aarch64*. Список доступных машин и типов процессоров можно получить так:

  ```
  $ ./build.sh list-arch
  ```

Если средства уже были собраны ранее и требуют лишь обновления, можно использовать ключ обновления «-u», чтобы пересобрать только изменившиеся средства:

```
$ ./build.sh -U -u -O ~/obj -j2 -m evbarm -a aarch64 tools
```

По завершении сборки средств выводится информация о них и о нескольких переменных окружения:

```
===> Tools built to /home/nia/obj/tooldir.NetBSD-9.99.81-amd64
===> build.sh ended:      Sun Apr 18 12:10:58 CEST 2021
===> Summary of results:
         build.sh command:    ./build.sh -U -u -O /home/nia/obj -j2 -m evbarm -a aarch64 tools
         build.sh started:    Sun Apr 18 11:17:46 CEST 2021
         NetBSD version:      9.99.81
         MACHINE:             evbarm
         MACHINE_ARCH:        aarch64
         Build platform:      NetBSD 9.99.81 amd64
         HOST_SH:             /bin/sh
         No $TOOLDIR/bin/nbmake, needs building.
         Bootstrapping nbmake
         MAKECONF file:       /etc/mk.conf
         TOOLDIR path:        /home/nia/obj/tooldir.NetBSD-9.99.81-amd64
         DESTDIR path:        /home/nia/obj/destdir.evbarm
         RELEASEDIR path:     /home/nia/obj/releasedir
         Created /home/nia/obj/tooldir.NetBSD-9.99.81-amd64/bin/nbmake
         Updated makewrapper: /home/nia/obj/tooldir.NetBSD-9.99.81-amd64/bin/nbmake-evbarm
         Tools built to /home/nia/obj/tooldir.NetBSD-9.99.81-amd64
         build.sh ended:      Sun Apr 18 12:10:58 CEST 2021
```

Во время сборки последовательно используются объектные каталоги, то есть сохраняются особые каталоги для объектных файлов и результатов компиляции, зависящих от платформы. В нашем примере они будут храниться в каталогах с именем «obj.evbarm», поскольку целевой платформой сборки является AArch64.

Сам инструментарий тоже входит в их число, но поскольку он работает и собран для системы amd64, он размещается в отдельном каталоге, указывающем, откуда выполняется кросс-сборка. Вот где находится наш инструментарий:

```
$ pwd
/home/nia/cvs/src
$ ls -d ~/obj/tooldir*
/home/nia/obj/tooldir.NetBSD-9.99.81-amd64
```

Итак, общее правило таково: для заданного сочетания «хост» — «цель» кросс-компилятор по умолчанию размещается в каталоге «tooldir.host». Полный список всех средств, создаваемых для кросс-компиляции всей операционной системы NetBSD, включает:

```
$ ls ~/obj/tooldir.NetBSD-9.99.81-amd64/bin
aarch64--netbsd-addr2line    nbconfig                    nbmkcsmapper
aarch64--netbsd-ar           nbcrunchgen                 nbmkdep
aarch64--netbsd-as           nbctags                     nbmkesdb
aarch64--netbsd-c++          nbctfconvert                nbmklocale
aarch64--netbsd-c++filt      nbctfmerge                  nbmknod
aarch64--netbsd-cpp          nbcvslatest                 nbmktemp
aarch64--netbsd-dbsym        nbdb                        nbmkubootimage
aarch64--netbsd-elfedit      nbdisklabel                 nbmsgc
aarch64--netbsd-fdisk        nbdtc                       nbmtree
```

```
aarch64--netbsd-g++          nbeqn                    nbnroff
aarch64--netbsd-gcc          nbfile                   nbpax
aarch64--netbsd-gcc-9.3.0    nbgenassym               nbpaxctl
aarch64--netbsd-gcc-ar       nbgencat                 nbperf
aarch64--netbsd-gcc-nm       nbgmake                  nbpic
aarch64--netbsd-gcc-ranlib   nbgpt                    nbpwd_mkdb
aarch64--netbsd-gcov         nbgrep                   nbrefer
aarch64--netbsd-gcov-dump    nbgroff                  nbrpcgen
aarch64--netbsd-gcov-tool    nbhexdump                nbsed
aarch64--netbsd-install      nbhost-mkdep             nbslc
aarch64--netbsd-ld           nbindxbib                nbsoelim
aarch64--netbsd-ld.bfd       nbinstall-info           nbsortinfo
aarch64--netbsd-mdsetimage   nbinstallboot            nbstat
aarch64--netbsd-nm           nbjoin                   nbstrfile
aarch64--netbsd-objcopy      nblex                    nbsunlabel
aarch64--netbsd-objdump      nbllvm-tblgen            nbtbl
aarch64--netbsd-ranlib       nblorder                 nbtexi2dvi
aarch64--netbsd-readelf      nbm4                     nbtexi2pdf
aarch64--netbsd-size         nbmake                   nbtexindex
aarch64--netbsd-strings      nbmake-evbarm            nbtic
aarch64--netbsd-strip        nbmakefs                 nbtsort
nbasn1_compile               nbmakeinfo               nbuudecode
nbawk                        nbmakekeys               nbxz
nbcap_mkdb                   nbmakestrs               nbyacc
nbcat                        nbmakewhatis             nbzic
nbcksum                      nbmandoc
nbcompile_et                 nbmenuc
```

Как видите, большинство средств, доступных в NetBSD изначально, присутствуют с некоторым префиксом, позволяющим определить целевую платформу для средств, специфичных для конкретной целевой платформы.

Один важный инструмент, который стоит здесь отметить, — «nbmake-evbarm». Это обёртка оболочки для совместимой с BSD команды make(1), настроенная на использование всех нужных команд из инструментария кросс-компиляции. Использование этой обёртки вместо /usr/bin/make позволяет кросс-компилировать программы, написанные с применением инфраструктуры Makefile NetBSD (см. src/share/mk). Через минуту мы воспользуемся этой обёрткой make(1), чтобы выполнить кросс-компиляцию ядра!

33.2. Настройка ядра вручную

Теперь, когда у нас есть работающий инструментарий, нужны «обычные» шаги для сборки ядра: создать файл конфигурации ядра, запустить config(1), а затем выполнить сборку. Поскольку программа config(1), создающая заголовочные файлы и Makefile для сборки ядра, зависит от платформы, необходимо использовать программу «nbconfig» из нашего нового инструментария. В остальном процедура точно такая же, как компиляция «обычного» ядра NetBSD. Используемые здесь команды:

```
$ cd /home/nia/cvs/src/sys/arch/evbarm/conf
$ cp GENERIC64 MYKERNEL
$ vi MYKERNEL
$ /home/nia/obj/tooldir.NetBSD-9.99.81-amd64/bin/nbconfig MYKERNEL
```

Вот и всё. Эта команда создала каталог `../compile/MYKERNEL` с рядом заголовочных файлов, определяющих сведения об устройствах, включаемых в ядро, и Makefile, настроенный на сборку всех необходимых для ядра файлов и их компоновку.

33.3. Сборка ядра вручную

У нас есть все файлы и средства, чтобы кросс-компилировать ядро для AArch64 на хост-системе с процессором Intel, так что приступим! Перейдя в каталог, созданный на предыдущем шаге, нужно использовать обёртку-сценарий `nbmake-evbarm` из инструментария, которая просто вызывает make(1) со всеми необходимыми настройками для кросс-компиляции под платформу AArch64:

```
$ cd ../compile/MYKERNEL/
$ /home/nia/obj/tooldir.NetBSD-9.99.81-amd64/bin/nbmake-evbarm depend
$ /home/nia/obj/tooldir.NetBSD-9.99.81-amd64/bin/nbmake-evbarm
```

Сборка некоторое время будет работать, а затем выдаст ядро:

```
...
   text    data     bss     dec     hex filename
11131832        3627920 1738912 16498664         fbbfe8 netbsd
...
$ ls -lh netbsd
-rwxr-xr-x  1 nia  wheel   16M Apr 18 12:46 netbsd
$ file netbsd
netbsd: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), statically linked, for
NetBSD 9.99.81, not stripped
```

> **ПРИМЕЧАНИЕ**
>
> Поскольку сборка выполняется для AArch64, были созданы и несколько других образов ядра в иных форматах, в том числе `netbsd.img` и `netbsd.bin` для раздела `/boot` U-Boot. В системах с более стандартными процедурами загрузки достаточно позаботиться только о `netbsd`.

Теперь ядро из файла `netbsd` можно перенести на машину (через scp, rsync и т. п.) и загрузить.

После настройки и кросс-компиляции ядра следующий логичный шаг — кросс-компиляция всей системы и приведение её к виду, готовому для распространения. Прежде чем перейти к нему, в следующем разделе будет показан альтернативный подход к кросс-компиляции ядра: использование сценария `build.sh` для настройки и кросс-компиляции ядра за один шаг.

33.4. Сборка ядра с помощью `build.sh`

Кросс-компилированное ядро можно собрать вручную, как описано в предыдущих разделах, или более простым способом — с помощью `build.sh`, который и будет показан здесь.

Подготовка файла конфигурации ядра выполняется так же, как описано выше:

```
$ cd /home/nia/cvs/src/sys/arch/evbarm/conf
$ cp GENERIC64 MYKERNEL
$ vi MYKERNEL
```

Затем отредактируйте *`MYKERNEL`* и, когда закончите, останется только собрать ядро с помощью `build.sh` (он также выполнит настройку, проделав шаги, показанные выше):

```
$ cd /home/nia/cvs/src
$ ./build.sh -U -u -j2 -O ~/obj -m evbarm -a aarch64 kernel=MYKERNEL
```

Обратите внимание, что указан ключ обновления («-u»): средства уже собраны, и пересобирать их все нет смысла. После сборки ядра `build.sh` выведет его местоположение и другие сведения:

```
...
===> Summary of results:
         build.sh command:    ./build.sh -O /home/nia/obj -U -u -j2 -m evbarm -a aarch64
kernel=MYKERNEL
         build.sh started:    Sun Apr 18 12:25:16 CEST 2021
         NetBSD version:      9.99.81
         MACHINE:             evbarm
         MACHINE_ARCH:        aarch64
         Build platform:      NetBSD 9.99.81 amd64
         HOST_SH:             /bin/sh
         MAKECONF file:       /etc/mk.conf
         TOOLDIR path:        /home/nia/obj/tooldir.NetBSD-9.99.81-amd64
         DESTDIR path:        /home/nia/obj/destdir.evbarm
         RELEASEDIR path:     /home/nia/obj/releasedir
         Updated makewrapper: /home/nia/obj/tooldir.NetBSD-9.99.81-amd64/bin/nbmake-evbarm
         Building kernel without building new tools
         Building kernel:     MYKERNEL
         Build directory:     /home/nia/obj/sys/arch/evbarm/compile/MYKERNEL
         Kernels built from MYKERNEL:
          /home/nia/obj/sys/arch/evbarm/compile/MYKERNEL/netbsd
         build.sh ended:      Sun Apr 18 12:46:26 CEST 2021
===> .
```

Здесь интересен путь к собранному ядру: `/home/nia/obj/sys/arch/evbarm/compile/`*`MYKERNEL`*`/netbsd`, — его можно использовать так же, как описано выше.

33.5. Сборка пользовательского окружения

К настоящему моменту, вероятно, уже становится ясно, что инструментарий на самом деле работает поэтапно. Сначала собирается инструментарий, затем ядро. Поскольку `build.sh` при каждом запуске пытается пересобрать средства, использование «update» экономит время. Вероятно, также ясно, что помимо нескольких ключей семантика `build.sh` в сущности сводится к `build.sh` *`command`*. Поэтому разумно заключить, что сборка всего пользовательского окружения и/или выпуска — это вопрос использования правильных команд.

Неудивительно, что сборка и создание выпуска выглядят так:

```
$ ./build.sh -U -u -j2 -O ~/obj -m evbarm -a aarch64 release
```

Эти команды скомпилируют всё пользовательское окружение NetBSD и поместят его в целевой объектный каталог, а затем соберут из него выпуск в подкаталоге release.

33.6. Сборка системы X Window

У проекта NetBSD есть собственная копия системы X Window, содержащая изменения, благодаря которым X хорошо работает на как можно большем числе платформ, поддерживаемых NetBSD. Поэтому желательно использовать версию системы X Window, доступную из NetBSD и для NetBSD, которую также можно кросс-компилировать почти так же, как ядро и базовую систему. Для этого исходные тексты «xsrc» необходимо выгрузить из CVS так же, как выгружались «src» и «pkgsrc», как описано в Глава 32, *Получение исходных текстов*.

После этого X можно кросс-компилировать для целевой платформы, добавив к build.sh ключ `-x`, например при создании полного выпуска:

```
$ ./build.sh -U -u -O ~/obj -j2 -x -X /home/nia/cvs/xsrc -m evbarm -a aarch64 release
```

Ключ `-x` автоматически задаёт переменную для сборки X11, а `-X` указывает каталог, содержащий исходные тексты X11.

33.7. Изменение поведения сборки

В системе сборки есть множество переменных, с помощью которых можно управлять, например, тем, куда попадают определённые файлы, какие средства используются (если используются), и т. д. Большинство из них описано в `src/BUILDING`. В этом разделе приведено несколько примеров изменения настроек по умолчанию, каждый из которых делается по-своему.

33.7.1. Изменение каталога назначения

Многие предпочитают следить за NetBSD-current и выполнять кросс-компиляцию для используемых ими архитектур. Логика проста: иногда появляется новая возможность или устройство, и кто-то может захотеть ими воспользоваться. Отслеживая изменения и время от времени выполняя сборку, можно быть уверенным, что эти архитектуры способны собрать собственный выпуск.

Разумно предположить, что при отслеживании и сборке более чем для одной архитектуры может возникнуть желание хранить результаты сборки не в месте по умолчанию. Есть два способа: использовать сценарий для задания нового DESTDIR или просто сделать это в интерактивном режиме. В любом случае его можно задать так же, как любую другую переменную (разумеется, в зависимости от вашей оболочки).

Для sh(1) или ksh(1) это:

```
$ export DESTDIR=/usr/builds/evbarm-aarch64
```

Для csh(1) команда такова:

```
$ setenv DESTDIR /usr/builds/evbarm-aarch64
```

Достаточно просто. При запуске сборки двоичные файлы и прочие файлы будут помещены в `/usr/builds`.

33.7.2. Статическая сборка

Инструментарий NetBSD по умолчанию собирает и компонует с разделяемыми библиотеками. Многие пользователи всё же предпочитают возможность статической компоновки. Иногда можно создать небольшую систему без разделяемых библиотек — это хороший пример полной статической сборки. Если на конкретной машине сборки всегда требуется определённым образом заданная переменная окружения, проще всего просто добавить изменённую настройку в `/etc/mk.conf`.

Чтобы машина сборки всегда собирала статически, просто добавьте в `/etc/mk.conf` следующую строку:

```
LDSTATIC=-static
```

33.7.3. Использование ключей `build.sh`

Помимо переменных окружения и `/etc/mk.conf`, на процесс сборки можно влиять рядом ключей самого сценария `build.sh`, как мы уже видели при принудительной сборке от непривилегированного (не root) пользователя, выборе целевой архитектуры или запрете удаления старых файлов перед сборкой. Все эти ключи можно вывести, выполнив **build.sh -h**:

```
$ cd /home/nia/cvs/src
$ build.sh -h

Usage: build.sh [-EhnoPRrUuxy] [-a arch] [-B buildid] [-C cdextras]
                [-c compiler] [-D dest] [-j njob] [-M obj] [-m mach]
                [-N noisy] [-O obj] [-R release] [-S seed] [-T tools]
                [-V var=[value]] [-w wrapper] [-X x11src] [-Y extsrcsrc]
                [-Z var]
                operation [...]

 Build operations (all imply "obj" and "tools"):
    build               Run "make build".
    distribution        Run "make distribution" (includes DESTDIR/etc/ files).
    release             Run "make release" (includes kernels & distrib media).

 Other operations:
    help                Show this message and exit.
    makewrapper         Create nbmake-${MACHINE} wrapper and nbmake.
                        Always performed.
    cleandir            Run "make cleandir".  [Default unless -u is used]
    dtb                 Build devicetree blobs.
    obj                 Run "make obj".  [Default unless -o is used]
    tools               Build and install tools.
    install=idir        Run "make installworld" to `idir' to install all sets
                        except `etc'.  Useful after "distribution" or "release"
    kernel=conf         Build kernel with config file `conf'
    kernel.gdb=conf     Build kernel (including netbsd.gdb) with config
                        file `conf'
    releasekernel=conf  Install kernel built by kernel=conf to RELEASEDIR.
    kernels             Build all kernels
    installmodules=idir Run "make installmodules" to `idir' to install all
                        kernel modules.
    modules             Build kernel modules.
    rumptest            Do a linktest for rump (for developers).
    sets                Create binary sets in
                        RELEASEDIR/RELEASEMACHINEDIR/binary/sets.
                        DESTDIR should be populated beforehand.
    distsets            Same as "distribution sets".
    sourcesets          Create source sets in RELEASEDIR/source/sets.
    syspkgs             Create syspkgs in
                        RELEASEDIR/RELEASEMACHINEDIR/binary/syspkgs.
    iso-image           Create CD-ROM image in RELEASEDIR/images.
    iso-image-source    Create CD-ROM image with source in RELEASEDIR/images.
    live-image          Create bootable live image in
                        RELEASEDIR/RELEASEMACHINEDIR/installation/liveimage.
    install-image       Create bootable installation image in
                        RELEASEDIR/RELEASEMACHINEDIR/installation/installimage.
    disk-image=target   Create bootable disk image in
                        RELEASEDIR/RELEASEMACHINEDIR/binary/gzimg/target.img.gz.
    params              Display various make(1) parameters.
    list-arch           Display a list of valid MACHINE/MACHINE_ARCH values,
                        and exit.  The list may be narrowed by passing glob
                        patterns or exact values in MACHINE or MACHINE_ARCH.

 Options:
    -a arch         Set MACHINE_ARCH to arch.  [Default: deduced from MACHINE]
```

```
    -B buildid     Set BUILDID to buildid.
    -C cdextras    Append cdextras to CDEXTRA variable for inclusion on CD-ROM.
    -c compiler    Select compiler:
                       clang
                       gcc
                   [Default: gcc]
    -D dest        Set DESTDIR to dest.  [Default: destdir.MACHINE]
    -E             Set "expert" mode; disables various safety checks.
                   Should not be used without expert knowledge of the build
                   system.
    -h             Print this help message.
    -j njob        Run up to njob jobs in parallel; see make(1) -j.
    -M obj         Set obj root directory to obj; sets MAKEOBJDIRPREFIX.
                   Unsets MAKEOBJDIR.
    -m mach        Set MACHINE to mach.  Some mach values are actually
                   aliases that set MACHINE/MACHINE_ARCH pairs.
                   [Default: deduced from the host system if the host
                   OS is NetBSD]
    -N noisy       Set the noisyness (MAKEVERBOSE) level of the build:
                       0   Minimal output ("quiet")
                       1   Describe what is occurring
                       2   Describe what is occurring and echo the actual
                           command
                       3   Ignore the effect of the "@" prefix in make commands
                       4   Trace shell commands using the shell's -x flag
                   [Default: 2]
    -n             Show commands that would be executed, but do not execute them.
    -O obj         Set obj root directory to obj; sets a MAKEOBJDIR pattern.
                   Unsets MAKEOBJDIRPREFIX.
    -o             Set MKOBJDIRS=no; do not create objdirs at start of build.
    -P             Set MKREPRO and MKREPRO_TIMESTAMP to the latest source
                   CVS timestamp for reproducible builds.
    -R release     Set RELEASEDIR to release.  [Default: releasedir]
    -r             Remove contents of TOOLDIR and DESTDIR before building.
    -S seed        Set BUILDSEED to seed.  [Default: NetBSD-majorversion]
    -T tools       Set TOOLDIR to tools.  If unset, and TOOLDIR is not set in
                   the environment, nbmake will be (re)built
                   unconditionally.
    -U             Set MKUNPRIVED=yes; build without requiring root privileges,
                   install from an UNPRIVED build with proper file permissions.
    -u             Set MKUPDATE=yes; do not run "make cleandir" first.
                   Without this, everything is rebuilt, including the tools.
    -V var=[value] Set variable `var' to `value'.
    -w wrapper     Create nbmake script as wrapper.
                   [Default: ${TOOLDIR}/bin/nbmake-${MACHINE}]
    -X x11src      Set X11SRCDIR to x11src.  [Default: /usr/xsrc]
    -x             Set MKX11=yes; build X11 from X11SRCDIR
    -Y extsrcsrc   Set EXTSRCSRCDIR to extsrcsrc.  [Default: /usr/extsrc]
    -y             Set MKEXTSRC=yes; build extsrc from EXTSRCSRCDIR
    -Z var         Unset ("zap") variable `var'.
```

Как видите, ряд ключей позволяет изменить стандартное поведение сборки. Некоторые из них уже были представлены, остальные можно задавать по необходимости.

33.7.4. Переменные make(1), используемые при сборке

Поведением сборок NetBSD управляют несколько переменных. Если не указано иное, эти переменные можно задать либо в окружении процесса, либо в файле конфигурации make(1), указанном в `MAKECONF`. Полный список этих параметров можно найти в файлах `BUILDING` и `share/mk/bsd.README` в корневом каталоге исходных текстов.

BUILDID

Идентификатор сборки. Идентификатор добавляется к именам объектных каталогов, и к нему можно обращаться в файле конфигурации make(1), чтобы задать дополнительные параметры сборки, например ключи компилятора.

DESTDIR

Каталог, в который помещается собранная система NetBSD. Если он задан, средствам компиляции передаются специальные ключи, предотвращающие их обычное использование `/usr/include`, `/usr/lib` и т. п. из хост-системы. Этот путь не должен заканчиваться символом косой черты (/). (Для установки в корневой каталог системы задайте для `DESTDIR` пустую строку.) Каталог должен находиться в файловой системе, поддерживающей длинные имена файлов и жёсткие ссылки.

По умолчанию — пустая строка, если `USETOOLS` равно «yes»; в противном случае переменная не задана. Примечание: `build.sh` предоставит значение по умолчанию (destdir.MACHINE в каталоге `.OBJDIR` верхнего уровня), если только он не запущен в режиме «expert».

EXTERNAL_TOOLCHAIN

Если задана пользователем, указывает на корень внешнего инструментария (например, `/usr/local/gnu`). Это включает инфраструктуру кросс-сборки, даже когда инструментарий по умолчанию недоступен (см. `TOOLCHAIN_MISSING` ниже).

По умолчанию: не задано

MAKEVERBOSE

Подробность сообщений сборки. Поддерживаются следующие значения:

0	Описательные сообщения не выводятся.
1	Описательные сообщения выводятся.
2	Описательные сообщения выводятся (с префиксом «#»), а вывод команд не подавляется.

По умолчанию: 2

MKCATPAGES

Можно задать значение «yes» или «no». Указывает, будут ли во время сборки создаваться предварительно отформатированные страницы руководства в виде простого текста.

По умолчанию: «yes»

MKDOC

Можно задать значение «yes» или «no». Указывает, будет ли во время сборки устанавливаться системная документация, предназначенная для `DESTDIR/usr/share/doc`.

По умолчанию: «yes»

MKHOSTOBJ

Можно задать значение «yes» или «no». Если задано «yes», то для программ, предназначенных для запуска на хосте компиляции, к имени объектного каталога, созданного командой «make obj», будет добавлен суффикс с именем, выпуском и архитектурой хост-операционной системы. Это позволяет нескольким хост-системам компилировать NetBSD для одной целевой платформы. Если задано «no», программы, собранные для запуска на хосте компиляции, будут использовать те же имена объектных каталогов, что и программы, собранные для запуска на целевой платформе.

По умолчанию: «no»

MKINFO

Можно задать значение «yes» или «no». Указывает, будут ли во время сборки создаваться и устанавливаться файлы GNU info, используемые для документации большинства средств компиляции.

По умолчанию: «yes»

MKLINT

Можно задать значение «yes» или «no». Указывает, будет ли lint(1) запускаться для отдельных частей исходного кода NetBSD во время сборки, а также будут ли библиотеки lint устанавливаться в `DESTDIR /usr/libdata/lint`.

По умолчанию: «yes»

MKMAN

Можно задать значение «yes» или «no». Указывает, будут ли во время сборки устанавливаться страницы руководства.

По умолчанию: «yes»

MKNLS

Можно задать значение «yes» или «no». Указывает, будут ли во время сборки компилироваться и устанавливаться файлы зон локали (locale zone files) системы Native Language System.

По умолчанию: «yes»

MKOBJ

Можно задать значение «yes» или «no». Указывает, будут ли создаваться объектные каталоги при выполнении «make obj». Если задано «no», все собранные файлы будут находиться внутри обычного дерева исходных текстов.

По умолчанию: «yes»

MKPIC

Можно задать значение «yes» или «no». Указывает, будут ли во время сборки создаваться и устанавливаться разделяемые объекты и библиотеки. Если задано «no», вся сборка будет статически скомпонована.

По умолчанию: зависит от платформы. На момент написания для всех платформ, кроме sh3, по умолчанию используется «yes»

MKPICINSTALL

Можно задать значение «yes» или «no». Указывает, устанавливаются ли во время сборки библиотеки формата ar(1) (`lib*_pic.a`), используемые для создания разделяемых библиотек.

По умолчанию: «yes»

MKPROFILE

Можно задать значение «yes» или «no». Указывает, будут ли во время сборки собираться и устанавливаться библиотеки с профилировкой (`lib*_p.a`).

По умолчанию: «yes»; однако на некоторых платформах `MKPROFILE` иногда отключается по умолчанию из-за проблем инструментария с профилированным кодом.

MKSHARE

Можно задать значение «yes» или «no». Указывает, будут ли во время сборки собираться и устанавливаться файлы, предназначенные для `DESTDIR /usr/share`. Если задано «no», то все `MKCATPAGES`, `MKDOC`, `MKINFO`, `MKMAN` и `MKNLS` будут безусловно установлены в «no».

По умолчанию: «yes»

MKTTINTERP

Можно задать значение «yes» или «no». Для сборок X определяет, включён ли интерпретатор байт-кода TrueType. Подробности см. на freetype.org.

По умолчанию: «no»

MKUNPRIVED

Можно задать значение «yes» или «no». Указывает, будет ли выполняться установка без привилегий. Владелец, группа, права доступа и флаги файлов не будут устанавливаться для устанавливаемых элементов; вместо этого сведения будут добавлены в файл `METALOG` в `DESTDIR`. Содержимое `METALOG` используется при создании tar-файлов дистрибутива, чтобы обеспечить сохранение правильного владельца файлов.

По умолчанию: «no»

MKUPDATE

Можно задать значение «yes» или «no». Указывает, будут ли все операции установки, предназначенные для записи в `DESTDIR`, сравнивать временные метки файлов перед установкой и пропускать этап установки, если целевые файлы не устарели. Это также влияет на полные сборки (см. ниже).

По умолчанию: «no»

MKLLVM

Можно задать значение «yes» или «no». Указывает, следует ли собирать и устанавливать Clang в качестве хост-компилятора.

По умолчанию: «no»

MKX11

Можно задать значение «yes» или «no». Указывает, собирается ли X11 из `X11SRCDIR`.

По умолчанию: «yes»

TOOLDIR

Каталог для размещения хост-средств после сборки. Этот каталог должен быть уникальным для данной хост-системы и дерева исходных текстов NetBSD. (Однако несколько целевых платформ могут совместно использовать один `TOOLDIR`; файлы, зависящие от целевой платформы, имеют уникальные имена.) Если он не задан, на основе сведений uname(1) о хост-платформе будет создан каталог по умолчанию в `.OBJDIR` дерева `src`.

По умолчанию: не задано.

USETOOLS

Указывает, следует ли использовать средства, заданные в `TOOLDIR`, в ходе выполняемой сборки. При кросс-компиляции необходимо задать «yes».

yes	Использовать средства из `TOOLDIR`.
no	Не использовать средства из `TOOLNAME`, но отказываться собирать встроенные компоненты средств компиляции, зависящие от версии этого средства.
never	Не использовать средства из `TOOLNAME` даже при сборке встроенных компонентов средств. Это похоже на традиционный метод сборки NetBSD, но при этом не проверяется, достаточно ли новы используемые средства компиляции для успешной сборки дерева. При сборке всего дерева исходных текстов NetBSD это может вызвать проблемы при сборке или во время работы.

По умолчанию: «yes», если собирается всё дерево исходных текстов NetBSD или его часть (определяется автоматически); в противном случае — «no» (чтобы сохранить традиционную семантику включаемых файлов `bsd.*.mk` команды make(1)).

X11SRCDIR

Каталог, содержащий исходные тексты X11.

По умолчанию: «usr/xsrc»

Следующие переменные влияют только на `Makefile` верхнего уровня и не влияют на ручную сборку поддеревьев исходного кода NetBSD.

INSTALLWORLDDIR

Каталог, в который устанавливает цель «make installworld».

По умолчанию: «/»

MKOBJDIRS

Можно задать значение «yes» или «no». Указывает, будут ли объектные каталоги создаваться автоматически (проходом «make obj») в начале сборки.

По умолчанию: «no»

MKUPDATE

Можно задать значение «yes» или «no». Если задано, то помимо описанного выше действия `MKUPDATE=yes` это подразумевает также действие `NOCLEANDIR` (то есть «make cleandir» не выполняется).

По умолчанию: «no»

NOCLEANDIR

Если задано, пропускается этап «make cleandir» полной сборки. В результате повторно компилируются только изменённые файлы в дереве исходных текстов. Это может ускорить сборку, когда в дереве обновляется лишь несколько файлов.

По умолчанию: не задано

NODISTRIBDIRS

Если задано, пропускается «make distrib-dirs» полной сборки. При этом не запускается mtree(8) для `DESTDIR`, что полезно в системах, где сборка ведётся непривилегированным пользователем, или там, где известно, что общесистемные файлы mtree не изменились.

По умолчанию: не задано

NOINCLUDES

Если задано, пропускается этап «make includes» полной сборки. В результате make(1) не считает некоторые программы устаревшими только из-за того, что изменились системные включаемые файлы. Однако на этот параметр не следует полагаться при произвольном обновлении всего дерева исходных текстов NetBSD; в этом случае рекомендуется использовать `MKUPDATE=yes`.

По умолчанию: не задано

RELEASEDIR

Если задано, указывает каталог, в который структура release(7) будет записана по завершении «make release».

По умолчанию: не задано

TOOLCHAIN_MISSING

Задаётся как «yes» на платформах, для которых нет работоспособного инструментария в дереве, или если вам нужен либо желателен инструментарий самой системы (то есть не кросс-средства, доступные через путь поиска вашей оболочки).

По умолчанию: зависит от целевой платформы; на платформах с инструментарием в дереве задаётся как «no».

Глава 34. Компиляция ядра

Есть несколько причин, по которым может понадобиться скомпилировать ядро NetBSD:

- установить исправления ошибок, обновления безопасности или новые возможности, пересобрав ядро из обновлённых исходных текстов;
- удалив из конфигурации неиспользуемые драйверы устройств и подсистемы ядра, можно значительно уменьшить размер ядра, а значит, потребление памяти и поверхность атаки. Делать это нужно с осторожностью, так как можно получить неработающую систему. Всегда сохраняйте копии старых работающих ядер;
- получить доступ к дополнительным возможностям, включив параметры или подсистемы ядра, некоторые из которых экспериментальны или отключены по умолчанию;
- глубже изучить систему.

34.1. Требования и порядок действий

Чтобы перекомпилировать ядро, у вас должен быть установлен набор компилятора (`comp.tgz`).

Основные шаги для получения обновлённого или настроенного ядра таковы:

1. Установить или обновить исходные тексты ядра
2. Создать или изменить файл конфигурации ядра
3. Собрать ядро по файлу конфигурации — вручную или с помощью **build.sh**
4. Установить ядро

34.2. Установка исходных текстов ядра

Исходные тексты ядра можно получить через AnonCVS (см. Глава 32, *Получение исходных текстов*) или из tar-архива `syssrc.tgz`, который находится в каталоге `source/sets/` используемого вами выпуска.

Если вы решили использовать AnonCVS для получения всего дерева исходных текстов, наберитесь терпения: операция может длиться много минут, поскольку репозиторий содержит тысячи файлов.

Если у вас есть tar-архив с исходными текстами, его можно распаковать от имени root:

```
# cd /
# tar zxf /path/to/syssrc.tgz
```

Даже если вы использовали tar-архив из выпуска, возможно, вам захочется применить AnonCVS, чтобы обновить исходные тексты изменениями, внесёнными после выпуска. Это особенно актуально, если вы обновляете ядро, чтобы включить исправление конкретной ошибки, в том числе уязвимости, описанной в рекомендации по безопасности NetBSD. Возможно, вам понадобится получить самые свежие исходные тексты из соответствующей ветви выпуска или ветви критических обновлений для вашей версии; рекомендации по безопасности обычно содержат сведения о датах или ревизиях файлов с нужными исправлениями. Дополнительные сведения о командах CVS, используемых для обновления исходных текстов из этих ветвей, см. в Раздел 32.4, «Получение через CVS».

Когда исходные тексты есть, можно создать собственное ядро: это не так сложно, как может показаться. На самом деле новое ядро создаётся за несколько шагов, которые описаны в следующих разделах.

34.3. Создание файла конфигурации ядра

> **ПРИМЕЧАНИЕ**
>
> Каталоги, описанные в этом разделе, относятся к amd64. Пользователям других архитектур нужно подставить соответствующие каталоги; их список можно найти в подкаталогах `src/sys/arch`.

Файл конфигурации ядра определяет тип, количество и характеристики устройств, поддерживаемых ядром, а также несколько параметров конфигурации ядра. Для платформы amd64 файлы конфигурации ядра находятся в каталоге `/usr/src/sys/arch/amd64/conf`.

Обратите внимание, что имена файлов конфигурации ядра исторически пишутся полностью заглавными буквами, поэтому их легко отличить от других файлов в этом каталоге:

```
$ cd /usr/src/sys/arch/amd64/conf/
$ ls
ALL                 INSTALL             XEN3_DOM0           kern.ldscript.kaslr
CVS                 INSTALL_XEN3_DOMU   XEN3_DOMU           majors.amd64
GENERIC             MODULAR             files.amd64         std.amd64
GENERIC_KASLR       Makefile.amd64      kern.ldscript       std.xen
GENERIC_USERMODE    NOCOMPAT            kern.ldscript.Xen
```

Проще всего создать новый файл, скопировав существующий и изменив его. Обычно лучший выбор на большинстве платформ — конфигурация GENERIC, так как она содержит большинство драйверов и параметров. В файле конфигурации есть комментарии с описанием параметров; более подробное описание приведено на странице руководства options(4). Итак, обычный порядок действий таков:

```
$ cp GENERIC MYKERNEL
$ vi MYKERNEL
```

Изменение файла конфигурации ядра в сущности сводится к трём операциям:

1. поддержка аппаратных устройств включается в ядро или исключается из него (например, поддержку SCSI можно удалить, если она не нужна);
2. поддержка возможностей ядра включается или отключается (например, включить поддержку клиента NFS, включить совместимость с Linux, …);
3. настройка параметров ядра.

Строки, начинающиеся с «#», — комментарии; строки отключаются путём комментирования и включаются путём удаления символа комментария. Лучше комментировать строки, а не удалять их: их всегда можно раскомментировать позже.

Чтобы определить, какие строки можно отключить, можно использовать вывод команды dmesg(8). Для каждой строки вида:

```
XXX at YYY
```

и *XXX*, и *YYY* должны быть активны в файле конфигурации ядра. Вероятно, придётся немного поэкспериментировать, прежде чем достичь минимальной конфигурации, но в настольной системе без SCSI и PCMCIA можно вдвое уменьшить размер ядра.

Можно также просмотреть параметры в файле конфигурации и отключить те, которые вам не нужны. У каждого параметра есть краткий описывающий его комментарий, которого обычно достаточно, чтобы понять, что этот параметр делает. Для многих параметров более длинное и подробное описание приведено на странице руководства options(4).

34.4. Сборка ядра вручную

На основе файла конфигурации ядра — одной из стандартных конфигураций или вашей собственной — необходимо собрать новое ядро.

Эти шаги можно выполнить вручную или с помощью команды **build.sh**, представленной в разделе Глава 33, *Кросс-компиляция NetBSD с помощью `build.sh`*. В этом разделе рассказывается, как собрать ядро для текущей системы вручную; в следующем разделе Раздел 34.5, «Сборка ядра с помощью `build.sh`» описано, как сделать то же самое с помощью **build.sh**.

- Настроить ядро
- Сгенерировать зависимости
- Скомпилировать ядро

34.4.1. Настройка ядра вручную

Закончив изменять файл конфигурации ядра (который мы назовём `MYKERNEL`), выполните следующую команду:

```
$ config MYKERNEL
```

Если `MYKERNEL` не содержит ошибок, программа config(1) создаст необходимые для компиляции ядра файлы; в противном случае нужно будет исправить ошибки, прежде чем снова запускать config(1).

> **ПРИМЕЧАНИЯ ПО КРОСС-КОМПИЛЯЦИИ**
>
> Поскольку программа config(1), создающая заголовочные файлы и Makefile для сборки ядра, зависит от платформы, необходимо использовать программу **nbconfig** из только что созданного инструментария (созданного, например, с помощью
>
> ```
> /usr/src/build.sh -m sparc64 tools
> ```
>
>). Помимо этого, процедура точно такая же, как компиляция «обычного» ядра NetBSD. Например, команда такова:
>
> ```
> % /usr/src/tooldir.NetBSD-9.0-amd64/bin/nbconfig MYKERNEL
> ```
>
> Эта команда создала каталог `../compile/MYKERNEL` с рядом заголовочных файлов, определяющих сведения об устройствах, которые нужно скомпилировать в ядро, и Makefile, настроенный на сборку всех необходимых для ядра файлов и их компоновку.

34.4.2. Генерация зависимостей и повторная компиляция вручную

Генерация зависимостей и компиляция ядра выполняются следующими командами:

```
$ cd ../compile/MYKERNEL
$ make depend
$ make
```

Бывает, что компиляция останавливается с ошибками; причины могут быть разными, но чаще всего это ошибка в файле конфигурации, которую не выявила config(1). Иногда сбой вызван проблемой с оборудованием (часто неисправными модулями памяти): компиляция создаёт для системы более высокую нагрузку, чем большинство приложений. Другая типичная ошибка такова: активный параметр B требует параметра A, который не активен. Полная компиляция ядра может длиться от нескольких минут до нескольких часов в зависимости от оборудования.

Результат успешного выполнения команды make — файл `netbsd` в каталоге compile, готовый к установке.

> **ПРИМЕЧАНИЯ ПО КРОСС-КОМПИЛЯЦИИ**
>
> Для кросс-компиляции ядра sparc64 необходимо использовать обёртку-сценарий оболочки `nbmake-sparc64` из инструментария кросс-компиляции, которая вызывает make(1) со всеми настройками, необходимыми для кросс-компиляции под платформу sparc64:
>
> ```
> % cd ../compile/MYKERNEL/
> % /usr/src/tooldir.NetBSD-9.0-amd64/bin/nbmake-sparc64 depend
> % /usr/src/tooldir.NetBSD-9.0-amd64/bin/nbmake-sparc64
> ```
>
> Сборка некоторое время будет работать, а затем выдаст ядро:
>
> ```
> ...
> text    data     bss     dec     hex filename
> 5016899  163728  628752 5809379  58a4e3 netbsd
> % ls -l netbsd
> -rwxr-xr-x  1 feyrer  666  5874663 Dec  2 23:17 netbsd
> % file netbsd
> netbsd: ELF 64-bit MSB executable, SPARC V9, version 1 (SYSV), statically linked, not
> stripped
> ```
>
> Теперь ядро из файла `netbsd` можно либо перенести на машину UltraSPARC (через NFS, FTP, scp и т. п.) и загрузить с жёсткого диска, если он есть, либо загрузить напрямую с машины кросс-разработки по NFS.

34.5. Сборка ядра с помощью `build.sh`

После создания и возможного редактирования файла конфигурации ядра ручные шаги настройки ядра, генерации зависимостей и повторной компиляции можно выполнить с помощью сценария `src/build.sh` — всё за один раз:

```
$ cd /usr/src
$ ./build.sh kernel=MYKERNEL
```

При этом выполняются те же шаги, что и выше, с одним небольшим отличием: перед компиляцией все старые объектные файлы удаляются, чтобы начать сборку с нуля. Обычно это излишне, и вполне можно сохранить старые файлы и пересобрать только те, чьи зависимости изменились. Для этого добавьте ключ `-u` к `build.sh`:

```
$ cd /usr/src
$ ./build.sh -u kernel=MYKERNEL
```

Завершив работу, `build.sh` выведет местоположение нового скомпилированного ядра. После этого его можно установить.

34.6. Установка нового ядра

Каким бы способом ни был получен новый файл ядра, теперь его нужно установить. Новый файл ядра следует скопировать в корневой каталог, сохранив предыдущую версию.

```
# mv /netbsd /netbsd.old
# mv netbsd /
```

Настройка может значительно уменьшить размер ядра. В следующем примере `netbsd.old` — установленное ядро, а `netbsd` — новое ядро.

```
-rwxr-xr-x  3 root  wheel  3523098 Dec 10 00:13 /netbsd
-rwxr-xr-x  3 root  wheel  7566271 Dec 10 00:13 /netbsd.old
```

Новое ядро вступает в силу после перезагрузки:

```
# shutdown -r now
```

34.7. Если что-то пошло не так

После перезапуска компьютера может оказаться, что новое ядро работает не так, как ожидалось, или вообще не загружается. Не волнуйтесь: если это случилось, просто перезагрузитесь с ранее сохранённым ядром и удалите новое (лучше перезагрузиться в режиме «single user»):

- Перезагрузите машину
- Нажмите пробел в приглашении загрузчика в течение 5 секунд обратного отсчёта

  ```
  boot:
  ```

- Введите

  ```
  > boot netbsd.old -s
  ```

- Теперь выполните следующие команды, чтобы восстановить предыдущую версию ядра:

  ```
  # fsck /
  # mount /
  ```

```
# mv netbsd.old netbsd
# reboot
```

Это вернёт вам ту работающую систему, с которой вы начинали, и вы сможете доработать свой файл конфигурации ядра, чтобы устранить проблему. Вообще, разумно начать с ядра GENERIC, а затем вносить постепенные изменения.

Глава 35. Обновление существующей системы из исходных текстов

Распространённый способ обновления системы NetBSD до более новой версии — пересобрать систему из исходных текстов и установить результат. Это работает как для стабильных выпусков, например NetBSD 11.0, так и для NetBSD-current. В частности, если вы используете стабильный выпуск NetBSD в производственной среде, рекомендуется регулярно выполнять эту процедуру, чтобы включать все исправления безопасности, внесённые в ветвь после её выпуска.

Есть множество способов пересобрать NetBSD из исходных текстов, и эта глава поможет вам разобраться в доступных вариантах. Сначала в ней показано, как выглядит процедура, выполняемая вручную, а затем описаны некоторые средства автоматизации, упрощающие процесс.

> **ПРИМЕЧАНИЕ**
>
> *Не забудьте проверить src/UPDATING на предмет последних изменений и особых указаний, которые могут потребоваться при обновлении системы.*

35.1. Порядок ручной сборки и обновления

Большинство следующих шагов можно выполнить как обычный пользователь. Только установка нового ядра и пользовательского окружения требует привилегий root. Хотя в следующих примерах в качестве рабочего каталога выбран `/usr`, процедуру можно выполнять и в домашнем каталоге пользователя. Обычные пользователи, как правило, не имеют прав на изменение `/usr`, но root может это изменить.

Наличие актуальных исходных текстов — обязательное условие для следующих шагов. В разделе Раздел 32.4, «Получение через CVS» рассказывается о способах получения или обновления исходных текстов для выпуска, стабильной или текущей ветви (с помощью CVS).

Всегда обращайтесь за подробностями к выводу **build.sh -h** и файлам `UPDATING` и `BUILDING` — это стоит того: есть *много* параметров, которые можно задать в командной строке или в `/etc/mk.conf`

35.1.1. Сборка нового пользовательского окружения

Первый шаг — собрать пользовательское окружение:

```
$ cd /usr/src
$ ./build.sh -O ../obj -T ../tools -U distribution
```

35.1.2. Сборка нового ядра

Следующий шаг — сборка ядра:

```
$ cd /usr/src
$ ./build.sh -O ../obj -T ../tools -U kernel=<KERNEL>
```

На платформах, поддерживающих модули, следующий шаг, если вы их используете, соберёт их:

```
$ ./build.sh -O ../obj -T ../tools -U modules
```

35.1.3. Установка ядра и пользовательского окружения

Установка нового ядра и модулей (если они вам нужны), перезагрузка (чтобы убедиться, что новое ядро работает) и установка нового пользовательского окружения — заключительные шаги процедуры обновления:

```
$ cd /usr/src
$ su
# ./build.sh -O ../obj -T ../tools -U installmodules=/
# mv /netbsd /netbsd.old
# mv /usr/obj/sys/arch/<ARCH>/compile/<KERNEL>/netbsd /
# shutdown -r now
 ...
$ cd /usr/src
$ su
# ./build.sh -O ../obj -T ../tools -U install=/
```

Если новое ядро `netbsd` не загружается успешно, можно вернуться к загрузке ядра `netbsd.old`.

Модули устанавливаются в `/stand/<ARCH>/<VERSION>/modules`, где — старший и младший номера выпуска, например 7.1 (но не номер патча, как 7.1.2, кроме версий в разработке, например 7.99.12). Так, например, установка модулей для 10.1 не перезапишет модули для 10.0, если что-то пойдёт не так и вам понадобится вернуться к загрузке более старого ядра 10.0.

35.1.4. Обновление файлов конфигурации системы

Обновление файлов конфигурации системы выполняется в два шага. Сначала с помощью postinstall(8) проверяются и исправляются вещи, которые легко автоматизировать. Затем с помощью etcupdate(8) объединяются остальные изменения файлов конфигурации.

```
# /usr/sbin/postinstall -s /usr/src check
# /usr/sbin/postinstall -s /usr/src fix
# /usr/sbin/etcupdate -s /usr/src
```

При необходимости перезагрузитесь, чтобы все работающие службы использовали новые двоичные файлы:

```
# shutdown -r now
```

35.1.5. Краткое описание

1. Из корня дерева исходных текстов:

   ```
   $ cd /usr/src
   ```

2. Соберите пользовательское окружение:

   ```
   $ ./build.sh -O ../obj -T ../tools -U -u distribution
   ```

3. Соберите ядро и, если нужно, модули:

```
$ ./build.sh -O ../obj -T ../tools -U -u kernel=GENERIC modules
```

4. Установите ядро:

```
$ cd ../obj/sys/arch/<ARCH>/compile/GENERIC
$ su
# mv /netbsd /netbsd.old
# cp netbsd /netbsd
```

5. Перезагрузитесь с новым ядром:

```
# shutdown -r now
```

6. Установите новое пользовательское окружение:

```
$ cd /usr/src
$ su
# ./build.sh -O ../obj -T ../tools -U install=/
```

7. Обновите систему и файлы конфигурации:

```
#  /usr/sbin/etcupdate -s /usr/src
```

> **ПРИМЕЧАНИЕ**
>
> В описанной выше процедуре ключ `-u` указывает на процесс обновления и на то, что перед началом сборки не следует выполнять операцию make clean. Это полезно при обновлении после предыдущей сборки и/или при чистой сборке. Ключ `-U` позволяет выполнить всю сборку непривилегированным пользователем, а затем установку от имени root.

35.2. Использование sysinst

С помощью `sysinst` можно также установить только что собранную систему. Шаги таковы:

1. Соберите полный выпуск:

```
$ ./build.sh -O ../obj -T ../tools -U -u -x release
```

2. Полученные наборы для установки будут находиться в каталоге `/usr/obj/releasedir/`.
3. Скопируйте установочное ядро в корневой каталог вашей системы NetBSD, перезагрузитесь с него и выполните обновление с помощью `sysinst` (см. Глава 4, *Обновление NetBSD*).

35.3. Использование sysbuild и sysupgrade

Средства sysbuild и sysupgrade (в настоящее время доступные в `pkgsrc/sysutils/sysbuild` и `pkgsrc/sysutils/sysupgrade` соответственно) автоматизируют весь процесс пересборки NetBSD из исходных текстов (*включая получение исходных текстов из репозитория CVS*) и установки результата с минимальными усилиями.

У обоих средств есть файлы конфигурации, определяющие, как собирать выпуск и как его устанавливать. Помимо прочего, в них указываются используемый репозиторий CVS, целевая архитектура, место размещения файлов сборки и шаги, выполняемые при обновлении. Файлы находятся в `/usr/pkg/etc/sysbuild/default.conf` и `/usr/pkg/etc/sysupgrade.conf`. Конфигурация по умолчанию обоих средств позволяет начать работу с минимальными усилиями.

В простейшем виде можно выполнить полную сборку NetBSD и обновить систему до неё, выполнив следующие команды:

```
# sysbuild build
# sysupgrade auto ~/sysbuild/release/$(uname -m)
```

Вот и всё, что требуется. Эти вызовы сделают следующее:

1. Загрузят деревья исходных текстов из CVS в `/usr/src` и `/usr/xsrc`. Последнее загружается только в том случае, если в вашей системе есть X11. А если исходные тексты уже есть в системе, они будут лишь обновлены до самой новой версии.
2. Соберут новый выпуск в `~/sysbuild/<machine>/`. Этот каталог, отдельный для каждой машины, будет содержать подкаталоги наподобие `obj`, `destdir` и т. п. Результаты сборки останутся в `~/sysbuild/release/<machine>/`.
3. Установят новое ядро и распакуют новые наборы, используя только что собранные файлы выпуска.
4. Запустят etcupdate и postinstall, чтобы помочь вам включить новые изменения конфигурации в вашу систему.

Подробнее см. страницы руководства sysbuild(1) и sysupgrade(8), а также комментарии в упомянутых файлах конфигурации.

35.3.1. Настройка: сборка без прав root

Приведённые выше команды показывают самый простой и базовый вызов этих средств с использованием *файлов конфигурации по умолчанию*. Один из недостатков — необходимость прав root во время сборки дерева исходных текстов, чтобы sysbuild мог обновлять деревья исходных текстов в `/usr/src` и `/usr/xsrc`. Освоив эту процедуру, рекомендуется отказаться от сборки с правами root; в этом разделе показано, что для этого нужно сделать с sysbuild.

Чтобы собирать без прав root, можно либо разместить деревья исходных текстов вне `/usr` (самый простой путь), либо дать своему пользователю права на изменение деревьев в `/usr` (хорошо, если вы хотите совместно использовать дерево исходных текстов с другими пользователями).

Если вы хотите хранить деревья исходных текстов в своём домашнем каталоге, что удобно при разработке, просто отредактируйте `/usr/pkg/etc/sysbuild.conf` и добавьте следующие настройки:

```
SRCDIR="${HOME}/sysbuild/src"
[ ! -f /etc/mtree/set.xbase ] || XSRCDIR="${HOME}/sysbuild/xsrc"
```

После этого вызов «sysbuild build», показанный выше, будет работать под непривилегированным пользователем. Процедура обновления тогда примет вид:

```
$ sysbuild build
... become root ...
```

```
# sysupgrade auto ~/sysbuild/release/$(uname -m)
```

Другой вариант, если вы хотите хранить деревья исходных текстов в местах, описанных в hier(7), — выполнить от имени root следующие действия:

```
# mkdir -p /usr/src /usr/xsrc
# chown -R <your-user>:wsrc /usr/src /usr/xsrc
... and optionally add <your-user> to wsrc in /etc/group ...
```

После этого файл конфигурации sysbuild по умолчанию позволит размещать файлы в этих местах и выполнять непривилегированные сборки.

35.3.2. Настройка: организация ночных сборок

Пакет `pkgsrc/sysutils/sysbuild-user` позволяет настроить и поддерживать непривилегированного системного пользователя для периодических (например, ночных) сборок из исходных текстов. Это очень удобно, чтобы пристально отслеживать NetBSD-current.

Создаваемый пользователь так и называется — sysbuild; по умолчанию он настроен на выполнение полной сборки системы за ночь. Результаты остаются в `/home/sysbuild/release/<machine>/` — удобном каталоге выпуска по умолчанию для sysupgrade. О любых сбоях сборки вам сообщат по электронной почте.

Поведение sysbuild для этого непривилегированного пользователя настраивается в `/home/sysbuild/default.conf`.

Взаимодействовать с sysbuild от имени этого непривилегированного пользователя можно, выполняя команды вида:

```
# su - sysbuild /usr/pkg/bin/sysbuild ...
```

35.4. Подробнее об обновлении файлов конфигурации и загрузки

`etcupdate` — сценарий, который помогает пользователям сравнивать, объединять и устанавливать новые файлы конфигурации и загрузки (файлы из дистрибутивного набора etc.tgz) в /dev, /etc и /root после обновления операционной системы. Обновление операционной системы могло быть выполнено как компиляцией исходных текстов, так и распаковкой дистрибутивных двоичных файлов.

35.4.1. Использование etcupdate с исходными текстами

Если исходные тексты находятся в /usr/src, достаточно следующей команды:

```
# etcupdate
```

А что, если исходные тексты NetBSD находятся в другом месте, например в `/home/jdoe/netbsd/src`? Не беспокойтесь: укажите etcupdate расположение дерева исходных текстов с помощью -s srcdir, и всё будет работать:

```
# etcupdate -s /home/jdoe/netbsd/src
```

35.4.2. Использование etcupdate с дистрибутивными наборами двоичных файлов

Иногда держать под рукой исходные тексты неудобно, но обновить файлы конфигурации и загрузки всё же нужно. Решение — передать etc.tgz (или xetc.tgz) в etcupdate с помощью ключа -s tgzfile.

```
# etcupdate -s /some/where/etc.tgz
```

35.4.3. Использование `etcmanage` вместо `etcupdate`

Сценарий `etcmanage` на Perl (доступный из pkgsrc/sysutils/etcmanage или как двоичный пакет) — альтернатива `etcupdate`. Его следует использовать следующим образом, в сочетании с postinstall(8):

```
# /usr/pkg/bin/etcmanage
# /usr/sbin/postinstall
```

Глава 36. Создание установочных носителей NetBSD

36.1. Создание стандартных установочных образов с помощью build.sh

Для некоторых архитектур можно собрать дисковый или ISO-образ, который загружается в программу установки. Для этого нужно запустить `./build.sh` с целями `install-image` (для USB-флешки) или `iso-image` (для DVD или CD):

```
$ ./build.sh -U -u -j2 -m amd64 -O ~/obj tools
$ ./build.sh -U -u -j2 -m amd64 -O ~/obj release
$ ./build.sh -U -u -j2 -m amd64 -O ~/obj install-image
```

Доступно множество других целей `./build.sh`, см. Глава 33, *Кросс-компиляция NetBSD с помощью `build.sh`*.

36.2. Создание собственных «живых» дисковых образов

Иногда вместо готовых скомпилированных образов может понадобиться создать собственный настроенный предустановленный («живой») образ, например для массового развёртывания на встраиваемых системах. В этом разделе описаны необходимые для этого шаги.

1. Нужно собрать выпуск NetBSD, чтобы было чем наполнить образ. Инструкции см. в Глава 33, *Кросс-компиляция NetBSD с помощью `build.sh`*.
2. Нужно написать или выбрать сценарий sh(1) из `src/distrib/utils/embedded/conf`. Изучите конфигурации по умолчанию и при необходимости настройте одну из них под свои задачи. Сценарии могут изменять системные файлы конфигурации и дополнять их.
3. Запустите утилиту:

   ```
   $ export MKDTB=no
   $ ./distrib/utils/embedded/mkimage -D obj/destdir.amd64 -K
   sys/arch/amd64/compile/obj/GENERIC/netbsd -h amd64 -x ./example-amd64-image.img
   ```

   - В данном случае мы собираем образ по конфигурации **amd64.conf**, поэтому указываем **-h amd64**.
   - Мы хотим включить наборы X11, поэтому указываем **-x**.
   - Тип устройства корневой файловой системы нужно указать, например, с помощью **-r sd**. Как правило, это ld(4) для устройств SD/MMC и sd(4) для USB-флешек.
   - Мы задаём в окружении MKDTB=no, чтобы не собирать двоичные файлы дерева устройств (device tree blobs), которые используются только на ARM и MIPS.
4. Внесите в образ последние дополнения. К ним может относиться установка пакетов с помощью pkg_add(1) и т. п. Мы монтируем образ как виртуальный диск с помощью vndconfig(8):

   ```
   # vndconfig vnd0 ./example-amd64-image.img
   # mount /dev/vnd0a /mnt
   ```

```
# pkg_add -P /mnt -K /usr/pkg/pkgdb -v darkstat-3.0.719.tgz
# umount /mnt
```

5. Запишите образ на носитель:

```
# dd if=example-amd64-image.img ibs=1m | progress dd of=/dev/rsd0 obs=1m
```

Приложение A. Сведения

A.1. Где взять этот документ

В настоящее время этот документ доступен в следующих форматах:

- HTML
- PDF
- PostScript в архиве gzip

Кроме того, это руководство иногда продаётся в печатном виде на выставках и конференциях, и вся прибыль передаётся NetBSD Foundation. Возможно, когда-нибудь станет доступна и печать по требованию. Если вы хотите получить печатный переплетённый экземпляр этого документа, напишите на <www@NetBSD.org>.

A.2. История руководства

Это руководство возникло как собрание отрывочных заметок, которые Федерико Лупи, первый автор руководства по NetBSD, писал в основном для себя. Поняв, что они могут быть полезны и другим пользователям NetBSD, он начал их собирать и создал первую версию руководства с помощью форматирующей программы groff. Чтобы «легко» получать большее разнообразие выходных форматов (например, HTML и PostScript/PDF), он совершил «ошибку», перейдя на SGML/DocBook — формат, в котором исходные тексты существуют и сейчас. Позднее сопровождение взяли на себя проект NetBSD и его разработчики, а формат был изменён на XML/DocBook — из-за лучших инструментов и несколько большего опыта в тонкой настройке.

При написании и форматировании руководства использовались следующие инструменты с открытым исходным кодом:

- редактор vi, входящий в состав NetBSD (nvi).
- анализатор libxslt из GNOME для преобразования XML/DocBook в HTML.
- система TeX из коллекции пакетов NetBSD. TeX используется как основа для получения форматов PS и PDF.
- программа tgif для рисования иллюстраций.
- программы gimp и xv для преобразования форматов изображений и небольших правок иллюстраций.

Большое спасибо всем, кто участвовал в разработке этих замечательных инструментов.

Приложение B. Вклад в руководство NetBSD

Существует интерес как к вводной, так и к углублённой документации по NetBSD: вероятно, это признак роста популярности этой операционной системы и увеличения числа её пользователей. Поэтому важно продолжать добавлять в руководство новый материал и улучшать существующий текст.

Каким бы ни был ваш уровень знаний NetBSD, вы можете участвовать в развитии этого руководства. В этом приложении рассказано, как это делать и что следует знать, прежде чем начинать.

Если вы новичок и это руководство оказалось вам полезным, присылайте свои замечания и предложения на `<www@NetBSD.org>`. Например, если вы попробовали что-то описанное здесь, и у вас это не получилось, или если вам кажется, что что-то объяснено неясно, или если у вас есть идея новой главы и т. п.: такая обратная связь очень полезна.

Если у вас есть опыт работы с NetBSD, подумайте о том, чтобы добавить в руководство новый материал: вы можете написать новую главу или улучшить существующую.

Что бы вы ни выбрали, не начинайте работу, не связавшись предварительно с нами, чтобы избежать дублирования усилий.

B.1. Отправка материалов

Исходные тексты руководства по NetBSD находятся в CVS в каталоге htdocs/docs/guide.

Если вы хотите добавить в руководство какой-либо материал, у вас есть несколько вариантов — в зависимости от объёма текста, который вы собираетесь написать. Если речь идёт лишь о небольшом исправлении, проще всего отправить его в виде diff к существующим исходным текстам по электронной почте на `<www@NetBSD.org>`; впрочем, туда же можно присылать любые небольшие изменения и предложения.

Если вы планируете написать значительный объём текста — раздел или главу, — можете выбрать один из многих форматов:

- XML/DocBook; это предпочтительный формат. Если вы выберете его, возьмите исходные тексты руководства и используйте их как образец отступов и разметки текста, чтобы оформление оставалось единообразным.
- текст; если оформление остаётся простым, преобразовать текст в формат XML несложно.
- другие форматы тоже принимаются, если вы действительно не можете использовать ни один из предыдущих.

B.2. Шаблон XML/DocBook

Для руководства я использую стиль оформления, похожий на стиль программы. Вот шаблон:

```
<chapter id="chap-xxxxx">
  <title>This is the title of the chapter</title>

  <para>
    This is the text of a paragraph.  This is the text of a paragraph.
```

```
    This is the text of a paragraph.  This is the text of a paragraph.
    This is the text of a paragraph.
  </para>

  <!-- ============================================================= -->

  <sect1>
    <title>This is the title of a sect1</title>

    <para>
      This is the text of a paragraph.  This is the text of a paragraph.
      This is the text of a paragraph.  This is the text of a paragraph.
      This is the text of a paragraph.
    </para>

    <!-- ........................................................... -->

    <sect2>
      <title>This is the title of a sect2</title>

      <para>
        A sect2 is nested inside a sect1.
      </para>
    </sect2>

  </sect1>

  <!-- ============================================================= -->

  <sect1>
    <title>This is the title of another sect1</title>

    <para>
      An itemized list:
      <itemizedlist>
        <listitem>
          <para>
            text
          </para>
        </listitem>
        <listitem>
          <para>
            text
          </para>
        </listitem>
      </itemizedlist>
    </para>

  </sect1>
</chapter>
```

По умолчанию принято следующее:

- два пробела на каждый уровень отступа
- строки длиной не более 72 символов.
- используйте разделительные строки (комментарии) между sect1/sect2.

Приложение C. Начало работы с XML/DocBook

В этом приложении описана установка инструментов, необходимых для получения отформатированной версии руководства по NetBSD. Кроме того, здесь приведены инструкции по сборке руководства.

C.1. Что такое XML/DocBook

XML (eXtensible Markup Language) — язык, который используется для определения других языков на основе разметки, то есть с помощью XML можно определить грамматику (допустимые конструкции) языков разметки. HTML, например, можно определить с помощью XML. Если вы программист, представляйте XML как BNF (Backus-Naur Form): инструмент, служащий для определения грамматик.

DocBook — это шаблон разметки, определённый с помощью XML; DocBook перечисляет допустимые теги, которые можно использовать в документе DocBook, и то, как их можно сочетать друг с другом. Если вы программист, представляйте DocBook как грамматику языка, заданную с помощью BNF. Например, в нём сказано, что теги:

```
<para> ... </para>
```

определяют абзац, а может находиться внутри , но не может находиться внутри .

Поэтому, когда вы пишете документ, вы пишете его на DocBook, а не на XML: в этом отношении DocBook является аналогом HTML (хотя разметка богаче, а некоторые понятия отличаются).

Спецификация DocBook (то есть список тегов и правил) называется DTD (Document Type Definition).

Коротко говоря, DTD определяет, как выглядят ваши исходные документы, но ничего не сообщает о формате итоговых (скомпилированных) документов. Требуется ещё один шаг: исходные тексты DocBook должны быть преобразованы в какое-то иное представление, например в HTML или PDF. Этот шаг выполняет инструмент наподобие Jade, который применяет к исходному документу преобразования DSSSL. DSSSL (Document Style Semantics and Specification Language) — формат, используемый для определения *таблиц стилей*, необходимых для преобразования DocBook в другие форматы. Структура сборки руководства поддерживает также язык таблиц стилей XSL (Extensible Stylesheet Language). Программа **xsltproc** используется для преобразования XML с помощью таблиц стилей XSL.

C.2. Установка необходимых инструментов

Все инструменты, необходимые для создания руководства в различных форматах, можно установить с помощью метапакетов *netbsd-www*, *netbsd-doc* и *netbsd-doc-print*. Вместе пакеты *netbsd-doc* и *netbsd-www* устанавливают всё, что нужно для создания HTML-версии руководства. Чтобы получить возможность создавать печатные форматы, например PostScript и PDF, установите метапакет *netbsd-doc-print*.

Предполагая, что актуальное дерево pkgsrc установлено в `/usr/pkgsrc`, установить все эти метапакеты можно так:

```
$ cd /usr/pkgsrc/meta-pkgs/netbsd-www
$ make install
$ cd /usr/pkgsrc/meta-pkgs/netbsd-doc
$ make install
$ cd /usr/pkgsrc/meta-pkgs/netbsd-doc-print
$ make install
```

C.3. Работа с инструментами

В этом разделе дан обзор того, как скомпилировать руководство из XML в любой из следующих целевых форматов: *html*, *html-split*, *ascii*, *ps* и *pdf*. По умолчанию создаются все форматы. Чтобы получить любой из перечисленных выходных форматов, запустите **make**, указав формат (или форматы) в качестве аргумента.

Рассмотрим несколько примеров.

Прежде чем смотреть на результат, полученный в любом из перечисленных форматов, нужно убедиться в целостности структуры XML. Это можно сделать, запустив **make lint**:

```
$ cd htdocs/docs/guide/en
$ make lint
```

Исправьте все ошибки, которые могут возникнуть. Работая над содержимым руководства, вы можете собирать HTML-версию, чтобы просматривать её при вычитке:

```
$ cd htdocs/docs/guide/en
$ make html-split
```

После этого обновите также версии руководства в форматах PostScript и PDF. Команда для этого:

```
$ cd htdocs/docs/guide/en
$ make pdf
```

Прежде чем фиксировать сгенерированные файлы, убедитесь, что сначала зафиксированы файлы XML, а затем заново сгенерируйте все форматы, то есть процедура будет примерно такой:

```
$ cd htdocs/docs/guide/en
$ cvs commit *.xml
$
$ make lint
$ make
$ make install-doc
$
$ cd ..
$ cvs commit en download
```

Если запустить **make** без аргументов, все форматы будут сгенерированы заново. Именно так по умолчанию собирается руководство для сайта NetBSD.org.

C.4. Ссылки

Официальная домашняя страница DocBook — место, где можно найти основную документацию по DocBook. Также можно прочитать в сети или загрузить копию книги DocBook: The Definitive Guide

Нормана Уолша (Norman Walsh) и Леонарда Мюллнера (Leonard Muellner).

По DSSSL начните с http://nwalsh.com.

XSL описан на http://www.w3.org/Style/XSL/.

Исходные тексты Jade/OpenJade и информацию о них можно найти на домашней странице OpenJade.

Если вы хотите получать документы в форматах PostScript и PDF из исходных текстов DocBook, обратите внимание на домашнюю страницу JadeTex.

Приложение D. Благодарности

Изначально руководство по NetBSD написал Федерико Лупи: он в одиночку вёл исходные тексты, координировал обновления и объединял все присланные материалы. С тех пор руководство обновляет и сопровождает команда www проекта NetBSD. Оно развивалось благодаря вкладу многих людей, которые добровольно тратили своё время и силы, присылали материалы, предложения и исправления.

D.1. Первоначальные благодарности

Вот кого Федерико благодарит в первую очередь:

- Paulo Aukar
- Grant Beattie — преобразование в XML DocBook.
- Manolo De Santis — глава об аудио
- Eric Delcamp — загрузочные дискеты
- Hubert Feyrer — за Глава 23, *Введение в организацию сетей TCP/IP*, включая раздел о протоколе Интернета нового поколения IPv6 и раздел о подключении к IPv6 и переходе на него через 6to4. Он также помог с переходом с SGML на XML.
- Jason R. Fink
- Daniel de Kok — исправления в главах об аудио и Linux.
- Reinoud Koornstra — глава о CVS и пересборка `/dev` в главе «Разное».
- Brian A. Seklecki `<lavalamp@burghcom.com>` — за главу о CCD.
- Guillain Seuillot
- Martti Kuparinen — документация по RAIDframe.
- David Magda

D.2. Текущие благодарности

В настоящее время этот документ сопровождает команда www проекта NetBSD. Благодаря её усилиям документ постоянно поддерживается в актуальном состоянии и всегда доступен в сети. Кроме того, особая благодарность (в алфавитном порядке):

- Hubert Feyrer — за то, что подготовил руководство к NetBSD 2.0, и за множество улучшений во всех главах.
- Jason R. Fink — за сопровождение этого документа и включение изменений.
- Joel Knight — за Глава 28, *Введение в протокол резервирования общего адреса (CARP)*. См. Раздел D.3.3, «Лицензия Joel Knight на статью о CARP» — там указана прилагаемая лицензия.
- Daniel de Kok — за постоянный вклад новыми главами, сопровождение существующих глав и работу по переводу.
- Hiroki Sato — за предоставленную возможность собирать версии этого документа в форматах PDF и PS.

- Jan Schaumann — за работу по сопровождению и управление www/htdocs.
- Lubomir Sedlacik — за некоторые подробности об использовании CGD для подкачки в Раздел 14.6, «Предложения и предупреждения».
- Dag-Erling Smørgrav — за статью в Глава 18, *Подключаемые модули аутентификации (PAM)*. См. Раздел D.3.2, «Лицензия Networks Associates Technology на статью о PAM» — там указана прилагаемая лицензия.
- Florian Stöhr — за Раздел 14.4, «Пример: зашифрованные CD/DVD».

D.3. Лицензии

D.3.1. Первоначальная лицензия Федерико Лупи на это руководство

Распространение и использование в исходной и двоичной формах, с изменениями или без них, разрешаются при соблюдении следующих условий:

1. При распространении исходного кода должны сохраняться приведённое выше уведомление об авторских правах, этот список условий и следующий отказ от ответственности.
2. При распространении в двоичной форме необходимо воспроизводить приведённое выше уведомление об авторских правах, этот список условий и следующий отказ от ответственности в документации и/или других материалах, поставляемых с распространением.
3. Все рекламные материалы, упоминающие возможности или использование этого программного обеспечения, должны содержать следующее уведомление: этот продукт включает программное обеспечение, разработанное Федерико Лупи для проекта NetBSD.
4. Имя автора не может использоваться для поддержки или продвижения продуктов, производных от этого программного обеспечения, без специального предварительного письменного разрешения.

ЭТО ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПРЕДОСТАВЛЯЕТСЯ АВТОРОМ «КАК ЕСТЬ», И ЛЮБЫЕ ЯВНЫЕ ИЛИ ПОДРАЗУМЕВАЕМЫЕ ГАРАНТИИ, ВКЛЮЧАЯ, НО НЕ ОГРАНИЧИВАЯСЬ ПОДРАЗУМЕВАЕМЫМИ ГАРАНТИЯМИ ТОВАРНОЙ ПРИГОДНОСТИ И ПРИГОДНОСТИ ДЛЯ КОНКРЕТНОЙ ЦЕЛИ, ИСКЛЮЧАЮТСЯ. НИ ПРИ КАКИХ ОБСТОЯТЕЛЬСТВАХ АВТОР НЕ НЕСЁТ ОТВЕТСТВЕННОСТИ ЗА ЛЮБЫЕ ПРЯМЫЕ, КОСВЕННЫЕ, СЛУЧАЙНЫЕ, ОСОБЫЕ, ШТРАФНЫЕ ИЛИ ПОСЛЕДУЮЩИЕ УБЫТКИ (ВКЛЮЧАЯ, НО НЕ ОГРАНИЧИВАЯСЬ ЗАКУПКОЙ ЗАМЕНЯЮЩИХ ТОВАРОВ ИЛИ УСЛУГ; ПОТЕРЮ ВОЗМОЖНОСТИ ИСПОЛЬЗОВАНИЯ, ДАННЫХ ИЛИ ПРИБЫЛИ; ПРЕРЫВАНИЕ ДЕЯТЕЛЬНОСТИ), ОДНАКО ВОЗНИКШИЕ ПО ЛЮБОЙ ПРИЧИНЕ И НА ЛЮБОМ ОСНОВАНИИ ОТВЕТСТВЕННОСТИ, БУДЬ ТО ПО КОНТРАКТУ, СТРОГОЙ ОТВЕТСТВЕННОСТИ ИЛИ ДЕЛИКТУ (ВКЛЮЧАЯ НЕБРЕЖНОСТЬ ИЛИ ИНОЕ), ВОЗНИКШИЕ ЛЮБЫМ ОБРАЗОМ ИЗ ИСПОЛЬЗОВАНИЯ ЭТОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ, ДАЖЕ ЕСЛИ БЫЛО СООБЩЕНО О ВОЗМОЖНОСТИ ТАКОГО УЩЕРБА.

D.3.2. Лицензия Networks Associates Technology на статью о PAM

```
Copyright (c) 2001-2003 Networks Associates Technology, Inc.

All rights reserved.


This software was developed for the FreeBSD Project by ThinkSec AS and
```

```
Network Associates Laboratories, the Security Research Division of
Network Associates, Inc.  under DARPA/SPAWAR contract N66001-01-C-8035
("CBOSS"), as part of the DARPA CHATS research program.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:
1. Redistributions of source code must retain the above copyright
   notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.
3. The name of the author may not be used to endorse or promote
   products derived from this software without specific prior written
   permission.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
ARE DISCLAIMED.  IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
SUCH DAMAGE.
```

D.3.3. Лицензия Joel Knight на статью о CARP

```
Copyright (c) 2005 Joel Knight <enabled@myrealbox.com>

Permission to use, copy, modify, and distribute this documentation for
```

> any purpose with or without fee is hereby granted, provided that the
> above copyright notice and this permission notice appear in all copies.
>
> THE DOCUMENTATION IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL
> WARRANTIES WITH REGARD TO THIS DOCUMENTATION INCLUDING ALL IMPLIED
> WARRANTIES OF MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE
> AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL
> DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR
> PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER
> TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
> PERFORMANCE OF THIS DOCUMENTATION

Приложение E. Библиография

[AeleenFrisch] Aeleen Frisch. © 1991. O'Reilly & Associates. *Основы системного администрирования*.

[CraigHunt] Craig Hunt. © 1993. O'Reilly & Associates. *Администрирование сетей TCP/IP*.

[RFC1034] P. V. Mockapetris. © 1987. *RFC 1034: доменные имена — концепции и средства*.

[RFC1035] P. V. Mockapetris. © 1987. *RFC 1035: доменные имена — реализация и спецификация*.

[RFC1055] J. L. Romkey. © 1988. *RFC 1055: нестандартный способ передачи IP-дейтаграмм по последовательным линиям: SLIP*.

[RFC1331] W. Simpson. © 1992. *RFC 1331: протокол Point-to-Point (PPP) для передачи многопротокольных дейтаграмм по двухточечным каналам*.

[RFC1332] G. McGregor. © 1992. *RFC 1332: протокол управления Internet Protocol для PPP (IPCP)*.

[RFC1933] R. Gilligan and E. Nordmark. © 1996. *RFC 1933: механизмы перехода для хостов и маршрутизаторов IPv6*.

[RFC2004] C. Perkins. © 1996. *RFC 2003: инкапсуляция IP в IP*.

[RFC2401] S. Kent and R. Atkinson. © 1998. *RFC 2401: архитектура безопасности протокола Интернета*.

[RFC2411] R. Thayer, N. Doraswamy, and R. Glenn. © 1998. *RFC 2411: путеводитель по документам о безопасности IP*.

[RFC2461] T. Narten, E. Nordmark, and W. Simpson. © 1998. *RFC 2461: обнаружение соседей для IP версии 6 (IPv6)*.

[RFC2529] B. Carpenter and C. Jung. © 1999. *RFC 2529: передача IPv6 через домены IPv4 без явных туннелей*.

[RFC3024] G. Montenegro. © 2001. *RFC 3024: обратное туннелирование для мобильного IP*.

[RFC3027] M. Holdrege and P. Srisuresh. © 2001. *RFC 3027: осложнения протоколов при работе с транслятором сетевых адресов IP*.

[RFC3056] B. Carpenter and K. Moore. © 2001. *RFC 3056: соединение доменов IPv6 через облака IPv4*.